University of Waterloo

Neural Networks

CS 479

Lecture Notes

Kenneth Zhang

Fall 2025

Contents

1 Lecture 1: Neurons and Membrane Potentials 1
- 1.1 Structure of a Neuron 1
- 1.2 Ion Channels and Pumps 1
- 1.3 Membrane Potential 2
- 1.4 Action Potential 2

2 Lecture 2: Hodgkin–Huxley Model, Spiking Neurons, and Artificial Neurons 3
- 2.1 The Hodgkin–Huxley Model 3
- 2.2 The Leaky Integrate-and-Fire Model 7
- 2.3 Artificial Neurons and Activation Functions 12

3 Lecture 3: Postsynaptic Currents, Spike Trains, and Spiking Neural Networks 16
- 3.1 Postsynaptic Current Filters 16
- 3.2 Spike Trains and Dirac Impulses 19
- 3.3 Filtering a Spike Train by Convolution 20
- 3.4 Connectivity and the Synaptic Weight Matrix 22
- 3.5 Exponential Synaptic Dynamics 23
- 3.6 The Full Leaky Integrate-and-Fire Network 25
- 3.7 Summary of the Computational Model 29

4 Lecture 4: Supervised Learning and the Universal Approximation Theorem 30
- 4.1 Supervised Learning: Inputs, Targets, and Predictions 30
- 4.2 The Objective of Learning 32
- 4.3 Feedforward Neural Networks and Hidden Units 33
- 4.4 The Universal Approximation Theorem 34
- 4.5 How Sigmoids Approximate Thresholds 36
- 4.6 A Complete Constructive Proof in One Dimension 37
- 4.7 Universal Approximation in Several Dimensions 41
- 4.8 An Example: Approximating a Continuous Response 43
- 4.9 What Universal Approximation Does and Does Not Guarantee 44
- 4.10 Additional Derivations: Training a Single-Hidden-Layer Network 44

5 Lecture 5: Loss Functions, Maximum Likelihood, and Gradient Descent . 46
- 5.1 The Learning Objective 46
- 5.2 Regression and Mean Squared Error 48
- 5.3 Binary Classification and Bernoulli Cross-Entropy 51
- 5.4 Multiclass Classification and Categorical Cross-Entropy 55
- 5.5 Gradient Descent and Gradient Ascent 60
- 5.6 Full-Batch, Stochastic, and Mini-Batch Gradients 65
- 5.7 Training Neural Networks by the Chain Rule 66
- 5.8 Worked Examples 68

6 Lecture 6: Backpropagation and the Chain Rule in Multilayer Networks . 71
- 6.1 From Gradient Descent to Backpropagation 71

6.2 The Multivariable Chain Rule . . . 72
6.3 Output-Layer Backpropagation . . . 73
6.4 Backpropagation Through the Hidden Layer . . . 76
6.5 Matrix Differentials and the Outer-Product Rule . . . 79
6.6 Backpropagation in an Arbitrarily Deep Network . . . 80
6.7 Backpropagation as Reverse-Mode Automatic Differentiation . . . 82
6.8 A Complete Numerical Example . . . 83
6.9 Empirical Risk, Mini-Batches, and Gradient Descent . . . 85
6.10 Computational Cost and the Value of Reusing Errors . . . 86
6.11 Activation Derivatives and Gradient Propagation . . . 86
6.12 Checking Derivatives and Common Sources of Error . . . 88

7 Lecture 7: Computational Graphs and Automatic Differentiation . . . 89
7.1 From Backpropagation to Automatic Differentiation . . . 89
7.2 Constructing a Graph from an Arithmetic Expression . . . 89
7.3 The Chain Rule and Gradient Propagation . . . 91
7.4 Reverse-Mode Automatic Differentiation on a DAG . . . 93
7.5 A Complete Backward Pass for $f(x, y) = \sin(x) + xy$ . . . 96
7.6 Implementing a Small Automatic Differentiation Engine . . . 98
7.7 Local Derivative Rules for Common Operations . . . 101
7.8 A Loss Function Built on the Same Graph . . . 102
7.9 Jacobians, Vector Products, and Differentiation Modes . . . 103
7.10 Backpropagation as a Special Case . . . 104
7.11 Computational Cost, Memory, and Practical Issues . . . 105

8 Lecture 8: Neural Network Training and Matrix Automatic Differentiation 107
8.1 From Automatic Differentiation to Optimization . . . 107
8.2 Computational Graphs in a Training Step . . . 108
8.3 A Neural Network as a Composition of Elementary Operations . . . 110
8.4 Vector-Valued Operations and Their Adjoints . . . 113
8.5 Matrix Automatic Differentiation . . . 113
8.6 Affine Layers, Activation Functions, and Mini-Batches . . . 116
8.7 Automatic Differentiation in a Training Program . . . 118
8.8 Gradient Descent as a Discretized Dynamical System . . . 120
8.9 Empirical Risk, Mini-Batches, and Stochastic Gradients . . . 122
8.10 Verifying Automatically Computed Derivatives . . . 123

9 Lecture 9–10: Generalization, Regularization, and Optimization of Neural Networks . . . 125
9.1 Lecture 9: Overfitting and Generalization . . . 125
9.2 Lecture 10: Stochastic Gradient Descent and Optimization Dynamics . . . 131
9.3 Deep Networks and Gradient Propagation . . . 137
9.4 Putting the Optimization Techniques Together . . . 142

10 Lecture 11: Convolutional Neural Networks: Kernels, Channels, Padding, and Stride 143
10.1 Spatial Structure and Local Connectivity 143
10.2 A Single Convolutional Kernel 144
10.3 Output Size for a Valid Convolution 146
10.4 Multiple Kernels and Output Feature Maps 147
10.5 Convolution with Multiple Input Channels 147
10.6 Padding: Convolutions Near Image Boundaries 149
10.7 Stride: How Far the Kernel Moves 151
10.8 Parameter Count and Computational Efficiency 152
10.9 Why Weight Sharing Gives Translation Equivariance 154
10.10 Convolution as a Structured Linear Map 155
10.11 Backpropagation Through a Convolutional Layer 156
10.12 Stacking Convolutional Layers and Receptive Fields 159
10.13 Additional Exercises 160

11 Lecture 12: Hopfield Networks, Associative Memory, and Energy-Based Learning 162
11.1 Binary States and Network Architecture 162
11.2 Storing Memories with Hebbian Learning 164
11.3 The Hopfield Energy Function 167
11.4 Energy Minimization and Gradient Calculus 170
11.5 Learning the Correlation Matrix from an Objective 172
11.6 Memory Retrieval, Distance, and Attractors 174
11.7 Further Consequences of Symmetric Binary Networks 176
11.8 Exercises and Complete Solutions 177

12 Lecture 13: Restricted Boltzmann Machines and Energy-Based Generative Learning 180
12.1 Architecture of a Restricted Boltzmann Machine 180
12.2 Conditional Independence and Sampling Rules 182
12.3 The Visible Distribution and Free Energy 184
12.4 Maximum-Likelihood Learning 186
12.5 Blocked Gibbs Sampling 190
12.6 Contrastive Divergence and Practical Training 191
12.7 A Fully Worked RBM Example 192
12.8 Further Mathematical Properties 194
12.9 Exercises and Solutions 195

13 Lecture 14: Autoencoders, Dimensionality Reduction, and Word Embeddings 198
13.1 Autoencoders and Reconstruction Learning 198
13.2 The Bottleneck and Dimensionality Reduction 200
13.3 Linear Autoencoders and Principal-Component Analysis 201
13.4 Tied Weights in Autoencoders 203
13.5 Discrete Words and Vector Embeddings 204

13.6 Context-Based Learning and the Skip-Gram Model . . . 206
13.7 Word2Vec Corpus Preparation . . . 208
13.8 Negative Sampling . . . 209
13.9 Cosine Similarity and Embedding Geometry . . . 211
13.10Connections and Limitations . . . 213
13.11Exercises and Complete Solutions . . . 213

14 Lecture 15: Variational Autoencoders and Variational Inference . . . 216
14.1 From Deterministic Autoencoders to Latent-Variable Models . . . 216
14.2 Probabilistic Decoders and Reconstruction Loss . . . 218
14.3 Approximate Posterior Inference . . . 219
14.4 The Evidence Lower Bound . . . 220
14.5 A Tractable Gaussian Variational Family . . . 221
14.6 The Reparameterization Trick . . . 223
14.7 The Complete VAE Objective and Training Algorithm . . . 226
14.8 Worked Example: A Linear Gaussian Latent Model . . . 228
14.9 Interpretation, Extensions, and Limitations . . . 229
14.10Exercises and Complete Solutions . . . 231

15 Lecture 16: Diffusion Models and Denoising Score Matching . . . 234
15.1 Forward Diffusion: Gradually Adding Gaussian Noise . . . 234
15.2 An Exact Formula for the Noisy Observation . . . 235
15.3 Denoising and the Reverse Conditional Distribution . . . 237
15.4 Reparameterizing the Reverse Mean with Predicted Noise . . . 239
15.5 Variational Training of the Reverse Diffusion Process . . . 240
15.6 What the Denoiser Learns . . . 243
15.7 Training and Sampling Algorithms . . . 244
15.8 A Complete Numerical Example . . . 245
15.9 Connections and Mathematical Caveats . . . 246
15.10Exercises . . . 247

16 Lecture 17: Recurrent Neural Networks and Learning from Sequences . . . 250
16.1 Sequences and the Recurrent Hidden State . . . 250
16.2 Output Distributions and Sequence Prediction . . . 252
16.3 Differentiating the Sequence Loss . . . 254
16.4 A Worked Numerical Example of BPTT . . . 258
16.5 Stacked Recurrent Neural Networks . . . 259
16.6 Vanishing and Exploding Gradients in RNNs . . . 261
16.7 Training and Computational Considerations . . . 262
16.8 Further Examples and Exercises . . . 265

17 Lecture 18: Vanishing Gradients and Gated Recurrent Units . . . 268
17.1 Loss of Long-Range Information in Simple Recurrent Networks . . . 268
17.2 Vanishing and Exploding Gradients in Nonlinear RNNs . . . 271
17.3 Gated Recurrent Units: The Main Idea . . . 273

17.4 The Full Gated Recurrent Unit . . . 275
17.5 Differentiating the GRU: Exact State Jacobian . . . 277
17.6 Examples and Interpretation . . . 281
17.7 Practical Training and Connections . . . 282
17.8 Further Examples and Exercises . . . 284

18 Lecture 19: Attention Mechanisms and Transformer Representations . . . 288
18.1 From Word Embeddings to Context-Dependent Representations . . . 288
18.2 Scaled Dot-Product Attention . . . 289
18.3 The Attention Matrix and a Worked Example . . . 293
18.4 Why Position Must Be Represented Explicitly . . . 295
18.5 Multi-Head Attention . . . 298
18.6 Attention as a Differentiable Computational Graph . . . 300
18.7 Masking, Cross-Attention, and Computational Cost . . . 302
18.8 Connection to Transformer Blocks . . . 304
18.9 Exercises and Solutions . . . 305

19 Lecture 20: Normalization and the Transformer Encoder–Decoder . . . 309
19.1 Why Normalize Hidden Activations? . . . 309
19.2 Batch Normalization . . . 310
19.3 Layer Normalization . . . 312
19.4 Backpropagation Through Normalization . . . 314
19.5 Residual Connections and Add & Norm . . . 316
19.6 Position-Wise Feedforward Networks . . . 317
19.7 Transformer Encoder Architecture . . . 318
19.8 Transformer Decoder Architecture . . . 319
19.9 From Decoder Hidden States to Token Probabilities . . . 321
19.10Parallel Training and Sequential Generation . . . 323
19.11Worked Transformer Block Example . . . 324
19.12Summary of the Architecture . . . 324
19.13Exercises . . . 325

20 Lecture 21: Adversarial Attacks and Robustness of Neural Networks . . . 328
20.1 Classification and the Geometry of Decision Boundaries . . . 328
20.2 Adversarial Examples and the Threat Model . . . 329
20.3 Decision-Boundary Distance and Local Robustness . . . 331
20.4 The Fast Gradient Sign Method . . . 333
20.5 Why High Input Dimension Matters . . . 335
20.6 Iterative Attacks and Projection . . . 336
20.7 Adversarial Robustness as an Optimization Problem . . . 336
20.8 Robust Classification Margins and Certificates . . . 338
20.9 A Small Worked Linear Attack . . . 339
20.10Implementation and Evaluation . . . 339
20.11Exercises . . . 340

21 Lecture 22: Adversarial Defense and TRADES Robust Training 344
21.1 From Adversarial Attacks to Adversarial Defense 344
21.2 Classification Boundaries and the Robustness Trade-off 345
21.3 Why the Zero–One Objective is Difficult to Optimize 347
21.4 Adversarial Training as Minimax Optimization 348
21.5 TRADES: Separating Accuracy from Boundary Stability 349
21.6 How Prediction Stability Relates to Classification Robustness 351
21.7 Inner Optimization: Finding Discrepant Nearby Inputs 352
21.8 Outer Optimization and Gradients of the TRADES Loss 354
21.9 Training Algorithm 356
21.10A Detailed Binary Logistic Example 357
21.11Choosing the Robustness Parameters 358
21.12Robust Risk, Empirical Risk, and Generalization 358
21.13Numerical Example: Robust and Clean Accuracy 359
21.14Worked Exercises 360

22 Lecture 23: Generative Adversarial Networks and Adversarial Generative Learning 363
22.1 From Discriminative Learning to Generative Learning 363
22.2 The Real-versus-Fake Classification Problem 364
22.3 The Optimal Discriminator for a Fixed Generator 366
22.4 The Jensen–Shannon Divergence and the GAN Equilibrium 367
22.5 Generator Gradients and the Saturation Problem 369
22.6 Training GANs by Alternating Optimization 371
22.7 Why GAN Optimization Can Be Unstable 372
22.8 Separated Supports and Weak Learning Signals 373
22.9 Mode Collapse and Distributional Coverage 374
22.10A Worked Discrete GAN Example 375
22.11A Gaussian Classification Example 376
22.12Practical Stabilization and Alternative Generative Objectives 376
22.13Worked Exercises 378

1 Lecture 1: Neurons and Membrane Potentials

1.1 Structure of a Neuron

A *neuron* is a specialized cell that receives, processes, and transmits signals. Its principal components are:

1. **Cell Body (Soma).** Contains the nucleus, which is surrounded by the cell membrane.
2. **Dendrites.** Branching structures that receive input signals from other neurons.
3. **Axon.** Transmits signals away from the soma.
4. **Synapse.** A junction at the end of an axon through which a neuron communicates with another cell.

1.2 Ion Channels and Pumps

The neuronal cell membrane contains ion channels and pumps that regulate the movement of ions between the inside and outside of the cell.

The principal mechanisms are:

1. **Sodium (Na^+) Channels.** Allow sodium ions to enter the cell. The concentration of Na^+ is higher outside the cell than inside. Hence, when sodium channels open, sodium ions tend to flow inward.

$$Na^+_{\text{outside}} \longrightarrow Na^+_{\text{inside}}.$$

2. **Potassium (K^+) Channels.** Allow potassium ions to leave the cell. The concentration of K^+ is higher inside the cell than outside. Hence, when potassium channels open, potassium ions tend to flow outward.

$$K^+_{\text{inside}} \longrightarrow K^+_{\text{outside}}.$$

3. **Sodium-Potassium (Na^+/K^+) Pump.** Actively transports three sodium ions out of the cell and two potassium ions into the cell during each cycle.

$$\begin{aligned} 3Na^+_{\text{inside}} &\longrightarrow 3Na^+_{\text{outside}}, \\ 2K^+_{\text{outside}} &\longrightarrow 2K^+_{\text{inside}}. \end{aligned}$$

   The pump maintains the ionic concentration gradients that are essential for sustaining the resting membrane potential.

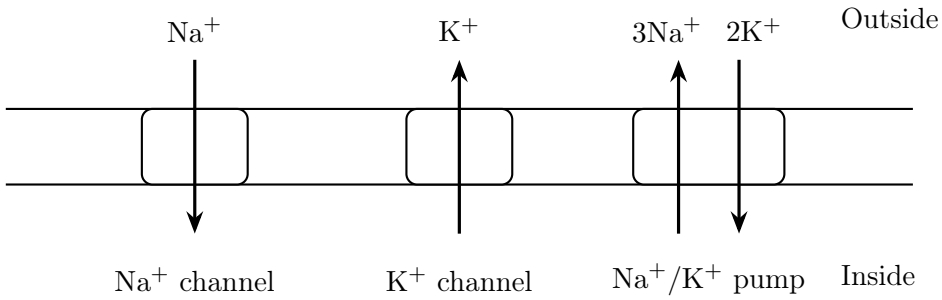


Figure 1: Ion transport across the neuronal cell membrane.

1.3 Membrane Potential

Definition 1.1 (Membrane Potential)**.** The *membrane potential*, denoted by V, is the electrical potential difference between the inside and outside of a cell, defined by

$$V := V_{\text{inside}} - V_{\text{outside}}.$$

For a typical neuron at rest, the membrane potential is approximately

$$\boxed{V_{\text{rest}} \approx -70\,\text{mV}.}$$

The negative sign indicates that the inside of the cell is electrically negative relative to the outside.

Remark 1.1. The resting membrane potential is maintained by the unequal distribution of ions across the membrane, its selective permeability to different ions, and the concentration gradients sustained by the sodium-potassium pump.

1.4 Action Potential

Definition 1.2 (Action Potential)**.** An *action potential* is a rapid, temporary change in the membrane potential of a neuron that allows an electrical signal to propagate along its axon.

Let V_{th} denote the threshold membrane potential required to initiate an action potential.

When the membrane potential rises sufficiently to reach the threshold,

$$V \geq V_{\text{th}},$$

voltage-gated sodium channels open, allowing additional sodium ions to enter the cell.

Consequently,

$$\text{Na}^+ \text{ channels open} \quad \Longrightarrow \quad \text{Na}^+ \text{ influx increases} \quad \Longrightarrow \quad V \text{ increases.}$$

This increase in membrane potential is called *depolarization*. The resulting positive feedback produces the rapid rising phase of the action potential.

2 Lecture 2: Hodgkin–Huxley Model, Spiking Neurons, and Artificial Neurons

2.1 The Hodgkin–Huxley Model

The *Hodgkin–Huxley (HH) model* describes how ionic currents across a neuronal membrane generate an action potential. It treats the membrane as a capacitor in parallel with conductances associated with sodium, potassium, and leakage currents. The sodium and potassium *channels*, rather than the sodium–potassium pump, are the voltage-dependent components responsible for the rapid dynamics of an action potential.

Let $V(t)$ denote the intracellular membrane potential relative to the extracellular potential. The state of the HH model consists of the voltage V and three gating variables $m, h, n \in [0, 1]$. The gating variables represent the fraction or probability of individual channel gates being in a permissive state, rather than the fraction of complete channels that are open.

2.1.1 Voltage-Dependent Gating Variables

Definition 2.1 (Gating variables)**.** The HH model uses three voltage-dependent gating variables:

- $n(t)$: activation variable for potassium channels;
- $m(t)$: activation variable for sodium channels;
- $h(t)$: inactivation variable for sodium channels.

The simplified gating interpretation assumes four independent activation gates for a potassium channel, and three activation gates together with one inactivation gate for a sodium channel.

Thus the fractions of fully conducting channels are represented by

$$\begin{aligned} \text{Potassium-channel open fraction} &= n^4, \\ \text{Sodium-channel open fraction} &= m^3 h. \end{aligned}$$

In particular, if n is the probability that one potassium activation gate is open, independence gives

$$\mathbb{P}(\text{all four activation gates open}) = n \cdot n \cdot n \cdot n = n^4.$$

Likewise, if the three sodium activation gates and the inactivation gate are independent, then

$$\mathbb{P}(\text{sodium channel conducts}) = m \cdot m \cdot m \cdot h = m^3 h.$$

The factor h is the probability that the sodium inactivation gate is *not blocking* the channel. Thus sodium conduction increases as m rises, but decreases as h falls.

Definition 2.2 (Gating dynamics)**.** For $x \in \{m, h, n\}$, the HH gating dynamics are

$$\boxed{\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{x_\infty(V) - x}{\tau_x(V)}.}$$

Here $x_\infty(V)$ is the steady-state value of the gate at fixed voltage V, and $\tau_x(V) > 0$ is its voltage-dependent time constant. Explicitly,

$$\begin{aligned}
\frac{\mathrm{d}n}{\mathrm{d}t} &= \frac{n_\infty(V) - n}{\tau_n(V)}, \\
\frac{\mathrm{d}m}{\mathrm{d}t} &= \frac{m_\infty(V) - m}{\tau_m(V)}, \\
\frac{\mathrm{d}h}{\mathrm{d}t} &= \frac{h_\infty(V) - h}{\tau_h(V)}.
\end{aligned}$$

These equations describe relaxation toward the voltage-dependent equilibrium: whenever $x < x_\infty(V)$, the gate variable increases; whenever $x > x_\infty(V)$, it decreases. With the voltage held fixed at V_0, the equation can be solved exactly:

$$\begin{aligned}
\frac{\mathrm{d}x}{\mathrm{d}t} &= \frac{x_\infty(V_0) - x}{\tau_x(V_0)}, \\
\frac{\mathrm{d}}{\mathrm{d}t}\big(x - x_\infty(V_0)\big) &= -\frac{x - x_\infty(V_0)}{\tau_x(V_0)}, \\
x(t) - x_\infty(V_0) &= \big(x(0) - x_\infty(V_0)\big) e^{-t/\tau_x(V_0)}, \\
x(t) &= \boxed{x_\infty(V_0) + \big(x(0) - x_\infty(V_0)\big) e^{-t/\tau_x(V_0)}}.
\end{aligned}$$

Thus the deviation from the equilibrium value decreases exponentially, with time constant $\tau_x(V_0)$.

Proposition 2.1 (Transition-rate representation)**.** *Suppose a gate switches between a closed state and an open state with voltage-dependent transition rates $\alpha_x(V) \geq 0$ and $\beta_x(V) \geq 0$. Its open-state probability $x(t)$ satisfies*

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \alpha_x(V)(1 - x) - \beta_x(V)x.$$

Provided $\alpha_x(V) + \beta_x(V) > 0$, this is equivalent to the relaxation equation with

$$\boxed{x_\infty(V) = \frac{\alpha_x(V)}{\alpha_x(V) + \beta_x(V)}, \qquad \tau_x(V) = \frac{1}{\alpha_x(V) + \beta_x(V)}.}$$

Proof. The increase in the open-state probability is the rate of opening multiplied by the closed probability, minus the rate of closing multiplied by the open probability. Therefore,

$$\begin{aligned}\frac{\mathrm{d}x}{\mathrm{d}t} &= \alpha_x(V)(1-x) - \beta_x(V)x \\ &= \alpha_x(V) - \big(\alpha_x(V) + \beta_x(V)\big)x \\ &= \big(\alpha_x(V) + \beta_x(V)\big)\left(\frac{\alpha_x(V)}{\alpha_x(V)+\beta_x(V)} - x\right) \\ &= \frac{x_\infty(V) - x}{\tau_x(V)}.\end{aligned}$$

The claimed expressions follow by comparison. In particular, the dynamics preserve $0 \le x \le 1$: at $x = 0$ the derivative is nonnegative, while at $x = 1$ it is nonpositive. □

Remark 2.1. During depolarization, $m_\infty(V)$ and $n_\infty(V)$ generally increase, while $h_\infty(V)$ generally decreases. Sodium activation m is relatively fast, sodium inactivation h is slower, and potassium activation n is delayed. These different response times help produce the rapid rise, fall, and after-hyperpolarization of an action potential.

2.1.2 Dynamics of the Membrane Potential

The membrane capacitance C stores electrical charge. Since the capacitor current is

$$I_C = C\frac{\mathrm{d}V}{\mathrm{d}t},$$

conservation of electrical current gives

$$C\frac{\mathrm{d}V}{\mathrm{d}t} = J_{\text{in}} - I_L - I_{\text{Na}} - I_{\text{K}},$$

where J_{in} is an applied inward current and the ionic currents use the outward-positive sign convention.

Definition 2.3 (Ionic currents)**.** The three currents in the HH model are

$$\begin{aligned}I_L &= g_L(V - V_L), \\ I_{\text{Na}} &= \bar{g}_{\text{Na}} m^3 h (V - V_{\text{Na}}), \\ I_{\text{K}} &= \bar{g}_{\text{K}} n^4 (V - V_{\text{K}}),\end{aligned}$$

where $g_L > 0$ is the leak conductance; $\bar{g}_{\text{Na}}, \bar{g}_{\text{K}} > 0$ are the maximum sodium and potassium conductances; and $V_L, V_{\text{Na}}, V_{\text{K}}$ are their reversal potentials.

Substituting the currents into the conservation equation gives the *Hodgkin–Huxley membrane equation*:

$$\boxed{C\frac{\mathrm{d}V}{\mathrm{d}t} = J_{\text{in}} - g_L(V - V_L) - \bar{g}_{\text{Na}} m^3 h (V - V_{\text{Na}}) - \bar{g}_{\text{K}} n^4 (V - V_{\text{K}}).}$$

Together with the gating equations, the complete four-dimensional HH system is

$$\begin{aligned}
C\frac{\mathrm{d}V}{\mathrm{d}t} &= J_{\text{in}} - g_L(V - V_L) - \bar{g}_{\text{Na}} m^3 h (V - V_{\text{Na}}) - \bar{g}_{\text{K}} n^4 (V - V_{\text{K}}), \\
\frac{\mathrm{d}m}{\mathrm{d}t} &= \frac{m_\infty(V) - m}{\tau_m(V)}, \\
\frac{\mathrm{d}h}{\mathrm{d}t} &= \frac{h_\infty(V) - h}{\tau_h(V)}, \\
\frac{\mathrm{d}n}{\mathrm{d}t} &= \frac{n_\infty(V) - n}{\tau_n(V)}.
\end{aligned}$$

Remark 2.2 (Direction of current). The reversal potential determines the sign of each ionic current. For typical neuronal voltages,

$$\begin{aligned}
V < V_{\text{Na}} \quad &\Longrightarrow \quad I_{\text{Na}} < 0 \qquad &&\text{(inward sodium current)}, \\
V > V_{\text{K}} \quad &\Longrightarrow \quad I_{\text{K}} > 0 \qquad &&\text{(outward potassium current)}.
\end{aligned}$$

The voltage equation subtracts these outward-positive currents. Hence sodium influx tends to increase V, while potassium efflux tends to decrease V. The leak current drives voltage toward V_L.

2.1.3 Action Potential Dynamics

An *action potential* is a brief excursion of the membrane potential above its resting level, followed by a return to rest. A typical resting potential is approximately $-70\,\text{mV}$, while an illustrative threshold is around $-55\,\text{mV}$. The precise values depend on the neuron and operating conditions; in the HH model the spike threshold is a dynamical property rather than a universal fixed voltage.

1. **Resting state.** Without sufficient stimulation, inward and outward currents balance, so $\mathrm{d}V/\mathrm{d}t \approx 0$. The membrane stays near its resting voltage.
2. **Depolarization.** A sufficiently strong input current raises V. Fast sodium activation increases m, opening more sodium channels and causing further sodium influx. This positive feedback creates the rising phase of the spike.
3. **Peak and sodium inactivation.** Sodium channels become inactivated as h decreases. Consequently the inward sodium current weakens, even if m remains elevated briefly.
4. **Repolarization.** Delayed activation of potassium channels increases n, resulting in greater potassium efflux and a reduction in V.
5. **After-hyperpolarization.** Because the potassium conductance closes gradually, potassium current can temporarily bring V below its resting level.
6. **Recovery.** Potassium activation diminishes and sodium inactivation recovers. The membrane returns near rest. The sodium–potassium pump maintains the underlying ionic gradients over longer timescales; it is not what directly drives each rapid falling phase.

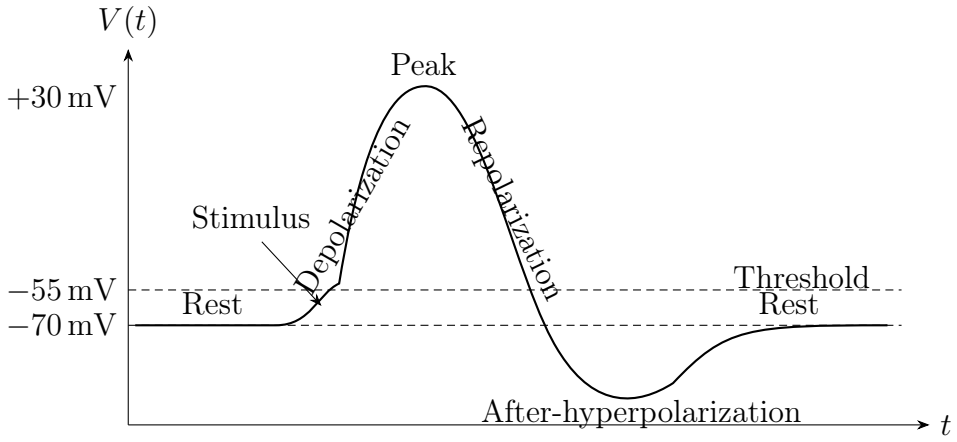


Figure 2: Schematic action potential. The indicated threshold and voltages are illustrative.

Remark 2.3 (Refractory periods). During the *absolute refractory period*, many sodium channels remain inactivated and an additional action potential cannot readily be initiated. During the *relative refractory period*, an action potential may occur but requires stronger stimulation because sodium channels have not fully recovered and/or potassium conductance remains elevated. These effects limit how frequently a neuron can spike.

2.2 The Leaky Integrate-and-Fire Model

The HH model resolves the ionic currents and the waveform of each action potential. The *leaky integrate-and-fire (LIF) model* instead describes the subthreshold membrane voltage and represents spikes by threshold crossings, resets, and optionally a refractory period. It does not attempt to reproduce the full spike waveform.

2.2.1 Subthreshold Voltage Dynamics

Definition 2.4 (LIF membrane equation)**.** Let $V(t)$ denote the membrane voltage, $C > 0$ the membrane capacitance, $R > 0$ the membrane resistance, V_L the leak reversal potential, and $J_{\text{in}}(t)$ the applied input current. Between spikes, the LIF model is

$$\boxed{C\frac{\mathrm{d}V}{\mathrm{d}t} = J_{\text{in}}(t) - \frac{V - V_L}{R}.}$$

The capacitor accumulates incoming charge, while the leak term opposes deviations from V_L. Define the *membrane time constant*

$$\tau_m := RC.$$

Multiplying the LIF equation by R gives

$$RC\frac{\mathrm{d}V}{\mathrm{d}t} = RJ_{\text{in}}(t) - (V - V_L),$$
$$\tau_m\frac{\mathrm{d}V}{\mathrm{d}t} = \boxed{V_L + RJ_{\text{in}}(t) - V}.$$

Remark 2.4 (Physical meaning of the time constant). Since R has units of resistance and C has units of capacitance, RC has units of time. A larger τ_m means that the voltage responds more slowly to changes in current and retains the effect of past inputs for longer. A smaller τ_m means faster relaxation toward equilibrium.

2.2.2 Solution of the Subthreshold Equation

Suppose that the applied current is constant, $J_{\text{in}}(t) = J_0$. Define the equilibrium voltage

$$V_\infty := V_L + RJ_0.$$

Then the differential equation becomes

$$\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} = V_\infty - V.$$

Proposition 2.2 (Voltage under constant input)**.** *For initial condition $V(0) = V_0$, the subthreshold LIF solution is*

$$\boxed{V(t) = V_\infty + (V_0 - V_\infty)e^{-t/\tau_m}.}$$

In particular, $V(t) \to V_\infty$ exponentially as $t \to \infty$, as long as no threshold crossing interrupts the dynamics.

Proof. Separating variables yields

$$\begin{aligned}
\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} &= V_\infty - V, \\
\frac{\mathrm{d}V}{V_\infty - V} &= \frac{\mathrm{d}t}{\tau_m}, \\
\int_{V_0}^{V(t)} \frac{\mathrm{d}u}{V_\infty - u} &= \int_0^t \frac{\mathrm{d}s}{\tau_m}, \\
-\log\big|V_\infty - V(t)\big| + \log\big|V_\infty - V_0\big| &= \frac{t}{\tau_m}, \\
\frac{V_\infty - V(t)}{V_\infty - V_0} &= e^{-t/\tau_m}, \\
V(t) &= V_\infty + (V_0 - V_\infty)e^{-t/\tau_m}.
\end{aligned}$$

The formula also applies to $V_0 = V_\infty$ by direct substitution. □

For a time-dependent input, the same equation has the integrating-factor solution

$$\begin{aligned}
\frac{\mathrm{d}V}{\mathrm{d}t} + \frac{V}{\tau_m} &= \frac{V_L + RJ_{\text{in}}(t)}{\tau_m}, \\
\frac{\mathrm{d}}{\mathrm{d}t}\left(e^{t/\tau_m}V(t)\right) &= \frac{e^{t/\tau_m}}{\tau_m}\big(V_L + RJ_{\text{in}}(t)\big), \\
V(t) &= \boxed{V_L + (V_0 - V_L)e^{-t/\tau_m} + \frac{1}{C}\int_0^t e^{-(t-s)/\tau_m} J_{\text{in}}(s)\,\mathrm{d}s}.
\end{aligned}$$

The integral is an exponentially weighted history of the input current. This is the *integration* in the name "integrate-and-fire"; the decay factor captures the *leak*.

2.2.3 Threshold, Reset, and Refractory Period

Definition 2.5 (Spike-generation rule)**.** Choose a threshold V_{th} and reset potential $V_r < V_{\text{th}}$. A spike time t_i is the first upward threshold crossing after the preceding refractory interval. For continuous subthreshold trajectories,

$$V(t_i^-) = V_{\text{th}}, \qquad V(t_i^+) = V_r.$$

Here t_i^- and t_i^+ denote the instants immediately before and after the modeled reset. A spike is recorded at t_i, but the spike waveform is not explicitly modeled.

If an absolute refractory period $\tau_{\text{ref}} > 0$ is imposed, the voltage is held at its reset value for

$$t \in [t_i, t_i + \tau_{\text{ref}}),$$

and the subthreshold equation resumes at $t_i + \tau_{\text{ref}}$. In some normalized descriptions the threshold is chosen as 1 and the reset voltage as 0.

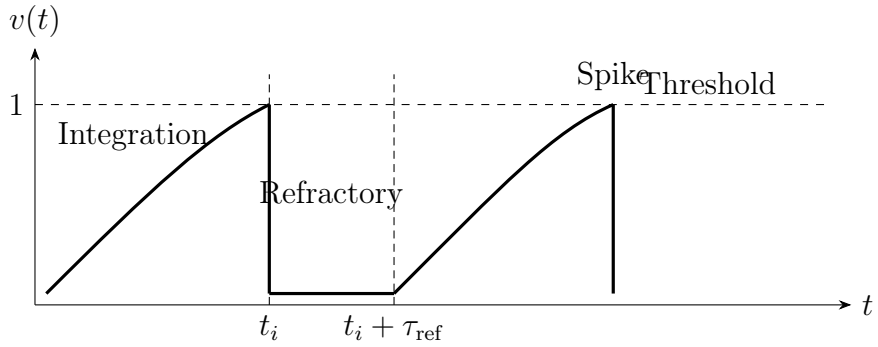


Figure 3: Schematic normalized LIF voltage showing threshold crossing, reset, and an absolute refractory interval.

2.2.4 Normalized LIF Dynamics

For $V_L < V_{\text{th}}$, introduce the dimensionless voltage and current

$$v := \frac{V - V_L}{V_{\text{th}} - V_L}, \qquad v_{\text{in}}(t) := \frac{RJ_{\text{in}}(t)}{V_{\text{th}} - V_L}.$$

Then $V = V_L + (V_{\text{th}} - V_L)v$, and the threshold is $v_{\text{th}} = 1$.

Substituting into the dimensional equation gives

$$\begin{aligned}\tau_m \frac{\mathrm{d}}{\mathrm{d}t}\big(V_L + (V_{\mathrm{th}} - V_L)v\big) &= RJ_{\mathrm{in}} - (V_{\mathrm{th}} - V_L)v,\\ \tau_m (V_{\mathrm{th}} - V_L)\frac{\mathrm{d}v}{\mathrm{d}t} &= RJ_{\mathrm{in}} - (V_{\mathrm{th}} - V_L)v,\\ \tau_m \frac{\mathrm{d}v}{\mathrm{d}t} &= \boxed{v_{\mathrm{in}}(t) - v}.\end{aligned}$$

Taking the reset voltage to equal the leak potential gives $v(0) = 0$ immediately after a refractory period. For constant v_{in}, the subthreshold solution is

$$\boxed{v(t) = v_{\mathrm{in}}\big(1 - e^{-t/\tau_m}\big).}$$

This shows explicitly that the potential approaches v_{in} from below when $v_{\mathrm{in}} > 0$.

2.2.5 Interspike Interval and Firing Rate

Under a constant input $v_{\mathrm{in}} > 1$, the normalized voltage eventually reaches the threshold $v = 1$. Define the time required to go from reset $v(0) = 0$ to threshold by

$$T := \inf\{t > 0 : v(t) = 1\}.$$

Proposition 2.3 (Time to threshold)**.** *For constant normalized input $v_{\mathrm{in}} > 1$, the time to threshold is*

$$\boxed{T = \tau_m \log\left(\frac{v_{\mathrm{in}}}{v_{\mathrm{in}} - 1}\right).}$$

If $v_{\mathrm{in}} \le 1$, threshold is never reached in finite time from reset, so $T = +\infty$.

Proof. From the explicit voltage solution, threshold crossing occurs when

$$\begin{aligned}1 &= v_{\mathrm{in}}\big(1 - e^{-T/\tau_m}\big),\\ \frac{1}{v_{\mathrm{in}}} &= 1 - e^{-T/\tau_m},\\ e^{-T/\tau_m} &= 1 - \frac{1}{v_{\mathrm{in}}} = \frac{v_{\mathrm{in}} - 1}{v_{\mathrm{in}}},\\ -\frac{T}{\tau_m} &= \log\left(\frac{v_{\mathrm{in}} - 1}{v_{\mathrm{in}}}\right),\\ T &= \tau_m \log\left(\frac{v_{\mathrm{in}}}{v_{\mathrm{in}} - 1}\right).\end{aligned}$$

For $v_{\mathrm{in}} < 1$, the trajectory converges to a voltage strictly below the threshold. At $v_{\mathrm{in}} = 1$ it approaches threshold asymptotically but never crosses it in finite time. □

Definition 2.6 (Steady-state firing rate)**.** With refractory period $\tau_{\text{ref}} \geq 0$, each complete spiking cycle consists of an integration period T followed by a refractory period τ_{ref}. The steady-state firing rate is therefore

$$G(v_{\text{in}}) := \frac{1}{T + \tau_{\text{ref}}}.$$

Consequently,

$$\boxed{G(v_{\text{in}}) = \begin{cases} \dfrac{1}{\tau_{\text{ref}} + \tau_m \log\left(\frac{v_{\text{in}}}{v_{\text{in}}-1}\right)}, & v_{\text{in}} > 1, \\ 0, & v_{\text{in}} \leq 1. \end{cases}}$$

The formula can equivalently be expressed using physical voltages. If the post-refractory voltage resets to V_r and the constant current equilibrium is $V_\infty = V_L + RJ_0 > V_{\text{th}}$, then

$$\begin{aligned} V_{\text{th}} &= V_\infty + (V_r - V_\infty)e^{-T/\tau_m}, \\ \frac{V_\infty - V_{\text{th}}}{V_\infty - V_r} &= e^{-T/\tau_m}, \\ T &= \boxed{\tau_m \log\left(\frac{V_\infty - V_r}{V_\infty - V_{\text{th}}}\right)}. \end{aligned}$$

The normalized formula is the special case $V_r = V_L$.

Proposition 2.4 (Firing-rate properties)**.** *For $v_{\text{in}} > 1$, the firing rate is strictly increasing in v_{in}. Moreover,*

$$\begin{aligned} \lim_{v_{\text{in}} \downarrow 1} G(v_{\text{in}}) &= 0, \\ \lim_{v_{\text{in}} \to \infty} G(v_{\text{in}}) &= \begin{cases} 1/\tau_{\text{ref}}, & \tau_{\text{ref}} > 0, \\ +\infty, & \tau_{\text{ref}} = 0. \end{cases} \end{aligned}$$

Proof. Write $T(v) = \tau_m \log(v/(v-1))$ for $v > 1$. Then

$$\begin{aligned} T'(v) &= \tau_m \left(\frac{1}{v} - \frac{1}{v-1}\right) \\ &= -\frac{\tau_m}{v(v-1)} < 0. \end{aligned}$$

Hence

$$G'(v) = -\frac{T'(v)}{(\tau_{\text{ref}} + T(v))^2} = \frac{\tau_m}{v(v-1)\big(\tau_{\text{ref}} + T(v)\big)^2} > 0.$$

As $v \downarrow 1$, $v/(v-1) \to \infty$, hence $T(v) \to \infty$ and $G(v) \to 0$. As $v \to \infty$, $v/(v-1) \to 1$ and $T(v) \to 0$, yielding the stated upper limit. □

Remark 2.5 (Interpretation). The firing-rate function G acts as an input–output nonlinearity: a subthreshold input gives zero firing rate, while a larger suprathreshold input leads to more frequent spikes. A positive refractory time places a hard upper bound on the firing rate. Thus the LIF neuron turns a continuous input current into a sequence of discrete spike events.

2.3 Artificial Neurons and Activation Functions

An *artificial neuron* abstracts the input–output computation of a biological neuron without explicitly simulating ion channels or membrane voltage. It forms an affine combination of its inputs and applies a nonlinear activation function.

Definition 2.7 (Artificial neuron)**.** For input vector $x \in \mathbb{R}^d$, weight vector $w \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$, define

$$z = w^\top x + b = \sum_{j=1}^{d} w_j x_j + b.$$

The output (or activation) is

$$\boxed{a = \sigma(z) = \sigma(w^\top x + b),}$$

where σ denotes the activation function.

Common activation functions include the following.

2.3.1 Sigmoid, Hyperbolic Tangent, and ReLU

Definition 2.8 (Sigmoid activation)**.** The *logistic sigmoid* is

$$\boxed{\sigma(z) = \frac{1}{1+e^{-z}}, \qquad z \in \mathbb{R}.}$$

Its range is $(0, 1)$, and its derivative is

$$\begin{aligned}\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) \\ &= \boxed{\sigma(z)(1-\sigma(z)).}\end{aligned}$$

Definition 2.9 (Hyperbolic tangent)**.** The *hyperbolic tangent* activation is

$$\boxed{\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},}$$

with range $(-1, 1)$ and derivative

$$\boxed{\frac{\mathrm{d}}{\mathrm{d}z}\tanh(z) = 1 - \tanh^2(z).}$$

Definition 2.10 (Rectified linear unit)**.** The *rectified linear unit (ReLU)* is

$$\boxed{\mathrm{ReLU}(z) = \max\{0, z\} = \begin{cases} 0, & z \le 0, \\ z, & z > 0. \end{cases}}$$

For $z \neq 0$ its derivative is

$$\mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

ReLU is not differentiable at $z = 0$; implementations generally select a subgradient there.

Remark 2.6 (Why nonlinear activation matters). Without an activation function, successive affine layers collapse into a single affine map. Indeed, if

$$h = W_1 x + b_1, \qquad y = W_2 h + b_2,$$

then

$$\begin{aligned} y &= W_2(W_1 x + b_1) + b_2 \\ &= (W_2 W_1)x + (W_2 b_1 + b_2). \end{aligned}$$

Thus adding affine layers alone does not create nonlinear expressive power.

2.3.2 Softmax Activation

While sigmoid, tanh, and ReLU act on scalar pre-activations, *softmax* converts a vector of scores into a vector of nonnegative values summing to one.

Definition 2.11 (Softmax)**.** For a vector $z = (z_1, \ldots, z_N) \in \mathbb{R}^N$, the softmax function is defined componentwise by

$$\boxed{\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\displaystyle\sum_{j=1}^{N} e^{z_j}}, \qquad i = 1, \ldots, N.}$$

Proposition 2.5 (Normalization and positivity)**.** *For every $z \in \mathbb{R}^N$,*

$$\mathrm{softmax}(z)_i > 0 \qquad \textit{and} \qquad \sum_{i=1}^{N} \mathrm{softmax}(z)_i = 1.$$

Therefore $\mathrm{softmax}(z)$ *is a probability vector.*

Proof. Each exponential is strictly positive, so the denominator is positive and every softmax component is positive. Furthermore,

$$\begin{aligned}\sum_{i=1}^{N} \operatorname{softmax}(z)_i &= \sum_{i=1}^{N} \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}} \\ &= \frac{\sum_{i=1}^{N} e^{z_i}}{\sum_{j=1}^{N} e^{z_j}} = 1.\end{aligned}$$

□

Proposition 2.6 (Shift invariance)**.** *For any $c \in \mathbb{R}$ and the all-ones vector $\mathbf{1} \in \mathbb{R}^N$,*

$$\boxed{\operatorname{softmax}(z + c\mathbf{1}) = \operatorname{softmax}(z).}$$

Proof. For each i,

$$\begin{aligned}\operatorname{softmax}(z + c\mathbf{1})_i &= \frac{e^{z_i + c}}{\sum_{j=1}^{N} e^{z_j + c}} \\ &= \frac{e^{c} e^{z_i}}{e^{c} \sum_{j=1}^{N} e^{z_j}} \\ &= \operatorname{softmax}(z)_i.\end{aligned}$$

□

Remark 2.7 (Numerically stable softmax). By shift invariance, taking $c = -\max_j z_j$ gives

$$\operatorname{softmax}(z)_i = \frac{e^{z_i - z_{\max}}}{\sum_{j=1}^{N} e^{z_j - z_{\max}}}, \qquad z_{\max} = \max_j z_j.$$

All exponential arguments are then nonpositive, reducing the risk of overflow.

Proposition 2.7 (Softmax Jacobian)**.** *Write $p_i = \operatorname{softmax}(z)_i$. For $1 \le i, k \le N$,*

$$\boxed{\frac{\partial p_i}{\partial z_k} = p_i(\delta_{ik} - p_k),}$$

where $\delta_{ik} = 1$ if $i = k$ and 0 otherwise. Equivalently,

$$D \operatorname{softmax}(z) = \operatorname{diag}(p) - pp^{\top}.$$

Proof. Let $S = \sum_{j=1}^{N} e^{z_j}$. Differentiating $p_i = e^{z_i}/S$ by the quotient rule gives

$$\begin{aligned}\frac{\partial p_i}{\partial z_k} &= \frac{\delta_{ik} e^{z_i} S - e^{z_i} e^{z_k}}{S^2} \\ &= \frac{e^{z_i}}{S}\left(\delta_{ik} - \frac{e^{z_k}}{S}\right) \\ &= p_i(\delta_{ik} - p_k).\end{aligned}$$

□

Remark 2.8 (Biological versus artificial neurons). The HH model describes the detailed biophysical dynamics of electrical signaling, whereas the LIF model preserves only the subthreshold dynamics and spike timings. A conventional artificial neuron abstracts this process further into the static mapping $x \mapsto \sigma(w^\top x + b)$. The models serve different purposes: reproducing membrane physics, simulating spike-based computation, and learning a nonlinear input–output transformation, respectively.

3 Lecture 3: Postsynaptic Currents, Spike Trains, and Spiking Neural Networks

3.1 Postsynaptic Current Filters

A presynaptic neuron communicates with a postsynaptic neuron by producing a sequence of action potentials, or *spikes*. Although a spike is a brief electrical event, its influence on a postsynaptic cell generally persists over a nonzero period of time. A *postsynaptic current (PSC) filter* describes the time course of this influence following a single incoming spike.

Definition 3.1 (Causal postsynaptic-current kernel)**.** A *causal postsynaptic-current kernel* is a function $h : \mathbb{R} \to \mathbb{R}$ such that $h(t) = 0$ for $t < 0$. A common family of nonnegative kernels is

$$\boxed{h(t) = \begin{cases} kt^n e^{-t/\tau_s}, & t \geq 0, \\ 0, & t < 0, \end{cases}}$$

where $n \in \{0, 1, 2, \dots\}$, $\tau_s > 0$ is the synaptic decay time constant, and $k > 0$ is a normalization constant. The input spike is taken to occur at $t = 0$.

The exponential factor causes the response to decay over time; the polynomial factor t^n determines the initial rise of the response. In particular, $n = 0$ gives a purely exponential response, while $n \geq 1$ produces a response that first rises and subsequently falls. This is a mathematical model for synaptic response dynamics, rather than a model of the action-potential waveform itself.

3.1.1 Normalization of the PSC Kernel

The kernel is often normalized to have unit integral:

$$\int_0^\infty h(t)\,\mathrm{d}t = 1.$$

This convention makes the filter a unit-area representation of a single spike, so that the total signed synaptic effect can be controlled separately by a connection weight.

Proposition 3.1 (Normalization constant)**.** *For $n \in \{0, 1, 2, \dots\}$, the kernel*

$$h(t) = kt^n e^{-t/\tau_s} \mathbf{1}_{\{t \geq 0\}}$$

has unit integral if and only if

$$\boxed{k = \frac{1}{n!\,\tau_s^{n+1}}.}$$

Thus the normalized kernel is

$$\boxed{h_n(t) = \frac{t^n}{n!\,\tau_s^{n+1}} e^{-t/\tau_s} \mathbf{1}_{\{t \geq 0\}}.}$$

Proof. Substituting the proposed kernel into the unit-area requirement gives

$$\begin{aligned} 1 &= \int_0^\infty h(t)\,\mathrm{d}t \\ &= k\int_0^\infty t^n e^{-t/\tau_s}\,\mathrm{d}t. \end{aligned}$$

Let $u = t/\tau_s$. Then $t = \tau_s u$ and $\mathrm{d}t = \tau_s\,\mathrm{d}u$, so

$$\begin{aligned} 1 &= k\int_0^\infty (\tau_s u)^n e^{-u}\tau_s\,\mathrm{d}u \\ &= k\tau_s^{n+1}\int_0^\infty u^n e^{-u}\,\mathrm{d}u \\ &= k\tau_s^{n+1}\Gamma(n+1) \\ &= k\tau_s^{n+1}n!. \end{aligned}$$

Solving for k yields $k = (n!\tau_s^{n+1})^{-1}$, as required. □

Remark 3.1 (Units and interpretation). A unit-area kernel has units of inverse time. The parameter τ_s is a characteristic decay timescale, not the exact delay until a postsynaptic action potential occurs. Whether the postsynaptic neuron fires also depends on its membrane state, threshold, synaptic strength, and other inputs.

3.1.2 Shape and Time Scale of the Filter

The polynomial–exponential family provides considerable flexibility in modeling the temporal influence of a spike. To examine the response shape, consider $t > 0$ and differentiate:

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t}h_n(t) &= \frac{1}{n!\tau_s^{n+1}}\frac{\mathrm{d}}{\mathrm{d}t}\left(t^n e^{-t/\tau_s}\right) \\ &= \frac{e^{-t/\tau_s}}{n!\tau_s^{n+1}}\left(nt^{n-1} - \frac{t^n}{\tau_s}\right) \\ &= \frac{t^{n-1}e^{-t/\tau_s}}{n!\tau_s^{n+1}}\left(n - \frac{t}{\tau_s}\right). \end{aligned}$$

For $n \geq 1$, this derivative is positive for $0 < t < n\tau_s$ and negative for $t > n\tau_s$. Therefore the response reaches its maximum at

$$\boxed{t_{\text{peak}} = n\tau_s \qquad (n \geq 1).}$$

For $n = 0$, the exponential response is maximal at $t = 0$ and immediately decays.

Proposition 3.2 (Temporal moments of the normalized filter)**.** *For h_n as above,*

$$\begin{aligned} \int_0^\infty t\,h_n(t)\,\mathrm{d}t &= (n+1)\tau_s, \\ \int_0^\infty \big(t - (n+1)\tau_s\big)^2 h_n(t)\,\mathrm{d}t &= (n+1)\tau_s^2. \end{aligned}$$

Proof. The first moment is

$$\begin{aligned}\int_0^\infty t h_n(t)\,\mathrm{d}t &= \frac{1}{n!\tau_s^{n+1}}\int_0^\infty t^{n+1}e^{-t/\tau_s}\,\mathrm{d}t \\ &= \frac{\tau_s^{n+2}(n+1)!}{n!\tau_s^{n+1}} \\ &= (n+1)\tau_s.\end{aligned}$$

Similarly,

$$\begin{aligned}\int_0^\infty t^2 h_n(t)\,\mathrm{d}t &= \frac{\tau_s^{n+3}(n+2)!}{n!\tau_s^{n+1}} \\ &= (n+1)(n+2)\tau_s^2.\end{aligned}$$

Subtracting the square of the first moment gives

$$\begin{aligned}&(n+1)(n+2)\tau_s^2 - (n+1)^2\tau_s^2 \\ &\quad = (n+1)\tau_s^2.\end{aligned}$$

□

A short value of τ_s creates a narrow response, while a large value produces a more persistent response. The handwritten lecture illustrates a fast timescale around 15 ms (associated with fast AMPA/GABA-type responses) and a slower timescale around 100 ms (associated with NMDA-type responses). These are illustrative timescales: real receptor dynamics depend on the synapse and receptor subtype.

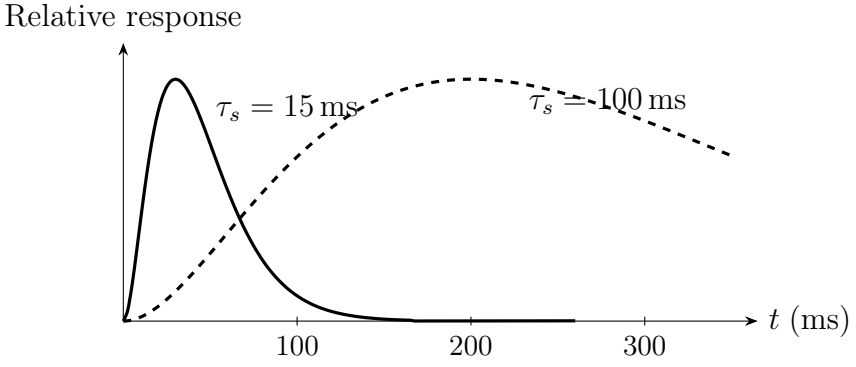


Figure 4: Fast and slow PSC responses from the $n = 2$ family. The curves are peak-normalized to compare temporal shapes, rather than unit-area-normalized. The dashed curve is still decaying beyond the plotted window.

3.2 Spike Trains and Dirac Impulses

Neurons typically generate multiple spikes. To describe their effect on synaptic input, it is convenient to idealize each spike as a point event, represented mathematically using the Dirac delta distribution.

Definition 3.2 (Dirac delta)**.** The *Dirac delta* at the origin, denoted δ, is characterized by its action on a suitable test function f:

$$\boxed{\int_{-\infty}^{\infty} f(t)\delta(t-t_0)\,\mathrm{d}t = f(t_0).}$$

Formally, $\delta(t-t_0)$ represents a unit impulse concentrated at time t_0.

Remark 3.2. Although the delta is often illustrated as an infinitely tall, zero-width impulse of unit area, it is not an ordinary real-valued function. It is a distribution (or, equivalently in this setting, a unit point mass), and its rigorous meaning is the integral identity above.

Definition 3.3 (Spike train)**.** Suppose a presynaptic neuron fires at times

$$t_1 < t_2 < \cdots < t_N.$$

Its *spike train* is

$$\boxed{a(t) = \sum_{p=1}^{N} \delta(t-t_p).}$$

Each spike contributes one unit impulse at its time of occurrence. For a locally finite infinite sequence of spike times, the same definition extends to a locally finite sum.

For a test function f, linearity gives

$$\begin{aligned}\int_{-\infty}^{\infty} f(t)a(t)\,\mathrm{d}t &= \int_{-\infty}^{\infty} f(t)\sum_{p=1}^{N}\delta(t-t_p)\,\mathrm{d}t \\ &= \sum_{p=1}^{N}\int_{-\infty}^{\infty} f(t)\delta(t-t_p)\,\mathrm{d}t \\ &= \boxed{\sum_{p=1}^{N} f(t_p)}.\end{aligned}$$

Thus an integral against the spike train is precisely a sum of contributions evaluated at the spike times. In particular, an interval containing exactly k spikes satisfies $\int a(t)\,\mathrm{d}t = k$ when its endpoints are not spike times.

3.3 Filtering a Spike Train by Convolution

A presynaptic spike does not need to cause an immediate voltage jump in the postsynaptic neuron. Instead, it generates a temporally distributed input with the shape prescribed by the PSC kernel.

Definition 3.4 (Filtered spike train)**.** Let $a(t) = \sum_{p=1}^{N} \delta(t - t_p)$ be a spike train and let h be a causal PSC filter. The associated *filtered synaptic input* is the convolution

$$\boxed{s(t) = (a * h)(t) := \int_{-\infty}^{\infty} a(u)h(t-u)\,\mathrm{d}u.}$$

Proposition 3.3 (Superposition of synaptic responses)**.** *For the spike train $a(t) = \sum_{p=1}^{N} \delta(t-t_p)$, the filtered input is*

$$\boxed{s(t) = \sum_{p=1}^{N} h(t - t_p).}$$

Hence every spike at time t_p contributes a copy of h shifted to begin at t_p.

Proof. Substitute the spike-train representation into the convolution integral:

$$\begin{aligned} s(t) &= (a * h)(t) \\ &= \int_{-\infty}^{\infty} a(u)h(t-u)\,\mathrm{d}u \\ &= \int_{-\infty}^{\infty} \left[\sum_{p=1}^{N} \delta(u - t_p)\right] h(t-u)\,\mathrm{d}u \\ &= \sum_{p=1}^{N} \int_{-\infty}^{\infty} \delta(u - t_p)h(t-u)\,\mathrm{d}u \\ &= \sum_{p=1}^{N} h(t - t_p). \end{aligned}$$

The final equality is the defining sifting property of the Dirac delta. Because h is causal, $h(t - t_p) = 0$ whenever $t < t_p$. □

This result makes the temporal computation explicit. A single input spike produces one shifted current response. A train of spikes produces a *sum* of shifted responses, with nearby responses overlapping and accumulating.

Example 3.1 (Two postsynaptic responses)**.** Suppose spikes arrive at $t_1 = 0$ and $t_2 = \Delta > 0$ and the PSC filter is

$$h(t) = \frac{1}{\tau_s} e^{-t/\tau_s} \mathbf{1}_{\{t \geq 0\}}.$$

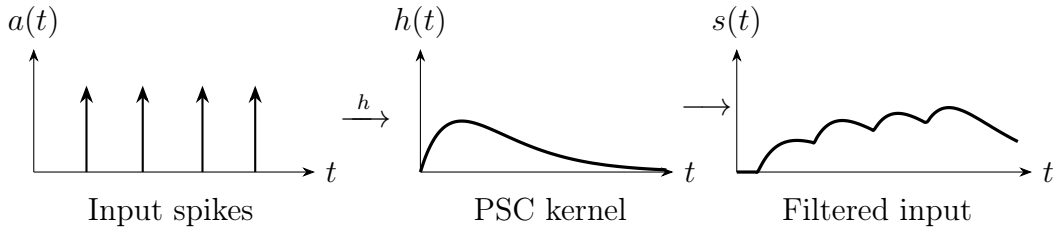


Figure 5: A spike train is filtered by assigning one causal PSC response to each spike and summing the overlapping responses. The curves are schematic.

Then

$$\begin{aligned} s(t) &= h(t) + h(t-\Delta) \\ &= \frac{1}{\tau_s} e^{-t/\tau_s} \mathbf{1}_{\{t \geq 0\}} + \frac{1}{\tau_s} e^{-(t-\Delta)/\tau_s} \mathbf{1}_{\{t \geq \Delta\}}. \end{aligned}$$

Consequently,

$$s(t) = \begin{cases} 0, & t < 0, \\ \frac{1}{\tau_s} e^{-t/\tau_s}, & 0 \leq t < \Delta, \\ \frac{1}{\tau_s} \left(e^{-t/\tau_s} + e^{-(t-\Delta)/\tau_s} \right), & t \geq \Delta. \end{cases}$$

The contribution from the first spike is still present when the second spike arrives, producing temporal summation.

Proposition 3.4 (Area of a filtered spike train)**.** *Suppose h is nonnegative and has unit integral. If a consists of N unit spikes, then*

$$\boxed{\int_{-\infty}^{\infty} s(t)\, \mathrm{d}t = N.}$$

Proof. By the superposition formula and linearity,

$$\begin{aligned} \int_{-\infty}^{\infty} s(t)\, \mathrm{d}t &= \sum_{p=1}^{N} \int_{-\infty}^{\infty} h(t - t_p)\, \mathrm{d}t \\ &= \sum_{p=1}^{N} \int_{-\infty}^{\infty} h(u)\, \mathrm{d}u \\ &= \sum_{p=1}^{N} 1 = N. \end{aligned}$$

□

3.4 Connectivity and the Synaptic Weight Matrix

In a network, postsynaptic neurons receive signals from multiple presynaptic neurons. A *synaptic weight* represents the strength and sign of the connection from one neuron to another.

3.4.1 Feedforward Matrix Representation

Let the input layer contain N neurons and the output layer contain M neurons. Define w_{ji} to be the weight from input neuron j to output neuron i. Under the *row-vector convention*, collect the inputs and outputs into

$$\boldsymbol{x} = (x_1, \ldots, x_N) \in \mathbb{R}^{1\times N}, \qquad \boldsymbol{y} = (y_1, \ldots, y_M) \in \mathbb{R}^{1\times M}.$$

Set

$$W = (w_{ji}) \in \mathbb{R}^{N\times M}, \qquad \boldsymbol{b} = (b_1, \ldots, b_M) \in \mathbb{R}^{1\times M}.$$

Definition 3.5 (Affine layer and activation)**.** The affine pre-activations are

$$\boxed{\boldsymbol{z} = \boldsymbol{x}W + \boldsymbol{b},}$$

and the output activations are

$$\boxed{\boldsymbol{y} = \sigma(\boldsymbol{z}) = \sigma(\boldsymbol{x}W + \boldsymbol{b}),}$$

where the activation function σ is applied componentwise.

For example, with two inputs and three outputs,

$$\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}, \qquad W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix},$$
$$\boldsymbol{b} = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix},$$

so that

$$\begin{aligned} \boldsymbol{z} &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} \\ &= \begin{bmatrix} x_1w_{11} + x_2w_{21} + b_1 & x_1w_{12} + x_2w_{22} + b_2 & x_1w_{13} + x_2w_{23} + b_3 \end{bmatrix}. \end{aligned}$$

Thus each output receives a weighted contribution from *every* input under full connectivity:

$$\boxed{z_i = \sum_{j=1}^{N} x_j w_{ji} + b_i, \qquad i = 1, \ldots, M.}$$

A fully connected layer has NM weights and M biases, giving $NM + M$ trainable parameters in total.

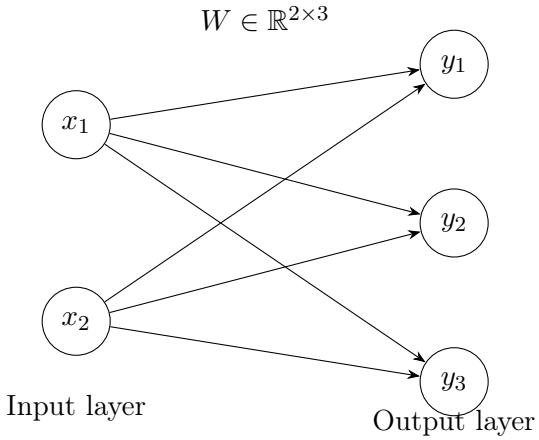


Figure 6: A fully connected layer with two input neurons and three output neurons. There is one independent weight w_{ji} on each directed connection $j \to i$.

3.4.2 Bias as an Additional Input

A bias can be represented as a connection from an artificial input node that is constantly equal to one. Define the augmented row vector and weight matrix

$$\widetilde{\boldsymbol{x}} = \begin{bmatrix} \boldsymbol{x} & 1 \end{bmatrix} \in \mathbb{R}^{1\times(N+1)}, \qquad \widetilde{W} = \begin{bmatrix} W \\ \boldsymbol{b} \end{bmatrix} \in \mathbb{R}^{(N+1)\times M}.$$

Then

$$\begin{aligned} \widetilde{\boldsymbol{x}}\widetilde{W} &= \begin{bmatrix} \boldsymbol{x} & 1 \end{bmatrix} \begin{bmatrix} W \\ \boldsymbol{b} \end{bmatrix} \\ &= \boldsymbol{x}W + \boldsymbol{b}. \end{aligned}$$

Hence the affine map may be written as a single matrix multiplication:

$$\boxed{\boldsymbol{z} = \widetilde{\boldsymbol{x}}\widetilde{W}.}$$

The extra row of $\widetilde{W}$ consists precisely of the bias parameters $b_1, \ldots, b_M$.

Remark 3.3 (Column-vector convention). If one instead defines $\boldsymbol{x} \in \mathbb{R}^{N\times 1}$, then the corresponding formula is $\boldsymbol{z} = W^\top \boldsymbol{x} + \boldsymbol{b}$ with $\boldsymbol{b} \in \mathbb{R}^{M\times 1}$. These formulas describe the same map; only the orientation conventions differ.

3.5 Exponential Synaptic Dynamics

For computational spiking neural networks, a particularly useful filter is the normalized exponential PSC kernel

$$\boxed{h(t) = \frac{1}{\tau_s} e^{-t/\tau_s} \mathbf{1}_{\{t\geq 0\}}.}$$

This is the case $n = 0$ of the general family. It has a simple ordinary differential equation between spikes and an equally simple update rule at spike times.

3.5.1 Exponential Decay Between Spikes

Assume no new spikes arrive over an interval beginning at t_0. The filtered input satisfies

$$\boxed{\tau_s \frac{\mathrm{d}s}{\mathrm{d}t} = -s.}$$

Proposition 3.5 (Exact synaptic decay)**.** *If $s(t_0) = s_0$ and no spike arrives on $[t_0, t]$, then*

$$\boxed{s(t) = s_0 e^{-(t-t_0)/\tau_s}.}$$

Proof. From $\tau_s \dot{s} = -s$, separation of variables gives

$$\begin{aligned}
\frac{\mathrm{d}s}{\mathrm{d}t} &= -\frac{s}{\tau_s}, \\
\frac{\mathrm{d}s}{s} &= -\frac{\mathrm{d}t}{\tau_s}, \\
\int_{s_0}^{s(t)} \frac{\mathrm{d}u}{u} &= -\frac{1}{\tau_s} \int_{t_0}^{t} \mathrm{d}r, \\
\log \left| \frac{s(t)}{s_0} \right| &= -\frac{t - t_0}{\tau_s}, \\
s(t) &= s_0 e^{-(t-t_0)/\tau_s}.
\end{aligned}$$

The same formula holds for $s_0 = 0$ by direct substitution. □

For a single unit spike at $t_0 = 0$, the initial jump is $s(0^+) = 1/\tau_s$, and the post-spike solution is

$$s(t) = \frac{1}{\tau_s} e^{-t/\tau_s}, \qquad t > 0,$$

which recovers the exponential PSC kernel. The factor $1/\tau_s$ is essential for unit-area normalization.

3.5.2 Deriving the Spike-Arrival Jump Rule

A filtered spike train may also be described by a differential equation driven by Dirac impulses:

$$\boxed{\tau_s \frac{\mathrm{d}s}{\mathrm{d}t} = -s + \sum_p w_p \delta(t - t_p).}$$

Here w_p denotes the weight of the spike arriving at t_p. Between arrivals, the delta terms vanish, and the state decays according to $\tau_s \dot{s} = -s$.

Proposition 3.6 (Jump at spike arrival)**.** *When a spike of weight w_p arrives at t_p, the synaptic state jumps by w_p/τ_s:*

$$\boxed{s(t_p^+) = s(t_p^-) + \frac{w_p}{\tau_s}.}$$

Proof. Integrate the impulse-driven equation across a small interval around the spike time:

$$\tau_s \int_{t_p-\varepsilon}^{t_p+\varepsilon} \frac{\mathrm{d}s}{\mathrm{d}t}\,\mathrm{d}t = -\int_{t_p-\varepsilon}^{t_p+\varepsilon} s(t)\,\mathrm{d}t + w_p \int_{t_p-\varepsilon}^{t_p+\varepsilon} \delta(t-t_p)\,\mathrm{d}t,$$
$$\tau_s\left(s(t_p+\varepsilon) - s(t_p-\varepsilon)\right) = -\int_{t_p-\varepsilon}^{t_p+\varepsilon} s(t)\,\mathrm{d}t + w_p.$$

Assuming the state has finite one-sided limits, the ordinary integral tends to zero as $\varepsilon \downarrow 0$. Hence

$$\tau_s\left(s(t_p^+) - s(t_p^-)\right) = w_p,$$
$$s(t_p^+) - s(t_p^-) = \frac{w_p}{\tau_s}.$$

□

Consequently, simulation of exponential synaptic dynamics only requires two operations: (i) exponentially decay the current synaptic state between events, and (ii) add w_p/τ_s whenever a spike arrives. For a weight $w_p < 0$, the jump is negative and represents inhibitory input in this signed-current approximation.

Remark 3.4 (Different weight conventions). The jump size depends on the kernel normalization. With $h(t) = \tau_s^{-1} e^{-t/\tau_s}\mathbf{1}_{\{t\geq 0\}}$, the jump is w_p/τ_s. With the unnormalized kernel $e^{-t/\tau_s}\mathbf{1}_{\{t\geq 0\}}$, the corresponding jump would be w_p. These conventions must not be mixed.

3.6 The Full Leaky Integrate-and-Fire Network

We now combine the postsynaptic filter with the leaky integrate-and-fire (LIF) voltage model developed in Lecture 2. Each neuron possesses a membrane-voltage state and an incoming synaptic-current state.

Definition 3.6 (Filtered LIF neuron)**.** For a postsynaptic neuron i, let $v_i(t)$ denote normalized membrane voltage and let $s_i(t)$ denote its filtered input. Between spike arrivals and outside the absolute refractory period, the dynamics are

$$\boxed{\begin{aligned} \tau_m \frac{\mathrm{d}v_i}{\mathrm{d}t} &= s_i - v_i, \\ \tau_s \frac{\mathrm{d}s_i}{\mathrm{d}t} &= -s_i. \end{aligned}}$$

The neuron emits a spike whenever its voltage reaches the threshold $v_i \geq 1$. Its voltage is then reset to 0 and held there for a prescribed refractory period $\tau_{\mathrm{ref}} \geq 0$.

The synaptic input $s_i(t)$ is an aggregate of filtered spikes from presynaptic neurons. If neuron j fires at times $t_{j,1}, t_{j,2}, \ldots$, then a network with exponential kernels can be described as

$$\boxed{s_i(t) = \sum_j \sum_p w_{ji}\, h(t - t_{j,p})}$$

for the contribution of spikes after the chosen initialization time. With h unit-area-normalized, w_{ji} absorbs the scaling needed to convert inverse-time filter values into the units of the input s_i. Equivalently, at an incoming spike from presynaptic neuron j at time t_*, neuron i updates

$$\boxed{s_i(t_*^+) \leftarrow s_i(t_*^-) + \frac{w_{ji}}{\tau_s}.}$$

The model thereby combines the connection weight w_{ji}, the synaptic time constant τ_s, and the membrane time constant τ_m.

3.6.1 How a Spiking Neuron Operates

A neuron's computation proceeds as follows:

1. Incoming spikes from presynaptic neurons alter the synaptic state s_i according to their connection weights.
2. Between incoming events, s_i decays exponentially at rate $1/\tau_s$.
3. Outside the refractory period, the membrane voltage v_i integrates the synaptic input while leaking toward zero through $-v_i$.
4. Once v_i reaches the normalized threshold 1, the neuron emits an output spike and records its spike time.
5. The membrane voltage is reset to 0 and, if $\tau_{\text{ref}} > 0$, remains there during the absolute refractory interval. The synaptic state may still evolve during refractoriness.
6. The emitted spike is transmitted along outgoing connections and produces appropriately weighted jumps in downstream synaptic states.

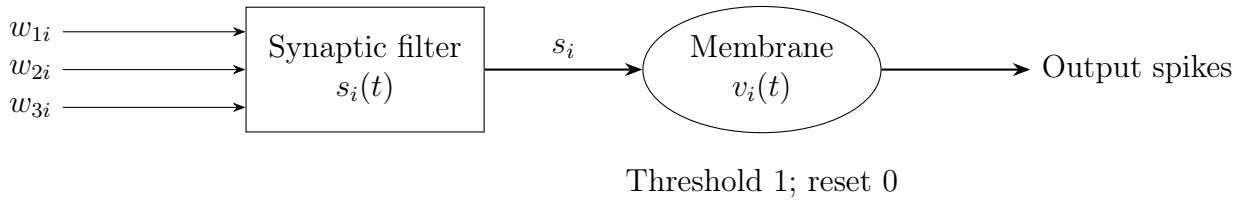


Figure 7: Functional structure of a filtered LIF neuron. Incoming weighted spikes update the synaptic filter, which drives the voltage state and subsequent spike generation.

3.6.2 Exact Evolution Between Events

Because the coupled system is linear between events, both states can be propagated analytically. Consider an interval $[t_0, t_0 + \Delta]$ in which no incoming spikes arrive, the neuron does not reach threshold, and the neuron is not refractory. Write

$$s_0 = s_i(t_0), \qquad v_0 = v_i(t_0), \qquad \Delta \geq 0.$$

The synaptic state is immediately

$$s_i(t_0 + u) = s_0 e^{-u/\tau_s}, \qquad 0 \leq u \leq \Delta.$$

Substitution into the membrane equation gives

$$\frac{\mathrm{d}v_i}{\mathrm{d}u} + \frac{v_i}{\tau_m} = \frac{s_0}{\tau_m} e^{-u/\tau_s}.$$

Proposition 3.7 (Exact membrane update between events)**.** *If $\tau_m \neq \tau_s$, the voltage after elapsed time Δ is*

$$\boxed{v_i(t_0 + \Delta) = v_0 e^{-\Delta/\tau_m} + \frac{s_0 \tau_s}{\tau_s - \tau_m} \left(e^{-\Delta/\tau_s} - e^{-\Delta/\tau_m} \right).}$$

If $\tau_m = \tau_s = \tau$, the corresponding formula is

$$\boxed{v_i(t_0 + \Delta) = e^{-\Delta/\tau} \left(v_0 + \frac{s_0 \Delta}{\tau} \right).}$$

Proof. Multiplying the voltage equation by the integrating factor e^{u/τ_m} gives

$$\frac{\mathrm{d}}{\mathrm{d}u} \left(e^{u/\tau_m} v_i(t_0 + u) \right) = \frac{s_0}{\tau_m} e^{u(1/\tau_m - 1/\tau_s)}.$$

Integrate from 0 to Δ:

$$e^{\Delta/\tau_m} v_i(t_0 + \Delta) - v_0 = \frac{s_0}{\tau_m} \int_0^\Delta e^{u(1/\tau_m - 1/\tau_s)} \, \mathrm{d}u.$$

When $\tau_m \neq \tau_s$, the exponent coefficient is nonzero, so

$$\begin{aligned} e^{\Delta/\tau_m} v_i(t_0 + \Delta) - v_0 &= \frac{s_0}{\tau_m} \frac{e^{\Delta(1/\tau_m - 1/\tau_s)} - 1}{1/\tau_m - 1/\tau_s} \\ &= \frac{s_0 \tau_s}{\tau_s - \tau_m} \left(e^{\Delta(1/\tau_m - 1/\tau_s)} - 1 \right). \end{aligned}$$

Multiplying by $e^{-\Delta/\tau_m}$ yields

$$v_i(t_0 + \Delta) = v_0 e^{-\Delta/\tau_m} + \frac{s_0 \tau_s}{\tau_s - \tau_m} \left(e^{-\Delta/\tau_s} - e^{-\Delta/\tau_m} \right).$$

If instead $\tau_m = \tau_s = \tau$, the integrand in the integrating-factor equation equals 1, and

$$\begin{aligned} e^{\Delta/\tau} v_i(t_0 + \Delta) - v_0 &= \frac{s_0}{\tau} \int_0^{\Delta} 1 \, \mathrm{d}u \\ &= \frac{s_0 \Delta}{\tau}. \end{aligned}$$

Rearranging proves the second expression. □

3.6.3 Voltage Response to a Single Synaptic Spike

Assume a neuron begins at rest, $v(0^-) = s(0^-) = 0$, and receives one input spike of weight w at $t = 0$. The jump rule gives

$$s(0^+) = \frac{w}{\tau_s}, \qquad v(0^+) = 0.$$

For $t > 0$ before any output spike or refractory interval, the exact membrane response is therefore

$$\boxed{v(t) = \begin{cases} \dfrac{w}{\tau_s - \tau_m} \left(e^{-t/\tau_s} - e^{-t/\tau_m} \right), & \tau_s \neq \tau_m, \\ \dfrac{wt}{\tau_m^2} e^{-t/\tau_m}, & \tau_s = \tau_m. \end{cases}}$$

The response is a difference of exponentials when the time constants differ, and a polynomial times an exponential when they coincide. The response is evaluated only until an output spike resets the model voltage.

Remark 3.5 (Two stages of temporal filtering). The PSC kernel filters the presynaptic spike train into $s(t)$, and the membrane equation filters $s(t)$ once more into $v(t)$. Thus even an exponential synaptic impulse can generate a membrane response that rises gradually and then decays. The synaptic timescale τ_s controls input persistence, while τ_m controls membrane integration and leak.

3.6.4 Continuous-Time State-Space Interpretation

Between events, define the state vector

$$\boldsymbol{q}_i(t) = \begin{bmatrix} v_i(t) \\ s_i(t) \end{bmatrix}.$$

The filtered LIF dynamics can be written as the linear system

$$\boxed{\frac{\mathrm{d}\boldsymbol{q}_i}{\mathrm{d}t} = A\boldsymbol{q}_i, \qquad A = \begin{bmatrix} -1/\tau_m & 1/\tau_m \\ 0 & -1/\tau_s \end{bmatrix}.}$$

Consequently, in an event-free, non-refractory interval,

$$\boxed{\boldsymbol{q}_i(t_0 + \Delta) = e^{A\Delta} \boldsymbol{q}_i(t_0).}$$

At a presynaptic spike from neuron j, the state is updated by

$$\boldsymbol{q}_i(t_*^+) = \boldsymbol{q}_i(t_*^-) + \begin{bmatrix} 0 \\ w_{ji}/\tau_s \end{bmatrix}.$$

At the postsynaptic neuron's own firing time, the membrane voltage is reset, whereas s_i is not necessarily reset. These continuous-time flows together with discrete jumps define a *hybrid dynamical system*.

Remark 3.6 (Dimensional consistency). In the normalized equations v_i and s_i have the same units (often both dimensionless), so the weight w_{ji} in $w_{ji}h(t)$ has units of time when h is normalized to unit area. An equivalent formulation can introduce a separate synaptic current or voltage scaling factor and use dimensionless weights. The lecture's compact notation absorbs this scaling into w_{ji}.

3.7 Summary of the Computational Model

The modeling pipeline developed in this lecture is

$$\begin{aligned} &\text{Presynaptic spikes} \longrightarrow \text{PSC filtering} \longrightarrow \text{Synaptic summation} \\ &\qquad \longrightarrow \text{LIF voltage} \longrightarrow \text{Output spikes.} \end{aligned}$$

More explicitly, for every neuron i:

$$\begin{aligned} a_j(t) &= \sum_p \delta(t - t_{j,p}), \\ h(t) &= \frac{1}{\tau_s} e^{-t/\tau_s} \mathbf{1}_{\{t \geq 0\}}, \\ s_i(t) &= \sum_j w_{ji}(h * a_j)(t), \\ \tau_m \dot{v}_i(t) &= s_i(t) - v_i(t), \quad \text{outside refractoriness}, \\ \tau_s \dot{s}_i(t) &= -s_i(t), \quad \text{between input spikes}, \\ s_i(t_*^+) &= s_i(t_*^-) + \frac{w_{ji}}{\tau_s} \quad \text{when neuron } j \text{ spikes at } t_*, \\ v_i(t_i^-) = 1 \quad &\Longrightarrow \quad \text{emit spike, set } v_i(t_i^+) = 0, \text{ and enter refractoriness.} \end{aligned}$$

The weight matrix determines *which* neurons communicate and by how much; the PSC filter determines *when* incoming spikes exert influence; and the LIF membrane equation determines *whether and when* a postsynaptic neuron fires.

4 Lecture 4: Supervised Learning and the Universal Approximation Theorem

4.1 Supervised Learning: Inputs, Targets, and Predictions

In the preceding lectures, we considered biological neurons, spiking-neuron models, and the computation performed by artificial neurons. We now consider how a network of artificial neurons can be trained to represent an unknown relationship between observed inputs and desired outputs. This is the central problem of *supervised learning*.

Let $d_x \geq 1$ be the input dimension. An input is represented by a feature vector $x \in \mathbb{R}^{d_x}$, and a target t is the quantity to be predicted from that input. Throughout this lecture, we write t for a target in order to distinguish it from the network's prediction $\widehat{y}$.

Definition 4.1 (Labeled dataset)**.** A *labeled dataset* of size N is a finite collection

$$\mathcal{D}_N = \{(x_n, t_n) : n = 1, \ldots, N\},$$

where $x_n \in \mathbb{R}^{d_x}$ and t_n belongs to an appropriate output space $\mathcal{Y}$.

There are two standard types of supervised-learning problems.

1. **Regression.** The target is a real-valued quantity or vector:

$$\mathcal{D}_N = \{(x_n, t_n) : x_n \in \mathbb{R}^{d_x},\ t_n \in \mathbb{R}^{d_y},\ n = 1, \ldots, N\}.$$

   The scalar-output case corresponds to $d_y = 1$. Examples include estimating the temperature at a future time, predicting a continuous physical signal, and estimating a real-valued response from measurements.

2. **Classification.** The target identifies one of K classes:

$$\mathcal{D}_N = \{(x_n, t_n) : x_n \in \mathbb{R}^{d_x},\ t_n \in \{1, 2, \ldots, K\}\}.$$

   In this case, the network may output a vector of class scores or class probabilities, which is then converted into a predicted label.

Definition 4.2 (Parameterized predictor)**.** A neural-network predictor is a map

$$f_\theta : \mathbb{R}^{d_x} \longrightarrow \mathcal{Y}_{\text{pred}}, \qquad \widehat{y} = f_\theta(x),$$

where θ collects all trainable parameters, including the connection weights and biases. The prediction on example n is

$$\widehat{y}_n = f_\theta(x_n).$$

Remark 4.1 (The role of biases). A bias shifts the input to a neuron before the activation function is applied. For a scalar neuron,

$$h(x) = \sigma(w^\top x + b).$$

Unlike the weight vector w, which determines the direction and scale of the linear response, the bias b determines the location of its activation transition. This shift will be central to the approximation argument below.

4.1.1 Loss Functions

To determine whether a prediction is close to its target, we specify a *loss function*.

Definition 4.3 (Loss)**.** A loss function is a map

$$\ell : \mathcal{Y}_{\text{pred}} \times \mathcal{Y} \longrightarrow [0, \infty]$$

that assigns a penalty $\ell(\widehat{y}, t)$ to predicting $\widehat{y}$ when the correct target is t.

For scalar regression, common losses are the squared-error and absolute-error losses:

$$\ell_{\text{sq}}(\widehat{y}, t) = (\widehat{y} - t)^2,$$
$$\ell_{\text{abs}}(\widehat{y}, t) = |\widehat{y} - t|.$$

For vector-valued regression, a corresponding squared-error loss is

$$\ell_{\text{sq}}(\widehat{y}, t) = \|\widehat{y} - t\|_2^2 = \sum_{k=1}^{d_y} (\widehat{y}_k - t_k)^2.$$

For classification, one direct loss is the zero–one loss

$$\ell_{0\text{–}1}(\widehat{t}, t) = \mathbf{1}_{\{\widehat{t} \neq t\}},$$

which counts a prediction as incorrect whenever its assigned class differs from the target. Because it is discontinuous in the class decision, differentiable surrogate losses are normally used to train networks.

If the network produces logits $z(x; \theta) \in \mathbb{R}^K$, let

$$p_k(x; \theta) = \frac{e^{z_k(x;\theta)}}{\sum_{j=1}^{K} e^{z_j(x;\theta)}}.$$

The cross-entropy loss for class $t \in \{1, \ldots, K\}$ is

$$\ell_{\text{CE}}(p, t) = -\log p_t.$$

The predicted class is usually $\widehat{t} = \arg\max_k p_k$.

4.2 The Objective of Learning

A supervised-learning problem is typically modeled by an unknown probability distribution P on input–target pairs (X, T). The data are observations from this distribution, while the network is intended to make accurate predictions on new observations drawn from the same underlying population.

Definition 4.4 (Population risk)**.** For a predictor f_θ and loss ℓ, the *population risk*, or expected prediction error, is

$$\mathcal{R}(\theta) := \mathbb{E}_{(X,T)\sim P}\big[\ell(f_\theta(X), T)\big] = \int \ell(f_\theta(x), t)\, P(\mathrm{d}x, \mathrm{d}t).$$

The ideal population-learning objective is

$$\inf_{\theta\in\Theta} \mathcal{R}(\theta),$$

where Θ denotes the permissible parameter set.

This expresses the objective written in the lecture notes: select weights and biases so that the average prediction error over the *entire data distribution* is as small as possible. The infimum notation does not assume that an optimal parameter θ necessarily exists.

Definition 4.5 (Empirical risk)**.** For an observed dataset $\mathcal{D}_N$, the *empirical risk* is

$$\widehat{\mathcal{R}}_N(\theta) := \frac{1}{N}\sum_{n=1}^{N} \ell(f_\theta(x_n), t_n).$$

An *empirical risk minimizer* (when one exists) satisfies

$$\widehat{\theta}_N \in \arg\min_{\theta\in\Theta} \widehat{\mathcal{R}}_N(\theta).$$

Remark 4.2 (Population versus training error). The population risk $\mathcal{R}(\theta)$ is an expectation under the unknown distribution P. The empirical risk $\widehat{\mathcal{R}}_N(\theta)$ is the finite sample average available for computation. They are not generally identical. Minimizing training error does not by itself guarantee low population error: a sufficiently flexible network may fit the observed training examples while performing poorly on new inputs.

Proposition 4.1 (Unbiased empirical risk at a fixed parameter)**.** *Suppose the examples (X_n, T_n) are independent and identically distributed according to P, and $\mathbb{E}|\ell(f_\theta(X), T)| < \infty$. For each* fixed θ*,*

$$\mathbb{E}\big[\widehat{\mathcal{R}}_N(\theta)\big] = \mathcal{R}(\theta).$$

If in addition the loss has finite variance $\sigma_\ell^2(\theta)$, then

$$\mathrm{Var}\big(\widehat{\mathcal{R}}_N(\theta)\big) = \frac{\sigma_\ell^2(\theta)}{N}.$$

Proof. By linearity of expectation and the identical distributions of the examples,

$$\begin{aligned}\mathbb{E}\big[\widehat{\mathcal{R}}_N(\theta)\big] &= \mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\ell(f_\theta(X_n),T_n)\right]\\ &= \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}\big[\ell(f_\theta(X_n),T_n)\big]\\ &= \frac{1}{N}\sum_{n=1}^{N}\mathcal{R}(\theta)\\ &= \mathcal{R}(\theta).\end{aligned}$$

Independence also gives

$$\begin{aligned}\operatorname{Var}\big(\widehat{\mathcal{R}}_N(\theta)\big) &= \frac{1}{N^2}\sum_{n=1}^{N}\operatorname{Var}\big(\ell(f_\theta(X_n),T_n)\big)\\ &= \frac{N\sigma_\ell^2(\theta)}{N^2}\\ &= \frac{\sigma_\ell^2(\theta)}{N}.\end{aligned}$$

□

Remark 4.3. The qualification "fixed θ" matters. When θ is selected by minimizing the same empirical risk, it depends on the training data. The proposition cannot be applied to the resulting random $\widehat{\theta}_N$ without additional analysis. This distinction is one reason that approximation ability and generalization ability are separate mathematical questions.

4.3 Feedforward Neural Networks and Hidden Units

A feedforward network maps an input through one or more collections of intermediate neurons, called *hidden layers*. The diagram in the handwritten notes illustrates a single-input, three-hidden-unit, single-output network. More generally, with d_x input coordinates and M hidden units, define

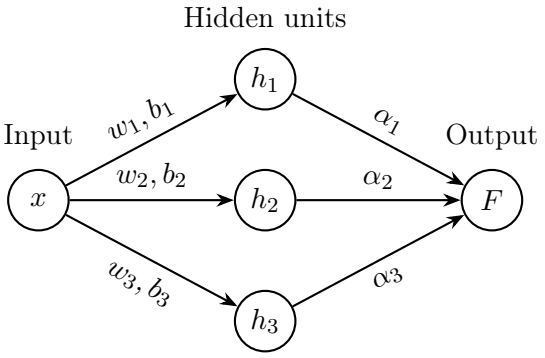
$$\begin{aligned}h_j(x) &= \sigma(w_j^\top x + b_j), \qquad j = 1,\dots,M,\\ F(x) &= c + \sum_{j=1}^{M}\alpha_j h_j(x)\\ &= c + \sum_{j=1}^{M}\alpha_j \sigma(w_j^\top x + b_j).\end{aligned}$$

Here $w_j \in \mathbb{R}^{d_x}$ is the input-to-hidden weight vector, $b_j \in \mathbb{R}$ is the hidden bias, $\alpha_j \in \mathbb{R}$ is the hidden-to-output weight, and $c \in \mathbb{R}$ is the output bias.

Definition 4.6 (Single-hidden-layer network class)**.** For an activation $\sigma : \mathbb{R} \to \mathbb{R}$ and a set $K \subseteq \mathbb{R}^d$, define

$$\mathcal{N}_\sigma(K) := \left\{ x \mapsto c + \sum_{j=1}^{M} \alpha_j \sigma(w_j^\top x + b_j) : M \in \mathbb{N},\ c, \alpha_j, b_j \in \mathbb{R},\ w_j \in \mathbb{R}^d \right\} \Big|_K .$$

The width M is allowed to depend on the function being approximated and on the desired approximation accuracy.



$$h_j(x) = \sigma(w_j^\top x + b_j)$$

Figure 8: A one-hidden-layer feedforward network. The output is an affine combination of nonlinear hidden-unit responses. In the scalar-input case, $w_j^\top x = w_j x$.

4.3.1 Why Nonlinearity is Essential

Without a nonlinear activation, the hidden units would be affine functions of the input, and any affine combination of them would remain affine. Indeed, replacing $\sigma(u)$ by u yields

$$\begin{aligned} F(x) &= c + \sum_{j=1}^{M} \alpha_j (w_j^\top x + b_j) \\ &= \left(\sum_{j=1}^{M} \alpha_j w_j \right)^\top x + \left(c + \sum_{j=1}^{M} \alpha_j b_j \right). \end{aligned}$$

Consequently, a linear network cannot represent a generic nonlinear continuous function, regardless of the number of hidden units. It is the nonlinearity σ that makes the approximation result possible.

4.4 The Universal Approximation Theorem

The central theoretical question is whether sums of nonlinear hidden-unit responses can approximate an arbitrary continuous target function with arbitrarily small error. The *universal approximation theorem* answers this question affirmatively for many activation functions.

Definition 4.7 (Continuous sigmoidal function)**.** An activation function $\sigma : \mathbb{R} \to \mathbb{R}$ is *continuous sigmoidal* if it is continuous and satisfies

$$\lim_{u \to -\infty} \sigma(u) = 0, \qquad \lim_{u \to +\infty} \sigma(u) = 1.$$

A standard example is the logistic sigmoid

$$\sigma(u) = \frac{1}{1 + e^{-u}}.$$

We call it *nondecreasing sigmoidal* if it additionally satisfies

$$u \leq v \quad \Longrightarrow \quad \sigma(u) \leq \sigma(v).$$

Remark 4.4. The logistic sigmoid is continuous, nondecreasing, and bounded between 0 and 1. Other continuous sigmoidal activations need not be monotone. The constructive one-dimensional proof below uses monotonicity; the multidimensional theorem proved subsequently requires only the stated limits and continuity.

Definition 4.8 (Uniform approximation and density)**.** Let $K \subset \mathbb{R}^d$ be compact and let $C(K)$ denote the continuous real-valued functions on K, equipped with the uniform norm

$$\|f\|_\infty := \sup_{x \in K} |f(x)|.$$

A class $\mathcal{A} \subset C(K)$ is *dense in* $C(K)$ if, for every $f \in C(K)$ and every $\varepsilon > 0$, there exists $g \in \mathcal{A}$ such that

$$\|f - g\|_\infty < \varepsilon.$$

Equivalently, $\overline{\mathcal{A}}^{\|\cdot\|_\infty} = C(K)$.

Theorem 4.1 (Universal approximation for continuous sigmoids)**.** *Let $K \subset \mathbb{R}^d$ be compact and let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous sigmoidal. For every $f \in C(K)$ and every $\varepsilon > 0$, there exist a finite integer $M \geq 1$, weights $w_j \in \mathbb{R}^d$, biases $b_j \in \mathbb{R}$, output coefficients $\alpha_j \in \mathbb{R}$, and an output bias $c \in \mathbb{R}$ such that*

$$\sup_{x \in K} \left| f(x) - c - \sum_{j=1}^{M} \alpha_j \sigma(w_j^\top x + b_j) \right| < \varepsilon.$$

In particular, $\mathcal{N}_\sigma(K)$ is dense in $C(K)$.

The theorem is an *existence result*. It does not say that any fixed width works for every target, that the learned parameters are easy to find, or that a network fitted to finitely many observations will generalize well. We first prove the mechanism constructively when $d = 1$, and then give a general proof for arbitrary input dimension.

4.5 How Sigmoids Approximate Thresholds

For the remainder of the one-dimensional construction, take $K = [a, b]$, with $a < b$, and assume σ is nondecreasing sigmoidal.

Definition 4.9 (Heaviside threshold)**.** For a threshold $r \in \mathbb{R}$, define

$$H_r(x) := \mathbf{1}_{\{x>r\}} = \begin{cases} 0, & x \le r, \\ 1, & x > r. \end{cases}$$

Lemma 4.1 (Steep sigmoids approximate a threshold away from its jump)**.** *For $k > 0$ and $r \in \mathbb{R}$, let*

$$S_{k,r}(x) := \sigma(k(x - r)).$$

For each $x \neq r$,

$$\lim_{k\to\infty} S_{k,r}(x) = H_r(x).$$

Moreover, for every $\delta > 0$,

$$\lim_{k\to\infty} \sup_{x\le r-\delta} |S_{k,r}(x)| = 0,$$
$$\lim_{k\to\infty} \sup_{x\ge r+\delta} |S_{k,r}(x) - 1| = 0.$$

Proof. If $x < r$ then $k(x - r) \to -\infty$, and the negative sigmoidal limit gives $S_{k,r}(x) \to 0$. If $x > r$, then $k(x - r) \to +\infty$, and $S_{k,r}(x) \to 1$. For $x \le r - \delta$, monotonicity gives

$$0 \le S_{k,r}(x) \le \sigma(-k\delta) \longrightarrow 0.$$

For $x \ge r + \delta$, similarly,

$$0 \le 1 - S_{k,r}(x) \le 1 - \sigma(k\delta) \longrightarrow 0.$$

Taking suprema proves both estimates. □

Remark 4.5 (Pointwise convergence is not uniform at the jump). At $x = r$, one always has $S_{k,r}(r) = \sigma(0)$, which generally differs from $H_r(r) = 0$. More importantly, a uniform limit of continuous functions is continuous, whereas H_r is discontinuous. Hence no sequence of continuous sigmoids can converge *uniformly* to H_r on an interval containing r. The step-function sketch in the handwritten notes correctly identifies the approximation mechanism, but the final uniform approximation requires controlling the transition regions using continuity of the target function. We do this below.

4.5.1 Localized Responses from Differences of Sigmoids

A difference of two shifted sigmoids behaves like the indicator of an interval. For $r < s$, define

$$P_{k;r,s}(x) := \sigma(k(x - r)) - \sigma(k(x - s)).$$

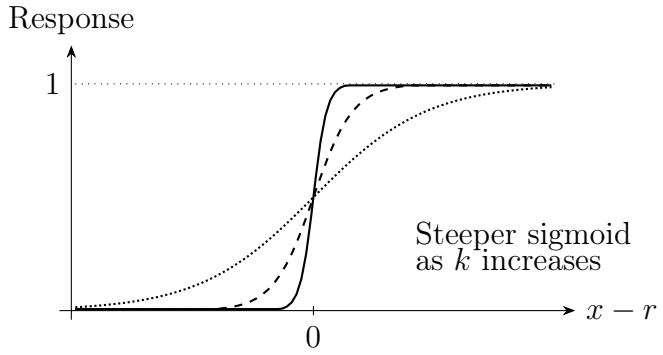


Figure 9: The scaled logistic response $\sigma(k(x-r))$. It approaches a hard threshold at r away from the discontinuity but not uniformly across that discontinuity.

Since σ is nondecreasing and $x - r > x - s$,

$$0 \leq P_{k;r,s}(x) \leq 1.$$

For $x \notin \{r, s\}$, taking $k \to \infty$ gives

$$P_{k;r,s}(x) \longrightarrow H_r(x) - H_s(x) = \mathbf{1}_{\{r<x\leq s\}}.$$

The exact values at the two endpoints are immaterial for the pointwise statement away from the jumps. The finite-k function is continuous, unlike the limiting interval indicator.

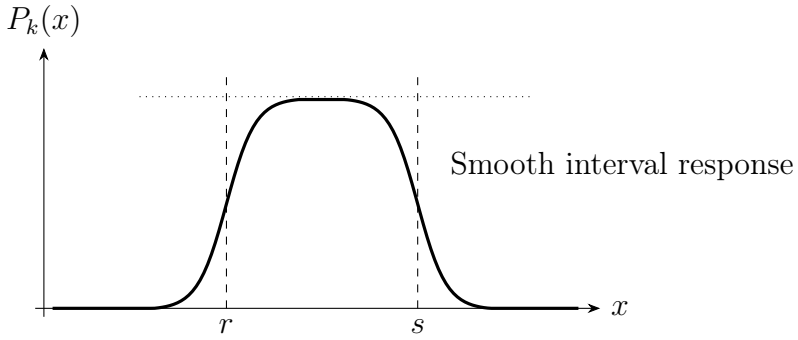


Figure 10: Subtracting two translated sigmoids produces a localized, approximately rectangular response.

4.6 A Complete Constructive Proof in One Dimension

The proof in the handwritten notes proposes approximating a continuous function by combining many short interval responses. The following argument makes that construction rigorous in the uniform norm, including the regions near interval boundaries.

Theorem 4.2 (Constructive one-dimensional universal approximation)**.** *Let $f \in C([a,b])$, let $\varepsilon > 0$, and let σ be a continuous nondecreasing sigmoidal function. Then there exist finitely many coefficients $\alpha_j, w_j, b_j, c \in \mathbb{R}$ such that*

$$\sup_{x\in[a,b]} \left| f(x) - c - \sum_{j=1}^{M} \alpha_j \sigma(w_j x + b_j) \right| < \varepsilon.$$

Proof. **Step 1: Use uniform continuity to select a partition.** Since $[a,b]$ is compact and f is continuous, f is uniformly continuous. Consequently, for the chosen $\varepsilon > 0$, there exists $\delta > 0$ such that

$$|x - y| < \delta \quad \Longrightarrow \quad |f(x) - f(y)| < \frac{\varepsilon}{2} \qquad (x, y \in [a,b]).$$

Choose an integer $m \geq 2$ sufficiently large that the equal mesh size

$$h := \frac{b-a}{m}$$

satisfies $2h < \delta$. Write

$$t_j := a + jh, \qquad j = 0, 1, \ldots, m,$$

and let the midpoint of interval $[t_j, t_{j+1}]$ be

$$u_j := \frac{t_j + t_{j+1}}{2}, \qquad j = 0, \ldots, m-1.$$

It follows that whenever $|x - u_j| < 2h$,

$$|f(x) - f(u_j)| < \frac{\varepsilon}{2}.$$

Step 2: Construct smooth weights that sum to one. For $k > 0$, define

$$\begin{aligned} p_0^{(k)}(x) &:= 1 - \sigma(k(x - t_1)), \\ p_j^{(k)}(x) &:= \sigma(k(x - t_j)) - \sigma(k(x - t_{j+1})), \qquad j = 1, \ldots, m-2, \\ p_{m-1}^{(k)}(x) &:= \sigma(k(x - t_{m-1})). \end{aligned}$$

Because σ is nondecreasing with values in $[0,1]$, each weight is nonnegative. Moreover the terms telescope:

$$\begin{aligned} \sum_{j=0}^{m-1} p_j^{(k)}(x) &= 1 - \sigma(k(x - t_1)) \\ &\quad + \sum_{j=1}^{m-2} \big[\sigma(k(x - t_j)) - \sigma(k(x - t_{j+1}))\big] \\ &\quad + \sigma(k(x - t_{m-1})) \\ &= 1. \end{aligned}$$

Thus the weights form a smooth partition of unity, meaning

$$p_j^{(k)}(x) \geq 0, \qquad \sum_{j=0}^{m-1} p_j^{(k)}(x) = 1.$$

Step 3: Form a weighted combination of nearby function values. Define

$$G_k(x) := \sum_{j=0}^{m-1} f(u_j) p_j^{(k)}(x).$$

By the partition-of-unity identity,

$$\begin{aligned} G_k(x) - f(x) &= \sum_{j=0}^{m-1} f(u_j) p_j^{(k)}(x) - f(x) \sum_{j=0}^{m-1} p_j^{(k)}(x) \\ &= \sum_{j=0}^{m-1} \big(f(u_j) - f(x)\big) p_j^{(k)}(x). \end{aligned}$$

Partition the indices into

$$\begin{aligned} A_x &:= \{j : |u_j - x| < 2h\}, \\ B_x &:= \{j : |u_j - x| \geq 2h\}. \end{aligned}$$

Writing $M_f := \sup_{x \in [a,b]} |f(x)| < \infty$, the triangle inequality gives

$$\begin{aligned} |G_k(x) - f(x)| &\leq \sum_{j \in A_x} |f(u_j) - f(x)| p_j^{(k)}(x) \\ &\quad + \sum_{j \in B_x} |f(u_j) - f(x)| p_j^{(k)}(x) \\ &< \frac{\varepsilon}{2} \sum_{j \in A_x} p_j^{(k)}(x) + 2M_f \sum_{j \in B_x} p_j^{(k)}(x) \\ &\leq \frac{\varepsilon}{2} + 2M_f \sum_{j \in B_x} p_j^{(k)}(x). \end{aligned}$$

Step 4: Show that distant intervals receive negligible weight. Define the two sigmoid-tail quantities

$$\eta_k := \max\big\{\sigma(-kh),\ 1 - \sigma(kh)\big\}.$$

By the two sigmoidal limits, $\eta_k \to 0$ as $k \to \infty$. We claim that if $|u_j - x| \geq 2h$, then

$$0 \leq p_j^{(k)}(x) \leq \eta_k.$$

Indeed, if $x \leq u_j - 2h$, then $x \leq t_j - 3h/2$, so (for an interior interval)

$$\begin{aligned} 0 \leq p_j^{(k)}(x) &\leq \sigma(k(x - t_j)) \\ &\leq \sigma(-3kh/2) \leq \sigma(-kh) \leq \eta_k. \end{aligned}$$

If $x \geq u_j + 2h$, then $x \geq t_{j+1} + 3h/2$, and similarly

$$\begin{aligned} 0 \leq p_j^{(k)}(x) &\leq 1 - \sigma(k(x - t_{j+1})) \\ &\leq 1 - \sigma(3kh/2) \leq 1 - \sigma(kh) \leq \eta_k. \end{aligned}$$

For the boundary indices $j = 0$ and $j = m - 1$, the same estimates follow directly from the definitions of $p_0^{(k)}$ and $p_{m-1}^{(k)}$; the geometrically impossible outward cases need not be considered because $x \in [a, b]$. There are at most m indices in B_x, so

$$\sum_{j \in B_x} p_j^{(k)}(x) \leq m\eta_k.$$

Substituting into the preceding error bound yields the uniform estimate

$$\sup_{x \in [a,b]} |G_k(x) - f(x)| \leq \frac{\varepsilon}{2} + 2M_f m\eta_k.$$

Because $\eta_k \to 0$, we may choose k sufficiently large that $2M_f m\eta_k < \varepsilon/2$ (if $M_f = 0$, the conclusion is immediate). Then

$$\sup_{x \in [a,b]} |G_k(x) - f(x)| < \varepsilon.$$

Step 5: Express the constructed function as a neural network. Expanding and collecting the sigmoid terms produces the telescoping representation

$$\begin{aligned} G_k(x) &= f(u_0)\big[1 - \sigma(k(x - t_1))\big] \\ &\quad + \sum_{j=1}^{m-2} f(u_j)\big[\sigma(k(x - t_j)) - \sigma(k(x - t_{j+1}))\big] \\ &\quad + f(u_{m-1})\sigma(k(x - t_{m-1})) \\ &= f(u_0) + \sum_{j=1}^{m-1} \big(f(u_j) - f(u_{j-1})\big)\sigma(k(x - t_j)). \end{aligned}$$

Thus G_k is exactly a single-hidden-layer network with output bias $c = f(u_0)$, hidden weights $w_j = k$, biases $b_j = -kt_j$, and output weights $\alpha_j = f(u_j) - f(u_{j-1})$. Since its uniform error is below ε, the proof is complete. □

Remark 4.6 (A constant can also be represented by a hidden unit). If one prefers the convention without a separate output bias, note that any continuous sigmoidal activation has a value $\sigma(b_0) \neq 0$ for some $b_0 \in \mathbb{R}$. The constant c may therefore be written as

$$c = \frac{c}{\sigma(b_0)}\sigma(0 \cdot x + b_0).$$

Hence allowing or omitting the explicit output bias does not alter the approximation class when network width is unrestricted.

4.6.1 Interpretation of the Construction

The proof implements the heuristic in the handwritten lecture: a sigmoidal hidden unit acts as an approximate threshold, and a difference of two such units acts as an approximate interval indicator. The target function is then approximated by summing appropriately weighted interval contributions. The crucial additional observation is that the *continuous* transitions can be made sufficiently narrow, and continuity of f ensures that the different neighboring sample values are close wherever their transition regions overlap.

The construction also clarifies the role of width and steepness. First choose a sufficiently fine partition so that f varies little within neighboring intervals. Then choose sufficiently steep sigmoids so that intervals far from x contribute negligibly. In particular, the order of parameter choices is

$$\varepsilon \;\longrightarrow\; \delta \;\longrightarrow\; m \;\longrightarrow\; k.$$

The number of required hidden units and the sizes of the weights may grow as the desired accuracy increases.

4.7 Universal Approximation in Several Dimensions

The one-dimensional partition argument illustrates the mechanism clearly, but a general input $x = (x_1, \dots, x_d)$ requires a result for functions on subsets of $\mathbb{R}^d$. We now prove the theorem previously stated for arbitrary compact K. The argument uses two classical facts from functional analysis: continuous linear functionals on $C(K)$ can be represented by finite signed measures, and a proper closed linear subspace of a normed space can be separated by a nonzero continuous linear functional.

Lemma 4.2 (Half-space separation of finite signed measures)**.** *Let μ be a finite signed Borel measure on $\mathbb{R}^d$, supported on a compact set K. Suppose that*

$$\mu\big(\{x : w^\top x > r\}\big) = 0 \qquad \text{for every } w \in \mathbb{R}^d \text{ and } r \in \mathbb{R}.$$

Then $\mu = 0$.

Proof. Fix $w \in \mathbb{R}^d$ and form the one-dimensional pushforward signed measure

$$\nu_w(B) := \mu\big(\{x : w^\top x \in B\}\big), \qquad B \in \mathcal{B}(\mathbb{R}).$$

By assumption, $\nu_w((r, \infty)) = 0$ for every r. Subtracting two such half-line values shows that $\nu_w((r, s]) = 0$ for every $r < s$; continuity of finite signed measures then shows that $\nu_w(\mathbb{R}) = 0$. Since these half-open intervals generate the Borel sets, ν_w vanishes identically.

For every w, therefore,

$$\begin{aligned} 0 &= \int_{\mathbb{R}} e^{iu}\, \nu_w(\mathrm{d}u) \\ &= \int_{\mathbb{R}^d} e^{iw^\top x}\, \mu(\mathrm{d}x). \end{aligned}$$

Thus the Fourier transform of μ vanishes everywhere. Uniqueness of the Fourier transform for finite signed measures implies $\mu = 0$.

For completeness, the final uniqueness step can be seen by applying Fourier inversion to any smooth, compactly supported test function φ. Since its Fourier transform $\widehat{\varphi}$ is integrable,

$$\begin{aligned}\int \varphi(x)\,\mu(\mathrm{d}x) &= \frac{1}{(2\pi)^d}\int_{\mathbb{R}^d} \widehat{\varphi}(\xi)\left(\int_{\mathbb{R}^d} e^{i\xi^\top x}\,\mu(\mathrm{d}x)\right)\mathrm{d}\xi \\ &= 0.\end{aligned}$$

Here we used the Fourier-transform convention for which the inversion formula carries $e^{i\xi^\top x}$. Smooth test functions determine finite signed measures, so $\mu = 0$. □

Lemma 4.3 (A continuous sigmoid is discriminatory)**.** *Let $K \subset \mathbb{R}^d$ be compact and let σ be continuous sigmoidal. If a finite signed Borel measure μ on K satisfies*

$$\int_K \sigma(w^\top x + b)\,\mu(\mathrm{d}x) = 0 \qquad \textit{for all } w \in \mathbb{R}^d,\ b \in \mathbb{R},$$

then $\mu = 0$.

Proof. Fix $w \in \mathbb{R}^d$, let ν_w be the pushforward of μ under $x \mapsto w^\top x$, and choose $r \in \mathbb{R}$. For every $k > 0$, the hypothesis applied to $(kw, -kr)$ gives

$$\begin{aligned}0 &= \int_K \sigma(k(w^\top x - r))\,\mu(\mathrm{d}x) \\ &= \int_{\mathbb{R}} \sigma(k(u - r))\,\nu_w(\mathrm{d}u).\end{aligned}$$

A finite signed measure has finite total variation. Since a continuous sigmoidal function is bounded, dominated convergence applies with respect to that total-variation measure. Whenever $\nu_w(\{r\}) = 0$, the pointwise limit of the integrand equals the indicator of (r, ∞) almost everywhere with respect to $|\nu_w|$. Therefore,

$$\begin{aligned}0 &= \lim_{k\to\infty}\int_{\mathbb{R}} \sigma(k(u - r))\,\nu_w(\mathrm{d}u) \\ &= \int_{\mathbb{R}} \mathbf{1}_{(r,\infty)}(u)\,\nu_w(\mathrm{d}u) \\ &= \nu_w((r, \infty)).\end{aligned}$$

The points with $|\nu_w|(\{r\}) > 0$ form an at most countable set. Given an arbitrary r, choose a sequence $r_n \downarrow r$ that avoids this exceptional set. Since $(r_n, \infty) \uparrow (r, \infty)$, continuity from below for finite signed measures gives

$$\nu_w((r, \infty)) = \lim_{n\to\infty} \nu_w((r_n, \infty)) = 0.$$

Thus every projected half-space has zero measure:

$$\mu\big(\{x : w^\top x > r\}\big) = 0 \qquad \text{for all } w, r.$$

The preceding lemma implies $\mu = 0$. $\square$

Proof of the multidimensional universal approximation theorem. Let $\mathcal{A}$ denote the linear span of the ridge functions

$$\mathcal{A} := \operatorname{span}\{x \mapsto \sigma(w^\top x + b) : w \in \mathbb{R}^d,\ b \in \mathbb{R}\}\big|_K.$$

Since σ is continuous, $\mathcal{A} \subset C(K)$. We claim that the uniform closure of $\mathcal{A}$ is all of $C(K)$.

Suppose instead that $\overline{\mathcal{A}} \neq C(K)$. By the Hahn–Banach separation theorem for a proper closed linear subspace, there exists a nonzero continuous linear functional $L : C(K) \to \mathbb{R}$ such that

$$L(g) = 0 \qquad \text{for every } g \in \mathcal{A}.$$

By the Riesz representation theorem, there is a nonzero finite signed Borel measure μ on K such that

$$L(g) = \int_K g(x)\,\mu(\mathrm{d}x), \qquad g \in C(K).$$

In particular, because each ridge function belongs to $\mathcal{A}$,

$$\int_K \sigma(w^\top x + b)\,\mu(\mathrm{d}x) = 0 \qquad \text{for every } w \in \mathbb{R}^d,\ b \in \mathbb{R}.$$

The discriminatory lemma forces $\mu = 0$, contradicting the fact that L is nonzero. Consequently,

$$\overline{\mathcal{A}}^{\|\cdot\|_\infty} = C(K).$$

The finite linear combinations defining $\mathcal{A}$ are precisely single-hidden-layer networks without a separate output bias. Adding an output bias cannot reduce their approximation power. Hence for every continuous f and every $\varepsilon > 0$ there is a finite network F satisfying $\|F - f\|_\infty < \varepsilon$, as claimed. $\square$

4.8 An Example: Approximating a Continuous Response

Consider the simple continuous regression target

$$f(x) = x^2, \qquad x \in [-1, 1].$$

The theorem guarantees that for every $\varepsilon > 0$ there exists a finite sigmoid network F with

$$\sup_{x \in [-1,1]} |F(x) - x^2| < \varepsilon.$$

The constructive proof tells us how to obtain such a network. Partition $[-1, 1]$ into equal subintervals, assign each subinterval the value of f at its midpoint, and blend those midpoint values using differences of steep sigmoids. The explicit network takes the form

$$F(x) = f(u_0) + \sum_{j=1}^{m-1} \big(f(u_j) - f(u_{j-1})\big)\sigma(k(x - t_j)).$$

No parameter-learning algorithm is needed to establish this *existence* statement. It supplies a candidate construction for a chosen partition and steepness, not necessarily the most economical representation of x^2.

4.9 What Universal Approximation Does and Does Not Guarantee

It is useful to distinguish three separate issues.

1. **Approximation.** Does the hypothesis class contain a network that represents the desired continuous function to a specified accuracy? This is what the universal approximation theorem establishes.
2. **Optimization.** Can an algorithm, such as gradient descent, find appropriate parameters from the observed data? Universal approximation alone gives no optimization guarantee.
3. **Generalization.** Does the network learned from finitely many examples have low expected error on new observations? This requires statistical assumptions, data, and further analysis beyond the approximation theorem.

Remark 4.7 (Why compactness and continuity matter). The theorem concerns uniform approximation of continuous functions on compact domains. It does not assert that a continuous neural network can uniformly approximate every discontinuous function. For example, the jump function H_0 cannot be uniformly approximated on $[-1, 1]$ by continuous networks: a uniform limit of continuous functions must be continuous. Similarly, the stated theorem does not provide a uniform approximation claim over an arbitrary unbounded domain such as all of $\mathbb{R}^d$.

Remark 4.8 (Training loss is not the approximation error). For a dataset $\mathcal{D}_N$, minimizing

$$\frac{1}{N}\sum_{n=1}^{N} \ell(F(x_n), t_n)$$

only constrains predictions at the observed inputs. In contrast, the universal approximation theorem controls

$$\sup_{x\in K} |F(x) - f(x)|,$$

which is a bound over *every* point in the entire compact input domain. These are fundamentally different statements.

4.10 Additional Derivations: Training a Single-Hidden-Layer Network

To connect the theorem's network representation with the supervised-learning objective, we derive the gradients for scalar regression. Consider

$$F_\theta(x) = c + \sum_{j=1}^{M} \alpha_j \sigma(w_j^\top x + b_j)$$

with the half-squared-error loss

$$\ell(F_\theta(x), t) = \frac{1}{2}(F_\theta(x) - t)^2.$$

Set

$$z_j = w_j^\top x + b_j, \qquad h_j = \sigma(z_j), \qquad e = F_\theta(x) - t.$$

By the chain rule,

$$\begin{aligned}
\frac{\partial \ell}{\partial F_\theta} &= e, \\
\frac{\partial F_\theta}{\partial \alpha_j} &= h_j, \\
\frac{\partial F_\theta}{\partial c} &= 1, \\
\frac{\partial F_\theta}{\partial b_j} &= \alpha_j \sigma'(z_j), \\
\nabla_{w_j} F_\theta &= \alpha_j \sigma'(z_j) x.
\end{aligned}$$

It follows that the individual-example parameter gradients are

$$\begin{aligned}
\frac{\partial \ell}{\partial \alpha_j} &= e\, \sigma(z_j), \\
\frac{\partial \ell}{\partial c} &= e, \\
\frac{\partial \ell}{\partial b_j} &= e\, \alpha_j \sigma'(z_j), \\
\nabla_{w_j} \ell &= e\, \alpha_j \sigma'(z_j) x.
\end{aligned}$$

For the empirical risk

$$\widehat{\mathcal{R}}_N(\theta) = \frac{1}{2N} \sum_{n=1}^{N} \big(F_\theta(x_n) - t_n\big)^2,$$

the gradient is the average of these contributions over the examples:

$$\nabla_\theta \widehat{\mathcal{R}}_N(\theta) = \frac{1}{N} \sum_{n=1}^{N} \nabla_\theta \ell(F_\theta(x_n), t_n).$$

A gradient-descent update with learning rate $\eta > 0$ is

$$\theta^{(r+1)} = \theta^{(r)} - \eta \nabla_\theta \widehat{\mathcal{R}}_N(\theta^{(r)}).$$

Thus the same weights and biases whose existence is established by universal approximation are the parameters adjusted during numerical training. The theorem guarantees that sufficiently expressive parameter choices *exist*, not that these gradient updates necessarily recover them.

5 Lecture 5: Loss Functions, Maximum Likelihood, and Gradient Descent

5.1 The Learning Objective

In Lecture 4, we introduced supervised learning as the problem of selecting parameters of a neural network so that its predictions are close to the corresponding targets. We now make this objective explicit, derive the loss functions used for regression and classification, and study how gradients can be used to fit the parameters.

5.1.1 Dataset, Network Outputs, and Parameters

Definition 5.1 (Supervised dataset)**.** Let $d_x \geq 1$ and $d_t \geq 1$. A supervised dataset of size N is

$$\mathcal{D}_N = \left\{ \left(x^{(d)}, t^{(d)} \right) : d = 1, \ldots, N \right\},$$

where $x^{(d)} \in \mathbb{R}^{d_x}$ is the feature vector and $t^{(d)}$ is its target. For regression, $t^{(d)} \in \mathbb{R}^{d_t}$; for classification the target is a class label or an equivalent binary or one-hot vector.

The superscript (d) identifies the training example, whereas a subscript identifies a coordinate. Thus

$$x^{(d)} = \begin{pmatrix} x_1^{(d)} & \cdots & x_{d_x}^{(d)} \end{pmatrix}^\top,$$
$$t^{(d)} = \begin{pmatrix} t_1^{(d)} & \cdots & t_{d_t}^{(d)} \end{pmatrix}^\top.$$

Let $\theta = (\theta_1, \ldots, \theta_p) \in \Theta \subseteq \mathbb{R}^p$ collect the weights and biases of a network. Denote its prediction by

$$y^{(d)} = f\left(x^{(d)}; \theta \right).$$

The symbol f represents the complete forward computation, including affine transformations and activation functions. The parameters θ are shared across examples; the input and output change from one example to another.

Definition 5.2 (Loss function)**.** A loss is a function $\ell(y, t)$ that measures the discrepancy between a prediction y and a target t. In the usual supervised-learning setting,

$$\ell : \mathcal{Y}_{\text{pred}} \times \mathcal{Y}_{\text{target}} \longrightarrow [0, +\infty].$$

A small loss indicates agreement between the prediction and the target, according to the criterion encoded by ℓ.

Definition 5.3 (Population and empirical risks)**.** Let (X, T) be a random input–target pair with unknown joint distribution P. The *population risk* of θ is

$$\begin{aligned}\mathcal{R}(\theta) &= \mathbb{E}_{(X,T)\sim P}\big[\ell\big(f(X;\theta), T\big)\big] \\ &= \int \ell\big(f(x;\theta), t\big)\, P(\mathrm{d}x, \mathrm{d}t).\end{aligned}$$

The corresponding *empirical risk* is the observed sample average

$$\widehat{\mathcal{R}}_N(\theta) = \frac{1}{N}\sum_{d=1}^{N} \ell\big(f(x^{(d)};\theta), t^{(d)}\big).$$

The ideal statistical objective and the computational training objective are therefore, respectively,

$$\begin{aligned}\text{Population:} \qquad & \inf_{\theta\in\Theta} \mathcal{R}(\theta), \\ \text{Empirical:} \qquad & \inf_{\theta\in\Theta} \widehat{\mathcal{R}}_N(\theta).\end{aligned}$$

We use an infimum because neither objective is automatically guaranteed to attain a minimum. The population expectation is usually unavailable: the data-generating distribution P is unknown. Training proceeds by minimizing the empirical average, perhaps with an additional regularization term.

Remark 5.1 (When a sample average estimates an expectation)*.* If the examples are independent and identically distributed from P and the loss is integrable, then for every *fixed* θ,

$$\mathbb{E}\big[\widehat{\mathcal{R}}_N(\theta)\big] = \mathcal{R}(\theta).$$

The strong law of large numbers additionally gives $\widehat{\mathcal{R}}_N(\theta) \to \mathcal{R}(\theta)$ almost surely for that fixed θ. This does not, without additional assumptions, imply uniform convergence over all θ or guarantee generalization for a parameter chosen using the training data.

Proposition 5.1 (Gradient of the empirical risk)**.** *Suppose each map $\theta \mapsto \ell(f(x^{(d)};\theta), t^{(d)})$ is differentiable. Then*

$$\nabla_\theta \widehat{\mathcal{R}}_N(\theta) = \frac{1}{N}\sum_{d=1}^{N} \nabla_\theta \ell\big(f(x^{(d)};\theta), t^{(d)}\big).$$

Proof. For each coordinate $j = 1, \ldots, p$, the finite sum can be differentiated term by term:

$$\begin{aligned}\frac{\partial \widehat{\mathcal{R}}_N}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j}\left[\frac{1}{N}\sum_{d=1}^{N} \ell\big(f(x^{(d)};\theta), t^{(d)}\big)\right] \\ &= \frac{1}{N}\sum_{d=1}^{N} \frac{\partial}{\partial \theta_j}\ell\big(f(x^{(d)};\theta), t^{(d)}\big).\end{aligned}$$

Stacking the p partial derivatives proves the result. □

Remark 5.2. Interchanging the gradient with the *population* expectation requires additional conditions, such as a suitable integrable bound on derivatives in a neighborhood of the parameter. Under such conditions,

$$\nabla_\theta \mathcal{R}(\theta) = \mathbb{E}\big[\nabla_\theta \ell\big(f(X;\theta), T\big)\big].$$

For empirical risk no such interchange issue arises because the sum is finite.

5.2 Regression and Mean Squared Error

Regression aims to predict a continuous scalar or vector target. The usual loss is based on Euclidean distance between the output and the target.

Definition 5.4 (Squared-error loss)**.** For $y, t \in \mathbb{R}^{d_t}$, define the *half-squared-error* loss by

$$\ell_{\mathrm{SE}}(y,t) = \frac{1}{2}\|y-t\|_2^2 = \frac{1}{2}\sum_{j=1}^{d_t}(y_j - t_j)^2.$$

For N training examples, the empirical objective is

$$\widehat{\mathcal{R}}_{\mathrm{SE}}(\theta) = \frac{1}{2N}\sum_{d=1}^{N}\sum_{j=1}^{d_t}\Big(f_j(x^{(d)};\theta) - t_j^{(d)}\Big)^2.$$

The factor $1/2$ makes differentiation cleaner. In some conventions "MSE" instead means $\frac{1}{N}\sum_d \|y^{(d)} - t^{(d)}\|_2^2$, possibly divided by the number of output coordinates as well. These constant factors do not change the minimizers, though they rescale the gradients.

Proposition 5.2 (Squared-error derivatives)**.** *With* $\ell_{\mathrm{SE}}(y,t) = \frac{1}{2}\|y-t\|_2^2$,

$$\nabla_y \ell_{\mathrm{SE}}(y,t) = y - t, \qquad \nabla_y^2 \ell_{\mathrm{SE}}(y,t) = I_{d_t}.$$

Proof. Differentiating componentwise, for $j, k \in \{1, \dots, d_t\}$,

$$\begin{aligned}
\frac{\partial \ell_{\mathrm{SE}}}{\partial y_j} &= \frac{1}{2}\frac{\partial}{\partial y_j}\sum_{k=1}^{d_t}(y_k - t_k)^2 \\
&= \frac{1}{2}\cdot 2(y_j - t_j) = y_j - t_j, \\
\frac{\partial^2 \ell_{\mathrm{SE}}}{\partial y_j \partial y_k} &= \begin{cases} 1, & j = k, \\ 0, & j \neq k. \end{cases}
\end{aligned}$$

Thus the Hessian with respect to y is the identity matrix. □

If $f(x;\theta)$ is differentiable in θ, the chain rule gives

$$\begin{aligned}
\nabla_\theta \ell_{\mathrm{SE}}\big(f(x;\theta), t\big) &= \big(D_\theta f(x;\theta)\big)^\top \nabla_y \ell_{\mathrm{SE}}(y,t)\big|_{y=f(x;\theta)} \\
&= \big(D_\theta f(x;\theta)\big)^\top \big(f(x;\theta) - t\big).
\end{aligned}$$

Here $D_\theta f$ is the $d_t \times p$ Jacobian of the network output with respect to its parameters. This relation exposes two ingredients of training: the prediction residual and the sensitivity of the network output to each parameter.

5.2.1 Why Squared Error Estimates the Conditional Mean

Proposition 5.3 (Optimal predictor under squared loss)**.** *Assume $T \in L^2$, and let $g(X)$ range over square-integrable functions of the input. The population mean-squared error*

$$\mathbb{E}\big[(g(X) - T)^2\big]$$

is minimized by the conditional mean $g^*(X) = \mathbb{E}[T \mid X]$, up to almost-sure equality.

Proof. Set $m(X) = \mathbb{E}[T \mid X]$. Decompose the prediction error as

$$g(X) - T = \big(g(X) - m(X)\big) + \big(m(X) - T\big).$$

Expanding the square and taking expectations gives

$$\begin{aligned}\mathbb{E}[(g(X) - T)^2] = \mathbb{E}[(g(X) - m(X))^2] + \mathbb{E}[(m(X) - T)^2] \\ + 2\mathbb{E}[(g(X) - m(X))(m(X) - T)].\end{aligned}$$

For the cross term, conditional expectation yields

$$\begin{aligned}&\mathbb{E}[(g(X) - m(X))(m(X) - T)] \\ &\quad = \mathbb{E}[(g(X) - m(X))\mathbb{E}[m(X) - T \mid X]] \\ &\quad = \mathbb{E}[(g(X) - m(X))(m(X) - m(X))] = 0.\end{aligned}$$

Consequently,

$$\begin{aligned}\mathbb{E}[(g(X) - T)^2] = \mathbb{E}[(m(X) - T)^2] + \mathbb{E}[(g(X) - m(X))^2] \\ \geq \mathbb{E}[(m(X) - T)^2],\end{aligned}$$

with equality exactly when $g(X) = m(X)$ almost surely. □

Remark 5.3 (What the loss asks the network to learn). Even if the true target contains randomness, squared-error regression aims to learn its conditional expectation, not necessarily the most likely target or an exact noiseless relation. Different losses generally elicit different summaries of the conditional distribution.

5.2.2 Squared Error as a Gaussian Negative Log-Likelihood

Suppose a scalar target follows a Gaussian conditional model with fixed noise variance:

$$T \mid X = x \sim \mathcal{N}\big(f(x;\theta), \sigma^2\big), \qquad \sigma^2 > 0.$$

For one observed target t, the conditional density is

$$p_\theta(t \mid x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(t - f(x;\theta))^2}{2\sigma^2}\right].$$

Taking the negative logarithm gives

$$\begin{aligned} -\log p_\theta(t \mid x) &= -\log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(t - f(x;\theta))^2}{2\sigma^2}\right)\right] \\ &= \frac{1}{2}\log(2\pi\sigma^2) + \frac{(t - f(x;\theta))^2}{2\sigma^2}. \end{aligned}$$

For an independent dataset, the negative log-likelihood is a constant plus

$$\frac{1}{2\sigma^2}\sum_{d=1}^{N}\big(t^{(d)} - f(x^{(d)};\theta)\big)^2.$$

Hence, for fixed σ^2, minimizing squared error is equivalent to maximizing the likelihood under a Gaussian observation model. This is a modeling interpretation, not a requirement that every regression problem actually have Gaussian noise.

5.2.3 A Linear-Regression Example

Consider $f(x; w, b) = w^\top x + b$, where $w \in \mathbb{R}^{d_x}$, and define the augmented vectors

$$\widetilde{x}^{(d)} = \begin{pmatrix} x^{(d)} \\ 1 \end{pmatrix}, \qquad \beta = \begin{pmatrix} w \\ b \end{pmatrix}.$$

Let $X \in \mathbb{R}^{N\times(d_x+1)}$ have rows $(\widetilde{x}^{(d)})^\top$, and let $t \in \mathbb{R}^N$ contain the scalar targets. The empirical half-squared-error is

$$J(\beta) = \frac{1}{2N}\|X\beta - t\|_2^2.$$

Expanding and differentiating yields

$$\begin{aligned} J(\beta) &= \frac{1}{2N}\big(\beta^\top X^\top X\beta - 2\beta^\top X^\top t + t^\top t\big), \\ \nabla_\beta J(\beta) &= \frac{1}{N}\big(X^\top X\beta - X^\top t\big), \\ \nabla_\beta^2 J(\beta) &= \frac{1}{N}X^\top X. \end{aligned}$$

The Hessian is positive semidefinite because $v^\top X^\top X v = \|Xv\|_2^2 \geq 0$. Any minimizer satisfies the normal equations

$$X^\top X\widehat{\beta} = X^\top t.$$

If X has full column rank, $X^\top X$ is invertible and the unique minimizer is

$$\widehat{\beta} = (X^\top X)^{-1}X^\top t.$$

For nonlinear neural networks, an analogous closed-form solution is generally unavailable, motivating iterative optimization.

5.3 Binary Classification and Bernoulli Cross-Entropy

In binary classification, each target belongs to one of two classes, usually represented by $t \in \{0, 1\}$. Rather than directly outputting a label, a neural network can produce a probability of class 1.

Definition 5.5 (Binary probabilistic predictor)**.** For an input x, write

$$y = f(x; \theta) \in (0, 1), \qquad y \approx \mathbb{P}(T = 1 \mid X = x).$$

The model assigns class probabilities

$$\mathbb{P}_\theta(T = 1 \mid X = x) = y, \qquad \mathbb{P}_\theta(T = 0 \mid X = x) = 1 - y.$$

A sigmoid output layer is one way to ensure that $y \in (0, 1)$.

5.3.1 Deriving the Bernoulli Likelihood

Definition 5.6 (Bernoulli distribution)**.** A Bernoulli variable T with success probability $y \in (0, 1)$ has probability mass function

$$\mathbb{P}(T = t) = y^t (1 - y)^{1-t}, \qquad t \in \{0, 1\}.$$

The compact formula reproduces the probabilities of both labels:

$$\begin{aligned} t = 1 : &\qquad y^1 (1 - y)^0 = y, \\ t = 0 : &\qquad y^0 (1 - y)^1 = 1 - y. \end{aligned}$$

Given N conditionally independent labels, with network outputs $y^{(d)} = f(x^{(d)}; \theta)$, the conditional likelihood is

$$\begin{aligned} L_N(\theta) &= \prod_{d=1}^{N} \mathbb{P}_\theta\big(T^{(d)} = t^{(d)} \mid X^{(d)} = x^{(d)}\big) \\ &= \prod_{d=1}^{N} \big(y^{(d)}\big)^{t^{(d)}} \big(1 - y^{(d)}\big)^{1-t^{(d)}}. \end{aligned}$$

Taking logarithms turns the product into a sum:

$$\begin{aligned} \log L_N(\theta) &= \sum_{d=1}^{N} \log \left[\big(y^{(d)}\big)^{t^{(d)}} \big(1 - y^{(d)}\big)^{1-t^{(d)}} \right] \\ &= \sum_{d=1}^{N} \left[t^{(d)} \log y^{(d)} + (1 - t^{(d)}) \log(1 - y^{(d)}) \right]. \end{aligned}$$

Maximizing the likelihood is thus equivalent to minimizing its negative logarithm. Dividing by N changes the numerical scale but not the maximizing or minimizing parameters.

Definition 5.7 (Binary cross-entropy)**.** The *binary cross-entropy* (BCE) of a predicted probability $y \in (0,1)$ and observed label $t \in \{0,1\}$ is

$$\ell_{\mathrm{BCE}}(y,t) = -\left[t \log y + (1-t)\log(1-y)\right].$$

Its empirical risk is

$$\widehat{\mathcal{R}}_{\mathrm{BCE}}(\theta) = -\frac{1}{N}\sum_{d=1}^{N}\left[t^{(d)}\log y^{(d)} + (1-t^{(d)})\log(1-y^{(d)})\right].$$

For a single example the loss specializes to

$$\ell_{\mathrm{BCE}}(y,t) = \begin{cases} -\log y, & t = 1, \\ -\log(1-y), & t = 0. \end{cases}$$

It assigns small loss to high probability on the observed class and a large loss to confidently wrong predictions. For example, predicting $y = 0.9$ when $t = 1$ incurs $-\log(0.9)$, whereas predicting $y = 0.1$ for the same target incurs $-\log(0.1)$.

Remark 5.4 (The sign of cross-entropy). Because $\log y \le 0$ and $\log(1-y) \le 0$ for $0 < y < 1$, the negative sign is essential. The *log-likelihood* is maximized, while its *negative* is the nonnegative loss minimized by gradient descent. At the boundary $y = 0$ or $y = 1$, a confidently incorrect prediction has infinite idealized cross-entropy.

5.3.2 Cross-Entropy Derivatives

Proposition 5.4 (Derivative with respect to predicted probability)**.** *For* $0 < y < 1$*,*

$$\frac{\partial \ell_{\mathrm{BCE}}(y,t)}{\partial y} = -\frac{t}{y} + \frac{1-t}{1-y} = \frac{y-t}{y(1-y)}.$$

Moreover,

$$\frac{\partial^2 \ell_{\mathrm{BCE}}(y,t)}{\partial y^2} = \frac{t}{y^2} + \frac{1-t}{(1-y)^2} > 0$$

for $t \in \{0,1\}$*.*

Proof. Term-by-term differentiation gives

$$\begin{aligned} \frac{\partial \ell_{\mathrm{BCE}}}{\partial y} &= -t\frac{1}{y} - (1-t)\frac{-1}{1-y} \\ &= -\frac{t}{y} + \frac{1-t}{1-y} \\ &= \frac{-t(1-y) + (1-t)y}{y(1-y)} \\ &= \frac{-t + ty + y - ty}{y(1-y)} = \frac{y-t}{y(1-y)}. \end{aligned}$$

Differentiating once more yields the stated positive second derivative. □

For the common sigmoid output $y = \sigma(z) = (1 + e^{-z})^{-1}$, the chain rule simplifies substantially.

Proposition 5.5 (Sigmoid plus BCE)**.** *If $y = \sigma(z)$ and $\ell(z, t) = \ell_{\mathrm{BCE}}(\sigma(z), t)$, then*

$$\frac{\partial \ell(z,t)}{\partial z} = \sigma(z) - t = y - t.$$

Proof. We know from Lecture 2 that $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = y(1 - y)$. Hence

$$\begin{aligned}\frac{\partial \ell}{\partial z} &= \frac{\partial \ell}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}z} \\ &= \frac{y - t}{y(1-y)}\, y(1-y) \\ &= y - t.\end{aligned}$$

The cancellation is one reason sigmoid–BCE is a convenient combination for binary probabilistic classification. □

5.3.3 Logistic Regression as a Single Artificial Neuron

Consider the affine logit

$$z = w^\top x + b, \qquad y = \sigma(z).$$

For one observation, differentiating the affine map gives

$$\nabla_w z = x, \qquad \frac{\partial z}{\partial b} = 1.$$

Thus

$$\begin{aligned}\nabla_w \ell_{\mathrm{BCE}}(\sigma(w^\top x + b), t) &= \frac{\partial \ell}{\partial z}\nabla_w z = (y - t)x, \\ \frac{\partial \ell_{\mathrm{BCE}}}{\partial b} &= y - t.\end{aligned}$$

For a full dataset,

$$\begin{aligned}\nabla_w \widehat{\mathcal{R}}_{\mathrm{BCE}}(w, b) &= \frac{1}{N}\sum_{d=1}^{N} \left(y^{(d)} - t^{(d)}\right) x^{(d)}, \\ \frac{\partial \widehat{\mathcal{R}}_{\mathrm{BCE}}}{\partial b} &= \frac{1}{N}\sum_{d=1}^{N} \left(y^{(d)} - t^{(d)}\right).\end{aligned}$$

The model's predicted log-odds are linear in the inputs:

$$\begin{aligned}\frac{y}{1-y} &= \frac{(1 + e^{-z})^{-1}}{e^{-z}(1 + e^{-z})^{-1}} = e^z, \\ \log\left(\frac{y}{1-y}\right) &= z = w^\top x + b.\end{aligned}$$

Consequently, the $y = 1/2$ decision boundary is the hyperplane $w^\top x + b = 0$.

Proposition 5.6 (Convexity of logistic-regression BCE)**.** *Let* $\widetilde{x} = (x^\top, 1)^\top$ *and* $\beta = (w^\top, b)^\top$. *For the sigmoid predictor* $y = \sigma(\beta^\top \widetilde{x})$*, the Hessian of the empirical BCE objective is*

$$\nabla_\beta^2 \widehat{\mathcal{R}}_{\mathrm{BCE}}(\beta) = \frac{1}{N} \sum_{d=1}^{N} y^{(d)}\big(1 - y^{(d)}\big)\widetilde{x}^{(d)}(\widetilde{x}^{(d)})^\top.$$

In particular, the objective is convex in β.

Proof. Differentiation of $\nabla_\beta \ell = (y - t)\widetilde{x}$ gives

$$\begin{aligned}\nabla_\beta^2 \ell &= \nabla_\beta y\, (\widetilde{x})^\top \\ &= \sigma'(\beta^\top \widetilde{x})\widetilde{x}\widetilde{x}^\top \\ &= y(1 - y)\widetilde{x}\widetilde{x}^\top.\end{aligned}$$

Taking the sample average gives the Hessian formula. For every v,

$$v^\top \nabla_\beta^2 \widehat{\mathcal{R}}_{\mathrm{BCE}}(\beta) v = \frac{1}{N} \sum_{d=1}^{N} y^{(d)}(1 - y^{(d)})\big(v^\top \widetilde{x}^{(d)}\big)^2 \geq 0.$$

Thus the Hessian is positive semidefinite and the empirical objective is convex. □

Remark 5.5. Convexity of logistic regression does not extend automatically to multilayer neural networks. In a deep network the logits are nonlinear functions of the trainable weights, and the resulting parameter optimization problem is generally nonconvex.

5.3.4 Why Cross-Entropy Estimates Probabilities

Proposition 5.7 (Optimal conditional binary prediction)**.** *Suppose that at a fixed input* x*, the true conditional probability of class* 1 *is* $q = \mathbb{P}(T = 1 \mid X = x) \in (0, 1)$. *The expected BCE of a candidate probability* $y \in (0, 1)$ *is*

$$r_q(y) = -q \log y - (1 - q) \log(1 - y).$$

Its unique minimizer is $y = q$.

Proof. By differentiation,

$$\begin{aligned}r_q'(y) &= -\frac{q}{y} + \frac{1 - q}{1 - y} = \frac{y - q}{y(1 - y)}, \\ r_q''(y) &= \frac{q}{y^2} + \frac{1 - q}{(1 - y)^2} > 0.\end{aligned}$$

The only stationary point is $y = q$. Strict positivity of the second derivative shows that this is the unique global minimum. □

Therefore BCE does not merely reward a correct hard class decision: at the population level, it rewards accurate predicted class probabilities. This property is important when a network's output is to be interpreted as a probability rather than just a score.

5.4 Multiclass Classification and Categorical Cross-Entropy

Now suppose there are $K \geq 2$ mutually exclusive classes, indexed by $\{1, \ldots, K\}$. A network outputs a vector of class probabilities $y = (y_1, \ldots, y_K)^\top$ with

$$y_k \geq 0, \qquad \sum_{k=1}^{K} y_k = 1.$$

Typically y is produced by a softmax output layer, so every coordinate is strictly positive.

Definition 5.8 (One-hot encoding)**.** For an observed class $c \in \{1, \ldots, K\}$, define the one-hot target $t \in \{0, 1\}^K$ by

$$t_k = \mathbf{1}_{\{k=c\}}, \qquad k = 1, \ldots, K.$$

Exactly one coordinate equals 1, and $\sum_{k=1}^{K} t_k = 1$.

For example, with $K = 4$ and true class $c = 3$,

$$t = \begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix}^\top.$$

5.4.1 Categorical Likelihood and Loss

For one observation the categorical probability mass function, written using one-hot targets, is

$$\begin{aligned} \mathbb{P}_\theta(T = c \mid X = x) &= y_c \\ &= \prod_{k=1}^{K} y_k^{t_k}. \end{aligned}$$

Indeed, all factors except the observed class appear to the power 0.

Definition 5.9 (Categorical cross-entropy)**.** The *categorical cross-entropy* (CCE) of a probability vector y and one-hot target t is

$$\ell_{\mathrm{CCE}}(y, t) = -\sum_{k=1}^{K} t_k \log y_k.$$

For an observed class c, this reduces to $\ell_{\mathrm{CCE}}(y, t) = -\log y_c$. Its empirical objective is

$$\widehat{\mathcal{R}}_{\mathrm{CCE}}(\theta) = -\frac{1}{N} \sum_{d=1}^{N} \sum_{k=1}^{K} t_k^{(d)} \log y_k^{(d)}.$$

The relation to maximum likelihood follows explicitly:

$$L_N(\theta) = \prod_{d=1}^{N} \prod_{k=1}^{K} \left(y_k^{(d)} \right)^{t_k^{(d)}},$$

$$\log L_N(\theta) = \sum_{d=1}^{N} \sum_{k=1}^{K} t_k^{(d)} \log y_k^{(d)},$$

$$-\frac{1}{N} \log L_N(\theta) = -\frac{1}{N} \sum_{d=1}^{N} \sum_{k=1}^{K} t_k^{(d)} \log y_k^{(d)}$$
$$= \widehat{\mathcal{R}}_{\mathrm{CCE}}(\theta).$$

Thus minimizing CCE is equivalent to maximizing the conditional likelihood of the observed class labels. The formula applies to binary classification as well if binary predictions are represented by the two probabilities $(1 - y, y)$.

5.4.2 The Softmax Output Layer

Definition 5.10 (Softmax)**.** For logits $z = (z_1, \ldots, z_K)^\top \in \mathbb{R}^K$, define

$$y_k = \mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K.$$

The output is a probability vector because each $e^{z_k} > 0$ and $\sum_k y_k = 1$.

For a one-hot target t, the loss can be rewritten directly in terms of logits:

$$\begin{aligned}
\ell_{\mathrm{CCE}}(\mathrm{softmax}(z), t) &= -\sum_{k=1}^{K} t_k \log \left(\frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \right) \\
&= -\sum_{k=1}^{K} t_k \left(z_k - \log \sum_{j=1}^{K} e^{z_j} \right) \\
&= -\sum_{k=1}^{K} t_k z_k + \left(\sum_{k=1}^{K} t_k \right) \log \sum_{j=1}^{K} e^{z_j} \\
&= \log \sum_{j=1}^{K} e^{z_j} - \sum_{k=1}^{K} t_k z_k.
\end{aligned}$$

For the observed class c, this is $\log(\sum_j e^{z_j}) - z_c$.

Remark 5.6 (Stable log-sum-exp). Because softmax is unchanged by subtracting a common constant from every logit, choose $a = \max_k z_k$. The loss becomes

$$\ell_{\mathrm{CCE}} = a + \log \left(\sum_{j=1}^{K} e^{z_j - a} \right) - z_c.$$

This numerically stable form avoids taking exponentials of very large positive logits and is preferred in implementations. The softmax and the logarithm should generally be combined in a stable loss routine.

5.4.3 The Softmax–Cross-Entropy Gradient

Proposition 5.8 (Derivative of softmax)**.** *For* $y_i = \mathrm{softmax}(z)_i$*,*

$$\frac{\partial y_i}{\partial z_j} = y_i\big(\delta_{ij} - y_j\big),$$

where δ_{ij} *is the Kronecker delta.*

Proof. With $S = \sum_{r=1}^{K} e^{z_r}$, the quotient rule gives

$$\begin{aligned}\frac{\partial y_i}{\partial z_j} &= \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} \\ &= \frac{e^{z_i}}{S}\left(\delta_{ij} - \frac{e^{z_j}}{S}\right) \\ &= y_i(\delta_{ij} - y_j).\end{aligned}$$

□

Proposition 5.9 (Softmax plus categorical cross-entropy)**.** *Let* $y = \mathrm{softmax}(z)$ *and* $\ell(z,t) = -\sum_{k=1}^{K} t_k \log y_k$*, where* t *is one-hot. Then*

$$\nabla_z \ell(z,t) = y - t.$$

Proof. By the multivariable chain rule,

$$\begin{aligned}\frac{\partial \ell}{\partial z_j} &= -\sum_{k=1}^{K} t_k \frac{1}{y_k}\frac{\partial y_k}{\partial z_j} \\ &= -\sum_{k=1}^{K} t_k(\delta_{kj} - y_j) \\ &= -t_j + y_j \sum_{k=1}^{K} t_k \\ &= -t_j + y_j,\end{aligned}$$

where the last step uses $\sum_k t_k = 1$. □

The resulting gradient $y - t$ has the same residual form as the gradient of half-squared error with respect to a linear output. However, its derivation reflects the combination of a probabilistic output model and negative log-likelihood.

5.4.4 A Linear Softmax Classifier

Let $W \in \mathbb{R}^{K \times d_x}$ and $b \in \mathbb{R}^K$ and define

$$z = Wx + b, \qquad y = \mathrm{softmax}(z).$$

For each weight W_{kj},

$$\begin{aligned}\frac{\partial \ell}{\partial W_{kj}} &= \sum_{r=1}^{K} \frac{\partial \ell}{\partial z_r} \frac{\partial z_r}{\partial W_{kj}} \\ &= \sum_{r=1}^{K} (y_r - t_r)\delta_{rk} x_j \\ &= (y_k - t_k) x_j.\end{aligned}$$

Similarly $\partial \ell / \partial b_k = y_k - t_k$, so the gradient is compactly

$$\nabla_W \ell = (y - t)x^\top, \qquad \nabla_b \ell = y - t.$$

For an empirical objective, average these matrices and vectors over the examples.

Proposition 5.10 (Convexity in the logits)**.** *For a one-hot target t, the function*

$$z \longmapsto \ell(z, t) = \log \sum_{k=1}^{K} e^{z_k} - t^\top z$$

is convex. Its Hessian is

$$\nabla_z^2 \ell = \operatorname{diag}(y) - yy^\top, \qquad y = \operatorname{softmax}(z).$$

Proof. The gradient is $y - t$, so the Hessian is exactly the Jacobian of softmax. For any $v \in \mathbb{R}^K$,

$$\begin{aligned}v^\top \nabla_z^2 \ell \, v &= \sum_{k=1}^{K} y_k v_k^2 - \left(\sum_{k=1}^{K} y_k v_k \right)^2 \\ &= \sum_{k=1}^{K} y_k \left(v_k - \sum_{j=1}^{K} y_j v_j \right)^2 \\ &\geq 0.\end{aligned}$$

The Hessian is therefore positive semidefinite, establishing convexity. □

Remark 5.7. Softmax probabilities are invariant under replacing z by $z + c\mathbf{1}$ for any constant c, so the Hessian has the all-ones vector in its nullspace. Thus the logit loss is convex but not strictly convex in all directions.

5.4.5 Entropy and Cross-Entropy at the Population Level

Suppose the true conditional class distribution at input x is $q = (q_1, \ldots, q_K)$, while the model predicts $y = (y_1, \ldots, y_K)$ with each $y_k > 0$. The expected categorical loss is

$$\mathbb{E}[-\log y_T \mid X = x] = -\sum_{k=1}^{K} q_k \log y_k.$$

Add and subtract $-\sum_k q_k \log q_k$ to obtain

$$\begin{aligned} -\sum_{k=1}^{K} q_k \log y_k &= -\sum_{k=1}^{K} q_k \log q_k + \sum_{k=1}^{K} q_k \log \frac{q_k}{y_k} \\ &= H(q) + D_{\mathrm{KL}}(q\|y), \end{aligned}$$

where $H(q) = -\sum_k q_k \log q_k$ is entropy and

$$D_{\mathrm{KL}}(q\|y) = \sum_{k:q_k>0} q_k \log \frac{q_k}{y_k}$$

is the Kullback–Leibler divergence, with the convention $0 \log 0 = 0$.

Lemma 5.1 (Nonnegativity of the categorical KL divergence)**.** *For any probability vectors q, y with $y_k > 0$,*

$$D_{\mathrm{KL}}(q\|y) \geq 0.$$

Equality holds only when $q = y$.

Proof. The inequality $\log u \leq u - 1$ holds for every $u > 0$. For each k with $q_k > 0$, set $u = y_k/q_k$ and multiply by q_k:

$$\begin{aligned} q_k \log \frac{y_k}{q_k} &\leq q_k \left(\frac{y_k}{q_k} - 1 \right) \\ &= y_k - q_k. \end{aligned}$$

Summing over coordinates for which $q_k > 0$, we obtain

$$\begin{aligned} -D_{\mathrm{KL}}(q\|y) &\leq \sum_{k:q_k>0} (y_k - q_k) \\ &= \sum_{k:q_k>0} y_k - 1 \leq 0. \end{aligned}$$

Equality requires equality in each logarithmic inequality and no mass outside the support of q, which together give $y = q$. □

Proposition 5.11 (Optimal multiclass probability prediction)**.** *The expected categorical cross-entropy at any fixed input is minimized when $y = q$, the true conditional probability vector, provided this vector lies in the allowed prediction set. If softmax restricts predictions to strictly positive coordinates and some $q_k = 0$, the optimum is approached in the limit rather than attained at finite logits.*

Proof. In the decomposition

$$\mathbb{E}[-\log y_T \mid X = x] = H(q) + D_{\mathrm{KL}}(q\|y),$$

the entropy $H(q)$ does not depend on the prediction y. The KL divergence is nonnegative and vanishes exactly when $y = q$. □

5.5 Gradient Descent and Gradient Ascent

The loss functions above produce scalar objectives defined on the parameter space. To fit the network, we need an algorithm that changes θ to improve the objective. Gradient methods repeatedly use local derivative information to select a direction of movement.

5.5.1 The Gradient and Its Geometric Meaning

Definition 5.11 (Gradient)**.** For a differentiable scalar function $E : \mathbb{R}^p \to \mathbb{R}$, its gradient is

$$\nabla E(\theta) = \begin{pmatrix} \dfrac{\partial E}{\partial \theta_1} \\ \vdots \\ \dfrac{\partial E}{\partial \theta_p} \end{pmatrix} \in \mathbb{R}^p.$$

The directional derivative in the unit direction u is

$$D_u E(\theta) = \nabla E(\theta)^\top u.$$

Proposition 5.12 (Steepest local increase and decrease)**.** *If $\nabla E(\theta) \neq 0$, the maximum and minimum of the directional derivative over all directions u with $\|u\|_2 = 1$ are*

$$\max_{\|u\|_2=1} D_u E(\theta) = \|\nabla E(\theta)\|_2,$$
$$\min_{\|u\|_2=1} D_u E(\theta) = -\|\nabla E(\theta)\|_2.$$

The maximizing direction is $u = \nabla E(\theta)/\|\nabla E(\theta)\|_2$; the minimizing direction is its negative.

Proof. For any unit vector u, the Cauchy–Schwarz inequality gives

$$\begin{aligned} -\|\nabla E(\theta)\|_2 &\leq \nabla E(\theta)^\top u \\ &\leq \|\nabla E(\theta)\|_2 \, \|u\|_2 \\ &= \|\nabla E(\theta)\|_2. \end{aligned}$$

Equality at the upper bound occurs when u is parallel to $\nabla E(\theta)$; equality at the lower bound occurs when u points in the opposite direction. □

For a small parameter displacement $\Delta\theta$, first-order Taylor expansion states

$$E(\theta + \Delta\theta) = E(\theta) + \nabla E(\theta)^\top \Delta\theta + o(\|\Delta\theta\|_2).$$

If $\Delta\theta = -\eta \nabla E(\theta)$ with $\eta > 0$, the first-order change becomes

$$E(\theta - \eta \nabla E(\theta)) - E(\theta) = -\eta \|\nabla E(\theta)\|_2^2 + o\big(\eta \|\nabla E(\theta)\|_2\big).$$

Thus a sufficiently small step along the *negative* gradient locally decreases the objective at a point with nonzero gradient.

5.5.2 Gradient Flow and Euler Discretization

Definition 5.12 (Gradient flow)**.** The continuous-time *gradient-descent flow* for an objective E is the differential equation

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla E\big(\theta(t)\big).$$

The analogous *gradient-ascent flow* is

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = +\nabla E\big(\theta(t)\big).$$

To derive the discrete update, choose a time step $\eta > 0$ and approximate the derivative by a forward difference:

$$\begin{aligned}
\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} &\approx \frac{\theta(t+\eta) - \theta(t)}{\eta}, \\
\frac{\theta(t+\eta) - \theta(t)}{\eta} &= -\nabla E\big(\theta(t)\big), \\
\theta(t+\eta) &= \theta(t) - \eta \nabla E\big(\theta(t)\big).
\end{aligned}$$

Writing $\theta^{(r)}$ for the rth iterate gives *gradient descent*:

$$\theta^{(r+1)} = \theta^{(r)} - \eta \nabla E(\theta^{(r)}).$$

For gradient ascent, the sign changes:

$$\theta^{(r+1)} = \theta^{(r)} + \eta \nabla E(\theta^{(r)}).$$

The positive scalar η is the *step size* or *learning rate*.

Remark 5.8 (Why a sign error matters). Gradient descent minimizes an error such as BCE or MSE, whereas gradient ascent maximizes an objective such as log-likelihood. Since BCE and CCE are *negative* log-likelihoods, descending on cross-entropy corresponds exactly to ascending on log-likelihood. One should not maximize a nonnegative loss merely because its derivation involved maximum likelihood.

Proposition 5.13 (Risk decreases along gradient flow)**.** *Let E be continuously differentiable and let $\theta(t)$ be a differentiable solution of $\dot{\theta} = -\nabla E(\theta)$. Then*

$$\frac{\mathrm{d}}{\mathrm{d}t} E(\theta(t)) = -\|\nabla E(\theta(t))\|_2^2 \leq 0.$$

In particular, the objective is nonincreasing along the flow.

Proof. By the multivariable chain rule and the definition of gradient flow,

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}t} E(\theta(t)) &= \nabla E(\theta(t))^{\top} \dot{\theta}(t) \\
&= \nabla E(\theta(t))^{\top} \big(-\nabla E(\theta(t))\big) \\
&= -\|\nabla E(\theta(t))\|_2^2 \leq 0.
\end{aligned}$$

□

Remark 5.9. The continuous-time flow has a built-in monotonicity property. Discrete gradient descent requires appropriate step-size choices: an excessively large learning rate can overshoot and increase the objective.

5.5.3 Two-Parameter Gradient Descent

For a function $E(a, b)$, the gradient is

$$\nabla E(a,b) = \begin{pmatrix} \dfrac{\partial E}{\partial a} \\ \dfrac{\partial E}{\partial b} \end{pmatrix}.$$

The simultaneous gradient descent update at iteration r is

$$a^{(r+1)} = a^{(r)} - \eta \frac{\partial E}{\partial a}\big(a^{(r)}, b^{(r)}\big),$$
$$b^{(r+1)} = b^{(r)} - \eta \frac{\partial E}{\partial b}\big(a^{(r)}, b^{(r)}\big).$$

Both partial derivatives are evaluated at the same current point $\big(a^{(r)}, b^{(r)}\big)$. The ascent update replaces $-\eta$ by $+\eta$.

Example 5.1 (A quadratic bowl)**.** Consider

$$E(a,b) = \frac{1}{2}(a-2)^2 + 2(b+1)^2.$$

Then

$$\nabla E(a,b) = \begin{pmatrix} a-2 \\ 4(b+1) \end{pmatrix}, \qquad \nabla^2 E = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

Gradient descent takes the form

$$a^{(r+1)} - 2 = (1-\eta)(a^{(r)} - 2),$$
$$b^{(r+1)} + 1 = (1-4\eta)(b^{(r)} + 1).$$

Iterating the recurrences gives

$$a^{(r)} - 2 = (1-\eta)^r (a^{(0)} - 2),$$
$$b^{(r)} + 1 = (1-4\eta)^r (b^{(0)} + 1).$$

Both errors converge to zero exactly when $|1-\eta| < 1$ and $|1-4\eta| < 1$, i.e. when $0 < \eta < 1/2$. The unique minimizer is $(a, b) = (2, -1)$.

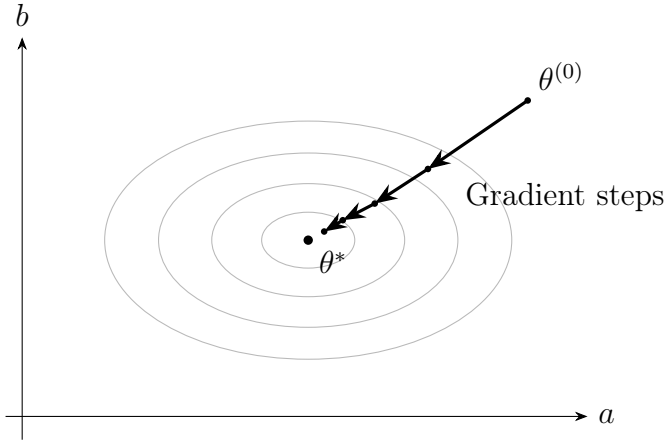


Figure 11: Illustration of gradient descent on elliptical level sets of a convex quadratic objective. Arrows move from a higher-loss parameter to lower-loss parameters.

5.5.4 Step Size and the Descent Lemma

To make the local descent claim quantitative, it is useful to control the rate at which the gradient can change.

Definition 5.13 (Lipschitz-continuous gradient)**.** A differentiable function $E : \mathbb{R}^p \to \mathbb{R}$ has an L-Lipschitz gradient, for $L > 0$, if

$$\|\nabla E(u) - \nabla E(v)\|_2 \le L\|u - v\|_2 \qquad \text{for all } u, v \in \mathbb{R}^p.$$

Such a function is also called L-smooth.

Lemma 5.2 (Descent lemma)**.** *If E has an L-Lipschitz gradient, then for all $u, v \in \mathbb{R}^p$,*

$$E(v) \le E(u) + \nabla E(u)^\top (v - u) + \frac{L}{2}\|v - u\|_2^2.$$

Proof. Set $d = v - u$ and define $\varphi(s) = E(u + sd)$ for $s \in [0, 1]$. The fundamental theorem of calculus and chain rule give

$$\begin{aligned} E(v) - E(u) &= \varphi(1) - \varphi(0) \\ &= \int_0^1 \varphi'(s)\,\mathrm{d}s \\ &= \int_0^1 \nabla E(u + sd)^\top d\,\mathrm{d}s \\ &= \nabla E(u)^\top d + \int_0^1 \big(\nabla E(u + sd) - \nabla E(u)\big)^\top d\,\mathrm{d}s. \end{aligned}$$

The Cauchy–Schwarz inequality and L-smoothness imply

$$\begin{aligned} \big|\big(\nabla E(u + sd) - \nabla E(u)\big)^\top d\big| &\le \|\nabla E(u + sd) - \nabla E(u)\|_2 \|d\|_2 \\ &\le Ls\|d\|_2^2. \end{aligned}$$

Therefore,

$$\begin{aligned} E(v) - E(u) &\le \nabla E(u)^\top d + \int_0^1 Ls\|d\|_2^2 \,\mathrm{d}s \\ &= \nabla E(u)^\top (v-u) + \frac{L}{2}\|v-u\|_2^2. \end{aligned}$$

□

Proposition 5.14 (Guaranteed decrease for a smooth objective)**.** *Let E be L-smooth and let*

$$\theta^+ = \theta - \eta \nabla E(\theta).$$

Then

$$E(\theta^+) \le E(\theta) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla E(\theta)\|_2^2.$$

In particular, for $0 < \eta < 2/L$, any step at a point with nonzero gradient strictly decreases the objective.

Proof. Apply the descent lemma with $u = \theta$ and $v = \theta^+$. Since

$$v - u = -\eta \nabla E(\theta),$$

we find

$$\begin{aligned} E(\theta^+) &\le E(\theta) + \nabla E(\theta)^\top\big(-\eta \nabla E(\theta)\big) + \frac{L}{2}\big\|-\eta \nabla E(\theta)\big\|_2^2 \\ &= E(\theta) - \eta\|\nabla E(\theta)\|_2^2 + \frac{L\eta^2}{2}\|\nabla E(\theta)\|_2^2 \\ &= E(\theta) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla E(\theta)\|_2^2. \end{aligned}$$

If $0 < \eta < 2/L$, the coefficient $1 - L\eta/2$ is positive. □

Corollary 5.1 (Gradient-norm convergence under standard conditions)**.** *Suppose E is globally L-smooth, is bounded below by $E_{\inf} > -\infty$, and gradient descent uses a constant step size $0 < \eta < 2/L$. Then*

$$\sum_{r=0}^{\infty} \|\nabla E(\theta^{(r)})\|_2^2 < \infty, \qquad \|\nabla E(\theta^{(r)})\|_2 \longrightarrow 0.$$

Proof. Set $c = \eta(1 - L\eta/2) > 0$. The previous proposition gives

$$c\|\nabla E(\theta^{(r)})\|_2^2 \le E(\theta^{(r)}) - E(\theta^{(r+1)}).$$

Summing from $r = 0$ to $M - 1$ telescopes:

$$c \sum_{r=0}^{M-1} \|\nabla E(\theta^{(r)})\|_2^2 \leq E(\theta^{(0)}) - E(\theta^{(M)}) \leq E(\theta^{(0)}) - E_{\inf}.$$

The infinite series of nonnegative terms therefore converges. Its terms tend to zero, proving the gradient-norm claim. □

Remark 5.10 (A stationary point is not always a global minimum). The conclusion $\|\nabla E(\theta^{(r)})\| \to 0$ does not imply that the parameter iterates converge, nor that they approach a global minimizer. For nonconvex neural-network losses, stationary points may include local minima, saddle points, and other critical configurations. The global L-smoothness assumption is also a mathematical idealization that must be checked for the specific objective.

5.6 Full-Batch, Stochastic, and Mini-Batch Gradients

For a dataset of size N, the empirical risk is a finite average

$$E_N(\theta) = \frac{1}{N} \sum_{d=1}^{N} \ell_d(\theta), \qquad \ell_d(\theta) = \ell\big(f(x^{(d)}; \theta), t^{(d)}\big).$$

Full-batch gradient descent computes the complete gradient

$$g_{\text{full}}(\theta) = \nabla E_N(\theta) = \frac{1}{N} \sum_{d=1}^{N} \nabla \ell_d(\theta)$$

and performs the update

$$\theta^{(r+1)} = \theta^{(r)} - \eta_r g_{\text{full}}(\theta^{(r)}).$$

When N is large, evaluating every example for every update can be expensive. Stochastic and mini-batch methods replace the full gradient with a sample estimate.

Definition 5.14 (Mini-batch gradient)**.** Let $B \subseteq \{1, \ldots, N\}$ be a mini-batch of size $m = |B| \geq 1$. Define

$$g_B(\theta) = \frac{1}{m} \sum_{d \in B} \nabla \ell_d(\theta).$$

A mini-batch stochastic gradient descent update is

$$\theta^{(r+1)} = \theta^{(r)} - \eta_r g_{B_r}(\theta^{(r)}).$$

The case $m = 1$ is ordinary stochastic gradient descent (SGD); $m = N$ reproduces full-batch gradient descent.

Proposition 5.15 (Unbiased mini-batch gradient)**.** *Fix θ and sample a batch B of size m uniformly without replacement from the N example indices. Then*

$$\mathbb{E}_B[g_B(\theta)] = \nabla E_N(\theta).$$

Proof. Let $I_d = \mathbf{1}_{\{d \in B\}}$. Uniform sampling without replacement gives $\mathbb{E}_B[I_d] = m/N$. Thus

$$\begin{aligned}
\mathbb{E}_B[g_B(\theta)] &= \mathbb{E}_B\left[\frac{1}{m}\sum_{d=1}^{N} I_d \nabla \ell_d(\theta)\right] \\
&= \frac{1}{m}\sum_{d=1}^{N} \mathbb{E}_B[I_d]\nabla \ell_d(\theta) \\
&= \frac{1}{m}\sum_{d=1}^{N} \frac{m}{N}\nabla \ell_d(\theta) \\
&= \frac{1}{N}\sum_{d=1}^{N} \nabla \ell_d(\theta) \\
&= \nabla E_N(\theta).
\end{aligned}$$

□

Remark 5.11 (Noise in stochastic training). The mini-batch gradient is generally random even when the current parameter θ is fixed. It is unbiased for the *empirical* gradient under the sampling rule above, not automatically for the population gradient. Smaller batches produce cheaper but typically noisier updates; larger batches are more computationally expensive per update but average over more examples.

5.7 Training Neural Networks by the Chain Rule

Gradient descent requires derivatives with respect to all weights and biases. Because a feedforward network is a composition of elementary functions, these derivatives can be obtained systematically by the chain rule. This is the mathematical basis of *backpropagation*.

5.7.1 A Single Neuron with Squared-Error Loss

Consider a scalar neuron with differentiable activation σ:

$$z = w^\top x + b, \qquad y = \sigma(z), \qquad \ell = \frac{1}{2}(y - t)^2.$$

Applying the chain rule step by step,

$$\begin{aligned}
\frac{\partial \ell}{\partial y} &= y - t, \\
\frac{\partial y}{\partial z} &= \sigma'(z),
\end{aligned}$$

$$\frac{\partial z}{\partial w_j} = x_j,$$
$$\frac{\partial z}{\partial b} = 1.$$

Therefore

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial y}\frac{\partial y}{\partial z} = (y-t)\sigma'(z),$$
$$\frac{\partial \ell}{\partial w_j} = \frac{\partial \ell}{\partial z}\frac{\partial z}{\partial w_j} = (y-t)\sigma'(z)x_j,$$
$$\frac{\partial \ell}{\partial b} = (y-t)\sigma'(z).$$

In vector form,

$$\nabla_w \ell = (y-t)\sigma'(z)x, \qquad \frac{\partial \ell}{\partial b} = (y-t)\sigma'(z).$$

This shows how a mismatch at the output is propagated back through the activation to the individual input weights.

5.7.2 Backpropagation Through a Network

Consider a network of L affine-plus-activation layers. Let $a^{(0)} = x$ and for $\ell = 1, \ldots, L$ define

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)},$$
$$a^{(\ell)} = \phi^{(\ell)}(z^{(\ell)}),$$

where each $\phi^{(\ell)}$ acts coordinatewise, except that the final activation can be softmax. The network output is $y = a^{(L)}$.

Definition 5.15 (Backpropagated error)**.** For a scalar loss $\ell(y, t)$, define the pre-activation error vector

$$\delta^{(\ell)} := \nabla_{z^{(\ell)}} \ell.$$

It measures how sensitive the loss is to a change in the pre-activation at layer ℓ.

Proposition 5.16 (Backpropagation recursion)**.** *Assume all relevant derivatives exist. For a hidden layer $\ell < L$ with coordinatewise activation $\phi^{(\ell)}$,*

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right) \odot \phi^{(\ell)\prime}(z^{(\ell)}),$$

where $\odot$ denotes componentwise multiplication. The parameter gradients at each layer are

$$\nabla_{W^{(\ell)}} \ell = \delta^{(\ell)} (a^{(\ell-1)})^\top, \qquad \nabla_{b^{(\ell)}} \ell = \delta^{(\ell)}.$$

For a sigmoid output with BCE, or a softmax output with CCE, the output-layer error simplifies to

$$\delta^{(L)} = y - t.$$

Proof. By the chain rule, the derivative of the loss with respect to hidden activation $a_i^{(\ell)}$ is

$$\begin{aligned}\frac{\partial \ell}{\partial a_i^{(\ell)}} &= \sum_j \frac{\partial \ell}{\partial z_j^{(\ell+1)}} \frac{\partial z_j^{(\ell+1)}}{\partial a_i^{(\ell)}} \\ &= \sum_j \delta_j^{(\ell+1)} W_{ji}^{(\ell+1)} \\ &= \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right)_i .\end{aligned}$$

Since $a_i^{(\ell)} = \phi^{(\ell)}(z_i^{(\ell)})$,

$$\begin{aligned}\delta_i^{(\ell)} &= \frac{\partial \ell}{\partial a_i^{(\ell)}} \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}} \\ &= \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right)_i \phi^{(\ell)\prime}(z_i^{(\ell)}),\end{aligned}$$

which is the recursion. Furthermore,

$$z_i^{(\ell)} = \sum_j W_{ij}^{(\ell)} a_j^{(\ell-1)} + b_i^{(\ell)},$$

so $\partial z_i^{(\ell)} / \partial W_{ij}^{(\ell)} = a_j^{(\ell-1)}$ and $\partial z_i^{(\ell)} / \partial b_i^{(\ell)} = 1$. Therefore,

$$\begin{aligned}\frac{\partial \ell}{\partial W_{ij}^{(\ell)}} &= \delta_i^{(\ell)} a_j^{(\ell-1)}, \\ \frac{\partial \ell}{\partial b_i^{(\ell)}} &= \delta_i^{(\ell)}.\end{aligned}$$

The output-layer formula follows from the previously derived sigmoid–BCE or softmax–CCE gradient identities. □

Remark 5.12. Backpropagation is a method for computing gradients efficiently; it is not itself an optimization method. Gradient descent, SGD, or a different optimizer then uses the computed gradients to update the parameters.

5.8 Worked Examples

Example 5.2 (One gradient-descent step for regression)**.** Consider the model $f(x; w, b) = wx + b$ and one observation $(x, t) = (2, 1)$. Start from $w^{(0)} = 0$ and $b^{(0)} = 0$. The loss is

$$E(w, b) = \frac{1}{2}(wx + b - 1)^2.$$

At the starting point, $y = 0$ and $y - t = -1$. Therefore

$$\begin{aligned}\frac{\partial E}{\partial w} &= (y - t)x = -2, \\ \frac{\partial E}{\partial b} &= y - t = -1.\end{aligned}$$

Using $\eta = 0.1$, the updated parameters are

$$w^{(1)} = 0 - 0.1(-2) = 0.2,$$
$$b^{(1)} = 0 - 0.1(-1) = 0.1.$$

The new prediction is $y^{(1)} = 0.2(2) + 0.1 = 0.5$. Hence

$$E(w^{(0)}, b^{(0)}) = \frac{1}{2}(0-1)^2 = 0.5,$$
$$E(w^{(1)}, b^{(1)}) = \frac{1}{2}(0.5-1)^2 = 0.125.$$

The step decreases the loss by 0.375.

Example 5.3 (One gradient-descent step for binary classification)**.** For one feature $x = 2$ and label $t = 1$, let

$$y = \sigma(wx + b), \qquad w^{(0)} = b^{(0)} = 0.$$

Initially $z = 0$ and $y = \sigma(0) = 1/2$. The BCE loss is

$$\ell = -\log(1/2) = \log 2.$$

Using the sigmoid–BCE derivative,

$$\frac{\partial \ell}{\partial w} = (y - t)x = (1/2 - 1)2 = -1,$$
$$\frac{\partial \ell}{\partial b} = y - t = -\frac{1}{2}.$$

With $\eta = 0.1$,

$$w^{(1)} = 0.1, \qquad b^{(1)} = 0.05.$$

The updated logit and predicted probability are

$$z^{(1)} = 0.1(2) + 0.05 = 0.25,$$
$$y^{(1)} = \sigma(0.25) > \frac{1}{2}.$$

Thus the predicted probability of the true class increases, and its negative log-likelihood decreases.

Example 5.4 (Categorical loss and output-layer gradient)**.** Suppose a three-class classifier assigns probabilities

$$y = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}, \qquad t = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

The observed label is class 2. The categorical cross-entropy is

$$\ell_{\mathrm{CCE}}(y, t) = -\sum_{k=1}^{3} t_k \log y_k = -\log(0.5) = \log 2.$$

If y is the softmax of logits z, then

$$\nabla_z \ell = y - t = \begin{pmatrix} 0.2 \\ -0.5 \\ 0.3 \end{pmatrix}.$$

Gradient descent moves the logits in direction $-(y - t)$: it tends to increase the true class logit and decrease the other logits locally.

6 Lecture 6: Backpropagation and the Chain Rule in Multilayer Networks

6.1 From Gradient Descent to Backpropagation

In Lecture 5, we formulated the training objective of a neural network as the minimization of a loss function over its parameters. If the parameter vector is $\theta \in \mathbb{R}^p$, an iteration of gradient descent has the form

$$\theta^{(r+1)} = \theta^{(r)} - \eta_r \nabla_\theta E(\theta^{(r)}).$$

The mathematical problem is therefore to calculate $\nabla_\theta E$ efficiently. In a multilayer network, every weight affects the loss indirectly through a sequence of intermediate pre-activations and activations. The resulting derivatives must account for all directed paths from the weight to the output.

Definition 6.1 (Backpropagation)**.** *Backpropagation* is an algorithm for computing derivatives of a scalar loss with respect to the parameters of a computational graph. It applies the multivariable chain rule in reverse order, reusing derivatives of intermediate quantities rather than recomputing every path independently.

Backpropagation computes the gradient; it is not itself an optimization method. Once the gradient is known, an optimizer such as gradient descent or stochastic gradient descent determines how the parameters change.

6.1.1 A One-Hidden-Layer Network

Consider a fully connected feedforward network with d input units, H hidden units, and K output units. For one input–target pair (x, t), write

$$x = (x_1, \dots, x_d)^\top \in \mathbb{R}^d,$$
$$t = (t_1, \dots, t_K)^\top \in \mathbb{R}^K.$$

Let w_{ki} connect input unit k to hidden unit i, and let m_{ij} connect hidden unit i to output unit j. Let a_i and b_j denote the corresponding biases. The network computes

$$\alpha_i = \sum_{k=1}^{d} w_{ki} x_k + a_i, \qquad h_i = \phi(\alpha_i), \qquad i = 1, \dots, H,$$
$$\beta_j = \sum_{i=1}^{H} m_{ij} h_i + b_j, \qquad y_j = \psi_j(\beta), \qquad j = 1, \dots, K,$$
$$E = \ell(y, t).$$

Here ϕ is the hidden activation function, and ψ is the output activation. When the output activation acts coordinatewise, $\psi_j(\beta) = \psi(\beta_j)$. In a softmax classifier, however, each ψ_j depends on the entire vector β.

The network defines the computation

$$x \longrightarrow \alpha \longrightarrow h \longrightarrow \beta \longrightarrow y \longrightarrow E.$$

The weights enter at different positions in this chain, and hence their gradients have different forms.

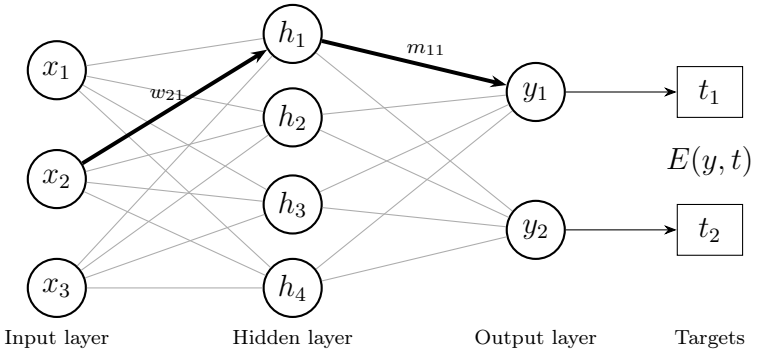


Figure 12: A network with three inputs, four hidden neurons, and two outputs. The highlighted connections illustrate the path from an input weight to the loss through a hidden unit and an output. Every hidden unit can affect every output in this fully connected example.

Remark 6.1 (Dependencies of the loss). The scalar loss $E = \ell(y, t)$ depends on a weight m_{ij} through the output pre-activation β_j, while it depends on an input–hidden weight w_{ki} through α_i, h_i, and potentially *every* output pre-activation β_j. The latter case requires a sum of chain-rule contributions.

6.2 The Multivariable Chain Rule

We first state the calculus principle underlying both the output-layer and hidden-layer calculations.

Proposition 6.1 (Chain rule for a scalar loss)**.** *Let $u \in \mathbb{R}^n$, let $v = F(u) \in \mathbb{R}^m$ be differentiable, and let $E = G(v) \in \mathbb{R}$ be differentiable. If $DF(u) \in \mathbb{R}^{m\times n}$ is the Jacobian of F, then*

$$\nabla_u E = DF(u)^\top \nabla_v G.$$

In coordinates,

$$\frac{\partial E}{\partial u_i} = \sum_{j=1}^{m} \frac{\partial E}{\partial v_j}\frac{\partial v_j}{\partial u_i}.$$

Proof. By the ordinary multivariable chain rule, the derivative with respect to the ith coordinate is

$$\begin{aligned}\frac{\partial E}{\partial u_i} &= \frac{\partial}{\partial u_i} G(F_1(u), \dots, F_m(u)) \\ &= \sum_{j=1}^{m} \frac{\partial G}{\partial v_j}(F(u)) \frac{\partial F_j}{\partial u_i}(u).\end{aligned}$$

Since $(DF(u))_{ji} = \partial F_j / \partial u_i$, the ith coordinate of $DF(u)^\top \nabla_v G$ equals the last sum. Stacking the coordinates establishes the vector identity. □

Example 6.1 (A branching computational graph)**.** Suppose $q = q(u)$ and two downstream quantities $r_1 = r_1(q)$ and $r_2 = r_2(q)$ enter the same loss, so $E = F(r_1, r_2)$. Then

$$\begin{aligned}\frac{\mathrm{d}E}{\mathrm{d}u} &= \frac{\partial E}{\partial r_1}\frac{\mathrm{d}r_1}{\mathrm{d}q}\frac{\mathrm{d}q}{\mathrm{d}u} + \frac{\partial E}{\partial r_2}\frac{\mathrm{d}r_2}{\mathrm{d}q}\frac{\mathrm{d}q}{\mathrm{d}u} \\ &= \left(\frac{\partial E}{\partial r_1}\frac{\mathrm{d}r_1}{\mathrm{d}q} + \frac{\partial E}{\partial r_2}\frac{\mathrm{d}r_2}{\mathrm{d}q}\right)\frac{\mathrm{d}q}{\mathrm{d}u}.\end{aligned}$$

The shared derivative with respect to q can be computed once and reused. This is the essential efficiency advantage of reverse-mode differentiation.

6.3 Output-Layer Backpropagation

The output-layer gradient is simplest because each output-layer weight appears directly in one output pre-activation.

6.3.1 A Single Hidden-to-Output Weight

Fix a hidden unit i and output unit j. Recall that

$$\beta_j = \sum_{q=1}^{H} m_{qj} h_q + b_j.$$

For the particular weight m_{ij}, all hidden activations are held fixed during differentiation with respect to that weight. We obtain

$$\begin{aligned}\frac{\partial \beta_j}{\partial m_{ij}} &= \frac{\partial}{\partial m_{ij}}(m_{1j}h_1 + \cdots + m_{ij}h_i + \cdots + m_{Hj}h_H + b_j) \\ &= h_i.\end{aligned}$$

Defining $\delta_j^{\text{out}} = \partial E / \partial \beta_j$, the chain rule gives

$$\begin{aligned}\frac{\partial E}{\partial m_{ij}} &= \sum_{q=1}^{K} \frac{\partial E}{\partial \beta_q}\frac{\partial \beta_q}{\partial m_{ij}} \\ &= \frac{\partial E}{\partial \beta_j}\frac{\partial \beta_j}{\partial m_{ij}}\end{aligned}$$

$$= h_i \delta_j^{\text{out}}.$$

The intermediate sum is included to emphasize that only β_j depends on m_{ij}.

Definition 6.2 (Output error signal)**.** The vector of derivatives of the loss with respect to the output pre-activations is

$$\delta^{\text{out}} := \nabla_\beta E = \begin{pmatrix} \partial E/\partial \beta_1 & \cdots & \partial E/\partial \beta_K \end{pmatrix}^\top \in \mathbb{R}^K.$$

This is often called the *output error signal*, or the output layer's backpropagated error.

Proposition 6.2 (Output weight and bias derivatives)**.** *For every hidden unit i and output unit j,*

$$\frac{\partial E}{\partial m_{ij}} = h_i \delta_j^{\text{out}},$$
$$\frac{\partial E}{\partial b_j} = \delta_j^{\text{out}}.$$

Proof. The weight derivative was derived above. For the bias, $\partial \beta_q / \partial b_j$ equals 1 when $q = j$ and 0 otherwise. Hence

$$\begin{aligned} \frac{\partial E}{\partial b_j} &= \sum_{q=1}^{K} \frac{\partial E}{\partial \beta_q} \frac{\partial \beta_q}{\partial b_j} \\ &= \frac{\partial E}{\partial \beta_j}. \end{aligned}$$

□

6.3.2 Vectorizing the Output Gradient

Let $M = (m_{ij}) \in \mathbb{R}^{H \times K}$, with rows indexed by hidden units and columns indexed by output units. Then

$$\beta = M^\top h + b, \qquad h \in \mathbb{R}^H, \quad b \in \mathbb{R}^K.$$

The gradient matrix has entries $(\nabla_M E)_{ij} = h_i \delta_j^{\text{out}}$, so

$$\begin{aligned} \nabla_M E &= \begin{pmatrix} h_1 \delta_1^{\text{out}} & \cdots & h_1 \delta_K^{\text{out}} \\ \vdots & \ddots & \vdots \\ h_H \delta_1^{\text{out}} & \cdots & h_H \delta_K^{\text{out}} \end{pmatrix} \\ &= h (\delta^{\text{out}})^\top, \qquad \nabla_b E = \delta^{\text{out}}. \end{aligned}$$

This is an *outer product*: a column vector of hidden activations multiplies a row vector of output errors.

Remark 6.2 (Dimensions are a consistency check). The shapes are

$$\underbrace{\nabla_M E}_{H\times K} = \underbrace{h}_{H\times 1}\underbrace{(\delta^{\text{out}})^\top}_{1\times K}.$$

With the alternative convention $\beta = \widetilde{M}h + b$ for $\widetilde{M} \in \mathbb{R}^{K\times H}$, the same derivative is $(\nabla_{\widetilde{M}} E) = \delta^{\text{out}} h^\top$. Both formulas are correct; their orientation follows the chosen definition of the matrix.

6.3.3 Calculating the Output Error from the Loss

The calculation of δ^{out} starts with the gradient of the loss with respect to the output y and passes backward through the output activation $y = \psi(\beta)$.

Proposition 6.3 (General output activation)**.** *If the output activation $\psi : \mathbb{R}^K \to \mathbb{R}^K$ is differentiable, then*

$$\delta^{\text{out}} = D\psi(\beta)^\top \nabla_y \ell(y, t).$$

When ψ acts coordinatewise using a differentiable scalar activation, this becomes

$$\delta^{\text{out}} = \psi'(\beta) \odot \nabla_y \ell(y, t),$$

where $\odot$ is componentwise multiplication.

Proof. The loss is the composition $E(\beta) = \ell(\psi(\beta), t)$. Applying the multivariable chain rule,

$$\frac{\partial E}{\partial \beta_j} = \sum_{q=1}^{K} \frac{\partial \ell}{\partial y_q}\frac{\partial \psi_q}{\partial \beta_j}.$$

This is precisely the jth coordinate of $D\psi(\beta)^\top \nabla_y \ell$. If ψ_q depends only on β_q, the Jacobian is diagonal with entries $\psi'(\beta_q)$, giving the componentwise formula. □

Example 6.2 (Linear output and squared error)**.** For $y = \beta$ and $E = \frac{1}{2}\sum_{j=1}^{K}(y_j - t_j)^2$, we have

$$\begin{aligned}
\frac{\partial E}{\partial y_j} &= y_j - t_j, \\
\frac{\partial y_j}{\partial \beta_j} &= 1, \\
\delta_j^{\text{out}} &= y_j - t_j.
\end{aligned}$$

Hence $\nabla_M E = h(y - t)^\top$ and $\nabla_b E = y - t$.

Example 6.3 (Sigmoid output and binary cross-entropy)**.** For one binary output, let $y = \sigma(\beta)$ and

$$E = -t \log y - (1 - t)\log(1 - y), \qquad t \in \{0, 1\}.$$

Since $\mathrm{d}y/\mathrm{d}\beta = y(1-y)$,

$$\begin{aligned}\frac{\mathrm{d}E}{\mathrm{d}\beta} &= \left(-\frac{t}{y} + \frac{1-t}{1-y}\right) y(1-y) \\ &= -t(1-y) + (1-t)y \\ &= y - t.\end{aligned}$$

Thus the output error signal equals the prediction residual even though the output activation is nonlinear.

Example 6.4 (Softmax output and categorical cross-entropy)**.** Let $y_j = e^{\beta_j} / \sum_q e^{\beta_q}$, let $t_j \geq 0$ with $\sum_j t_j = 1$, and let

$$E = -\sum_{j=1}^{K} t_j \log y_j.$$

Writing $S = \sum_q e^{\beta_q}$, we have $\log y_j = \beta_j - \log S$ and therefore

$$\begin{aligned}E &= -\sum_{j=1}^{K} t_j \beta_j + \left(\sum_{j=1}^{K} t_j\right) \log S \\ &= -\sum_{j=1}^{K} t_j \beta_j + \log S, \\ \frac{\partial E}{\partial \beta_k} &= -t_k + \frac{e^{\beta_k}}{S} \\ &= y_k - t_k.\end{aligned}$$

The softmax Jacobian is not diagonal; the simplified result comes from differentiating the softmax and cross-entropy combination, not from treating softmax as a coordinatewise activation.

6.4 Backpropagation Through the Hidden Layer

A weight connecting an input to a hidden neuron influences *all* output units connected to that hidden neuron. We must accumulate the gradient contributions from every output branch before continuing backward.

6.4.1 One Input-to-Hidden Weight

Fix input coordinate k and hidden coordinate i. The hidden pre-activation is

$$\alpha_i = \sum_{r=1}^{d} w_{ri} x_r + a_i, \qquad h_i = \phi(\alpha_i).$$

To compute $\partial E / \partial w_{ki}$, apply the chain rule twice:

$$\frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial \alpha_i} \frac{\partial \alpha_i}{\partial w_{ki}} = x_k \frac{\partial E}{\partial \alpha_i},$$

because $\partial\alpha_i/\partial w_{ki} = x_k$. The quantity to determine is $\partial E/\partial\alpha_i$, since it contains the influence of α_i on all K outputs.

By the chain rule through the activation,

$$\begin{aligned}\frac{\partial E}{\partial \alpha_i} &= \frac{\partial E}{\partial h_i}\frac{\partial h_i}{\partial \alpha_i} \\ &= \phi'(\alpha_i)\frac{\partial E}{\partial h_i}.\end{aligned}$$

But every output pre-activation is $\beta_j = \sum_q m_{qj} h_q + b_j$, so

$$\begin{aligned}\frac{\partial E}{\partial h_i} &= \sum_{j=1}^{K} \frac{\partial E}{\partial \beta_j}\frac{\partial \beta_j}{\partial h_i} \\ &= \sum_{j=1}^{K} \delta_j^{\text{out}} m_{ij}.\end{aligned}$$

Combining these equations gives

$$\begin{aligned}\frac{\partial E}{\partial w_{ki}} &= \frac{\partial E}{\partial \alpha_i} x_k \\ &= x_k \phi'(\alpha_i)\frac{\partial E}{\partial h_i} \\ &= x_k \phi'(\alpha_i)\sum_{j=1}^{K} m_{ij}\delta_j^{\text{out}}.\end{aligned}$$

Definition 6.3 (Hidden error signal)**.** For hidden unit i, define

$$\delta_i^{\text{hid}} := \frac{\partial E}{\partial \alpha_i}.$$

Then

$$\begin{aligned}\delta_i^{\text{hid}} &= \phi'(\alpha_i)\sum_{j=1}^{K} m_{ij}\delta_j^{\text{out}}, \\ \frac{\partial E}{\partial w_{ki}} &= x_k \delta_i^{\text{hid}}, \\ \frac{\partial E}{\partial a_i} &= \delta_i^{\text{hid}}.\end{aligned}$$

Remark 6.3 (The key summation). There is a *sum* over output neurons j in the hidden error signal. A hidden activation h_i generally has multiple downstream effects. Computing only one path would omit some of the derivative of the loss. Conversely, computing $\partial E/\partial h_i$ once and then reusing it for each input-to-hidden weight avoids repeating the downstream calculation.

6.4.2 An Explicit Two-Output Derivation

For the network shown in the handwritten lecture, suppose $d = 3$, $H = 4$, and $K = 2$, and consider w_{21}, the weight from x_2 to hidden unit h_1. Let

$$\begin{aligned}
\alpha_1 &= w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + a_1, \\
h_1 &= \phi(\alpha_1), \\
\beta_1 &= m_{11}h_1 + m_{21}h_2 + m_{31}h_3 + m_{41}h_4 + b_1, \\
\beta_2 &= m_{12}h_1 + m_{22}h_2 + m_{32}h_3 + m_{42}h_4 + b_2.
\end{aligned}$$

Differentiating with respect to the particular weight w_{21} yields

$$\begin{aligned}
\frac{\partial E}{\partial w_{21}} &= \frac{\partial E}{\partial \alpha_1}\frac{\partial \alpha_1}{\partial w_{21}} \\
&= x_2 \frac{\partial E}{\partial \alpha_1} \\
&= x_2 \frac{\partial E}{\partial h_1}\frac{\partial h_1}{\partial \alpha_1} \\
&= x_2 \phi'(\alpha_1)\left(\frac{\partial E}{\partial \beta_1}\frac{\partial \beta_1}{\partial h_1} + \frac{\partial E}{\partial \beta_2}\frac{\partial \beta_2}{\partial h_1}\right) \\
&= x_2 \phi'(\alpha_1)\left(m_{11}\frac{\partial E}{\partial \beta_1} + m_{12}\frac{\partial E}{\partial \beta_2}\right) \\
&= x_2 \phi'(\alpha_1)\left(m_{11}\delta_1^{\text{out}} + m_{12}\delta_2^{\text{out}}\right).
\end{aligned}$$

The two terms correspond to the two different routes through which h_1 affects the final loss.

6.4.3 Vectorizing the Hidden-Layer Gradient

Let $W = (w_{ki}) \in \mathbb{R}^{d\times H}$, $a \in \mathbb{R}^H$, and write the entire first affine map as

$$\alpha = W^\top x + a, \qquad h = \phi(\alpha).$$

The derivative with respect to h is

$$\nabla_h E = M\delta^{\text{out}},$$

since the Jacobian of $\beta = M^\top h + b$ is $M^\top$ and its transpose is M. Using the componentwise activation derivative, define

$$\begin{aligned}
\delta^{\text{hid}} &:= \nabla_\alpha E \\
&= \phi'(\alpha) \odot \nabla_h E \\
&= \phi'(\alpha) \odot (M\delta^{\text{out}}).
\end{aligned}$$

The input-to-hidden weight and bias gradients now become

$$\begin{aligned}
\nabla_W E &= x(\delta^{\text{hid}})^\top, \\
\nabla_a E &= \delta^{\text{hid}}.
\end{aligned}$$

Theorem 6.1 (Gradients of a two-layer feedforward network)**.** *Suppose ϕ is differentiable coordinatewise, ψ is differentiable, and ℓ is differentiable with respect to the network output. For*

$$\begin{aligned} \alpha &= W^\top x + a, & h &= \phi(\alpha), \\ \beta &= M^\top h + b, & y &= \psi(\beta), \\ E &= \ell(y, t), \end{aligned}$$

with $W \in \mathbb{R}^{d\times H}$ and $M \in \mathbb{R}^{H\times K}$, let

$$\begin{aligned} \delta^{\text{out}} &= D\psi(\beta)^\top \nabla_y \ell(y, t), \\ \delta^{\text{hid}} &= \phi'(\alpha) \odot (M\delta^{\text{out}}). \end{aligned}$$

Then all gradients are given by

$$\begin{aligned} \nabla_M E &= h(\delta^{\text{out}})^\top, & \nabla_b E &= \delta^{\text{out}}, \\ \nabla_W E &= x(\delta^{\text{hid}})^\top, & \nabla_a E &= \delta^{\text{hid}}. \end{aligned}$$

Proof. The output-layer derivation gives $\partial E/\partial m_{ij} = h_i \delta^{\text{out}}_j$ and $\partial E/\partial b_j = \delta^{\text{out}}_j$. The hidden-layer chain rule gives

$$\frac{\partial E}{\partial \alpha_i} = \phi'(\alpha_i) \sum_{j=1}^{K} m_{ij} \delta^{\text{out}}_j,$$

while $\partial \alpha_i/\partial w_{ki} = x_k$ and $\partial \alpha_i/\partial a_i = 1$. Consequently $\partial E/\partial w_{ki} = x_k \delta^{\text{hid}}_i$ and $\partial E/\partial a_i = \delta^{\text{hid}}_i$. Writing the componentwise identities as outer products proves the result. □

Remark 6.4 (Computational order). The equations should be evaluated in the following order: compute the full forward pass; differentiate the loss at the output; compute δ^{out}; compute δ^{hid}; finally form the weight and bias gradients. No hidden-layer gradient can be evaluated before the downstream output error has been propagated backward.

6.5 Matrix Differentials and the Outer-Product Rule

An alternative derivation avoids coordinatewise sums and clarifies why each layer's weight gradient is an outer product.

Lemma 6.1 (Affine-layer differential identity)**.** *Let $z = Au + b$, where $A \in \mathbb{R}^{m\times n}$, $u \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, and let $E = E(z)$ be a differentiable scalar function. Write $\delta = \nabla_z E \in \mathbb{R}^m$. Then*

$$\begin{aligned} \nabla_A E &= \delta u^\top, \\ \nabla_u E &= A^\top \delta, \\ \nabla_b E &= \delta. \end{aligned}$$

Proof. The differential of the affine map is

$$\mathrm{d}z = (\mathrm{d}A)u + A(\mathrm{d}u) + \mathrm{d}b.$$

The scalar differential of the loss is

$$\begin{aligned}
\mathrm{d}E &= \delta^\top \mathrm{d}z \\
&= \delta^\top (\mathrm{d}A)u + \delta^\top A(\mathrm{d}u) + \delta^\top \mathrm{d}b \\
&= \operatorname{tr}\big(u\delta^\top \mathrm{d}A\big) + (A^\top \delta)^\top \mathrm{d}u + \delta^\top \mathrm{d}b \\
&= \operatorname{tr}\big((\delta u^\top)^\top \mathrm{d}A\big) + (A^\top \delta)^\top \mathrm{d}u + \delta^\top \mathrm{d}b.
\end{aligned}$$

The matrix-gradient convention $\mathrm{d}E = \operatorname{tr}((\nabla_A E)^\top \mathrm{d}A) + (\nabla_u E)^\top \mathrm{d}u + (\nabla_b E)^\top \mathrm{d}b$ identifies the three derivatives. □

Remark 6.5. Using the transposed parameterization $z = A^\top u + b$ changes the first identity to $\nabla_A E = u\delta^\top$ and the input derivative to $\nabla_u E = A\delta$. This is the orientation adopted for the one-hidden-layer handwritten calculations above.

6.6 Backpropagation in an Arbitrarily Deep Network

The same chain rule extends to any finite number of layers. To avoid ambiguity about matrix transposes and layer indices, we now use the standard *column-output* convention.

6.6.1 Forward Propagation and Notation

Let $L \geq 1$ be the number of parameterized layers, with n_0 input coordinates and n_ℓ units in layer ℓ. Define

$$\begin{aligned}
h^{(0)} &= x \in \mathbb{R}^{n_0}, \\
z^{(\ell)} &= W^{(\ell)} h^{(\ell-1)} + b^{(\ell)} \in \mathbb{R}^{n_\ell}, \\
h^{(\ell)} &= \phi_\ell(z^{(\ell)}) \in \mathbb{R}^{n_\ell}, \qquad \ell = 1, \ldots, L, \\
E &= \ell\big(h^{(L)}, t\big).
\end{aligned}$$

Here $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $b^{(\ell)} \in \mathbb{R}^{n_\ell}$. Except where explicitly noted, the activation function ϕ_ℓ acts coordinatewise. The output layer may instead be a non-coordinatewise map such as softmax.

Definition 6.4 (Layerwise backpropagation error)**.** At layer ℓ, the backpropagated error is

$$\delta^{(\ell)} := \nabla_{z^{(\ell)}} E = \begin{pmatrix} \partial E/\partial z_1^{(\ell)} & \cdots & \partial E/\partial z_{n_\ell}^{(\ell)} \end{pmatrix}^\top.$$

This is a vector of derivatives with respect to *pre-activations*, not with respect to the weights themselves.

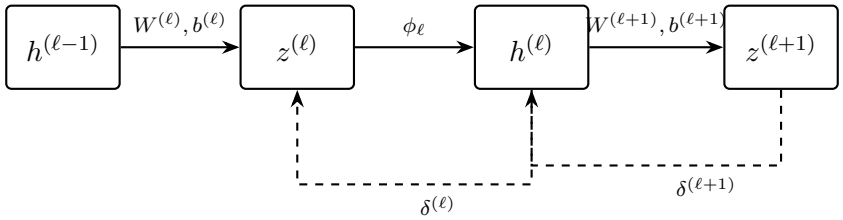


Figure 13: Forward information flows through affine maps and activations; in the backward pass, the downstream error signal is multiplied by a transposed weight matrix and by activation derivatives.

6.6.2 The Backward Recurrence

Theorem 6.2 (General backpropagation recursion)**.** *Assume all relevant activations and the loss are differentiable. For the network defined above, the output error is*

$$\delta^{(L)} = D\phi_L(z^{(L)})^\top \nabla_{h^{(L)}} E.$$

For every hidden layer $\ell = L-1, \dots, 1$, the error satisfies

$$\delta^{(\ell)} = \phi'_\ell(z^{(\ell)}) \odot \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right).$$

The derivatives with respect to parameters at layer ℓ are

$$\begin{aligned} \nabla_{W^{(\ell)}} E &= \delta^{(\ell)} (h^{(\ell-1)})^\top, \\ \nabla_{b^{(\ell)}} E &= \delta^{(\ell)}. \end{aligned}$$

Proof. At the output, $E = \ell(\phi_L(z^{(L)}), t)$. By the chain rule,

$$\delta^{(L)} = D\phi_L(z^{(L)})^\top \nabla_{h^{(L)}} E.$$

Now take $1 \le \ell < L$ and suppose the downstream vector $\delta^{(\ell+1)}$ is already known. The next pre-activation is

$$z^{(\ell+1)} = W^{(\ell+1)} h^{(\ell)} + b^{(\ell+1)}.$$

By the affine chain rule,

$$\begin{aligned} \nabla_{h^{(\ell)}} E &= (W^{(\ell+1)})^\top \nabla_{z^{(\ell+1)}} E \\ &= (W^{(\ell+1)})^\top \delta^{(\ell+1)}. \end{aligned}$$

Since $h^{(\ell)} = \phi_\ell(z^{(\ell)})$, another application of the chain rule gives

$$\begin{aligned} \delta^{(\ell)} &= D\phi_\ell(z^{(\ell)})^\top \nabla_{h^{(\ell)}} E \\ &= D\phi_\ell(z^{(\ell)})^\top (W^{(\ell+1)})^\top \delta^{(\ell+1)}. \end{aligned}$$

When ϕ_ℓ is coordinatewise, the Jacobian is diagonal, with diagonal entries $\phi'_\ell(z_i^{(\ell)})$. Matrix multiplication by that diagonal Jacobian is equivalent to the stated componentwise product.

Finally, the layerwise affine map is $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$. Applying the affine-layer differential identity with $u = h^{(\ell-1)}$ and $\delta = \delta^{(\ell)}$ gives

$$\nabla_{W^{(\ell)}} E = \delta^{(\ell)} (h^{(\ell-1)})^\top,$$
$$\nabla_{b^{(\ell)}} E = \delta^{(\ell)}.$$

The argument applies inductively from layer L back to layer 1. □

6.6.3 Coordinatewise Form and Interpretation

For $i = 1, \ldots, n_\ell$ and $j = 1, \ldots, n_{\ell-1}$, the matrix recurrence is equivalent to

$$\delta_i^{(\ell)} = \phi_\ell'(z_i^{(\ell)}) \sum_{q=1}^{n_{\ell+1}} W_{qi}^{(\ell+1)} \delta_q^{(\ell+1)},$$
$$\frac{\partial E}{\partial W_{ij}^{(\ell)}} = \delta_i^{(\ell)} h_j^{(\ell-1)},$$
$$\frac{\partial E}{\partial b_i^{(\ell)}} = \delta_i^{(\ell)}.$$

Thus each hidden unit collects downstream errors through its outgoing weights, multiplies that quantity by the local activation derivative, and passes it further backward. Each individual weight gradient is the product of a presynaptic activation and a postsynaptic error signal.

Remark 6.6 (Correct indexing). The error at layer ℓ uses the weights of the *following* layer, namely $W^{(\ell+1)}$, not $W^{(\ell)}$. Also, in the present column-output convention the backward matrix is $(W^{(\ell+1)})^\top$. If instead one defines $z^{(\ell)} = (W^{(\ell)})^\top h^{(\ell-1)} + b^{(\ell)}$, the backward factor becomes $W^{(\ell+1)}$, and the layer's weight gradient is $h^{(\ell-1)}(\delta^{(\ell)})^\top$. Both parameterizations describe the same computation; their transposes must not be mixed.

6.7 Backpropagation as Reverse-Mode Automatic Differentiation

The recursion above is a specific instance of reverse-mode automatic differentiation, where a scalar output depends on a possibly very large number of inputs or parameters.

Proposition 6.4 (Reverse accumulation on a computational graph)**.** *Consider a differentiable acyclic computational graph ending in a scalar E. Let a node v feed directly into nodes $u_1, \ldots, u_r$. Once each downstream derivative $\partial E/\partial u_j$ is known,*

$$\frac{\partial E}{\partial v} = \sum_{j=1}^{r} \frac{\partial E}{\partial u_j} \frac{\partial u_j}{\partial v}.$$

Processing nodes in reverse topological order computes the gradient with respect to all graph inputs and parameters.

Proof. The loss depends on v through the collection of its direct downstream children. Applying the multivariable chain rule gives the displayed sum. Because the graph has no directed cycles, all downstream children can be processed before their parents. Repeating the identity therefore propagates derivatives to the graph's input nodes. □

Remark 6.7 (Gradient accumulation versus parameter sharing). When a node has several outgoing edges, its derivative accumulates contributions from all children. If the *same parameter* is used at several different graph locations, its final gradient is likewise the sum of the gradients arising from every occurrence. This occurs in recurrent and convolutional architectures as well as in certain tied-weight feedforward networks.

6.8 A Complete Numerical Example

We illustrate a complete forward and backward pass without hiding any intermediate step. Consider $d = H = K = 2$, hidden ReLU activation, identity output activation, and half-squared-error loss. Set

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad W = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad a = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$
$$M = \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad t = \begin{pmatrix} 3 \\ 2 \end{pmatrix}.$$

All hidden pre-activations in this example are strictly positive, so the ReLU derivatives are unambiguous.

6.8.1 Forward Pass

Using $\alpha = W^\top x + a$,

$$\alpha = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$
$$h = \mathrm{ReLU}(\alpha) = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

The output pre-activation and prediction are

$$\beta = M^\top h + b = \begin{pmatrix} 1 & 2 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \end{pmatrix},$$
$$y = \beta = \begin{pmatrix} 5 \\ 1 \end{pmatrix}.$$

Therefore

$$\begin{aligned} E &= \frac{1}{2}\|y-t\|_2^2 \\ &= \frac{1}{2}\left((5-3)^2+(1-2)^2\right) \\ &= \frac{5}{2}. \end{aligned}$$

6.8.2 Backward Pass Through the Output Layer

The output error for identity activation with squared loss is

$$\delta^{\text{out}} = y - t = \begin{pmatrix} 5 \\ 1 \end{pmatrix} - \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

Thus

$$\begin{aligned} \nabla_M E &= h(\delta^{\text{out}})^\top \\ &= \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} 2 & -1 \end{pmatrix} \\ &= \begin{pmatrix} 2 & -1 \\ 4 & -2 \end{pmatrix}, \\ \nabla_b E &= \begin{pmatrix} 2 \\ -1 \end{pmatrix}. \end{aligned}$$

6.8.3 Backward Pass Through the Hidden Layer

Since $\alpha_1, \alpha_2 > 0$, the ReLU derivatives are both 1. Hence

$$\begin{aligned} M\delta^{\text{out}} &= \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} \\ &= \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \\ \delta^{\text{hid}} &= \text{ReLU}'(\alpha) \odot (M\delta^{\text{out}}) \\ &= \begin{pmatrix} 1 \\ 1 \end{pmatrix} \odot \begin{pmatrix} 3 \\ 3 \end{pmatrix} \\ &= \begin{pmatrix} 3 \\ 3 \end{pmatrix}. \end{aligned}$$

The first-layer gradients are

$$\nabla_W E = x(\delta^{\text{hid}})^\top$$

$$= \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} 3 & 3 \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ 6 & 6 \end{pmatrix},$$

$$\nabla_a E = \begin{pmatrix} 3 \\ 3 \end{pmatrix}.$$

Remark 6.8 (Reading an individual gradient). The entry $\partial E/\partial w_{21} = 6$ can be checked by the path formula. The input coordinate is $x_2 = 2$, the hidden derivative is 1, and the two output contributions are $m_{11}\delta_1^{\text{out}} + m_{12}\delta_2^{\text{out}} = 1(2) + (-1)(-1) = 3$. Their product equals $2 \cdot 1 \cdot 3 = 6$, matching the matrix result.

6.9 Empirical Risk, Mini-Batches, and Gradient Descent

The preceding calculations use a single training example. For a dataset $\{(x^{(d)}, t^{(d)})\}_{d=1}^N$, define the empirical loss

$$\widehat{\mathcal{R}}_N(\theta) = \frac{1}{N} \sum_{d=1}^{N} \ell(f(x^{(d)}; \theta), t^{(d)}).$$

Because this is a finite sum, its gradient is the average of the examplewise gradients:

$$\nabla_\theta \widehat{\mathcal{R}}_N(\theta) = \frac{1}{N} \sum_{d=1}^{N} \nabla_\theta \ell(f(x^{(d)}; \theta), t^{(d)}).$$

Backpropagation is run on each example or, more efficiently in practice, on matrix-valued mini-batches.

Proposition 6.5 (Batch weight gradients)**.** *Consider the column-output layer $Z = WH + B$, with $H \in \mathbb{R}^{n_{\text{in}} \times B}$ containing B activation vectors in its columns and $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$. Suppose the batch objective is the arithmetic mean of individual losses and that $\Delta \in \mathbb{R}^{n_{\text{out}} \times B}$ contains the individual* unaveraged *pre-activation error vectors as columns. Then*

$$\nabla_W E_{\text{batch}} = \frac{1}{B} \Delta H^\top, \qquad \nabla_b E_{\text{batch}} = \frac{1}{B} \Delta \mathbf{1}_B.$$

Proof. For sample d, the layer gradient is $\delta^{(d)}(h^{(d)})^\top$. Averaging these matrices gives

$$\begin{aligned} \nabla_W E_{\text{batch}} &= \frac{1}{B} \sum_{d=1}^{B} \delta^{(d)} (h^{(d)})^\top \\ &= \frac{1}{B} \begin{pmatrix} \delta^{(1)} & \cdots & \delta^{(B)} \end{pmatrix} \begin{pmatrix} (h^{(1)})^\top \\ \vdots \\ (h^{(B)})^\top \end{pmatrix} \\ &= \frac{1}{B} \Delta H^\top. \end{aligned}$$

The bias gradient is the average of the error vectors, giving the second identity. □

Once the parameter gradients have been computed, gradient descent takes the updates

$$W^{(\ell)} \longleftarrow W^{(\ell)} - \eta \nabla_{W^{(\ell)}} E,$$
$$b^{(\ell)} \longleftarrow b^{(\ell)} - \eta \nabla_{b^{(\ell)}} E,$$

for each layer ℓ. Here the superscript (ℓ) indicates the layer, while successive training iterations are indicated by separate iteration counters when necessary. All gradients used in one simultaneous update are evaluated at the *same* current parameter values.

6.10 Computational Cost and the Value of Reusing Errors

For a fully connected layer with $n_{\ell-1}$ inputs and n_ℓ outputs, its affine forward computation requires on the order of $n_\ell n_{\ell-1}$ multiply-add operations. Forming the backward product $(W^{(\ell+1)})^\top \delta^{(\ell+1)}$ and the outer-product parameter gradient has comparable cost.

Proposition 6.6 (Arithmetic complexity of backpropagation)**.** *For a network of L dense layers with widths $n_0, \dots, n_L$, and componentwise activations of constant cost per coordinate, the arithmetic cost of one forward and backward pass for a single example is*

$$O\left(\sum_{\ell=1}^{L} n_\ell n_{\ell-1}\right),$$

assuming the output loss and activation also admit evaluation and differentiation within comparable cost.

Proof. At layer ℓ, computing $W^{(\ell)} h^{(\ell-1)}$ requires $O(n_\ell n_{\ell-1})$ arithmetic operations; applying the activation requires $O(n_\ell)$. During the backward pass, multiplication by $(W^{(\ell)})^\top$ also requires $O(n_\ell n_{\ell-1})$ operations, and constructing $\delta^{(\ell)}(h^{(\ell-1)})^\top$ requires $O(n_\ell n_{\ell-1})$. Summing over layers proves the bound. □

The contrast is with computing the derivative with respect to every weight separately by tracing all of its downstream paths. Reverse accumulation shares intermediate derivatives. This is especially important when a network has many more trainable parameters than output coordinates; training losses are usually scalar.

6.11 Activation Derivatives and Gradient Propagation

The recursion reveals how activation functions and weights influence the magnitude of a gradient as it travels backward through the network. The relevant hidden-layer identity is

$$\delta^{(\ell)} = \phi'_\ell(z^{(\ell)}) \odot (W^{(\ell+1)})^\top \delta^{(\ell+1)}.$$

It contains a local derivative and a linear transformation of downstream error.

Remark 6.9 (Sigmoid and hyperbolic tangent). For the logistic sigmoid and hyperbolic tangent,

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)),$$
$$\tanh'(z) = 1 - \tanh^2(z).$$

The sigmoid derivative is at most $1/4$ in magnitude, and both derivatives approach zero for strongly saturated activations. In sufficiently deep networks, repeated multiplication by small derivatives can make early-layer gradients very small.

Remark 6.10 (ReLU). For $\mathrm{ReLU}(z) = \max\{0, z\}$, the derivative is 1 when $z > 0$ and 0 when $z < 0$. It is not classically differentiable at $z = 0$; optimization software typically adopts a designated subgradient, often 0. An inactive unit passes no local derivative through its incoming weights for that example.

Proposition 6.7 (A norm bound for backward signals)**.** *Suppose the hidden activation in layer ℓ is coordinatewise differentiable and satisfies $\max_i |\phi'_\ell(z_i^{(\ell)})| \le c_\ell$. Then, using the Euclidean norm and induced spectral norm,*

$$\|\delta^{(\ell)}\|_2 \le c_\ell \|W^{(\ell+1)}\|_2 \|\delta^{(\ell+1)}\|_2.$$

Consequently, for $1 \le r < L$,

$$\|\delta^{(r)}\|_2 \le \left(\prod_{\ell=r}^{L-1} c_\ell \|W^{(\ell+1)}\|_2 \right) \|\delta^{(L)}\|_2.$$

Proof. Let $D_\ell = \mathrm{diag}(\phi'_\ell(z^{(\ell)}))$. The backward recurrence is $\delta^{(\ell)} = D_\ell (W^{(\ell+1)})^\top \delta^{(\ell+1)}$. Submultiplicativity gives

$$\begin{aligned} \|\delta^{(\ell)}\|_2 &\le \|D_\ell\|_2 \|(W^{(\ell+1)})^\top\|_2 \|\delta^{(\ell+1)}\|_2 \\ &\le c_\ell \|W^{(\ell+1)}\|_2 \|\delta^{(\ell+1)}\|_2. \end{aligned}$$

Iterating the inequality proves the final bound. □

Remark 6.11 (Vanishing versus exploding gradients). The preceding upper bound helps explain *vanishing gradients* when the product of layerwise factors is small. Large products can permit *exploding gradients*, though the upper bound alone does not guarantee that they occur; cancellations and the directions of the matrices also matter. Initialization, normalization, activation selection, residual connections, and gradient clipping are common practical responses.

6.12 Checking Derivatives and Common Sources of Error

6.12.1 Finite-Difference Gradient Checking

Although backpropagation is an analytic application of the chain rule, a numerical derivative can help detect implementation mistakes. For a scalar parameter θ_j and step size $\varepsilon > 0$, the centered finite difference is

$$D_\varepsilon E(\theta)_j = \frac{E(\theta + \varepsilon e_j) - E(\theta - \varepsilon e_j)}{2\varepsilon},$$

where e_j denotes the jth coordinate vector.

Proposition 6.8 (Error of a centered difference)**.** *Suppose E is three times continuously differentiable in a neighborhood of θ along coordinate j. Then*

$$D_\varepsilon E(\theta)_j = \frac{\partial E}{\partial \theta_j}(\theta) + O(\varepsilon^2) \quad \text{as } \varepsilon \downarrow 0.$$

Proof. Applying Taylor's theorem along coordinate j,

$$E(\theta + \varepsilon e_j) = E(\theta) + \varepsilon \partial_j E(\theta) + \frac{\varepsilon^2}{2}\partial_{jj} E(\theta) + \frac{\varepsilon^3}{6}\partial_{jjj} E(\theta) + o(\varepsilon^3),$$
$$E(\theta - \varepsilon e_j) = E(\theta) - \varepsilon \partial_j E(\theta) + \frac{\varepsilon^2}{2}\partial_{jj} E(\theta) - \frac{\varepsilon^3}{6}\partial_{jjj} E(\theta) + o(\varepsilon^3).$$

Subtracting and dividing by 2ε yields

$$D_\varepsilon E(\theta)_j = \partial_j E(\theta) + \frac{\varepsilon^2}{6}\partial_{jjj} E(\theta) + o(\varepsilon^2).$$

□

In floating-point arithmetic, excessively small ε can cause cancellation and rounding error. Gradient checking is typically performed using a moderate small step and on a limited subset of parameters, not as a replacement for backpropagation in training. At nondifferentiable activation points, an ordinary centered difference need not match the chosen subgradient.

6.12.2 Three Essential Consistency Checks

1. **Shapes.** For $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$, the error $\delta^{(\ell)}$ has length n_ℓ, the weight gradient has shape $n_\ell \times n_{\ell-1}$, and the bias gradient has length n_ℓ.
2. **Shared downstream paths.** Every outgoing edge from a hidden unit contributes to its incoming error. Omitting the sum over outgoing units yields an incomplete derivative.
3. **Loss scaling.** If a loss averages over B examples, the gradient must reflect the same average. Applying the factor $1/B$ twice shrinks the gradient incorrectly; omitting it changes its scale.

7 Lecture 7: Computational Graphs and Automatic Differentiation

7.1 From Backpropagation to Automatic Differentiation

In Lecture 6, we derived the gradient of a multilayer neural network by applying the multivariable chain rule backward through its layers. This calculation is a special case of a more general procedure called *automatic differentiation*. Rather than deriving a new gradient formula for each model, automatic differentiation decomposes the computation into elementary operations, records how intermediate quantities depend on one another, and systematically combines their local derivatives.

The fundamental distinction is between *evaluating* a function and *differentiating* the program that evaluates it. A function may be represented by a complicated expression, but its evaluation can be organized into simple operations such as addition, multiplication, and the application of an activation function. Each operation has an elementary derivative. The derivatives of the complete computation are obtained by composing these local derivatives.

Definition 7.1 (Computational graph)**.** A *computational graph* is a directed graph whose nodes represent variables and elementary operations. An edge from a variable or operation output u to an operation ϕ means that u is used as an input to ϕ. The output of ϕ can in turn be used as an input to later operations. A graph formed by a finite, nonrecursive sequence of computations is a *directed acyclic graph* (DAG).

The graph is directed in the order of computation: inputs are evaluated first, and each operation is evaluated after its inputs have been computed. Derivative information is propagated in the opposite direction when using reverse-mode automatic differentiation.

Remark 7.1 (Connection to neural networks). For a feedforward neural network, the nodes represent such quantities as affine pre-activations, hidden activations, network predictions, and the loss. The layer-by-layer backpropagation formulas from Lecture 6 are exactly the local chain-rule updates associated with these nodes. The same mechanism works for arbitrary differentiable computational graphs, even when their structure is not arranged into uniform neural-network layers.

7.2 Constructing a Graph from an Arithmetic Expression

7.2.1 A Running Example

Consider the scalar function

$$f(x, y) = \sin(x) + xy, \qquad (x, y) \in \mathbb{R}^2.$$

Instead of evaluating the expression in one step, define the intermediate variables

$$a = \sin(x),$$
$$b = xy,$$
$$f = a + b.$$

The computation therefore consists of three elementary operations:

1. The *sine* operation takes x as input and produces a.
2. The *multiplication* operation takes (x, y) as inputs and produces b.
3. The *addition* operation takes (a, b) as inputs and produces f.

Notice that the input x is used twice: it is an argument of both the sine operation and the multiplication operation. Thus the graph has two distinct paths from x to the final output f.

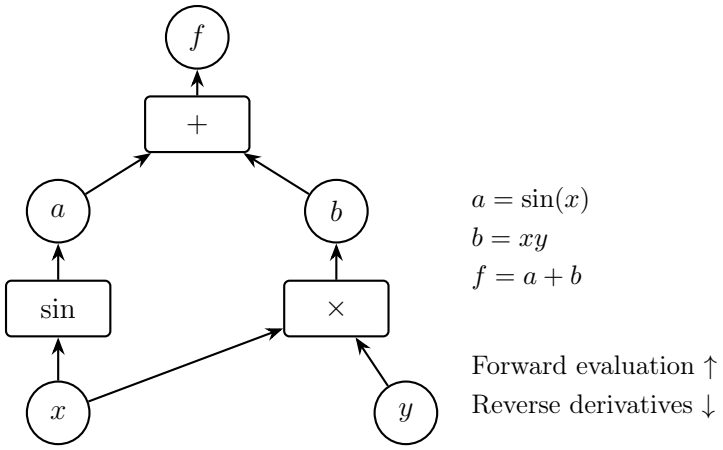


Figure 14: Computational graph for $f(x, y) = \sin(x) + xy$. The input x has two outgoing paths, so its derivative receives contributions from both paths.

7.2.2 Values, Dependencies, and Creators

Definition 7.2 (Variable and operation records)**.** A variable node v may store the following information:

- its numerical value v.val;
- its accumulated derivative v.grad;
- the operation v.creator that produced it, when one exists.

An operation node stores its input arguments, a rule for computing its forward value, and a rule for propagating a derivative backward to each input. A leaf variable such as a supplied input x or a trainable parameter has no creator operation within the current graph.

For the running example, the forward values are obtained in the order

$$\begin{aligned} x.\mathrm{val} &= x, \qquad y.\mathrm{val} = y, \\ a.\mathrm{val} &= \sin(x.\mathrm{val}), \\ b.\mathrm{val} &= x.\mathrm{val}\, y.\mathrm{val}, \\ f.\mathrm{val} &= a.\mathrm{val} + b.\mathrm{val}. \end{aligned}$$

The creator associated with a is the sine operation, the creator of b is multiplication, and the creator of f is addition. The stored dependencies make it possible to revisit the same operations in the reverse direction when calculating gradients.

Example 7.1 (Arithmetic nodes)**.** For the elementary addition operation $c = a + b$, the forward rule and local derivatives are

$$\begin{aligned} c.\mathrm{val} &= a.\mathrm{val} + b.\mathrm{val}, \\ \frac{\partial c}{\partial a} &= 1, \qquad \frac{\partial c}{\partial b} = 1. \end{aligned}$$

For multiplication $c = ab$, they are

$$\begin{aligned} c.\mathrm{val} &= a.\mathrm{val}\, b.\mathrm{val}, \\ \frac{\partial c}{\partial a} &= b.\mathrm{val}, \qquad \frac{\partial c}{\partial b} = a.\mathrm{val}. \end{aligned}$$

Thus an addition operation must store its two dependencies, while a multiplication operation also needs access to their forward values when computing its backward derivatives.

7.3 The Chain Rule and Gradient Propagation

7.3.1 A Chain of Three Functions

The simplest computational graph is a chain. Let

$$h = H(x), \qquad g = G(h), \qquad f = F(g),$$

where $F, G, H : \mathbb{R} \to \mathbb{R}$ are differentiable. The composition is

$$f(x) = F(G(H(x))).$$

Applying the ordinary chain rule gives

$$\begin{aligned} \frac{\mathrm{d}f}{\mathrm{d}x} &= \frac{\mathrm{d}F}{\mathrm{d}g}\frac{\mathrm{d}g}{\mathrm{d}x} \\ &= \frac{\mathrm{d}F}{\mathrm{d}g}\frac{\mathrm{d}G}{\mathrm{d}h}\frac{\mathrm{d}h}{\mathrm{d}x} \\ &= F'(g)G'(h)H'(x). \end{aligned}$$

The product can be computed incrementally in reverse order: first differentiate f with respect to g, then propagate this derivative to h, then to x.

Definition 7.3 (Adjoint or reverse-mode gradient)**.** Suppose a scalar output f depends on an intermediate scalar variable v in the computational graph. The *adjoint* of v is

$$\bar{v} := \frac{\partial f}{\partial v},$$

where the derivative treats v as an independent input to the portion of the graph downstream from that node. The value $\bar{v}$ is the accumulated sensitivity of the final output to a change in v. In code, it is typically stored as v.grad.

For the chain above, the reverse calculation reads

$$\begin{aligned}
\bar{f} &= \frac{\partial f}{\partial f} = 1, \\
\bar{g} &= \bar{f}\frac{\partial f}{\partial g} = F'(g), \\
\bar{h} &= \bar{g}\frac{\partial g}{\partial h} = F'(g)G'(h), \\
\bar{x} &= \bar{h}\frac{\partial h}{\partial x} = F'(g)G'(h)H'(x).
\end{aligned}$$

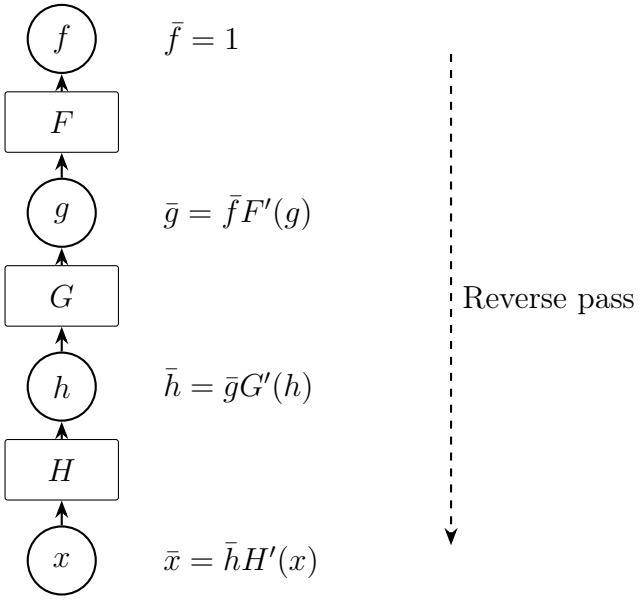


Figure 15: A composition of three differentiable functions. The adjoint starts at the output with value one and is transported backward using local derivatives.

7.3.2 Branching and the Multivariable Chain Rule

In a general computational graph, one variable can influence several later variables. A derivative must then account for every downstream path.

Proposition 7.1 (Chain rule with multiple branches)**.** *Let u be scalar, let $v_1(u), \dots, v_k(u)$ be differentiable, and let $f = F(v_1, \dots, v_k)$ for a differentiable function $F : \mathbb{R}^k \to \mathbb{R}$. Then*

$$\frac{\mathrm{d}f}{\mathrm{d}u} = \sum_{j=1}^{k} \frac{\partial F}{\partial v_j} \frac{\mathrm{d}v_j}{\mathrm{d}u}.$$

Proof. The multivariable differential of F is

$$\mathrm{d}F = \sum_{j=1}^{k} \frac{\partial F}{\partial v_j} \, \mathrm{d}v_j.$$

Each v_j is a differentiable function of u, so

$$\mathrm{d}v_j = \frac{\mathrm{d}v_j}{\mathrm{d}u} \, \mathrm{d}u.$$

Substituting the latter into the former and comparing coefficients of $\mathrm{d}u$ gives

$$\mathrm{d}F = \sum_{j=1}^{k} \frac{\partial F}{\partial v_j} \frac{\mathrm{d}v_j}{\mathrm{d}u} \, \mathrm{d}u,$$

$$\frac{\mathrm{d}f}{\mathrm{d}u} = \sum_{j=1}^{k} \frac{\partial F}{\partial v_j} \frac{\mathrm{d}v_j}{\mathrm{d}u}.$$

□

This is why autodifferentiation implementations use an *accumulation* instruction such as

$$u.\mathrm{grad} \mathrel{+}= \text{contribution},$$

rather than simply overwriting the gradient at every backward operation. A node receiving contributions from several descendants needs their sum.

7.4 Reverse-Mode Automatic Differentiation on a DAG

7.4.1 Local Backward Rules

Suppose an operation has scalar output $z = \phi(u_1, \dots, u_k)$ and the final scalar objective is f. If the accumulated adjoint of the output is $\bar{z} = \partial f / \partial z$, then the operation passes to its ith input the contribution

$$\Delta \bar{u}_i = \bar{z} \frac{\partial \phi}{\partial u_i}.$$

If u_i contributes to additional operations, these contributions must be added to its existing adjoint:

$$\bar{u}_i \mathrel{+}= \bar{z}\frac{\partial \phi}{\partial u_i}.$$

For the three operations appearing in $f(x, y) = \sin(x) + xy$, the backward rules are

$$\begin{aligned} z = u + v: &\quad \bar{u} \mathrel{+}= \bar{z}, && \bar{v} \mathrel{+}= \bar{z}, \\ z = uv: &\quad \bar{u} \mathrel{+}= v\bar{z}, && \bar{v} \mathrel{+}= u\bar{z}, \\ z = \sin(u): &\quad \bar{u} \mathrel{+}= \cos(u)\bar{z}. \end{aligned}$$

Notice that these are rules for the *incoming contribution* to a node, not necessarily the final value of its adjoint.

Definition 7.4 (Topological ordering)**.** A topological ordering of a finite directed acyclic computational graph is an ordering of its nodes such that every node is placed after all of its input dependencies. Evaluating operations in a topological order gives a valid forward pass. Visiting the nodes in the reverse of such an order gives a valid backward pass.

7.4.2 Reverse Accumulation Theorem

Theorem 7.1 (Reverse accumulation for a scalar computational graph)**.** *Consider a finite differentiable DAG with scalar input nodes $x_1, \ldots, x_d$, scalar intermediate nodes, and scalar output f. Each non-input node v has the form*

$$v = \phi_v(u_1, \ldots, u_{k_v}),$$

where u_i are its parent nodes and ϕ_v is differentiable at the computed input values. Initialize all adjoints to zero, set $\bar{f} = 1$, and process the non-input nodes in reverse topological order. For each processed operation $v = \phi_v(u_1, \ldots, u_{k_v})$, update

$$\bar{u}_i \mathrel{+}= \bar{v}\frac{\partial \phi_v}{\partial u_i}, \qquad i = 1, \ldots, k_v.$$

When the procedure terminates, each input adjoint equals the corresponding partial derivative:

$$\bar{x}_j = \frac{\partial f}{\partial x_j}, \qquad j = 1, \ldots, d.$$

If a node is used several times as an argument, each occurrence contributes separately.

Proof. For each node u, let $\mathcal{C}(u)$ denote the collection of its immediate downstream children, with repeated edges counted separately. The multivariable chain rule gives the identity

$$\frac{\partial f}{\partial u} = \sum_{v \in \mathcal{C}(u)} \frac{\partial f}{\partial v}\frac{\partial v}{\partial u},$$

for $u \neq f$, where the derivative of v with respect to u holds its other input arguments fixed. This identity follows by writing the downstream portion of the graph as a function of the immediate children of u and applying the chain rule.

We prove the algorithm is correct by reverse induction along a topological order. At the output node, $\bar{f} = 1 = \partial f / \partial f$. Now suppose that, when node u is reached, all its downstream children have already been processed and their adjoints equal their true derivatives with respect to f. Each such child v has contributed the quantity

$$\bar{v}\frac{\partial v}{\partial u} = \frac{\partial f}{\partial v}\frac{\partial v}{\partial u}$$

to $\bar{u}$. Every child is processed before u because the traversal is reverse topological. Consequently, by the time u is processed,

$$\begin{aligned}\bar{u} &= \sum_{v \in \mathcal{C}(u)} \bar{v}\frac{\partial v}{\partial u} \\ &= \sum_{v \in \mathcal{C}(u)} \frac{\partial f}{\partial v}\frac{\partial v}{\partial u} \\ &= \frac{\partial f}{\partial u}.\end{aligned}$$

The induction continues to all input nodes, establishing the result. When the same node appears in multiple argument positions of an operation, the corresponding chain-rule terms are separate and are both included. □

Remark 7.2 (The role of the reverse traversal). It is not enough to visit arbitrary nodes starting near the output. If a node has multiple children, its adjoint must first receive the contribution from *every* child before that node's local derivative is propagated farther backward. A reverse topological traversal guarantees this ordering.

7.4.3 Pathwise Interpretation

There is another way to view the same result. Let $\mathcal{P}(x_j, f)$ be the set of all directed paths from an input x_j to output f, and let $\partial v / \partial u$ denote the derivative along an edge $u \to v$. Repeated applications of the chain rule yield the path-sum identity

$$\frac{\partial f}{\partial x_j} = \sum_{p \in \mathcal{P}(x_j, f)} \prod_{(u \to v) \in p} \frac{\partial v}{\partial u}.$$

An input used in several independent parts of a program contributes through every path. Reverse-mode autodifferentiation computes this sum efficiently by combining downstream contributions at shared nodes rather than separately enumerating every path.

Example 7.2 (A variable reused within one operation)**.** Let $f(x) = x^2$, represented as a multiplication node $f = x \times x$. The local backward rule supplies two terms to the same input:

$$\bar{x} \mathrel{+}= \bar{f}\frac{\partial(x_1 x_2)}{\partial x_1}\bigg|_{x_1=x_2=x} = \bar{f}\, x,$$
$$\bar{x} \mathrel{+}= \bar{f}\frac{\partial(x_1 x_2)}{\partial x_2}\bigg|_{x_1=x_2=x} = \bar{f}\, x.$$

With $\bar{f} = 1$, the accumulated derivative is $\bar{x} = 2x$. Forgetting either contribution would incorrectly produce only x.

7.5 A Complete Backward Pass for $f(x, y) = \sin(x) + xy$

7.5.1 Forward Evaluation

The computational graph has the intermediate states

$$a = \sin(x),$$
$$b = xy,$$
$$f = a + b.$$

All values are determined by the two supplied inputs x and y. No derivatives need to be computed during this stage, although some forward values are retained for the backward stage.

7.5.2 Initialize the Output Gradient

Because the output is differentiated with respect to itself, reverse mode starts with the seed

$$\bar{f} = \frac{\partial f}{\partial f} = 1.$$

The addition node $f = a + b$ has local partial derivatives

$$\frac{\partial f}{\partial a} = 1, \qquad \frac{\partial f}{\partial b} = 1.$$

Consequently, the two intermediate adjoints become

$$\bar{a} = \bar{f}\frac{\partial f}{\partial a} = (1)(1) = 1,$$
$$\bar{b} = \bar{f}\frac{\partial f}{\partial b} = (1)(1) = 1.$$

7.5.3 Backpropagate Through Sine and Multiplication

The sine node contributes to x because $a = \sin(x)$. Its local derivative is

$$\frac{\partial a}{\partial x} = \cos(x).$$

Hence its contribution to the input derivative is

$$\Delta\bar{x}\big|_a = \bar{a}\frac{\partial a}{\partial x} = (1)\cos(x) = \cos(x).$$

The multiplication node $b = xy$ has two local partial derivatives,

$$\frac{\partial b}{\partial x} = y, \qquad \frac{\partial b}{\partial y} = x.$$

It therefore contributes

$$\Delta\bar{x}\big|_b = \bar{b}\frac{\partial b}{\partial x} = (1)y = y,$$
$$\Delta\bar{y}\big|_b = \bar{b}\frac{\partial b}{\partial y} = (1)x = x.$$

Both paths into x must be accumulated. The complete result is

$$\begin{aligned}\frac{\partial f}{\partial x} &= \Delta\bar{x}\big|_a + \Delta\bar{x}\big|_b \\ &= \cos(x) + y, \\ \frac{\partial f}{\partial y} &= \Delta\bar{y}\big|_b = x.\end{aligned}$$

Thus the gradient of the original function is

$$\nabla f(x, y) = \begin{pmatrix}\cos(x) + y \\ x\end{pmatrix}.$$

This agrees with directly differentiating $f(x, y) = \sin(x) + xy$.

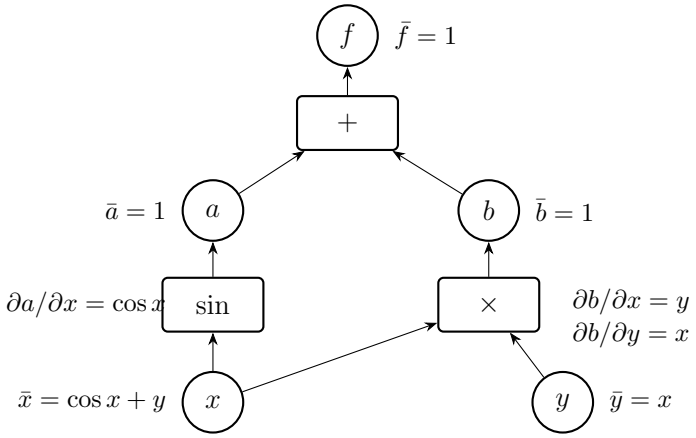


Figure 16: The complete reverse pass. Addition sends an adjoint of one to both branches. Sine and multiplication then send their local derivatives to the inputs; the two contributions to x are added.

Example 7.3 (Numerical forward and backward pass)**.** Let $x = 1$ and $y = 2$. The forward pass computes

$$\begin{aligned} a &= \sin(1) \approx 0.841471, \\ b &= (1)(2) = 2, \\ f &= a + b \approx 2.841471. \end{aligned}$$

The backward pass gives

$$\begin{aligned} \bar{f} &= 1, \\ \bar{a} &= 1, \qquad \bar{b} = 1, \\ \bar{x} &= \cos(1) + 2 \approx 2.540302, \\ \bar{y} &= 1. \end{aligned}$$

The reported sensitivities have the interpretation that a small perturbation $(\Delta x, \Delta y)$ changes the output, to first order, by

$$\Delta f \approx 2.540302\,\Delta x + 1 \cdot \Delta y.$$

7.6 Implementing a Small Automatic Differentiation Engine

7.6.1 Variable Objects and Operation Objects

A simple autodifferentiation library can represent each scalar result by a `Var` object. Its forward value is stored in `val`, its derivative in `grad`, and its dependency information through the operation that created it. For example, addition of two variable objects does not merely return a number: it returns a new variable that remembers which operands produced it.

Remark 7.3 (Creator and argument storage). It is convenient either (i) to use separate variable and operation records, with each non-leaf variable remembering its creator, or (ii) to store the operation and its operand references directly on the resulting variable. Both representations describe the same computational DAG. The example below uses the second representation to make the backward rules concise.

7.6.2 Reference Implementation in Python

The following minimal implementation supports scalar addition, multiplication, sine, graph construction, and reverse-mode gradient accumulation. Its `backward` method creates a topological ordering of dependencies and then invokes local backward rules in reverse order.

```
import math

class Var:
    def __init__(self, val, parents=(), op="leaf"):
        self.val = float(val)
        self.grad = 0.0
        self.parents = tuple(parents)
        self.op = op
        self._backward = lambda: None

    def __add__(self, other):
        other = other if isinstance(other, Var) else Var(other)
        out = Var(self.val + other.val, (self, other), "+")

        def local_backward():
            self.grad += out.grad
            other.grad += out.grad
        out._backward = local_backward
        return out

    __radd__ = __add__

    def __mul__(self, other):
        other = other if isinstance(other, Var) else Var(other)
        out = Var(self.val * other.val, (self, other), "*")

        def local_backward():
            self.grad += other.val * out.grad
            other.grad += self.val * out.grad
        out._backward = local_backward
        return out

    __rmul__ = __mul__

    def sin(self):
        out = Var(math.sin(self.val), (self,), "sin")

        def local_backward():
            self.grad += math.cos(self.val) * out.grad
        out._backward = local_backward
        return out

    def backward(self):
```

```
        topo = []
        visited = set()

        def visit(v):
            if v in visited:
                return
            visited.add(v)
            for p in v.parents:
                visit(p)
            topo.append(v)

        visit(self)
        for v in topo:
            v.grad = 0.0
        self.grad = 1.0
        for v in reversed(topo):
            v._backward()

x = Var(1.0)
y = Var(2.0)
a = x.sin()
b = x * y
f = a + b
f.backward()

print(f.val)  # 2.8414709848...
print(x.grad) # 2.5403023059...
print(y.grad) # 1.0
```

Remark 7.4 (Why `+=` matters). Both `a = x.sin()` and `b = x * y` depend on the same `x` object. The backward rule of `a` adds $\cos(x)$ to `x.grad`, while the backward rule of `b` adds y. Replacing the accumulation operator `+=` with assignment would discard whichever contribution arrived first.

Remark 7.5 (Repeated backward passes). The method above resets the gradients of all nodes in the graph before running each backward pass. This prevents accidental accumulation across separate calls and makes the example reproducible. Some machine-learning libraries instead allow intentional accumulation across multiple backward passes; in that setting one must explicitly zero gradients before beginning a new optimization step. A practical implementation may also free saved intermediate state after a backward pass unless graph retention is requested.

7.7 Local Derivative Rules for Common Operations

A reverse-mode system is built from a relatively small collection of elementary differentiable operations. Their backward behavior follows directly from ordinary calculus.

Proposition 7.2 (Scalar reverse rules)**.** *Let $z = \phi(u, v)$ or $z = \phi(u)$, and suppose $\bar{z}$ has been computed. The following rules give the contributions to the input adjoints at differentiable points:*

$$\begin{aligned}
z = u + v: &\qquad \Delta\bar{u} = \bar{z}, \quad \Delta\bar{v} = \bar{z};\\
z = uv: &\qquad \Delta\bar{u} = v\bar{z}, \quad \Delta\bar{v} = u\bar{z};\\
z = u/v,\ v \neq 0: &\qquad \Delta\bar{u} = \frac{\bar{z}}{v}, \quad \Delta\bar{v} = -\frac{u\bar{z}}{v^2};\\
z = \sin u: &\qquad \Delta\bar{u} = \cos(u)\bar{z};\\
z = \exp u: &\qquad \Delta\bar{u} = e^u\bar{z};\\
z = \log u,\ u > 0: &\qquad \Delta\bar{u} = \frac{\bar{z}}{u};\\
z = \sigma(u): &\qquad \Delta\bar{u} = \sigma(u)(1 - \sigma(u))\bar{z}.
\end{aligned}$$

Here $\sigma(u) = 1/(1 + e^{-u})$ is the logistic sigmoid.

Proof. For each operation, the scalar chain rule gives

$$\frac{\partial f}{\partial u} = \frac{\partial f}{\partial z}\frac{\partial z}{\partial u} = \bar{z}\frac{\partial \phi}{\partial u}.$$

The listed formulas follow by differentiating the corresponding elementary function. For instance, for $z = u/v$,

$$\begin{aligned}
\frac{\partial z}{\partial u} &= \frac{1}{v},\\
\frac{\partial z}{\partial v} &= -\frac{u}{v^2},
\end{aligned}$$

which gives the division rule. For sigmoid, direct differentiation yields

$$\begin{aligned}
\sigma'(u) &= \frac{e^{-u}}{(1 + e^{-u})^2}\\
&= \frac{1}{1 + e^{-u}}\left(1 - \frac{1}{1 + e^{-u}}\right)\\
&= \sigma(u)(1 - \sigma(u)).
\end{aligned}$$

□

Remark 7.6 (Nondifferentiable operations). Operations such as $z = \max\{0, u\}$ are differentiable away from $u = 0$, but not differentiable at the kink. Automatic differentiation systems implement a chosen subgradient convention at such points. The reverse chain rule is mathematically exact for differentiable operations; at nondifferentiable points the value returned depends on the adopted convention.

7.8 A Loss Function Built on the Same Graph

A neural-network calculation usually ends not with a prediction alone but with a scalar loss. To connect the running example to optimization, let

$$s(x,y) = \sin(x) + xy, \qquad E(x,y) = \frac{1}{2}(s(x,y) - t)^2,$$

where $t \in \mathbb{R}$ is a fixed target. Introduce two additional nodes,

$$r = s - t,$$
$$E = \frac{1}{2}r^2.$$

The reverse pass starts from $\bar{E} = 1$ and proceeds through the loss nodes:

$$\bar{r} = \bar{E}\frac{\partial E}{\partial r} = r,$$
$$\bar{s} = \bar{r}\frac{\partial r}{\partial s} = r = s - t.$$

The scalar $\bar{s}$ is the upstream gradient entering the original graph. The derivatives of the prediction were already obtained, namely $\partial s/\partial x = \cos x + y$ and $\partial s/\partial y = x$. Therefore,

$$\begin{aligned}
\frac{\partial E}{\partial x} &= \frac{\partial E}{\partial s}\frac{\partial s}{\partial x} \\
&= (s - t)(\cos x + y) \\
&= (\sin x + xy - t)(\cos x + y), \\
\frac{\partial E}{\partial y} &= \frac{\partial E}{\partial s}\frac{\partial s}{\partial y} \\
&= (s - t)x \\
&= (\sin x + xy - t)x.
\end{aligned}$$

In vector form,

$$\nabla_{(x,y)} E = (s - t)\begin{pmatrix} \cos x + y \\ x \end{pmatrix}.$$

Thus automatic differentiation separates the gradient of the final loss with respect to the prediction from the derivative of the prediction with respect to its inputs or parameters.

Example 7.4 (One scalar gradient-descent update)**.** Treat x as a trainable scalar parameter, hold y and t fixed, and use a learning rate $\eta > 0$. Gradient descent updates

$$\begin{aligned}
x^{(r+1)} &= x^{(r)} - \eta\frac{\partial E}{\partial x}(x^{(r)}, y) \\
&= x^{(r)} - \eta\big(\sin x^{(r)} + x^{(r)}y - t\big)\big(\cos x^{(r)} + y\big).
\end{aligned}$$

The optimizer uses a gradient computed by automatic differentiation, but the reverse traversal does not itself select the learning rate or perform this update.

7.9 Jacobians, Vector Products, and Differentiation Modes

The scalar examples reveal the basic idea, but network computations usually transform vectors and matrices. In this setting, the chain rule is naturally expressed in terms of Jacobians and their products with vectors.

Definition 7.5 (Jacobian matrix)**.** For a differentiable map $F : \mathbb{R}^n \to \mathbb{R}^m$, its Jacobian is the $m \times n$ matrix

$$J_F(x) := DF(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial x_1} & \cdots & \frac{\partial F_m}{\partial x_n} \end{pmatrix}.$$

For a differentiable composition $F \circ G$, the Jacobian chain rule is

$$J_{F\circ G}(x) = J_F(G(x))J_G(x).$$

Definition 7.6 (Jacobian–vector product)**.** For a direction $p \in \mathbb{R}^n$, the *Jacobian–vector product* (JVP) of F at x in direction p is

$$J_F(x)p \in \mathbb{R}^m.$$

It equals the directional derivative $\mathrm{D}F(x)[p]$.

Definition 7.7 (Vector–Jacobian product)**.** For an output cotangent $q \in \mathbb{R}^m$, the *vector–Jacobian product* (VJP), in column-vector gradient convention, is

$$J_F(x)^\top q \in \mathbb{R}^n.$$

It transports a downstream gradient or adjoint of the output back to the input.

Proposition 7.3 (Forward- and reverse-mode chain rules)**.** *Let $h = G(x)$ and $y = F(h)$, with $G : \mathbb{R}^n \to \mathbb{R}^k$ and $F : \mathbb{R}^k \to \mathbb{R}^m$ differentiable. Then:*

1. *For a given input direction $p \in \mathbb{R}^n$, the forward tangent propagates as*

$$\begin{aligned} \dot{h} &= J_G(x)p, \\ \dot{y} &= J_F(h)\dot{h} = J_F(h)J_G(x)p. \end{aligned}$$

2. *For a given output adjoint $q \in \mathbb{R}^m$, the reverse cotangent propagates as*

$$\begin{aligned} \bar{h} &= J_F(h)^\top q, \\ \bar{x} &= J_G(x)^\top \bar{h} = J_G(x)^\top J_F(h)^\top q. \end{aligned}$$

Proof. The Jacobian chain rule gives $J_{F\circ G}(x) = J_F(h)J_G(x)$. Multiplying on the right by p gives the first statement. Taking transposes and multiplying by q gives

$$\begin{aligned} J_{F\circ G}(x)^\top q &= (J_F(h)J_G(x))^\top q \\ &= J_G(x)^\top J_F(h)^\top q, \end{aligned}$$

which is the reverse statement. □

7.9.1 Forward Mode Versus Reverse Mode

Forward mode propagates a perturbation in the *input* direction through the computation. Reverse mode propagates a scalar sensitivity or cotangent from the *output* back to the inputs. Neither method needs to materialize a full Jacobian when only its product with a vector is required.

Remark 7.7 (Why reverse mode is used for training). For a scalar loss $E : \mathbb{R}^p \to \mathbb{R}$ with many parameters, reverse mode computes

$$\nabla_\theta E = J_E(\theta)^\top(1)$$

with one reverse sweep after a forward evaluation, up to constant-factor overhead for each elementary operation. By contrast, straightforward forward mode computes one directional derivative for each chosen input direction; computing all p partial derivatives separately generally requires p such directions. For a map with few inputs but many outputs, the opposite tradeoff may favor forward mode.

7.10 Backpropagation as a Special Case

For a network layer with input $h \in \mathbb{R}^d$, weight matrix $W \in \mathbb{R}^{k\times d}$, bias $b \in \mathbb{R}^k$, and pre-activation

$$z = Wh + b,$$

let E be a scalar loss computed downstream. Define the reverse signal $\bar{z} := \nabla_z E \in \mathbb{R}^k$. We obtain the input, weight, and bias derivatives directly from the differential of the affine operation.

Proposition 7.4 (Backward rule for an affine layer)**.** *For $z = Wh + b$ and scalar differentiable downstream loss E,*

$$\begin{aligned}\nabla_h E &= W^\top \bar{z},\\ \nabla_W E &= \bar{z}h^\top,\\ \nabla_b E &= \bar{z}.\end{aligned}$$

Proof. For simultaneous perturbations $(\mathrm{d}W, \mathrm{d}h, \mathrm{d}b)$,

$$\mathrm{d}z = (\mathrm{d}W)h + W(\mathrm{d}h) + \mathrm{d}b.$$

Using the column-vector gradient convention,

$$\begin{aligned}\mathrm{d}E &= \bar{z}^\top \mathrm{d}z\\ &= \bar{z}^\top(\mathrm{d}W)h + \bar{z}^\top W(\mathrm{d}h) + \bar{z}^\top \mathrm{d}b\\ &= \mathrm{tr}\big((\bar{z}h^\top)^\top \mathrm{d}W\big) + (W^\top\bar{z})^\top \mathrm{d}h + \bar{z}^\top \mathrm{d}b.\end{aligned}$$

The coefficients of the independent differentials identify the three gradients. □

If the layer applies a coordinatewise differentiable nonlinearity $h_{\text{out}} = \phi(z)$, the incoming gradient $\bar{h}_{\text{out}}$ produces

$$\bar{z} = \bar{h}_{\text{out}} \odot \phi'(z).$$

Combining this rule with the affine rule reproduces the layerwise recurrence from Lecture 6:

$$\begin{aligned} \delta^{(\ell)} &= \big(W^{(\ell+1)}\big)^\top \delta^{(\ell+1)} \odot \phi'_\ell(z^{(\ell)}), \\ \nabla_{W^{(\ell)}} E &= \delta^{(\ell)} \big(h^{(\ell-1)}\big)^\top, \\ \nabla_{b^{(\ell)}} E &= \delta^{(\ell)}. \end{aligned}$$

Here the convention is $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$. The previously derived backpropagation formulas are consequently local automatic-differentiation rules composed across a neural-network graph.

7.11 Computational Cost, Memory, and Practical Issues

Proposition 7.5 (Reverse-pass operation count)**.** *Suppose a scalar-output DAG contains M elementary operation instances, each of bounded arity, and each operation's value and local derivatives can be computed with bounded arithmetic cost. A forward evaluation followed by one reverse-mode gradient pass requires $O(M)$ arithmetic operations, excluding any unbounded work hidden inside an elementary operation. It can compute derivatives with respect to all scalar input variables in the graph.*

Proof. In a topological traversal, each operation is evaluated once. With bounded cost per operation, the forward pass uses $O(M)$ operations. In the backward pass, each operation is visited once, and a bounded number of local derivatives is computed and accumulated into its parents. This also uses $O(M)$ operations. Since the input gradients are the accumulated adjoints at the input nodes, no separate reverse traversal is required for each input coordinate. □

Remark 7.8 (Memory requirements). A conventional reverse pass needs access to the computational graph and selected forward values. Saving these values can require memory proportional to the size of the executed graph. Recomputing selected intermediates during the backward pass, often called *checkpointing*, can reduce memory usage at the expense of additional computation.

7.11.1 Gradient Checking

An implementation can be checked against finite differences. For a differentiable scalar function $f : \mathbb{R}^d \to \mathbb{R}$, a centered approximation to its ith partial derivative is

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \varepsilon e_i) - f(x - \varepsilon e_i)}{2\varepsilon},$$

where e_i is the ith coordinate vector and $\varepsilon > 0$ is small. If the third directional derivative is bounded near x, Taylor expansion gives an $O(\varepsilon^2)$ truncation error for the centered difference. Excessively small ε can nevertheless lead to floating-point cancellation error.

For $f(x, y) = \sin(x) + xy$, the centered finite differences are

$$\begin{aligned}
\frac{f(x+\varepsilon, y) - f(x-\varepsilon, y)}{2\varepsilon} &= \frac{\sin(x+\varepsilon) - \sin(x-\varepsilon)}{2\varepsilon} + y \\
&= \cos(x)\frac{\sin\varepsilon}{\varepsilon} + y \longrightarrow \cos(x) + y, \\
\frac{f(x, y+\varepsilon) - f(x, y-\varepsilon)}{2\varepsilon} &= \frac{x(y+\varepsilon) - x(y-\varepsilon)}{2\varepsilon} \\
&= x.
\end{aligned}$$

This independently recovers the two derivatives obtained by the reverse pass.

7.11.2 Common Implementation Pitfalls

1. **Overwriting accumulated gradients.** A variable may feed more than one operation. Each distinct downstream branch must add its contribution.
2. **Traversing in the wrong order.** A node must wait until every dependent child has sent its gradient. Reverse topological ordering ensures correctness.
3. **Losing the graph.** Converting a tracked variable to an ordinary numeric value in the middle of a computation can sever its dependence on the inputs and prevent the derivative from propagating.
4. **Confusing gradient computation with optimization.** The backward pass computes $\nabla_\theta E$, whereas gradient descent or another optimizer uses that gradient to change θ.
5. **Mixing dimensions.** With column gradients, a vector operation $z = F(x)$ passes gradients backward by $J_F(x)^\top \bar{z}$, not by $J_F(x)\bar{z}$.
6. **Forgetting nondifferentiable points.** A numerical library may select subgradients at kinks; such values are conventions rather than ordinary derivatives.

8 Lecture 8: Neural Network Training and Matrix Automatic Differentiation

8.1 From Automatic Differentiation to Optimization

In Lecture 7, we represented a differentiable computation as a directed acyclic graph and obtained derivatives by traversing this graph in reverse. This lecture combines that mechanism with gradient descent to train neural networks. The central distinction is that automatic differentiation computes a gradient, while an optimization algorithm uses that gradient to change the parameters.

Let $\theta = (\theta_1, \ldots, \theta_p) \in \mathbb{R}^p$ denote the trainable parameters of a model, and let $E : \mathbb{R}^p \to \mathbb{R}$ be a differentiable scalar loss. Our objective is to solve

$$\min_{\theta \in \mathbb{R}^p} E(\theta).$$

For a neural network, the coordinates of θ are typically weights and biases; the loss also depends on the data, which are kept fixed when differentiating with respect to θ.

Definition 8.1 (Gradient)**.** The gradient of a differentiable scalar function $E : \mathbb{R}^p \to \mathbb{R}$ is the vector

$$\nabla_\theta E(\theta) = \begin{pmatrix} \dfrac{\partial E}{\partial \theta_1} \\ \vdots \\ \dfrac{\partial E}{\partial \theta_p} \end{pmatrix}.$$

It collects the partial derivatives of the loss with respect to all trainable parameters, evaluated at the current parameter values.

The *gradient descent* iteration, with learning rate $\eta > 0$, is

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta E(\theta^{(k)}).$$

The update is repeated until a stopping criterion is met, such as a maximum number of iterations, sufficiently small gradients, or satisfactory validation performance. In a nonconvex neural-network problem, gradient descent does not in general guarantee finding a global minimizer.

Proposition 8.1 (The negative gradient is a local descent direction)**.** *Suppose E is differentiable at θ and $\nabla E(\theta) \neq 0$. Then the directional derivative of E in direction $d = -\nabla E(\theta)$ is strictly negative.*

Proof. For any direction $d \in \mathbb{R}^p$, differentiability gives

$$E(\theta + td) = E(\theta) + t\nabla E(\theta)^\top d + o(t), \quad \frac{\mathrm{d}}{\mathrm{d}t} E(\theta + td)\Big|_{t=0} = \nabla E(\theta)^\top d.$$

Choosing $d = -\nabla E(\theta)$ yields

$$\nabla E(\theta)^\top d = -\nabla E(\theta)^\top \nabla E(\theta) = -\|\nabla E(\theta)\|_2^2 < 0.$$

Thus sufficiently small positive steps along the negative gradient decrease E locally. □

Remark 8.1 (What automatic differentiation returns). The gradient $\nabla_\theta E$ is a mathematical quantity. A reverse-mode automatic differentiation (AD) system evaluates the loss numerically and computes its partial derivatives by the chain rule. An optimizer subsequently performs the separate assignment $\theta \leftarrow \theta - \eta \nabla_\theta E$. Calling a backward operation alone does not update any parameter.

8.2 Computational Graphs in a Training Step

8.2.1 Forward and Reverse Computation

Consider a scalar loss formed by successive differentiable operations:

$$\theta \longmapsto v_1 \longmapsto v_2 \longmapsto \cdots \longmapsto v_M = E.$$

The arrows represent data dependencies, not necessarily a single chain: one intermediate value may feed several later operations. A computational graph may therefore have branches and merges. During the *forward pass*, each operation computes its output from the values of its parents. During the *backward pass*, the gradient at each node is assembled from derivatives of the downstream operations that use it.

For a scalar-valued output, the *adjoint* of a scalar intermediate variable v is

$$\bar{v} := \frac{\partial E}{\partial v}.$$

It measures how an infinitesimal perturbation of that intermediate value affects the final loss, with the rest of the computational graph fixed at its evaluated values.

Definition 8.2 (Local reverse-mode update)**.** Suppose a graph operation produces $v = \phi(u_1, \ldots, u_r)$, and the upstream adjoint $\bar{v} = \partial E/\partial v$ is known. The operation adds the contributions

$$\bar{u}_i \ \mathrel{+}= \ \bar{v}\frac{\partial \phi}{\partial u_i}, \qquad i = 1, \ldots, r.$$

Here += denotes accumulation rather than replacement. The backward traversal begins by seeding the scalar loss with $\bar{E} = 1$.

If u contributes to the loss along multiple outgoing edges, all contributions must be summed:

$$\bar{u} = \sum_{v \in \mathcal{C}(u)} \bar{v} \frac{\partial v}{\partial u},$$

where $\mathcal{C}(u)$ is the collection of immediate graph operations that consume u. This is precisely the multivariable chain rule applied at a branching node.

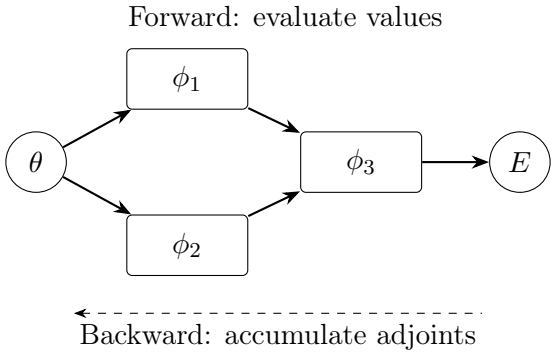


Figure 17: A branching computational graph. Both paths from θ contribute to its final gradient.

Theorem 8.1 (Correctness of reverse accumulation)**.** *Let a finite directed acyclic computational graph consist of differentiable elementary operations and have one scalar loss E. Initialize the adjoint of E to* 1 *and all other adjoints to zero. Process operations in reverse topological order and, for each operation, add its local chain-rule contributions to all parents. At termination, the adjoint stored at every input or parameter node u equals $\partial E/\partial u$.*

Proof. Topologically order the computed variables $v_1, \ldots, v_M = E$ so that every parent of v_j appears before v_j. We proceed by backward induction. For the final output E, $\bar{E} = 1 = \partial E/\partial E$. Suppose that, by the time node v_j is processed in reverse order, all child adjoints equal their derivatives of E. Since all directed paths from v_j to E begin with an edge from v_j to one of its immediate children, the multivariable chain rule gives

$$\begin{aligned} \frac{\partial E}{\partial v_j} &= \sum_{v_k \in \mathcal{C}(v_j)} \frac{\partial E}{\partial v_k} \frac{\partial v_k}{\partial v_j} \\ &= \sum_{v_k \in \mathcal{C}(v_j)} \bar{v}_k \frac{\partial v_k}{\partial v_j}. \end{aligned}$$

Every child v_k is processed before v_j, and its local update adds exactly the displayed summand to $\bar{v}_j$. Therefore the accumulated adjoint is $\partial E/\partial v_j$. Reverse induction reaches every parameter and input node. □

8.2.2 Pseudocode for Training

The two tasks, differentiation and optimization, can now be combined into one algorithm. A framework generally exposes the intermediate operations through a computational graph and stores a gradient buffer for each trainable parameter.

1. Initialize the parameter vector θ and a learning rate $\eta > 0$.
2. For every training iteration, clear the accumulated parameter gradients.
3. Evaluate the network prediction and its scalar loss (forward pass).
4. Traverse the graph backward, beginning with derivative 1 at the loss (backward pass).
5. Apply the gradient update $\theta \leftarrow \theta - \eta \nabla_\theta E$.

Mathematically the steps are

$$\begin{aligned} \widehat{y}^{(k)} &= f_{\theta^{(k)}}(x), \\ E^{(k)} &= \ell\big(\widehat{y}^{(k)}, t\big), \\ g^{(k)} &= \nabla_\theta E^{(k)}, \\ \theta^{(k+1)} &= \theta^{(k)} - \eta g^{(k)}. \end{aligned}$$

The parameters in $f_{\theta^{(k)}}$ are held fixed while evaluating the forward and backward passes for iteration k.

Remark 8.2 (Gradient buffers and accumulation). In many automatic differentiation libraries, repeated calls to a backward routine *add* to existing parameter gradients. Thus the gradient buffer must normally be cleared before each optimizer update, unless accumulation across multiple micro-batches is intentional. In PyTorch, for example, `optimizer.zero_grad()` or `model.zero_grad()` clears parameter gradients; `loss.backward()` computes them. It is not generally correct to call `loss.zero_grad()` for this purpose.

8.3 A Neural Network as a Composition of Elementary Operations

8.3.1 The Four-Operation Representation

The handwritten lecture expresses a simple network as a nested composition

$$E = D\big(C(B(A(x))), t\big).$$

Here A, B, C, and D stand for the following operations:

1. A is an identity or input-preprocessing map that passes x to the network.
2. B combines the input with trainable weights and biases to create a pre-activation.
3. C applies an activation function to form the network prediction.
4. D compares the prediction with a target t to compute a scalar loss.

In the simplest version, let $x \in \mathbb{R}^d$, $w \in \mathbb{R}^d$, $b \in \mathbb{R}$, and $t \in \mathbb{R}$. Define

$$\begin{aligned} a &= A(x) = x, \\ z &= B(a) = w^\top a + b, \\ y &= C(z) = \sigma(z), \\ E &= D(y, t) = \frac{1}{2}(y - t)^2. \end{aligned}$$

The model depends on (w, b) through the full chain

$$(w, b) \longmapsto z \longmapsto y \longmapsto E.$$

Although D is written as a function of y and t, only y changes when differentiating the loss with respect to the network parameters for a fixed labeled example (x, t).

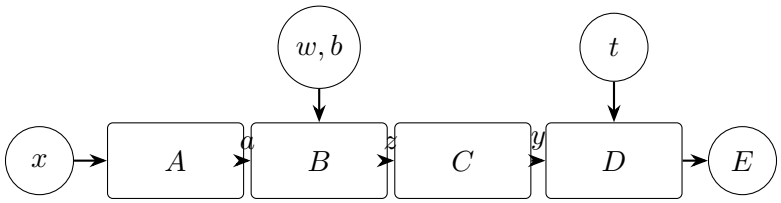


Figure 18: The input, affine transform, activation, and loss as an AD graph. Weights and biases enter at the affine operation.

8.3.2 Complete Scalar Backward Pass

We now compute every derivative for the one-output model. First,

$$\begin{aligned} \frac{\partial E}{\partial y} &= \frac{\partial}{\partial y}\left(\frac{1}{2}(y - t)^2\right) = y - t, \\ \frac{\partial y}{\partial z} &= \sigma'(z), \\ \frac{\partial z}{\partial w_j} &= x_j, \qquad \frac{\partial z}{\partial b} = 1. \end{aligned}$$

Combining these elementary derivatives by the chain rule,

$$\begin{aligned} \frac{\partial E}{\partial z} &= \frac{\partial E}{\partial y}\frac{\partial y}{\partial z} = (y - t)\sigma'(z), \\ \frac{\partial E}{\partial w_j} &= \frac{\partial E}{\partial z}\frac{\partial z}{\partial w_j} = (y - t)\sigma'(z)x_j, \\ \frac{\partial E}{\partial b} &= \frac{\partial E}{\partial z}\frac{\partial z}{\partial b} = (y - t)\sigma'(z). \end{aligned}$$

For the logistic sigmoid,

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)) = y(1 - y).$$

Hence the complete gradient is

$$\nabla_w E = (y - t)y(1 - y)x,$$
$$\frac{\partial E}{\partial b} = (y - t)y(1 - y).$$

Defining the pre-activation error $\delta := \partial E/\partial z$, we can write the result more compactly as

$$\delta = (y - t)\sigma'(z), \qquad \nabla_w E = \delta x, \qquad \frac{\partial E}{\partial b} = \delta.$$

This δ is the scalar backward signal passed through the activation to the affine operation.

Example 8.1 (A numerical training step)**.** Take

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad w = \begin{pmatrix} 0.2 \\ -0.1 \end{pmatrix}, \qquad b = 0.1, \qquad t = 1.$$

The forward computation is

$$z = w^\top x + b = (0.2)(1) + (-0.1)(2) + 0.1 = 0.1,$$
$$y = \sigma(0.1) = \frac{1}{1 + e^{-0.1}} \approx 0.524979,$$
$$E = \frac{1}{2}(y - 1)^2 \approx 0.112823.$$

The reverse computation is

$$\delta = (y - 1)y(1 - y) \approx -0.118459,$$
$$\nabla_w E = \delta x \approx \begin{pmatrix} -0.118459 \\ -0.236918 \end{pmatrix},$$
$$\frac{\partial E}{\partial b} \approx -0.118459.$$

One step of gradient descent with $\eta = 0.1$ produces

$$w_{\text{new}} = w - 0.1\nabla_w E \approx \begin{pmatrix} 0.211846 \\ -0.076308 \end{pmatrix},$$
$$b_{\text{new}} = b - 0.1\frac{\partial E}{\partial b} \approx 0.111846.$$

Because the target is greater than the present output, both the updated bias and the weight associated with the positive first input increase. The second weight becomes less negative, which also increases the pre-activation at this input.

8.4 Vector-Valued Operations and Their Adjoints

The previous example had scalar intermediate values z and y. Modern neural networks generally operate on vectors and matrices. The derivative of a vector-valued mapping is a Jacobian, but the reverse pass need not form that Jacobian explicitly.

Definition 8.3 (Vector–Jacobian product)**.** Let $F : \mathbb{R}^n \to \mathbb{R}^m$ be differentiable and define $y = F(x)$. Let $E : \mathbb{R}^m \to \mathbb{R}$ be a scalar loss and $s = \nabla_y E \in \mathbb{R}^m$. The reverse-mode update at F is

$$\nabla_x (E \circ F)(x) = J_F(x)^\top s,$$

where $J_F(x) \in \mathbb{R}^{m \times n}$ is the Jacobian of F. The mapping $s \mapsto J_F(x)^\top s$ is called a *vector–Jacobian product* (VJP).

Derivation. Writing $F = (F_1, \dots, F_m)$, the multivariable chain rule gives, for each j,

$$\begin{aligned} \frac{\partial (E \circ F)}{\partial x_j} &= \sum_{i=1}^{m} \frac{\partial E}{\partial y_i} \frac{\partial F_i}{\partial x_j} \\ &= \sum_{i=1}^{m} s_i (J_F)_{ij} \\ &= (J_F^\top s)_j. \end{aligned}$$

Collecting all n coordinates proves the formula. □

Remark 8.3 (Why the VJP matters). A full Jacobian may be enormous. To obtain gradients of a scalar loss, reverse-mode AD only requires multiplication of the local Jacobian transpose by the known downstream gradient. Elementary operations implement this VJP directly, often without ever constructing a dense Jacobian.

8.5 Matrix Automatic Differentiation

8.5.1 Notation and Matrix Gradients

Let

$$A \in \mathbb{R}^{m \times k}, \qquad B \in \mathbb{R}^{k \times n}, \qquad Y = AB \in \mathbb{R}^{m \times n}.$$

Suppose $L : \mathbb{R}^{m \times n} \to \mathbb{R}$ is differentiable at Y and define the upstream sensitivity

$$S := \nabla_Y L \in \mathbb{R}^{m \times n}, \qquad S_{ij} := \frac{\partial L}{\partial Y_{ij}}.$$

The gradient with respect to a matrix is defined to have the same shape as that matrix:

$$(\nabla_A L)_{pq} := \frac{\partial L}{\partial A_{pq}}, \qquad (\nabla_B L)_{qr} := \frac{\partial L}{\partial B_{qr}}.$$

We use the Frobenius inner product

$$\langle U, V \rangle_F := \operatorname{tr}(U^\top V) = \sum_{i,j} U_{ij} V_{ij}.$$

The differential of a scalar matrix function is then

$$\mathrm{d}L = \langle \nabla_Y L, \mathrm{d}Y \rangle_F = \operatorname{tr}(S^\top \mathrm{d}Y).$$

Theorem 8.2 (Backward rules for matrix multiplication)**.** *For $Y = AB$ and scalar loss $L(Y)$ as above, the gradients of L with respect to the two factors are*

$$\nabla_A L = SB^\top \in \mathbb{R}^{m \times k},$$
$$\nabla_B L = A^\top S \in \mathbb{R}^{k \times n}.$$

The rules hold for any differentiable scalar loss L; no special loss function is required.

Entrywise proof. By the definition of matrix multiplication,

$$Y_{ij} = \sum_{r=1}^{k} A_{ir} B_{rj}.$$

For fixed indices p, q, differentiating with respect to A_{pq} gives

$$\frac{\partial Y_{ij}}{\partial A_{pq}} = \begin{cases} B_{qj}, & i = p, \\ 0, & i \neq p. \end{cases}$$

The chain rule therefore yields

$$\begin{aligned} \frac{\partial L}{\partial A_{pq}} &= \sum_{i=1}^{m} \sum_{j=1}^{n} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial A_{pq}} \\ &= \sum_{j=1}^{n} S_{pj} B_{qj} \\ &= \sum_{j=1}^{n} S_{pj} (B^\top)_{jq} \\ &= (SB^\top)_{pq}. \end{aligned}$$

Similarly, for fixed indices q, r,

$$\frac{\partial Y_{ij}}{\partial B_{qr}} = \begin{cases} A_{iq}, & j = r, \\ 0, & j \neq r, \end{cases}$$

and hence

$$\begin{aligned}\frac{\partial L}{\partial B_{qr}} &= \sum_{i=1}^{m}\sum_{j=1}^{n} S_{ij}\frac{\partial Y_{ij}}{\partial B_{qr}} \\ &= \sum_{i=1}^{m} A_{iq}S_{ir} \\ &= \sum_{i=1}^{m} (A^\top)_{qi}S_{ir} \\ &= (A^\top S)_{qr}.\end{aligned}$$

Since the identities hold for every matrix entry, they prove both gradient formulas. □

Differential proof. The first-order differential of $Y = AB$ is

$$\mathrm{d}Y = (\mathrm{d}A)B + A(\mathrm{d}B).$$

Substitute this into the differential of the loss:

$$\begin{aligned}\mathrm{d}L &= \operatorname{tr}\big(S^\top \mathrm{d}Y\big) \\ &= \operatorname{tr}\big(S^\top (\mathrm{d}A)B\big) + \operatorname{tr}\big(S^\top A(\mathrm{d}B)\big) \\ &= \operatorname{tr}\big(BS^\top \mathrm{d}A\big) + \operatorname{tr}\big(S^\top A\mathrm{d}B\big) \\ &= \operatorname{tr}\big((SB^\top)^\top \mathrm{d}A\big) + \operatorname{tr}\big((A^\top S)^\top \mathrm{d}B\big) \\ &= \langle SB^\top, \mathrm{d}A\rangle_F + \langle A^\top S, \mathrm{d}B\rangle_F.\end{aligned}$$

The defining property of a matrix gradient identifies the two coefficients of $\mathrm{d}A$ and $\mathrm{d}B$ as $\nabla_A L$ and $\nabla_B L$, respectively. □

Remark 8.4 (Checking matrix dimensions). Always confirm that the backward gradients have the shapes of the inputs:

$$\begin{aligned}SB^\top &: (m \times n)(n \times k) \to (m \times k), \\ A^\top S &: (k \times m)(m \times n) \to (k \times n).\end{aligned}$$

These shape checks also reveal why reversing the order of the factors or forgetting a transpose gives an incorrect update.

8.5.2 A Numerical Matrix Example

Example 8.2 (Gradient of a matrix-product loss)**.** Let

$$A = \begin{pmatrix} 1 & 2 \\ 0 & -1 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 1 \\ -1 & 3 \end{pmatrix}, \qquad T = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

Choose the squared-error loss

$$Y = AB, \qquad L(Y) = \frac{1}{2}\|Y - T\|_F^2.$$

The forward product and residual are

$$Y = \begin{pmatrix} 1(2)+2(-1) & 1(1)+2(3) \\ 0(2)+(-1)(-1) & 0(1)+(-1)(3) \end{pmatrix} = \begin{pmatrix} 0 & 7 \\ 1 & -3 \end{pmatrix},$$
$$S = \nabla_Y L = Y - T = \begin{pmatrix} -1 & 5 \\ 1 & -4 \end{pmatrix}.$$

The backward rules give

$$\nabla_A L = SB^\top = \begin{pmatrix} -1 & 5 \\ 1 & -4 \end{pmatrix}\begin{pmatrix} 2 & -1 \\ 1 & 3 \end{pmatrix} = \begin{pmatrix} 3 & 16 \\ -2 & -13 \end{pmatrix},$$
$$\nabla_B L = A^\top S = \begin{pmatrix} 1 & 0 \\ 2 & -1 \end{pmatrix}\begin{pmatrix} -1 & 5 \\ 1 & -4 \end{pmatrix} = \begin{pmatrix} -1 & 5 \\ -3 & 14 \end{pmatrix}.$$

Both gradients have the same dimensions as the matrices they differentiate.

8.6 Affine Layers, Activation Functions, and Mini-Batches

8.6.1 A Single Affine Layer

A dense neural-network layer can be written, for a column-vector input, as

$$z = Wx + b,$$

with $W \in \mathbb{R}^{r\times d}$, $x \in \mathbb{R}^d$, $b \in \mathbb{R}^r$, and $z \in \mathbb{R}^r$. Let the loss be $E(z)$ and define $\delta := \nabla_z E \in \mathbb{R}^r$. The differential of the layer is

$$\mathrm{d}z = (\mathrm{d}W)x + W(\mathrm{d}x) + \mathrm{d}b.$$

Thus,

$$\begin{aligned} \mathrm{d}E &= \delta^\top \mathrm{d}z \\ &= \delta^\top(\mathrm{d}W)x + \delta^\top W(\mathrm{d}x) + \delta^\top \mathrm{d}b \\ &= \langle \delta x^\top, \mathrm{d}W\rangle_F + (W^\top\delta)^\top \mathrm{d}x + \delta^\top \mathrm{d}b. \end{aligned}$$

Consequently,

$$\begin{aligned} \nabla_W E &= \delta x^\top, \\ \nabla_x E &= W^\top \delta, \\ \nabla_b E &= \delta. \end{aligned}$$

The first formula computes the weight gradient; the second sends the error backward to preceding layers; the third computes the bias gradient.

8.6.2 Elementwise Activation

Suppose $y = \sigma(z)$, where σ acts on each coordinate independently and the upstream gradient is $s = \nabla_y E$. The ith component satisfies

$$\begin{aligned}\frac{\partial E}{\partial z_i} &= \sum_{j=1}^{r} \frac{\partial E}{\partial y_j}\frac{\partial y_j}{\partial z_i} \\ &= \frac{\partial E}{\partial y_i}\sigma'(z_i) = s_i\sigma'(z_i).\end{aligned}$$

Using $\odot$ for elementwise multiplication,

$$\nabla_z E = s \odot \sigma'(z).$$

For a logistic sigmoid, $\sigma'(z) = \sigma(z) \odot (1 - \sigma(z))$, with the arithmetic understood elementwise. Combining an affine transform and activation therefore gives the complete reverse pass

$$\begin{aligned}z &= Wx + b, \\ y &= \sigma(z), \\ s &= \nabla_y E, \\ \delta &= s \odot \sigma'(z), \\ \nabla_W E &= \delta x^\top, \\ \nabla_b E &= \delta, \\ \nabla_x E &= W^\top \delta.\end{aligned}$$

These are the usual backpropagation formulas, derived here as the adjoints of elementary tensor operations.

8.6.3 Vectorization Across a Batch

Suppose we process N examples simultaneously. Let the columns of $X \in \mathbb{R}^{d\times N}$ represent input vectors, and let

$$Z = WX + b\mathbf{1}_N^\top \in \mathbb{R}^{r\times N},$$

where $\mathbf{1}_N \in \mathbb{R}^N$ is the vector of ones. Set $Y = \sigma(Z)$ elementwise and let E be a scalar loss computed from Y and the targets. Write

$$S := \nabla_Y E, \qquad \Delta := S \odot \sigma'(Z).$$

Applying the matrix multiplication rule and accounting for the repeated bias gives

$$\begin{aligned}\nabla_W E &= \Delta X^\top \in \mathbb{R}^{r\times d}, \\ \nabla_X E &= W^\top \Delta \in \mathbb{R}^{d\times N}, \\ \nabla_b E &= \Delta\mathbf{1}_N \in \mathbb{R}^r.\end{aligned}$$

Bias accumulation over the batch. Since $Z_{ij} = \sum_{k=1}^{d} W_{ik} X_{kj} + b_i$, we have

$$\frac{\partial Z_{ij}}{\partial b_p} = \begin{cases} 1, & i = p, \\ 0, & i \neq p. \end{cases}$$

Therefore,

$$\begin{aligned} \frac{\partial E}{\partial b_p} &= \sum_{i=1}^{r} \sum_{j=1}^{N} \frac{\partial E}{\partial Z_{ij}} \frac{\partial Z_{ij}}{\partial b_p} \\ &= \sum_{j=1}^{N} \Delta_{pj} = (\Delta \mathbf{1}_N)_p. \end{aligned}$$

Thus each shared bias accumulates the gradients over the batch. If E is a *mean* loss, its $1/N$ factor is already incorporated in S and hence in Δ. □

Remark 8.5 (Row-major data conventions). Some libraries store a batch as rows, using $X \in \mathbb{R}^{N \times d}$ and $Z = XW + b$ with $W \in \mathbb{R}^{d \times r}$. For this convention, if $\Delta = \nabla_Z E$, the backward rules are

$$\nabla_X E = \Delta W^\top, \qquad \nabla_W E = X^\top \Delta, \qquad \nabla_b E = \Delta^\top \mathbf{1}_N.$$

The mathematics is identical, but transposes change because the storage convention is different.

8.7 Automatic Differentiation in a Training Program

8.7.1 A Minimal PyTorch Example

The following program trains a network containing a single affine map followed by a sigmoid. It demonstrates the forward pass, loss evaluation, gradient reset, backward pass, and parameter update. The notation follows the conceptual sequence of the handwritten lecture.

```
import torch
from torch import nn

# A small batch of inputs and binary targets.
x = torch.tensor([[1.0, 2.0], [2.0, -1.0]])
t = torch.tensor([[1.0], [0.0]])

net = nn.Sequential(
    nn.Linear(2, 1),
    nn.Sigmoid(),
)
loss_fn = nn.MSELoss(reduction="mean")
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
```

```
for epoch in range(100):
    optimizer.zero_grad()   # Clear previously accumulated grads.
    y = net(x)              # Forward pass.
    loss = loss_fn(y, t)    # Scalar training loss.
    loss.backward()         # Reverse-mode differentiation.
    optimizer.step()        # Gradient descent parameter update.
```

The underlying operations are those of the four-stage graph:

$$
\begin{aligned}
&A : x \longmapsto x,\\
&B : x \longmapsto Wx + b,\\
&C : z \longmapsto \sigma(z),\\
&D : (y, t) \longmapsto \frac{1}{2N} \sum_{i=1}^{N} (y_i - t_i)^2.
\end{aligned}
$$

The library loss `nn.MSELoss(reduction="mean")` uses the mean of squared errors *without* the factor 1/2. That convention multiplies the associated analytical gradients by two relative to the displayed mathematical D. Either convention is valid, provided the implemented loss and its derivative are used consistently.

Remark 8.6 (Differentiation versus parameter updates). The operation `loss.backward()` populates the gradient buffers of leaf tensors that require gradients. It does not modify the weights. The optimizer step uses those buffers to change weights, while the subsequent forward pass constructs a new graph with the updated parameters. To avoid accidentally differentiating through parameter updates, optimizers carry out their assignments outside the recorded differentiation graph.

8.7.2 Why Gradients Must Be Reset

Suppose a parameter w has gradient buffer $g = 0$ initially. After differentiating one loss E_1, it contains

$$g \leftarrow 0 + \frac{\partial E_1}{\partial w}.$$

After a second backward call without resetting the buffer,

$$g \leftarrow \frac{\partial E_1}{\partial w} + \frac{\partial E_2}{\partial w}.$$

If this was intentional, then g is the gradient of the combined loss $E_1 + E_2$. Otherwise it is not the gradient of E_2 alone, and updating with it changes the intended optimization algorithm.

Example 8.3 (Gradient accumulation across micro-batches)**.** Suppose a batch is split into M equal-sized micro-batches, each with mean loss E_j. The desired overall mean objective is

$$E_{\text{batch}} = \frac{1}{M} \sum_{j=1}^{M} E_j.$$

By linearity of differentiation,

$$\nabla_\theta E_{\text{batch}} = \sum_{j=1}^{M} \nabla_\theta \left(\frac{E_j}{M} \right).$$

Thus one may reset gradients once, call backward on each E_j/M to accumulate gradients, and only then take one optimizer step. For unequal micro-batch sizes, each mean micro-batch loss must instead be weighted by its fraction of the total number of examples.

8.8 Gradient Descent as a Discretized Dynamical System

8.8.1 Continuous-Time Gradient Flow

Gradient descent can be viewed as the forward Euler discretization of a continuous-time differential equation. Define the *gradient flow* by

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = -\nabla E(\theta(t)).$$

With timestep $\eta > 0$, forward Euler gives

$$\begin{aligned} \frac{\theta(t+\eta) - \theta(t)}{\eta} &\approx -\nabla E(\theta(t)), \\ \theta(t+\eta) &\approx \theta(t) - \eta \nabla E(\theta(t)). \end{aligned}$$

This is precisely the discrete gradient descent update. The learning rate plays the role of an integration timestep: excessively large timesteps can produce instability, even when the continuous dynamics move downhill.

Proposition 8.2 (Energy dissipation along gradient flow)**.** *Suppose E is continuously differentiable and $\theta(t)$ is a differentiable solution of $\dot{\theta} = -\nabla E(\theta)$. Then*

$$\frac{\mathrm{d}}{\mathrm{d}t} E(\theta(t)) = -\|\nabla E(\theta(t))\|_2^2 \leq 0.$$

Proof. By the chain rule,

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t} E(\theta(t)) &= \nabla E(\theta(t))^\top \dot{\theta}(t) \\ &= \nabla E(\theta(t))^\top \big(-\nabla E(\theta(t)) \big) \\ &= -\|\nabla E(\theta(t))\|_2^2. \end{aligned}$$

The loss can therefore increase along a gradient-flow trajectory only if the defining equation is violated; it is constant at times when the gradient vanishes. □

8.8.2 A Sufficient Condition for Descent

The differential equation guarantees monotone loss reduction, but the discrete update needs a restriction on its step size. We derive one standard condition rather than assuming gradient descent always decreases the loss.

Definition 8.4 (Lipschitz-continuous gradient)**.** A differentiable function $E : \mathbb{R}^p \to \mathbb{R}$ has an L-Lipschitz gradient (is L-smooth) if

$$\|\nabla E(u) - \nabla E(v)\|_2 \le L\|u - v\|_2 \qquad \text{for all } u, v \in \mathbb{R}^p,$$

for some $L > 0$.

Lemma 8.1 (Descent inequality)**.** *If E has L-Lipschitz gradient, then for any $u, v \in \mathbb{R}^p$,*

$$E(v) \le E(u) + \nabla E(u)^\top (v - u) + \frac{L}{2}\|v - u\|_2^2.$$

Proof. Set $d = v - u$ and $\phi(s) = E(u + sd)$ for $s \in [0, 1]$. By the fundamental theorem of calculus,

$$\begin{aligned} E(v) - E(u) &= \int_0^1 \phi'(s)\,\mathrm{d}s \\ &= \int_0^1 \nabla E(u + sd)^\top d\,\mathrm{d}s \\ &= \nabla E(u)^\top d + \int_0^1 [\nabla E(u + sd) - \nabla E(u)]^\top d\,\mathrm{d}s. \end{aligned}$$

Using Cauchy–Schwarz and the Lipschitz condition,

$$\begin{aligned} [\nabla E(u + sd) - \nabla E(u)]^\top d &\le \|\nabla E(u + sd) - \nabla E(u)\|_2 \|d\|_2 \\ &\le Ls\|d\|_2^2. \end{aligned}$$

Integrating s from 0 to 1 gives the result. □

Theorem 8.3 (One-step decrease for gradient descent)**.** *Let E have an L-Lipschitz gradient, and set $\theta^+ = \theta - \eta\nabla E(\theta)$. Then*

$$E(\theta^+) \le E(\theta) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla E(\theta)\|_2^2.$$

In particular, for $0 < \eta < 2/L$, each step strictly decreases the loss unless the gradient is zero.

Proof. Apply the preceding lemma with $u = \theta$ and $v = \theta^+ = \theta - \eta g$, where $g = \nabla E(\theta)$:

$$\begin{aligned} E(\theta^+) &\le E(\theta) + g^\top(-\eta g) + \frac{L}{2}\| - \eta g\|_2^2 \\ &= E(\theta) - \eta\|g\|_2^2 + \frac{L\eta^2}{2}\|g\|_2^2 \\ &= E(\theta) - \eta\left(1 - \frac{L\eta}{2}\right)\|g\|_2^2. \end{aligned}$$

When $0 < \eta < 2/L$, the factor multiplying $\|g\|^2$ is positive. □

Corollary 8.1 (Approach toward stationarity)**.** *Under the assumptions of the preceding theorem, assume also that E is bounded below by $E_{\inf} \in \mathbb{R}$, and fix $0 < \eta < 2/L$. The iterates of gradient descent satisfy*

$$\sum_{k=0}^{\infty} \|\nabla E(\theta^{(k)})\|_2^2 < \infty, \qquad \|\nabla E(\theta^{(k)})\|_2 \longrightarrow 0.$$

Proof. Let $c = \eta(1 - L\eta/2) > 0$. Summing the one-step inequalities for $k = 0, \dots, K-1$ yields

$$\begin{aligned} c\sum_{k=0}^{K-1} \|\nabla E(\theta^{(k)})\|_2^2 &\le \sum_{k=0}^{K-1}[E(\theta^{(k)}) - E(\theta^{(k+1)})] \\ &= E(\theta^{(0)}) - E(\theta^{(K)}) \\ &\le E(\theta^{(0)}) - E_{\inf}. \end{aligned}$$

The increasing nonnegative partial sums are bounded, so the series converges and its nonnegative terms tend to zero. □

Remark 8.7. This is a statement about gradient magnitudes, not global optimality. The conclusion remains compatible with convergence toward a local minimum, a saddle point, or other stationary configurations. A practical optimizer may also employ momentum, adaptive steps, stochastic gradients, or weight decay, in which case its exact parameter-update formula differs from plain gradient descent.

8.9 Empirical Risk, Mini-Batches, and Stochastic Gradients

For a dataset $\mathcal{D} = \{(x_i, t_i)\}_{i=1}^N$, define the empirical objective

$$E_N(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f_\theta(x_i), t_i\big).$$

Assuming differentiation can be taken term by term, its gradient is

$$\nabla E_N(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \ell\big(f_\theta(x_i), t_i\big).$$

Computing this sum over every example at each iteration is full-batch gradient descent. For larger datasets, a *mini-batch* $\mathcal{B} \subseteq \{1, \dots, N\}$ gives the approximate gradient

$$g_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell\big(f_\theta(x_i), t_i\big).$$

The stochastic-gradient update is $\theta^+ = \theta - \eta g_{\mathcal{B}}(\theta)$.

Proposition 8.3 (Unbiased sampling gradient)**.** *Fix θ. If indices $I_1, \dots, I_b$ are sampled independently and uniformly from $\{1, \dots, N\}$, then*

$$g_b(\theta) = \frac{1}{b} \sum_{j=1}^{b} \nabla_\theta \ell\big(f_\theta(x_{I_j}), t_{I_j}\big)$$

satisfies $\mathbb{E}[g_b(\theta)] = \nabla E_N(\theta)$, with expectation taken over the sampled indices.

Proof. By linearity of expectation,

$$\begin{aligned}
\mathbb{E}[g_b(\theta)] &= \frac{1}{b} \sum_{j=1}^{b} \mathbb{E}\left[\nabla_\theta \ell(f_\theta(x_{I_j}), t_{I_j})\right] \\
&= \frac{1}{b} \sum_{j=1}^{b} \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \ell(f_\theta(x_i), t_i) \\
&= \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \ell(f_\theta(x_i), t_i) \\
&= \nabla E_N(\theta).
\end{aligned}$$

□

Remark 8.8 (Batch scaling). If the objective is the average loss, gradients should be scaled as derivatives of that average, not the unnormalized sum. Multiplying the loss by a positive constant multiplies its gradient by the same constant and therefore changes the effective step size unless the learning rate is adjusted.

8.10 Verifying Automatically Computed Derivatives

Although a framework implements automatic differentiation, numerical mistakes can still arise from incorrect graph construction, unintended parameter sharing, shape errors, or inconsistent loss scaling. Gradients can be checked against finite differences.

Definition 8.5 (Central finite-difference check)**.** For a scalar loss $E : \mathbb{R}^p \to \mathbb{R}$, a parameter vector θ, a coordinate direction e_j, and $h > 0$, define

$$D_j^{(h)} E(\theta) := \frac{E(\theta + h e_j) - E(\theta - h e_j)}{2h}.$$

If E has a sufficiently smooth third derivative in the jth coordinate, then

$$D_j^{(h)} E(\theta) = \frac{\partial E}{\partial \theta_j}(\theta) + O(h^2) \qquad (h \to 0).$$

Derivation of the error order. Taylor expansion in the direction e_j gives

$$E(\theta + he_j) = E(\theta) + h\partial_j E(\theta) + \frac{h^2}{2}\partial_{jj} E(\theta) + O(h^3),$$
$$E(\theta - he_j) = E(\theta) - h\partial_j E(\theta) + \frac{h^2}{2}\partial_{jj} E(\theta) + O(h^3).$$

Subtracting these expansions and dividing by $2h$ leaves

$$D_j^{(h)} E(\theta) = \partial_j E(\theta) + O(h^2).$$

□

For a vector parameter of moderate size, a convenient relative discrepancy is

$$\frac{\|g_{\mathrm{AD}} - g_{\mathrm{FD}}\|_2}{\max\{1, \|g_{\mathrm{AD}}\|_2, \|g_{\mathrm{FD}}\|_2\}},$$

where g_{AD} is the automatically differentiated gradient and g_{FD} is the finite-difference approximation. Very small h is not always better because floating-point rounding errors become more significant as h decreases.

Remark 8.9 (Non-differentiability and graph details). The usual derivative-based statements apply at points where the elementary operations are differentiable. Operations such as ReLU at zero, maxima at ties, clipping at a boundary, and discrete control-flow choices require separate treatment. AD libraries often use a specified derivative or subgradient convention at such points. The presence of a backward result is not, by itself, evidence that the overall training objective is smooth.

9 Lecture 9–10: Generalization, Regularization, and Optimization of Neural Networks

These two lectures connect the statistical question of *what a neural network learns* to the computational question of *how its parameters are optimized*. Lecture 9 introduces overfitting and regularization, particularly weight decay. Lecture 10 develops stochastic gradient descent, momentum, and the problems of vanishing or exploding gradients in deep networks. Their unifying theme is that a low training loss alone does not guarantee either successful generalization or reliable optimization.

9.1 Lecture 9: Overfitting and Generalization

9.1.1 The Learning Problem and Its Two Risks

Consider a supervised-learning problem with input space $\mathcal{X}$, target space $\mathcal{Y}$, and an unknown data-generating distribution P on $\mathcal{X} \times \mathcal{Y}$. Let

$$D_{\text{tr}} = \{(x_i, y_i)\}_{i=1}^{n}$$

be the training dataset, sampled independently from P. A neural network with parameters $\theta \in \mathbb{R}^p$ produces predictions $f_\theta(x)$. The loss incurred on one input–target pair is $\ell(f_\theta(x), y)$.

Definition 9.1 (Population and empirical risks)**.** The *population risk*, also called expected or generalization risk, is

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P}\big[\ell(f_\theta(X), Y)\big].$$

The *empirical training risk* is the average loss on the observed training set,

$$\widehat{R}_{\text{tr}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Both are functions of the model parameters, but only the empirical risk is directly computable from the finite training sample.

The training process typically produces

$$\widehat{\theta} \in \arg\min_{\theta\in\mathbb{R}^p} \widehat{R}_{\text{tr}}(\theta),$$

or, more generally, a parameter vector with small empirical risk. In contrast, the statistical goal is to achieve small $R(\widehat{\theta})$ on previously unseen examples. The empirical and population objectives need not be minimized by the same parameter vector.

Definition 9.2 (Generalization gap)**.** The *generalization gap* of a fitted model $\widehat{\theta}$ on its training data is

$$\mathrm{Gap}(\widehat{\theta}) = R(\widehat{\theta}) - \widehat{R}_{\mathrm{tr}}(\widehat{\theta}).$$

A large positive gap means the model's training performance is substantially better than its expected performance on new examples.

A model *overfits* when it captures sample-specific details, including noise or incidental patterns, rather than learning only patterns that transfer to new data. An expressive network can often drive training error very low even when its error on new observations remains comparatively high.

Remark 9.1 (A point about expectations). For a fixed parameter vector θ chosen independently of the training sample,

$$\mathbb{E}_{D_{\mathrm{tr}}}\big[\widehat{R}_{\mathrm{tr}}(\theta)\big] = R(\theta).$$

However, $\widehat{\theta}$ is chosen *using* D_{tr}; therefore one cannot replace θ by $\widehat{\theta}$ in the preceding argument without handling that dependence. Fitting parameters to the same observations used for performance assessment can produce optimistic training-error estimates.

Proposition 9.1 (Training-set optimization creates optimism)**.** *Let $\Theta_0 \subseteq \mathbb{R}^p$ be any nonempty parameter set for which the following expectations exist. Then*

$$\mathbb{E}\left[\inf_{\theta\in\Theta_0} \widehat{R}_{\mathrm{tr}}(\theta)\right] \le \inf_{\theta\in\Theta_0} R(\theta).$$

Thus minimizing over parameters before evaluating the training loss tends to make the optimized empirical loss an optimistic estimate of the best achievable population risk within the class.

Proof. For any fixed $\theta_0 \in \Theta_0$,

$$\begin{aligned}
\inf_{\theta\in\Theta_0} \widehat{R}_{\mathrm{tr}}(\theta) &\le \widehat{R}_{\mathrm{tr}}(\theta_0), \\
\mathbb{E}\left[\inf_{\theta\in\Theta_0} \widehat{R}_{\mathrm{tr}}(\theta)\right] &\le \mathbb{E}\big[\widehat{R}_{\mathrm{tr}}(\theta_0)\big] \\
&= R(\theta_0).
\end{aligned}$$

This holds for every $\theta_0 \in \Theta_0$, so taking the infimum over θ_0 gives the claim. □

9.1.2 Training, Validation, and Test Sets

The *training set* is used to compute losses, gradients, and parameter updates. A separate *validation set* is used to select hyperparameters such as regularization strength, network architecture, learning rate, or the number of training epochs. A *test set* is reserved for evaluating the final selected procedure after tuning is complete.

If the validation and test examples are independent of the training data, and $\widehat{\theta}$ is determined solely by the training data, then a fixed-model evaluation on the test set provides an unbiased

estimate of the model's population risk. Once the test set itself is used repeatedly for model selection, this independence no longer holds for the selected model, and the test estimate may become optimistic as well.

For a validation sample D_{val} of size n_{val}, define

$$\widehat{R}_{\text{val}}(\theta) = \frac{1}{n_{\text{val}}} \sum_{i=1}^{n_{\text{val}}} \ell\big(f_\theta(x_i^{\text{val}}), y_i^{\text{val}}\big).$$

A typical procedure is to train candidates indexed by a hyperparameter λ and choose

$$\widehat{\lambda} \in \arg\min_{\lambda \in \Lambda} \widehat{R}_{\text{val}}\big(\widehat{\theta}_\lambda\big).$$

Only after this choice is fixed should the test set be used for a final, independent performance estimate.

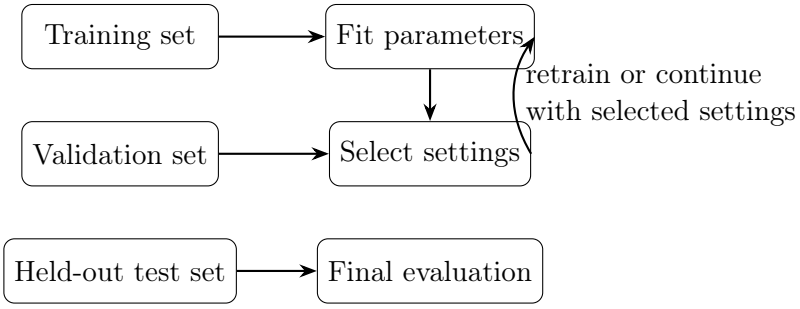


Figure 19: A conceptual data-splitting workflow. The test set is not part of the parameter- or hyperparameter-selection loop.

9.1.3 Recognizing Overfitting

A typical pattern as training progresses is that training loss decreases steadily while validation loss first decreases and subsequently rises. That rise indicates that continued fitting may be worsening performance on unseen observations, even as optimization of the training objective continues.

Remark 9.2 (Optimization versus generalization). A lower training loss demonstrates improved fit to the observed training data, not necessarily better prediction on new data. This distinction is especially important for high-capacity neural networks: even nearly perfect interpolation of the training labels may coexist with very good or very poor generalization, depending on the data distribution, architecture, and training procedure.

9.1.4 Regularization by Weight Decay

The Lecture 9 notes introduce *regularization* as a way to discourage excessively large or complicated fitted parameters. A simple and widely used example is L^2 weight decay.

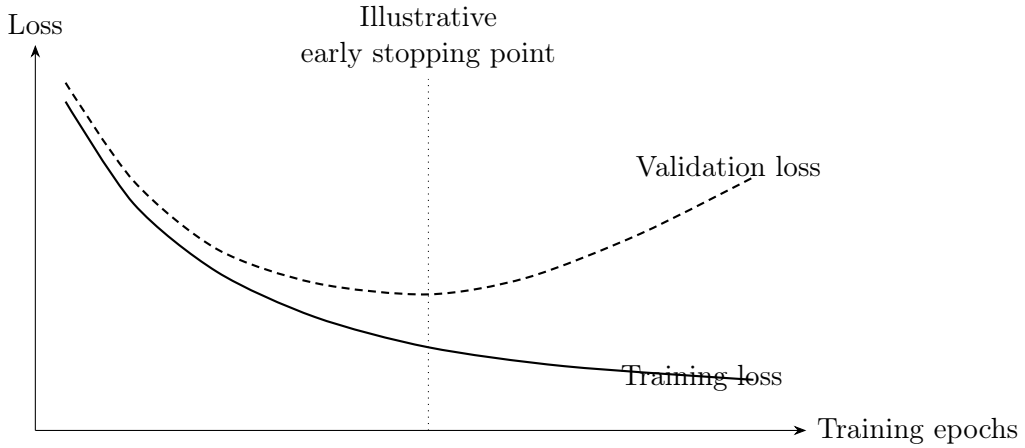


Figure 20: Schematic training and validation curves. The precise shape is problem-dependent; overfitting need not always produce this exact pattern.

Definition 9.3 (L^2-regularized empirical risk)**.** Let θ collect the parameters to be regularized and let $\lambda \geq 0$ be the regularization coefficient. The regularized objective is

$$J_\lambda(\theta) = \widehat{R}_{\mathrm{tr}}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2 = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) + \frac{\lambda}{2}\sum_{j=1}^{p} \theta_j^2.$$

When desired, the penalty may be applied only to a designated collection of weight parameters, with biases excluded.

The factor $1/2$ simplifies differentiation and has no substantive effect because it can be absorbed into λ. The penalty is small for weights near zero and grows quadratically with their magnitudes.

Proposition 9.2 (Gradient of weight decay)**.** *If $\widehat{R}_{\mathrm{tr}}$ is differentiable, then*

$$\nabla_\theta J_\lambda(\theta) = \nabla_\theta \widehat{R}_{\mathrm{tr}}(\theta) + \lambda\theta.$$

Consequently, a gradient-descent step of size $\eta > 0$ is

$$\theta_{t+1} = \theta_t - \eta\nabla_\theta J_\lambda(\theta_t) = (1 - \eta\lambda)\theta_t - \eta\nabla_\theta \widehat{R}_{\mathrm{tr}}(\theta_t).$$

Proof. Write $\theta = (\theta_1, \ldots, \theta_p)$. Componentwise,

$$\begin{aligned}\frac{\partial}{\partial\theta_j}\left(\frac{\lambda}{2}\|\theta\|_2^2\right) &= \frac{\partial}{\partial\theta_j}\left(\frac{\lambda}{2}\sum_{k=1}^{p}\theta_k^2\right) \\ &= \frac{\lambda}{2}\cdot 2\theta_j = \lambda\theta_j.\end{aligned}$$

Collecting these partial derivatives gives $\nabla(\lambda\|\theta\|_2^2/2) = \lambda\theta$. Substituting into the ordinary gradient update yields the final formula. □

The factor $(1-\eta\lambda)$ explains the term *weight decay*: for $0 < \eta\lambda < 1$, the regularizer contracts the old parameter vector toward the origin at every step, in addition to the ordinary gradient update.

Remark 9.3 (Coupled and decoupled weight decay). For plain SGD, adding $\lambda\theta$ to the gradient is algebraically equivalent to multiplying the old parameter by $1-\eta\lambda$ before taking the data-gradient step. With adaptive optimizers such as Adam, adding $\lambda\theta$ to the gradient is generally *not* the same as decoupled weight decay, because adaptive rescaling also acts on the penalty gradient. The distinction matters when interpreting implementation details.

9.1.5 A Closed-Form Illustration: Ridge Regression

Consider linear regression $f_\theta(x) = x^\top\theta$ with design matrix $X \in \mathbb{R}^{n\times p}$ and response vector $y \in \mathbb{R}^n$. With quadratic loss and weight decay, consider

$$J_\lambda(\theta) = \frac{1}{2n}\|X\theta - y\|_2^2 + \frac{\lambda}{2}\|\theta\|_2^2.$$

Expanding the objective yields

$$\begin{aligned} J_\lambda(\theta) &= \frac{1}{2n}(X\theta - y)^\top(X\theta - y) + \frac{\lambda}{2}\theta^\top\theta \\ &= \frac{1}{2n}\theta^\top X^\top X\theta - \frac{1}{n}y^\top X\theta + \frac{1}{2n}y^\top y + \frac{\lambda}{2}\theta^\top\theta. \end{aligned}$$

Differentiating,

$$\begin{aligned} \nabla_\theta J_\lambda(\theta) &= \frac{1}{n}X^\top X\theta - \frac{1}{n}X^\top y + \lambda\theta \\ &= \left(\frac{1}{n}X^\top X + \lambda I_p\right)\theta - \frac{1}{n}X^\top y. \end{aligned}$$

Setting the gradient equal to zero gives

$$\begin{aligned} \left(\frac{1}{n}X^\top X + \lambda I_p\right)\widehat{\theta}_\lambda &= \frac{1}{n}X^\top y, \\ (X^\top X + n\lambda I_p)\widehat{\theta}_\lambda &= X^\top y. \end{aligned}$$

For $\lambda > 0$, the coefficient matrix is positive definite, so the unique minimizer is

$$\widehat{\theta}_\lambda = (X^\top X + n\lambda I_p)^{-1}X^\top y.$$

The added $n\lambda I_p$ improves invertibility even when $X^\top X$ is singular.

Proposition 9.3 (Shrinkage along singular directions)**.** *Suppose $X = U\Sigma V^\top$ is a singular-value decomposition with singular values $s_1, \dots, s_r > 0$. Then*

$$\widehat{\theta}_\lambda = \sum_{j=1}^{r} \frac{s_j}{s_j^2 + n\lambda}\langle u_j, y\rangle v_j.$$

Relative to the unregularized least-squares coefficient $\langle u_j, y\rangle / s_j$, the jth component is multiplied by

$$\frac{s_j^2}{s_j^2 + n\lambda} \in (0, 1).$$

Proof. Using the singular-value decomposition,

$$\begin{aligned}(X^\top X + n\lambda I_p)^{-1} X^\top y &= (V\Sigma^\top \Sigma V^\top + n\lambda I_p)^{-1} V \Sigma^\top U^\top y \\ &= V(\Sigma^\top \Sigma + n\lambda I_p)^{-1} \Sigma^\top U^\top y \\ &= \sum_{j=1}^{r} \frac{s_j}{s_j^2 + n\lambda} \langle u_j, y\rangle v_j.\end{aligned}$$

The shrinkage factor follows by dividing by the corresponding unregularized coefficient. □

The result illustrates why weight decay can reduce sensitivity to directions of parameter space that are poorly supported by the training data. However, shrinking weights does not by itself guarantee improved population performance for every dataset or every regularization strength.

9.1.6 L^1 Penalties and Sparsity

Lecture 9 also contrasts L^2 penalties with L^1 regularization. The L^1-regularized objective is

$$J_{\lambda,1}(\theta) = \widehat{R}_{\mathrm{tr}}(\theta) + \lambda \|\theta\|_1, \qquad \|\theta\|_1 = \sum_{j=1}^{p} |\theta_j|.$$

The absolute-value penalty is not differentiable at zero, so an ordinary gradient formula does not apply everywhere. Its special geometry tends to produce *exactly zero* fitted coefficients in many convex learning problems.

Proposition 9.4 (Soft-thresholding)**.** *For $z \in \mathbb{R}$ and $\alpha > 0$, the unique minimizer of*

$$\min_{w \in \mathbb{R}} \left\{ \frac{1}{2}(w - z)^2 + \alpha |w| \right\}$$

is

$$S_\alpha(z) = \operatorname{sign}(z)(|z| - \alpha)_+ = \begin{cases} z - \alpha, & z > \alpha, \\ 0, & |z| \le \alpha, \\ z + \alpha, & z < -\alpha. \end{cases}$$

Here $u_+ = \max\{u, 0\}$ and $\operatorname{sign}(0) = 0$.

Proof. For $w > 0$, the derivative of the objective is $w - z + \alpha$. Its zero is $w = z - \alpha$, which lies in $(0, \infty)$ only if $z > \alpha$. For $w < 0$, the derivative is $w - z - \alpha$, giving $w = z + \alpha < 0$ if $z < -\alpha$. At $w = 0$, the subdifferential is

$$\partial \left(\frac{1}{2}(w - z)^2 + \alpha |w| \right) \Big|_{w=0} = -z + \alpha[-1, 1].$$

It contains zero precisely when $|z| \le \alpha$. Since the quadratic part is strictly convex, these cases give the unique global minimizer. □

Remark 9.4. L^2 regularization continuously shrinks coefficients but does not typically set them exactly to zero. L^1 regularization can produce sparse parameter vectors because its update includes a threshold at which small coefficients become exactly zero. In general neural networks, sparsity and generalization effects also depend on parameterization and optimizer choice.

9.1.7 Regularization as a Statistical Tradeoff

Increasing regularization generally reduces the effective freedom of a model while increasing the amount of bias imposed by the penalty. Too little regularization may permit excessive fitting of the training sample; too much may cause underfitting. The value of λ therefore needs to be selected on held-out validation data or by another appropriate model-selection procedure.

A further technique is *early stopping*: monitor validation loss while training and retain a parameter checkpoint chosen using validation performance. Early stopping can act as an implicit regularizer, but it is not identical to weight decay, and its effect depends on the optimization dynamics.

9.2 Lecture 10: Stochastic Gradient Descent and Optimization Dynamics

9.2.1 Full-Batch, Stochastic, and Mini-Batch Gradients

Consider the empirical risk

$$F(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta), \qquad \ell_i(\theta) := \ell(f_\theta(x_i), y_i).$$

Assume that every ℓ_i is differentiable. The exact training-set gradient is

$$\nabla F(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i(\theta).$$

Computing this sum on a very large training set can be expensive. Rather than using all observations at each parameter update, one may choose a random batch B_t and compute a gradient estimate.

Definition 9.4 (Mini-batch gradient)**.** For a batch $B_t \subseteq \{1, \dots, n\}$ of size b, the mini-batch gradient is

$$g_t = \frac{1}{b}\sum_{i \in B_t} \nabla \ell_i(\theta_t).$$

The mini-batch SGD update is

$$\theta_{t+1} = \theta_t - \eta_t g_t.$$

With batch size $b = 1$, the update is often called *stochastic gradient descent* in the narrow sense. Full-batch gradient descent corresponds to $b = n$ when the batch contains the entire training set. In practice the term SGD is frequently also used for mini-batch updates.

Proposition 9.5 (Unbiasedness of the mini-batch estimator)**.** *Suppose $I_1, \dots, I_b$ are sampled independently and uniformly from $\{1, \dots, n\}$, and define*

$$g_B(\theta) = \frac{1}{b} \sum_{j=1}^{b} \nabla \ell_{I_j}(\theta).$$

Then, conditional on a fixed parameter vector θ,

$$\mathbb{E}[g_B(\theta)] = \nabla F(\theta).$$

The same conclusion holds for a uniformly selected size-b subset sampled without replacement.

Proof. Under sampling with replacement,

$$\begin{aligned} \mathbb{E}[g_B(\theta)] &= \mathbb{E}\left[\frac{1}{b} \sum_{j=1}^{b} \nabla \ell_{I_j}(\theta)\right] \\ &= \frac{1}{b} \sum_{j=1}^{b} \mathbb{E}\big[\nabla \ell_{I_j}(\theta)\big] \\ &= \frac{1}{b} \sum_{j=1}^{b} \left(\frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta)\right) \\ &= \nabla F(\theta). \end{aligned}$$

For a uniform subset without replacement, each data index is included with probability b/n. Hence

$$\begin{aligned} \mathbb{E}\left[\frac{1}{b} \sum_{i \in B} \nabla \ell_i(\theta)\right] &= \frac{1}{b} \sum_{i=1}^{n} \mathbb{P}(i \in B) \nabla \ell_i(\theta) \\ &= \frac{1}{b} \cdot \frac{b}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta) \\ &= \nabla F(\theta). \end{aligned}$$

□

Proposition 9.6 (Variance decreases with batch size)**.** *With independent uniform sampling with replacement, put*

$$\bar{g} = \nabla F(\theta), \qquad \Sigma(\theta) = \frac{1}{n} \sum_{i=1}^{n} (\nabla \ell_i(\theta) - \bar{g})(\nabla \ell_i(\theta) - \bar{g})^\top.$$

Then

$$\operatorname{Cov}(g_B(\theta)) = \frac{1}{b}\Sigma(\theta), \qquad \mathbb{E}\|g_B(\theta) - \bar{g}\|_2^2 = \frac{1}{b}\operatorname{tr}\Sigma(\theta).$$

Proof. Let $Z_j = \nabla \ell_{I_j}(\theta)$, so that $g_B = b^{-1}\sum_j Z_j$ and $\operatorname{Cov}(Z_j) = \Sigma$. Independence makes all cross-covariances vanish:

$$\begin{aligned}\operatorname{Cov}(g_B) &= \operatorname{Cov}\left(\frac{1}{b}\sum_{j=1}^{b} Z_j\right) \\ &= \frac{1}{b^2}\sum_{j=1}^{b}\operatorname{Cov}(Z_j) \\ &= \frac{1}{b}\Sigma.\end{aligned}$$

The second identity follows from the trace formula for the second moment of a centered random vector. □

Larger batches yield less noisy gradients but generally require more work per update. Smaller batches make each update cheaper and introduce randomness, which can change how the optimizer navigates a nonconvex landscape. Such randomness is not a general guarantee of escaping poor local minima or reaching a global optimum.

Remark 9.5 (Epochs and iterations). An *iteration* is one update of the parameter vector. An *epoch* is approximately one complete pass through the training dataset. If the dataset contains n observations and batches have size b, an epoch involves about $\lceil n/b \rceil$ mini-batch updates. With random sampling with replacement, not every example is necessarily visited once per epoch.

9.2.2 A Descent Estimate for Noisy Gradients

To understand why step size and batch size matter jointly, we first recall a smoothness condition.

Definition 9.5 (L-smoothness)**.** A differentiable function $F : \mathbb{R}^p \to \mathbb{R}$ has an L-Lipschitz gradient if

$$\|\nabla F(u) - \nabla F(v)\|_2 \le L\|u - v\|_2 \quad \text{for all } u, v \in \mathbb{R}^p.$$

Such a function is called L-smooth.

Lemma 9.1 (Descent lemma)**.** *If F is L-smooth, then*

$$F(v) \le F(u) + \nabla F(u)^\top (v - u) + \frac{L}{2}\|v - u\|_2^2.$$

Proof. Set $d = v - u$ and write $\phi(t) = F(u + td)$. By the fundamental theorem of calculus,

$$\begin{aligned} F(v) - F(u) &= \int_0^1 \nabla F(u + td)^\top d \, \mathrm{d}t \\ &= \nabla F(u)^\top d + \int_0^1 \big(\nabla F(u + td) - \nabla F(u)\big)^\top d \, \mathrm{d}t \\ &\le \nabla F(u)^\top d + \int_0^1 \|\nabla F(u + td) - \nabla F(u)\|_2 \|d\|_2 \, \mathrm{d}t \\ &\le \nabla F(u)^\top d + \int_0^1 Lt\|d\|_2^2 \, \mathrm{d}t \\ &= \nabla F(u)^\top d + \frac{L}{2}\|d\|_2^2. \end{aligned}$$

□

Proposition 9.7 (One-step expected progress of SGD)**.** *Suppose F is L-smooth, g_t is conditionally unbiased given θ_t, and*

$$\mathbb{E}\big[\|g_t - \nabla F(\theta_t)\|_2^2 \mid \theta_t\big] \le \frac{\sigma^2}{b}.$$

For the update $\theta_{t+1} = \theta_t - \eta g_t$ with $0 < \eta \le 1/L$,

$$\mathbb{E}[F(\theta_{t+1}) \mid \theta_t] \le F(\theta_t) - \frac{\eta}{2}\|\nabla F(\theta_t)\|_2^2 + \frac{L\eta^2\sigma^2}{2b}.$$

Proof. The descent lemma with $u = \theta_t$ and $v = \theta_t - \eta g_t$ gives

$$F(\theta_{t+1}) \le F(\theta_t) - \eta \nabla F(\theta_t)^\top g_t + \frac{L\eta^2}{2}\|g_t\|_2^2.$$

Taking conditional expectation and using unbiasedness,

$$\begin{aligned} \mathbb{E}[\nabla F(\theta_t)^\top g_t \mid \theta_t] &= \|\nabla F(\theta_t)\|_2^2, \\ \mathbb{E}[\|g_t\|_2^2 \mid \theta_t] &= \|\nabla F(\theta_t)\|_2^2 + \mathbb{E}[\|g_t - \nabla F(\theta_t)\|_2^2 \mid \theta_t] \\ &\le \|\nabla F(\theta_t)\|_2^2 + \frac{\sigma^2}{b}. \end{aligned}$$

Therefore,

$$\begin{aligned} \mathbb{E}[F(\theta_{t+1}) \mid \theta_t] &\le F(\theta_t) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla F(\theta_t)\|_2^2 + \frac{L\eta^2\sigma^2}{2b} \\ &\le F(\theta_t) - \frac{\eta}{2}\|\nabla F(\theta_t)\|_2^2 + \frac{L\eta^2\sigma^2}{2b}, \end{aligned}$$

since $\eta \le 1/L$. □

The first negative term rewards progress in regions with a large gradient. The positive term quantifies how noisy gradients can offset that improvement. The bound predicts that choosing a smaller learning rate or a larger batch size reduces the effect of gradient noise.

9.2.3 Momentum: Accumulating Past Gradients

Ordinary gradient descent uses only the current gradient. In narrow valleys, this may cause back-and-forth oscillation in steep directions while progress along shallow directions remains slow. *Momentum* introduces a state variable that remembers previous updates and combines them across iterations.

Definition 9.6 (Classical momentum)**.** Choose a learning rate $\eta > 0$ and a momentum coefficient $\beta \in [0, 1)$. Define the velocity v_t and parameter updates by

$$\begin{aligned} v_{t+1} &= \beta v_t - \eta g_t, \\ \theta_{t+1} &= \theta_t + v_{t+1}, \end{aligned}$$

with $v_0 = 0$. Equivalently, with $m_{t+1} = -v_{t+1}/\eta$,

$$\begin{aligned} m_{t+1} &= \beta m_t + g_t, \\ \theta_{t+1} &= \theta_t - \eta m_{t+1}. \end{aligned}$$

The signs depend on whether the auxiliary state is described as a velocity pointing toward descent or as an accumulated gradient pointing toward ascent. Both conventions implement the same algorithm when used consistently.

Proposition 9.8 (Momentum as a geometric sum)**.** *If $m_0 = 0$, then for every $t \geq 0$,*

$$m_{t+1} = \sum_{k=0}^{t} \beta^{t-k} g_k.$$

Proof. For $t = 0$, the recursion gives $m_1 = g_0$, which agrees with the sum. Suppose the expression holds for m_t. Then

$$\begin{aligned} m_{t+1} &= \beta m_t + g_t \\ &= \beta \sum_{k=0}^{t-1} \beta^{t-1-k} g_k + g_t \\ &= \sum_{k=0}^{t-1} \beta^{t-k} g_k + \beta^0 g_t \\ &= \sum_{k=0}^{t} \beta^{t-k} g_k. \end{aligned}$$

The result follows by induction. □

Thus recent gradients receive greater weight than older ones, and a succession of similarly directed gradients builds a larger effective step. Gradients that alternate in sign along one direction tend to cancel, reducing oscillation. If $g_t \equiv g$ is constant, then

$$\begin{aligned} m_{t+1} &= \sum_{k=0}^{t} \beta^{t-k} g \\ &= \sum_{j=0}^{t} \beta^{j} g \\ &= \frac{1-\beta^{t+1}}{1-\beta} g \xrightarrow[t\to\infty]{} \frac{g}{1-\beta}. \end{aligned}$$

This illustrates momentum's accumulation of a persistent descent direction. It is not a statement that the gradient of a changing objective remains constant along the optimization trajectory.

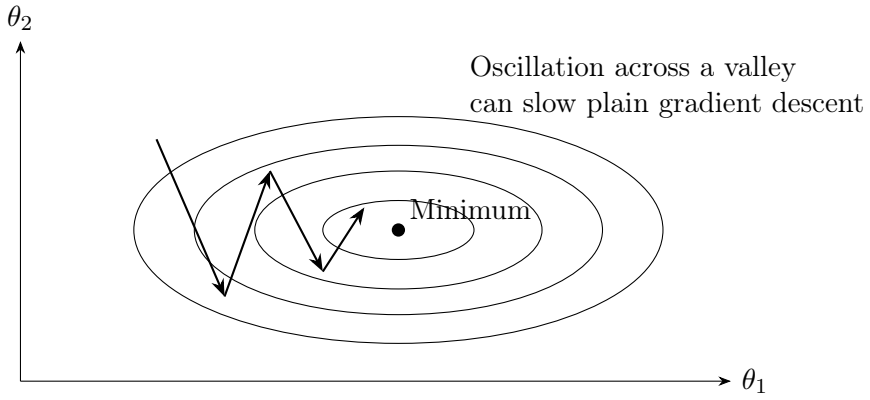


Figure 21: A schematic narrow-valley optimization landscape. Momentum can suppress alternating update directions while preserving progress along a consistent direction.

9.2.4 A Quadratic Model for Momentum

A one-dimensional quadratic makes momentum dynamics explicit. Let

$$F(\theta) = \frac{a}{2}\theta^2, \qquad a > 0,$$

so $\nabla F(\theta) = a\theta$. Classical momentum gives

$$\begin{aligned} \theta_{t+1} &= \theta_t + v_{t+1}, \\ v_{t+1} &= \beta v_t - \eta a \theta_t. \end{aligned}$$

For $t \geq 1$, note that $v_t = \theta_t - \theta_{t-1}$. Consequently,

$$\begin{aligned} \theta_{t+1} &= \theta_t + \beta(\theta_t - \theta_{t-1}) - \eta a \theta_t \\ &= (1 + \beta - \eta a)\theta_t - \beta \theta_{t-1}. \end{aligned}$$

Seeking solutions of the form $\theta_t = r^t$ yields the characteristic equation

$$r^2 - (1 + \beta - \eta a)r + \beta = 0.$$

Its roots are

$$r_{\pm} = \frac{1 + \beta - \eta a \pm \sqrt{(1 + \beta - \eta a)^2 - 4\beta}}{2}.$$

Convergence of this linear recurrence requires both roots to lie strictly inside the unit circle. Thus a large learning rate can make momentum unstable; momentum is an acceleration mechanism, not a guarantee of convergence for arbitrary settings.

9.3 Deep Networks and Gradient Propagation

9.3.1 The Backpropagation Chain

Let a depth-L feedforward network be defined by

$$\begin{aligned} h^{(0)} &= x, \\ z^{(\ell)} &= W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \\ h^{(\ell)} &= \sigma_\ell(z^{(\ell)}), \qquad \ell = 1, \dots, L. \end{aligned}$$

The activations act componentwise unless otherwise specified. For a scalar loss $E = \ell(h^{(L)}, y)$, the gradient with respect to early layers is obtained by repeated application of the chain rule.

Define the layer Jacobian

$$J_\ell := \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} = D_\ell W^{(\ell)}, \qquad D_\ell := \operatorname{diag}\big(\sigma_\ell'(z^{(\ell)})\big).$$

Then

$$\begin{aligned} \nabla_{h^{(L-1)}} E &= J_L^\top \nabla_{h^{(L)}} E, \\ \nabla_{h^{(L-2)}} E &= J_{L-1}^\top J_L^\top \nabla_{h^{(L)}} E, \\ &\vdots \\ \nabla_{h^{(0)}} E &= J_1^\top J_2^\top \cdots J_L^\top \nabla_{h^{(L)}} E. \end{aligned}$$

Thus the backward signal is transformed by a product of Jacobian matrices. The size of this product is central to understanding vanishing and exploding gradients.

9.3.2 The Vanishing-Gradient Problem

The logistic sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

with derivative

$$\begin{aligned}\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \sigma(z)(1-\sigma(z)).\end{aligned}$$

Because $0 < \sigma(z) < 1$, the derivative is strictly positive. Moreover,

$$\begin{aligned}\sigma(z)(1-\sigma(z)) &= \frac{1}{4} - \left(\sigma(z) - \frac{1}{2}\right)^2 \\ &\leq \frac{1}{4}.\end{aligned}$$

Hence

$$0 < \sigma'(z) \leq \frac{1}{4},$$

with equality at $z = 0$. At very positive or very negative inputs, the sigmoid saturates and $\sigma'(z)$ approaches zero.

For the illustrative preactivation $z = 5$,

$$\begin{aligned}\sigma(5) &= \frac{1}{1+e^{-5}} \approx 0.993307, \\ \sigma'(5) &= \sigma(5)(1-\sigma(5)) \approx 0.006648.\end{aligned}$$

Near the origin, for example at $z = 0.1$,

$$\begin{aligned}\sigma(0.1) &\approx 0.524979, \\ \sigma'(0.1) &\approx 0.249376.\end{aligned}$$

Thus the first derivative is about 37.5 times smaller than the second. This is the numerical phenomenon emphasized in the handwritten lecture.

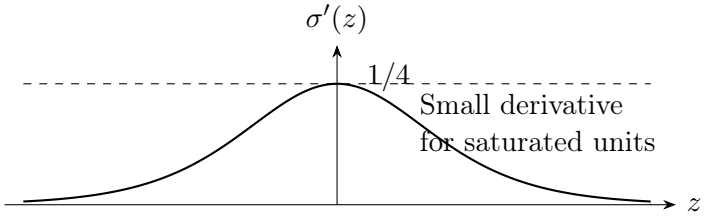


Figure 22: The sigmoid derivative is maximized at zero and approaches zero in the saturated tails.

Proposition 9.9 (Jacobian-product bound)**.** *For the Euclidean norm on vectors and its induced spectral norm on matrices, the propagated input gradient satisfies*

$$\|\nabla_{h^{(0)}} E\| \leq \left(\prod_{\ell=1}^{L} \|J_\ell\|\right) \|\nabla_{h^{(L)}} E\|.$$

If $\|J_\ell\| \le \rho < 1$ *for every layer, then*

$$\|\nabla_{h^{(0)}} E\| \le \rho^L \|\nabla_{h^{(L)}} E\|.$$

Proof. Repeated backpropagation gives

$$\nabla_{h^{(0)}} E = J_1^\top \cdots J_L^\top \nabla_{h^{(L)}} E.$$

Submultiplicativity of the spectral norm yields

$$\begin{aligned}\|\nabla_{h^{(0)}} E\| &\le \|J_1^\top\| \cdots \|J_L^\top\| \|\nabla_{h^{(L)}} E\| \\ &= \left(\prod_{\ell=1}^{L} \|J_\ell\|\right) \|\nabla_{h^{(L)}} E\|,\end{aligned}$$

since the spectral norm is invariant under transposition. If each factor is at most ρ, their product is at most ρ^L. □

Remark 9.6 (A scalar example). If a signal passes through four sigmoid derivatives, each at its maximal value $1/4$, and the intervening scalar weights are 1, then the chain-rule multiplier is

$$\left(\frac{1}{4}\right)^4 = \frac{1}{256} \approx 0.00390625.$$

At ten such layers it is $4^{-10} \approx 9.54 \times 10^{-7}$. This calculation isolates the role of activation derivatives. In a real neural network, the weight matrices can either amplify or reduce the backward signal, so sigmoid depth alone does not completely determine gradient magnitude.

9.3.3 Residual and Skip Connections

One response to vanishing gradients is to introduce a direct path around one or more nonlinear transformations. A *skip connection* carries information from an earlier layer into a later layer without forcing it through every intermediate transformation.

Definition 9.7 (Residual block)**.** A residual block takes an input h and produces

$$\widetilde{h} = h + F(h; W),$$

where F is a parameterized transformation and the input and output dimensions agree. In an index notation,

$$h^{(\ell+1)} = h^{(\ell)} + F_\ell(h^{(\ell)}; W^{(\ell)}).$$

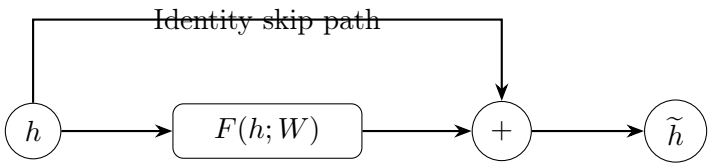


Figure 23: A residual connection adds the block input to the learned transformation output.

Proposition 9.10 (Backpropagation through a residual block)**.** *Let $\widetilde{h} = h + F(h; W)$ and let a differentiable loss E depend on $\widetilde{h}$. Then*

$$\nabla_h E = \big(I + J_F(h; W)\big)^\top \nabla_{\widetilde{h}} E = \nabla_{\widetilde{h}} E + J_F(h; W)^\top \nabla_{\widetilde{h}} E,$$

where $J_F = \partial F / \partial h$.

Proof. Differentiating $\widetilde{h} = h + F(h; W)$ with respect to h gives

$$\frac{\partial \widetilde{h}}{\partial h} = I + \frac{\partial F}{\partial h} = I + J_F.$$

The multivariate chain rule yields

$$\begin{aligned} \nabla_h E &= \left(\frac{\partial \widetilde{h}}{\partial h}\right)^\top \nabla_{\widetilde{h}} E \\ &= (I + J_F)^\top \nabla_{\widetilde{h}} E \\ &= \nabla_{\widetilde{h}} E + J_F^\top \nabla_{\widetilde{h}} E. \end{aligned}$$

□

The first summand provides a direct, unmultiplied gradient contribution through the identity path. This makes optimization of deep networks easier, although the second summand can interfere with the first, and the presence of a skip connection does not establish a uniform nonvanishing gradient bound for every choice of weights.

For two residual blocks, the total Jacobian is

$$\frac{\partial h^{(2)}}{\partial h^{(0)}} = (I + J_{F_1})(I + J_{F_0}) = I + J_{F_0} + J_{F_1} + J_{F_1} J_{F_0}.$$

The expansion contains an identity term corresponding to bypassing both residual transformations, two single-transformation paths, and one path through both transformations. This algebra captures the multiple backward routes indicated by the handwritten sketch.

9.3.4 Exploding Gradients

The opposite difficulty occurs when a chain of derivatives repeatedly amplifies gradient magnitude. The same Jacobian-product formula that produces vanishing gradients can produce *exploding gradients* when expansion dominates.

For a scalar chain $h_\ell = \sigma(w_\ell h_{\ell-1} + b_\ell)$,

$$\frac{\partial h_L}{\partial h_0} = \prod_{\ell=1}^{L} w_\ell \sigma'(z_\ell), \qquad z_\ell = w_\ell h_{\ell-1} + b_\ell.$$

If $|w_\ell \sigma'(z_\ell)| > 1$ along enough layers, this derivative can become large.

Example 9.1 (Exploding gradients despite a bounded sigmoid derivative)**.** Let every layer have $w_\ell = 8$, $b_\ell = -4$, logistic activation, and input $h_0 = 1/2$. Then

$$z_\ell = 8 \cdot \frac{1}{2} - 4 = 0,$$
$$h_\ell = \sigma(0) = \frac{1}{2},$$
$$w_\ell \sigma'(z_\ell) = 8 \cdot \frac{1}{4} = 2.$$

Thus the forward activations remain at $1/2$, while

$$\frac{\partial h_L}{\partial h_0} = 2^L.$$

After four such layers, the backward multiplier is $2^4 = 16$; after ten layers, it is $2^{10} = 1024$. The example shows that small activation derivatives do not alone rule out exploding gradients: the associated weights also matter.

Large gradients can cause erratic parameter updates, numerical overflow, or unstable loss values. Potential remedies include careful initialization, normalization, appropriate learning rates, residual architectures, and gradient clipping. Their effects and guarantees vary across architectures and datasets.

9.3.5 Gradient Clipping

Gradient clipping restricts the magnitude of a gradient before the parameter update. The norm-based version preserves its direction unless the gradient is zero.

Definition 9.8 (Global norm clipping)**.** Let $g \in \mathbb{R}^p$ be a gradient and let $\tau > 0$ be a clipping threshold. Define

$$\operatorname{clip}_\tau(g) = \begin{cases} g, & \|g\|_2 \le \tau, \\ \tau \dfrac{g}{\|g\|_2}, & \|g\|_2 > \tau. \end{cases}$$

Equivalently,

$$\operatorname{clip}_\tau(g) = g \min\left\{1, \frac{\tau}{\|g\|_2}\right\}$$

for $g \neq 0$, with $\operatorname{clip}_\tau(0) = 0$.

Proposition 9.11 (Clipping is Euclidean projection onto a ball)**.** *Let $B_\tau = \{u \in \mathbb{R}^p : \|u\|_2 \le \tau\}$. Then*

$$\operatorname{clip}_\tau(g) = \operatorname*{arg\,min}_{u \in B_\tau} \frac{1}{2}\|u - g\|_2^2.$$

In particular,

$$\|\operatorname{clip}_\tau(g)\|_2 = \min\{\|g\|_2, \tau\},$$

and for $g \neq 0$ the clipped gradient points in the same direction as g.

Proof. If $\|g\|_2 \le \tau$, the feasible choice $u = g$ makes the objective zero, which is minimal. Now suppose $\|g\|_2 > \tau$. For any feasible u, the Cauchy–Schwarz inequality gives

$$\begin{aligned}\|u - g\|_2^2 &= \|u\|_2^2 + \|g\|_2^2 - 2u^\top g \\ &\ge \|u\|_2^2 + \|g\|_2^2 - 2\|u\|_2\|g\|_2 \\ &= (\|g\|_2 - \|u\|_2)^2 \\ &\ge (\|g\|_2 - \tau)^2.\end{aligned}$$

Equality is attained by $u = \tau g/\|g\|_2$. This proves optimality and the remaining identities. □

With clipping, an SGD update becomes

$$\theta_{t+1} = \theta_t - \eta_t \operatorname{clip}_\tau(g_t).$$

This prevents a single update from having norm greater than $\eta_t\tau$ under ordinary SGD. It does not by itself correct the conditioning of the loss landscape or prevent gradients from vanishing. Furthermore, clipping is a nonlinear transformation, so a clipped stochastic gradient is generally not an unbiased estimator of the original gradient.

9.4 Putting the Optimization Techniques Together

A common training procedure uses mini-batches for affordable gradient estimates, momentum for accumulated directions, weight decay for parameter regularization, and optional gradient clipping for controlling unusually large gradient updates.

Example 9.2 (One iteration of mini-batch SGD with momentum and clipping)**.** Suppose the current mini-batch gradient is g_t, the accumulated momentum is m_t, and the parameters to be penalized are θ_t. A mathematically explicit algorithm is

$$\begin{aligned}\widetilde{g}_t &= g_t + \lambda\theta_t, \\ \overline{g}_t &= \operatorname{clip}_\tau(\widetilde{g}_t), \\ m_{t+1} &= \beta m_t + \overline{g}_t, \\ \theta_{t+1} &= \theta_t - \eta_t m_{t+1}.\end{aligned}$$

This is one valid ordering: clip the regularized gradient and then accumulate momentum. Other practical optimizers clip before adding weight decay, apply weight decay in a decoupled step, or clip the accumulated update instead; these choices are not algebraically identical.

Remark 9.7 (What each mechanism addresses)*.* The mechanisms serve related but different purposes. Validation estimates performance on held-out data; weight decay discourages large parameter magnitudes; mini-batching reduces per-update data-processing cost; momentum accumulates information across updates; residual connections create direct information and gradient paths; and clipping bounds the size of an applied gradient. None of these methods alone guarantees global optimality or perfect generalization.

10 Lecture 11: Convolutional Neural Networks: Kernels, Channels, Padding, and Stride

In the preceding lectures, a neural network was described as a composition of affine transformations and nonlinear activation functions. A fully connected layer treats every input coordinate as potentially connected to every output coordinate. When the input is an image, however, its coordinates have a spatial organization: neighboring pixels usually contain related information, and the same local pattern may appear at many locations. *Convolutional neural networks* exploit these properties by applying small, shared sets of weights across an image.

The central objects in this lecture are a *kernel* (also called a filter), an *input feature map*, and an *output feature map*. We will begin with a single-channel image, derive the convolutional operation carefully, extend it to multiple kernels and multiple channels, and then examine the effects of padding and stride. Finally, we will derive the parameter count and the backward derivatives needed to train a convolutional layer.

10.1 Spatial Structure and Local Connectivity

Suppose a grayscale image has height H and width W. We represent it as a matrix

$$F = (F_{i,j}) \in \mathbb{R}^{H \times W}, \qquad 0 \le i < H, \quad 0 \le j < W.$$

The entries $F_{i,j}$ are pixel intensities or, at a later network layer, entries of a learned feature map. Row index i increases vertically, and column index j increases horizontally. Throughout the derivations below, array indices begin at zero.

A fully connected affine layer operating on this image would first flatten F into a vector in $\mathbb{R}^{HW}$. If the layer has M output units, it computes

$$y = A\,\mathrm{vec}(F) + b, \qquad A \in \mathbb{R}^{M \times HW}, \quad b \in \mathbb{R}^{M}.$$

Hence the layer contains MHW weights and M biases. The structure of A does not, by itself, express the fact that nearby pixels are spatially related.

A convolutional layer introduces two useful constraints:

1. **Local connectivity.** An output at location (i, j) depends on a small neighborhood of the input instead of the entire image.
2. **Parameter sharing.** The same kernel coefficients are reused at every admissible spatial location.

These constraints reduce the number of independently learned coefficients and establish a useful spatial structure in the output. In particular, the layer can recognize a similar local pattern at different locations without learning an unrelated set of weights at every position.

10.2 A Single Convolutional Kernel

10.2.1 The Kernel as a Local Pattern Detector

Definition 10.1 (Kernel and local image patch)**.** Let $D \in \mathbb{N}$ be a kernel width and let

$$g = (g_{p,q}) \in \mathbb{R}^{D \times D}.$$

The *kernel* g consists of D^2 trainable coefficients. For an image $F \in \mathbb{R}^{H \times W}$, the $D \times D$ patch beginning at location (i, j) is

$$F^{(i,j)} = (F_{i+p,j+q})_{0 \le p,q < D},$$

provided that all indices in the patch are within the image boundaries.

The kernel produces a scalar by taking the inner product of its coefficients with those of the current patch. When i, j identify the upper-left corner of a valid patch, set

$$\langle F^{(i,j)}, g \rangle = \sum_{p=0}^{D-1} \sum_{q=0}^{D-1} F_{i+p,j+q}\, g_{p,q}.$$

The kernel g is then moved to another position and the same computation is repeated.

Definition 10.2 (Single-channel convolutional preactivation)**.** For a single input channel, one $D \times D$ kernel, unit stride, and no padding, the output *preactivation* at position (i, j) is

$$z_{i,j} = \sum_{p=0}^{D-1} \sum_{q=0}^{D-1} F_{i+p,j+q}\, g_{p,q} + b,$$

where $b \in \mathbb{R}$ is a single bias shared across all output locations. This definition is valid for

$$0 \le i \le H - D, \qquad 0 \le j \le W - D.$$

The array $Z = (z_{i,j})$ is called the output *feature map* before an activation function is applied.

Notice that b is *not* a separate parameter for each output position. In exactly the same manner as the kernel coefficients, it is shared across the map.

Remark 10.1 (Convolution versus cross-correlation). The operation used in most modern convolutional neural-network implementations is, strictly speaking, *cross-correlation*: the coefficients $g_{p,q}$ are not reversed before being multiplied by the patch. The conventional discrete mathematical convolution instead reverses the kernel. In deep learning, the cross-correlation operation is nevertheless commonly called *convolution*; we follow that convention unless explicitly stated otherwise. Because the kernel is learned, reversing its indexing does not reduce the set of filters that can be represented.

10.2.2 A Kernel Sliding Over an Image

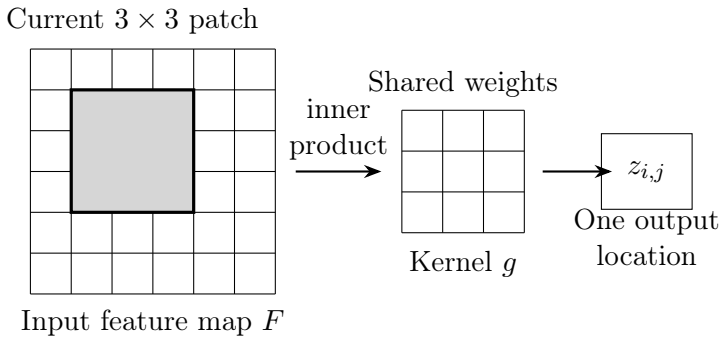


Figure 24: A convolutional kernel compares its entries with an input patch. The same coefficients are reused when the window moves to another position. The shaded 3×3 patch is shown only schematically.

The parameter-sharing property means that, although the layer computes many different outputs $z_{i,j}$, it learns only the D^2 kernel coefficients and one bias. The output values differ because each location supplies a different patch of the image, not because the parameters change with location.

10.2.3 Worked Example of a Valid Convolution

Example 10.1 (A 2×2 filter on a 4×4 image)**.** Consider

$$F = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}, \qquad g = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}, \qquad b = -1.$$

At the upper-left position, the local patch is

$$F^{(0,0)} = \begin{pmatrix} 1 & 2 \\ 5 & 6 \end{pmatrix}.$$

Its output is obtained by multiplying corresponding entries:

$$\begin{aligned} z_{0,0} &= 1 \cdot 1 + 2 \cdot 0 + 5 \cdot (-1) + 6 \cdot 2 - 1 \\ &= 1 + 0 - 5 + 12 - 1 \\ &= 7. \end{aligned}$$

At the position immediately to the right,

$$\begin{aligned} z_{0,1} &= 2 \cdot 1 + 3 \cdot 0 + 6 \cdot (-1) + 7 \cdot 2 - 1 \\ &= 2 - 6 + 14 - 1 \\ &= 9. \end{aligned}$$

Repeating the calculation for every valid patch produces

$$Z = \begin{pmatrix} 7 & 9 & 11 \\ 15 & 17 & 19 \\ 23 & 25 & 27 \end{pmatrix}.$$

There are 3×3 valid patch locations: the 2×2 kernel cannot begin in the final row or column without extending beyond the image. The kernel and bias contribute only $4 + 1 = 5$ trainable parameters, even though nine output values are produced.

10.3 Output Size for a Valid Convolution

The spatial dimensions of the output depend on the input dimensions, kernel dimensions, padding, and stride. We begin with the simplest case.

Proposition 10.1 (Output size with unit stride and no padding)**.** *Let $F \in \mathbb{R}^{H \times W}$ and $g \in \mathbb{R}^{D_h \times D_w}$, where $1 \le D_h \le H$ and $1 \le D_w \le W$. With unit stride and no padding, the output has dimensions*

$$H_{\text{out}} = H - D_h + 1, \qquad W_{\text{out}} = W - D_w + 1.$$

For a square $D \times D$ kernel, the output is $(H - D + 1) \times (W - D + 1)$.

Proof. A valid patch starting in row i must satisfy

$$0 \le i, \qquad i + (D_h - 1) \le H - 1.$$

Therefore

$$0 \le i \le H - D_h.$$

There are $H - D_h + 1$ integer choices for i. Likewise, the column index must satisfy

$$0 \le j \le W - D_w,$$

which provides $W - D_w + 1$ choices. Each permissible pair (i, j) produces exactly one output value, proving the formula. □

Example 10.2 (Shrinkage without padding)**.** An input of size 32×32 convolved with a 5×5 kernel at unit stride and without padding produces

$$H_{\text{out}} = W_{\text{out}} = 32 - 5 + 1 = 28.$$

Thus one filter produces a 28×28 output feature map.

10.4 Multiple Kernels and Output Feature Maps

A convolutional layer rarely contains only one kernel. Different kernels can learn different local patterns, and each produces an independent output feature map.

Definition 10.3 (A bank of convolutional kernels)**.** Let $K \geq 1$ be the number of output kernels. For a single-channel input, suppose

$$g^{(k)} \in \mathbb{R}^{D \times D}, \qquad b^{(k)} \in \mathbb{R}, \qquad k = 1, \ldots, K.$$

The output of kernel k is

$$\begin{aligned} z_{i,j}^{(k)} &= \langle F^{(i,j)}, g^{(k)} \rangle + b^{(k)} \\ &= \sum_{p=0}^{D-1} \sum_{q=0}^{D-1} F_{i+p,j+q} g_{p,q}^{(k)} + b^{(k)}. \end{aligned}$$

The K output feature maps can be stacked to form

$$Z \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times K}.$$

The superscript (k) is a kernel or output-channel label, *not* an exponent. Every output channel is computed from its own kernel and bias, but all the outputs are functions of the same input feature map.

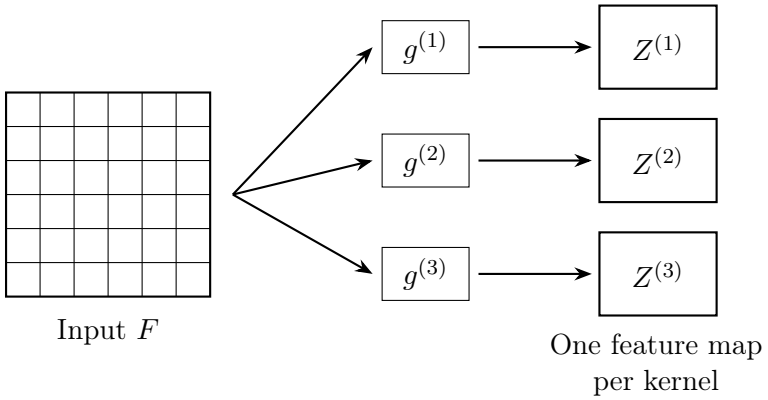


Figure 25: A bank of three filters produces three output feature maps. A layer with K filters produces K such maps.

10.5 Convolution with Multiple Input Channels

A color image generally contains three channels (for example, red, green, and blue). Intermediate feature maps in a neural network may have tens or hundreds of channels. A convolutional layer must therefore extend its kernel across *all* input channels rather than apply an unrelated two-dimensional kernel to each one and stop there.

10.5.1 Three-Dimensional Kernels

Definition 10.4 (Multichannel input and kernel)**.** Let an input feature tensor have dimensions

$$F \in \mathbb{R}^{H \times W \times C},$$

where C is the number of input channels. One convolutional filter now has shape

$$g \in \mathbb{R}^{D_h \times D_w \times C}.$$

With unit stride and no padding, its preactivation is

$$z_{i,j} = \sum_{c=0}^{C-1} \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p,j+q,c}\, g_{p,q,c} + b.$$

Thus the filter evaluates a separate local inner product in every input channel and *sums the results across channels* to produce one scalar at location (i, j). For fixed i, j,

$$\begin{aligned} z_{i,j} &= \sum_{c=0}^{C-1} \langle F_c^{(i,j)}, g_c \rangle + b \\ &= \langle F^{(i,j)}, g \rangle + b, \end{aligned}$$

where the final inner product is over the entire $D_h \times D_w \times C$ local patch. Each channel has its own kernel slice $g_{:,:,c}$, but all slices belong to the *same output filter*.

Remark 10.2 (Input channels are not output channels). The number C counts the channels supplied *to* the layer. The number K counts the filters used by the layer and therefore the output channels it creates. A single filter spans all C input channels, and its output is one feature map, not C separate maps.

10.5.2 The General C-Input, K-Output Layer

Definition 10.5 (Multichannel convolutional layer)**.** Let $F \in \mathbb{R}^{H \times W \times C}$ and let

$$G \in \mathbb{R}^{D_h \times D_w \times C \times K}, \qquad b \in \mathbb{R}^K.$$

For output channel $k = 0, \ldots, K-1$, the kth preactivation feature map is

$$z_{i,j,k} = \sum_{c=0}^{C-1} \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p,j+q,c} G_{p,q,c,k} + b_k.$$

Its output tensor is

$$Z \in \mathbb{R}^{(H-D_h+1) \times (W-D_w+1) \times K}$$

for valid unit-stride convolution.

The layer may then apply a pointwise activation σ to obtain

$$A_{i,j,k} = \sigma(z_{i,j,k}).$$

For example, a rectified linear unit gives $A_{i,j,k} = \max\{0, z_{i,j,k}\}$. The activated tensor can serve as the multichannel input of a subsequent convolutional layer.

Example 10.3 (RGB image with multiple output channels)**.** Let a color image have shape $64 \times 64 \times 3$, and let a convolutional layer use $K = 16$ kernels of spatial size 3×3. Each of the 16 filters has shape $3 \times 3 \times 3$, and the complete weight tensor has shape

$$G \in \mathbb{R}^{3\times3\times3\times16}.$$

With stride one and no padding, the output has shape

$$62 \times 62 \times 16.$$

There are

$$3 \cdot 3 \cdot 3 \cdot 16 + 16 = 448$$

trainable coefficients when one bias is used per output channel.

10.6 Padding: Convolutions Near Image Boundaries

For a valid convolution, the filter is evaluated only when it lies fully within the original image. As a consequence, the output is smaller than the input when the kernel has width greater than one. *Padding* lets us specify values outside the original image so that a kernel can also be evaluated near its boundaries.

Definition 10.6 (Zero padding)**.** For nonnegative integers P_h, P_w, define the zero-padded input $\widetilde{F}$ by surrounding F with P_h rows of zeros at the top and bottom and P_w columns of zeros on the left and right. Then

$$\widetilde{F} \in \mathbb{R}^{(H+2P_h)\times(W+2P_w)\times C},$$

and, for $0 \le i < H$, $0 \le j < W$,

$$\widetilde{F}_{i+P_h,j+P_w,c} = F_{i,j,c}.$$

All other entries of $\widetilde{F}$ are zero.

The convolution is then applied to $\widetilde{F}$ rather than directly to F. Zero padding permits the kernel to overlap the boundary without referring to unspecified data. The padded entries are artificial zeros; they are not learned and should not be interpreted as measured image pixels.

Proposition 10.2 (Output size with symmetric padding)**.** *For kernel shape $D_h \times D_w$, unit stride, and symmetric padding amounts P_h, P_w, the output dimensions are*

$$H_{\text{out}} = H + 2P_h - D_h + 1, \qquad W_{\text{out}} = W + 2P_w - D_w + 1,$$

provided both resulting dimensions are positive.

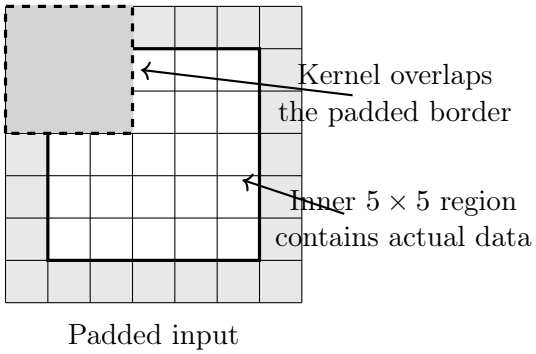


Figure 26: A 5×5 input with one zero-padded row or column on each edge. The dashed 3×3 window extends into the added border.

Proof. The padded input dimensions are $H + 2P_h$ and $W + 2P_w$. Applying the valid-convolution counting argument to this larger array gives

$$H_{\text{out}} = (H + 2P_h) - D_h + 1,$$
$$W_{\text{out}} = (W + 2P_w) - D_w + 1.$$

This is precisely the stated formula. □

10.6.1 Valid and Same Padding

The words *valid* and *same* are commonly used to describe padding conventions.

- **Valid convolution:** no padding; a kernel is used only when its entire window lies inside the original image.
- **Same convolution:** padding is chosen so that, when stride is one, the output height and width match the input height and width.

For an odd square kernel of width D and stride one, symmetric padding

$$P_h = P_w = \frac{D-1}{2}$$

gives

$$\begin{aligned} H_{\text{out}} &= H + 2\left(\frac{D-1}{2}\right) - D + 1 \\ &= H, \\ W_{\text{out}} &= W + 2\left(\frac{D-1}{2}\right) - D + 1 \\ &= W. \end{aligned}$$

For even kernel widths, exact size preservation may require asymmetric padding. The actual distribution of the added zeros depends on the framework's conventions.

Example 10.4 (The difference made by padding)**.** For a 28×28 image and a 5×5 kernel at unit stride:

$$\begin{aligned} \text{No padding:} \quad & H_{\text{out}} = 28 - 5 + 1 = 24, \\ & Z \in \mathbb{R}^{24 \times 24}; \\ \text{Two cells of padding per side:} \quad & H_{\text{out}} = 28 + 2 \cdot 2 - 5 + 1 = 28, \\ & Z \in \mathbb{R}^{28 \times 28}. \end{aligned}$$

Thus padding controls whether the feature map shrinks at this layer.

10.7 Stride: How Far the Kernel Moves

Until now the kernel has moved by one pixel from one output location to the next. A *stride* specifies how far the convolution window moves between successive applications.

Definition 10.7 (Stride)**.** Let $s_h, s_w \in \mathbb{N}$. The *vertical stride* s_h and *horizontal stride* s_w specify how many padded-input rows and columns the kernel moves between neighboring output positions. For a single channel, the strided convolution is

$$z_{i,j} = \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} \widetilde{F}_{s_h i+p, s_w j+q}\, g_{p,q} + b.$$

The usual unit-stride convolution is the case $s_h = s_w = 1$.

A larger stride samples fewer starting positions and therefore produces a smaller output. Notice that stride changes the spacing of the sampling locations; it does not itself change how many weights are stored in the kernel.

Proposition 10.3 (General output-size formula)**.** *For input dimensions $H \times W$, kernel dimensions $D_h \times D_w$, symmetric padding P_h, P_w, and strides s_h, s_w, assume at least one valid kernel position exists in each direction. Then*

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{H + 2P_h - D_h}{s_h} \right\rfloor + 1, \\ W_{\text{out}} &= \left\lfloor \frac{W + 2P_w - D_w}{s_w} \right\rfloor + 1. \end{aligned}$$

Proof. A kernel starting at vertical output index i begins in padded-input row $s_h i$ and ends in row $s_h i + D_h - 1$. It lies within the padded input if and only if

$$\begin{aligned} s_h i + D_h - 1 &\le H + 2P_h - 1, \\ s_h i &\le H + 2P_h - D_h, \\ i &\le \frac{H + 2P_h - D_h}{s_h}. \end{aligned}$$

The permissible integer values are therefore

$$i = 0, 1, \ldots, \left\lfloor \frac{H + 2P_h - D_h}{s_h} \right\rfloor.$$

Counting these indices gives the height formula. The identical argument applied horizontally gives the width formula. □

Example 10.5 (A strided convolution)**.** Let $H = W = 7$, let the kernel be 3×3, let the stride be $s_h = s_w = 2$, and use one zero of padding on each side. Then

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{7 + 2 - 3}{2} \right\rfloor + 1 \\ &= \lfloor 3 \rfloor + 1 = 4. \end{aligned}$$

Similarly $W_{\text{out}} = 4$. One kernel therefore gives a 4×4 output, whereas unit stride with the same padding would give a 7×7 output.

Remark 10.3 (The floor matters). The division in the size formula need not be exact. For example, an input of height 8, a kernel of height 3, zero padding, and stride 2 gives

$$H_{\text{out}} = \left\lfloor \frac{8 - 3}{2} \right\rfloor + 1 = 3.$$

The window begins at rows 0, 2, and 4; beginning at row 6 would make it extend beyond the input. A framework's `same` padding convention with stride greater than one may add asymmetric padding so that the output dimension instead equals $\lceil H/s_h \rceil$.

10.8 Parameter Count and Computational Efficiency

The lecture's final comparison concerns the dramatic reduction in independently learned parameters when a spatially structured input is processed by a convolutional layer rather than by a dense affine transformation.

Proposition 10.4 (Number of parameters in a convolutional layer)**.** *For C input channels, K output channels, and spatial kernel shape $D_h \times D_w$, a standard convolutional layer has*

$$N_{\text{param}} = K(CD_hD_w + 1)$$

trainable parameters if each output channel has one learned bias. Without bias terms, it has KCD_hD_w parameters. The count is independent of the input image's height and width.

Proof. Each of the K output filters contains a coefficient for every combination of one input channel and one spatial kernel position. There are CD_hD_w such combinations per filter, hence KCD_hD_w weights. An additional bias is learned for each output filter, contributing K parameters. Summing these gives

$$KCD_hD_w + K = K(CD_hD_w + 1).$$

The original image dimensions H and W do not enter this count, because the same coefficients are reused at all spatial positions. □

10.8.1 A Fully Connected Layer on a 100-by-100 Image

Consider the example from the handwritten notes: a grayscale image of size 100×100 and a dense hidden representation also containing 100×100 units. Flattening gives

$$\mathrm{vec}(F) \in \mathbb{R}^{10{,}000}, \qquad y \in \mathbb{R}^{10{,}000}.$$

A fully connected transformation requires a weight matrix

$$A \in \mathbb{R}^{10{,}000 \times 10{,}000}.$$

Consequently,

$$\begin{aligned} N_{\text{weights}} &= 10{,}000 \cdot 10{,}000 \\ &= 100{,}000{,}000 = 10^8. \end{aligned}$$

If the layer also learns a bias for each output unit, its total count is

$$10^8 + 10^4 = 100{,}010{,}000.$$

Even one dense layer of this shape contains one hundred million distinct weight coefficients.

10.8.2 A Convolutional Layer with 64 Kernels

Now use 64 kernels, each of spatial size 5×5, on a single-channel input. Each filter contains

$$\begin{aligned} 5 \cdot 5 \cdot 1 &= 25 \quad \text{weights}, \\ 25 + 1 &= 26 \quad \text{parameters including bias}. \end{aligned}$$

For all 64 filters,

$$\begin{aligned} N_{\text{param}} &= 64(5 \cdot 5 \cdot 1 + 1) \\ &= 64 \cdot 26 \\ &= 1664. \end{aligned}$$

Thus the convolutional layer learns only 1,664 coefficients, as opposed to the more than 10^8 coefficients in the dense example.

Remark 10.4 (Comparing models with different output sizes). The dense and convolutional layers above do *not* necessarily have equal output dimensions or equivalent expressive constraints: the convolutional layer creates 64 spatial feature maps, whereas the dense example creates a flat vector with 10,000 coordinates. The calculation illustrates the importance of weight sharing, rather than establishing a direct same-architecture comparison. With two-dimensional `same` padding and stride one, the convolutional output has size $100 \times 100 \times 64$, so the number of

activation values can actually exceed that of the dense example even though the number of trainable parameters is far smaller.

For an RGB input, $C = 3$ and the same convolutional layer would instead have

$$64(5 \cdot 5 \cdot 3 + 1) = 64 \cdot 76 = 4864$$

parameters. A change in the input-channel count affects the parameter count; a change in the input's spatial height or width does not.

10.8.3 Parameters and Computation Are Distinct

A small number of parameters does not automatically imply a small number of arithmetic operations. Every output location must evaluate its local inner product. For standard convolution, ignoring bias additions and nonlinearities, the number of scalar multiplications is approximately

$$N_{\text{mult}} = H_{\text{out}} W_{\text{out}} K C D_h D_w.$$

For a $100 \times 100 \times 1$ input, a 5×5 kernel, 64 output channels, stride one, and same padding, this becomes

$$\begin{aligned} N_{\text{mult}} &= 100 \cdot 100 \cdot 64 \cdot 1 \cdot 5 \cdot 5 \\ &= 16{,}000{,}000. \end{aligned}$$

These computations repeatedly use the same 1,600 kernel weights. In practice, convolution can be implemented using specialized algorithms or optimized matrix operations, but its computational cost still depends strongly on the output feature-map dimensions.

10.9 Why Weight Sharing Gives Translation Equivariance

One conceptual advantage of convolution is that a translated input produces a correspondingly translated feature map, provided boundaries are handled consistently. This property is called *translation equivariance*, not translation invariance.

Definition 10.8 (Translation operator)**.** For an array F defined on the two-dimensional integer lattice $\mathbb{Z}^2$, define its translation by $(a, b) \in \mathbb{Z}^2$ as

$$(T_{a,b}F)_{i,j} = F_{i-a,j-b}.$$

An operator $\mathcal{C}$ is translation-equivariant when

$$\mathcal{C}(T_{a,b}F) = T_{a,b}(\mathcal{C}F)$$

for every translation to which both sides apply.

Proposition 10.5 (Equivariance of unit-stride cross-correlation)**.** *Let $g \in \mathbb{R}^{D_h \times D_w}$ and let $F : \mathbb{Z}^2 \to \mathbb{R}$ be any image for which the local sums below are defined. Set*

$$(\mathcal{C}_g F)_{i,j} = \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p,j+q} g_{p,q}.$$

Then for every $(a, b) \in \mathbb{Z}^2$,

$$\mathcal{C}_g(T_{a,b}F) = T_{a,b}(\mathcal{C}_g F).$$

A spatially shared bias and a pointwise activation preserve this property.

Proof. By the definition of $T_{a,b}$,

$$\begin{aligned}
(\mathcal{C}_g(T_{a,b}F))_{i,j} &= \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} (T_{a,b}F)_{i+p,j+q} g_{p,q} \\
&= \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p-a,j+q-b} g_{p,q} \\
&= (\mathcal{C}_g F)_{i-a,j-b} \\
&= (T_{a,b}(\mathcal{C}_g F))_{i,j}.
\end{aligned}$$

A constant bias b is the same at every spatial position, so translating the output does not alter it. If σ is applied independently at every location, then

$$\sigma((T_{a,b}Z)_{i,j}) = \sigma(Z_{i-a,j-b}) = (T_{a,b}\sigma(Z))_{i,j},$$

which proves the corresponding statement for pointwise activations. □

Remark 10.5 (Boundary effects and stride). The proposition assumes an infinite lattice or a translation convention consistent with the local operator. Finite image boundaries and padding can break exact equivariance when content crosses the boundary. Likewise, stride $s > 1$ generally preserves equivariance only under translations aligned with the stride grid; a one-pixel input shift need not correspond to a simple one-pixel output shift. A convolutional layer is also not generally *translation-invariant*: the location of a detected pattern is represented by the location of its response.

10.10 Convolution as a Structured Linear Map

A convolutional layer is affine in its input when the filter coefficients are fixed. It can therefore be represented as a matrix multiplication, but the corresponding matrix has many zeros and repeated coefficients.

Example 10.6 (A one-dimensional convolution matrix)**.** Consider input $x = (x_0, x_1, x_2, x_3)^\top$ and a two-element kernel $g = (a, b)$. A valid stride-one cross-correlation gives

$$\begin{aligned} z_0 &= ax_0 + bx_1, \\ z_1 &= ax_1 + bx_2, \\ z_2 &= ax_2 + bx_3. \end{aligned}$$

The entire mapping can be written

$$\begin{pmatrix} z_0 \\ z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} a & b & 0 & 0 \\ 0 & a & b & 0 \\ 0 & 0 & a & b \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

The matrix has 8 nonzero entries, but there are only two independent learned weights, a and b. Their repetition encodes parameter sharing.

Two-dimensional convolution can similarly be written using a sparse, structured matrix acting on a flattened image. The useful computational perspective is that convolution is a *constrained affine map*: the same kernel values recur along the spatial diagonals corresponding to shifted patches, and entries corresponding to nonlocal connections are zero.

Remark 10.6 (What a filter learns). A hand-specified kernel can represent local smoothing, differences, or edge-sensitive patterns. In a neural network the kernel is not usually specified manually: its coefficients are optimized from data to reduce the task loss. Its learned response can be passed through a nonlinear activation and then supplied to deeper convolutional layers, enabling compositions of increasingly complex local features.

10.11 Backpropagation Through a Convolutional Layer

The previous lectures developed reverse-mode automatic differentiation and backpropagation for affine and nonlinear layers. We now derive the analogous gradients for a convolutional layer directly from the indexed forward equation.

For clarity, consider first one input channel, one output kernel, stride one, and valid convolution:

$$z_{i,j} = \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p,j+q} g_{p,q} + b.$$

Let E be a differentiable scalar loss depending on all output preactivations, and define the upstream gradient

$$\delta_{i,j} := \frac{\partial E}{\partial z_{i,j}}.$$

Proposition 10.6 (Gradients with respect to the kernel and bias)**.** *For every kernel position* (p, q)*,*

$$\frac{\partial E}{\partial g_{p,q}} = \sum_{i=0}^{H-D_h} \sum_{j=0}^{W-D_w} \delta_{i,j} F_{i+p,j+q}.$$

The shared-bias gradient is

$$\frac{\partial E}{\partial b} = \sum_{i=0}^{H-D_h} \sum_{j=0}^{W-D_w} \delta_{i,j}.$$

Proof. The parameter $g_{p,q}$ influences every output location whose patch contains the corresponding kernel coefficient. Applying the multivariable chain rule,

$$\begin{aligned}
\frac{\partial E}{\partial g_{p,q}} &= \sum_{i,j} \frac{\partial E}{\partial z_{i,j}} \frac{\partial z_{i,j}}{\partial g_{p,q}} \\
&= \sum_{i,j} \delta_{i,j} \frac{\partial}{\partial g_{p,q}} \left(\sum_{r=0}^{D_h-1} \sum_{s=0}^{D_w-1} F_{i+r,j+s} g_{r,s} + b \right) \\
&= \sum_{i,j} \delta_{i,j} F_{i+p,j+q}.
\end{aligned}$$

Here the sums over (i, j) run over all valid output positions. Similarly, every preactivation contains b with derivative one, giving

$$\begin{aligned}
\frac{\partial E}{\partial b} &= \sum_{i,j} \frac{\partial E}{\partial z_{i,j}} \frac{\partial z_{i,j}}{\partial b} \\
&= \sum_{i,j} \delta_{i,j}.
\end{aligned}$$

□

Remark 10.7 (Why gradients must be summed). The same kernel parameter is reused at many positions during the forward pass. Therefore its derivative accumulates contributions from *every* output position in which that parameter was used. This is the convolutional analogue of gradient accumulation at a shared node in a computational graph.

Proposition 10.7 (Gradient with respect to the input)**.** *For a pixel* $F_{u,v}$*, the gradient is*

$$\frac{\partial E}{\partial F_{u,v}} = \sum_{\substack{0 \le i \le H-D_h,\ 0 \le j \le W-D_w \\ 0 \le u-i < D_h,\ 0 \le v-j < D_w}} \delta_{i,j}\, g_{u-i,v-j}.$$

Equivalently, extend the kernel by zero outside its $D_h \times D_w$ *support and write*

$$\frac{\partial E}{\partial F_{u,v}} = \sum_{i=0}^{H-D_h} \sum_{j=0}^{W-D_w} \delta_{i,j} g_{u-i,v-j}.$$

Proof. A particular $F_{u,v}$ contributes to output $z_{i,j}$ precisely when there exist indices p, q with

$$u = i + p, \qquad v = j + q, \qquad 0 \le p < D_h, \quad 0 \le q < D_w.$$

In that case,

$$\frac{\partial z_{i,j}}{\partial F_{u,v}} = g_{u-i,v-j};$$

otherwise the derivative is zero. By the chain rule,

$$\begin{aligned}\frac{\partial E}{\partial F_{u,v}} &= \sum_{i,j} \frac{\partial E}{\partial z_{i,j}} \frac{\partial z_{i,j}}{\partial F_{u,v}} \\ &= \sum_{\substack{i,j \text{ valid} \\ 0 \le u-i < D_h,\, 0 \le v-j < D_w}} \delta_{i,j} g_{u-i,v-j},\end{aligned}$$

which is the first expression. The zero-extension convention yields the second expression. □

10.11.1 Multichannel Backpropagation

For the full convolutional layer,

$$z_{i,j,k} = \sum_{c=0}^{C-1} \sum_{p=0}^{D_h-1} \sum_{q=0}^{D_w-1} F_{i+p,j+q,c} G_{p,q,c,k} + b_k.$$

Define $\delta_{i,j,k} = \partial E / \partial z_{i,j,k}$. The analogous backward equations are

$$\begin{aligned}\frac{\partial E}{\partial G_{p,q,c,k}} &= \sum_{i,j} \delta_{i,j,k} F_{i+p,j+q,c}, \\ \frac{\partial E}{\partial b_k} &= \sum_{i,j} \delta_{i,j,k}, \\ \frac{\partial E}{\partial F_{u,v,c}} &= \sum_{k=0}^{K-1} \sum_{\substack{i,j \text{ valid} \\ 0 \le u-i < D_h,\, 0 \le v-j < D_w}} \delta_{i,j,k} G_{u-i,v-j,c,k}.\end{aligned}$$

Thus parameter gradients accumulate over spatial locations, and input gradients also accumulate over all output channels receiving information from that input channel.

With padding and stride, the same chain rule applies after replacing F by the padded tensor $\widetilde{F}$ and replacing the input location $(i + p, j + q)$ by $(s_h i + p, s_w j + q)$. Gradients in added padding cells are discarded when returning the gradient with respect to the original unpadded input.

Example 10.7 (A small kernel-gradient calculation)**.** Take a one-dimensional input $x = (1, 2, 3)^\top$ and a kernel $g = (a, b)$, so

$$z_0 = a + 2b, \qquad z_1 = 2a + 3b.$$

For a loss $E(z_0, z_1)$ with upstream derivatives $\delta_0 = \partial E/\partial z_0$ and $\delta_1 = \partial E/\partial z_1$,

$$\begin{aligned}\frac{\partial E}{\partial a} &= \delta_0 \frac{\partial z_0}{\partial a} + \delta_1 \frac{\partial z_1}{\partial a} \\ &= \delta_0 + 2\delta_1, \\ \frac{\partial E}{\partial b} &= \delta_0 \frac{\partial z_0}{\partial b} + \delta_1 \frac{\partial z_1}{\partial b} \\ &= 2\delta_0 + 3\delta_1.\end{aligned}$$

Each kernel coefficient receives two contributions because each was used at two distinct output positions.

10.12 Stacking Convolutional Layers and Receptive Fields

A convolutional layer extracts local features, and subsequent layers can process neighborhoods of those features. In this way, later units may depend on progressively larger regions of the original image.

Definition 10.9 (Receptive field)**.** The *receptive field* of a unit at a particular network layer is the set of input-image positions that can affect the value of that unit through the network's computational graph.

For a chain of layers in one spatial dimension (or applied independently to either spatial direction), let D_ℓ and s_ℓ be the kernel size and stride at layer ℓ. Let R_ℓ denote the receptive-field width in input coordinates of one layer-ℓ output and let J_ℓ denote the spacing, in input coordinates, between adjacent layer-ℓ receptive-field centers. Starting from

$$R_0 = 1, \qquad J_0 = 1,$$

the recursion is

$$J_\ell = J_{\ell-1} s_\ell, \qquad R_\ell = R_{\ell-1} + (D_\ell - 1) J_{\ell-1}.$$

For unit strides at every layer, $J_\ell = 1$ and therefore

$$R_L = 1 + \sum_{\ell=1}^{L} (D_\ell - 1).$$

For example, stacking three 3×3 convolutions at unit stride gives a 7×7 receptive field in the original input (away from boundaries):

$$\begin{aligned}R_1 &= 1 + (3 - 1) = 3, \\ R_2 &= 3 + (3 - 1) = 5, \\ R_3 &= 5 + (3 - 1) = 7.\end{aligned}$$

This calculation concerns which input positions can influence a unit; it does not assert that every position must have a nonzero influence for every parameter choice.

Remark 10.8 (Scope of the lecture). The core handwritten lecture introduces kernels, output feature maps, multiple channels, padding, stride, and the parameter-count comparison. Translation equivariance, explicit convolutional backpropagation, and receptive-field formulas are mathematical extensions included here to connect those ideas with the gradient methods and multilayer architectures developed earlier in the course.

10.13 Additional Exercises

Exercise 1.

A tensor $F \in \mathbb{R}^{48\times 64\times 3}$ is passed through $K = 32$ filters of spatial size 5×3, with zero padding $P_h = 2$, $P_w = 1$ and strides $s_h = 2$, $s_w = 2$. Find the output tensor dimensions and the number of trainable parameters when each output channel has one bias.

Solution. The spatial dimensions are

$$H_{\text{out}} = \left\lfloor \frac{48 + 2(2) - 5}{2} \right\rfloor + 1 = \left\lfloor \frac{47}{2} \right\rfloor + 1 = 24,$$
$$W_{\text{out}} = \left\lfloor \frac{64 + 2(1) - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 32.$$

Thus the output is $24 \times 32 \times 32$. The parameter count is

$$\begin{aligned} N_{\text{param}} &= K(CD_hD_w + 1) \\ &= 32(3 \cdot 5 \cdot 3 + 1) \\ &= 32 \cdot 46 = 1472. \end{aligned}$$

■

Exercise 2.

For an $H \times W$ single-channel input, compare two ways of producing K scalar output maps of the same spatial dimensions: (i) a convolution with a $D \times D$ kernel for each output channel, and (ii) a completely unrestricted dense map from HW inputs to $H_{\text{out}}W_{\text{out}}K$ outputs. Express both parameter counts.

Solution. A convolutional layer with one bias per output channel has

$$K(D^2 + 1)$$

parameters, independently of the spatial output size. A dense layer with HW input features and $H_{\text{out}}W_{\text{out}}K$ outputs has

$$\begin{aligned} N_{\text{dense}} &= (HW)(H_{\text{out}}W_{\text{out}}K) + H_{\text{out}}W_{\text{out}}K \\ &= H_{\text{out}}W_{\text{out}}K(HW + 1). \end{aligned}$$

The latter assigns independently learnable weights to all input–output pairs, whereas convolution restricts each output location to a local patch and ties the patch weights across locations. ■

Exercise 3.

For a single-channel valid convolution with kernel $g \in \mathbb{R}^{2\times 2}$, input $F \in \mathbb{R}^{3\times 3}$, and an upstream gradient $\delta \in \mathbb{R}^{2\times 2}$, write the gradient with respect to the top-left kernel coefficient $g_{0,0}$ and the central input pixel $F_{1,1}$.

Solution. The kernel coefficient $g_{0,0}$ is used at the top-left corner of each patch, so

$$\frac{\partial E}{\partial g_{0,0}} = \delta_{0,0}F_{0,0} + \delta_{0,1}F_{0,1} + \delta_{1,0}F_{1,0} + \delta_{1,1}F_{1,1}.$$

The central input pixel is included in all four output patches, but it aligns with a different kernel coefficient in each patch. Hence

$$\frac{\partial E}{\partial F_{1,1}} = \delta_{0,0}g_{1,1} + \delta_{0,1}g_{1,0} + \delta_{1,0}g_{0,1} + \delta_{1,1}g_{0,0}.$$

■

Exercise 4.

A convolutional network contains a first layer with kernel width 5 and stride 2, followed by a second layer with kernel width 3 and stride 1, and a third layer with kernel width 3 and stride 2. Compute the final receptive-field width and cumulative input-coordinate stride.

Solution. Start with $(R_0, J_0) = (1, 1)$. At the first layer,

$$R_1 = 1 + (5-1)\cdot 1 = 5, \qquad J_1 = 1 \cdot 2 = 2.$$

At the second layer,

$$R_2 = 5 + (3-1)\cdot 2 = 9, \qquad J_2 = 2 \cdot 1 = 2.$$

At the third layer,

$$R_3 = 9 + (3-1)\cdot 2 = 13, \qquad J_3 = 2 \cdot 2 = 4.$$

The final receptive field covers up to 13 consecutive input positions in that spatial direction, and neighboring final-layer units have receptive-field centers separated by 4 input positions. ■

11 Lecture 12: Hopfield Networks, Associative Memory, and Energy-Based Learning

A feedforward neural network maps an input to an output by evaluating a sequence of layers. A *Hopfield network* has a different purpose: it stores a collection of patterns and attempts to reconstruct a stored pattern from a partial or corrupted observation. Its interconnected neurons repeatedly revise their states until the system reaches a stable configuration. The resulting dynamics are interpreted as a form of *associative memory*, or *content-addressable memory*: the memory is retrieved from a cue resembling it, rather than from an explicit memory address.

The key mathematical feature is the existence of an *energy function*. Under appropriate assumptions on the connection weights and the update scheme, each state update cannot increase this energy. The network therefore evolves toward a configuration that is locally stable under individual-neuron changes. We will derive the storage rule, construct the energy function, prove its descent property, and connect the resulting model to matrix calculus and gradient-based learning.

11.1 Binary States and Network Architecture

11.1.1 States, Connections, and Biases

Definition 11.1 (Binary Hopfield network)**.** A binary Hopfield network with N neurons has a state vector

$$x = (x_1, \dots, x_N)^\top \in \{-1, +1\}^N.$$

Its parameters are a connection matrix $W = (w_{ij}) \in \mathbb{R}^{N \times N}$ and a bias vector $b \in \mathbb{R}^N$. We assume that

$$W = W^\top, \qquad w_{ii} = 0 \quad (1 \le i \le N).$$

The entry w_{ij} describes the interaction between neurons i and j. Symmetry means $w_{ij} = w_{ji}$, and the zero-diagonal condition excludes self-connections.

Each neuron has two possible states, -1 and $+1$. Consequently, an N-neuron system possesses 2^N possible joint configurations. The network is typically recurrent: neurons interact through connections that may form cycles, in contrast to the directed acyclic computation graphs used for ordinary feedforward backpropagation.

Remark 11.1 (Interpretation of connection signs). If $w_{ij} > 0$, the pairwise interaction favors $x_i = x_j$; if $w_{ij} < 0$, it favors $x_i = -x_j$. A zero weight contributes no direct interaction. These statements will follow from the pairwise contribution $-w_{ij}x_ix_j$ to the energy.

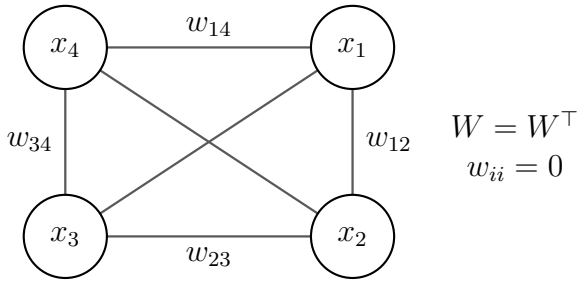


Figure 27: Four fully interconnected binary neurons. Each undirected edge represents a pair of equal weights $w_{ij} = w_{ji}$. There are no self-connections.

11.1.2 Local Fields and Threshold Updates

Definition 11.2 (Local field)**.** The local field at neuron i is

$$h_i(x) := \sum_{j \neq i} w_{ij} x_j + b_i.$$

For the whole network, the field vector is

$$h(x) = Wx + b.$$

A neuron updates its state according to the sign of its local field. To avoid ambiguity when the field is exactly zero, we adopt the tie-preserving rule

$$T_i(x) = \begin{cases} +1, & h_i(x) > 0, \\ -1, & h_i(x) < 0, \\ x_i, & h_i(x) = 0. \end{cases}$$

An *asynchronous update* changes one coordinate at a time, replacing x_i by $T_i(x)$ and leaving all other coordinates unchanged. A *synchronous update* changes all coordinates simultaneously using the fields of the old state. The distinction will be essential to the energy-descent theorem.

Remark 11.2 (Bias versus threshold convention)*.* The handwritten lecture also expresses the update as the sign of $\sum_{j \neq i} w_{ij} x_j - \theta_i$. This is exactly the convention above when $b_i = -\theta_i$. With the energy convention $-b^\top x$ used below, the local field must be $Wx + b$, *not* $Wx - b$. Either convention is valid when its corresponding energy is used consistently.

11.1.3 Fixed Points and Pattern Retrieval

Definition 11.3 (Fixed point)**.** A configuration $x^* \in \{-1, +1\}^N$ is an asynchronous-update *fixed point* if every coordinate satisfies

$$T_i(x^*) = x_i^*, \qquad i = 1, \ldots, N.$$

Equivalently, under the tie-preserving rule,

$$x_i^* h_i(x^*) \geq 0 \qquad \text{for every } i.$$

It is a *strict fixed point* if $x_i^* h_i(x^*) > 0$ for every i.

An intended memory should be a fixed point, preferably one that attracts nearby states. If a corrupted input evolves into the intended pattern, the network has performed pattern completion. The model does not, however, guarantee that the selected attractor is the nearest memory under every choice of weights.

11.2 Storing Memories with Hebbian Learning

Suppose we want to store m binary patterns, called *memories*,

$$\xi^{(1)}, \dots, \xi^{(m)} \in \{-1, +1\}^N.$$

It is useful to place the patterns into a matrix

$$X = \begin{bmatrix} \xi^{(1)} & \xi^{(2)} & \cdots & \xi^{(m)} \end{bmatrix} \in \mathbb{R}^{N \times m}.$$

A simple learning rule adjusts connections according to whether two neurons tend to share the same state across the stored patterns.

11.2.1 Correlation and the Hebbian Weight Matrix

Definition 11.4 (Normalized Hebbian storage rule)**.** For $i \neq j$, define the weight

$$w_{ij} = \frac{1}{m} \sum_{s=1}^{m} \xi_i^{(s)} \xi_j^{(s)}, \qquad w_{ii} = 0.$$

Equivalently, the complete weight matrix is

$$W = \frac{1}{m} \sum_{s=1}^{m} \xi^{(s)} (\xi^{(s)})^\top - I_N = \frac{1}{m} X X^\top - I_N.$$

The factor $1/m$ controls the scale of the weights; alternative normalizations, particularly $1/N$, also occur in the Hopfield-network literature.

The diagonal subtraction is exact because each pattern is binary:

$$\begin{aligned} \left(\frac{1}{m} X X^\top \right)_{ii} &= \frac{1}{m} \sum_{s=1}^{m} \xi_i^{(s)} \xi_i^{(s)} \\ &= \frac{1}{m} \sum_{s=1}^{m} (\xi_i^{(s)})^2 \\ &= \frac{1}{m} \sum_{s=1}^{m} 1 = 1. \end{aligned}$$

Thus subtracting I_N removes all self-connections while preserving all off-diagonal correlations.

For a fixed pair $i \neq j$, the product $\xi_i^{(s)}\xi_j^{(s)}$ is $+1$ when the two neurons share the same sign and -1 otherwise. Hence

$$\begin{aligned} w_{ij} &= \frac{1}{m}\Big(\#\{s : \xi_i^{(s)} = \xi_j^{(s)}\} - \#\{s : \xi_i^{(s)} \neq \xi_j^{(s)}\}\Big) \\ &= 2\frac{\#\{s : \xi_i^{(s)} = \xi_j^{(s)}\}}{m} - 1. \end{aligned}$$

This is an empirical co-activation statistic, not necessarily a centered covariance. Since $|\xi_i^{(s)}\xi_j^{(s)}| = 1$, the normalized weights satisfy $|w_{ij}| \leq 1$ for $i \neq j$.

Proposition 11.1 (Structure of the Hebbian matrix)**.** *Let $W = m^{-1}XX^\top - I_N$. Then*

1. *W is symmetric and has zero diagonal;*
2. *$W + I_N$ is positive semidefinite;*
3. *$\operatorname{rank}(W + I_N) \leq m$;*
4. *for every $v \in \mathbb{R}^N$,*

$$v^\top W v = \frac{1}{m}\|X^\top v\|_2^2 - \|v\|_2^2.$$

Proof. Symmetry follows from $(XX^\top)^\top = XX^\top$, while the diagonal calculation above proves $w_{ii} = 0$. Moreover, for every $v \in \mathbb{R}^N$,

$$\begin{aligned} v^\top(W + I_N)v &= \frac{1}{m}v^\top XX^\top v \\ &= \frac{1}{m}(X^\top v)^\top(X^\top v) \\ &= \frac{1}{m}\|X^\top v\|_2^2 \geq 0. \end{aligned}$$

Thus $W + I_N$ is positive semidefinite. Since X has m columns,

$$\operatorname{rank}(W + I_N) = \operatorname{rank}(XX^\top) = \operatorname{rank}(X) \leq m.$$

Subtracting $\|v\|_2^2$ from the preceding identity gives the last assertion. □

11.2.2 A Four-Neuron Example

Consider one intended memory,

$$\xi = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}^\top.$$

Since $m = 1$, Hebbian storage gives

$$\xi\xi^\top = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix},$$

$$W = \xi\xi^\top - I_4 = \begin{bmatrix} 0 & -1 & 1 & -1 \\ -1 & 0 & -1 & 1 \\ 1 & -1 & 0 & -1 \\ -1 & 1 & -1 & 0 \end{bmatrix}.$$

With zero bias, the field at the stored pattern is

$$\begin{aligned} W\xi &= (\xi\xi^\top - I_4)\xi \\ &= \xi(\xi^\top\xi) - \xi \\ &= (4-1)\xi = 3\xi. \end{aligned}$$

Every neuron therefore agrees with its local field. The pattern is a strict fixed point. Its sign-reversed counterpart $-\xi$ is also a strict fixed point, because

$$W(-\xi) = -W\xi = -3\xi = 3(-\xi).$$

This elementary example shows that one stored memory can correspond to more than one attractor.

Proposition 11.2 (Stability of a single stored pattern)**.** *Suppose $m = 1$, $b = 0$, and $N \geq 2$. With $W = \xi\xi^\top - I_N$, both ξ and $-\xi$ are strict fixed points.*

Proof. Because $\xi^\top\xi = N$,

$$W\xi = \xi(\xi^\top\xi) - \xi = (N-1)\xi.$$

Thus $\xi_i(W\xi)_i = N - 1 > 0$ for all i. Similarly, $(-\xi)_i(W(-\xi))_i = N - 1 > 0$. □

11.2.3 Multiple Memories and Interference

An intended memory $\xi^{(r)}$ is not automatically a fixed point when other patterns are stored. Under the normalized Hebbian rule and zero bias,

$$\begin{aligned} W\xi^{(r)} &= \left(\frac{1}{m}\sum_{s=1}^{m} \xi^{(s)}(\xi^{(s)})^\top - I_N\right)\xi^{(r)} \\ &= \frac{1}{m}\sum_{s=1}^{m} \xi^{(s)}\langle\xi^{(s)}, \xi^{(r)}\rangle - \xi^{(r)} \\ &= \left(\frac{N}{m} - 1\right)\xi^{(r)} + \frac{1}{m}\sum_{s\neq r} \xi^{(s)}\langle\xi^{(s)}, \xi^{(r)}\rangle. \end{aligned}$$

The first term reinforces the intended memory, while the second is *crosstalk interference* caused by correlations with the other stored patterns.

Proposition 11.3 (A sufficient fixed-point condition)**.** *With $b = 0$, the stored pattern $\xi^{(r)}$ is a strict fixed point whenever, for every coordinate i,*

$$\frac{N}{m} - 1 + \frac{1}{m} \sum_{s \neq r} \xi_i^{(r)} \xi_i^{(s)} \langle \xi^{(s)}, \xi^{(r)} \rangle > 0.$$

In particular, if all m memories are pairwise orthogonal and $N > m$, every stored memory is a strict fixed point.

Proof. By definition, strict stability is equivalent to $\xi_i^{(r)}(W\xi^{(r)})_i > 0$ for every i. Multiplying the preceding field identity by $\xi_i^{(r)}$ and using $(\xi_i^{(r)})^2 = 1$ produces the displayed condition. Under pairwise orthogonality, every term in the crosstalk sum vanishes, leaving $N/m - 1 > 0$. □

Remark 11.3 (What Hebbian storage guarantees). The weights encode correlations between training patterns, but there is no universal guarantee that all supplied patterns will remain strict fixed points. Highly correlated memories may interfere, and additional fixed points not equal to any stored memory may exist. The convergence theorem later in this lecture guarantees arrival at a stable configuration, not recovery of a particular desired memory.

11.3 The Hopfield Energy Function

11.3.1 From Pairwise Interactions to a Scalar Energy

Definition 11.5 (Hopfield energy)**.** For a symmetric zero-diagonal connection matrix W and bias b, the energy of the binary configuration x is

$$E(x) = -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} w_{ij} x_i x_j - \sum_{i=1}^{N} b_i x_i.$$

In matrix notation,

$$E(x) = -\frac{1}{2} x^\top W x - b^\top x, \qquad x \in \{-1, +1\}^N.$$

The factor $1/2$ corrects the double-counting of pairwise interactions: symmetry means the ordered pair (i, j) and the ordered pair (j, i) make equal contributions. Since $w_{ii} = 0$, the expression can also be written as

$$E(x) = - \sum_{1 \leq i < j \leq N} w_{ij} x_i x_j - b^\top x.$$

For one edge (i, j), if $w_{ij} > 0$, agreeing states make the interaction energy negative and thus favorable. If $w_{ij} < 0$, opposite states make it negative and favorable. The bias term $-b_i x_i$ favors alignment with the sign of b_i.

11.3.2 Exact Energy Change When One Neuron Flips

Lemma 11.1 (Single-coordinate energy difference)**.** *Let $x \in \{-1,+1\}^N$ and fix an index i. Let x' agree with x except possibly at coordinate i. Then*

$$E(x') - E(x) = -(x'_i - x_i)h_i(x), \qquad h_i(x) = \sum_{j \neq i} w_{ij}x_j + b_i.$$

If $x'_i = -x_i$, this becomes

$$E(x') - E(x) = 2x_i h_i(x).$$

Proof. All contributions to E that do not contain x_i cancel in the difference. Using symmetry $w_{ij} = w_{ji}$ and the zero diagonal,

$$\begin{aligned} E(x') - E(x) &= -\frac{1}{2}\sum_{j \neq i} w_{ij}(x'_i - x_i)x_j - \frac{1}{2}\sum_{j \neq i} w_{ji}x_j(x'_i - x_i) - b_i(x'_i - x_i) \\ &= -\sum_{j \neq i} w_{ij}(x'_i - x_i)x_j - b_i(x'_i - x_i) \\ &= -(x'_i - x_i)\left(\sum_{j \neq i} w_{ij}x_j + b_i\right) \\ &= -(x'_i - x_i)h_i(x). \end{aligned}$$

If $x'_i = -x_i$, then $x'_i - x_i = -2x_i$, which gives the second formula. □

The formula is exact: it does not use a small-step approximation or assume differentiability of the binary state space. In particular, an energy-reducing flip is available at i precisely when $x_i h_i(x) < 0$. A strict fixed point has positive energy cost for every single-bit flip.

11.3.3 Energy Descent and Finite-Time Convergence

Proposition 11.4 (Energy cannot increase under asynchronous updates)**.** *Assume $W = W^\top$, $w_{ii} = 0$, and the tie-preserving update rule. If x' is obtained from x by updating exactly one coordinate i, then*

$$E(x') \leq E(x).$$

If that update changes the neuron's state, then the inequality is strict.

Proof. If $h_i(x) = 0$, the tie-preserving rule leaves x_i unchanged, so the energy is constant. If $h_i(x) \neq 0$, the update sets $x'_i = \operatorname{sign}(h_i(x))$. By the energy-difference lemma,

$$\begin{aligned} E(x') - E(x) &= -(x'_i - x_i)h_i(x) \\ &= -x'_i h_i(x) + x_i h_i(x) \\ &= -|h_i(x)| + x_i h_i(x) \\ &\leq -|h_i(x)| + |x_i|\,|h_i(x)| \\ &= 0. \end{aligned}$$

Equality can occur only when the old state already agrees with the nonzero local field, in which case $x_i' = x_i$. Thus any actual state change strictly decreases energy. □

Proposition 11.5 (Finite convergence to a fixed point)**.** *Assume the hypotheses of the preceding proposition. Suppose the asynchronous update schedule is* fair*: every coordinate is revisited until no coordinate can change. Then, starting from any initial binary state, the system makes finitely many state-changing updates and ends at a fixed point.*

Proof. There are only 2^N binary configurations. Every state-changing update produces a strict decrease in E. Hence a configuration can never be revisited after a state change: revisiting the same configuration would reproduce the same energy, contradicting strict descent. Thus at most $2^N - 1$ state changes are possible.

After the last state change, the state remains constant. If it were not a fixed point, at least one coordinate would disagree with its nonzero local field. Fairness would eventually select that coordinate and force another state change, contradicting the choice of the last change. Therefore the terminal state is a fixed point. □

Remark 11.4 (Necessary assumptions)*.* This proof relies on symmetry, zero self-connections, asynchronous updates, and the rule that ties do not flip a neuron's state. With synchronous updates, the energy need not decrease after each iteration and the network can exhibit two-cycles. For example, for two neurons with

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}, \qquad b = 0,$$

a synchronous update sends $(+1, +1)$ to $(-1, -1)$ and back to $(+1, +1)$ indefinitely. Each such state has energy $+1$. By contrast, asynchronous updates settle in one of the two opposite-sign states, each of energy -1.

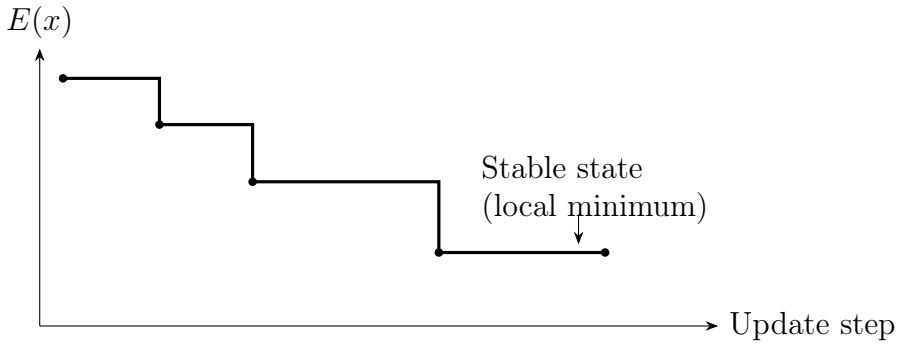


Figure 28: Schematic energy evolution under asynchronous binary updates. Energy decreases strictly when a neuron flips and remains unchanged for an update that leaves the state fixed.

11.4 Energy Minimization and Gradient Calculus

The energy-descent theorem operates on a discrete state space, where a coordinate can only take the values ± 1. It is nevertheless useful to extend the polynomial energy expression to all of $\mathbb{R}^N$ and compute ordinary gradients. These continuous derivatives help explain the relation between Hopfield dynamics and optimization; they should not be mistaken for a proof that the discrete network follows a continuous gradient-flow trajectory.

11.4.1 Gradient with Respect to the State

Proposition 11.6 (Continuous-state energy gradient)**.** *Consider the extension $E : \mathbb{R}^N \to \mathbb{R}$,*

$$E(x) = -\frac{1}{2} x^\top W x - b^\top x,$$

where W is a fixed matrix. Its Euclidean gradient is

$$\nabla_x E(x) = -\frac{1}{2}(W + W^\top)x - b.$$

For symmetric W, this reduces to

$$\nabla_x E(x) = -Wx - b.$$

Proof. Expanding the quadratic form in coordinates gives

$$E(x) = -\frac{1}{2} \sum_{j=1}^{N} \sum_{k=1}^{N} x_j w_{jk} x_k - \sum_{j=1}^{N} b_j x_j.$$

For each i,

$$\begin{aligned} \frac{\partial E}{\partial x_i} &= -\frac{1}{2} \sum_{k=1}^{N} w_{ik} x_k - \frac{1}{2} \sum_{j=1}^{N} x_j w_{ji} - b_i \\ &= -\frac{1}{2}\big((Wx)_i + (W^\top x)_i\big) - b_i. \end{aligned}$$

Stacking these N derivatives yields the claim. If $W = W^\top$, the two matrix terms coincide. □

The negative continuous gradient is $Wx + b$, precisely the local-field vector. The binary update uses the *sign* of the field rather than an infinitesimal displacement in that direction:

$$-\nabla_x E(x) = Wx + b, \qquad T_i(x) = \operatorname{sign}(h_i(x)) \quad \text{when } h_i(x) \neq 0.$$

Remark 11.5 (Correct matrix derivative). The expression $-\frac{1}{2}(W - W^\top)x - b$ sometimes appears in informal handwritten manipulations. It is not the derivative of the stated energy; the correct expression is $-\frac{1}{2}(W + W^\top)x - b$. In particular, the minus-sign expression would incorrectly remove the quadratic interaction whenever W is symmetric.

11.4.2 Gradient with Respect to the Weight Matrix

Now hold the state x fixed and treat the entries of W as variables. By the trace identity

$$x^\top W x = \operatorname{tr}(x^\top W x) = \operatorname{tr}(W x x^\top),$$

we can write the weight-dependent energy as

$$E(W \mid x) = -\frac{1}{2}\operatorname{tr}(W x x^\top) - b^\top x.$$

Proposition 11.7 (Unconstrained weight and bias gradients)**.** *For fixed x, considering all N^2 matrix entries independently,*

$$\nabla_W E(W \mid x) = -\frac{1}{2} x x^\top, \qquad \nabla_b E(W \mid x) = -x.$$

In coordinates,

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{2} x_i x_j, \qquad \frac{\partial E}{\partial b_i} = -x_i.$$

Proof. Expanding gives

$$E(W \mid x) = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{ij} x_i x_j - \sum_{i=1}^{N} b_i x_i.$$

Holding x fixed and differentiating with respect to an individual entry w_{ij} gives

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{2} x_i x_j.$$

Similarly, $\partial E/\partial b_i = -x_i$. Collecting the entries produces the matrix gradient. □

Remark 11.6 (Tied symmetric parameters). The preceding derivative regards w_{ij} and w_{ji} as independent variables. If symmetry is enforced by parameterizing the unordered edge $i < j$ with a single number $a_{ij} = w_{ij} = w_{ji}$, then

$$E(W \mid x) = -\sum_{i<j} a_{ij} x_i x_j - b^\top x \quad \Longrightarrow \quad \frac{\partial E}{\partial a_{ij}} = -x_i x_j.$$

There is no contradiction: a tied edge parameter changes *two* entries of W, so its derivative has twice the magnitude of the unconstrained entrywise derivative.

11.4.3 Projecting onto Symmetric Zero-Diagonal Matrices

A raw gradient step need not respect a parameter constraint. Define the linear space

$$\mathcal{S}_0 := \{A \in \mathbb{R}^{N\times N} : A = A^\top,\ A_{ii} = 0\}.$$

Its orthogonal projection in the Frobenius inner product is

$$\mathcal{P}_0(A) := \frac{1}{2}(A + A^\top) - \operatorname{diag}\left(\frac{1}{2}(A + A^\top)\right),$$

where $\operatorname{diag}(M)$ denotes the diagonal matrix formed from the diagonal entries of M.

Proposition 11.8 (Projected gradient rule)**.** *Given $W \in \mathcal{S}_0$, a constrained descent step for a differentiable objective $J(W)$ can be written as*

$$W^+ = W - \eta \mathcal{P}_0(\nabla_W J(W)), \qquad \eta > 0.$$

Then $W^+ \in \mathcal{S}_0$. For the single-pattern Hopfield energy, this specializes to

$$W^+ = W + \frac{\eta}{2}(xx^\top - I_N).$$

Proof. The matrix $\mathcal{P}_0(A)$ is symmetric by construction and has zero diagonal. Since $\mathcal{S}_0$ is a vector space, the difference between $W \in \mathcal{S}_0$ and a scalar multiple of $\mathcal{P}_0(A)$ also lies in $\mathcal{S}_0$. For $J(W) = E(W \mid x)$, the unconstrained gradient is $-xx^\top/2$. Because $xx^\top$ is symmetric and has all diagonal entries equal to one,

$$\begin{aligned}\mathcal{P}_0(\nabla_W E) &= \mathcal{P}_0\left(-\frac{1}{2}xx^\top\right) \\ &= -\frac{1}{2}(xx^\top - I_N).\end{aligned}$$

Substitution proves the update formula. □

Remark 11.7 (Projection versus adding the identity). Simply writing $\nabla_W E = -xx^\top + I_N$ changes the true unconstrained gradient. This expression may, however, be used to encode the desired *off-diagonal* update direction: $-xx^\top + I_N$ has zero diagonal and agrees with the negative off-diagonal correlation direction up to a constant learning-rate convention. Projection makes the constraint explicit and avoids confusing a gradient with a corrected update rule.

11.5 Learning the Correlation Matrix from an Objective

The Hebbian storage matrix can also be characterized as the solution of a simple regularized matrix optimization problem. This provides a precise connection between correlation storage and gradient-based learning, while also explaining why minimizing the bare Hopfield energy over W is not by itself a well-posed learning objective.

11.5.1 Why Minimizing Stored-Pattern Energy Alone Is Insufficient

Suppose we average the energy of the m memories while holding $b = 0$:

$$\begin{aligned}\overline{E}(W) &:= \frac{1}{m}\sum_{s=1}^{m} E(\xi^{(s)} \mid W) \\ &= -\frac{1}{2m}\sum_{s=1}^{m} (\xi^{(s)})^\top W \xi^{(s)} \\ &= -\frac{1}{2}\operatorname{tr}\left(W\frac{1}{m}XX^\top\right).\end{aligned}$$

This function is linear in W. If the coefficient has any nonzero component in the feasible zero-diagonal space, multiplying a favorable W by an arbitrarily large positive constant drives the average energy to $-\infty$. Therefore the unregularized average energy generally has no finite minimizer over all symmetric zero-diagonal weights.

11.5.2 A Regularized Correlation Objective

Definition 11.6 (Regularized Hebbian objective)**.** Fix the pattern matrix X and define the symmetric correlation matrix

$$C := \frac{1}{m}XX^\top.$$

For $W \in \mathcal{S}_0$, let

$$J(W) := \frac{1}{2}\|W\|_F^2 - \langle W, C\rangle_F,$$

where $\langle A, B\rangle_F := \operatorname{tr}(A^\top B)$ and $\|A\|_F^2 := \langle A, A\rangle_F$.

Proposition 11.9 (Hebbian weights as a unique optimum)**.** *The unique minimizer of J over $\mathcal{S}_0$ is the normalized Hebbian matrix*

$$W^* = C - I_N = \frac{1}{m}XX^\top - I_N.$$

Proof. Complete the square in the Frobenius norm:

$$\begin{aligned}J(W) &= \frac{1}{2}\|W\|_F^2 - \langle W, C\rangle_F \\ &= \frac{1}{2}\left(\|W\|_F^2 - 2\langle W, C\rangle_F + \|C\|_F^2\right) - \frac{1}{2}\|C\|_F^2 \\ &= \frac{1}{2}\|W - C\|_F^2 - \frac{1}{2}\|C\|_F^2.\end{aligned}$$

Thus minimizing J over $\mathcal{S}_0$ is the orthogonal projection of C onto $\mathcal{S}_0$. Since C is symmetric and $C_{ii} = 1$ for every i, this projection is $C - I_N$. Uniqueness follows from strict convexity of the squared Frobenius norm. □

The projected gradient of J is

$$\mathcal{P}_0(\nabla J(W)) = \mathcal{P}_0(W - C) = W - (C - I_N).$$

A projected gradient step with learning rate $\eta > 0$ becomes

$$\begin{aligned} W^{(k+1)} &= W^{(k)} - \eta\big(W^{(k)} - (C - I_N)\big) \\ &= (1 - \eta)W^{(k)} + \eta(C - I_N). \end{aligned}$$

Therefore, starting with $W^{(0)} \in \mathcal{S}_0$ and taking $0 < \eta < 2$, the parameter error evolves as

$$W^{(k+1)} - W^* = (1 - \eta)(W^{(k)} - W^*),$$

so

$$\|W^{(k)} - W^*\|_F = |1 - \eta|^k \|W^{(0)} - W^*\|_F \longrightarrow 0.$$

At $\eta = 1$, the projection produces the Hebbian matrix in a single step.

Remark 11.8 (Energy learning versus retrieval). Two different procedures appear in Hopfield networks:

1. **Storage:** choose W from example memories, e.g. by Hebbian correlation or the regularized objective above.
2. **Retrieval:** hold W fixed and update the binary state x to decrease the Hopfield energy.

The energy descent of the *state dynamics* should not be confused with optimizing the weight parameters using gradient descent.

11.6 Memory Retrieval, Distance, and Attractors

11.6.1 Hamming Distance for Binary Patterns

Definition 11.7 (Hamming distance)**.** For binary vectors $x, y \in \{-1, +1\}^N$, their Hamming distance is

$$d_H(x, y) := \#\{i : x_i \neq y_i\}.$$

Equivalently,

$$d_H(x, y) = \frac{N - x^\top y}{2}.$$

Derivation of the inner-product identity. For any coordinate i, the product $x_i y_i$ equals $+1$ if the two bits agree and -1 if they differ. If $d = d_H(x, y)$, then $N - d$ bits agree and d differ, so

$$\begin{aligned} x^\top y &= \sum_{i=1}^{N} x_i y_i \\ &= (N - d)(+1) + d(-1) \\ &= N - 2d. \end{aligned}$$

Solving for d gives $d = (N - x^\top y)/2$. $\square$

A corrupted cue can be modeled as an intended memory with a small number of flipped coordinates. A Hopfield network sometimes corrects these flips, but energy minimization is not generally equivalent to minimizing Hamming distance to the memory set.

Definition 11.8 (Basin of attraction)**.** Fix a deterministic asynchronous update schedule. For a fixed point x^*, its *basin of attraction* is the set of starting configurations whose iterated updates eventually reach x^*.

When an input belongs to the basin of a stored memory, retrieval succeeds. When it belongs to the basin of an unintended fixed point, the network may settle on a *spurious attractor*. In particular, the finite-convergence theorem only identifies the final configuration as a fixed point, without specifying which one.

11.6.2 A Worked Retrieval Example

Return to the four-neuron memory

$$\xi = (1, -1, 1, -1)^\top, \qquad W = \xi\xi^\top - I_4, \qquad b = 0.$$

Suppose a single bit is corrupted and the initial cue is

$$x^{(0)} = (-1, -1, 1, -1)^\top.$$

For neuron 1, the local field is

$$\begin{aligned} h_1(x^{(0)}) &= w_{12}x_2^{(0)} + w_{13}x_3^{(0)} + w_{14}x_4^{(0)} \\ &= (-1)(-1) + (1)(1) + (-1)(-1) \\ &= 3 > 0. \end{aligned}$$

Thus an asynchronous update at neuron 1 changes its state from -1 to $+1$, giving

$$x^{(1)} = (1, -1, 1, -1)^\top = \xi.$$

By the single-coordinate energy lemma, the energy change is

$$E(x^{(1)}) - E(x^{(0)}) = 2x_1^{(0)}h_1(x^{(0)}) = 2(-1)(3) = -6.$$

We can verify the result directly. Since $W = \xi\xi^\top - I_4$ and $b = 0$,

$$\begin{aligned} E(x) &= -\frac{1}{2}x^\top(\xi\xi^\top - I_4)x \\ &= -\frac{1}{2}\Big((\xi^\top x)^2 - x^\top x\Big) \\ &= -\frac{1}{2}\Big((\xi^\top x)^2 - 4\Big). \end{aligned}$$

The corrupted state differs in one bit, so $\xi^\top x^{(0)} = 4 - 2 = 2$, while $\xi^\top \xi = 4$. Therefore,

$$E(x^{(0)}) = -\frac{1}{2}(2^2 - 4) = 0,$$
$$E(\xi) = -\frac{1}{2}(4^2 - 4) = -6,$$
$$E(\xi) - E(x^{(0)}) = -6.$$

The network has corrected the flipped bit by following a direction of strictly decreasing energy.

11.7 Further Consequences of Symmetric Binary Networks

11.7.1 Existence of Stable States

Proposition 11.10 (Existence of a fixed point)**.** *Every symmetric zero-diagonal binary Hopfield network with a tie-preserving asynchronous update rule has at least one fixed point.*

Proof. Because $\{-1, +1\}^N$ is finite, E attains a global minimum at some x^*. If an update of coordinate i changed the state, the energy-descent proposition would imply $E(x') < E(x^*)$, contradicting global minimality. Therefore no coordinate can change under its update rule, and x^* is a fixed point. □

11.7.2 Fixed Points as Local Energy Minima

Proposition 11.11 (One-flip local optimality)**.** *A configuration x^* is an asynchronous fixed point under the tie-preserving rule if and only if*

$$E(x^*) \le E(x^{*(i)}) \qquad \text{for every } i,$$

where $x^{*(i)}$ is obtained from x^* by flipping only coordinate i.

Proof. For each i, the energy-difference lemma gives

$$E(x^{*(i)}) - E(x^*) = 2x_i^* h_i(x^*).$$

The right-hand side is nonnegative for every i exactly when $x_i^* h_i(x^*) \ge 0$ for every i. By the fixed-point definition, this is equivalent to $T_i(x^*) = x_i^*$ at every coordinate. □

A fixed point is therefore a *one-bit local minimum* of the discrete energy, not necessarily a global minimum. Local minima are the mathematical source of both pattern completion and retrieval errors.

11.7.3 Effects of Rescaling the Stored Weights

If $b = 0$ and W is multiplied by a positive scalar $c > 0$, the update rule does not change, because

$$\operatorname{sign}(cWx) = \operatorname{sign}(Wx)$$

for every nonzero local field. Moreover, the energy is multiplied by c:

$$E_{cW}(x) = -\frac{1}{2}x^\top (cW)x = cE_W(x).$$

Thus rescaling all weights by the same positive factor does not affect deterministic zero-bias fixed points or their asynchronous transitions, although it changes the numerical energy scale. With nonzero biases, the same invariance holds only if the biases are rescaled by c as well.

11.8 Exercises and Complete Solutions

Exercise 1: Energy Difference Under a Bit Flip.

Let $W = W^\top$ and $w_{ii} = 0$. Show that the energy difference between two configurations differing only in coordinate i is $2x_i((Wx)_i + b_i)$ when the coordinate is flipped. Explain why the sign rule lowers energy.

Solution. Denote the flipped configuration by x' and observe that $x'_i = -x_i$ while $x'_j = x_j$ for $j \neq i$. The terms in the energy not involving coordinate i cancel. Therefore,

$$\begin{aligned} E(x') - E(x) &= -\sum_{j \neq i} w_{ij}(x'_i - x_i)x_j - b_i(x'_i - x_i) \\ &= -(x'_i - x_i)\left(\sum_{j \neq i} w_{ij}x_j + b_i\right) \\ &= -(-2x_i)((Wx)_i + b_i) \\ &= 2x_i((Wx)_i + b_i). \end{aligned}$$

A flip is performed only when the old sign disagrees with the nonzero local field. In that case $x_i((Wx)_i + b_i) < 0$, making the energy difference strictly negative. ■

Exercise 2: Memory Correlation and Orthogonality.

Let $N = 4$ and consider the two patterns

$$\xi^{(1)} = (1, 1, -1, -1)^\top, \qquad \xi^{(2)} = (1, -1, 1, -1)^\top.$$

Compute their inner product and the normalized Hebbian weight matrix. Verify that each pattern is a strict fixed point when $b = 0$.

Solution. The inner product is

$$\begin{aligned} (\xi^{(1)})^\top \xi^{(2)} &= (1)(1) + (1)(-1) + (-1)(1) + (-1)(-1) \\ &= 1 - 1 - 1 + 1 = 0. \end{aligned}$$

Thus the memories are orthogonal. With $m = 2$,

$$W = \frac{1}{2}\left(\xi^{(1)}(\xi^{(1)})^\top + \xi^{(2)}(\xi^{(2)})^\top\right) - I_4$$
$$= \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{bmatrix}.$$

For $r = 1, 2$, orthogonality implies

$$W\xi^{(r)} = \left(\frac{4}{2} - 1\right)\xi^{(r)} + \frac{1}{2}\xi^{(3-r)}\langle\xi^{(3-r)}, \xi^{(r)}\rangle$$
$$= \xi^{(r)}.$$

Hence $\xi_i^{(r)}(W\xi^{(r)})_i = 1 > 0$ for every i, so each pattern is a strict fixed point. ■

Exercise 3: Asynchronous and Synchronous Updates.

Let

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}, \qquad b = 0, \qquad x^{(0)} = (1, 1)^\top.$$

Determine the synchronous trajectory and an asynchronous trajectory in which neuron 1 is updated first.

Solution. At $x^{(0)} = (1, 1)^\top$, the local-field vector is

$$Wx^{(0)} = (-1, -1)^\top.$$

Synchronous updating changes both coordinates, so

$$x^{(1)} = (-1, -1)^\top.$$

At this state the local fields are $(1, 1)^\top$, giving $x^{(2)} = (1, 1)^\top$. The trajectory therefore repeats with period two.

For asynchronous updating, first update neuron 1. Its field is -1, so

$$x^{(1)} = (-1, 1)^\top.$$

Now $Wx^{(1)} = (-1, 1)^\top$, so both coordinates agree with their local fields. The trajectory has reached a fixed point. The initial energy is

$$E(1, 1) = -w_{12}(1)(1) = 1,$$

whereas the terminal energy is

$$E(-1, 1) = -w_{12}(-1)(1) = -1.$$

Thus asynchronous dynamics decrease energy while synchronous dynamics cycle. ■

Exercise 4: Deriving the Hebbian Objective.

Let $C = m^{-1}XX^\top$. Solve

$$\min_{W=W^\top,\,\mathrm{diag}(W)=0} \left\{ \frac{1}{2}\|W\|_F^2 - \langle W, C\rangle_F \right\}.$$

Solution. Expanding the square yields

$$\frac{1}{2}\|W\|_F^2 - \langle W, C\rangle_F = \frac{1}{2}\|W - C\|_F^2 - \frac{1}{2}\|C\|_F^2.$$

Because C is symmetric, for every off-diagonal entry the minimum occurs at $w_{ij} = C_{ij}$. The diagonal entries must equal zero. Therefore the unique minimizer is the matrix obtained from C by setting its diagonal to zero. Since $C_{ii} = 1$, this matrix is

$$W^* = C - I_N = \frac{1}{m}XX^\top - I_N.$$

This is exactly the normalized Hebbian rule. ■

12 Lecture 13: Restricted Boltzmann Machines and Energy-Based Generative Learning

A *Restricted Boltzmann machine* (RBM) is a probabilistic neural network that learns a distribution over observed data. It is closely related to the energy-based Hopfield networks introduced in Lecture 12, but it differs in an important way: rather than merely evolving toward an energy minimum, an RBM assigns a probability to every possible configuration and is trained to assign high probability to observed examples. It can subsequently generate new data by sampling from its learned probability distribution.

The central mathematical objects are an *energy function*, a *Boltzmann probability distribution*, and a partition function. We first establish how these quantities determine the joint and visible-state distributions, then derive maximum-likelihood learning and its *positive* and *negative* phases. Finally, we explain why Gibbs sampling is especially convenient for the restricted architecture and how it leads to practical approximate learning rules.

12.1 Architecture of a Restricted Boltzmann Machine

12.1.1 Visible and Hidden Units

Definition 12.1 (Binary restricted Boltzmann machine)**.** An RBM with N visible units and M hidden units has two binary state vectors,

$$v = (v_1, \dots, v_N)^\top \in \{0,1\}^N, \qquad h = (h_1, \dots, h_M)^\top \in \{0,1\}^M.$$

Its parameters are a weight matrix and two bias vectors,

$$W = (w_{ij}) \in \mathbb{R}^{N \times M}, \qquad b \in \mathbb{R}^N, \qquad c \in \mathbb{R}^M.$$

The visible units represent an observation or data point, while the hidden units represent unobserved features or latent factors. The word *restricted* means that edges connect visible units to hidden units, but there are no visible-to-visible or hidden-to-hidden edges.

The RBM is an undirected, bipartite graphical model. Every pair (v_i, h_j) may interact through weight w_{ij}. The absence of within-layer connections is crucial: conditional on one layer, all states in the other layer become independent. This will allow all hidden variables, and then all visible variables, to be sampled simultaneously.

Remark 12.1 (Symmetry of an undirected connection). An edge from v_i to h_j has the same weight when viewed from either endpoint. The forward affine transformation from visible to hidden uses $W^\top v$, while the reverse transformation from hidden to visible uses Wh. These are the same parameters, not independent learned matrices. The observation in the handwritten lecture that W and $W^\top$ define equivalent descriptions refers to exchanging which partition is designated visible and which hidden; W itself need not be square or symmetric.

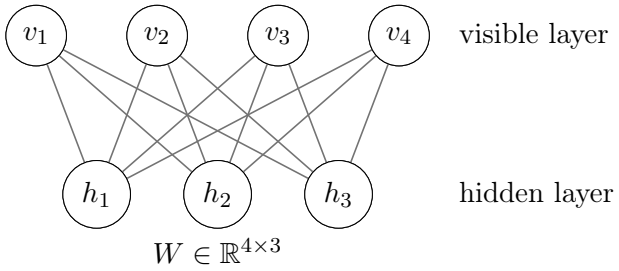


Figure 29: An RBM with four visible and three hidden units. There are no within-layer edges; all interactions pass between the layers.

12.1.2 Energy and the Boltzmann Law

Definition 12.2 (RBM energy)**.** The energy of a joint configuration (v, h) is

$$\begin{aligned} E_\theta(v,h) &= -\sum_{i=1}^{N}\sum_{j=1}^{M} v_i w_{ij} h_j - \sum_{i=1}^{N} b_i v_i - \sum_{j=1}^{M} c_j h_j \\ &= -v^\top W h - b^\top v - c^\top h, \end{aligned}$$

where $\theta = (W, b, c)$ denotes the collection of model parameters.

The interaction $-v^\top W h$ rewards aligned activations along positive-weight edges. The bias terms play roles analogous to external fields in statistical mechanics: b_i controls the baseline tendency for visible unit i to activate, and c_j controls the baseline tendency for hidden unit j to activate.

Definition 12.3 (Partition function and joint Boltzmann distribution)**.** The *partition function* is

$$Z_\theta := \sum_{v\in\{0,1\}^N} \sum_{h\in\{0,1\}^M} \exp\bigl(-E_\theta(v,h)\bigr).$$

The associated joint probability mass function is

$$p_\theta(v,h) := \frac{\exp\bigl(-E_\theta(v,h)\bigr)}{Z_\theta}.$$

Because the state space has 2^{N+M} elements and each Boltzmann factor is positive, $0 < Z_\theta < \infty$. Thus p_θ is a well-defined probability distribution. Moreover,

$$\sum_{v,h} p_\theta(v,h) = \frac{1}{Z_\theta}\sum_{v,h} e^{-E_\theta(v,h)} = 1.$$

For two states (v, h) and (v', h'),

$$\frac{p_\theta(v,h)}{p_\theta(v',h')} = \frac{e^{-E_\theta(v,h)}}{e^{-E_\theta(v',h')}} = e^{-(E_\theta(v,h)-E_\theta(v',h'))}.$$

In particular, lower-energy states are more probable. This is the precise meaning of the lecture's statement that an RBM visits lower-energy states more frequently, provided its sampling procedure has reached the intended distribution.

Remark 12.2 (Temperature convention). The general Boltzmann law has the form $p(v,h) \propto \exp\{-E(v,h)/T\}$ for temperature $T > 0$. Throughout these notes we set $T = 1$, absorbing inverse temperature into the model parameters. A deterministic minimum-energy state and a sample from the Boltzmann distribution are different objects: the latter can visit every finite-energy configuration.

12.2 Conditional Independence and Sampling Rules

12.2.1 Distribution of Hidden Units Given Visible Units

Suppose the visible configuration v is fixed. Terms that depend only on v cancel in the conditional distribution. We have

$$\begin{aligned} p_\theta(h \mid v) &= \frac{p_\theta(v,h)}{\sum_{\tilde{h}} p_\theta(v,\tilde{h})} \\ &= \frac{\exp\left(v^\top W h + b^\top v + c^\top h\right)}{\sum_{\tilde{h}} \exp\left(v^\top W \tilde{h} + b^\top v + c^\top \tilde{h}\right)} \\ &= \frac{\exp\left(\sum_{j=1}^{M} h_j (c_j + \sum_{i=1}^{N} w_{ij} v_i)\right)}{\sum_{\tilde{h}\in\{0,1\}^M} \exp\left(\sum_{j=1}^{M} \tilde{h}_j (c_j + \sum_{i=1}^{N} w_{ij} v_i)\right)}. \end{aligned}$$

Define the hidden pre-activation

$$a_j(v) := c_j + \sum_{i=1}^{N} w_{ij} v_i = c_j + (W^\top v)_j.$$

Because $h_j \in \{0,1\}$, the denominator factors:

$$\begin{aligned} \sum_{\tilde{h}\in\{0,1\}^M} \exp\left(\sum_{j=1}^{M} \tilde{h}_j a_j(v)\right) &= \sum_{\tilde{h}_1=0}^{1} \cdots \sum_{\tilde{h}_M=0}^{1} \prod_{j=1}^{M} e^{\tilde{h}_j a_j(v)} \\ &= \prod_{j=1}^{M} \left(\sum_{\tilde{h}_j=0}^{1} e^{\tilde{h}_j a_j(v)}\right) \\ &= \prod_{j=1}^{M} (1 + e^{a_j(v)}). \end{aligned}$$

Therefore,

$$p_\theta(h \mid v) = \prod_{j=1}^{M} \frac{e^{h_j a_j(v)}}{1 + e^{a_j(v)}}.$$

Proposition 12.1 (Independent Bernoulli hidden units)**.** *Conditional on a fixed visible vector v, the hidden units are independent Bernoulli random variables satisfying*

$$p_\theta(h_j = 1 \mid v) = \sigma\left(c_j + \sum_{i=1}^{N} w_{ij} v_i\right), \qquad \sigma(z) := \frac{1}{1+e^{-z}}.$$

In vector notation the conditional mean is

$$\mathbb{E}_\theta[h \mid v] = \sigma(W^\top v + c),$$

where σ is applied componentwise.

Proof. The factorization above shows conditional independence. Moreover,

$$p_\theta(h_j = 1 \mid v) = \frac{e^{a_j(v)}}{1+e^{a_j(v)}} = \frac{1}{1+e^{-a_j(v)}} = \sigma(a_j(v)).$$

Since a Bernoulli random variable has mean equal to its success probability, the conditional mean statement follows. □

12.2.2 Distribution of Visible Units Given Hidden Units

By reversing the roles of the two layers, define

$$d_i(h) := b_i + \sum_{j=1}^{M} w_{ij} h_j = b_i + (Wh)_i.$$

An analogous calculation yields

$$p_\theta(v \mid h) = \prod_{i=1}^{N} \frac{e^{v_i d_i(h)}}{1+e^{d_i(h)}},$$

$$p_\theta(v_i = 1 \mid h) = \sigma\left(b_i + \sum_{j=1}^{M} w_{ij} h_j\right),$$

$$\mathbb{E}_\theta[v \mid h] = \sigma(Wh + b).$$

Remark 12.3 (Deterministic activations versus binary samples). The quantities $\sigma(W^\top v + c)$ and $\sigma(Wh + b)$ are conditional probabilities or conditional means, not binary hidden and visible states. A genuine Gibbs sampler draws Bernoulli values using these probabilities. Merely replacing a binary layer by its mean is a deterministic approximation and does not, in general, sample from the RBM distribution.

12.3 The Visible Distribution and Free Energy

Training observations contain visible data v, but not the corresponding hidden states. The likelihood of a visible sample is thus obtained by *marginalizing* the hidden variables:

$$\begin{aligned} p_\theta(v) &= \sum_{h \in \{0,1\}^M} p_\theta(v,h) \\ &= \frac{1}{Z_\theta} \sum_{h \in \{0,1\}^M} e^{-E_\theta(v,h)}. \end{aligned}$$

Definition 12.4 (Unnormalized visible probability and free energy)**.** Define

$$\widetilde{p}_\theta(v) := \sum_h e^{-E_\theta(v,h)} \quad \text{and} \quad F_\theta(v) := -\log \widetilde{p}_\theta(v).$$

Then

$$p_\theta(v) = \frac{e^{-F_\theta(v)}}{Z_\theta}, \qquad Z_\theta = \sum_v e^{-F_\theta(v)}.$$

Proposition 12.2 (Explicit free-energy formula)**.** *For the binary RBM,*

$$F_\theta(v) = -b^\top v - \sum_{j=1}^{M} \log\left(1 + \exp\left(c_j + \sum_{i=1}^{N} w_{ij} v_i\right)\right).$$

Proof. Using the energy and factoring the hidden-state sum gives

$$\begin{aligned} \widetilde{p}_\theta(v) &= \sum_h \exp\left(b^\top v + \sum_{j=1}^{M} h_j a_j(v)\right) \\ &= e^{b^\top v} \prod_{j=1}^{M} \left(\sum_{h_j=0}^{1} e^{h_j a_j(v)}\right) \\ &= e^{b^\top v} \prod_{j=1}^{M} (1 + e^{a_j(v)}). \end{aligned}$$

Taking the negative logarithm gives

$$\begin{aligned} F_\theta(v) &= -\log \widetilde{p}_\theta(v) \\ &= -b^\top v - \sum_{j=1}^{M} \log(1 + e^{a_j(v)}). \end{aligned}$$

□

The function $\log(1 + e^z)$ is called the *softplus*. Thus marginalizing the hidden layer yields a tractable expression for the unnormalized likelihood of each visible pattern even though the full partition function typically remains expensive to compute.

Remark 12.4 (Computational difficulty). The sum defining Z_θ involves 2^{N+M} joint states. By eliminating the hidden states exactly, it can be reduced to a sum over only the 2^N visible configurations,

$$Z_\theta = \sum_{v \in \{0,1\}^N} e^{b^\top v} \prod_{j=1}^{M} (1 + e^{c_j + (W^\top v)_j}).$$

For large N, the remaining sum is still exponential. This difficulty motivates sampling-based approximations during learning.

12.3.1 Derivatives of the Free Energy

The softplus derivative is the logistic sigmoid:

$$\frac{\mathrm{d}}{\mathrm{d}z} \log(1 + e^z) = \frac{e^z}{1 + e^z} = \sigma(z).$$

Consequently,

$$\begin{aligned}
\frac{\partial F_\theta(v)}{\partial w_{ij}} &= -v_i \sigma\left(c_j + (W^\top v)_j\right), \\
\frac{\partial F_\theta(v)}{\partial b_i} &= -v_i, \\
\frac{\partial F_\theta(v)}{\partial c_j} &= -\sigma\left(c_j + (W^\top v)_j\right).
\end{aligned}$$

These are exact derivatives; no hidden-state sampling is necessary to evaluate them for a given visible input.

Proposition 12.3 (Free-energy derivative identity)**.** *For any model parameter ϑ appearing differentiably in E_θ,*

$$\frac{\partial F_\theta(v)}{\partial \vartheta} = \mathbb{E}_{p_\theta(h|v)} \left[\frac{\partial E_\theta(v,h)}{\partial \vartheta} \right].$$

Proof. Since $F_\theta(v) = -\log \sum_h e^{-E_\theta(v,h)}$,

$$\begin{aligned}
\frac{\partial F_\theta(v)}{\partial \vartheta} &= -\frac{\sum_h e^{-E_\theta(v,h)} \left(-\frac{\partial E_\theta(v,h)}{\partial \vartheta} \right)}{\sum_{\tilde{h}} e^{-E_\theta(v,\tilde{h})}} \\
&= \sum_h \frac{e^{-E_\theta(v,h)}}{\sum_{\tilde{h}} e^{-E_\theta(v,\tilde{h})}} \frac{\partial E_\theta(v,h)}{\partial \vartheta} \\
&= \sum_h p_\theta(h \mid v) \frac{\partial E_\theta(v,h)}{\partial \vartheta}.
\end{aligned}$$

□

12.4 Maximum-Likelihood Learning

12.4.1 Population and Empirical Likelihood

Suppose the training samples $v^{(1)}, \dots, v^{(n)}$ are observations of a random visible vector V with unknown data distribution p_{data}. We seek parameters for which the RBM visible marginal $p_\theta(v)$ approximates $p_{\text{data}}(v)$.

Definition 12.5 (Population negative log-likelihood)**.** The population negative log-likelihood, also called *cross-entropy risk*, is

$$\mathcal{L}(\theta) := \mathbb{E}_{V \sim p_{\text{data}}}[-\log p_\theta(V)].$$

For a finite training set the corresponding empirical objective is

$$\widehat{\mathcal{L}}_n(\theta) = -\frac{1}{n} \sum_{s=1}^{n} \log p_\theta(v^{(s)}).$$

Maximum likelihood seeks to minimize either the population risk, in theory, or its empirical approximation, in practice.

For a discrete empirical sample, maximizing the product of probabilities is equivalent to minimizing the empirical negative log-likelihood:

$$\begin{aligned} \arg\max_\theta \prod_{s=1}^{n} p_\theta(v^{(s)}) &= \arg\max_\theta \sum_{s=1}^{n} \log p_\theta(v^{(s)}) \\ &= \arg\min_\theta \left(-\frac{1}{n} \sum_{s=1}^{n} \log p_\theta(v^{(s)}) \right). \end{aligned}$$

Since $p_\theta(v) = e^{-F_\theta(v)}/Z_\theta$,

$$\begin{aligned} -\log p_\theta(v) &= -\log \left(\frac{e^{-F_\theta(v)}}{Z_\theta} \right) \\ &= F_\theta(v) + \log Z_\theta \\ &= -\log \left(\sum_h e^{-E_\theta(v,h)} \right) + \log \left(\sum_{\tilde{v},\tilde{h}} e^{-E_\theta(\tilde{v},\tilde{h})} \right). \end{aligned}$$

The first term concerns the observed data and is associated with the *positive phase*. The second term concerns the full model distribution and is associated with the *negative phase*.

Proposition 12.4 (Likelihood and Kullback–Leibler divergence)**.** *For a fixed data distribution* p_{data}*,*

$$\mathcal{L}(\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta),$$

where

$$H(p_{\text{data}}) = -\sum_v p_{\text{data}}(v) \log p_{\text{data}}(v),$$

$$D_{\text{KL}}(p_{\text{data}} \| p_\theta) = \sum_v p_{\text{data}}(v) \log \frac{p_{\text{data}}(v)}{p_\theta(v)}.$$

Thus maximum likelihood minimizes the forward KL divergence, because the entropy term does not depend on θ.

Proof. Expanding the KL divergence,

$$\begin{aligned} D_{\text{KL}}(p_{\text{data}} \| p_\theta) &= \sum_v p_{\text{data}}(v) \log p_{\text{data}}(v) - \sum_v p_{\text{data}}(v) \log p_\theta(v) \\ &= -H(p_{\text{data}}) + \mathcal{L}(\theta). \end{aligned}$$

Rearranging gives the result. □

12.4.2 Derivative of the Log-Partition Function

Lemma 12.1 (Log-partition derivative)**.** *For any scalar parameter ϑ of the RBM,*

$$\frac{\partial \log Z_\theta}{\partial \vartheta} = -\mathbb{E}_{p_\theta(v,h)} \left[\frac{\partial E_\theta(V, H)}{\partial \vartheta} \right].$$

Proof. Differentiating the finite sum gives

$$\begin{aligned} \frac{\partial \log Z_\theta}{\partial \vartheta} &= \frac{1}{Z_\theta} \frac{\partial Z_\theta}{\partial \vartheta} \\ &= \frac{1}{Z_\theta} \sum_{v,h} \frac{\partial}{\partial \vartheta} e^{-E_\theta(v,h)} \\ &= -\frac{1}{Z_\theta} \sum_{v,h} e^{-E_\theta(v,h)} \frac{\partial E_\theta(v,h)}{\partial \vartheta} \\ &= -\sum_{v,h} p_\theta(v,h) \frac{\partial E_\theta(v,h)}{\partial \vartheta}. \end{aligned}$$

□

12.4.3 The Two-Phase Learning Rule

Theorem 12.1 (Exact negative log-likelihood gradient)**.** *For one visible observation v, let $\ell_\theta(v) := -\log p_\theta(v)$. Then*

$$\nabla_\theta \ell_\theta(v) = \mathbb{E}_{p_\theta(h|v)}[\nabla_\theta E_\theta(v, H)] - \mathbb{E}_{p_\theta(\tilde{v},\tilde{h})}[\nabla_\theta E_\theta(\tilde{V}, \tilde{H})].$$

The first expectation is the positive phase (data clamped); the second is the negative phase (unclamped model).

Proof. Starting with the visible negative log-likelihood,

$$\ell_\theta(v) = -\log \sum_h e^{-E_\theta(v,h)} + \log Z_\theta.$$

Differentiating each term separately, the free-energy identity gives

$$\nabla_\theta \left[-\log \sum_h e^{-E_\theta(v,h)} \right] = \mathbb{E}_{p_\theta(h|v)}[\nabla_\theta E_\theta(v, H)],$$

and the log-partition identity gives

$$\nabla_\theta \log Z_\theta = -\mathbb{E}_{p_\theta(\tilde{v},\tilde{h})}[\nabla_\theta E_\theta(\tilde{V}, \tilde{H})].$$

Summing proves the claimed formula. □

For the population loss, averaging over observations leads to

$$\begin{aligned}\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{V\sim p_{\text{data}}} \mathbb{E}_{H\sim p_\theta(h|V)}[\nabla_\theta E_\theta(V, H)] \\ - \mathbb{E}_{(V,H)\sim p_\theta}[\nabla_\theta E_\theta(V, H)].\end{aligned}$$

The data expectation depends on the observed distribution; the model expectation must be computed or approximated using states generated by the current RBM.

Remark 12.5 (Physical interpretation). Gradient descent on the negative log-likelihood changes the parameters in the direction that lowers the energy of configurations compatible with the training data and raises the energy of configurations that the current model tends to produce. This is the probabilistic counterpart of fitting attractive low-energy states in Hopfield networks, with an additional competing term needed to normalize a probability distribution.

12.4.4 Exact Weight and Bias Gradients

For the binary energy,

$$\begin{aligned}\frac{\partial E_\theta(v,h)}{\partial w_{ij}} &= -v_i h_j, \\ \frac{\partial E_\theta(v,h)}{\partial b_i} &= -v_i, \\ \frac{\partial E_\theta(v,h)}{\partial c_j} &= -h_j.\end{aligned}$$

Substituting into the two-phase learning rule gives the exact population gradients:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_{ij}} &= -\mathbb{E}_{p_{\text{data}}(v)p_\theta(h|v)}[v_i h_j] + \mathbb{E}_{p_\theta(v,h)}[v_i h_j], \\ \frac{\partial \mathcal{L}}{\partial b_i} &= -\mathbb{E}_{p_{\text{data}}}[v_i] + \mathbb{E}_{p_\theta}[v_i], \\ \frac{\partial \mathcal{L}}{\partial c_j} &= -\mathbb{E}_{p_{\text{data}}(v)p_\theta(h|v)}[h_j] + \mathbb{E}_{p_\theta}[h_j].\end{aligned}$$

We use the notation

$$\langle f(V,H)\rangle_{\text{data}} := \mathbb{E}_{V\sim p_{\text{data}}}\mathbb{E}_{H\sim p_\theta(h|V)}[f(V,H)],$$
$$\langle f(V,H)\rangle_{\text{model}} := \mathbb{E}_{(V,H)\sim p_\theta}[f(V,H)].$$

Then the weight gradient becomes

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = -\langle V_i H_j\rangle_{\text{data}} + \langle V_i H_j\rangle_{\text{model}}.$$

Thus a gradient-descent update with learning rate $\eta > 0$ is

$$\begin{aligned} w_{ij}^{\text{new}} &= w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}} \\ &= w_{ij} + \eta\left(\langle V_i H_j\rangle_{\text{data}} - \langle V_i H_j\rangle_{\text{model}}\right). \end{aligned}$$

Similarly,

$$\begin{aligned} b_i^{\text{new}} &= b_i + \eta\left(\langle V_i\rangle_{\text{data}} - \langle V_i\rangle_{\text{model}}\right), \\ c_j^{\text{new}} &= c_j + \eta\left(\langle H_j\rangle_{\text{data}} - \langle H_j\rangle_{\text{model}}\right). \end{aligned}$$

Proposition 12.5 (Matrix-form gradient)**.** *Let $V \in \mathbb{R}^N$ and $H \in \mathbb{R}^M$ denote column state vectors. The gradient of the population negative log-likelihood with respect to W is*

$$\nabla_W \mathcal{L} = -\langle V H^\top\rangle_{\text{data}} + \langle V H^\top\rangle_{\text{model}}.$$

Consequently,

$$W^{\text{new}} = W + \eta\left(\langle V H^\top\rangle_{\text{data}} - \langle V H^\top\rangle_{\text{model}}\right).$$

Proof. The (i,j) entry of $VH^\top$ is $V_i H_j$. The entrywise gradient formula therefore gives the matrix identity. The update follows by subtracting $\eta\nabla_W\mathcal{L}$. □

The positive-phase moment can be obtained from the exact hidden conditional mean:

$$\begin{aligned} \langle V H^\top\rangle_{\text{data}} &= \mathbb{E}_{V\sim p_{\text{data}}}[V\mathbb{E}(H^\top \mid V)] \\ &= \mathbb{E}_{V\sim p_{\text{data}}}[V\sigma(W^\top V + c)^\top]. \end{aligned}$$

For the empirical dataset this is approximated by

$$\langle V H^\top\rangle_{\text{data}} \approx \frac{1}{n}\sum_{s=1}^{n} v^{(s)}\sigma(W^\top v^{(s)} + c)^\top.$$

Evaluating the model moment exactly is generally intractable because it requires an expectation over exponentially many joint configurations.

12.5 Blocked Gibbs Sampling

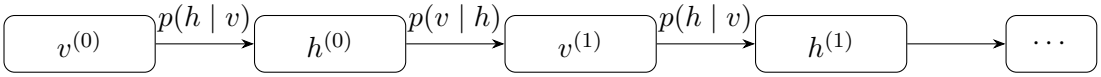
12.5.1 Alternating Conditional Draws

Because the two conditional distributions factorize, one can sample every hidden variable simultaneously once v is known, and sample every visible variable simultaneously once h is known.

Definition 12.6 (One full Gibbs step)**.** Given a visible configuration $v^{(t)}$, a full blocked Gibbs step consists of

$$h_j^{(t)} \sim \text{Bernoulli}\left(\sigma\left(c_j + (W^\top v^{(t)})_j\right)\right), \quad j = 1, \dots, M,$$
$$v_i^{(t+1)} \sim \text{Bernoulli}\left(\sigma\left(b_i + (W h^{(t)})_i\right)\right), \quad i = 1, \dots, N.$$

All draws within a layer are conditionally independent. After obtaining $v^{(t+1)}$, the process is repeated.



clamped data or initial state reconstruction

Figure 30: Alternating blocked Gibbs sampling in the RBM. The two layers are conditionally independent given each other, which permits parallel layerwise sampling.

12.5.2 Why the Sampler Targets the RBM Distribution

Proposition 12.6 (Invariance of the joint distribution)**.** *Suppose $(V, H) \sim p_\theta(v, h)$. If we replace H by an independent conditional draw $H' \sim p_\theta(h \mid V)$ while leaving V unchanged, then $(V, H') \sim p_\theta$. The analogous statement holds when the visible layer is resampled conditional on the hidden layer. Consequently, a full alternating Gibbs step preserves p_θ.*

Proof. For any v, h', the probability of the updated pair is

$$\begin{aligned}
\mathbb{P}(V = v, H' = h') &= \sum_h \mathbb{P}(V = v, H = h)\, p_\theta(h' \mid v) \\
&= p_\theta(h' \mid v) \sum_h p_\theta(v, h) \\
&= p_\theta(h' \mid v) p_\theta(v) \\
&= p_\theta(v, h').
\end{aligned}$$

The second update is proved in the same way, interchanging visible and hidden variables. Composing the updates therefore preserves the joint law. □

Remark 12.6 (Convergence versus stationary distribution). For finite parameters all Bernoulli conditional probabilities are strictly between 0 and 1. The induced Markov chain on the finite visible-state space has positive transition probabilities and therefore converges to its unique stationary visible marginal $p_\theta(v)$. This guarantees convergence in the infinite-time limit, but not that a small fixed number of Gibbs steps yields a sufficiently accurate model sample. Strong interactions can cause very slow mixing.

12.6 Contrastive Divergence and Practical Training

12.6.1 Approximating the Negative Phase

In an ideal maximum-likelihood update, we would generate exact samples from the equilibrium model distribution $p_\theta(v, h)$ and use them to estimate the model expectations. In large RBMs, waiting for a Gibbs chain to mix at every optimization step can be expensive.

Definition 12.7 (Contrastive divergence with k steps)**.** In *contrastive divergence*, denoted CD-k, each chain is initialized at a training example $v^{(0)}$. We then alternate hidden and visible Gibbs updates for k steps to obtain a reconstructed sample $v^{(k)}$. A practical weight update is

$$\Delta W_{\mathrm{CD}-k} = \eta \left[v^{(0)} \sigma(W^\top v^{(0)} + c)^\top - v^{(k)} \sigma(W^\top v^{(k)} + c)^\top \right].$$

For a mini-batch, the expression is averaged over the training examples and their associated chains.

One may alternatively use sampled hidden vectors $h^{(0)}$ and $h^{(k)}$ in both terms. Using their conditional means, as above, reduces Monte Carlo variability for those factors. The corresponding bias approximations are

$$\begin{aligned} \Delta b_{\mathrm{CD}-k} &= \eta \big(v^{(0)} - v^{(k)} \big), \\ \Delta c_{\mathrm{CD}-k} &= \eta \left[\sigma(W^\top v^{(0)} + c) - \sigma(W^\top v^{(k)} + c) \right]. \end{aligned}$$

Remark 12.7 (Why CD-k is only approximate). If $v^{(k)}$ were drawn exactly from $p_\theta(v)$, the expected negative term would equal the true model moment. In general the distribution after only k steps, initialized at a data sample, is not p_θ. Thus CD-k estimates a *short-chain* negative phase and does not provide an unbiased maximum-likelihood gradient for finite k. Moreover, the usual implementation does not differentiate through the sampling distribution's dependence on θ.

12.6.2 Persistent Contrastive Divergence

A second strategy maintains a collection of Gibbs chains across parameter updates, rather than restarting them at the observed data after every update. This is called *persistent contrastive divergence* (PCD). As the parameters change, the persistent chains are advanced a small number of steps and used to estimate the negative phase. If the learning dynamics evolve slowly enough relative to the mixing of the chains, persistent samples can track the model distribution more closely than very short chains repeatedly initialized at the data.

Neither CD-k nor PCD automatically guarantees exact maximum-likelihood gradients at a finite computational budget. Their role is to trade statistical accuracy in estimating the negative phase for practical efficiency.

12.7 A Fully Worked RBM Example

Consider $N = 2$ visible units and $M = 1$ hidden unit. Set

$$W = \begin{pmatrix} \log 2 \\ \log 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad c = 0.$$

Then the energy is

$$\begin{aligned} E(v_1, v_2, h) &= -v_1 h \log 2 - v_2 h \log 3, \\ e^{-E(v_1,v_2,h)} &= 2^{v_1 h} 3^{v_2 h}. \end{aligned}$$

12.7.1 Exact Partition Function and Visible Probabilities

For $h = 0$, every visible state has weight 1, so

$$\sum_{v_1,v_2} e^{-E(v_1,v_2,0)} = 4.$$

For $h = 1$, the weights factor:

$$\begin{aligned} \sum_{v_1,v_2} e^{-E(v_1,v_2,1)} &= \sum_{v_1=0}^{1} \sum_{v_2=0}^{1} 2^{v_1} 3^{v_2} \\ &= (1+2)(1+3) = 12. \end{aligned}$$

Hence $Z = 4 + 12 = 16$. Marginalizing h gives

$$p_\theta(v_1, v_2) = \frac{1 + 2^{v_1} 3^{v_2}}{16}.$$

The four visible probabilities are

(v_1, v_2)	$\widetilde{p}_\theta(v_1, v_2)$	$p_\theta(v_1, v_2)$
$(0, 0)$	2	2/16
$(1, 0)$	3	3/16
$(0, 1)$	4	4/16
$(1, 1)$	7	7/16

The probabilities sum to $(2 + 3 + 4 + 7)/16 = 1$.

12.7.2 Exact Conditionals

The hidden success probability is

$$\begin{aligned} p_\theta(h = 1 \mid v_1, v_2) &= \sigma(v_1 \log 2 + v_2 \log 3) \\ &= \frac{2^{v_1} 3^{v_2}}{1 + 2^{v_1} 3^{v_2}}. \end{aligned}$$

For instance,

$$p_\theta(h = 1 \mid 1, 0) = \frac{2}{3}, \qquad p_\theta(h = 1 \mid 1, 1) = \frac{6}{7}.$$

Given $h = 1$, the visible variables are independent and satisfy

$$\begin{aligned} p_\theta(v_1 = 1 \mid h = 1) &= \sigma(\log 2) = \frac{2}{3}, \\ p_\theta(v_2 = 1 \mid h = 1) &= \sigma(\log 3) = \frac{3}{4}. \end{aligned}$$

Given $h = 0$, both have conditional success probability $1/2$.

12.7.3 An Exact One-Sample Gradient Step

Suppose the observed data example is $v^{\text{data}} = (1, 0)^\top$. Then

$$\mathbb{E}_\theta[h \mid v^{\text{data}}] = \frac{2}{3}.$$

The positive-phase statistics are

$$\langle v_1 h \rangle_{\text{data}} = \frac{2}{3}, \qquad \langle v_2 h \rangle_{\text{data}} = 0, \qquad \langle h \rangle_{\text{data}} = \frac{2}{3}.$$

The exact model moments can be computed by direct enumeration. Since the $h = 1$ contribution to the unnormalized distribution is $2^{v_1} 3^{v_2}$,

$$\begin{aligned} \langle v_1 h \rangle_{\text{model}} &= \frac{2 + 6}{16} = \frac{1}{2}, \\ \langle v_2 h \rangle_{\text{model}} &= \frac{3 + 6}{16} = \frac{9}{16}, \\ \langle h \rangle_{\text{model}} &= \frac{12}{16} = \frac{3}{4}. \end{aligned}$$

For the visible variables,

$$\begin{aligned} \langle v_1 \rangle_{\text{model}} &= \frac{3 + 7}{16} = \frac{5}{8}, \\ \langle v_2 \rangle_{\text{model}} &= \frac{4 + 7}{16} = \frac{11}{16}. \end{aligned}$$

Thus one *exact* negative-log-likelihood gradient step with learning rate η yields

$$\Delta w_{11} = \eta \left(\frac{2}{3} - \frac{1}{2} \right) = \frac{\eta}{6},$$

$$\Delta w_{21} = \eta\left(0 - \frac{9}{16}\right) = -\frac{9\eta}{16},$$
$$\Delta b_1 = \eta\left(1 - \frac{5}{8}\right) = \frac{3\eta}{8},$$
$$\Delta b_2 = \eta\left(0 - \frac{11}{16}\right) = -\frac{11\eta}{16},$$
$$\Delta c_1 = \eta\left(\frac{2}{3} - \frac{3}{4}\right) = -\frac{\eta}{12}.$$

The update strengthens the observed co-activation (v_1, h) relative to the model's existing correlation, while suppressing the model's excessive tendency to activate v_2. This makes the positive-minus-negative phase interpretation concrete.

12.8 Further Mathematical Properties

12.8.1 The Log-Partition Function and Covariance

The model is an exponential family in its natural parameters. Writing the vector of sufficient statistics as

$$T(v, h) := \big(\{v_i h_j\}_{i,j}, \{v_i\}_i, \{h_j\}_j\big),$$

we have

$$p_\theta(v, h) = \exp\{\theta^\top T(v, h) - A(\theta)\}, \qquad A(\theta) = \log Z_\theta,$$

where θ is the concatenation of entries of W, b, c.

Proposition 12.7 (Moment and covariance identities)**.** *For the joint exponential-family representation,*

$$\nabla_\theta A(\theta) = \mathbb{E}_\theta[T(V, H)],$$
$$\nabla^2_\theta A(\theta) = \mathrm{Cov}_\theta(T(V, H), T(V, H)).$$

Consequently A is convex in the RBM parameters.

Proof. Differentiate the finite sum $Z_\theta = \sum_{v,h} e^{\theta^\top T(v,h)}$:

$$\frac{\partial A}{\partial \theta_a} = \frac{\sum_{v,h} T_a(v, h) e^{\theta^\top T(v,h)}}{Z_\theta} = \mathbb{E}_\theta[T_a].$$

Differentiating again,

$$\frac{\partial^2 A}{\partial \theta_a \partial \theta_b} = \mathbb{E}_\theta[T_a T_b] - \mathbb{E}_\theta[T_a]\mathbb{E}_\theta[T_b]$$
$$= \mathrm{Cov}_\theta(T_a, T_b).$$

For any vector u, the quadratic form of the Hessian is

$$u^\top \nabla^2 A(\theta) u = \mathrm{Var}_\theta(u^\top T(V, H)) \geq 0.$$

Hence the Hessian is positive semidefinite and A is convex. □

Remark 12.8 (Why RBM likelihood optimization can be nonconvex). Although the log-partition function is convex in the joint exponential-family parameters, the observed-data log-likelihood contains a hidden-state log-sum-exp term. The difference of these convex expressions is not generally concave or convex. Thus training an RBM may involve local optima, plateaus, and sensitivity to initialization.

12.8.2 Connection with the Hopfield Network

Both Hopfield networks and RBMs employ an energy function with pairwise interactions and biases. A binary Hopfield network evolves a single layer of spins and often retrieves stored patterns through asynchronous energy descent. An RBM instead has visible and hidden binary variables, assigns a normalized probability to their joint states, and learns generatively by maximum likelihood.

The absence of within-layer RBM connections enables tractable conditional distributions and alternating Gibbs updates. By contrast, dense within-layer interactions in the Hopfield model generally prevent such parallel conditional sampling. The RBM's generative objective is not simply to minimize energy at training points: the partition function penalizes making too many other configurations highly probable.

12.9 Exercises and Solutions

Exercise 1.

Prove directly from the conditional distribution that the log-odds of hidden activation are affine in the visible state:

$$\log \frac{p_\theta(h_j = 1 \mid v)}{p_\theta(h_j = 0 \mid v)} = c_j + (W^\top v)_j.$$

Solution. Writing $a_j(v) = c_j + (W^\top v)_j$, we have

$$\frac{p_\theta(h_j = 1 \mid v)}{p_\theta(h_j = 0 \mid v)} = \frac{e^{a_j(v)}/(1 + e^{a_j(v)})}{1/(1 + e^{a_j(v)})} = e^{a_j(v)}.$$

Taking logarithms gives the claim. ■

Exercise 2.

Consider a single visible and a single hidden unit, with parameters $w, b, c \in \mathbb{R}$. Find Z and the visible marginal exactly.

Solution. The energy is $E(v,h) = -wvh - bv - ch$. Sum first over h:

$$\begin{aligned} Z &= \sum_{v=0}^{1} e^{bv} \sum_{h=0}^{1} e^{h(wv+c)} \\ &= (1 + e^{c}) + e^{b}(1 + e^{w+c}). \end{aligned}$$

Therefore,

$$\begin{aligned} p_\theta(v=0) &= \frac{1 + e^{c}}{Z}, \\ p_\theta(v=1) &= \frac{e^{b}(1 + e^{w+c})}{Z}. \end{aligned}$$

Their sum is one by construction. ■

Exercise 3.

Show that the free-energy difference determines the relative probability of two visible configurations v, v'.

Solution. The partition function cancels:

$$\frac{p_\theta(v)}{p_\theta(v')} = \frac{e^{-F_\theta(v)}/Z_\theta}{e^{-F_\theta(v')}/Z_\theta} = e^{-F_\theta(v)+F_\theta(v')}.$$

Hence the visible state with smaller free energy has greater marginal probability. ■

Exercise 4.

Show that a finite mixture of observed binary data can produce a positive phase exactly from hidden conditional means, without drawing hidden samples.

Solution. For the empirical data law $\widehat{p}_{\text{data}} = n^{-1} \sum_{s=1}^{n} \delta_{v^{(s)}}$, the tower property yields

$$\begin{aligned} \mathbb{E}_{\widehat{p}_{\text{data}}(v)p_\theta(h|v)}[V H^\top] &= \frac{1}{n} \sum_{s=1}^{n} \mathbb{E}_\theta[v^{(s)} H^\top \mid V = v^{(s)}] \\ &= \frac{1}{n} \sum_{s=1}^{n} v^{(s)} \mathbb{E}_\theta[H^\top \mid V = v^{(s)}] \\ &= \frac{1}{n} \sum_{s=1}^{n} v^{(s)} \sigma(W^\top v^{(s)} + c)^\top. \end{aligned}$$

Thus the finite-sample positive-phase moment can be calculated exactly for the current parameters. ■

Exercise 5.

For a model whose Gibbs chain has reached stationarity, explain why the sample average

$$\frac{1}{m}\sum_{r=1}^{m} v^{(r)} h^{(r)\top}$$

approximates the negative-phase statistic, and distinguish this from CD-1.

Solution. At stationarity, each joint state has marginal law $p_\theta(v, h)$, so by linearity of expectation,

$$\mathbb{E}\left[\frac{1}{m}\sum_{r=1}^{m} V^{(r)} H^{(r)\top}\right] = \mathbb{E}_{p_\theta}[V H^\top].$$

Dependence between successive Markov-chain samples affects the variance of the average, but does not change this expectation. Under ergodicity, long-run time averages converge to the model expectation. In CD-1, the chain starts at a data example and takes just one alternating Gibbs step; its final distribution generally differs from p_θ, so the corresponding negative-phase estimate need not have the correct expectation. ■

13 Lecture 14: Autoencoders, Dimensionality Reduction, and Word Embeddings

In the preceding lectures we studied discriminative neural networks, whose outputs are targets such as class labels, and energy-based generative networks, which model probability distributions over data. We now consider a different way to learn representations: a network is trained to reconstruct its own input. A compressed internal representation must retain the information most useful for reconstruction. We then turn from numerical representations of images and other data to representations of discrete words, where a dense vector can encode useful similarities that a one-hot vector cannot express directly.

The first part develops *autoencoders*, including reconstruction losses, bottleneck constraints, tied weights, and the relationship to principal-component analysis. The second part develops *word embeddings*, the skip-gram predictive model, the softmax objective, negative sampling, and geometric similarity. Throughout, we distinguish what follows directly from the lecture's architecture from additional mathematical consequences and practical extensions.

13.1 Autoencoders and Reconstruction Learning

13.1.1 The Encoder–Decoder Architecture

Definition 13.1 (Autoencoder)**.** Let $x \in \mathbb{R}^d$ denote an observation. An *autoencoder* consists of an encoder f_θ and a decoder g_ϕ, with

$$f_\theta : \mathbb{R}^d \longrightarrow \mathbb{R}^k, \qquad z = f_\theta(x),$$
$$g_\phi : \mathbb{R}^k \longrightarrow \mathbb{R}^d, \qquad \widehat{x} = g_\phi(z).$$

The vector z is the *latent representation* or *code*; $\widehat{x}$ is the reconstruction. The composite model is

$$\widehat{x} = (g_\phi \circ f_\theta)(x).$$

Training attempts to make the reconstructed observation close to the original observation in a specified reconstruction loss $\ell(x, \widehat{x})$.

A typical neural autoencoder takes the form

$$z = \sigma_e(W_e x + b_e),$$
$$\widehat{x} = \sigma_d(W_d z + b_d),$$

where $W_e \in \mathbb{R}^{k \times d}$, $W_d \in \mathbb{R}^{d \times k}$, $b_e \in \mathbb{R}^k$ and $b_d \in \mathbb{R}^d$. For deep autoencoders, each of the encoder and decoder is a composition of several transformations; the same conceptual separation still applies.

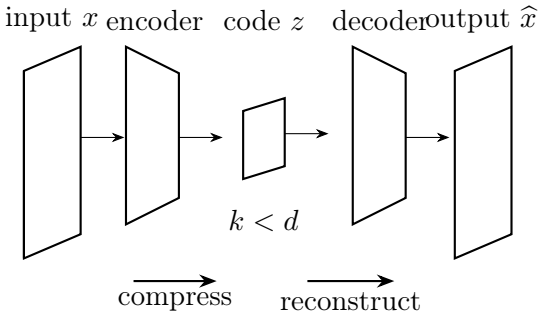


Figure 31: An undercomplete autoencoder. The encoder maps the observation to a lower-dimensional latent code; the decoder reconstructs the observation from that code.

Remark 13.1 (The autoencoder is not an ordinary supervised classifier). The input x is also the reconstruction target. There need not be a separate class label y. In a finite sample $\{x^{(1)}, \dots, x^{(m)}\}$, the training pairs can be regarded as $(x^{(s)}, x^{(s)})$ for $s = 1, \dots, m$. Because the method uses the observations themselves as targets, it is generally categorized as self-supervised representation learning.

13.1.2 Reconstruction Losses

Definition 13.2 (Empirical reconstruction risk)**.** For observations $x^{(1)}, \dots, x^{(m)}$, define

$$\widehat{\mathcal{L}}(\theta, \phi) = \frac{1}{m} \sum_{s=1}^{m} \ell\big(x^{(s)}, g_\phi(f_\theta(x^{(s)}))\big) .$$

The parameters are learned by seeking a minimizer

$$(\widehat{\theta}, \widehat{\phi}) \in \arg\min_{\theta, \phi} \widehat{\mathcal{L}}(\theta, \phi).$$

If an observation is drawn from an underlying distribution P_X, the corresponding population objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{X \sim P_X}[\ell(X, g_\phi(f_\theta(X)))] .$$

For continuous-valued observations, the conventional squared-error loss is

$$\ell_{\mathrm{MSE}}(x, \widehat{x}) = \frac{1}{d} \sum_{i=1}^{d} (x_i - \widehat{x}_i)^2 = \frac{1}{d} \|x - \widehat{x}\|_2^2.$$

The scaling by $1/d$ is a convention: removing it changes the gradient magnitude but not the set of minimizers in an otherwise unchanged optimization problem.

If the coordinates $x_i \in \{0, 1\}$ are modeled as conditionally independent Bernoulli variables and decoder outputs $\widehat{x}_i \in (0, 1)$ represent probabilities, binary cross-entropy provides an alternative:

$$\ell_{\mathrm{BCE}}(x, \widehat{x}) = -\sum_{i=1}^{d} [x_i \log \widehat{x}_i + (1 - x_i) \log(1 - \widehat{x}_i)] .$$

Its probabilistic justification follows from the Bernoulli likelihood:

$$p_\phi(x \mid z) = \prod_{i=1}^{d} \widehat{x}_i(z)^{x_i}(1-\widehat{x}_i(z))^{1-x_i},$$

$$-\log p_\phi(x \mid z) = -\sum_{i=1}^{d} \left[x_i \log \widehat{x}_i(z) + (1-x_i)\log(1-\widehat{x}_i(z))\right].$$

Hence minimizing binary cross-entropy is equivalent to maximizing conditional Bernoulli log-likelihood under this decoder model. Squared error similarly corresponds to a Gaussian observation model with fixed isotropic variance, up to constants and positive rescaling.

Remark 13.2 (Reconstruction and representation quality). A low reconstruction loss is evidence that the code retains information sufficient for the trained decoder to reconstruct examples. It does not, by itself, guarantee that the learned coordinates have a unique interpretation, form distinct semantic factors, or generalize well to unseen observations. The usefulness of z depends on the data, model restrictions, training procedure, and downstream task.

13.2 The Bottleneck and Dimensionality Reduction

13.2.1 Capacity of a Discrete Code

The handwritten lecture motivates compression using an eight-bit observation. A binary vector of length eight has

$$|\{0,1\}^8| = 2^8 = 256$$

possible configurations. By contrast, a three-bit code has

$$|\{0,1\}^3| = 2^3 = 8$$

possible configurations. A three-bit representation cannot losslessly encode all 256 eight-bit patterns by a deterministic encoder and decoder, but it can represent any chosen set of at most eight distinct patterns without collisions.

Proposition 13.1 (Exact reconstruction requires distinct codes)**.** *Let $\mathcal{X}$ be a finite collection of M distinct inputs and $\mathcal{Z}$ a finite code space with K distinct codes. If a deterministic encoder $f : \mathcal{X} \to \mathcal{Z}$ and decoder $g : \mathcal{Z} \to \mathcal{X}$ satisfy*

$$g(f(x)) = x \qquad \text{for every } x \in \mathcal{X},$$

then f is injective and $K \geq M$.

Proof. Suppose $f(x) = f(x')$ for $x, x' \in \mathcal{X}$. Applying the decoder to both sides gives

$$x = g(f(x)) = g(f(x')) = x'.$$

Thus distinct inputs must have distinct codes, so f is injective. An injective map from a set of M elements to a set of K elements requires $K \geq M$. □

Example 13.1 (An eight-bit observation with eight admissible patterns)**.** Suppose the data distribution is concentrated on the eight patterns

$$\mathcal{X} = \{(u_1, u_2, u_3, u_1, u_2, u_3, 0, 1) : u_1, u_2, u_3 \in \{0, 1\}\}.$$

Although each observation has eight bits, it is determined by only the first three. The code $z = (u_1, u_2, u_3)$ permits exact reconstruction by

$$g(z_1, z_2, z_3) = (z_1, z_2, z_3, z_1, z_2, z_3, 0, 1).$$

This illustrates the distinction between the ambient representation length and the smaller number of independent degrees of freedom actually needed for the data.

For real-valued codes, a k-dimensional bottleneck does *not* automatically imply that only 2^k inputs can be represented: each coordinate takes values in $\mathbb{R}$, not merely $\{0, 1\}$. Real-valued compression instead limits the form of the continuous representation, and further assumptions about continuity, network architecture, precision, or regularization determine the effective information capacity.

13.2.2 Undercomplete and Overcomplete Representations

Definition 13.3 (Latent dimensionality)**.** An autoencoder with input dimension d and latent dimension k is called *undercomplete* if $k < d$, *complete* if $k = d$, and *overcomplete* if $k > d$.

Undercompleteness encourages the network to compress, since all reconstructed information must pass through fewer coordinates. Nevertheless, with sufficient expressive power and a finite training set, an undercomplete network can sometimes memorize examples instead of learning useful structure. Conversely, an overcomplete network can learn meaningful representations if an additional constraint prevents a trivial identity mapping.

Common restrictions include ℓ_1 penalties on hidden activations, noise added to the input (denoising autoencoders), penalties on encoder sensitivity (contractive autoencoders), and restrictions on model capacity. These are extensions of the basic reconstruction architecture, rather than properties that follow from $k < d$ alone.

13.3 Linear Autoencoders and Principal-Component Analysis

A particularly instructive case arises when both encoder and decoder are linear. Its solution connects autoencoders to familiar linear-algebraic dimensionality reduction.

Definition 13.4 (Centered linear autoencoder)**.** Let $X \in \mathbb{R}^{d \times m}$ be a matrix whose columns are centered observations, so $X\mathbf{1}_m = 0$. A linear autoencoder with latent dimension k maps

$$\begin{aligned} z &= W_e x, & W_e &\in \mathbb{R}^{k \times d}, \\ \widehat{x} &= W_d z = W_d W_e x, & W_d &\in \mathbb{R}^{d \times k}. \end{aligned}$$

Its empirical squared reconstruction error is

$$J(W_d, W_e) = \frac{1}{m}\|X - W_d W_e X\|_F^2.$$

The matrix $A = W_d W_e$ has rank at most k. Therefore linear reconstruction is constrained to lie in the image of a rank-k linear map.

Theorem 13.1 (Optimal subspace of a linear autoencoder)**.** *Let $X = U\Sigma V^\top$ be a singular-value decomposition and let U_k contain k leading left singular vectors. For $0 \le k \le \operatorname{rank}(X)$, the minimum possible linear-autoencoder error is*

$$\min_{W_d, W_e} \frac{1}{m}\|X - W_d W_e X\|_F^2 = \frac{1}{m}\sum_{j>k} \sigma_j^2,$$

and one minimizer is given by

$$W_e = U_k^\top, \qquad W_d = U_k.$$

Thus the optimal k-dimensional reconstruction subspace is a principal-component subspace.

Proof. For any matrices W_d, W_e of the indicated sizes,

$$\operatorname{rank}(W_d W_e X) \le k.$$

The rank-k approximation theorem for the singular-value decomposition states that every matrix Y with $\operatorname{rank}(Y) \le k$ satisfies

$$\|X - Y\|_F^2 \ge \sum_{j>k} \sigma_j^2.$$

Hence the reconstruction error has at least this value. Choose $W_e = U_k^\top$ and $W_d = U_k$. Since the columns of U_k are orthonormal,

$$\begin{aligned} W_d W_e X &= U_k U_k^\top U \Sigma V^\top \\ &= U_k \Sigma_k V_k^\top, \end{aligned}$$

which is the truncated rank-k singular-value decomposition of X and attains the lower bound. Dividing by m completes the proof. □

Remark 13.3 (Nonuniqueness of the latent coordinates). Even if the optimal reconstruction subspace is uniquely determined, encoder and decoder weights generally are not. For any invertible $R \in \mathbb{R}^{k\times k}$,

$$(W_d R^{-1})(R W_e) = W_d W_e.$$

Thus a change of latent basis leaves all reconstructions unchanged. The representation coordinates themselves are not identifiable without further constraints.

13.4 Tied Weights in Autoencoders

The lecture next considers *tied weights*: the decoder uses the transpose of the encoder weight matrix instead of learning a completely independent matrix.

Definition 13.5 (Tied-weight autoencoder)**.** For $W \in \mathbb{R}^{k \times d}$, define

$$a = Wx + b_e, \qquad z = \sigma_e(a),$$
$$c = W^\top z + b_d, \qquad \widehat{x} = \sigma_d(c).$$

The decoder's matrix is $W^\top$. The network has kd distinct weight parameters rather than $2kd$, although it still has encoder and decoder bias parameters if those are included.

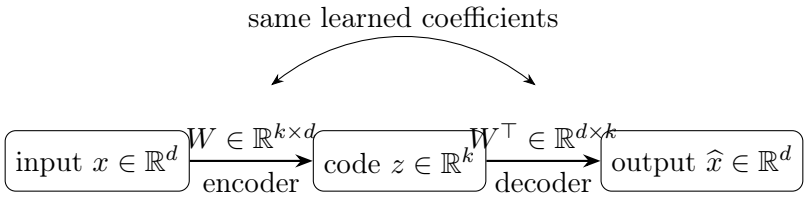


Figure 32: Tied encoder and decoder weights. The decoder uses the transpose of the encoder matrix; it does not introduce another independent set of matrix coefficients.

An important computational subtlety is that a shared parameter receives *gradient contributions from both of its appearances* in the computation graph. We derive these terms next.

13.4.1 Backpropagation Through Tied Weights

For a single observation, take the squared reconstruction loss

$$L(W, b_e, b_d) = \frac{1}{2}\|\widehat{x} - x\|_2^2.$$

Let $\odot$ denote coordinatewise multiplication. Introduce the decoder and encoder error vectors

$$\delta_d = \nabla_c L = (\widehat{x} - x) \odot \sigma_d'(c),$$
$$\delta_e = \nabla_a L = (W\delta_d) \odot \sigma_e'(a).$$

The second identity follows from $c = W^\top z + b_d$: its Jacobian with respect to z is $W^\top$, so the reverse-mode derivative is $W\delta_d$.

Proposition 13.2 (Tied-weight gradient)**.** *For the tied-weight autoencoder, the gradient with respect to the shared matrix W is*

$$\nabla_W L = \delta_e x^\top + z\delta_d^\top.$$

The bias gradients are $\nabla_{b_e} L = \delta_e$ and $\nabla_{b_d} L = \delta_d$.

Proof. The matrix W appears in two places. Its encoder occurrence is $a = Wx + b_e$ and its decoder occurrence is $c = W^\top z + b_d$. Consider an arbitrary variation $\mathrm{d}W$. The forward variations are

$$\begin{aligned}
\mathrm{d}a &= (\mathrm{d}W)x, \\
\mathrm{d}z &= \operatorname{diag}(\sigma_e'(a))(\mathrm{d}W)x, \\
\mathrm{d}c &= (\mathrm{d}W)^\top z + W^\top \mathrm{d}z.
\end{aligned}$$

Taking the differential of the loss and grouping its two contributions gives

$$\begin{aligned}
\mathrm{d}L &= \delta_d^\top \mathrm{d}c \\
&= \delta_d^\top (\mathrm{d}W)^\top z + \delta_d^\top W^\top \operatorname{diag}(\sigma_e'(a))(\mathrm{d}W)x \\
&= \delta_d^\top (\mathrm{d}W)^\top z + \delta_e^\top (\mathrm{d}W)x \\
&= \operatorname{tr}\big((z\delta_d^\top)^\top \mathrm{d}W\big) + \operatorname{tr}\big((\delta_e x^\top)^\top \mathrm{d}W\big) \\
&= \operatorname{tr}\big((\delta_e x^\top + z\delta_d^\top)^\top \mathrm{d}W\big).
\end{aligned}$$

By the definition of the matrix gradient, $\nabla_W L = \delta_e x^\top + z\delta_d^\top$. The bias formulas follow directly by differentiating their affine additions. □

Remark 13.4 (Why the second term matters). If the encoder and decoder weights were independent, their gradients would be $\delta_e x^\top$ and $\delta_d z^\top$, respectively. Because tied weights identify the decoder matrix with $W^\top$, a gradient step on the shared parameter must add the correctly transposed decoder contribution. Ignoring it would not be gradient descent on the stated tied-weight loss.

13.5 Discrete Words and Vector Embeddings

An image is naturally represented by numerical pixel measurements. A word is a discrete symbol, and its spelling or dictionary index is not a useful measure of semantic distance. To apply neural computation to words, we map each word to a learned vector.

13.5.1 One-Hot Encoding

Let $\mathcal{V} = \{w_1, \ldots, w_N\}$ be a vocabulary of N word types.

Definition 13.6 (One-hot word vector)**.** The one-hot encoding of vocabulary item w_i is the coordinate vector $e_i \in \mathbb{R}^N$ whose entries are

$$(e_i)_j = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases}$$

Thus one-hot word vectors are all unit vectors and are mutually orthogonal when the words differ.

For example, in a vocabulary containing "cat," "dog," and "car," the encodings may be

$$e_{\text{cat}} = (1, 0, 0)^\top,$$
$$e_{\text{dog}} = (0, 1, 0)^\top,$$
$$e_{\text{car}} = (0, 0, 1)^\top.$$

Every pair of distinct such vectors has inner product zero, cosine similarity zero, and Euclidean distance $\sqrt{2}$. Consequently one-hot encodings alone do not express that "cat" and "dog" are more semantically related to one another than either is to "car."

13.5.2 The Embedding Matrix

Definition 13.7 (Word embedding)**.** An embedding of the vocabulary into $\mathbb{R}^H$ assigns each word w_i a dense vector $u_i \in \mathbb{R}^H$. Collecting these vectors in a matrix

$$U = \begin{bmatrix} u_1 & \cdots & u_N \end{bmatrix} \in \mathbb{R}^{H \times N},$$

the embedding lookup is the linear operation

$$u_i = U e_i.$$

An embedding lookup is simply selection of a column of U: multiplication by e_i keeps column i and removes all others.

Proposition 13.3 (Gradient of an embedding lookup)**.** *Let $u = Ue_i$ and let $L(u)$ be a differentiable scalar loss. Then*

$$\nabla_U L = (\nabla_u L) e_i^\top.$$

In particular, the gradient is zero in every column except column i.

Proof. With e_i fixed, the differential of the lookup is $\mathrm{d}u = (\mathrm{d}U)e_i$. Therefore,

$$\begin{aligned} \mathrm{d}L &= (\nabla_u L)^\top \mathrm{d}u \\ &= (\nabla_u L)^\top (\mathrm{d}U) e_i \\ &= \operatorname{tr}\Big(\big((\nabla_u L) e_i^\top \big)^\top \mathrm{d}U \Big). \end{aligned}$$

The matrix-gradient identity follows. Since e_i is zero outside position i, only the corresponding column of the gradient can be nonzero. □

Remark 13.5 (Embedding dimensionality). If the vocabulary contains N words and the embedding dimension is H, the embedding matrix contains HN learned coefficients. Choosing $H \ll N$ makes the embedding of an individual word compact, but the vocabulary-wide parameter matrix may still be large. Further, word embeddings need not occupy all H dimensions independently or with fixed semantic meanings.

13.6 Context-Based Learning and the Skip-Gram Model

Words that frequently occur in similar contexts often acquire related learned representations. This motivates training embeddings from the task of predicting a neighboring word given a central word.

13.6.1 Context Windows and Training Pairs

Consider a token sequence $w_1, w_2, \ldots, w_T$. For a chosen context radius $c \in \mathbb{N}$, training pairs are formed from positions $(t, t+j)$ with $-c \le j \le c$, $j \neq 0$, and valid sequence indices. The central word is w_t, and each w_{t+j} is a context word.

For example, the sentence "a cat sleeps on the sofa" contains a training pair ("cat", "sleeps") if the context radius includes the next token. Such pairs supply the supervision: no semantic class labels are needed.

Definition 13.8 (Skip-gram predictive model)**.** Let $u_i \in \mathbb{R}^H$ be the *input* or center-word vector for word w_i, and $v_j \in \mathbb{R}^H$ the *output* or context-word vector for word w_j. Define the score

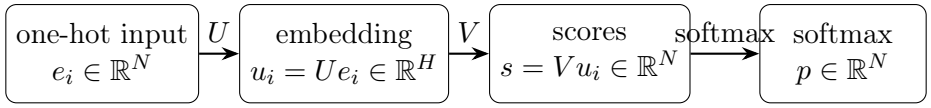
$$s_{ji} = v_j^\top u_i.$$

The full-softmax skip-gram conditional probability is

$$p_\theta(w_j \mid w_i) = \frac{\exp(v_j^\top u_i)}{\sum_{r=1}^{N} \exp(v_r^\top u_i)}.$$

The parameter collection θ contains both the center-word and context-word embedding matrices.

The lecture's diagram shows a one-hot central-word input, a learned hidden embedding vector, and an output distribution over the vocabulary. The central word selects a column of the input embedding matrix. A second matrix produces the vocabulary logits, and softmax normalizes them into a probability vector.



predict context word w_j from center word w_i

Figure 33: Full-softmax skip-gram architecture. An input word is embedded in $\mathbb{R}^H$; context scores are mapped to a probability distribution across the N-word vocabulary.

Remark 13.6 (Two learned vector sets). The center vectors u_i and context vectors v_i are generally separate parameters. Unlike a tied-weight autoencoder, a standard skip-gram model need not require the output matrix to be the transpose of the input matrix. After training, one may use the input vectors, the context vectors, or a combination for downstream representation tasks.

13.6.2 The Softmax Cross-Entropy Objective

Let $\mathcal{D}$ denote the multiset of central-word/context-word index pairs extracted from a corpus. The average negative log-likelihood is

$$\begin{aligned} J(\theta) &= -\frac{1}{|\mathcal{D}|} \sum_{(i,j)\in\mathcal{D}} \log p_\theta(w_j \mid w_i) \\ &= \frac{1}{|\mathcal{D}|} \sum_{(i,j)\in\mathcal{D}} \left[-v_j^\top u_i + \log \left(\sum_{r=1}^{N} e^{v_r^\top u_i} \right) \right]. \end{aligned}$$

Each training pair encourages the model to assign a larger score to its observed context word relative to other vocabulary words.

Proposition 13.4 (Full-softmax skip-gram gradients)**.** *Fix one observed pair (i, j) and write $p_r = p_\theta(w_r \mid w_i)$. For the pairwise negative log-likelihood*

$$L(i, j) = -\log p_j,$$

the gradients are

$$\begin{aligned} \nabla_{u_i} L(i, j) &= \sum_{r=1}^{N} p_r v_r - v_j, \\ \nabla_{v_r} L(i, j) &= (p_r - \mathbf{1}_{\{r=j\}}) u_i, \qquad r = 1, \ldots, N. \end{aligned}$$

Proof. The loss is

$$L(i, j) = -v_j^\top u_i + \log \left(\sum_{r=1}^{N} e^{v_r^\top u_i} \right).$$

Differentiating with respect to the center vector gives

$$\begin{aligned} \nabla_{u_i} L(i, j) &= -v_j + \frac{\sum_{r=1}^{N} e^{v_r^\top u_i} v_r}{\sum_{r=1}^{N} e^{v_r^\top u_i}} \\ &= -v_j + \sum_{r=1}^{N} p_r v_r. \end{aligned}$$

Similarly, for a fixed context-vector index r,

$$\begin{aligned} \nabla_{v_r} L(i, j) &= -\mathbf{1}_{\{r=j\}} u_i + \frac{e^{v_r^\top u_i}}{\sum_{q=1}^{N} e^{v_q^\top u_i}} u_i \\ &= (p_r - \mathbf{1}_{\{r=j\}}) u_i. \end{aligned}$$

These are precisely the softmax cross-entropy derivatives from earlier lectures, specialized to bilinear scores. □

Remark 13.7 (Computational cost). One pairwise full-softmax gradient requires the scores of all N vocabulary words. Computing each inner product costs order H, so the principal arithmetic cost is $O(NH)$ per pair. This may be expensive for a very large vocabulary. Negative sampling, developed next, reduces the number of compared words per training example.

13.7 Word2Vec Corpus Preparation

The handwritten lecture also identifies two practical ideas associated with Word2Vec: treating frequent multiword expressions as single vocabulary items and reducing the dominance of extremely frequent words. These are changes to the training corpus and sampling procedure, not additional layers in the skip-gram network.

13.7.1 Frequent Bigrams and Phrases

A *bigram* consists of two consecutive tokens. Some bigrams represent a meaning not well described by independently processing their component words, such as a proper name or established phrase. A corpus-processing pipeline may replace a frequently occurring expression (w_i, w_j) with one vocabulary item $w_i_w_j$. For instance, two tokens representing a place name may become one training token. The combined token receives its own embedding vector and can participate as either a center or context item.

There is no universal rule requiring that every adjacent word pair be joined: doing so would inflate the vocabulary dramatically. Phrase detection usually depends on co-occurrence frequencies or other statistical criteria. The lecture notes motivate the bigram treatment but do not specify a particular phrase-selection score.

13.7.2 Subsampling Frequent Words

Very frequent words can dominate the number of context pairs while providing relatively weak discrimination about their neighbors. A training pipeline may randomly discard occurrences of such words more often than it discards rare words. One possible frequency-dependent retention rule, used in Word2Vec-style implementations, is

$$p_{\text{keep}}(w) = \min\left\{1, \sqrt{\frac{t}{f(w)}} + \frac{t}{f(w)}\right\},$$

where $f(w)$ denotes the empirical fraction of tokens equal to w and $t > 0$ is a small threshold. For $f(w) \gg t$, the retention probability becomes small; less frequent words are kept more often. This is an example of a practical rule rather than a formula derived in the handwritten lecture.

Subsampling changes the distribution of observed training pairs, and hence the effective training objective. It should not be confused with *negative sampling*: subsampling removes selected corpus-token occurrences before or during pair construction, whereas negative sampling creates artificial contrastive context examples for the prediction loss.

13.8 Negative Sampling

The handwritten notes mention *negative sampling* as part of Word2Vec. Its essential idea is to turn the context-prediction task into discrimination between an observed pair and deliberately sampled mismatched pairs.

Definition 13.9 (Skip-gram with negative sampling)**.** For an observed center/context pair (i, j), draw K negative context indices $n_1, \dots, n_K$ from a chosen noise distribution q on the vocabulary. With logistic sigmoid $\sigma(t) = 1/(1 + e^{-t})$, define the negative-sampling loss

$$L_{\mathrm{NS}}(i, j, n_{1:K}) = -\log \sigma(v_j^\top u_i) - \sum_{s=1}^{K} \log \sigma(-v_{n_s}^\top u_i).$$

Training minimizes this loss for observed pairs and sampled negative examples.

The positive term assigns high probability to an observed center/context pair. Each negative term favors a low score for a sampled mismatched pair. The loss is *not* the same as the full-softmax log-likelihood; it defines a different sampling-based objective.

13.8.1 Derivative of Logistic Pairwise Loss

Recall the derivative

$$\sigma'(t) = \sigma(t)(1 - \sigma(t)).$$

Consequently,

$$\frac{\mathrm{d}}{\mathrm{d}t}[-\log \sigma(t)] = -\frac{\sigma'(t)}{\sigma(t)} = -(1 - \sigma(t)) = \sigma(t) - 1,$$
$$\frac{\mathrm{d}}{\mathrm{d}t}[-\log \sigma(-t)] = \frac{\sigma'(-t)}{\sigma(-t)} = 1 - \sigma(-t) = \sigma(t).$$

Thus both positive and negative terms have simple scalar derivatives.

Proposition 13.5 (Negative-sampling gradients)**.** *Write*

$$a = v_j^\top u_i, \qquad b_s = v_{n_s}^\top u_i.$$

For the negative-sampling loss, the center-vector gradient is

$$\nabla_{u_i} L_{\mathrm{NS}} = (\sigma(a) - 1)v_j + \sum_{s=1}^{K} \sigma(b_s) v_{n_s}.$$

The positive context-vector contribution is

$$(\nabla_{v_j} L_{\mathrm{NS}})_{\mathrm{positive}} = (\sigma(a) - 1)u_i,$$

and each occurrence of a negative index n_s contributes $\sigma(b_s)u_i$ to its context-vector gradient. If an index appears multiple times, including as both a positive and sampled negative index, all its contributions are added.

Proof. Apply the one-dimensional derivatives above to each bilinear argument. Since

$$\nabla_{u_i}(v_r^\top u_i) = v_r,$$
$$\nabla_{v_r}(v_r^\top u_i) = u_i,$$

the multivariable chain rule gives

$$\nabla_{u_i}[-\log\sigma(a)] = (\sigma(a)-1)v_j,$$
$$\nabla_{u_i}[-\log\sigma(-b_s)] = \sigma(b_s)v_{n_s}.$$

Summing the positive and negative contributions proves the center-vector expression; the context-vector results follow identically. □

With K negatives, the pairwise loss involves $K+1$ inner products, each taking $O(H)$ arithmetic operations. The principal cost is therefore $O((K+1)H)$, which can be much smaller than $O(NH)$ when $K \ll N$.

Remark 13.8 (Sampling distribution and collisions). The noise distribution q is a modeling and algorithmic choice; it need not be uniform. Common implementations use a distribution based on corpus frequencies raised to a fractional power. Negative indices may repeat, and an independently sampled negative index may coincide with a positive word. In the literal independent-sampling objective above, such collisions are permitted and their contributions are combined. Implementations may additionally suppress certain collisions, but that changes the exact sampling rule.

13.8.2 A Worked Pairwise Update

Example 13.2 (One negative sample)**.** Consider a two-dimensional center embedding and one positive and one negative context embedding,

$$u = (1,0)^\top, \qquad v_+ = (0,1)^\top, \qquad v_- = (1,0)^\top.$$

The scores and sigmoid values are

$$a = v_+^\top u = 0, \qquad \sigma(a) = \tfrac{1}{2},$$
$$b = v_-^\top u = 1, \qquad \sigma(b) = \frac{e}{1+e}.$$

Therefore,

$$\nabla_u L_{\mathrm{NS}} = -\tfrac{1}{2}v_+ + \frac{e}{1+e}v_-$$
$$= \begin{bmatrix} e/(1+e) \\ -1/2 \end{bmatrix},$$
$$\nabla_{v_+} L_{\mathrm{NS}} = -\tfrac{1}{2}u = \begin{bmatrix} -1/2 \\ 0 \end{bmatrix},$$

$$\nabla_{v_-} L_{\mathrm{NS}} = \frac{e}{1+e} u = \begin{bmatrix} e/(1+e) \\ 0 \end{bmatrix}.$$

A learning-rate-η gradient step decreases the positive-pair loss by shifting u and v_+ toward a larger positive score, while shifting the negative-pair score downward to first order. Whether the two actual vector directions move closer in Euclidean angle at every step depends on their norms and the other terms; the loss acts on inner products, not directly on cosine similarity.

13.9 Cosine Similarity and Embedding Geometry

After training, the geometric relationship between embedding vectors can be used to compare words. The handwritten lecture illustrates this with angles between three vectors.

Definition 13.10 (Cosine similarity)**.** For nonzero vectors $u, v \in \mathbb{R}^H$, their cosine similarity is

$$\mathrm{cosim}(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}.$$

This value lies in $[-1, 1]$. Its geometric interpretation is the cosine of the angle between the two vectors.

The range bound follows from Cauchy–Schwarz:

$$|u^\top v| \le \|u\|_2 \|v\|_2.$$

Positive values indicate an acute angle; zero indicates orthogonality; and negative values indicate an obtuse angle. In applications, vectors with greater cosine similarity are often treated as more closely related under the learned representation, though semantic quality is not guaranteed by geometry alone.

Proposition 13.6 (Scale invariance and normalized Euclidean distance)**.** *For nonzero vectors u, v and positive scalars $a, b > 0$,*

$$\mathrm{cosim}(au, bv) = \mathrm{cosim}(u, v).$$

Let $\widetilde{u} = u/\|u\|_2$ and $\widetilde{v} = v/\|v\|_2$. Then

$$\|\widetilde{u} - \widetilde{v}\|_2^2 = 2 - 2\,\mathrm{cosim}(u, v).$$

Hence ranking pairs by cosine similarity is equivalent to ranking them by Euclidean distance after unit normalization, in reverse order.

Proof. Positive rescaling cancels between the numerator and denominator:

$$\mathrm{cosim}(au, bv) = \frac{ab\, u^\top v}{a\|u\|_2 b\|v\|_2} = \mathrm{cosim}(u, v).$$

Since $\|\widetilde{u}\|_2 = \|\widetilde{v}\|_2 = 1$, expansion of the squared norm gives

$$\begin{aligned}\|\widetilde{u} - \widetilde{v}\|_2^2 &= \|\widetilde{u}\|_2^2 + \|\widetilde{v}\|_2^2 - 2\widetilde{u}^\top \widetilde{v} \\ &= 2 - 2\,\mathrm{cosim}(u, v).\end{aligned}$$

□

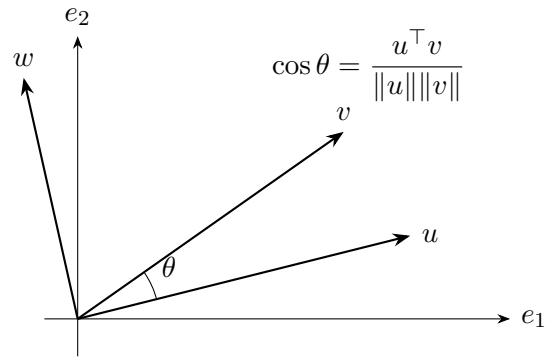


Figure 34: Cosine similarity measures angular alignment. The vectors u and v are more aligned than u and w in this schematic.

13.9.1 Derivative of Cosine Similarity

If cosine similarity itself is used inside an optimization objective, its derivative is useful. Set

$$c(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}, \qquad u, v \neq 0.$$

Treating v as fixed and writing $r = \|u\|_2$, we have $\nabla_u r = u/r$. Therefore,

$$\begin{aligned}\nabla_u c(u, v) &= \frac{v}{\|u\|_2\|v\|_2} - \frac{u^\top v}{\|v\|_2}\frac{u}{\|u\|_2^3} \\ &= \frac{v}{\|u\|_2\|v\|_2} - c(u, v)\frac{u}{\|u\|_2^2}.\end{aligned}$$

In particular,

$$u^\top \nabla_u c(u, v) = \frac{u^\top v}{\|u\|_2\|v\|_2} - c(u, v)\frac{\|u\|_2^2}{\|u\|_2^2} = 0.$$

Thus the cosine gradient is orthogonal to u: to first order, increasing cosine similarity changes its direction rather than its norm. This is consistent with scale invariance.

Remark 13.9 (Zero vectors and numerical stability). Cosine similarity is undefined when one of its arguments is zero. Numerical implementations often replace norms by $\max(\|u\|_2, \varepsilon)$ with a small $\varepsilon > 0$. This is a practical modification of the mathematical definition, not an identity at zero.

13.10 Connections and Limitations

The autoencoder and skip-gram models both learn hidden representations, but they use different prediction tasks. An autoencoder compresses an observation in order to reconstruct that observation. Skip-gram compresses a one-hot word identity into a vector that can be used to predict surrounding words. The first exploits regularities that support reconstruction; the second exploits regularities in context co-occurrence.

The learned word embeddings are often called *distributed representations* because a word's properties may be encoded across many continuous coordinates, while individual coordinates may influence many different word relationships. However, neither the one-hot-to-embedding mapping nor the context-prediction objective mathematically guarantees a human-interpretable meaning for any particular coordinate.

A further limitation is that standard Word2Vec assigns a single vector to each vocabulary word type. The same word appearing in different sentences receives the same static lookup vector, rather than a context-specific embedding computed anew for each occurrence. This motivates later context-dependent language models, but it does not alter the methods derived in this lecture.

13.11 Exercises and Complete Solutions

Exercise 1.

Suppose a dataset contains ten distinct binary vectors in $\{0,1\}^8$. Can a deterministic autoencoder with a three-bit binary code reconstruct all ten vectors exactly? What is the minimum number of binary latent coordinates required for a collision-free encoding of all ten observations?

Solution. A three-bit code has $2^3 = 8$ possible values. By the injectivity proposition, exact deterministic reconstruction of ten distinct observations would require at least ten distinct codes. Since $8 < 10$, a three-bit code is insufficient. With k binary latent coordinates we require $2^k \geq 10$, or $k \geq \lceil \log_2 10 \rceil = 4$. Four coordinates provide sixteen possible codes, so they are sufficient in cardinality, although learning an exact implementation may require appropriate encoder and decoder capacity. ■

Exercise 2.

Let $u = Ue_i$ for $U \in \mathbb{R}^{H \times N}$ and let $L(u) = \frac{1}{2}\|u - t\|_2^2$ with a fixed target $t \in \mathbb{R}^H$. Derive $\nabla_U L$ and one gradient-descent update.

Solution. We have

$$\nabla_u L = u - t = Ue_i - t,$$
$$\nabla_U L = (Ue_i - t)e_i^\top.$$

With learning rate $\eta > 0$,

$$U_{\text{new}} = U - \eta(Ue_i - t)e_i^\top.$$

Only column i changes. Its new value is

$$u_i^{\text{new}} = u_i - \eta(u_i - t) = (1 - \eta)u_i + \eta t.$$

All other columns remain unchanged. ■

Exercise 3.

For a two-word vocabulary, let a central word have context scores $s_1 = 0$ and $s_2 = \log 3$. Compute the softmax probabilities and the cross-entropy when word 2 is observed.

Solution. The exponentiated scores are $e^{s_1} = 1$ and $e^{s_2} = 3$, so

$$p_1 = \frac{1}{1+3} = \frac{1}{4},$$
$$p_2 = \frac{3}{1+3} = \frac{3}{4}.$$

When context word 2 is the target, the negative log-likelihood equals

$$L = -\log p_2 = -\log \frac{3}{4} = \log \frac{4}{3}.$$

The derivatives with respect to the two logits are $(p_1, p_2 - 1) = (1/4, -1/4)$. ■

Exercise 4.

Suppose $u = (3, 4)^\top$ and $v = (4, -3)^\top$. Find their cosine similarity and the distance between their normalized vectors. What is their geometric relationship?

Solution. Their inner product is $u^\top v = 3 \cdot 4 + 4 \cdot (-3) = 0$, while $\|u\|_2 = \|v\|_2 = 5$. Therefore,

$$\operatorname{cosim}(u, v) = 0,$$
$$\left\| \frac{u}{5} - \frac{v}{5} \right\|_2^2 = 2 - 2 \cdot 0 = 2.$$

The normalized Euclidean distance is $\sqrt{2}$. The vectors are orthogonal. ■

Exercise 5.

Consider the one-sample tied linear autoencoder with no biases, $z = Wx$, $\widehat{x} = W^\top z$, and $L = \frac{1}{2}\|\widehat{x} - x\|_2^2$. Derive the gradient with respect to W directly in terms of x, z, and the reconstruction residual $r = \widehat{x} - x$.

Solution. Since both activation functions are the identity, the decoder error is $\delta_d = r$ and the encoder error is $\delta_e = Wr$. The tied-weight gradient proposition gives

$$\begin{aligned}\nabla_W L &= (Wr)x^\top + zr^\top \\ &= W(W^\top Wx - x)x^\top + (Wx)(W^\top Wx - x)^\top.\end{aligned}$$

Both terms have shape $k \times d$. The first is the gradient contribution from the encoder occurrence of W; the second is the gradient contribution from the decoder occurrence of $W^\top$. ■

Exercise 6.

For a positive skip-gram pair with score $a = 0$ and one negative sample with score $b = 0$, calculate the scalar derivatives of the negative-sampling loss with respect to the two scores. Explain the sign of each derivative.

Solution. Because $\sigma(0) = 1/2$,

$$\begin{aligned}\frac{\partial L_{\mathrm{NS}}}{\partial a} &= \sigma(a) - 1 = -\frac{1}{2}, \\ \frac{\partial L_{\mathrm{NS}}}{\partial b} &= \sigma(b) = \frac{1}{2}.\end{aligned}$$

A gradient-descent step moves the positive score upward because its derivative is negative, and moves the negative score downward because its derivative is positive. This is precisely the discrimination encouraged by the negative-sampling objective. ■

14 Lecture 15: Variational Autoencoders and Variational Inference

In Lecture 14, an autoencoder mapped an observation x to a latent code z and then reconstructed the observation. The latent code of an ordinary deterministic autoencoder need not have a tractable probability distribution. Consequently, selecting an arbitrary point in latent space and decoding it need not generate a realistic new observation. In this lecture we make the latent-variable model *probabilistic*: we specify a distribution from which latent variables can be sampled, a conditional distribution that generates observations, and an encoder that approximates the posterior distribution of latent variables given observations.

The resulting model is a *variational autoencoder* (VAE). Its training objective follows from a lower bound on marginal log-likelihood called the *evidence lower bound* (ELBO). The main mathematical issues are (i) the intractability of the latent-variable marginal likelihood, (ii) the construction of a tractable variational objective, and (iii) the differentiation of expectations over distributions whose parameters are learned.

14.1 From Deterministic Autoencoders to Latent-Variable Models

14.1.1 The Reconstruction Viewpoint

Recall the deterministic autoencoder

$$z = f_\phi(x), \qquad \widehat{x} = g_\theta(z),$$

$$\widehat{x} = g_\theta(f_\phi(x)), \qquad (\phi, \theta) \in \arg\min_{\phi,\theta} \frac{1}{m} \sum_{s=1}^{m} \ell(x^{(s)}, \widehat{x}^{(s)}).$$

Here f_ϕ compresses the observation and g_θ reconstructs it. Even if the training reconstruction error is small, this objective does not prescribe which distribution should be used to generate latent codes at test time. In particular, the empirical collection of codes $\{f_\phi(x^{(s)})\}_{s=1}^m$ need not resemble a spherical Gaussian distribution or fill the space between training codes.

The VAE replaces the deterministic encoding by a conditional *distribution* $q_\phi(z \mid x)$, and treats the decoder as a conditional generative distribution $p_\theta(x \mid z)$. It also fixes a simple prior distribution $p(z)$, commonly a standard multivariate Gaussian.

Definition 14.1 (Latent-variable generative model)**.** Let $z \in \mathbb{R}^d$ be a latent variable and $x \in \mathbb{R}^D$ be an observation. A latent-variable model specifies

$$z \sim p(z),$$
$$x \mid z \sim p_\theta(x \mid z).$$

Its joint density is

$$p_\theta(x, z) = p_\theta(x \mid z) p(z),$$

and its marginal or *evidence* for an observation is

$$p_\theta(x) = \int_{\mathbb{R}^d} p_\theta(x \mid z) p(z) \, \mathrm{d}z.$$

The decoder parameters θ determine the conditional distribution $p_\theta(x \mid z)$. For simplicity the prior does not depend on θ; a learned prior can be considered as an extension.

The generative procedure is immediate: first sample z from $p(z)$, and then sample x from $p_\theta(x \mid z)$. Training must make the marginal distribution of these generated observations approximate the distribution of the training data.

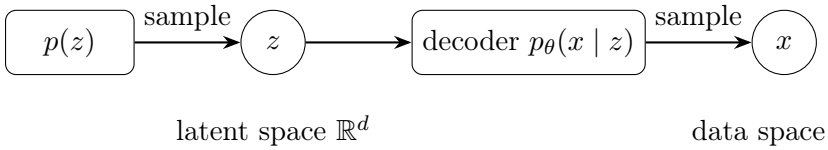


Figure 35: The direction of the generative model is $z \to x$. Sampling a latent vector is not sufficient by itself: the decoder specifies the observation distribution conditional on that vector.

Remark 14.1 (Distribution notation). The expression $p_\theta(x \mid z)$ denotes the *likelihood* or conditional observation model. The posterior $p_\theta(z \mid x)$ instead conditions on x and is generally a different, harder-to-evaluate distribution. The encoder $q_\phi(z \mid x)$ will approximate the latter. Keeping these three distributions distinct is essential for the derivations below.

14.1.2 Maximum-Likelihood Training

Suppose the data are independent observations $\mathcal{D} = \{x^{(1)}, \ldots, x^{(m)}\}$ drawn from an unknown data distribution P_{data}. A natural learning principle is maximum likelihood.

Definition 14.2 (Marginal maximum likelihood)**.** The empirical negative log-likelihood of the generative model is

$$\begin{aligned} \widehat{\mathcal{L}}_{\text{NLL}}(\theta) &= -\frac{1}{m} \sum_{s=1}^{m} \log p_\theta(x^{(s)}) \\ &= -\frac{1}{m} \sum_{s=1}^{m} \log \left(\int_{\mathbb{R}^d} p_\theta(x^{(s)} \mid z) p(z) \, \mathrm{d}z \right). \end{aligned}$$

Maximum-likelihood training minimizes this loss.

Taking a population expectation gives

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{X \sim P_{\text{data}}} \log p_\theta(X).$$

When both P_{data} and p_θ admit densities with respect to the same reference measure, the identity

$$\begin{aligned}\mathrm{KL}(P_{\text{data}}\|p_\theta) &= \mathbb{E}_{P_{\text{data}}} \log \frac{p_{\text{data}}(X)}{p_\theta(X)} \\ &= -H(P_{\text{data}}) - \mathbb{E}_{P_{\text{data}}} \log p_\theta(X)\end{aligned}$$

shows that minimizing population negative log-likelihood is equivalent, up to the entropy of the data distribution, to minimizing the forward KL divergence from the data distribution to the model. This interpretation requires the relevant quantities to be well-defined and finite.

The obstacle is that the integral over all latent variables is usually not available in closed form when the decoder is a nonlinear neural network. Importantly, moving the logarithm inside the integral would not preserve equality:

$$\log\left(\int p_\theta(x \mid z)p(z)\,\mathrm{d}z\right) \neq \int \log p_\theta(x \mid z)p(z)\,\mathrm{d}z$$

in general. A principled lower bound is needed instead.

14.2 Probabilistic Decoders and Reconstruction Loss

14.2.1 Gaussian Observation Model

A common choice for continuously valued data is a Gaussian decoder. Let $\mu_\theta : \mathbb{R}^d \to \mathbb{R}^D$ be a neural network, and for now suppose the observation noise variance $\sigma_x^2 > 0$ is fixed:

$$p_\theta(x \mid z) = \mathcal{N}\big(x; \mu_\theta(z), \sigma_x^2 I_D\big)\,.$$

By the multivariate normal density formula,

$$\begin{aligned}p_\theta(x \mid z) &= (2\pi\sigma_x^2)^{-D/2} \exp\left(-\frac{\|x - \mu_\theta(z)\|_2^2}{2\sigma_x^2}\right), \\ \log p_\theta(x \mid z) &= -\frac{D}{2}\log(2\pi\sigma_x^2) - \frac{1}{2\sigma_x^2}\|x - \mu_\theta(z)\|_2^2, \\ -\log p_\theta(x \mid z) &= \frac{1}{2\sigma_x^2}\|x - \mu_\theta(z)\|_2^2 + \frac{D}{2}\log(2\pi\sigma_x^2).\end{aligned}$$

Thus the negative Gaussian log-likelihood equals a scaled squared reconstruction error, plus a constant independent of θ when σ_x^2 is fixed. The scaling matters when this term is combined with a KL penalty; it is not generally harmless to rescale just one term of the VAE objective.

Remark 14.2 (Decoder sampling versus decoder mean). Under the Gaussian decoder, the neural network outputs the mean $\mu_\theta(z)$, not necessarily the observation itself. Sampling from the model gives $X = \mu_\theta(z) + \sigma_x \eta$ with $\eta \sim \mathcal{N}(0, I_D)$. Displaying the mean $\widehat{x} = \mu_\theta(z)$ is a common visualization choice, but it omits this additional observation noise.

For a diagonal covariance predicted by the decoder,

$$p_\theta(x \mid z) = \mathcal{N}\big(x; \mu_\theta(z), \mathrm{diag}(s_{\theta,1}^2(z), \ldots, s_{\theta,D}^2(z))\big)\,,$$

the negative log-likelihood becomes

$$-\log p_\theta(x \mid z) = \frac{1}{2}\sum_{i=1}^{D}\left[\frac{(x_i - \mu_{\theta,i}(z))^2}{s^2_{\theta,i}(z)} + \log s^2_{\theta,i}(z) + \log(2\pi)\right].$$

The logarithmic variance term can no longer be discarded, since the predicted variances depend on θ.

14.2.2 Other Observation Distributions

If an observation consists of binary coordinates, one may instead use a Bernoulli decoder with success probabilities $\pi_{\theta,i}(z)$:

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \pi_{\theta,i}(z)^{x_i}(1 - \pi_{\theta,i}(z))^{1-x_i},$$

$$-\log p_\theta(x \mid z) = -\sum_{i=1}^{D}\left[x_i \log \pi_{\theta,i}(z) + (1 - x_i)\log(1 - \pi_{\theta,i}(z))\right].$$

This is binary cross-entropy. For categorical observations, a categorical likelihood gives a softmax cross-entropy reconstruction term. The decoder distribution should match the nature and support of the modeled observations; the VAE objective itself is not restricted to mean squared error.

14.3 Approximate Posterior Inference

14.3.1 The Posterior is Generally Intractable

Bayes' rule gives the latent posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)} = \frac{p_\theta(x \mid z)p(z)}{\int p_\theta(x \mid u)p(u)\,\mathrm{d}u}.$$

Even if both the prior and the decoder likelihood can be evaluated, the denominator is the intractable evidence. Hence it is difficult to sample exactly from $p_\theta(z \mid x)$ or directly compute expectations under this posterior.

Definition 14.3 (Variational encoder)**.** A *variational encoder* or *recognition model* is a parameterized conditional density $q_\phi(z \mid x)$ intended to approximate the posterior $p_\theta(z \mid x)$. The encoder is *amortized*: a single parameter vector ϕ defines an approximate posterior for every observation x, rather than optimizing separate variational parameters afresh for each observation.

In the standard Gaussian VAE, a neural network returns vectors $\mu_\phi(x) \in \mathbb{R}^d$ and $\sigma_\phi(x) \in (0,\infty)^d$, defining

$$q_\phi(z \mid x) = \mathcal{N}\big(z; \mu_\phi(x), \operatorname{diag}(\sigma^2_{\phi,1}(x), \ldots, \sigma^2_{\phi,d}(x))\big).$$

This distribution is easy to sample and evaluate. Its expressiveness is nevertheless limited by the assumed diagonal-Gaussian family.

14.4 The Evidence Lower Bound

14.4.1 Derivation by Jensen's Inequality

Fix an observation x and suppose $q_\phi(z \mid x)$ is a probability density supported wherever the relevant ratio is evaluated. Multiplying and dividing the evidence integrand by $q_\phi(z \mid x)$ gives

$$\begin{aligned}\log p_\theta(x) &= \log \int p_\theta(x, z)\, \mathrm{d}z \\ &= \log \int q_\phi(z \mid x) \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\, \mathrm{d}z \\ &= \log \mathbb{E}_{Z \sim q_\phi(\cdot \mid x)} \left[\frac{p_\theta(x, Z)}{q_\phi(Z \mid x)} \right].\end{aligned}$$

Because the logarithm is concave, Jensen's inequality yields

$$\begin{aligned}\log p_\theta(x) &\geq \mathbb{E}_{q_\phi(z \mid x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right] \\ &= \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x, z)] - \mathbb{E}_{q_\phi(z \mid x)}[\log q_\phi(z \mid x)].\end{aligned}$$

Definition 14.4 (Evidence lower bound)**.** The *evidence lower bound*, abbreviated ELBO, is

$$\mathcal{E}(\theta, \phi; x) := \mathbb{E}_{q_\phi(z \mid x)} \left[\log p_\theta(x, z) - \log q_\phi(z \mid x) \right].$$

Then $\mathcal{E}(\theta, \phi; x) \leq \log p_\theta(x)$ whenever the quantities are defined.

Since $p_\theta(x, z) = p_\theta(x \mid z) p(z)$, we obtain the *reconstruction–regularization decomposition*:

$$\begin{aligned}\mathcal{E}(\theta, \phi; x) &= \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi(z \mid x)} \left[\log \frac{p(z)}{q_\phi(z \mid x)} \right] \\ &= \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \| p(z)\big).\end{aligned}$$

Thus maximizing the ELBO rewards reconstructions having high decoder log-likelihood, while penalizing variational posteriors that deviate from the prior.

Remark 14.3 (The minus sign). The training *loss* is normally the negative ELBO:

$$\mathcal{L}_{\mathrm{VAE}}(\theta, \phi; x) = -\mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] + \mathrm{KL}\big(q_\phi(z \mid x) \| p(z)\big).$$

The reconstruction loss and KL divergence both appear with positive signs when the objective is written as a minimization problem. Confusing the signs of the ELBO and negative ELBO reverses the optimization direction.

14.4.2 Exact Identity and the Variational Gap

Jensen's inequality explains the bound; the following identity explains exactly when it becomes tight.

Proposition 14.1 (ELBO identity)**.** *Assume the densities and the relevant KL divergence are well-defined, with $p_\theta(x) > 0$. Then*

$$\log p_\theta(x) = \mathcal{E}(\theta, \phi; x) + \mathrm{KL}\big(q_\phi(z \mid x) \| p_\theta(z \mid x)\big).$$

Consequently, the lower bound is exact if and only if $q_\phi(z \mid x) = p_\theta(z \mid x)$ almost everywhere, whenever equality in the KL divergence characterizes equality of the densities.

Proof. By Bayes' rule,

$$\log p_\theta(z \mid x) = \log p_\theta(x, z) - \log p_\theta(x).$$

Substitute this expression into the KL divergence:

$$\begin{aligned}
\mathrm{KL}\big(q_\phi(z \mid x) \| p_\theta(z \mid x)\big) &= \mathbb{E}_{q_\phi}\left[\log q_\phi(z \mid x) - \log p_\theta(z \mid x)\right] \\
&= \mathbb{E}_{q_\phi}\left[\log q_\phi(z \mid x) - \log p_\theta(x, z) + \log p_\theta(x)\right] \\
&= \log p_\theta(x) - \mathbb{E}_{q_\phi}\left[\log p_\theta(x, z) - \log q_\phi(z \mid x)\right] \\
&= \log p_\theta(x) - \mathcal{E}(\theta, \phi; x).
\end{aligned}$$

Rearranging proves the identity. Nonnegativity of KL divergence gives the lower bound, and its equality condition gives the final assertion. □

Remark 14.4 (Two distinct KL divergences). The exact gap between evidence and ELBO is $\mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$. In contrast, the regularizer *inside* the conveniently computable ELBO is $\mathrm{KL}(q_\phi(z \mid x) \| p(z))$. The first compares the encoder with the intractable posterior; the second compares it with the simple prior. They cannot be interchanged.

14.4.3 Connection to Coordinate Optimization

For a fixed decoder parameter θ, maximizing the ELBO over all possible probability densities q recovers the exact posterior. Indeed,

$$\begin{aligned}
\arg\max_q \mathcal{E}(\theta, q; x) &= \arg\min_q \mathrm{KL}\big(q(z \mid x) \| p_\theta(z \mid x)\big) \\
&= p_\theta(z \mid x),
\end{aligned}$$

provided the full posterior belongs to the optimization class. A VAE restricts q to a learned neural family $\{q_\phi\}$ and trains θ and ϕ jointly. Because the family may not contain the exact posterior, its optimal ELBO can remain strictly below the evidence.

14.5 A Tractable Gaussian Variational Family

14.5.1 Standard Gaussian Prior and Diagonal Gaussian Posterior

The lecture uses the standard normal prior

$$p(z) = \mathcal{N}(z; 0, I_d),$$

and an encoder with

$$q_\phi(z \mid x) = \mathcal{N}(z; \mu, \operatorname{diag}(\sigma_1^2, \dots, \sigma_d^2)), \qquad \mu = \mu_\phi(x), \quad \sigma_i = \sigma_{\phi,i}(x) > 0.$$

In this case the KL penalty has a closed form and requires no Monte Carlo approximation.

Proposition 14.2 (Gaussian KL formula)**.** *For $q = \mathcal{N}(\mu, \operatorname{diag}(\sigma_1^2, \dots, \sigma_d^2))$ and $p = \mathcal{N}(0, I_d)$,*

$$\mathrm{KL}(q\|p) = \frac{1}{2}\sum_{i=1}^{d}\big(\mu_i^2 + \sigma_i^2 - 1 - \log \sigma_i^2\big).$$

Proof. The log-densities are

$$\log q(z) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{i=1}^{d}\log \sigma_i^2 - \frac{1}{2}\sum_{i=1}^{d}\frac{(z_i - \mu_i)^2}{\sigma_i^2},$$
$$\log p(z) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{i=1}^{d} z_i^2.$$

Subtracting gives

$$\log \frac{q(z)}{p(z)} = -\frac{1}{2}\sum_{i=1}^{d}\log \sigma_i^2 - \frac{1}{2}\sum_{i=1}^{d}\frac{(z_i - \mu_i)^2}{\sigma_i^2} + \frac{1}{2}\sum_{i=1}^{d} z_i^2.$$

The two Gaussian moment identities needed for the expectation are

$$\mathbb{E}_q[(Z_i - \mu_i)^2] = \sigma_i^2,$$
$$\mathbb{E}_q[Z_i^2] = \operatorname{Var}_q(Z_i) + (\mathbb{E}_q Z_i)^2 = \sigma_i^2 + \mu_i^2.$$

Consequently,

$$\begin{aligned}\mathrm{KL}(q\|p) &= \mathbb{E}_q \log \frac{q(Z)}{p(Z)} \\ &= -\frac{1}{2}\sum_{i=1}^{d}\log \sigma_i^2 - \frac{1}{2}\sum_{i=1}^{d}\frac{\sigma_i^2}{\sigma_i^2} + \frac{1}{2}\sum_{i=1}^{d}(\sigma_i^2 + \mu_i^2) \\ &= \frac{1}{2}\sum_{i=1}^{d}\big(\mu_i^2 + \sigma_i^2 - 1 - \log \sigma_i^2\big).\end{aligned}$$

□

Remark 14.5 (Why the KL is nonnegative). For $u > 0$, $u - 1 - \log u \geq 0$, with equality only at $u = 1$. Indeed, differentiating $h(u) = u - 1 - \log u$ gives

$$h'(u) = 1 - u^{-1}, \qquad h''(u) = u^{-2} > 0.$$

Thus h has its unique minimum at $u = 1$. Each KL summand consists of $\mu_i^2 \geq 0$ and $h(\sigma_i^2) \geq 0$; the KL is zero precisely when $\mu = 0$ and every $\sigma_i^2 = 1$.

14.5.2 Parameterization by Log-Variance

Neural networks must enforce positive variances. Instead of directly predicting σ_i^2, an encoder commonly predicts

$$a_i = \log \sigma_i^2 \in \mathbb{R},$$

so that $\sigma_i^2 = e^{a_i}$ and $\sigma_i = e^{a_i/2}$. The KL then has the numerically convenient expression

$$\mathrm{KL}(q_\phi(z \mid x) \| p(z)) = \frac{1}{2} \sum_{i=1}^{d} \left(\mu_i^2 + e^{a_i} - 1 - a_i \right).$$

Differentiating explicitly yields

$$\begin{aligned} \frac{\partial \mathrm{KL}}{\partial \mu_i} &= \mu_i, \\ \frac{\partial \mathrm{KL}}{\partial a_i} &= \frac{1}{2}(e^{a_i} - 1), \\ \frac{\partial \mathrm{KL}}{\partial \sigma_i} &= \sigma_i - \frac{1}{\sigma_i}. \end{aligned}$$

These are derivatives with respect to the encoder outputs, not directly with respect to all weights ϕ; the latter require an additional application of the chain rule.

Example 14.1 (Two-dimensional Gaussian KL)**.** Suppose $\mu = (1, -\frac{1}{2})^\top$ and $\mathrm{diag}(\sigma_1^2, \sigma_2^2) = \mathrm{diag}(\frac{1}{4}, 4)$. Then

$$\begin{aligned} \mathrm{KL}(q\|p) &= \frac{1}{2} \left[1 + \frac{1}{4} - 1 - \log \frac{1}{4} + \frac{1}{4} + 4 - 1 - \log 4 \right] \\ &= \frac{1}{2} \left[\frac{1}{4} + \log 4 + \frac{13}{4} - \log 4 \right] \\ &= \frac{7}{4}. \end{aligned}$$

The logarithmic terms cancel in this particular example; the KL is nevertheless positive because the posterior mean and variances differ from those of the standard Gaussian.

14.6 The Reparameterization Trick

14.6.1 Why Direct Sampling Obstructs Ordinary Backpropagation

The reconstruction term is

$$R(\theta, \phi; x) := \mathbb{E}_{Z \sim q_\phi(\cdot|x)}[\log p_\theta(x \mid Z)].$$

The distribution of Z depends on the encoder parameters ϕ. A naive computation that samples Z by an opaque random sampling step and then treats the sample as a constant loses the derivative of R with respect to ϕ. Thus the decoder receives a reconstruction gradient, but the encoder may not receive the intended learning signal.

For a diagonal Gaussian, however, we can express the random draw as a deterministic differentiable transformation of parameter-free noise.

Definition 14.5 (Gaussian reparameterization)**.** Let $\varepsilon \sim \mathcal{N}(0, I_d)$ independently of x and the network parameters. Define

$$Z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon,$$

where $\odot$ denotes componentwise multiplication. This expression is the *reparameterization trick* for a diagonal Gaussian encoder.

Proposition 14.3 (Distribution of the reparameterized sample)**.** *Conditional on a fixed observation x, the random vector $Z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$ has distribution*

$$Z \mid x \sim \mathcal{N}\big(\mu_\phi(x), \operatorname{diag}(\sigma^2_{\phi,1}(x), \ldots, \sigma^2_{\phi,d}(x))\big)\,.$$

Proof. Conditioning on x makes the encoder outputs constant. For each coordinate,

$$\begin{aligned}\mathbb{E}[Z_i \mid x] &= \mu_{\phi,i}(x) + \sigma_{\phi,i}(x)\mathbb{E}[\varepsilon_i] = \mu_{\phi,i}(x),\\ \operatorname{Var}(Z_i \mid x) &= \sigma^2_{\phi,i}(x)\operatorname{Var}(\varepsilon_i) = \sigma^2_{\phi,i}(x).\end{aligned}$$

The independent standard-Gaussian coordinates remain jointly Gaussian under this affine transformation and have zero cross-covariances. Their joint covariance is therefore $\operatorname{diag}(\sigma^2_\phi(x))$, proving the claim. □

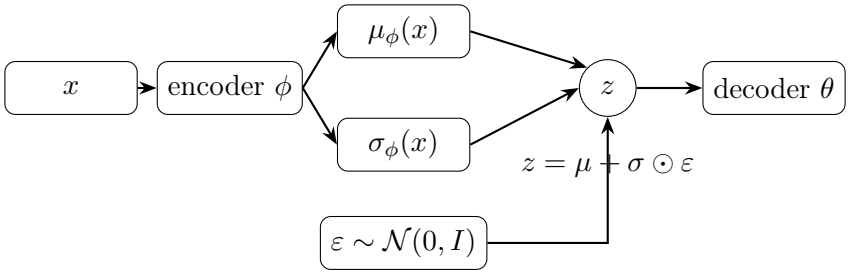


Figure 36: Reparameterization separates randomness (ε) from the differentiable encoder outputs (μ and σ). The reconstruction gradient can now propagate from the decoder through z and into the encoder.

14.6.2 Differentiating Through the Sample

Let $r_\theta(x, z) = \log p_\theta(x \mid z)$, and define $T_\phi(x, \varepsilon) = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$. Then

$$R(\theta, \phi; x) = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I_d)}[r_\theta(x, T_\phi(x, \varepsilon))].$$

Under standard differentiability and integrability assumptions permitting differentiation under the expectation, the chain rule gives

$$\begin{aligned}\nabla_\phi R(\theta, \phi; x) &= \mathbb{E}_\varepsilon\left[(D_\phi T_\phi(x, \varepsilon))^\top \nabla_z r_\theta(x, z)\big|_{z = T_\phi(x,\varepsilon)}\right],\\ \nabla_\theta R(\theta, \phi; x) &= \mathbb{E}_\varepsilon\left[\nabla_\theta r_\theta(x, T_\phi(x, \varepsilon))\right].\end{aligned}$$

The Jacobian of the sample with respect to the encoder parameters is

$$D_\phi T_\phi(x,\varepsilon) = D_\phi \mu_\phi(x) + \operatorname{diag}(\varepsilon) D_\phi \sigma_\phi(x).$$

At the individual-coordinate level, if $z_i = \mu_i + \sigma_i \varepsilon_i$, then

$$\frac{\partial z_i}{\partial \mu_i} = 1, \qquad \frac{\partial z_i}{\partial \sigma_i} = \varepsilon_i, \qquad \frac{\partial z_i}{\partial a_i} = \frac{1}{2} e^{a_i/2} \varepsilon_i \quad (a_i = \log \sigma_i^2).$$

For a differentiable scalar reconstruction loss $\ell(x,z)$, this implies

$$\begin{aligned} \frac{\partial \ell}{\partial \mu_i} &= \frac{\partial \ell}{\partial z_i}, \\ \frac{\partial \ell}{\partial \sigma_i} &= \frac{\partial \ell}{\partial z_i} \varepsilon_i, \\ \frac{\partial \ell}{\partial a_i} &= \frac{1}{2} \frac{\partial \ell}{\partial z_i} \sigma_i \varepsilon_i. \end{aligned}$$

These relations express how a single reconstruction loss supplies gradient information to both the encoder mean and variance outputs.

Remark 14.6 (What reparameterization does and does not do). The trick does not remove stochasticity. It moves the stochastic part to a distribution that is independent of the encoder parameters. A finite collection of samples still gives a Monte Carlo approximation, but ordinary backpropagation can now differentiate that approximation with respect to ϕ. When differentiation and expectation can be interchanged, the corresponding sample-average pathwise gradient is unbiased for the reconstruction-term gradient.

14.6.3 Monte Carlo Approximation

With independent samples $\varepsilon^{(1)}, \dots, \varepsilon^{(K)} \sim \mathcal{N}(0, I_d)$, let

$$z^{(k)} = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon^{(k)}.$$

Then

$$\begin{aligned} \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] &= \mathbb{E}_\varepsilon[\log p_\theta(x \mid T_\phi(x,\varepsilon))] \\ &\approx \frac{1}{K} \sum_{k=1}^{K} \log p_\theta(x \mid z^{(k)}). \end{aligned}$$

The estimator is unbiased for the expectation when the expectation is finite, and it converges almost surely as $K \to \infty$ under the usual law-of-large-numbers assumptions. A single latent sample per observation is frequently sufficient for stochastic training, because the data mini-batch also supplies averaging.

14.7 The Complete VAE Objective and Training Algorithm

14.7.1 Combining the Two Terms

For a Gaussian decoder with fixed variance σ_x^2 and a Gaussian encoder with diagonal covariance, the negative ELBO for one observation is

$$\begin{aligned}\mathcal{L}_{\mathrm{VAE}}(\theta,\phi;x) &= \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x \mid z)] + \mathrm{KL}\big(q_\phi(z \mid x)\|\mathcal{N}(0,I_d)\big)\\ &= \frac{1}{2\sigma_x^2}\mathbb{E}_{q_\phi(z|x)}\|x-\mu_\theta(z)\|_2^2 + \frac{D}{2}\log(2\pi\sigma_x^2)\\ &\quad + \frac{1}{2}\sum_{i=1}^{d}\big(\mu_{\phi,i}(x)^2 + \sigma_{\phi,i}(x)^2 - 1 - \log\sigma_{\phi,i}(x)^2\big).\end{aligned}$$

Dropping the fixed additive Gaussian constant and approximating the expectation with K reparameterized samples gives the computable objective

$$\begin{aligned}\widehat{\mathcal{L}}_{\mathrm{VAE}}(\theta,\phi;x) &= \frac{1}{2K\sigma_x^2}\sum_{k=1}^{K}\|x-\mu_\theta(z^{(k)})\|_2^2\\ &\quad + \frac{1}{2}\sum_{i=1}^{d}\big(\mu_i(x)^2 + e^{a_i(x)} - 1 - a_i(x)\big),\\ z^{(k)} &= \mu_\phi(x) + e^{a_\phi(x)/2}\odot\varepsilon^{(k)}, \quad \varepsilon^{(k)}\sim\mathcal{N}(0,I_d).\end{aligned}$$

The decoder learns to make reconstructed observations likely; the encoder learns latent distributions that both facilitate reconstruction and stay close to the prior.

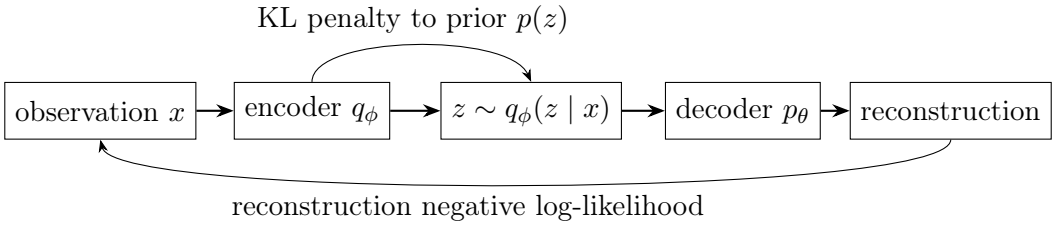


Figure 37: The VAE training objective has two contributions: reconstruct the observation from a sampled code and regularize the encoder distribution toward a chosen prior.

14.7.2 Mini-Batch Optimization

Let $B \subset \{1,\dots,m\}$ be a mini-batch of size b. The mini-batch objective is

$$\widehat{\mathcal{L}}_B(\theta,\phi) = \frac{1}{b}\sum_{s\in B}\widehat{\mathcal{L}}_{\mathrm{VAE}}(\theta,\phi;x^{(s)}).$$

For learning rate $\eta_t > 0$, a stochastic-gradient update is

$$\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \widehat{\mathcal{L}}_B(\theta_t, \phi_t),$$
$$\phi_{t+1} = \phi_t - \eta_t \nabla_\phi \widehat{\mathcal{L}}_B(\theta_t, \phi_t).$$

In practice both gradients are computed in a single reverse-mode differentiation pass through the encoder, the reparameterized sample, and the decoder.

A typical training iteration proceeds as follows:

1. Obtain a mini-batch of observations $x^{(s)}$.
2. Apply the encoder to calculate $\mu_\phi(x^{(s)})$ and $a_\phi(x^{(s)}) = \log \sigma^2_\phi(x^{(s)})$.
3. Draw parameter-independent noise $\varepsilon^{(s)} \sim \mathcal{N}(0, I_d)$ and form $z^{(s)} = \mu_\phi(x^{(s)}) + e^{a_\phi(x^{(s)})/2} \odot \varepsilon^{(s)}$.
4. Evaluate the decoder likelihood or decoder mean $\mu_\theta(z^{(s)})$.
5. Compute the reconstruction negative log-likelihood and the analytic KL term.
6. Average losses across the mini-batch, clear old gradient buffers, backpropagate, and update θ and ϕ.

14.7.3 Illustrative PyTorch Implementation

The following pseudocode uses a Gaussian decoder with fixed observation variance and a diagonal-Gaussian recognition model. The encoder returns μ and $\log \sigma^2$; the decoder returns $\mu_\theta(z)$. The objective preserves the mathematically correct summation across observed coordinates and averages over observations.

```
for x in data_loader:
    optimizer.zero_grad()

    mu, log_var = encoder(x)
    eps = torch.randn_like(mu)
    std = torch.exp(0.5 * log_var)
    z = mu + std * eps

    x_mean = decoder(z)
    reconstruction_nll = (
        (x - x_mean).square().flatten(start_dim=1).sum(dim=1)
        / (2.0 * observation_variance)
    )
    kl = 0.5 * (
        mu.square() + log_var.exp() - 1.0 - log_var
    ).sum(dim=1)

    loss = (reconstruction_nll + kl).mean()
    loss.backward()
    optimizer.step()
```

The Gaussian normalization constant is omitted because the observation variance is fixed. If it is trainable, the appropriate normalization term must be included. At generation time, one draws $z \sim \mathcal{N}(0, I_d)$ and samples from $p_\theta(x \mid z)$, rather than running the encoder on a training observation.

14.8 Worked Example: A Linear Gaussian Latent Model

The standard neural VAE generally has an intractable posterior. In a linear Gaussian model, however, the posterior can be found exactly, making it possible to see how the ELBO behaves.

Example 14.2 (Exact posterior in one dimension)**.** Let

$$Z \sim \mathcal{N}(0, 1), \qquad X \mid Z = z \sim \mathcal{N}(az, \sigma^2),$$

where $a \in \mathbb{R}$ and $\sigma^2 > 0$. For a fixed observed value x, Bayes' formula shows that the posterior density is proportional to

$$\begin{aligned} p(z \mid x) &\propto \exp\left(-\frac{z^2}{2} - \frac{(x - az)^2}{2\sigma^2}\right) \\ &= \exp\left[-\frac{1}{2}\left(\left(1 + \frac{a^2}{\sigma^2}\right) z^2 - \frac{2ax}{\sigma^2} z + \frac{x^2}{\sigma^2}\right)\right]. \end{aligned}$$

Set

$$v = \left(1 + \frac{a^2}{\sigma^2}\right)^{-1} = \frac{\sigma^2}{a^2 + \sigma^2}, \qquad m = v\frac{ax}{\sigma^2} = \frac{ax}{a^2 + \sigma^2}.$$

Completing the square in the exponent gives

$$\begin{aligned} \left(1 + \frac{a^2}{\sigma^2}\right) z^2 - \frac{2ax}{\sigma^2} z &= \frac{1}{v}(z^2 - 2mz) \\ &= \frac{1}{v}\big((z - m)^2 - m^2\big). \end{aligned}$$

All terms independent of z enter the normalizing constant, so

$$p(z \mid x) = \mathcal{N}\left(z; \frac{ax}{a^2 + \sigma^2}, \frac{\sigma^2}{a^2 + \sigma^2}\right).$$

In particular, the posterior mean depends linearly on x and the posterior variance is smaller than or equal to the prior variance 1, with strict inequality whenever $a \neq 0$.

The marginal is also Gaussian:

$$\begin{aligned} X &= aZ + \sigma\eta, \qquad Z, \eta \overset{\text{ind}}{\sim} \mathcal{N}(0, 1), \\ \mathbb{E}[X] &= 0, \\ \operatorname{Var}(X) &= a^2 \operatorname{Var}(Z) + \sigma^2 \operatorname{Var}(\eta) = a^2 + \sigma^2, \\ X &\sim \mathcal{N}(0, a^2 + \sigma^2). \end{aligned}$$

Therefore

$$\log p(x) = -\frac{1}{2}\log(2\pi(a^2+\sigma^2)) - \frac{x^2}{2(a^2+\sigma^2)}.$$

If the variational family contains $\mathcal{N}(m, v)$ and the encoder selects this exact posterior, the KL to the exact posterior vanishes. The ELBO identity then gives $\mathcal{E}(x) = \log p(x)$. This provides a direct consistency check of the variational derivation.

14.9 Interpretation, Extensions, and Limitations

14.9.1 What the Two Parts of the Objective Encourage

The reconstruction contribution encourages z to retain information about x that is useful for predicting the observation. The KL contribution encourages $q_\phi(z \mid x)$ to remain close to the prior, which facilitates sampling codes from the prior after training. These pressures compete: if the encoder is forced to equal the prior for every observation, Z contains no information about which observation was supplied. Conversely, an unconstrained encoder may represent observations accurately without providing a convenient sampling distribution.

The KL regularizer does not guarantee that every latent point leads to a meaningful observation, nor that the learned coordinates correspond to unique semantic factors. It controls a distributional discrepancy, not a pointwise reconstruction property.

Proposition 14.4 (Average posterior KL decomposition)**.** *Define the aggregated posterior*

$$q_\phi(z) := \int q_\phi(z \mid x) p_{\text{data}}(x)\,\mathrm{d}x.$$

Under the usual absolute-continuity and integrability assumptions,

$$\mathbb{E}_{X\sim p_{\text{data}}}\mathrm{KL}\big(q_\phi(z \mid X)\|p(z)\big) = I_q(X;Z) + \mathrm{KL}\big(q_\phi(z)\|p(z)\big),$$

where $I_q(X;Z)$ is the mutual information calculated under $q_\phi(x,z) = p_{\text{data}}(x)q_\phi(z \mid x)$.

Proof. Insert and subtract $\log q_\phi(z)$ in the definition:

$$\begin{aligned}
\mathbb{E}_X \mathrm{KL}(q_\phi(z \mid X)\|p(z)) &= \iint p_{\text{data}}(x) q_\phi(z \mid x) \log \frac{q_\phi(z \mid x)}{p(z)}\,\mathrm{d}z\,\mathrm{d}x \\
&= \iint q_\phi(x,z) \log \frac{q_\phi(z \mid x)}{q_\phi(z)}\,\mathrm{d}z\,\mathrm{d}x \\
&\quad + \iint q_\phi(x,z) \log \frac{q_\phi(z)}{p(z)}\,\mathrm{d}z\,\mathrm{d}x \\
&= I_q(X;Z) + \int q_\phi(z) \log \frac{q_\phi(z)}{p(z)}\,\mathrm{d}z.
\end{aligned}$$

The final integral is $\mathrm{KL}(q_\phi(z)\|p(z))$. □

This identity clarifies a subtlety: minimizing the average KL not only helps match the aggregated posterior to the prior, but also tends to reduce how much information the latent variable carries about its input.

14.9.2 Beta-VAEs and the Loss-Weighting Convention

A common extension weights the KL term by $\beta > 0$:

$$\mathcal{L}_\beta(\theta, \phi; x) = -\mathbb{E}_{q_\phi(z|x)} \log p_\theta(x \mid z) + \beta \, \mathrm{KL}\big(q_\phi(z \mid x) \| p(z)\big).$$

For $\beta = 1$ this is the ordinary negative ELBO. For $\beta \neq 1$ it is generally a modified training criterion rather than the negative of the same standard evidence lower bound. Larger β penalizes deviations from the prior more strongly, often at the cost of reconstruction quality; smaller β weakens this penalty.

A numerical implementation that uses a *mean* squared error across coordinates rather than the Gaussian negative log-likelihood *sum* implicitly changes the relative reconstruction/KL weight. To preserve the probability-model interpretation, first derive the likelihood with its proper variance and summation convention, then introduce any desired alternative weighting explicitly.

14.9.3 Posterior Collapse

A sufficiently expressive decoder may model the observations while ignoring z, leading to an encoder close to the prior for most data points:

$$q_\phi(z \mid x) \approx p(z).$$

Then the expected KL is small, but the latent representation conveys little information about x. This phenomenon is called *posterior collapse*. It is consistent with the average KL decomposition above. Decoder capacity, optimization dynamics, and deliberate KL weighting or schedules can affect whether collapse occurs.

14.9.4 Conceptual Summary

The probabilistic model specifies $p(z)$ and $p_\theta(x \mid z)$; the approximate inference model specifies $q_\phi(z \mid x)$. Evidence maximization is difficult because $p_\theta(x)$ requires latent marginalization. The ELBO supplies a computable lower bound by introducing a variational posterior. For diagonal Gaussians, the prior KL is analytic; the reconstruction expectation is estimated with samples. Reparameterization makes those samples differentiable transformations of parameter-free noise, enabling ordinary backpropagation.

14.10 Exercises and Complete Solutions

Exercise 1: Derive the ELBO in two different ways..

Starting from the model evidence, derive the ELBO using Jensen's inequality. Then derive the exact decomposition of log-evidence into the ELBO and a nonnegative posterior KL divergence.

Solution. For any density $q(z \mid x)$ with suitable support,

$$\begin{aligned}\log p_\theta(x) &= \log \int q(z \mid x) \frac{p_\theta(x, z)}{q(z \mid x)} \, \mathrm{d}z \\ &= \log \mathbb{E}_q \left[\frac{p_\theta(x, Z)}{q(Z \mid x)} \right] \\ &\geq \mathbb{E}_q [\log p_\theta(x, Z) - \log q(Z \mid x)] \\ &= \mathbb{E}_q \log p_\theta(x \mid Z) - \mathrm{KL}(q(z \mid x) \| p(z)).\end{aligned}$$

The last line is the ELBO. For the exact decomposition, use

$$\begin{aligned}\mathrm{KL}(q \| p_\theta(\cdot \mid x)) &= \mathbb{E}_q \left[\log q(Z \mid x) - \log p_\theta(Z \mid x) \right] \\ &= \mathbb{E}_q \left[\log q(Z \mid x) - \log p_\theta(x, Z) + \log p_\theta(x) \right] \\ &= \log p_\theta(x) - \mathcal{E}(\theta, q; x).\end{aligned}$$

Rearranging gives the required identity. The first derivation explains why the ELBO is a bound; the second gives its precise gap. ■

Exercise 2: Diagonal-Gaussian KL and its gradients..

Let $q(z) = \mathcal{N}(\mu, \mathrm{diag}(e^{a_1}, \ldots, e^{a_d}))$ and $p(z) = \mathcal{N}(0, I_d)$. Derive $\mathrm{KL}(q \| p)$ and its derivatives with respect to μ_i and a_i.

Solution. The diagonal covariance has $\sigma_i^2 = e^{a_i}$, so the Gaussian KL formula gives

$$\begin{aligned}\mathrm{KL}(q \| p) &= \frac{1}{2} \sum_{i=1}^{d} (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2) \\ &= \frac{1}{2} \sum_{i=1}^{d} (e^{a_i} + \mu_i^2 - 1 - a_i).\end{aligned}$$

Termwise differentiation gives

$$\begin{aligned}\frac{\partial \mathrm{KL}}{\partial \mu_i} &= \frac{1}{2}(2\mu_i) = \mu_i, \\ \frac{\partial \mathrm{KL}}{\partial a_i} &= \frac{1}{2}(e^{a_i} - 1).\end{aligned}$$

These terms are zero at $\mu_i = 0$ and $a_i = 0$, the values corresponding to the standard normal prior. ■

Exercise 3: Differentiate a reparameterized expectation..

Let $\varepsilon \sim \mathcal{N}(0,1)$, $Z = \mu + \sigma\varepsilon$ with $\sigma > 0$, and $F(\mu,\sigma) = \mathbb{E}[(x - aZ)^2]$. Find F, $\partial F/\partial\mu$, and $\partial F/\partial\sigma$, first by taking expectations and then by differentiating through the sample.

Solution. Expanding the square,

$$\begin{aligned}(x - aZ)^2 &= (x - a\mu - a\sigma\varepsilon)^2 \\ &= (x - a\mu)^2 - 2a\sigma(x - a\mu)\varepsilon + a^2\sigma^2\varepsilon^2.\end{aligned}$$

Since $\mathbb{E}\varepsilon = 0$ and $\mathbb{E}\varepsilon^2 = 1$,

$$F(\mu,\sigma) = (x - a\mu)^2 + a^2\sigma^2.$$

Direct differentiation gives

$$\frac{\partial F}{\partial\mu} = -2a(x - a\mu), \qquad \frac{\partial F}{\partial\sigma} = 2a^2\sigma.$$

Alternatively, differentiating the samplewise loss $\ell = (x - aZ)^2$ gives

$$\begin{aligned}\frac{\partial\ell}{\partial Z} &= -2a(x - aZ), \\ \frac{\partial Z}{\partial\mu} = 1, &\qquad \frac{\partial Z}{\partial\sigma} = \varepsilon.\end{aligned}$$

Thus

$$\begin{aligned}\mathbb{E}\frac{\partial\ell}{\partial\mu} &= \mathbb{E}[-2a(x - a\mu - a\sigma\varepsilon)] = -2a(x - a\mu), \\ \mathbb{E}\frac{\partial\ell}{\partial\sigma} &= \mathbb{E}[-2a(x - a\mu - a\sigma\varepsilon)\varepsilon] = 2a^2\sigma,\end{aligned}$$

matching the derivatives obtained from the explicit expectation. ■

Exercise 4: Find the exact posterior in a Gaussian generative model..

Suppose $Z \sim \mathcal{N}(0,1)$ and $X \mid Z = z \sim \mathcal{N}(2z,1)$. Compute the marginal distribution of X, the posterior distribution $Z \mid X = x$, and the posterior distribution at $x = 5$.

Solution. Write $X = 2Z + \eta$ where $\eta \sim \mathcal{N}(0,1)$ is independent of Z. Then

$$X \sim \mathcal{N}(0, 2^2 + 1) = \mathcal{N}(0,5).$$

From the one-dimensional linear-Gaussian posterior formula with $a = 2$ and $\sigma^2 = 1$,

$$Z \mid X = x \sim \mathcal{N}\left(\frac{2x}{5}, \frac{1}{5}\right).$$

Setting $x = 5$ gives

$$Z \mid X = 5 \sim \mathcal{N}(2, 1/5).$$

If the variational encoder represents this Gaussian exactly, the variational gap is zero at each value of x. ■

Exercise 5: Check the effect of reconstruction-loss scaling..

Suppose an observation has dimension D and a Gaussian decoder has fixed variance σ_x^2. Explain how replacing the summed squared-error negative log-likelihood by mean squared error changes the relative weighting of the KL term.

Solution. The parameter-dependent Gaussian reconstruction loss is

$$L_{\text{rec}} = \frac{1}{2\sigma_x^2} \sum_{i=1}^{D} (x_i - \mu_{\theta,i}(z))^2.$$

The coordinatewise mean squared error is

$$\text{MSE} = \frac{1}{D} \sum_{i=1}^{D} (x_i - \mu_{\theta,i}(z))^2.$$

Hence $L_{\text{rec}} = \frac{D}{2\sigma_x^2}\text{MSE}$. The standard negative ELBO has objective

$$\frac{D}{2\sigma_x^2}\text{MSE} + \text{KL}.$$

Using MSE+KL without adjusting coefficients instead multiplies the reconstruction contribution by $2\sigma_x^2/D$. Equivalently, relative to the original Gaussian interpretation, it changes the effective KL weight. This change may be intentionally desirable, but it is not algebraically neutral. ■

15 Lecture 16: Diffusion Models and Denoising Score Matching

In Lecture 15, a variational autoencoder represented observations using a latent-variable distribution and learned a decoder by maximizing an evidence lower bound. A *denoising diffusion probabilistic model* (DDPM) takes a different approach: rather than compressing the observation into a single low-dimensional code, it repeatedly perturbs an observation with Gaussian noise, producing a chain of increasingly noisy latent variables. Learning then consists of fitting a stochastic process that approximately reverses those perturbations. Beginning with a Gaussian noise sample and executing the learned reverse transitions produces a new observation.

The essential ideas are (i) a forward process whose conditional distributions can be computed exactly, (ii) Gaussian reverse conditionals given the clean observation, (iii) a variational objective for learning an approximate reverse process, and (iv) a particularly simple equivalent parameterization in which the neural network predicts the noise introduced by the forward process.

15.1 Forward Diffusion: Gradually Adding Gaussian Noise

15.1.1 The Forward Markov Chain

Let $x_0 \in \mathbb{R}^d$ be a data observation, with $x_0 \sim q_{\text{data}}$. Fix an integer $T \geq 1$ and a deterministic *noise schedule*

$$0 < \beta_t < 1, \qquad t = 1, \dots, T.$$

We define

$$\alpha_t := 1 - \beta_t, \qquad \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s, \qquad \bar{\alpha}_0 := 1.$$

Note carefully the difference between α_t, which controls a *single* diffusion step, and $\bar{\alpha}_t$, which measures the accumulated signal fraction after t steps. Because $0 < \alpha_t < 1$, the product $\bar{\alpha}_t$ is decreasing in t.

Definition 15.1 (Forward diffusion process)**.** Define the Gaussian Markov chain

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$
$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{\alpha_t}\, x_{t-1}, \beta_t I_d\big).$$

Equivalently, using independent random vectors $\varepsilon_t \sim \mathcal{N}(0, I_d)$,

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t, \qquad t = 1, \dots, T.$$

The notation $\mathcal{N}(x; \mu, \Sigma)$ denotes the Gaussian density evaluated at x, with mean μ and covariance Σ.

The chain follows

$$x_0 \longrightarrow x_1 \longrightarrow \cdots \longrightarrow x_{T-1} \longrightarrow x_T.$$

At each step the current state is attenuated by $\sqrt{\alpha_t}$ and fresh independent Gaussian noise of variance β_t is added. The forward transition is fixed by the schedule, not learned by a neural network.

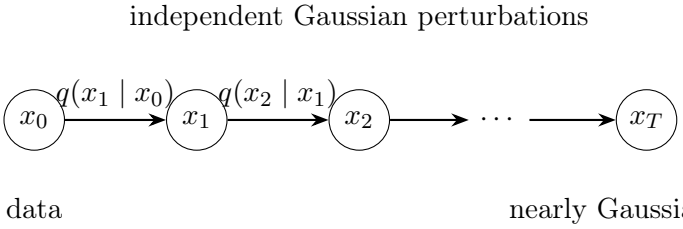


Figure 38: The forward diffusion chain. The forward transitions are fixed Gaussian kernels; the learned reverse model will traverse the chain in the opposite direction.

A common schedule increases β_t over time. For example, the handwritten lecture illustrates linearly spaced noise levels beginning around 10^{-4} and ending around 0.02. Such values are examples rather than mathematical requirements, and whether x_T is nearly pure Gaussian noise depends on the *product* $\bar{\alpha}_T$, not just on the last value β_T.

15.1.2 Gaussian Reparameterization

Recall that if $Z \sim \mathcal{N}(0, I_d)$, then $\mu + AZ \sim \mathcal{N}(\mu, AA^\top)$. For the forward transition, choose $A = \sqrt{\beta_t} I_d$. Therefore,

$$\begin{aligned} x_t \mid x_{t-1} &\sim \mathcal{N}(\sqrt{\alpha_t}\, x_{t-1}, \beta_t I_d) \\ &\iff \quad x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, I_d). \end{aligned}$$

In particular,

$$\begin{aligned} \mathbb{E}[x_t \mid x_{t-1}] &= \sqrt{\alpha_t}\, x_{t-1}, \\ \mathrm{Cov}(x_t \mid x_{t-1}) &= \beta_t I_d. \end{aligned}$$

The mean is contracted toward zero at each step, while the covariance of the fresh increment is $\beta_t I_d$.

15.2 An Exact Formula for the Noisy Observation

A key convenience of Gaussian forward diffusion is that x_t can be sampled directly from x_0, without simulating all t intermediate transitions.

Proposition 15.1 (Closed-form forward marginal)**.** *For every $t \in \{1, \dots, T\}$,*

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t) I_d\big).$$

Equivalently, conditional on x_0,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I_d).$$

Proof. We prove the claim by induction. For $t = 1$, $\bar{\alpha}_1 = \alpha_1$ and $1 - \bar{\alpha}_1 = \beta_1$, so the assertion is exactly the defining forward transition.

Assume for time $t - 1$ that

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\, \varepsilon',$$

where $\varepsilon' \sim \mathcal{N}(0, I_d)$. Let $\varepsilon_t \sim \mathcal{N}(0, I_d)$ be independent of ε'. Then

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t \\ &= \sqrt{\alpha_t \bar{\alpha}_{t-1}}\, x_0 + \sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})}\, \varepsilon' + \sqrt{\beta_t}\, \varepsilon_t \\ &= \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})}\, \varepsilon' + \sqrt{\beta_t}\, \varepsilon_t. \end{aligned}$$

The final two summands are independent centered Gaussian vectors. Their covariance is

$$\begin{aligned} \big[\alpha_t (1 - \bar{\alpha}_{t-1}) + \beta_t\big] I_d &= \big[\alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t\big] I_d \\ &= (1 - \bar{\alpha}_t) I_d. \end{aligned}$$

Therefore their sum has the same distribution as $\sqrt{1 - \bar{\alpha}_t}\, \varepsilon$ for $\varepsilon \sim \mathcal{N}(0, I_d)$, proving the result. □

The same identity can be obtained by expanding every independent innovation:

$$\begin{aligned} x_t &= \sqrt{\bar{\alpha}_t}\, x_0 + \sum_{j=1}^{t} \left(\sqrt{\beta_j} \prod_{s=j+1}^{t} \sqrt{\alpha_s} \right) \varepsilon_j, \\ \mathrm{Cov}(x_t \mid x_0) &= \left(\sum_{j=1}^{t} \beta_j \prod_{s=j+1}^{t} \alpha_s \right) I_d \\ &= \left(1 - \prod_{s=1}^{t} \alpha_s \right) I_d = (1 - \bar{\alpha}_t) I_d. \end{aligned}$$

The penultimate equality is a telescoping identity, which follows directly from $\beta_j = 1 - \alpha_j$.

Remark 15.1 (Signal-to-noise ratio). Conditional on x_0, the coefficient of the signal is $\sqrt{\bar{\alpha}_t}$ while the standard deviation of the Gaussian perturbation is $\sqrt{1 - \bar{\alpha}_t}$. The scalar signal-to-noise ratio associated with these coefficients is

$$\mathrm{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}.$$

As $\bar{\alpha}_t$ decreases, the noisy observation reveals progressively less about the original data point.

15.2.1 Why the Terminal Distribution Is Approximately Gaussian

Conditionally on x_0,

$$x_T = \sqrt{\bar{\alpha}_T}\, x_0 + \sqrt{1-\bar{\alpha}_T}\, \varepsilon.$$

When $\bar{\alpha}_T \ll 1$, the data-dependent term is small and the noise term has variance close to one. Consequently the *marginal* $q(x_T)$, obtained by averaging over $x_0 \sim q_{\text{data}}$, is approximately $\mathcal{N}(0, I_d)$ under appropriate moment assumptions on the data. For a fixed x_0, the conditional law is instead exactly

$$q(x_T \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_T}\, x_0, (1-\bar{\alpha}_T) I_d\big).$$

We should not confuse either distribution with the one-step transition $q(x_T \mid x_{T-1})$, whose mean is $\sqrt{\alpha_T} x_{T-1}$.

Proposition 15.2 (Convergence to a standard Gaussian)**.** *Consider a sequence of noise schedules for which $\bar{\alpha}_T \to 0$. If X_0 is a finite random vector independent of $\varepsilon \sim \mathcal{N}(0, I_d)$, then*

$$\sqrt{\bar{\alpha}_T} X_0 + \sqrt{1-\bar{\alpha}_T}\, \varepsilon \ \xrightarrow{\mathcal{D}}\ \mathcal{N}(0, I_d).$$

If additionally $\mathbb{E}\|X_0\|_2^2 < \infty$, the displayed coupling converges to ε in mean square.

Proof. The factors $\sqrt{\bar{\alpha}_T} \to 0$ and $\sqrt{1-\bar{\alpha}_T} \to 1$. Hence the first term converges to zero in probability and the second to ε, giving convergence in distribution. For the second claim, independence yields

$$\begin{aligned} &\mathbb{E}\left\| \sqrt{\bar{\alpha}_T} X_0 + \big(\sqrt{1-\bar{\alpha}_T} - 1\big)\varepsilon \right\|_2^2 \\ &\qquad = \bar{\alpha}_T \mathbb{E}\|X_0\|_2^2 + d\big(\sqrt{1-\bar{\alpha}_T} - 1\big)^2 \longrightarrow 0. \end{aligned}$$

□

15.3 Denoising and the Reverse Conditional Distribution

15.3.1 Why Direct Inversion Is Not Available

If both x_0 and the realized noise ε were known, the closed-form forward relation could be rearranged:

$$\begin{aligned} x_t &= \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \\ x_0 &= \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}, \\ \varepsilon &= \frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{\sqrt{1-\bar{\alpha}_t}}. \end{aligned}$$

At generation time, however, we only have a noisy state x_t; neither the original observation nor the actual injected noise is available. There is no deterministic inverse $x_t \mapsto x_0$ in general. Moreover, when $\bar{\alpha}_t$ is very small, division by $\sqrt{\bar{\alpha}_t}$ amplifies estimation errors. The model must *learn* to infer the relevant noise from x_t and the timestep t.

The forward process is Markovian. For the reverse direction, it is particularly useful to condition on the clean observation x_0 as well. The resulting posterior can be calculated exactly because it is the product of Gaussian densities.

15.3.2 Exact Gaussian Forward Posterior

Proposition 15.3 (Posterior of the previous diffusion state)**.** *For $t \geq 2$,*

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(x_{t-1}; \widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t I_d\big),$$

where

$$\begin{aligned}
\widetilde{\beta}_t &= \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t, \\
\widetilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t.
\end{aligned}$$

At $t = 1$ the clean starting state is known, so $q(x_0 \mid x_1, x_0)$ is a point mass at x_0, not a nondegenerate Gaussian.

Proof. By the conditional independence structure of the forward Markov chain,

$$\begin{aligned}
q(x_{t-1} \mid x_t, x_0) &\propto q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0) \\
&\propto \exp\left\{ -\frac{\|x_t - \sqrt{\alpha_t} x_{t-1}\|_2^2}{2\beta_t} - \frac{\|x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0\|_2^2}{2(1-\bar{\alpha}_{t-1})} \right\}.
\end{aligned}$$

Treat $u = x_{t-1}$ as the variable and expand only the terms that depend on u. Up to a normalizing constant, the exponent is

$$-\frac{1}{2}\left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} \right) \|u\|_2^2 - 2 \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0 \right)^{\top} u \right].$$

The scalar precision is

$$\begin{aligned}
\widetilde{\beta}_t^{-1} &= \frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} \\
&= \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})} \\
&= \frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})},
\end{aligned}$$

where we used $\alpha_t + \beta_t = 1$ and $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. Completing the square gives the covariance $\widetilde{\beta}_t I_d$ and mean

$$\begin{aligned}
\widetilde{\mu}_t &= \widetilde{\beta}_t \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0 \right) \\
&= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} x_0.
\end{aligned}$$

□

The central difficulty is now sharply stated: we know the functional form of $q(x_{t-1} \mid x_t, x_0)$, but it depends on x_0, which is unknown during generation. We therefore replace the exact posterior mean by a *learned* mean depending only on x_t and t.

15.4 Reparameterizing the Reverse Mean with Predicted Noise

15.4.1 The Noise-Mean Identity

Recall the direct noisy observation

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon.$$

Substitute the formal identity

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}$$

into the exact posterior mean. This yields

$$\begin{aligned}
\widetilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon}{\sqrt{\bar{\alpha}_t}}\right) + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t \\
&= \frac{\beta_t}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}\varepsilon + \frac{\alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} x_t \\
&= \frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} x_t - \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}\varepsilon \\
&= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right).
\end{aligned}$$

This is the fundamental noise-parameterization identity in the handwritten lecture.

Definition 15.2 (Noise-prediction network)**.** Let $\varepsilon_\theta(x_t, t)$ be a neural network taking a noisy sample and its timestep as inputs and producing a vector in $\mathbb{R}^d$. The reverse-process mean is parameterized as

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right).$$

A common fixed-variance reverse transition is

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I_d\big),$$

where $\sigma_t^2 > 0$ is chosen according to a noise-variance schedule. At $t \geq 2$, common choices include $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \widetilde{\beta}_t$.

The reverse model does not need to output a reconstructed clean observation directly. It can instead learn the noise in the observed state. If ε_θ is a good predictor of the forward noise, substituting it for ε makes μ_θ a good predictor of the exact posterior mean.

Proposition 15.4 (Reconstruction from predicted noise)**.** *A prediction of the clean observation from x_t is*

$$\widehat{x}_{0,\theta}(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

When this estimate is substituted into $\widetilde{\mu}_t(x_t, x_0)$, the resulting mean equals $\mu_\theta(x_t, t)$.

Proof. The estimated x_0 is obtained by rearranging the direct-noising relation with ε_θ in place of ε. Substitution into the exact posterior mean uses precisely the algebra above, so the two parameterizations coincide. □

Remark 15.2 (Different Gaussian conditionals). Three distributions must be distinguished. The forward transition $q(x_t \mid x_{t-1})$ is known; the forward posterior $q(x_{t-1} \mid x_t, x_0)$ is known *given* x_0; the reverse generative transition $p_\theta(x_{t-1} \mid x_t)$ is learned and does not depend on x_0. In general, the true unconditional reverse conditional $q(x_{t-1} \mid x_t)$ is a mixture over possible x_0 and need not itself be exactly Gaussian.

15.5 Variational Training of the Reverse Diffusion Process

15.5.1 Generative Model and Evidence Lower Bound

The reverse generative model starts from a simple terminal prior and moves backward through the chain:

$$\begin{aligned} p(x_T) &= \mathcal{N}(x_T; 0, I_d), \\ p_\theta(x_{0:T}) &= p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \\ p_\theta(x_0) &= \int_{\mathbb{R}^{dT}} p_\theta(x_{0:T})\, \mathrm{d}x_1 \cdots \mathrm{d}x_T. \end{aligned}$$

The forward chain $q(x_{1:T} \mid x_0)$ is an auxiliary distribution which makes the intractable marginal likelihood accessible through a lower bound.

Proposition 15.5 (Diffusion variational bound)**.** *For any observation x_0 for which the relevant densities and expectations are well-defined,*

$$-\log p_\theta(x_0) \le \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})} \right] =: \mathcal{L}_{\mathrm{VLB}}(x_0; \theta).$$

Furthermore, this bound decomposes into

$$\begin{aligned} \mathcal{L}_{\mathrm{VLB}}(x_0; \theta) = \underbrace{D_{\mathrm{KL}}\big(q(x_T \mid x_0) \| p(x_T)\big)}_{L_T} + \underbrace{\mathbb{E}_q[-\log p_\theta(x_0 \mid x_1)]}_{L_0} \\ + \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(x_t|x_0)} D_{\mathrm{KL}}\big(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t)\big)}_{L_{t-1}}. \end{aligned}$$

Proof. By inserting the auxiliary density into the marginal likelihood and using Jensen's inequality,

$$\begin{aligned}\log p_\theta(x_0) &= \log \int q(x_{1:T} \mid x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \, \mathrm{d}x_{1:T} \\ &\geq \mathbb{E}_q \left[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0) \right].\end{aligned}$$

Negating both sides proves the bound. To obtain the decomposition, Bayes' rule and the Markov property give the identity

$$q(x_{1:T} \mid x_0) = q(x_T \mid x_0) \prod_{t=2}^{T} q(x_{t-1} \mid x_t, x_0).$$

Therefore the log density ratio can be expanded as

$$\begin{aligned}\log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})} = \; & \log \frac{q(x_T \mid x_0)}{p(x_T)} \\ & + \sum_{t=2}^{T} \log \frac{q(x_{t-1} \mid x_t, x_0)}{p_\theta(x_{t-1} \mid x_t)} - \log p_\theta(x_0 \mid x_1).\end{aligned}$$

Taking the expectation with respect to the forward chain and applying the definition of conditional KL divergence produces the desired result. □

The terminal term L_T is independent of θ if the forward schedule and terminal prior are fixed. The $t = 1$ contribution L_0 is a reconstruction or observation-likelihood term. The intermediate terms compare two Gaussian distributions and lead directly to an elementary squared-error objective.

15.5.2 The Gaussian KL and Noise-Matching Loss

Lemma 15.1 (KL for Gaussians with equal isotropic covariance)**.** *Let* $P = \mathcal{N}(\mu_P, \sigma^2 I_d)$ *and* $Q = \mathcal{N}(\mu_Q, \sigma^2 I_d)$ *with* $\sigma^2 > 0$*. Then*

$$D_{\mathrm{KL}}(P \| Q) = \frac{\|\mu_P - \mu_Q\|_2^2}{2\sigma^2}.$$

Proof. The Gaussian log-density constants cancel. For $X \sim P$,

$$\begin{aligned}D_{\mathrm{KL}}(P \| Q) &= \mathbb{E}_P \left[\log \frac{p(X)}{q(X)} \right] \\ &= \frac{1}{2\sigma^2} \mathbb{E}_P \left[\|X - \mu_Q\|_2^2 - \|X - \mu_P\|_2^2 \right] \\ &= \frac{1}{2\sigma^2} \left[d\sigma^2 + \|\mu_P - \mu_Q\|_2^2 - d\sigma^2 \right] \\ &= \frac{\|\mu_P - \mu_Q\|_2^2}{2\sigma^2}.\end{aligned}$$

□

Now take $q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\widetilde{\mu}_t, \widetilde{\beta}_t I_d)$ and $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta, \sigma_t^2 I_d)$. If the variances agree, the lemma applies directly. More generally, when σ_t^2 and $\widetilde{\beta}_t$ are fixed rather than learned, the additional covariance-dependent terms in the Gaussian KL are constant in θ, so the parameter-dependent part is always

$$\frac{1}{2\sigma_t^2}\|\widetilde{\mu}_t - \mu_\theta\|_2^2.$$

Using the noise parameterization, the mean difference is

$$\begin{aligned}\widetilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right) - \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) \\ &= \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}\big(\varepsilon_\theta(x_t, t) - \varepsilon\big).\end{aligned}$$

Consequently,

$$\frac{1}{2\sigma_t^2}\|\widetilde{\mu}_t - \mu_\theta\|_2^2 = \frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2.$$

Proposition 15.6 (Noise-prediction form of the KL terms)**.** *For fixed reverse variances $\sigma_t^2 > 0$ and $t \geq 2$, the parameter-dependent part of the intermediate variational term is*

$$\begin{aligned}L_{t-1}(\theta) &= \lambda_t \mathbb{E}_{x_0,\varepsilon}\left\|\varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, t\right)\right\|_2^2 + C_t, \\ \lambda_t &:= \frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)},\end{aligned}$$

where C_t does not depend on θ, $x_0 \sim q_{\text{data}}$, and $\varepsilon \sim \mathcal{N}(0, I_d)$ independently.

Proof. The mean-squared identity above gives the parameter-dependent KL term. By the exact forward marginal, sampling $x_t \mid x_0$ is equivalent to sampling the displayed ε and forming $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$. Taking the expectation over the data and forward noise completes the formula. □

Remark 15.3 (Why the training objective can omit KL weights). The commonly used *simplified DDPM loss* replaces the timestep-dependent coefficient λ_t by a constant weight and usually samples the timestep uniformly:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{\substack{x_0 \sim q_{\text{data}},\ t \sim \text{Unif}\{1,\dots,T\}, \\ \varepsilon \sim \mathcal{N}(0, I_d)}}\left\|\varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, t\right)\right\|_2^2.$$

This is the loss emphasized in the handwritten lecture. It is closely motivated by the variational KL terms, but dropping λ_t and treating the $t = 1$ reconstruction contribution in the same way generally changes the objective: the simplified loss is not an exact algebraic restatement of the full negative ELBO.

15.6 What the Denoiser Learns

15.6.1 Optimal Prediction under Squared Error

The loss has an immediate regression interpretation. For a fixed timestep, let X_t denote the noisy observation and let ε be the noise used to construct it.

Proposition 15.7 (Conditional expectation minimizes noise-prediction MSE)**.** *Among all measurable functions f such that the relevant expectations are finite, the minimizer of*

$$\mathbb{E}\|\varepsilon - f(X_t)\|_2^2$$

is $f^*(X_t) = \mathbb{E}[\varepsilon \mid X_t]$ almost surely.

Proof. Write $m(X_t) = \mathbb{E}[\varepsilon \mid X_t]$ and decompose

$$\varepsilon - f(X_t) = \big(\varepsilon - m(X_t)\big) + \big(m(X_t) - f(X_t)\big).$$

Squaring, taking expectations, and using

$$\mathbb{E}\left[(\varepsilon - m(X_t))^\top (m(X_t) - f(X_t))\right] = 0$$

by the defining orthogonality property of conditional expectation, gives

$$\mathbb{E}\|\varepsilon - f(X_t)\|_2^2 = \mathbb{E}\|\varepsilon - m(X_t)\|_2^2 + \mathbb{E}\|m(X_t) - f(X_t)\|_2^2.$$

The second term is nonnegative and vanishes precisely for $f = m$ almost surely. □

Thus the ideal denoiser does not identify the *realized* noise for every possible noisy input; it estimates the conditional average noise under the data and forward-noising distributions. This distinction matters because different clean inputs and noise realizations can produce the same x_t.

15.6.2 Connection to the Score Function

For $t \geq 1$, the noisy marginal density is

$$q_t(x) = \int q_{\text{data}}(x_0)\mathcal{N}(x; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I_d)\mathrm{d}x_0.$$

The *score* of this density is $\nabla_x \log q_t(x)$. Under conditions permitting differentiation under the integral sign, differentiation of the Gaussian kernel yields

$$\begin{aligned}
\nabla_x q_t(x) &= -\int \frac{x - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t} q_{\text{data}}(x_0) q(x \mid x_0)\,\mathrm{d}x_0, \\
\nabla_x \log q_t(x) &= -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\mathbb{E}[\varepsilon \mid X_t = x].
\end{aligned}$$

Therefore the optimal noise predictor also supplies an estimate of the score:

$$\nabla_x \log q_t(x) \approx -\frac{\varepsilon_\theta(x,t)}{\sqrt{1-\bar{\alpha}_t}}.$$

This provides the connection between DDPM noise prediction and *denoising score matching*. Both attempt to learn how the log density changes as the state moves in data space, at several levels of injected noise.

Remark 15.4 (Connection with the clean-sample estimate). Combining the conditional expectation identity with the noise-prediction reconstruction formula gives

$$\mathbb{E}[X_0 \mid X_t = x] = \frac{x - \sqrt{1-\bar{\alpha}_t}\,\mathbb{E}[\varepsilon \mid X_t = x]}{\sqrt{\bar{\alpha}_t}}$$

whenever the indicated conditional expectations exist. In a sufficiently expressive model, the denoiser can therefore be interpreted as estimating either the conditional mean noise or the conditional mean clean input.

15.7 Training and Sampling Algorithms

15.7.1 Efficient Stochastic Training

The closed-form marginal eliminates the need to run the full forward Markov chain during each training step. One noisy pair can be constructed directly from x_0 and the selected timestep.

Definition 15.3 (Simplified diffusion training step)**.** Given parameters θ, one stochastic update proceeds as follows:

(i) Sample a training observation $x_0 \sim q_{\text{data}}$.
(ii) Sample a timestep $t \sim \text{Unif}\{1, \dots, T\}$.
(iii) Independently sample $\varepsilon \sim \mathcal{N}(0, I_d)$.
(iv) Form $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$.
(v) Evaluate $\ell(\theta) = \|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2$ and update θ with a gradient-based optimizer.

If θ has components $\theta_1, \dots, \theta_p$, and the loss is differentiable in θ, then the sample gradient is

$$\begin{aligned}\nabla_\theta \ell &= \nabla_\theta\big(\varepsilon - \varepsilon_\theta(x_t,t)\big)^\top\big(\varepsilon - \varepsilon_\theta(x_t,t)\big) \\ &= -2\big[D_\theta \varepsilon_\theta(x_t,t)\big]^\top\big(\varepsilon - \varepsilon_\theta(x_t,t)\big).\end{aligned}$$

Here $D_\theta \varepsilon_\theta \in \mathbb{R}^{d\times p}$ is the Jacobian of the predicted noise with respect to model parameters, and backpropagation computes the vector–Jacobian product efficiently.

15.7.2 Ancestral Reverse Sampling

After training, one samples from the reverse model by beginning at a standard Gaussian state and traversing time backward.

Definition 15.4 (Reverse diffusion sampling)**.** Initialize $x_T \sim \mathcal{N}(0, I_d)$. For $t = T, T-1, \ldots, 1$, compute

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

and set

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z_t, \qquad z_t \sim \mathcal{N}(0, I_d),$$

using independent z_t for $t \geq 2$ and taking $z_1 = 0$ in the standard sampling convention. The final x_0 is the generated sample.

Choosing $\sigma_t^2 = \beta_t$ for $t \geq 2$ gives the update

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sqrt{\beta_t}\, z_t.$$

Alternatively one can use $\sigma_t^2 = \widetilde{\beta}_t$ for $t \geq 2$. At $t = 1$, $\widetilde{\beta}_1 = 0$, consistent with a deterministic final denoising step under the usual simplified sampling rule.

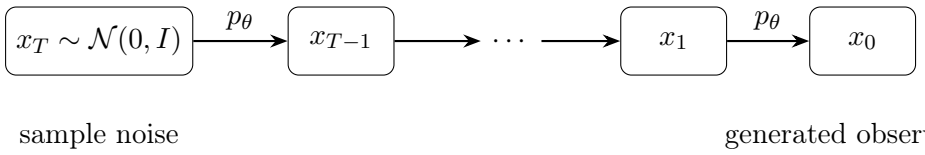


Figure 39: The learned reverse chain removes noise step by step, beginning from a Gaussian prior and ending with a generated data sample.

The reverse sampler has a different computational cost than the training step: one training example requires only one selected denoising evaluation, while one ancestral sample ordinarily requires T neural-network evaluations.

15.8 A Complete Numerical Example

Example 15.1 (Two diffusion steps in one dimension)**.** Consider $d = 1$ and the schedule

$$\beta_1 = 0.1, \qquad \beta_2 = 0.2, \qquad \alpha_1 = 0.9, \qquad \alpha_2 = 0.8.$$

Then

$$\bar{\alpha}_1 = 0.9, \qquad \bar{\alpha}_2 = 0.9 \cdot 0.8 = 0.72.$$

For $x_0 = 2$ and an illustrative sampled noise value $\varepsilon = 0.5$, the direct forward-noising formula gives

$$\begin{aligned} x_2 &= \sqrt{0.72}\,(2) + \sqrt{0.28}\,(0.5) \\ &\approx 1.697056 + 0.264575 = 1.961631. \end{aligned}$$

This is a single draw from $q(x_2 \mid x_0) = \mathcal{N}(\sqrt{0.72}\, x_0, 0.28)$, not the conditional mean.

The exact posterior variance for the preceding state is

$$\widetilde{\beta}_2 = \frac{1-\bar{\alpha}_1}{1-\bar{\alpha}_2}\beta_2 = \frac{0.1}{0.28}\cdot 0.2 = \frac{1}{14} \approx 0.0714286.$$

The posterior mean can be found directly using the known x_0:

$$\begin{aligned}\widetilde{\mu}_2(x_2, x_0) &= \frac{\sqrt{0.9}\,(0.2)}{0.28}\,(2) + \frac{\sqrt{0.8}\,(0.1)}{0.28}\,(1.961631)\\ &\approx 1.9819.\end{aligned}$$

Alternatively, use the equivalent noise formula:

$$\begin{aligned}\widetilde{\mu}_2 &= \frac{1}{\sqrt{0.8}}\left(x_2 - \frac{0.2}{\sqrt{0.28}}\varepsilon\right)\\ &= \frac{1}{\sqrt{0.8}}\left(1.961631 - \frac{0.2}{\sqrt{0.28}}(0.5)\right)\\ &\approx 1.9819.\end{aligned}$$

The two methods agree because they represent the same conditional Gaussian distribution. During *generation*, x_0 and the realized ε are unavailable; the learned predictor $\varepsilon_\theta(x_2, 2)$ substitutes for ε in the second formula.

15.9 Connections and Mathematical Caveats

15.9.1 Reverse Denoising as Approximate Bayesian Inference

The forward Gaussian channel defines a joint distribution over x_0 and x_t. The reverse network learns a transformation that encodes information about how likely clean data are to have produced the observed noisy state. Thus reverse diffusion is a form of approximate Bayesian inference performed repeatedly across noise scales. The exact posterior conditional on x_0 is Gaussian, while the distribution of x_0 conditional on x_t is governed by the unknown data distribution. This is why denoising is a learning problem rather than a purely algebraic inversion problem.

15.9.2 The Role of the Noise Schedule

The choice of $\{\beta_t\}$ affects the relationship between time and signal-to-noise ratio, the approximation $q(x_T) \approx \mathcal{N}(0, I_d)$, the magnitude of the reverse stochastic increments, and the relative weighting of the KL-derived training objectives. Schedules are a modeling choice, not a universal mathematical constant. In particular, a schedule with $\bar{\alpha}_T$ not close to zero does not justify replacing the terminal marginal by a standard Gaussian without approximation error.

15.9.3 Comparison with Variational Autoencoders

Both VAEs and diffusion models learn to generate observations from latent random variables and can be trained using variational ideas. A VAE usually uses a direct decoder $p_\theta(x \mid z)$ from

one latent variable z and an amortized encoder $q_\phi(z \mid x)$. A DDPM instead uses a sequence of latent variables $x_1, \dots, x_T$ with a known forward process and a sequence of learned reverse transitions. The diffusion loss can be expressed as noise prediction because the intermediate conditional Gaussians are analytically tractable.

15.10 Exercises

Exercise 1.

Suppose $\beta_t = \beta \in (0, 1)$ is constant. Determine $\bar{\alpha}_t$, the conditional distribution $q(x_t \mid x_0)$, and the limit as $t \to \infty$.

Solution. Since $\alpha_t = 1 - \beta$,

$$\begin{aligned} \bar{\alpha}_t &= (1-\beta)^t, \\ q(x_t \mid x_0) &= \mathcal{N}\left(\sqrt{(1-\beta)^t}\, x_0, \big(1 - (1-\beta)^t\big) I_d\right). \end{aligned}$$

Because $0 < 1 - \beta < 1$, $(1-\beta)^t \to 0$, so for any fixed x_0 the conditional distribution converges to $\mathcal{N}(0, I_d)$. ■

Exercise 2.

Prove the telescoping identity

$$\sum_{j=1}^{t} \beta_j \prod_{s=j+1}^{t} \alpha_s = 1 - \bar{\alpha}_t.$$

Solution. Using $\beta_j = 1 - \alpha_j$,

$$\begin{aligned} \sum_{j=1}^{t} (1-\alpha_j) \prod_{s=j+1}^{t} \alpha_s &= \sum_{j=1}^{t} \left(\prod_{s=j+1}^{t} \alpha_s - \prod_{s=j}^{t} \alpha_s \right) \\ &= 1 - \prod_{s=1}^{t} \alpha_s = 1 - \bar{\alpha}_t. \end{aligned}$$

Here an empty product is interpreted as 1. ■

Exercise 3.

With $\sigma_t^2 = \beta_t$ for $t \geq 2$, simplify the coefficient λ_t in the noise-matching KL loss. Determine how the result changes when $\sigma_t^2 = \widetilde{\beta}_t$.

Solution. By definition,

$$\lambda_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)}.$$

With $\sigma_t^2 = \beta_t$,

$$\lambda_t = \frac{\beta_t}{2\alpha_t(1-\bar{\alpha}_t)}.$$

With $\sigma_t^2 = \widetilde{\beta}_t = \beta_t(1-\bar{\alpha}_{t-1})/(1-\bar{\alpha}_t)$,

$$\lambda_t = \frac{\beta_t}{2\alpha_t(1-\bar{\alpha}_{t-1})}.$$

Both formulas apply for $t \geq 2$, where the posterior variance is nonzero. ■

Exercise 4.

Suppose an oracle returns the exact noise ε used to generate $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$. Show that the clean-observation estimator and the learned reverse mean recover their exact posterior counterparts.

Solution. Set $\varepsilon_\theta(x_t, t) = \varepsilon$. Then

$$\widehat{x}_{0,\theta}(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon}{\sqrt{\bar{\alpha}_t}} = x_0.$$

Substitution into μ_θ gives

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right) = \widetilde{\mu}_t(x_t, x_0).$$

For $t \geq 2$, this establishes equality of the means. The sampled reverse step can still contain additional Gaussian randomness because the posterior variance is positive. ■

Exercise 5.

Let $X_0 \sim \mathcal{N}(0, I_d)$, independent of all forward noises. Determine the unconditional distribution of X_t for any schedule, and explain what is unusual about this special data distribution.

Solution. Using the exact forward marginal,

$$X_t = \sqrt{\bar{\alpha}_t}X_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon.$$

The two summands are independent centered Gaussian vectors, so

$$\mathrm{Cov}(X_t) = \bar{\alpha}_t I_d + (1-\bar{\alpha}_t)I_d = I_d.$$

Thus $X_t \sim \mathcal{N}(0, I_d)$ for every t. The initial distribution is already the terminal Gaussian distribution, so the forward diffusion does not change its marginal distribution. In ordinary generative modeling the data marginal is not already standard Gaussian, making the gradual transition informative. ■

Exercise 6.

Derive the conditional score of $q(x_t \mid x_0)$ with respect to x_t, and compare it with the score of the unconditional noisy marginal $q_t(x_t)$.

Solution. Since $q(x_t \mid x_0)$ is Gaussian with mean $\sqrt{\bar{\alpha}_t}x_0$ and covariance $(1-\bar{\alpha}_t)I_d$,

$$\begin{aligned}\nabla_{x_t} \log q(x_t \mid x_0) &= -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t} \\ &= -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}.\end{aligned}$$

The marginal score averages these conditional scores under the posterior of X_0 given $X_t = x_t$:

$$\nabla_{x_t} \log q_t(x_t) = -\frac{\mathbb{E}[\varepsilon \mid X_t = x_t]}{\sqrt{1-\bar{\alpha}_t}}.$$

The conditional score uses the realized noise, whereas the marginal score uses its conditional expectation given the noisy observation. ■

16 Lecture 17: Recurrent Neural Networks and Learning from Sequences

Many machine-learning tasks involve an ordered sequence of observations rather than a fixed-dimensional input. Examples include predicting the next word in a sentence, classifying an evolving sensor trace, recognizing speech, and forecasting a time series. In each case, the significance of an input depends not only on its value, but also on the observations that came before it. A *recurrent neural network* (RNN) represents the information available at time t using a hidden state that is updated sequentially. The same parameters are reused at every time step.

Our central questions are: how does an RNN process a sequence, how does it turn its hidden state into a probability distribution over outputs, how do we formulate a sequence-level maximum-likelihood objective, and how are its shared parameters differentiated through time? We subsequently extend the model to multiple recurrent layers.

16.1 Sequences and the Recurrent Hidden State

16.1.1 Notation and Dimensions

Let $T \geq 1$ denote a sequence length. At time $t \in \{1, \ldots, T\}$, let

$$x^{(t)} \in \mathbb{R}^{1\times d}, \qquad h^{(t)} \in \mathbb{R}^{1\times H}, \qquad y^{(t)} \in \mathbb{R}^{1\times K}.$$

Here d is the input dimension, H the number of hidden units, and K the number of possible output classes. We use *row-vector notation*, consistent with the lecture's expressions $x^{(t)}U + h^{(t-1)}W$ and $h^{(t)}V$. Each $y^{(t)}$ will be a vector of class probabilities. The superscript (t) indicates time, not exponentiation. The trainable parameters are

$$\begin{aligned}
U &\in \mathbb{R}^{d\times H} && \text{(input-to-hidden weights)},\\
W &\in \mathbb{R}^{H\times H} && \text{(hidden-to-hidden recurrent weights)},\\
V &\in \mathbb{R}^{H\times K} && \text{(hidden-to-output weights)},\\
b &\in \mathbb{R}^{1\times H} && \text{(hidden bias)},\\
c &\in \mathbb{R}^{1\times K} && \text{(output bias)}.
\end{aligned}$$

The initial hidden state $h^{(0)} \in \mathbb{R}^{1\times H}$ is either prescribed (often zero) or treated as another learnable parameter.

Definition 16.1 (A simple recurrent neural network)**.** Given a componentwise activation function $\phi : \mathbb{R} \to \mathbb{R}$, the recurrent state and output equations are

$$\begin{aligned}
a^{(t)} &= x^{(t)}U + h^{(t-1)}W + b,\\
h^{(t)} &= \phi(a^{(t)}),\\
o^{(t)} &= h^{(t)}V + c,\\
y^{(t)} &= \mathrm{softmax}(o^{(t)}), \qquad t = 1, \ldots, T.
\end{aligned}$$

The vector $a^{(t)}$ is the *hidden pre-activation*, $h^{(t)}$ the *hidden state*, and $o^{(t)}$ the output *logits*.

For example, ϕ is commonly the hyperbolic tangent,

$$\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \phi'(z) = 1 - \tanh^2(z).$$

Other nonlinearities can be used, but the simple RNN with a tanh state is especially convenient for studying differentiation through time. Unlike a feedforward hidden layer, the state receives *both* a new input $x^{(t)}$ and the previous state $h^{(t-1)}$.

Remark 16.1 (What the three weight matrices do). The term $x^{(t)}U$ embeds the current observation into the hidden-state space. The term $h^{(t-1)}W$ carries information forward from past observations. Finally, $h^{(t)}V + c$ maps the current state into K output logits. These are distinct mappings, with different source and destination dimensions.

16.1.2 An RNN as a Network Unrolled in Time

The recurrent diagram has two equivalent interpretations: a compact graph with a self-loop on the hidden layer, or an *unrolled computational graph* with one copy of the hidden computation for each time step. The copies do not have different parameters: they share the same U, W, V, b, c.

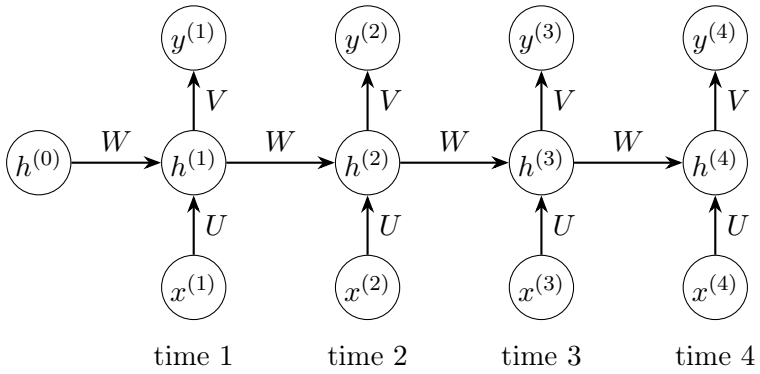


Figure 40: An RNN unrolled over four time steps. Each recurrent cell receives its input from below and the preceding hidden state from the left. All horizontal arrows use the same W, all input arrows the same U, and all output arrows the same V.

In particular, for the first three steps we have

$$\begin{aligned} h^{(1)} &= \phi(x^{(1)}U + h^{(0)}W + b), \\ h^{(2)} &= \phi(x^{(2)}U + h^{(1)}W + b), \\ h^{(3)} &= \phi(x^{(3)}U + h^{(2)}W + b). \end{aligned}$$

Substituting recursively exhibits the dependence on the entire observed history:

$$\begin{aligned} h^{(2)} &= \phi\big(x^{(2)}U + \phi(x^{(1)}U + h^{(0)}W + b)W + b\big)\,, \\ h^{(3)} &= \phi\big(x^{(3)}U + \phi\big(x^{(2)}U + \phi(x^{(1)}U + h^{(0)}W + b)W + b\big)\,W + b\big)\,. \end{aligned}$$

This repeated composition is the mathematical source of recurrent memory. It also explains why derivatives with respect to early inputs or shared parameters involve products of Jacobians over time.

Proposition 16.1 (Causality of the hidden state)**.** *If $h^{(0)}$ is fixed and the state update is deterministic, then for every $t \geq 1$ there is a function F_t such that*

$$h^{(t)} = F_t\big(x^{(1)}, \ldots, x^{(t)}\big).$$

In particular, the state at time t does not require any future input $x^{(s)}$, $s > t$.

Proof. For $t = 1$, set $F_1(x^{(1)}) = \phi(x^{(1)}U + h^{(0)}W + b)$. Suppose the claim holds at time $t-1$. Then

$$\begin{aligned} h^{(t)} &= \phi\big(x^{(t)}U + h^{(t-1)}W + b\big) \\ &= \phi\big(x^{(t)}U + F_{t-1}(x^{(1)}, \ldots, x^{(t-1)})W + b\big), \end{aligned}$$

which defines the required F_t. The result follows by induction. □

Remark 16.2 (A compressed state is not necessarily sufficient). The hidden state is a learned summary of the input prefix, but it need not retain every detail of the history. Nor does a generic RNN guarantee a mathematically sufficient statistic for a true data-generating process. Its effectiveness depends on the parameters, hidden dimension, training objective, and the task.

16.2 Output Distributions and Sequence Prediction

16.2.1 Softmax at Each Time Step

For a logit vector $o^{(t)} = (o_1^{(t)}, \ldots, o_K^{(t)})$, define

$$y_k^{(t)} = \frac{\exp(o_k^{(t)})}{\sum_{j=1}^{K} \exp(o_j^{(t)})}, \qquad k = 1, \ldots, K.$$

This gives $y_k^{(t)} > 0$ and $\sum_{k=1}^{K} y_k^{(t)} = 1$. We can therefore interpret

$$p_\theta\big(Y^{(t)} = k \mid x^{(1)}, \ldots, x^{(t)}\big) := y_k^{(t)},$$

where $\theta = (U, W, V, b, c)$ denotes all trainable parameters. The superscript (t) on $Y^{(t)}$ indexes a discrete class label at time t.

Definition 16.2 (Sequence-to-sequence classification)**.** In a many-to-many prediction problem, each input $x^{(t)}$ is associated with a target class $k_t \in \{1, \ldots, K\}$. The RNN produces the sequence of conditional probability vectors $y^{(1)}, \ldots, y^{(T)}$ and incurs a loss at each time step.

If $q^{(t)} \in \{0,1\}^{1\times K}$ is the one-hot target vector with $q^{(t)}_{k_t} = 1$, then the time-t categorical negative log-likelihood is

$$\begin{aligned}\ell_t(\theta) &= -\log y^{(t)}_{k_t} = -\sum_{k=1}^{K} q^{(t)}_k \log y^{(t)}_k \\ &= -o^{(t)}_{k_t} + \log\left(\sum_{j=1}^{K} e^{o^{(t)}_j}\right).\end{aligned}$$

The last expression is often preferable for stable implementation using a combined log-softmax and cross-entropy operation.

16.2.2 Conditional Factorization and Sequence Likelihood

The conditional model described so far uses $x^{(1:t)}$ to produce the distribution of $Y^{(t)}$. If we *define* its sequence-label distribution as a product of per-step categorical conditionals, we obtain

$$\begin{aligned}p_\theta(k_{1:T} \mid x_{1:T}) &= \prod_{t=1}^{T} p_\theta(k_t \mid x_{1:t}) \\ &= \prod_{t=1}^{T} y^{(t)}_{k_t}.\end{aligned}$$

This product asserts conditional independence of the labels given the deterministic input-driven states under the chosen model. The fact that the RNN has a hidden state by itself does not imply such independence for an arbitrary data-generating distribution; it is a modeling choice.

Taking the negative logarithm yields

$$\begin{aligned}-\log p_\theta(k_{1:T} \mid x_{1:T}) &= -\log \prod_{t=1}^{T} y^{(t)}_{k_t} \\ &= -\sum_{t=1}^{T} \log y^{(t)}_{k_t} \\ &= \sum_{t=1}^{T} \ell_t(\theta).\end{aligned}$$

This is the *sum-of-steps training objective* used in the handwritten lecture.

For autoregressive language modeling, the inputs at time t are selected from preceding observed tokens; one commonly defines

$$p_\theta(w_1, \ldots, w_T) = \prod_{t=1}^{T} p_\theta(w_t \mid w_1, \ldots, w_{t-1}).$$

In that setting the chain-rule factorization is a factorization of the joint token distribution, and the RNN approximates each conditional distribution. The indexing of inputs and targets must be shifted consistently: to predict w_t, the network cannot already be given w_t as an input.

16.2.3 Expected Negative Log-Likelihood and Maximum Likelihood

Let a supervised training set consist of N independent observed sequences,

$$\mathcal{D} = \left\{ (x^{(n)}_{1:T_n}, k^{(n)}_{1:T_n}) \right\}_{n=1}^{N}.$$

The maximum-likelihood estimator is a minimizer of the empirical negative log-likelihood:

$$\begin{aligned} \hat{\theta} &\in \arg\max_{\theta} \sum_{n=1}^{N} \log p_\theta(k^{(n)}_{1:T_n} \mid x^{(n)}_{1:T_n}) \\ &= \arg\min_{\theta} \left\{ -\frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} \log y^{(n,t)}_{k^{(n)}_t} \right\}. \end{aligned}$$

The corresponding population objective is

$$\mathcal{R}(\theta) = \mathbb{E}_{(X,K)\sim P_{\text{data}}} \left[-\sum_{t=1}^{T(X)} \log p_\theta(K_t \mid X_{1:t}) \right].$$

Minimizing $\mathcal{R}(\theta)$ is the ideal objective, but it involves the unknown data distribution. Training instead minimizes its empirical approximation over the available sample. The handwritten notes use the phrase *expected loss*; the empirical and population objectives are conceptually different.

Remark 16.3 (Sum versus mean loss). One may divide the sequence loss by T, by the number of valid target tokens, or by the number of sequences, depending on the task. Such normalizations change the weighting of examples and the numerical gradient scale, but not the core likelihood calculation for a fixed-length data set.

16.3 Differentiating the Sequence Loss

Because the parameters U, W, V, b, c are reused at each step, the gradient must collect contributions from *every* use of each parameter. This is *backpropagation through time* (BPTT): ordinary reverse-mode automatic differentiation applied to the time-unrolled computational graph.

16.3.1 Derivative of Softmax Cross-Entropy

Let $p = \text{softmax}(o)$ and let q be a one-hot target. We first derive the gradient of the single-step loss

$$\ell(o, q) = -\sum_{k=1}^{K} q_k \log p_k.$$

Proposition 16.2 (Softmax cross-entropy derivative)**.** *If $\sum_k q_k = 1$, then, in row-vector notation,*

$$\frac{\partial \ell}{\partial o} = p - q.$$

Equivalently, $\partial \ell / \partial o_j = p_j - q_j$ for every j.

Proof. Let $Z = \sum_{j=1}^{K} e^{o_j}$. Because $\log p_k = o_k - \log Z$,

$$\begin{aligned}\ell(o, q) &= -\sum_{k=1}^{K} q_k (o_k - \log Z) \\ &= -\sum_{k=1}^{K} q_k o_k + \log Z.\end{aligned}$$

Differentiating with respect to o_j gives

$$\begin{aligned}\frac{\partial \ell}{\partial o_j} &= -q_j + \frac{1}{Z}\frac{\partial Z}{\partial o_j} \\ &= -q_j + \frac{e^{o_j}}{\sum_{k=1}^{K} e^{o_k}} \\ &= p_j - q_j.\end{aligned}$$

□

Define the row-vector *output error* at time t as

$$e^{(t)} := \frac{\partial \ell_t}{\partial o^{(t)}} = y^{(t)} - q^{(t)} \in \mathbb{R}^{1\times K}.$$

As each output logit depends affinely on $h^{(t)}$, its direct contribution to the state gradient is

$$\left.\frac{\partial \mathcal{L}}{\partial h^{(t)}}\right|_{\text{output at } t} = e^{(t)} V^{\top} \in \mathbb{R}^{1\times H}.$$

We emphasize *direct*: the hidden state also affects the future via the recurrent connection.

16.3.2 Backward Recursion Through Time

Let

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t, \qquad \delta^{(t)} := \frac{\partial \mathcal{L}}{\partial a^{(t)}} \in \mathbb{R}^{1\times H}.$$

Here and below $\partial \mathcal{L}/\partial v$ denotes the row of derivatives with respect to the components of the row-vector v. Since $h^{(t)}$ influences both the output logits $o^{(t)}$ and the next pre-activation $a^{(t+1)}$, the total derivative obeys

$$\frac{\partial \mathcal{L}}{\partial h^{(t)}} = \underbrace{e^{(t)} V^{\top}}_{\text{current output}} + \underbrace{\delta^{(t+1)} W^{\top}}_{\text{future recurrent effect}}, \qquad t < T.$$

At the final time, there is no future recurrent contribution. Set $\delta^{(T+1)} = 0$ to combine all times in one equation.

Theorem 16.1 (Backpropagation through time)**.** *Assume ϕ is differentiable, and define $\delta^{(T+1)} = 0$. Then for $t = T, T-1, \ldots, 1$,*

$$\delta^{(t)} = \big(e^{(t)}V^\top + \delta^{(t+1)}W^\top\big) \odot \phi'\big(a^{(t)}\big),$$

where $\odot$ denotes componentwise multiplication. The gradients of the shared parameters are

$$\nabla_V \mathcal{L} = \sum_{t=1}^{T} (h^{(t)})^\top e^{(t)}, \qquad \nabla_c \mathcal{L} = \sum_{t=1}^{T} e^{(t)},$$
$$\nabla_U \mathcal{L} = \sum_{t=1}^{T} (x^{(t)})^\top \delta^{(t)}, \qquad \nabla_b \mathcal{L} = \sum_{t=1}^{T} \delta^{(t)},$$
$$\nabla_W \mathcal{L} = \sum_{t=1}^{T} (h^{(t-1)})^\top \delta^{(t)}.$$

If the initial hidden state is learnable, then

$$\frac{\partial \mathcal{L}}{\partial h^{(0)}} = \delta^{(1)} W^\top.$$

Proof. First consider the direct output dependence $o^{(t)} = h^{(t)}V + c$. For individual coordinates,

$$\begin{aligned}
\frac{\partial \ell_t}{\partial V_{jk}} &= \sum_{r=1}^{K} \frac{\partial \ell_t}{\partial o_r^{(t)}} \frac{\partial o_r^{(t)}}{\partial V_{jk}} \\
&= \sum_{r=1}^{K} e_r^{(t)} h_j^{(t)} \mathbf{1}_{\{r=k\}} \\
&= h_j^{(t)} e_k^{(t)}.
\end{aligned}$$

Thus the direct output-layer contributions are $(h^{(t)})^\top e^{(t)}$ and $e^{(t)}$, summed over all time steps.

Next the state $h^{(t)}$ has two outgoing paths: one to $o^{(t)}$, and one to $a^{(t+1)} = x^{(t+1)}U + h^{(t)}W + b$. The chain rule gives

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial h_j^{(t)}} &= \sum_{k=1}^{K} e_k^{(t)} V_{jk} + \sum_{r=1}^{H} \delta_r^{(t+1)} W_{jr} \\
&= \big(e^{(t)}V^\top + \delta^{(t+1)}W^\top\big)_j .
\end{aligned}$$

Because $h_j^{(t)} = \phi(a_j^{(t)})$, another application of the chain rule gives

$$\begin{aligned}
\delta_j^{(t)} &= \frac{\partial \mathcal{L}}{\partial h_j^{(t)}} \frac{\partial h_j^{(t)}}{\partial a_j^{(t)}} \\
&= \big(e^{(t)}V^\top + \delta^{(t+1)}W^\top\big)_j \, \phi'(a_j^{(t)}),
\end{aligned}$$

which establishes the backward recurrence.

Finally, each shared input weight U_{ij} appears in every $a_j^{(t)}$, so

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial U_{ij}} &= \sum_{t=1}^{T}\sum_{r=1}^{H} \frac{\partial \mathcal{L}}{\partial a_r^{(t)}}\frac{\partial a_r^{(t)}}{\partial U_{ij}} \\ &= \sum_{t=1}^{T} \delta_j^{(t)} x_i^{(t)}.\end{aligned}$$

Similarly,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_{ij}} &= \sum_{t=1}^{T} h_i^{(t-1)}\delta_j^{(t)}, \\ \frac{\partial \mathcal{L}}{\partial b_j} &= \sum_{t=1}^{T} \delta_j^{(t)}.\end{aligned}$$

These coordinate expressions are precisely the stated matrix gradients. The formula for $h^{(0)}$ follows from its single outgoing dependence on $a^{(1)}$. □

Remark 16.4 (Why a future gradient is necessary). If one differentiated only ℓ_t with respect to the hidden state at time t, the term $\delta^{(t+1)}W^\top$ would be missing. This would treat the hidden state as if it had no effect on later predictions and would not compute the gradient of the full sequence loss. Backpropagation through time is simply the ordinary chain rule applied to the entire unrolled graph.

16.3.3 The Gradient as a Sum Over Temporal Paths

The recurrence can be expanded to see precisely how future time steps affect an earlier hidden state. Let

$$D_t := \operatorname{diag}\big(\phi'(a^{(t)})\big) \in \mathbb{R}^{H\times H}.$$

Since $\delta^{(t)} = (e^{(t)}V^\top + \delta^{(t+1)}W^\top)D_t$,

$$\begin{aligned}\delta^{(t)} &= e^{(t)}V^\top D_t + \delta^{(t+1)}W^\top D_t \\ &= e^{(t)}V^\top D_t + e^{(t+1)}V^\top D_{t+1}W^\top D_t + \delta^{(t+2)}W^\top D_{t+1}W^\top D_t \\ &= e^{(t)}V^\top D_t + e^{(t+1)}V^\top D_{t+1}W^\top D_t \\ &\quad + e^{(t+2)}V^\top D_{t+2}W^\top D_{t+1}W^\top D_t + \cdots .\end{aligned}$$

Because $\delta^{(T+1)} = 0$, this expansion terminates at $s = T$. The contribution from the output loss at any particular time $s > t$ to the earlier pre-activation gradient is

$$e^{(s)}V^\top D_s W^\top D_{s-1}W^\top \cdots W^\top D_t,$$

while the contribution for $s = t$ is simply $e^{(t)}V^\top D_t$. This expression makes clear that gradients from a late prediction must travel through many recurrent Jacobians before reaching an early state.

16.4 A Worked Numerical Example of BPTT

It is instructive to compute the entire backward pass in the simplest scalar recurrent model. This example uses a linear hidden activation and squared-error outputs so that every gradient is visible without the additional algebra of softmax.

Example 16.1 (Two-step scalar recurrent network)**.** Set $h^{(0)} = 0$ and consider

$$h^{(t)} = ux^{(t)} + wh^{(t-1)} + b, \qquad \hat{y}^{(t)} = vh^{(t)} + c,$$

with inputs $x^{(1)} = 1$, $x^{(2)} = 2$ and targets $r^{(1)} = 1$, $r^{(2)} = 3$. Let the parameters be

$$u = 1, \qquad w = \frac{1}{2}, \qquad v = 2, \qquad b = c = 0.$$

The loss is

$$\mathcal{L} = \frac{1}{2}\sum_{t=1}^{2}\big(\hat{y}^{(t)} - r^{(t)}\big)^2.$$

The forward pass gives

$$\begin{aligned} h^{(1)} &= 1 \cdot 1 + \frac{1}{2}\cdot 0 = 1, & \hat{y}^{(1)} &= 2, \\ h^{(2)} &= 1 \cdot 2 + \frac{1}{2}\cdot 1 = \frac{5}{2}, & \hat{y}^{(2)} &= 5, \\ \mathcal{L} &= \frac{1}{2}(2-1)^2 + \frac{1}{2}(5-3)^2 & &= \frac{5}{2}. \end{aligned}$$

For each output, the derivative with respect to its prediction is $e^{(t)} = \hat{y}^{(t)} - r^{(t)}$, hence $e^{(1)} = 1$ and $e^{(2)} = 2$. The hidden activation is the identity, so its derivative is 1. At the last step,

$$\delta^{(2)} = e^{(2)}v = 2 \cdot 2 = 4.$$

At the first step, the *future* contribution must be included:

$$\begin{aligned} \delta^{(1)} &= e^{(1)}v + w\delta^{(2)} \\ &= 1 \cdot 2 + \frac{1}{2}\cdot 4 = 4. \end{aligned}$$

Therefore,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial u} &= x^{(1)}\delta^{(1)} + x^{(2)}\delta^{(2)} = 1 \cdot 4 + 2 \cdot 4 = 12, \\ \frac{\partial \mathcal{L}}{\partial w} &= h^{(0)}\delta^{(1)} + h^{(1)}\delta^{(2)} = 0 \cdot 4 + 1 \cdot 4 = 4, \\ \frac{\partial \mathcal{L}}{\partial v} &= h^{(1)}e^{(1)} + h^{(2)}e^{(2)} = 1 \cdot 1 + \frac{5}{2}\cdot 2 = 6, \\ \frac{\partial \mathcal{L}}{\partial b} &= \delta^{(1)} + \delta^{(2)} = 8, \\ \frac{\partial \mathcal{L}}{\partial c} &= e^{(1)} + e^{(2)} = 3. \end{aligned}$$

If one omitted the recurrent contribution $w\delta^{(2)}$, the first hidden-state gradient would have been 2 instead of 4, and the derivative with respect to u would incorrectly be 10 instead of 12.

16.5 Stacked Recurrent Neural Networks

The handwritten lecture also extends the simple recurrent model by replacing a single hidden state with several hidden layers at each time step. Each layer receives two streams of information: the same-time output of the preceding layer and its own state from the preceding time step.

16.5.1 Forward Equations for Multiple Recurrent Layers

Suppose there are L recurrent layers. Let $H_0 = d$ and H_ℓ denote the hidden dimension of layer ℓ. Define the layer-zero state by

$$h^{(t,0)} := x^{(t)} \in \mathbb{R}^{1\times H_0}.$$

For each layer $\ell = 1, \dots, L$, introduce

$$U_\ell \in \mathbb{R}^{H_{\ell-1}\times H_\ell}, \qquad W_\ell \in \mathbb{R}^{H_\ell \times H_\ell}, \qquad b_\ell \in \mathbb{R}^{1\times H_\ell}.$$

Definition 16.3 (Stacked RNN)**.** The hidden-state recurrence for layer ℓ is

$$\begin{aligned} a^{(t,\ell)} &= h^{(t,\ell-1)}U_\ell + h^{(t-1,\ell)}W_\ell + b_\ell, \\ h^{(t,\ell)} &= \phi_\ell(a^{(t,\ell)}), \qquad \ell = 1, \dots, L. \end{aligned}$$

The top hidden state produces the output distribution

$$y^{(t)} = \operatorname{softmax}\big(h^{(t,L)}V + c\big), \qquad V \in \mathbb{R}^{H_L\times K}.$$

For example, with two recurrent layers,

$$\begin{aligned} h^{(t,1)} &= \phi_1\big(x^{(t)}U_1 + h^{(t-1,1)}W_1 + b_1\big), \\ h^{(t,2)} &= \phi_2\big(h^{(t,1)}U_2 + h^{(t-1,2)}W_2 + b_2\big), \\ y^{(t)} &= \operatorname{softmax}\big(h^{(t,2)}V + c\big). \end{aligned}$$

Thus the state $h^{(t,2)}$ combines information transmitted *vertically* by layer 1 at time t and *horizontally* by layer 2 from time $t-1$.

16.5.2 Backpropagation in Time and Depth

For the stacked model, each hidden unit may influence the loss in three ways: its contribution to the current output (only at the top layer), its contribution to the same-time higher layer, and its contribution to the next-time state of its own layer. Thus the backward recursion must propagate gradients *both upward in depth and backward in time*.

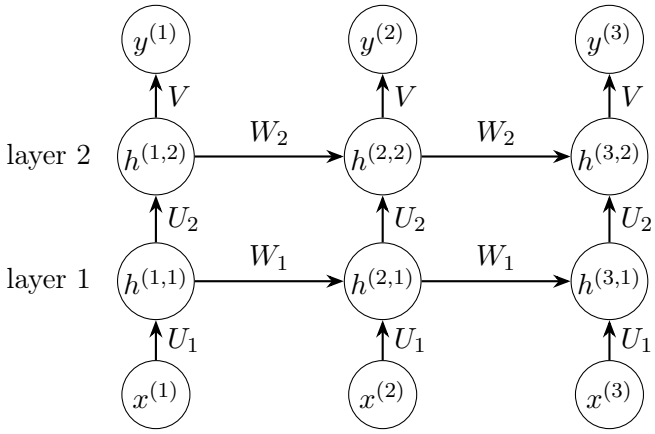


Figure 41: A two-layer RNN unrolled across three time steps. Vertical arrows perform computation across layers, and horizontal arrows propagate recurrent state through time.

Proposition 16.3 (Stacked RNN backward recursion)**.** *Let*

$$\delta^{(t,\ell)} := \frac{\partial \mathcal{L}}{\partial a^{(t,\ell)}}, \qquad e^{(t)} := y^{(t)} - q^{(t)}.$$

Set $\delta^{(T+1,\ell)} = 0$ *and* $\delta^{(t,L+1)} = 0$*. Then*

$$\delta^{(t,\ell)} = \left[\mathbf{1}_{\{\ell=L\}} e^{(t)} V^\top + \mathbf{1}_{\{\ell<L\}} \delta^{(t,\ell+1)} U_{\ell+1}^\top + \delta^{(t+1,\ell)} W_\ell^\top\right] \odot \phi_\ell'(a^{(t,\ell)}).$$

The gradients of the recurrent-layer parameters are

$$\nabla_{U_\ell} \mathcal{L} = \sum_{t=1}^{T} (h^{(t,\ell-1)})^\top \delta^{(t,\ell)},$$
$$\nabla_{W_\ell} \mathcal{L} = \sum_{t=1}^{T} (h^{(t-1,\ell)})^\top \delta^{(t,\ell)},$$
$$\nabla_{b_\ell} \mathcal{L} = \sum_{t=1}^{T} \delta^{(t,\ell)}.$$

Proof. The output-layer term is obtained exactly as in the single-layer case. For a layer $\ell < L$, its state enters the next layer's pre-activation through $h^{(t,\ell)} U_{\ell+1}$. Hence this same-time path contributes

$$\delta^{(t,\ell+1)} U_{\ell+1}^\top$$

to the gradient with respect to $h^{(t,\ell)}$. Independently, the hidden state enters the next-time pre-activation in its own layer through $h^{(t,\ell)} W_\ell$, contributing

$$\delta^{(t+1,\ell)} W_\ell^\top.$$

At $\ell = L$, the direct output term contributes $e^{(t)} V^\top$. Summing the outgoing gradient contributions and multiplying componentwise by $\phi_\ell'(a^{(t,\ell)})$ gives the stated recurrence. Finally, U_ℓ, W_ℓ, b_ℓ

are used at every time step, so their gradients add the corresponding local partial derivatives over $t = 1, \dots, T$. □

Remark 16.5 (Order of backward computation). An implementation computes the forward states by increasing time t and, within each time step, increasing layer ℓ. In the backward pass, one traverses the unrolled graph in the opposite topological order: from the last time toward the first, and from the top layer toward the bottom where dependencies permit. Automatic differentiation performs this ordering automatically.

16.6 Vanishing and Exploding Gradients in RNNs

The recurrence was designed to preserve information over time, yet training long-range dependencies can be difficult. The reason is that the chain rule multiplies the Jacobian of the recurrent state map repeatedly.

16.6.1 Jacobian of a Single Recurrent Step

With row-vector states, the linearization of the map $h^{(t-1)} \mapsto h^{(t)}$ is

$$\mathrm{d}h^{(t)} = \mathrm{d}h^{(t-1)} W D_t, \qquad D_t = \operatorname{diag}\big(\phi'(a^{(t)})\big).$$

Hence for $s < t$,

$$\mathrm{d}h^{(t)} = \mathrm{d}h^{(s)} \big(W D_{s+1}\big)\big(W D_{s+2}\big) \cdots \big(W D_t\big).$$

This product governs the sensitivity of the later hidden state to the earlier hidden state.

Proposition 16.4 (A sufficient gradient-decay bound)**.** *Assume there exists $M \geq 0$ such that $|\phi'(z)| \leq M$ for all relevant activations. For the spectral matrix norm $\|\cdot\|_2$,*

$$\left\| \prod_{r=s+1}^{t} W D_r \right\|_2 \leq \big(M\|W\|_2\big)^{t-s}.$$

Consequently, if $M\|W\|_2 < 1$, sensitivity to sufficiently distant states decays at least geometrically in the time separation.

Proof. For every r, the diagonal structure gives

$$\|D_r\|_2 = \max_j |\phi'(a_j^{(r)})| \leq M.$$

By submultiplicativity of the spectral norm,

$$\begin{aligned} \left\| \prod_{r=s+1}^{t} W D_r \right\|_2 &\leq \prod_{r=s+1}^{t} \|W\|_2 \|D_r\|_2 \\ &\leq \prod_{r=s+1}^{t} M\|W\|_2 \\ &= \big(M\|W\|_2\big)^{t-s}. \end{aligned}$$

If the base is smaller than one, this upper bound converges to zero as the separation grows. □

For $\phi = \tanh$, we have $0 < \phi'(z) \leq 1$ and $\phi'(z) \to 0$ as $|z| \to \infty$. Saturated activations can therefore further suppress the backward signal. Conversely, a bound $M\|W\|_2 > 1$ allows amplification but does not itself prove exploding gradients, since the actual matrices and directions also matter.

Remark 16.6 (Why long-range dependencies are difficult). A useful effect of $x^{(s)}$ on a prediction at a much later time t must survive many recurrent transformations. When backward sensitivities repeatedly shrink, early parameters receive weak learning signals. When sensitivities repeatedly grow, updates may become unstable. These phenomena are analogous to the depth-wise vanishing and exploding gradients studied in Lecture 10, except that the relevant depth here is the *number of time steps*.

16.6.2 Gradient Clipping and Gated Recurrence

One practical way to address large gradients is to clip the full gradient by its Euclidean norm. If $g = \nabla_\theta \mathcal{L}$ and $\tau > 0$, define

$$\widetilde{g} = \begin{cases} g, & \|g\|_2 \leq \tau, \\ \tau \dfrac{g}{\|g\|_2}, & \|g\|_2 > \tau. \end{cases}$$

The update $\theta \leftarrow \theta - \eta\widetilde{g}$ preserves the gradient direction while limiting its magnitude. Clipping addresses exploding gradients directly, but does not by itself cure vanishing gradients.

More elaborate recurrent cells, such as long short-term memory networks (LSTMs) and gated recurrent units (GRUs), use learned gates to control which information is retained, forgotten, or passed forward. Their purpose is to make long-range dependence easier to learn. These architectures extend the simple recurrence in this lecture; their exact gating equations are not needed for the current derivations.

16.7 Training and Computational Considerations

16.7.1 Forward and Backward Training Algorithm

For a labeled sequence and given initial state $h^{(0)}$, a complete training step is:

1. Initialize the hidden state, and run the recurrent computation for $t = 1, \ldots, T$, retaining $a^{(t)}$, $h^{(t)}$, $o^{(t)}$, and $y^{(t)}$.
2. Accumulate the negative log-likelihood $\mathcal{L} = \sum_{t=1}^{T} -\log y_{k_t}^{(t)}$.
3. Compute the output errors $e^{(t)} = y^{(t)} - q^{(t)}$.
4. Run the BPTT recurrence for $t = T, T-1, \ldots, 1$ to obtain $\delta^{(t)}$.
5. Sum the gradient contributions for each shared parameter, then optionally clip the gradient norm.
6. Update $\theta \leftarrow \theta - \eta\nabla_\theta \mathcal{L}$, or apply a chosen optimizer such as stochastic gradient descent or Adam.

For a mini-batch of sequences, add or average the parameter gradients over sequences, usually ignoring padded time steps through a binary loss mask.

16.7.2 Complexity and Parameter Sharing

A single-layer RNN with input dimension d, hidden dimension H, output dimension K, and length T performs, to leading order, matrix-vector products costing

$$O\big(T(dH + H^2 + HK)\big).$$

The number of trainable parameters is

$$P = dH + H^2 + HK + H + K.$$

Importantly, P does *not* increase with sequence length T because the same weights are reused at every time step. Exact BPTT generally stores intermediate hidden states and activations at a memory cost of order $O(TH)$ per sequence, apart from the parameters and optimizer state.

Proposition 16.5 (Shared-parameter gradient accumulation)**.** *Suppose a scalar parameter ϑ appears in T different nodes of an unrolled computational graph through copies $\vartheta_1, \dots, \vartheta_T$ that are all constrained to equal ϑ. Then*

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\vartheta} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \vartheta_t}\bigg|_{\vartheta_1 = \cdots = \vartheta_T = \vartheta}.$$

Proof. Consider the untied loss function $F(\vartheta_1, \dots, \vartheta_T)$ and define $\mathcal{L}(\vartheta) = F(\vartheta, \dots, \vartheta)$. By the multivariate chain rule,

$$\begin{aligned} \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\vartheta} &= \sum_{t=1}^{T} \frac{\partial F}{\partial \vartheta_t} \frac{\mathrm{d}\vartheta_t}{\mathrm{d}\vartheta} \\ &= \sum_{t=1}^{T} \frac{\partial F}{\partial \vartheta_t}, \end{aligned}$$

because each $\vartheta_t = \vartheta$ has derivative one. The same reasoning applies entrywise to shared matrices. □

16.7.3 Variable-Length Sequences and Truncation

Real sequences often have different lengths. For a mini-batch of B sequences padded to length $T_{\max}$, let $m_{n,t} \in \{0, 1\}$ indicate whether time step t is a valid target for example n. A token-normalized loss is

$$\mathcal{L}_{\text{batch}} = -\frac{\displaystyle\sum_{n=1}^{B} \sum_{t=1}^{T_{\max}} m_{n,t} \log y_{k_{n,t}}^{(n,t)}}{\displaystyle\sum_{n=1}^{B} \sum_{t=1}^{T_{\max}} m_{n,t}}.$$

This prevents padding positions from contributing fictitious target losses. For very long sequences, *truncated BPTT* differentiates through a limited number of recent time steps rather than through the full history. It lowers memory and computation for each update, but in general changes the gradient by discarding contributions that cross the truncation boundary.

16.7.4 A Minimal Training Implementation

The following PyTorch example performs a single training step for an input-driven RNN predicting a class at each time step. The inputs have shape (B, T, d), and the integer labels have shape (B, T).

```
import torch
from torch import nn

B, T, d, H, K = 8, 12, 5, 16, 4
x = torch.randn(B, T, d)
target = torch.randint(0, K, (B, T))

class SimpleRNN(nn.Module):
    def __init__(self, d, H, K):
        super().__init__()
        self.U = nn.Linear(d, H, bias=True)
        self.W = nn.Linear(H, H, bias=False)
        self.V = nn.Linear(H, K, bias=True)

    def forward(self, x):
        B, T, _ = x.shape
        h = x.new_zeros(B, self.W.out_features)
        outputs = []
        for t in range(T):
            h = torch.tanh(self.U(x[:, t]) + self.W(h))
            outputs.append(self.V(h))
        return torch.stack(outputs, dim=1)

net = SimpleRNN(d, H, K)
optimizer = torch.optim.Adam(net.parameters(), lr=1e-3)

optimizer.zero_grad()
logits = net(x)                          # (B, T, K)
loss = nn.functional.cross_entropy(
    logits.reshape(-1, K), target.reshape(-1)
)
loss.backward()                           # BPTT via autograd
torch.nn.utils.clip_grad_norm_(net.parameters(), 1.0)
```

```
optimizer.step()
```

Although U, W, and V appear as separate linear modules, the code shares their parameters across all time steps by calling the *same* modules in the loop. The loss function accepts logits directly and internally computes a numerically stable log-softmax. Consequently the code does not explicitly call softmax before the cross-entropy computation.

16.8 Further Examples and Exercises

Exercise 1.

Prove that, for a fixed initial state and a linear recurrent cell with no activation,

$$h^{(t)} = x^{(t)}U + h^{(t-1)}W + b,$$

the hidden state can be expressed explicitly in terms of the input history.

Solution. Repeated substitution gives

$$\begin{aligned} h^{(t)} &= \big(x^{(t-1)}U + h^{(t-2)}W + b\big)W + x^{(t)}U + b \\ &= x^{(t)}U + x^{(t-1)}UW + bW + b + h^{(t-2)}W^2. \end{aligned}$$

Continuing to time zero yields

$$h^{(t)} = h^{(0)}W^t + \sum_{s=1}^{t} x^{(s)}UW^{t-s} + b\sum_{j=0}^{t-1} W^j.$$

Each input $x^{(s)}$ is propagated through $t-s$ recurrent transitions. If $\|W\|_2 < 1$, its influence is geometrically suppressed as $t-s$ grows; this simple linear case motivates the more general Jacobian analysis. ■

Exercise 2.

Suppose a two-class RNN has logits $o = (\alpha, \beta)$ at a given time. Derive the cross-entropy loss and its derivative when the first class is correct.

Solution. The first-class probability is $p_1 = e^{\alpha}/(e^{\alpha} + e^{\beta})$, so

$$\begin{aligned} \ell &= -\log p_1 = -\alpha + \log(e^{\alpha} + e^{\beta}) \\ &= \log\big(1 + e^{\beta-\alpha}\big). \end{aligned}$$

Consequently,

$$\begin{aligned} \frac{\partial \ell}{\partial \alpha} &= p_1 - 1, \\ \frac{\partial \ell}{\partial \beta} &= p_2, \end{aligned}$$

which is $p - q$ for the one-hot target $q = (1, 0)$. ■

Exercise 3.

For a single-layer RNN with $d = 10$, $H = 32$, and $K = 5$, compute the number of trainable parameters, including both biases but treating $h^{(0)} = 0$ as fixed.

Solution. The input-to-hidden matrix contributes $10 \cdot 32 = 320$ entries, the recurrent matrix contributes $32^2 = 1024$, and the output matrix contributes $32 \cdot 5 = 160$. The biases contribute $32 + 5 = 37$. Thus

$$P = 320 + 1024 + 160 + 37 = 1541.$$

This number is independent of the sequence length. ■

Exercise 4.

In the two-step scalar example above, suppose we replace $w = 1/2$ by $w = 0$. Explain how this changes the contribution of the second loss term to the derivative at the first hidden state.

Solution. When $w = 0$, the second state is $h^{(2)} = ux^{(2)} + b$, which is independent of $h^{(1)}$. Hence

$$\frac{\partial \ell_2}{\partial h^{(1)}} = \frac{\partial \ell_2}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} = \frac{\partial \ell_2}{\partial h^{(2)}} \, w = 0.$$

The recurrent contribution disappears, and the first hidden state receives gradients only from the first output. ■

Exercise 5.

Consider a two-layer RNN with layer dimensions $H_0 = d$, $H_1 = p$, $H_2 = q$ and output dimension K. Compute its parameter count when all biases are included.

Solution. Layer 1 has $dp + p^2 + p$ parameters. Layer 2 has $pq + q^2 + q$ parameters. The output map has $qK + K$ parameters. Thus the total is

$$P = dp + p^2 + p + pq + q^2 + q + qK + K.$$

Again, time unfolding creates more operation nodes but no additional independent weight matrices. ■

Exercise 6.

Suppose $\phi = \tanh$ and $\|W\|_2 \le \rho < 1$. Show that any direct effect of perturbing $h^{(s)}$ on $h^{(t)}$ for $s < t$ is bounded by ρ^{t-s} times the perturbation size, to first order.

Solution. For tanh, $|\phi'(z)| \le 1$, so $\|D_r\|_2 \le 1$. The recurrent Jacobian product satisfies

$$\left\| \prod_{r=s+1}^{t} W D_r \right\|_2 \le \prod_{r=s+1}^{t} \|W\|_2 \|D_r\|_2 \le \rho^{t-s}.$$

For a sufficiently small perturbation $\Delta h^{(s)}$, linearization therefore gives

$$\|\Delta h^{(t)}\|_2 \le \rho^{t-s} \|\Delta h^{(s)}\|_2 + o(\|\Delta h^{(s)}\|_2).$$

Thus the local sensitivity of a far-future state to an early perturbation is geometrically small under the stated sufficient condition. ■

17 Lecture 18: Vanishing Gradients and Gated Recurrent Units

In Lecture 17, a recurrent neural network (RNN) maintained a hidden state by repeatedly applying the same nonlinear transformation to the latest input and the preceding state. In principle, the hidden state can summarize arbitrarily long input histories. In practice, however, as the sequence length grows, a simple RNN may lose information about early inputs and its gradients may become extremely small or large. The problem arises because information and derivatives must pass through many successive state transitions.

The present lecture begins with an exact scalar calculation that isolates this phenomenon. We then investigate the corresponding nonlinear and multidimensional gradient calculation, and introduce *gated recurrent units* (GRUs), which use learnable gates to control how much previous information is retained, how much of a new candidate state is accepted, and how much old state enters the candidate-state computation. Throughout, time is indexed by $t = 1, \dots, T$.

17.1 Loss of Long-Range Information in Simple Recurrent Networks

17.1.1 A Linear Scalar Recurrence

To separate the effect of recurrence from that of the activation function, begin with the scalar model

$$h_t = wh_{t-1} + z_t, \qquad t \geq 1,$$

where $w \in \mathbb{R}$ is a fixed recurrent weight, $z_t \in \mathbb{R}$ is the transformed input at time t, and h_0 is the initial hidden state. This is the linear version of the recurrent model $h_t = \phi(wh_{t-1} + z_t)$.

The first few steps are

$$\begin{aligned} h_1 &= wh_0 + z_1, \\ h_2 &= wh_1 + z_2 = w^2 h_0 + wz_1 + z_2, \\ h_3 &= wh_2 + z_3 = w^3 h_0 + w^2 z_1 + wz_2 + z_3. \end{aligned}$$

The pattern suggests that a past input z_k reaches h_t multiplied by the factor w^{t-k}.

Proposition 17.1 (Explicit solution of the scalar recurrence)**.** *For every integer $t \geq 1$, the solution is*

$$h_t = w^t h_0 + \sum_{k=1}^{t} w^{t-k} z_k.$$

Proof. The result is immediate for $t = 1$. Suppose it holds for time $t - 1$. Then

$$\begin{aligned} h_t &= wh_{t-1} + z_t \\ &= w\left(w^{t-1}h_0 + \sum_{k=1}^{t-1} w^{t-1-k}z_k\right) + z_t \\ &= w^t h_0 + \sum_{k=1}^{t-1} w^{t-k}z_k + w^{t-t}z_t \\ &= w^t h_0 + \sum_{k=1}^{t} w^{t-k}z_k. \end{aligned}$$

Thus the identity holds for all t by induction. □

Remark 17.1 (The contribution of an earlier input). Fix $k \le t$. If z_k is changed by Δz_k while all other inputs remain fixed, then

$$\Delta h_t = w^{t-k}\Delta z_k.$$

The effect depends on the elapsed time $t - k$, not merely on the input's magnitude. This is the precise sense in which past information can be forgotten or amplified by the recurrent dynamics.

Proposition 17.2 (Three recurrent regimes)**.** *For the scalar recurrence with a fixed time separation $t - k \to \infty$:*

1. *If $|w| < 1$, then $w^{t-k} \to 0$, so the influence of the fixed earlier input z_k vanishes geometrically.*
2. *If $|w| > 1$, then $|w^{t-k}| \to \infty$, so a perturbation of z_k is amplified geometrically.*
3. *If $|w| = 1$, the magnitude of the factor remains one. In particular, $w = 1$ preserves signs while $w = -1$ alternates signs.*

Proof. The result follows from the standard behavior of geometric sequences $|w|^m$ as $m \to \infty$. □

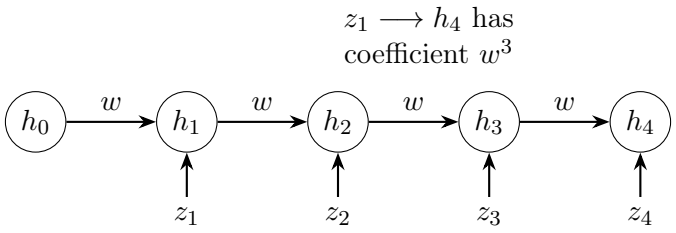


Figure 42: A scalar recurrence unrolled in time. The influence of an earlier input is multiplied by one copy of w per intervening state transition.

17.1.2 Constant Inputs and the Long-Run State

Suppose $z_t = z$ for every t. If $w \neq 1$, the geometric-series formula gives

$$\begin{aligned} h_t &= w^t h_0 + z \sum_{j=0}^{t-1} w^j \\ &= w^t h_0 + z \frac{1 - w^t}{1 - w} \\ &= \frac{z}{1 - w} + w^t \left(h_0 - \frac{z}{1 - w} \right). \end{aligned}$$

For $|w| < 1$, the transient term vanishes; therefore

$$\lim_{t \to \infty} h_t = \frac{z}{1 - w}.$$

The same fixed point is found directly by setting $h_t = h_{t-1} = h_*$:

$$h_* = w h_* + z \quad \Longleftrightarrow \quad (1 - w) h_* = z \quad \Longleftrightarrow \quad h_* = \frac{z}{1 - w}.$$

When $w = 1$, instead, $h_t = h_0 + tz$, which typically grows without bound unless $z = 0$.

Remark 17.2. Stability of the state and preservation of a long-range dependence are different concepts. When $|w| < 1$, the state may converge to a well-defined fixed point under constant input, yet it necessarily loses the influence of any one fixed early observation. Conversely, $|w| > 1$ may preserve an appreciable derivative to an old input, but the forward state and backward gradient can become unstable.

17.1.3 Backward Sensitivity and Gradient Magnitude

Let $L = \ell(h_T)$ be a differentiable scalar loss depending on the terminal state. By the chain rule,

$$\begin{aligned} \frac{\partial L}{\partial z_k} &= \frac{\partial L}{\partial h_T} \frac{\partial h_T}{\partial z_k} \\ &= \ell'(h_T) w^{T-k}. \end{aligned}$$

Similarly, for $s < T$,

$$\begin{aligned} \frac{\partial h_T}{\partial h_s} &= \prod_{t=s+1}^{T} \frac{\partial h_t}{\partial h_{t-1}} \\ &= \prod_{t=s+1}^{T} w = w^{T-s}, \\ \frac{\partial L}{\partial h_s} &= \ell'(h_T) w^{T-s}. \end{aligned}$$

Thus the same geometric factor controls both *forward memory* of an earlier input and *backward sensitivity* to an earlier state.

Example 17.1 (Long-range signal decay)**.** With $w = 0.8$, an input occurring 30 steps before the final time has multiplier

$$|w|^{30} = 0.8^{30} \approx 1.24 \times 10^{-3}.$$

For $w = 1.1$, the corresponding multiplier is

$$|w|^{30} = 1.1^{30} \approx 17.45.$$

The first recurrence almost forgets the early input; the second amplifies its effect. Neither case by itself proves anything about optimization of a nonlinear RNN, but both illustrate why the product of many recurrent factors is central.

17.2 Vanishing and Exploding Gradients in Nonlinear RNNs

17.2.1 The Recurrent Jacobian

Return to the row-vector convention used in Lecture 17. Let

$$\begin{aligned} x_t &\in \mathbb{R}^{1\times d}, & h_t &\in \mathbb{R}^{1\times H},\\ U &\in \mathbb{R}^{d\times H}, & W &\in \mathbb{R}^{H\times H}, \end{aligned}$$

and consider the recurrence

$$a_t = x_t U + h_{t-1} W + b, \qquad h_t = \phi(a_t),$$

where ϕ acts componentwise. Let

$$D_t = \operatorname{diag}\big(\phi'(a_{t,1}), \dots, \phi'(a_{t,H})\big).$$

For an infinitesimal change in the preceding row-vector state, keeping x_t fixed,

$$\begin{aligned} \mathrm{d}a_t &= \mathrm{d}h_{t-1} W,\\ \mathrm{d}h_t &= \mathrm{d}a_t D_t = \mathrm{d}h_{t-1} W D_t. \end{aligned}$$

Thus the time-t recurrent Jacobian, in this row-differential convention, is $J_t = W D_t$.

Proposition 17.3 (Sensitivity across many recurrent steps)**.** *For any $0 \le s < t \le T$, the sensitivity of h_t to h_s along the realized sequence is represented by*

$$J_{s\to t} = J_{s+1} J_{s+2} \cdots J_t = \prod_{r=s+1}^{t} (W D_r),$$

with factors ordered from earlier to later times. For any differential $\mathrm{d}h_s$,

$$\mathrm{d}h_t = \mathrm{d}h_s J_{s\to t}.$$

Proof. Apply $\mathrm{d}h_r = \mathrm{d}h_{r-1} J_r$ successively for $r = s+1, \dots, t$:

$$\begin{aligned} \mathrm{d}h_{s+1} &= \mathrm{d}h_s J_{s+1}, \\ \mathrm{d}h_{s+2} &= \mathrm{d}h_s J_{s+1} J_{s+2}, \\ &\vdots \\ \mathrm{d}h_t &= \mathrm{d}h_s J_{s+1} \cdots J_t. \end{aligned}$$

□

Theorem 17.1 (A sufficient vanishing-gradient condition)**.** *Suppose $|\phi'(u)| \le M$ for every u encountered during the recurrence. Then, in the spectral matrix norm,*

$$\|J_{s\to t}\|_2 \le \big(M\|W\|_2\big)^{t-s}.$$

Consequently, if $M\|W\|_2 < 1$, the sensitivity of h_t to h_s vanishes at least geometrically as $t - s \to \infty$.

Proof. Since D_r is diagonal,

$$\|D_r\|_2 = \max_i |\phi'(a_{r,i})| \le M.$$

Submultiplicativity of the induced operator norm gives

$$\begin{aligned} \|J_{s\to t}\|_2 &\le \prod_{r=s+1}^{t} \|W D_r\|_2 \\ &\le \prod_{r=s+1}^{t} \|W\|_2 \|D_r\|_2 \\ &\le \prod_{r=s+1}^{t} M\|W\|_2 \\ &= \big(M\|W\|_2\big)^{t-s}. \end{aligned}$$

The stated conclusion follows whenever the base is less than one. □

For the hyperbolic tangent activation,

$$\phi(u) = \tanh(u), \qquad \phi'(u) = 1 - \tanh^2(u) = \operatorname{sech}^2(u),$$

so $0 < \phi'(u) \le 1$, and $\phi'(u) \to 0$ when $|u| \to \infty$. Activation saturation can therefore reduce the gradient even when $\|W\|_2$ is not small.

Remark 17.3 (Necessary versus sufficient conditions). The condition $M\|W\|_2 < 1$ is sufficient, but not necessary, for decay. Conversely, $M\|W\|_2 > 1$ does not imply that gradients explode: an upper bound larger than one proves no amplification, and the actual product depends on matrix directions, eigenstructure, and activation derivatives. In particular, a highly saturated tanh can suppress a gradient even when W has large norm.

17.2.2 Gradient Propagation in Backpropagation Through Time

Write $\bar{h}_t = \nabla_{h_t} L$ for the row-vector gradient of a scalar loss. If there is no direct loss at an intermediate time, then

$$\begin{aligned} \bar{h}_{t-1} &= \bar{h}_t J_t^\top, \\ \bar{h}_s &= \bar{h}_t J_t^\top J_{t-1}^\top \cdots J_{s+1}^\top. \end{aligned}$$

The transpose order follows because $\mathrm{d}h_t = \mathrm{d}h_{t-1} J_t$ and

$$\mathrm{d}L = \bar{h}_t (\mathrm{d}h_t)^\top = \bar{h}_t J_t^\top (\mathrm{d}h_{t-1})^\top.$$

When the loss depends on outputs from several time steps, the backward recursion adds their direct contributions to $\bar{h}_t$. This is the same reverse accumulation through the unrolled graph derived in Lecture 17.

The vanishing-gradient phenomenon makes the training of long-range dependencies difficult: a target observed at time T may provide almost no gradient signal to operations occurring near time 1. Exploding gradients can instead create highly unstable updates. Gradient clipping can control the *size* of an exploding gradient, but does not create a reliable memory mechanism on its own.

17.3 Gated Recurrent Units: The Main Idea

17.3.1 The Candidate State and Update Gate

A GRU supplements the conventional candidate hidden state with a *gate* whose entries lie between zero and one. The gate is learned from the current input and preceding state.

Definition 17.1 (Update-gated recurrent cell)**.** Let $x_t \in \mathbb{R}^{1\times d}$ and $h_{t-1} \in \mathbb{R}^{1\times H}$. Define a candidate hidden state $\widetilde{h}_t$ and an update gate g_t by

$$\begin{aligned} \widetilde{h}_t &= \tanh\big(x_t U_h + h_{t-1} W_h + b_h\big), \\ g_t &= \sigma\big(x_t U_g + h_{t-1} W_g + b_g\big), \end{aligned}$$

where $U_h, U_g \in \mathbb{R}^{d\times H}$, $W_h, W_g \in \mathbb{R}^{H\times H}$, and $b_h, b_g \in \mathbb{R}^{1\times H}$. The state update is

$$h_t = g_t \odot \widetilde{h}_t + (\mathbf{1} - g_t) \odot h_{t-1},$$

where $\odot$ denotes elementwise (Hadamard) multiplication.

The logistic sigmoid is

$$\sigma(u) = \frac{1}{1+e^{-u}}, \qquad \sigma'(u) = \sigma(u)\big(1-\sigma(u)\big).$$

Consequently, for finite gate pre-activations, $0 < g_{t,j} < 1$ for every hidden coordinate j. The update is therefore a componentwise interpolation between an old state and a new candidate. An equivalent form is

$$\begin{aligned} h_t &= g_t \odot \widetilde{h}_t + (\mathbf{1} - g_t) \odot h_{t-1} \\ &= h_{t-1} + g_t \odot (\widetilde{h}_t - h_{t-1}). \end{aligned}$$

Remark 17.4 (Interpretation of the update gate). For hidden coordinate j:

1. If $g_{t,j} \approx 0$, then $h_{t,j} \approx h_{t-1,j}$; the old state is retained.
2. If $g_{t,j} \approx 1$, then $h_{t,j} \approx \widetilde{h}_{t,j}$; the new candidate mostly replaces the old state.
3. For intermediate gate values, the state mixes old and candidate information.

A separate gate is learned for each coordinate and each time step. Unlike a single fixed recurrent coefficient w, the balance changes with the incoming data and the previous state.

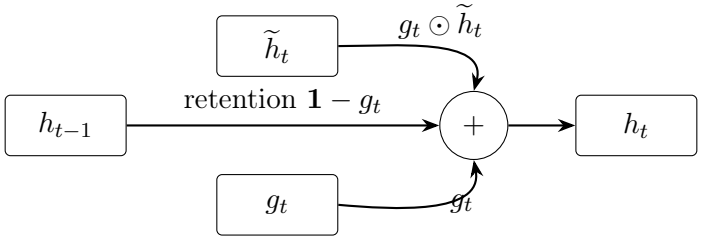


Figure 43: Conceptual GRU update rule. The update gate determines how much candidate information is introduced and how much previous hidden state is retained.

17.3.2 A Controlled Memory Path

Suppose for the moment that $g_t = g \in (0, 1)$ is a *fixed scalar* independent of the hidden state, and the candidate values $\widetilde{h}_t$ are supplied externally. Then

$$h_t = (1 - g)h_{t-1} + g\widetilde{h}_t.$$

This is a linear recurrence with coefficient $1 - g$. Applying the earlier formula yields

$$h_t = (1 - g)^t h_0 + \sum_{k=1}^{t} g(1 - g)^{t-k}\widetilde{h}_k.$$

If g is small, then $1 - g$ is close to one, so earlier information can persist for many time steps. For instance, a fixed $g = 0.01$ gives retention $(0.99)^{100} \approx 0.366$ across one hundred steps, substantially larger than the factor $(0.8)^{100}$ in a simple scalar recurrence with $w = 0.8$.

Proposition 17.4 (Effective memory timescale under fixed gating)**.** *In the scalar fixed-gate recurrence, the contribution of h_s to h_t has coefficient $(1-g)^{t-s}$. For $0 < g < 1$, writing*

$$(1-g)^m = e^{-m/\tau_{\text{eff}}},$$

its effective decay timescale is

$$\tau_{\text{eff}} = -\frac{1}{\log(1-g)}.$$

As $g \downarrow 0$, $\tau_{\text{eff}} \sim 1/g$.

Proof. Taking logarithms gives $m\log(1-g) = -m/\tau_{\text{eff}}$. Solving for the timescale yields the expression. Finally, the Taylor expansion $\log(1-g) = -g + O(g^2)$ proves $\tau_{\text{eff}} \sim 1/g$. □

Remark 17.5 (An important qualification). The previous calculation assumes that the gate and candidate do not themselves depend on the earlier state. In an actual GRU they do: derivatives through the candidate and through the gates must also be included. A direct retention path makes gradient preservation *possible*, but does not guarantee that every gradient remains large or stable. We derive the exact Jacobian below.

17.4 The Full Gated Recurrent Unit

17.4.1 Reset Gate and Candidate Formation

The update gate controls how strongly the new candidate replaces the old state. A second gate, the *reset gate*, controls how much of the old state is used *to construct that candidate*.

Definition 17.2 (Full GRU cell)**.** For every time t, let

$$\begin{aligned}
r_t &= \sigma\big(x_t U_r + h_{t-1} W_r + b_r\big),\\
g_t &= \sigma\big(x_t U_g + h_{t-1} W_g + b_g\big),\\
\widetilde{h}_t &= \tanh\big(x_t U_h + (r_t \odot h_{t-1}) W_h + b_h\big),\\
h_t &= g_t \odot \widetilde{h}_t + (\mathbf{1} - g_t) \odot h_{t-1}.
\end{aligned}$$

All $r_t, g_t, \widetilde{h}_t, h_t$ belong to $\mathbb{R}^{1\times H}$. Each $U_\bullet \in \mathbb{R}^{d\times H}$, $W_\bullet \in \mathbb{R}^{H\times H}$, and $b_\bullet \in \mathbb{R}^{1\times H}$.

The quantities r_t and g_t have different jobs:

- The *reset gate* r_t filters h_{t-1} before it enters the new candidate. If $r_t \approx \mathbf{0}$, the candidate depends predominantly on the current input; if $r_t \approx \mathbf{1}$, the old hidden state participates fully in candidate formation.
- The *update gate* g_t controls how much of that candidate is actually accepted into h_t. If $g_t \approx \mathbf{0}$, the old state is carried forward almost unchanged; if $g_t \approx \mathbf{1}$, the candidate is adopted.

Remark 17.6 (Alternative notation). Some references use an update gate z_t with the opposite interpolation convention,

$$h_t = z_t \odot h_{t-1} + (\mathbf{1} - z_t) \odot \widetilde{h}_t.$$

The conventions are equivalent under $z_t = \mathbf{1} - g_t$. This lecture uses g_t for the *amount of new candidate accepted*, matching the handwritten notes. GRU implementations can also place the reset multiplication on a slightly different side of the recurrent matrix; the equations must be checked before copying weights between conventions.

17.4.2 Unrolling a GRU Across a Sequence

The same six weight matrices and three bias vectors are shared at every time. A hidden state can therefore remain approximately constant during intervals in which g_t is small, and can be rapidly updated when the learned gate responds to a relevant input.

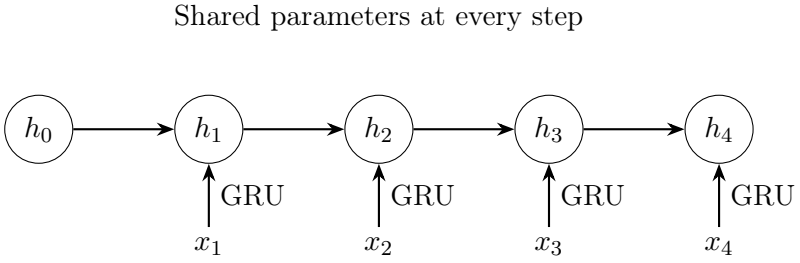


Figure 44: A GRU unrolled in time. The gating computations at each step depend on the current input and the preceding hidden state; all steps share the same parameters.

17.4.3 Boundedness of the GRU Hidden State

An advantage of using a tanh candidate together with a convex interpolation is that the state cannot escape the componentwise interval $[-1, 1]$ if it starts there.

Proposition 17.5 (Componentwise invariant interval)**.** *Suppose $h_{0,j} \in [-1, 1]$ for every j and the candidate is formed with componentwise* tanh*. For any input sequence and any finite GRU parameters, every subsequent coordinate satisfies*

$$h_{t,j} \in [-1, 1], \qquad t \geq 0.$$

Proof. The candidate obeys $-1 < \widetilde{h}_{t,j} < 1$, while sigmoid gates obey $0 < g_{t,j} < 1$. The update is

$$h_{t,j} = g_{t,j}\widetilde{h}_{t,j} + (1 - g_{t,j})h_{t-1,j}.$$

It is a convex combination of two numbers in $[-1, 1]$. Therefore it belongs to the same interval. Induction over t proves the claim. □

Remark 17.7. A bounded hidden state does *not* imply bounded parameter gradients uniformly in the sequence length. Derivatives can be sensitive to gate weights and repeated Jacobian products even when the forward state remains within a compact interval.

17.5 Differentiating the GRU: Exact State Jacobian

The key reason to study gated recurrence mathematically is to understand its backward sensitivity. A GRU includes a direct route from h_{t-1} to h_t, but it also includes indirect routes through the update gate, reset gate, and candidate. The chain rule must account for all of them.

17.5.1 Matrix-Differential Derivation

Fix a time t and suppress the time subscript temporarily. Write

$$\begin{aligned} p &:= h_{t-1}, & x &:= x_t, \\ a_r &= xU_r + pW_r + b_r, & r &= \sigma(a_r), \\ a_g &= xU_g + pW_g + b_g, & g &= \sigma(a_g), \\ a_h &= xU_h + (p \odot r)W_h + b_h, & c &= \tanh(a_h), \\ h &= g \odot c + (\mathbf{1} - g) \odot p. \end{aligned}$$

We use c as temporary shorthand for the candidate $\widetilde{h}_t$; it should not be confused with the output bias used in earlier lectures. Define diagonal matrices

$$\begin{aligned} D_p &:= \operatorname{diag}(p), & D_r &:= \operatorname{diag}(r), \\ D_g &:= \operatorname{diag}(g), & D_{1-g} &:= \operatorname{diag}(\mathbf{1} - g), \\ D_{c-p} &:= \operatorname{diag}(c - p), \\ D_{\sigma,r} &:= \operatorname{diag}(r \odot (\mathbf{1} - r)), \\ D_{\sigma,g} &:= \operatorname{diag}(g \odot (\mathbf{1} - g)), \\ D_{\tanh,c} &:= \operatorname{diag}(\mathbf{1} - c \odot c). \end{aligned}$$

The notation $\mathbf{1} - c \odot c$ means the vector with entries $1 - c_j^2$.

First differentiate the two gates with respect to p, keeping x fixed:

$$\begin{aligned} \mathrm{d}a_r &= \mathrm{d}p\, W_r, \\ \mathrm{d}r &= \mathrm{d}p\, W_r D_{\sigma,r}, \\ \mathrm{d}a_g &= \mathrm{d}p\, W_g, \\ \mathrm{d}g &= \mathrm{d}p\, W_g D_{\sigma,g}. \end{aligned}$$

Next differentiate the gated old state. The Hadamard product rule gives

$$\begin{aligned} \mathrm{d}(p \odot r) &= \mathrm{d}p\, D_r + \mathrm{d}r\, D_p \\ &= \mathrm{d}p\big(D_r + W_r D_{\sigma,r} D_p\big). \end{aligned}$$

Hence the candidate-state differential is

$$\begin{aligned}
\mathrm{d}a_h &= \mathrm{d}(p \odot r) W_h \\
&= \mathrm{d}p \big(D_r + W_r D_{\sigma,r} D_p\big) W_h, \\
\mathrm{d}c &= \mathrm{d}a_h D_{\tanh,c} \\
&= \mathrm{d}p \big(D_r + W_r D_{\sigma,r} D_p\big) W_h D_{\tanh,c}.
\end{aligned}$$

Finally, differentiating the state update yields

$$\begin{aligned}
\mathrm{d}h &= \mathrm{d}(g \odot c) + \mathrm{d}((\mathbf{1} - g) \odot p) \\
&= \mathrm{d}g\, D_c + \mathrm{d}c\, D_g - \mathrm{d}g\, D_p + \mathrm{d}p\, D_{1-g} \\
&= \mathrm{d}p\, D_{1-g} + \mathrm{d}g\, D_{c-p} + \mathrm{d}c\, D_g,
\end{aligned}$$

where $D_c = \mathrm{diag}(c)$. Substituting the gate and candidate differentials gives

$$\begin{aligned}
\mathrm{d}h = \mathrm{d}p \Big[& D_{1-g} + W_g D_{\sigma,g} D_{c-p} \\
& + \big(D_r + W_r D_{\sigma,r} D_p\big) W_h D_{\tanh,c} D_g \Big].
\end{aligned}$$

Theorem 17.2 (Exact GRU recurrent Jacobian)**.** *Under the row-vector convention* $\mathrm{d}h_t = \mathrm{d}h_{t-1} J_t$ *with current input held fixed, the GRU recurrent Jacobian is*

$$\begin{aligned}
J_t = & D_{\mathbf{1}-g_t} + W_g D_{\sigma,g_t} D_{\tilde{h}_t - h_{t-1}} \\
& + \big(D_{r_t} + W_r D_{\sigma,r_t} D_{h_{t-1}}\big) W_h D_{\tanh,\tilde{h}_t} D_{g_t}.
\end{aligned}$$

All diagonal matrices are evaluated at time t. The three terms arise respectively from direct old-state retention, dependence of the update gate on the old state, and dependence of the candidate on the old state, including its reset gate.

Proof. The result is the coefficient of $\mathrm{d}p$ in the preceding matrix-differential calculation, after restoring the time indices. Each differential follows from the scalar chain rule applied coordinatewise and the product rule for the Hadamard multiplication. □

Remark 17.8 (When the direct path is approximately an identity). If g_t is close to $\mathbf{0}$, the first term $D_{\mathbf{1}-g_t}$ is close to the identity. When the other derivative contributions are small as well, J_t is close to I_H, so gradient information can travel through that step with little change. This explains the motivation for gating. The exact Jacobian nevertheless contains the two additional terms above, so neither $J_t = I_H$ nor $\|J_t\|_2 \le 1$ holds automatically.

17.5.2 Backward Differentiation of One GRU Cell

For implementation, it is often clearer to backpropagate through the individual operations than to construct J_t explicitly. Given a scalar loss L, let a bar denote the row-vector gradient of L with respect to a node, for example

$$\bar{h} := \nabla_h L, \qquad \bar{c} := \nabla_c L, \qquad \bar{a}_h := \nabla_{a_h} L.$$

The GRU computational graph has the sequence of operations

$$(p, x) \longmapsto (r, g) \longmapsto c \longmapsto h.$$

Proposition 17.6 (Backward rules for a single GRU cell)**.** *With the notation introduced above and an incoming adjoint $\bar{h}$, reverse accumulation gives*

$$\begin{aligned}
\bar{c} &= \bar{h} \odot g, \\
\bar{g} &= \bar{h} \odot (c - p), \\
\bar{p}^{\,\mathrm{direct}} &= \bar{h} \odot (\mathbf{1} - g), \\
\bar{a}_h &= \bar{c} \odot (\mathbf{1} - c \odot c), \\
\bar{q} &= \bar{a}_h W_h^{\top}, \qquad q := p \odot r, \\
\bar{p}^{\,\mathrm{cand}} &= \bar{q} \odot r, \\
\bar{r} &= \bar{q} \odot p, \\
\bar{a}_r &= \bar{r} \odot r \odot (\mathbf{1} - r), \\
\bar{a}_g &= \bar{g} \odot g \odot (\mathbf{1} - g), \\
\bar{p} &= \bar{p}^{\,\mathrm{direct}} + \bar{p}^{\,\mathrm{cand}} + \bar{a}_r W_r^{\top} + \bar{a}_g W_g^{\top}.
\end{aligned}$$

The parameter gradients arising at this time step are

$$\begin{aligned}
&\nabla_{U_h} L = x^{\top} \bar{a}_h, && \nabla_{W_h} L = (p \odot r)^{\top} \bar{a}_h, && \nabla_{b_h} L = \bar{a}_h, \\
&\nabla_{U_r} L = x^{\top} \bar{a}_r, && \nabla_{W_r} L = p^{\top} \bar{a}_r, && \nabla_{b_r} L = \bar{a}_r, \\
&\nabla_{U_g} L = x^{\top} \bar{a}_g, && \nabla_{W_g} L = p^{\top} \bar{a}_g, && \nabla_{b_g} L = \bar{a}_g.
\end{aligned}$$

Proof. Differentiate the update $h = g \odot c + (\mathbf{1} - g) \odot p$ and pair its differential with $\bar{h}$:

$$\begin{aligned}
\mathrm{d}L &= \bar{h} (\mathrm{d}h)^{\top} \\
&= \bar{h} \big(\mathrm{d}c\, D_g + \mathrm{d}g\, D_{c-p} + \mathrm{d}p\, D_{\mathbf{1}-g} \big)^{\top}.
\end{aligned}$$

Since diagonal matrices equal their transposes, the coefficients of $\mathrm{d}c$, $\mathrm{d}g$, and the direct $\mathrm{d}p$ are precisely $\bar{h} \odot g$, $\bar{h} \odot (c - p)$, and $\bar{h} \odot (\mathbf{1} - g)$.

The candidate $c = \tanh(a_h)$ has coordinatewise derivative $1 - c_j^2$, giving

$$\bar{a}_h = \bar{c} \odot (\mathbf{1} - c \odot c).$$

For $a_h = x U_h + q W_h + b_h$ with $q = p \odot r$, the affine-layer backward rule gives $\bar{q} = \bar{a}_h W_h^{\top}$ and the stated weight and bias gradients. The product $q = p \odot r$ sends adjoints $\bar{q} \odot r$ toward p and $\bar{q} \odot p$ toward r. Finally, the sigmoid derivatives give $\bar{a}_r$ and $\bar{a}_g$; the two recurrent affine maps send additional adjoints $\bar{a}_r W_r^{\top}$ and $\bar{a}_g W_g^{\top}$ toward p. Adding the contributions from every path yields $\bar{p}$. □

Remark 17.9 (Shapes of the parameter gradients). Every $\bar{a}_\bullet$ is a row vector in $\mathbb{R}^{1\times H}$. Therefore

$$\begin{aligned} x^\top \bar{a}_\bullet &\in \mathbb{R}^{d\times H}, \\ p^\top \bar{a}_\bullet &\in \mathbb{R}^{H\times H}, \\ (p \odot r)^\top \bar{a}_h &\in \mathbb{R}^{H\times H}, \end{aligned}$$

matching the corresponding input and recurrent weight matrices. Checking these dimensions provides a useful safeguard against transpose mistakes.

17.5.3 Backpropagation Through Time for a GRU

The preceding proposition handles a single time step. A loss over a sequence requires the adjoints of later steps to be propagated backward to earlier states. Let

$$L(\theta) = \sum_{t=1}^{T} \ell_t(h_t),$$

where θ denotes the parameters shared across all time steps. The backward pass begins at time T, processes each GRU cell in reverse order, adds the direct gradient from ℓ_t, and accumulates parameter gradients.

Theorem 17.3 (GRU backpropagation through time)**.** *Let J_t denote the GRU recurrent Jacobian derived above and define $d_t = \nabla_{h_t}\ell_t(h_t)$. Set $\bar{h}_{T+1} = 0$. The total hidden-state gradients obey*

$$\bar{h}_t = d_t + \bar{h}_{t+1} J_{t+1}^\top, \qquad t = T, T-1, \ldots, 1.$$

Equivalently, $\bar{h}_t$ is obtained by summing the direct loss gradient at time t and all future gradient paths through the recurrent transitions. For every shared parameter θ_j, its full gradient is the sum of its local time-step contributions:

$$\nabla_{\theta_j} L = \sum_{t=1}^{T} \nabla_{\theta_j} L\big|_{\text{cell } t},$$

where the cell-t contribution is computed using the total adjoint $\bar{h}_t$.

Proof. For $t = T$, the terminal hidden state influences no later hidden states, so $\bar{h}_T = d_T$. At any earlier step, h_t affects the total objective through its direct loss ℓ_t and through the transition $h_{t+1} = F_\theta(h_t, x_{t+1})$. The multivariable chain rule gives

$$\begin{aligned} \nabla_{h_t} L &= \nabla_{h_t}\ell_t + \nabla_{h_{t+1}} L \, J_{t+1}^\top \\ &= d_t + \bar{h}_{t+1} J_{t+1}^\top. \end{aligned}$$

Induction backward through time proves the recurrence. Each occurrence of θ_j in the unrolled computational graph contributes a partial derivative to $\mathrm{d}L$; since the copies are numerically identical shared parameters, these local contributions add to give the gradient of L with respect to the one parameter θ_j. □

17.6 Examples and Interpretation

17.6.1 A One-Dimensional Gated Update

Consider a scalar state $h_{t-1} = 0.8$ and candidate $\widetilde{h}_t = -0.2$. If the update gate is $g_t = 0.25$, then

$$\begin{aligned} h_t &= g_t \widetilde{h}_t + (1 - g_t) h_{t-1} \\ &= 0.25(-0.2) + 0.75(0.8) \\ &= -0.05 + 0.60 = 0.55. \end{aligned}$$

The state moves toward the candidate but retains most of its previous value. By comparison, when $g_t = 0.9$,

$$h_t = 0.9(-0.2) + 0.1(0.8) = -0.10,$$

so the candidate almost completely replaces the preceding value.

If $r_t = 0.1$, the previous state enters candidate construction through

$$r_t h_{t-1} = 0.1(0.8) = 0.08;$$

if $r_t = 0.9$, the contribution is 0.72. The reset gate therefore changes the *information available to compute the candidate*, while the update gate changes the *amount of that candidate accepted afterward*.

17.6.2 A Gate-Induced Identity Path

Rewrite the update as

$$h_t = h_{t-1} + g_t \odot \left(\widetilde{h}_t - h_{t-1} \right).$$

This resembles the residual update studied for feedforward networks: the previous state passes directly to the next step, while an increment corrects it. But there is a difference: the increment is controlled by a data-dependent gate, and the candidate and gate themselves depend on the previous state.

For the scalar update, differentiating without freezing g_t or $\widetilde{h}_t$ gives

$$\frac{\partial h_t}{\partial h_{t-1}} = 1 - g_t + (\widetilde{h}_t - h_{t-1}) \frac{\partial g_t}{\partial h_{t-1}} + g_t \frac{\partial \widetilde{h}_t}{\partial h_{t-1}}.$$

This scalar formula is the one-coordinate counterpart of the exact matrix Jacobian. The direct term $1 - g_t$ provides a retention pathway, but the other two terms may add or cancel sensitivity.

Proposition 17.7 (Perfect retention in the ideal-gate limit)**.** *If a GRU step has $g_t = \mathbf{0}$ exactly, then $h_t = h_{t-1}$ regardless of the candidate and reset gate. If, in addition, the update gate is locally constant with respect to h_{t-1} at that step, the recurrent Jacobian is $J_t = I_H$.*

Proof. The state identity follows by substitution into the update rule. In the Jacobian formula, $D_{\mathbf{1}-g_t} = I_H$. The candidate term contains $D_{g_t} = 0$, and the gate-derivative term vanishes if the update gate is locally constant. Thus $J_t = I_H$. □

Remark 17.10. A logistic sigmoid with finite input never gives a gate equal to zero exactly; the proposition describes an idealized limiting case. In practice, gates close to zero can approximate identity-like state transitions. Gating improves the mechanism for maintaining long-range information, but finite-precision optimization and saturation of sigmoid gates introduce additional practical difficulties.

17.6.3 Resetting Irrelevant Context

Suppose a sequence contains multiple unrelated segments, such as sentences separated by a strong delimiter. When constructing the first new candidate after a delimiter, it may be useful to suppress the old context. If $r_t \approx \mathbf{0}$, then

$$\begin{aligned}\widetilde{h}_t &= \tanh\big(x_t U_h + (r_t \odot h_{t-1}) W_h + b_h\big) \\ &\approx \tanh(x_t U_h + b_h).\end{aligned}$$

This candidate is largely independent of the preceding hidden state. Choosing g_t relatively large then replaces the old state with information associated with the new segment. A GRU can therefore *selectively forget* old context and *selectively retain* relevant context, depending on the learned gates.

17.7 Practical Training and Connections

17.7.1 Parameters and Computational Cost

A simple tanh RNN with input dimension d and hidden dimension H has

$$dH + H^2 + H$$

parameters in its recurrent cell. The GRU has three sets of input weights, recurrent weights, and biases, giving

$$3(dH + H^2 + H)$$

parameters in the convention above. The extra parameters are used to compute the update and reset gates and the candidate state. Additional output layers introduce further parameters unrelated to the recurrent cell.

Remark 17.11. The increased parameter count is the cost of a more flexible state transition. Gated cells are often advantageous for long sequences, but no architecture is guaranteed to outperform a simple RNN for every dataset or task. The useful choice depends on sequence length, data availability, optimization, and the kinds of temporal dependence present.

17.7.2 Relation to LSTMs and Residual Networks

A long short-term memory network (LSTM) also uses gates to regulate state information, but typically maintains a separate cell state as well as a hidden output state. A GRU uses a single hidden state and blends old and candidate states directly. Both architectures create controlled paths for information to persist over many time steps, helping address the simple RNN's difficulty with long-range dependencies.

The update $h_t = h_{t-1} + g_t \odot (\widetilde{h}_t - h_{t-1})$ also resembles a residual network step $y = x + F(x)$. In both cases an additive direct path can improve gradient flow. Unlike a generic residual block, a GRU continuously adjusts the increment using a learned gate tied to the sequence dynamics.

17.7.3 A Minimal PyTorch GRU Cell

The following implementation uses exactly the equations of this lecture. It is intentionally written in elementary operations rather than calling a library GRU directly so that each gate and the Hadamard products can be seen explicitly.

```
import torch
from torch import nn

class LectureGRUCell(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.reset = nn.Linear(input_dim + hidden_dim, hidden_dim)
        self.update = nn.Linear(input_dim + hidden_dim, hidden_dim)
        self.candidate_input = nn.Linear(input_dim, hidden_dim)
        self.candidate_hidden = nn.Linear(hidden_dim, hidden_dim,
                                          bias=False)

    def forward(self, x, previous):
        combined = torch.cat([x, previous], dim=-1)
        r = torch.sigmoid(self.reset(combined))
        g = torch.sigmoid(self.update(combined))
        candidate = torch.tanh(
            self.candidate_input(x)
            + self.candidate_hidden(r * previous)
        )
        return g * candidate + (1 - g) * previous

# x_sequence: [batch, time, input_dim]
# h0:         [batch, hidden_dim]
cell = LectureGRUCell(input_dim=5, hidden_dim=8)
h = torch.zeros(4, 8)
x_sequence = torch.randn(4, 20, 5)
```

```
outputs = []
for t in range(x_sequence.shape[1]):
    h = cell(x_sequence[:, t, :], h)
    outputs.append(h)
all_states = torch.stack(outputs, dim=1)
loss = all_states.square().mean()
loss.backward()  # Reverse accumulation through all time steps
```

Remark 17.12 (Implementation conventions). The call to `nn.Linear` internally uses row-vector batches and stores its weight array transposed relative to the mathematical expression xU. This is an implementation detail; the resulting affine maps have the correct dimensions. Built-in GRU modules sometimes adopt a different reset-placement convention, and they usually define an update gate with the complementary interpretation.

17.7.4 Gradient Clipping and Truncated BPTT

Because the exact GRU Jacobian can still have a norm larger than one, exploding gradients remain possible. A practical training step may clip the total gradient $G = \nabla_\theta L$ at a prescribed threshold $\tau > 0$:

$$\widetilde{G} = \begin{cases} G, & \|G\|_2 \le \tau, \\ \tau \dfrac{G}{\|G\|_2}, & \|G\|_2 > \tau. \end{cases}$$

The update then uses $\theta \leftarrow \theta - \eta \widetilde{G}$. Clipping limits the step's gradient magnitude without changing its direction when $G \neq 0$.

Another technique, *truncated backpropagation through time*, differentiates through a window of only K time steps instead of the entire sequence. This reduces memory and computation, but discards some learning signals from dependencies spanning more than K steps. Gating and truncation address different concerns: gating changes the dynamics of information preservation, while truncation changes how far the training algorithm propagates derivatives.

17.8 Further Examples and Exercises

Exercise 1.

Consider $h_t = w h_{t-1} + z_t$ with $h_0 = 2$, $w = \frac{1}{2}$, and $z_t = 3$ for all $t \ge 1$. Derive h_t, compute its limit, and calculate $\partial h_{10} / \partial z_2$.

Solution. From the closed-form recurrence,

$$\begin{aligned} h_t &= \left(\frac{1}{2}\right)^t 2 + 3 \sum_{k=1}^{t} \left(\frac{1}{2}\right)^{t-k} \\ &= 2^{1-t} + 3 \frac{1 - 2^{-t}}{1 - 1/2} \end{aligned}$$

$$= 6 - 4 \cdot 2^{-t}.$$

Therefore $\lim_{t\to\infty} h_t = 6$. Since z_2 enters the time-10 state with factor w^{10-2},

$$\frac{\partial h_{10}}{\partial z_2} = \left(\frac{1}{2}\right)^8 = \frac{1}{256}.$$

■

Exercise 2.

Let $h_t = \tanh(wh_{t-1} + z_t)$ for a scalar w. Express $\partial h_T/\partial h_s$ and prove that its absolute value is bounded by $|w|^{T-s}$.

Solution. Differentiating one time step gives

$$\frac{\partial h_t}{\partial h_{t-1}} = w\big(1 - h_t^2\big).$$

The chain rule therefore yields

$$\frac{\partial h_T}{\partial h_s} = \prod_{t=s+1}^{T} w(1 - h_t^2) = w^{T-s} \prod_{t=s+1}^{T} (1 - h_t^2).$$

Because $|h_t| < 1$ for finite pre-activations, $0 < 1 - h_t^2 \le 1$. Hence

$$\left|\frac{\partial h_T}{\partial h_s}\right| \le |w|^{T-s}.$$

The bound can be much smaller if any time step has a saturated activation. ■

Exercise 3.

For a scalar fixed-gate recurrence $h_t = (1-g)h_{t-1} + g\widetilde{h}_t$ with constant $g \in (0,1)$, suppose $h_0 = 1$ and $\widetilde{h}_t = 0$ for all t. Determine the number of time steps needed to reduce $|h_t|$ to at most $\varepsilon \in (0,1)$.

Solution. The explicit solution is $h_t = (1-g)^t$. We require

$$(1-g)^t \le \varepsilon,$$
$$t \log(1-g) \le \log \varepsilon.$$

Since $\log(1-g) < 0$, division reverses the inequality:

$$t \ge \frac{\log \varepsilon}{\log(1-g)}.$$

Thus the smallest integer is

$$t_{\min} = \left\lceil \frac{\log \varepsilon}{\log(1-g)} \right\rceil.$$

For small g, $\log(1-g) \approx -g$, so $t_{\min}$ is approximately $\log(1/\varepsilon)/g$. ■

Exercise 4.

In a scalar GRU, suppose $h_{t-1} = 0.6$, $r_t = 0.2$, $g_t = 0.3$, $x_t = 1$, and the candidate parameters are $U_h = 0.5$, $W_h = 1$, $b_h = 0$. Compute the candidate and updated state.

Solution. The reset-filtered previous state is $r_t h_{t-1} = 0.12$. Hence

$$\begin{aligned}\widetilde{h}_t &= \tanh(x_t U_h + r_t h_{t-1} W_h + b_h) \\ &= \tanh(0.5 + 0.12) = \tanh(0.62) \approx 0.5511.\end{aligned}$$

The updated state is

$$\begin{aligned}h_t &= g_t \widetilde{h}_t + (1 - g_t) h_{t-1} \\ &= 0.3(0.5511) + 0.7(0.6) \\ &\approx 0.5853.\end{aligned}$$

The result remains close to the old state because the update gate is below one half. ■

Exercise 5.

For the scalar update $h = gc + (1-g)p$, derive the partial derivatives with respect to g, c, and p when the other two quantities are held fixed. Then explain why these are *not* the full derivative $\mathrm{d}h/\mathrm{d}p$ in a GRU.

Solution. The local partial derivatives are

$$\frac{\partial h}{\partial g} = c - p, \qquad \frac{\partial h}{\partial c} = g, \qquad \left.\frac{\partial h}{\partial p}\right|_{g,c} = 1 - g.$$

However, in a GRU both g and c depend on p. Applying the total derivative gives

$$\begin{aligned}\frac{\mathrm{d}h}{\mathrm{d}p} &= \frac{\partial h}{\partial p} + \frac{\partial h}{\partial g}\frac{\mathrm{d}g}{\mathrm{d}p} + \frac{\partial h}{\partial c}\frac{\mathrm{d}c}{\mathrm{d}p} \\ &= 1 - g + (c - p)\frac{\mathrm{d}g}{\mathrm{d}p} + g\frac{\mathrm{d}c}{\mathrm{d}p}.\end{aligned}$$

Ignoring the last two terms would discard gradient paths through the update gate and candidate. ■

Exercise 6.

With input dimension $d = 12$ and hidden dimension $H = 40$, compare the number of recurrent-cell parameters in a simple RNN and a GRU, including biases but excluding any output layer.

Solution. The simple RNN contains

$$\begin{aligned} dH + H^2 + H &= 12(40) + 40^2 + 40 \\ &= 480 + 1600 + 40 = 2120 \end{aligned}$$

parameters. The GRU contains three affine branches of these dimensions, and therefore

$$3(dH + H^2 + H) = 3(2120) = 6360$$

parameters. The GRU has three times the recurrent-cell parameters under the conventions of this lecture. ■

Exercise 7.

Assume all recurrent gates in an H-dimensional GRU are locally constant with respect to h_{t-1}, and that the candidate is also locally constant with respect to h_{t-1}. Identify J_t and express $J_{s\to t}$ across many steps. What happens if every update gate equals a common scalar g?

Solution. Under the stated assumptions, all indirect derivative terms in the GRU Jacobian vanish. Hence

$$J_t = D_{\mathbf{1}-g_t}.$$

Because these are diagonal matrices,

$$J_{s\to t} = \prod_{r=s+1}^{t} D_{\mathbf{1}-g_r} = \operatorname{diag}\left(\prod_{r=s+1}^{t} (1-g_{r,1}), \dots, \prod_{r=s+1}^{t} (1-g_{r,H}) \right).$$

If every coordinate of every gate equals the same constant g, then

$$J_{s\to t} = (1-g)^{t-s} I_H.$$

Thus small fixed update gates result in slower gradient decay along the direct retention path. ■

18 Lecture 19: Attention Mechanisms and Transformer Representations

In Lectures 17 and 18, recurrent neural networks processed an input sequence by propagating a hidden state through time. Consequently, the representation at a late position depended on earlier inputs through a potentially long chain of state transitions. The central idea of *attention* is to replace this fixed chain of intermediate transformations with a learned, input-dependent rule that directly combines information from different positions in a sequence.

For a sequence of word embeddings, attention allows the representation of a word to depend on whichever other words are relevant to it. These dependencies are represented by similarity scores between a *query* and a collection of *keys*, and the resulting weights are used to average corresponding *values*. This lecture develops the mechanism in scalar, vector, and matrix notation; explains the role of positional encodings; and extends a single attention head to multi-head attention.

18.1 From Word Embeddings to Context-Dependent Representations

18.1.1 Embedding a Sequence

Consider a vocabulary $\mathcal{V}$ with N_{vocab} words. An embedding model assigns each vocabulary item w a vector $e(w) \in \mathbb{R}^d$. In Lecture 14, the embedding was a learned vector intended to capture semantic information about a word. Such a vector is *static* if it is the same whenever that word appears, regardless of surrounding words.

As an example, consider the phrases "river bank" and "bank loan." A fixed embedding of the word *bank* does not itself encode which meaning is intended in a particular sentence. A contextual model instead produces a new vector that incorporates information from other words in the sequence.

For a sequence of n tokens $(w_1, \dots, w_n)$, let

$$x_i = e(w_i) \in \mathbb{R}^d, \qquad i = 1, \dots, n,$$

and assemble the word embeddings into a row-oriented matrix

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

Each row corresponds to a token position; each column corresponds to one embedding coordinate. We use row vectors inside matrices even though the individual x_i are written as column vectors.

Definition 18.1 (Contextual representation)**.** A *contextual representation* of position i is a vector $h_i \in \mathbb{R}^{d_v}$ computed using not only x_i, but also some or all of $x_1, \ldots, x_n$. An attention mechanism forms such a representation through data-dependent weights:

$$h_i = \sum_{j=1}^{n} a_{ij} v_j, \qquad a_{ij} \geq 0, \quad \sum_{j=1}^{n} a_{ij} = 1,$$

where $v_j \in \mathbb{R}^{d_v}$ is a value vector and a_{ij} measures how strongly position i attends to position j.

The weights a_{ij} are not fixed coefficients shared across all sentences. They depend on the particular input sequence and therefore adapt to its content. Moreover, each output position i receives its own collection of weights $(a_{i1}, \ldots, a_{in})$.

18.1.2 Queries, Keys, and Values

Definition 18.2 (Learned attention projections)**.** Choose a query/key dimension $d_k \geq 1$ and a value dimension $d_v \geq 1$. The trainable projections are

$$W^{(Q)} \in \mathbb{R}^{d \times d_k}, \qquad W^{(K)} \in \mathbb{R}^{d \times d_k}, \qquad W^{(V)} \in \mathbb{R}^{d \times d_v}.$$

For token i, define

$$\begin{aligned} q_i &= (W^{(Q)})^\top x_i \in \mathbb{R}^{d_k}, \\ k_i &= (W^{(K)})^\top x_i \in \mathbb{R}^{d_k}, \\ v_i &= (W^{(V)})^\top x_i \in \mathbb{R}^{d_v}. \end{aligned}$$

The three vectors are respectively the *query*, *key*, and *value* of the token.

The query q_i describes the information requested by position i. The key k_j represents features against which that query is matched, while v_j is the information made available by position j. It is important that a token contributes three distinct learned vectors, even though they arise from the same input embedding.

In matrix notation, all n positions can be projected simultaneously:

$$\begin{aligned} Q &= XW^{(Q)} \in \mathbb{R}^{n \times d_k}, \\ K &= XW^{(K)} \in \mathbb{R}^{n \times d_k}, \\ V &= XW^{(V)} \in \mathbb{R}^{n \times d_v}. \end{aligned}$$

Thus the ith rows of Q, K, and V are respectively $q_i^\top$, $k_i^\top$, and $v_i^\top$.

18.2 Scaled Dot-Product Attention

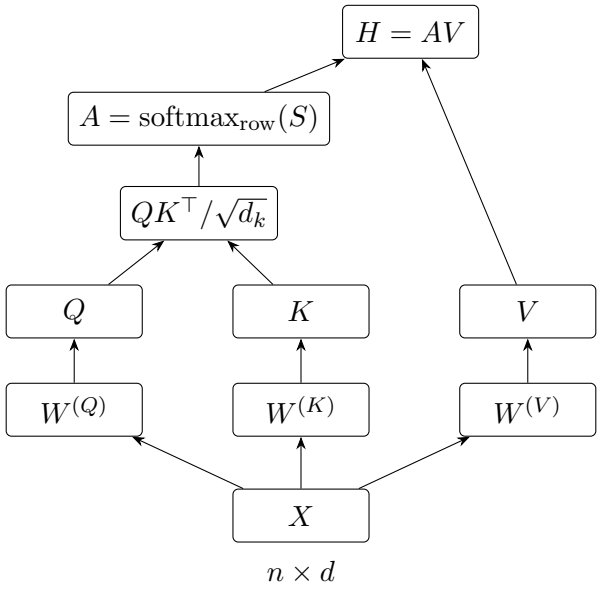


Figure 45: The three learned projections and the subsequent scaled dot-product attention computation. The softmax is applied separately to each row of the score matrix.

18.2.1 Similarity Scores

The query at position i is compared with the key at position j by their Euclidean inner product. The unnormalized similarity is

$$q_i^\top k_j = \sum_{\ell=1}^{d_k} q_{i\ell} k_{j\ell}.$$

A large positive inner product indicates strong alignment in the learned query/key space. A large negative product indicates opposite alignment, and a small magnitude indicates weak alignment. Since the projections are learned, the relevant geometry is determined by the training objective.

Definition 18.3 (Scaled attention scores)**.** For $i, j \in \{1, \ldots, n\}$ define

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}.$$

The complete score matrix is

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}.$$

For fixed i, the row $(s_{i1}, \ldots, s_{in})$ contains the scores obtained by comparing query q_i with *every* key in the input sequence. Therefore one row specifies all the candidate positions that token i may attend to.

18.2.2 Why Scale the Dot Product?

The denominator stabilizes the magnitude of the scores as the query/key dimension increases. The following calculation gives a common statistical motivation; it is not a claim that learned query and key coordinates remain independent during training.

Proposition 18.1 (Variance of a random dot product)**.** *Suppose $q = (q_1, \dots, q_{d_k})$ and $k = (k_1, \dots, k_{d_k})$ have independent, centered components such that $\mathbb{E}[q_\ell^2] = \mathbb{E}[k_\ell^2] = 1$. Assume all q_ℓ are independent of all k_r, and distinct coordinate pairs are independent. Then*

$$\mathbb{E}[q^\top k] = 0, \qquad \mathrm{Var}(q^\top k) = d_k,$$

and consequently

$$\mathrm{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) = 1.$$

Proof. Linearity of expectation and independence imply

$$\mathbb{E}[q^\top k] = \sum_{\ell=1}^{d_k} \mathbb{E}[q_\ell k_\ell] = \sum_{\ell=1}^{d_k} \mathbb{E}[q_\ell]\mathbb{E}[k_\ell] = 0.$$

Because the coordinate products have zero mean and are mutually independent,

$$\begin{aligned}
\mathrm{Var}(q^\top k) &= \mathrm{Var}\left(\sum_{\ell=1}^{d_k} q_\ell k_\ell\right) \\
&= \sum_{\ell=1}^{d_k} \mathrm{Var}(q_\ell k_\ell) \\
&= \sum_{\ell=1}^{d_k} \mathbb{E}[q_\ell^2 k_\ell^2] \\
&= \sum_{\ell=1}^{d_k} \mathbb{E}[q_\ell^2]\mathbb{E}[k_\ell^2] = d_k.
\end{aligned}$$

Dividing by $\sqrt{d_k}$ scales the variance by $1/d_k$, proving the claim. □

Remark 18.1 (Softmax saturation). If the scores in a row differ by extremely large amounts, softmax can become nearly one-hot, and its derivatives with respect to the scores become small. Scaling by $\sqrt{d_k}$ reduces the typical score magnitude under the assumptions above, helping prevent saturation at initialization. It does not ensure that saturation cannot occur later.

18.2.3 Converting Scores into Attention Weights

Definition 18.4 (Row-wise softmax)**.** The *attention-weight matrix* is obtained by applying softmax separately to each row of S:

$$A = \mathrm{softmax}_{\mathrm{row}}(S) \in \mathbb{R}^{n \times n}, \qquad a_{ij} = \frac{e^{s_{ij}}}{\sum_{r=1}^{n} e^{s_{ir}}}.$$

Equivalently,

$$a_{ij} = \frac{\exp\left(q_i^\top k_j / \sqrt{d_k}\right)}{\sum_{r=1}^{n} \exp\left(q_i^\top k_r / \sqrt{d_k}\right)}.$$

Proposition 18.2 (Attention weights form row distributions)**.** *For each query position i,*

$$a_{ij} > 0 \quad (j = 1, \ldots, n), \qquad \sum_{j=1}^{n} a_{ij} = 1.$$

Equivalently, A is row-stochastic: $A\mathbf{1}_n = \mathbf{1}_n$.

Proof. Every exponential is strictly positive. Summing across the ith row gives

$$\begin{aligned} \sum_{j=1}^{n} a_{ij} &= \sum_{j=1}^{n} \frac{e^{s_{ij}}}{\sum_{r=1}^{n} e^{s_{ir}}} \\ &= \frac{\sum_{j=1}^{n} e^{s_{ij}}}{\sum_{r=1}^{n} e^{s_{ir}}} = 1. \end{aligned}$$

□

The weights encode a *soft selection* of relevant positions. Unlike a hard maximum operation, the softmax makes the weights differentiable with respect to the scores, which in turn depend differentiably on the learned projections.

18.2.4 Weighted Combination of Values

Definition 18.5 (Single-head attention)**.** The output of scaled dot-product attention is

$$H = AV = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{n \times d_v}.$$

The contextual output at position i is

$$h_i = \sum_{j=1}^{n} a_{ij} v_j \in \mathbb{R}^{d_v}.$$

We can verify this directly from the matrix product. Since A has shape $n \times n$ and V has shape $n \times d_v$,

$$H_{i\ell} = (AV)_{i\ell} = \sum_{j=1}^{n} A_{ij} V_{j\ell} = \sum_{j=1}^{n} a_{ij} (v_j)_\ell,$$

for each output coordinate $\ell = 1, \ldots, d_v$. Hence the ith output row is exactly the weighted average of the value vectors.

Proposition 18.3 (Convex-hull property)**.** *For each i, the output vector h_i belongs to the convex hull of $\{v_1, \ldots, v_n\}$. In particular,*

$$\|h_i\|_2 \le \max_{1 \le j \le n} \|v_j\|_2.$$

Proof. The weights a_{ij} are nonnegative and sum to one, so $h_i = \sum_j a_{ij} v_j$ is a convex combination by definition. By the triangle inequality,

$$\begin{aligned} \|h_i\|_2 &= \left\| \sum_{j=1}^{n} a_{ij} v_j \right\|_2 \le \sum_{j=1}^{n} a_{ij} \|v_j\|_2 \\ &\le \left(\max_j \|v_j\|_2 \right) \sum_{j=1}^{n} a_{ij} = \max_j \|v_j\|_2. \end{aligned}$$

□

Remark 18.2. The convex-hull property concerns the *values of a single head before its output projection*. Once different heads are concatenated and multiplied by a learned output matrix, the final representation need not lie in the convex hull of any one head's value vectors.

18.3 The Attention Matrix and a Worked Example

18.3.1 Interpreting the Attention Matrix

The $n \times n$ attention matrix can be thought of as a table whose rows correspond to *receiving* or *querying* positions and whose columns correspond to *source* or *key/value* positions:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}.$$

For example, the coefficient a_{23} is the weight assigned by query position 2 to the value at position 3. In general a_{23} need not equal a_{32}, even if $q_i = k_i$ for all i, because different rows have different normalization denominators.

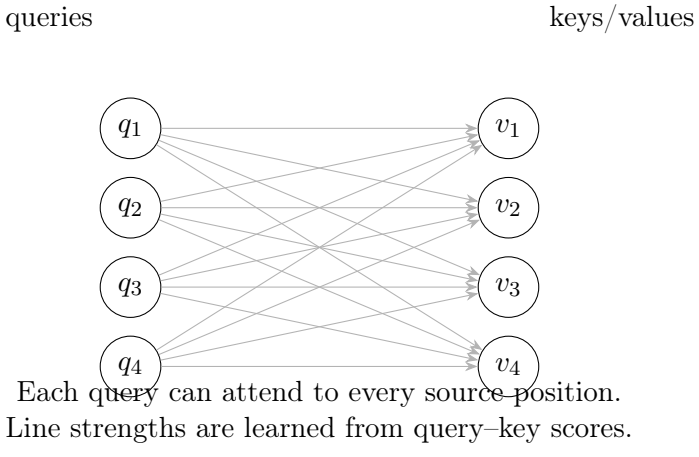


Figure 46: Full self-attention. The n^2 directed query-to-source comparisons are computed simultaneously by $QK^\top$.

18.3.2 An Explicit Three-Token Calculation

Example 18.1 (Scaled self-attention with three tokens)**.** Let $n = 3$, $d = d_k = d_v = 2$, and choose all three projection matrices to be the 2×2 identity. Consider

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad Q = K = V = X.$$

Then

$$\begin{aligned} QK^\top &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \qquad S = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Write $c = e^{1/\sqrt{2}}$. The denominator for rows 1 and 3 is $2c + 1$, whereas the denominator for row 2 is $c + 2$. Therefore,

$$A = \begin{bmatrix} \frac{c}{2c+1} & \frac{1}{2c+1} & \frac{c}{2c+1} \\ \frac{1}{c+2} & \frac{c}{c+2} & \frac{1}{c+2} \\ \frac{c}{2c+1} & \frac{1}{2c+1} & \frac{c}{2c+1} \end{bmatrix}.$$

The weighted-value matrix is

$$H = AV = \begin{bmatrix} \frac{c}{2c+1} & \frac{1}{2c+1} & \frac{c}{2c+1} \\ \frac{1}{c+2} & \frac{c}{c+2} & \frac{1}{c+2} \\ \frac{c}{2c+1} & \frac{1}{2c+1} & \frac{c}{2c+1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$
$$= \begin{bmatrix} \frac{2c}{2c+1} & \frac{1}{2c+1} \\ \frac{2}{c+2} & \frac{c}{c+2} \\ \frac{2c}{2c+1} & \frac{1}{2c+1} \end{bmatrix}.$$

The first and third input rows are identical, so the first and third output rows are identical. This is an important clue: without positional information, self-attention has no built-in method of distinguishing identical tokens at different positions.

Remark 18.3 (Numerically stable row normalization). For each query row i, let $m_i = \max_j s_{ij}$. By the shift invariance of softmax,

$$a_{ij} = \frac{\exp(s_{ij} - m_i)}{\sum_{r=1}^{n} \exp(s_{ir} - m_i)}.$$

This form is preferable in floating-point arithmetic because the exponentials do not overflow. In an implementation, the subtraction must be carried out *row by row*, not by using an unrelated maximum over the entire matrix.

18.4 Why Position Must Be Represented Explicitly

18.4.1 Attention without Positional Information

The attention computations $Q = XW^{(Q)}$, $K = XW^{(K)}$, and $V = XW^{(V)}$ do not inherently depend on the order in which token rows appear. An important structural property is that self-attention is *permutation equivariant* when no position-dependent information is included.

Proposition 18.4 (Permutation equivariance of self-attention)**.** *Let $P \in \mathbb{R}^{n \times n}$ be a permutation matrix, and define*

$$F(X) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{XW^{(Q)}(XW^{(K)})^{\top}}{\sqrt{d_k}}\right) XW^{(V)}.$$

Then

$$F(PX) = PF(X).$$

Thus permuting the input token positions simply permutes the output positions in the same way, rather than giving the model independent information about the original order.

Proof. If $X' = PX$, then

$$Q' = PQ, \qquad K' = PK, \qquad V' = PV.$$

Consequently,

$$S' = \frac{Q'(K')^\top}{\sqrt{d_k}} = \frac{PQK^\top P^\top}{\sqrt{d_k}} = PSP^\top.$$

The matrix $PSP^\top$ is formed by permuting the rows and columns of S using the same permutation. Since the softmax of a permuted vector is the correspondingly permuted softmax vector, applying row-wise softmax gives

$$A' = \mathrm{softmax}_{\mathrm{row}}(PSP^\top) = P\,\mathrm{softmax}_{\mathrm{row}}(S)P^\top = PAP^\top.$$

Therefore

$$F(PX) = A'V' = PAP^\top PV = PA(P^\top P)V = PF(X),$$

using $P^\top P = I_n$. □

Remark 18.4 (Why order matters). The sentences "dog bites man" and "man bites dog" contain the same words but have different meanings. A model intended to interpret sequences must distinguish their positions or relative order. Vanilla attention can condition on the token embeddings, but it does not supply position labels by itself. A positional representation breaks the pure permutation symmetry exhibited above.

18.4.2 Absolute Positional Encodings

Definition 18.6 (Additive positional encoding)**.** Let $p_i \in \mathbb{R}^d$ denote the positional vector assigned to sequence position i. Define the position-aware input

$$\widetilde{x}_i = x_i + p_i, \qquad \widetilde{X} = X + P_{\mathrm{pos}},$$

where the ith row of $P_{\mathrm{pos}} \in \mathbb{R}^{n\times d}$ is $p_i^\top$. Self-attention then computes

$$\begin{aligned} Q &= \widetilde{X}W^{(Q)}, \\ K &= \widetilde{X}W^{(K)}, \\ V &= \widetilde{X}W^{(V)}. \end{aligned}$$

The positional table P_{pos} may be learned, or it may be defined by a deterministic formula. The handwritten lecture uses *sinusoidal positional encodings*, which assign different frequencies to successive pairs of embedding coordinates.

Definition 18.7 (Sinusoidal positional encoding)**.** Assume that d is even, index the sequence positions by $p = 0, 1, \ldots, n-1$, and index embedding coordinates from 0 to $d-1$. For $j = 0, 1, \ldots, d/2 - 1$, define

$$\mathrm{PE}(p, 2j) = \sin\left(\frac{p}{10000^{2j/d}}\right),$$
$$\mathrm{PE}(p, 2j+1) = \cos\left(\frac{p}{10000^{2j/d}}\right).$$

The positional vector $p_p \in \mathbb{R}^d$ is obtained by collecting these values over all coordinate pairs.

For each j, define a frequency $\omega_j = 10000^{-2j/d}$. The corresponding coordinate pair of the positional vector is

$$\begin{bmatrix} \sin(\omega_j p) \\ \cos(\omega_j p) \end{bmatrix}.$$

Different pairs oscillate at different frequencies. Consequently, a position is represented simultaneously on multiple length scales.

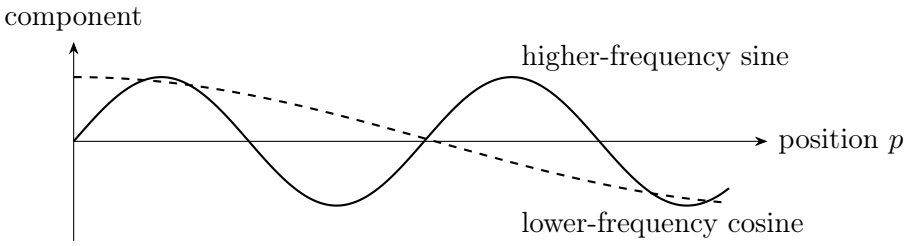


Figure 47: Illustrative sinusoidal components with different frequencies. In the standard encoding, the frequencies are selected by powers of $10000^{-2j/d}$.

Proposition 18.5 (Relative shifts act linearly on each frequency pair)**.** *Fix a frequency $\omega > 0$ and define*

$$u(p) = \begin{bmatrix} \sin(\omega p) \\ \cos(\omega p) \end{bmatrix}.$$

For any displacement $r \in \mathbb{R}$,

$$u(p+r) = R_\omega(r)u(p), \quad R_\omega(r) = \begin{bmatrix} \cos(\omega r) & \sin(\omega r) \\ -\sin(\omega r) & \cos(\omega r) \end{bmatrix}.$$

The matrix $R_\omega(r)$ is orthogonal and depends only on the displacement r, not on the original position p.

Proof. The angle-addition formulas give

$$\sin(\omega(p+r)) = \sin(\omega p)\cos(\omega r) + \cos(\omega p)\sin(\omega r),$$
$$\cos(\omega(p+r)) = \cos(\omega p)\cos(\omega r) - \sin(\omega p)\sin(\omega r).$$

Writing these two equalities in matrix form yields the stated representation. Direct multiplication shows $R_\omega(r)^\top R_\omega(r) = I_2$. □

Remark 18.5 (Interpretation and limitations). The previous proposition explains a convenient algebraic property of sinusoidal encodings: a fixed shift of position acts by a fixed rotation in each two-dimensional frequency subspace. It does not, by itself, mean that an arbitrary learned attention head automatically computes relative positions. Learned projections must still use the positional information in an appropriate way. Furthermore, adding positional vectors is a design choice; other attention architectures incorporate relative position directly in their scores.

18.5 Multi-Head Attention

18.5.1 Motivation for Multiple Heads

A single attention matrix assigns one normalized collection of source weights to each query position. A different collection of projections may identify a different notion of relevance. For example, one attention head might emphasize local grammatical dependencies while another might respond to distant semantic associations. These interpretations are possibilities rather than guarantees: the actual learned behavior depends on data, initialization, and training.

The *multi-head attention* construction computes several separate attention mechanisms in parallel, each with its own query, key, and value projections, and then combines their outputs with a final learned linear map.

Definition 18.8 (Multi-head self-attention)**.** Let the number of heads be $H_0 \geq 1$. For each head $r \in \{1, \ldots, H_0\}$ choose projection matrices

$$W_r^{(Q)}, W_r^{(K)} \in \mathbb{R}^{d \times d_k}, \qquad W_r^{(V)} \in \mathbb{R}^{d \times d_v}.$$

Given a position-aware input $\widetilde{X} \in \mathbb{R}^{n \times d}$, set

$$\begin{aligned} Q_r &= \widetilde{X} W_r^{(Q)}, \\ K_r &= \widetilde{X} W_r^{(K)}, \\ V_r &= \widetilde{X} W_r^{(V)}, \\ A_r &= \mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_r K_r^\top}{\sqrt{d_k}}\right), \\ H_r &= A_r V_r \in \mathbb{R}^{n \times d_v}. \end{aligned}$$

Concatenate the head outputs along the feature dimension and apply an output projection $W^{(O)} \in \mathbb{R}^{H_0 d_v \times d_{\mathrm{out}}}$:

$$\mathrm{MultiHead}(\widetilde{X}) = \mathrm{Concat}(H_1, \ldots, H_{H_0}) W^{(O)} \in \mathbb{R}^{n \times d_{\mathrm{out}}}.$$

The symbol H_0 counts the heads and is distinct from H_r, the output matrix of head r. In many common architectures, $d_{\mathrm{out}} = d$ so that the multi-head module can be combined with its input

in a residual connection. When $d_k = d_v = d/H_0$ and d is divisible by H_0, concatenating H_0 heads restores the original feature dimension d before applying $W^{(O)}$.

18.5.2 Dimension Check and Parameter Count

For every head r, the dimensions are

$$\begin{aligned} \widetilde{X} &\in \mathbb{R}^{n\times d}, \\ Q_r, K_r &\in \mathbb{R}^{n\times d_k}, \\ V_r, H_r &\in \mathbb{R}^{n\times d_v}, \\ Q_r K_r^\top, A_r &\in \mathbb{R}^{n\times n}. \end{aligned}$$

Hence

$$\mathrm{Concat}(H_1, \ldots, H_{H_0}) \in \mathbb{R}^{n\times H_0 d_v},$$

which can indeed be multiplied by $W^{(O)} \in \mathbb{R}^{H_0 d_v \times d_{\text{out}}}$.

Proposition 18.6 (Number of weight parameters)**.** *Without biases, a multi-head attention block with H_0 heads has*

$$H_0(2dd_k + dd_v) + H_0 d_v d_{\text{out}}$$

trainable projection weights. If $d_k = d_v = d/H_0$ and $d_{\text{out}} = d$, the total simplifies to $4d^2$.

Proof. Each head has dd_k query weights, dd_k key weights, and dd_v value weights, for a subtotal of $2dd_k + dd_v$. There are H_0 heads. The final output projection contributes $H_0 d_v d_{\text{out}}$ weights. Under the stated dimensions,

$$H_0\left(2d\frac{d}{H_0} + d\frac{d}{H_0}\right) + H_0\frac{d}{H_0}d = 3d^2 + d^2 = 4d^2.$$

□

Remark 18.6. The parameter count depends primarily on the model width d, while the $n \times n$ score matrices determine a major part of the computational and memory costs. These are different resource constraints: increasing the sequence length may greatly increase attention memory even without changing the number of learned weights.

18.5.3 Multi-Head Attention with Position Encoding

Putting the previous constructions together, a minimal attention block can be written as the composition

$$\begin{aligned} \widetilde{X} &= X + P_{\text{pos}}, \\ H_r &= \mathrm{softmax}_{\text{row}}\left(\frac{(\widetilde{X}W_r^{(Q)})(\widetilde{X}W_r^{(K)})^\top}{\sqrt{d_k}}\right)(\widetilde{X}W_r^{(V)}), \\ Y &= \mathrm{Concat}(H_1, \ldots, H_{H_0})W^{(O)}. \end{aligned}$$

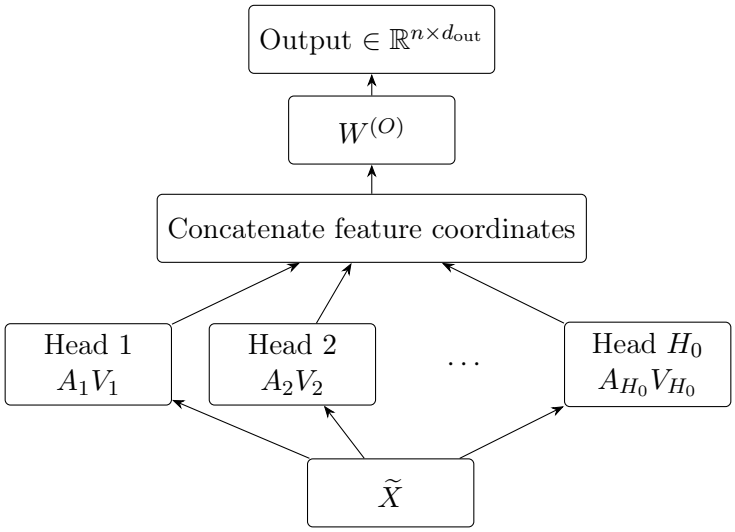


Figure 48: Multi-head attention: several separate Q/K/V projections and attention matrices generate feature vectors that are concatenated and linearly projected.

This corresponds to the sequence in the handwritten notes: token embeddings are augmented by position information, projected into query/key/value spaces, weighted through attention, and finally combined across heads.

18.6 Attention as a Differentiable Computational Graph

The projections and softmax weights are learned by gradient-based optimization. Since Lecture 7 developed reverse-mode automatic differentiation and Lecture 8 developed matrix backward rules, it is useful to derive the backward pass of an attention head explicitly.

18.6.1 Derivative of the Softmax Function

Proposition 18.7 (Softmax Jacobian)**.** *Let $a_j = e^{s_j} / \sum_{r=1}^{n} e^{s_r}$. For $j, \ell \in \{1, \dots, n\}$,*

$$\frac{\partial a_j}{\partial s_\ell} = a_j(\delta_{j\ell} - a_\ell),$$

where $\delta_{j\ell}$ is the Kronecker delta. Thus

$$D\,\mathrm{softmax}(s) = \mathrm{diag}(a) - aa^\top.$$

Proof. Writing $Z = \sum_{r=1}^{n} e^{s_r}$ and applying the quotient rule,

$$\begin{aligned}\frac{\partial a_j}{\partial s_\ell} &= \frac{\delta_{j\ell} e^{s_j} Z - e^{s_j} e^{s_\ell}}{Z^2} \\ &= \frac{e^{s_j}}{Z}\left(\delta_{j\ell} - \frac{e^{s_\ell}}{Z}\right) = a_j(\delta_{j\ell} - a_\ell).\end{aligned}$$

□

Given an upstream gradient $g_a \in \mathbb{R}^n$, the downstream gradient through a softmax row is

$$\begin{aligned} g_s &= \big(\operatorname{diag}(a) - aa^\top\big) g_a \\ &= \operatorname{diag}(a) g_a - a(a^\top g_a) \\ &= a \odot \big(g_a - (a^\top g_a)\mathbf{1}_n\big). \end{aligned}$$

Here $\odot$ denotes entrywise multiplication. This form requires only vector operations and avoids materializing the entire $n \times n$ softmax Jacobian for each row.

18.6.2 Matrix Backward Rules for a Single Attention Head

Consider

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A = \operatorname{softmax}_{\text{row}}(S), \qquad H = AV,$$

and let $L = L(H)$ be a differentiable scalar loss. The gradient of L with respect to a matrix is defined using the Frobenius inner product:

$$\mathrm{d}L = \langle \nabla_M L, \mathrm{d}M \rangle_F, \qquad \langle B, C \rangle_F = \operatorname{tr}(B^\top C).$$

Denote the upstream gradient by $G_H = \nabla_H L \in \mathbb{R}^{n \times d_v}$.

Theorem 18.1 (Backpropagation through scaled dot-product attention)**.** *For the single-head attention map above, let*

$$\begin{aligned} G_V &= A^\top G_H, \\ G_A &= G_H V^\top. \end{aligned}$$

For each row i, let $a_i = (a_{i1}, \ldots, a_{in})^\top$ and $g_{A,i}$ be the transpose of row i of G_A. Define G_S row by row as

$$g_{S,i} = a_i \odot \big(g_{A,i} - (a_i^\top g_{A,i})\mathbf{1}_n\big).$$

Then the gradients with respect to the queries and keys are

$$G_Q = \frac{G_S K}{\sqrt{d_k}}, \qquad G_K = \frac{G_S^\top Q}{\sqrt{d_k}}.$$

Consequently, when $Q = \widetilde{X} W^{(Q)}$, $K = \widetilde{X} W^{(K)}$, and $V = \widetilde{X} W^{(V)}$,

$$\begin{aligned} \nabla_{W^{(Q)}} L &= \widetilde{X}^\top G_Q, \\ \nabla_{W^{(K)}} L &= \widetilde{X}^\top G_K, \\ \nabla_{W^{(V)}} L &= \widetilde{X}^\top G_V, \\ \nabla_{\widetilde{X}} L &= G_Q (W^{(Q)})^\top + G_K (W^{(K)})^\top + G_V (W^{(V)})^\top. \end{aligned}$$

Proof. First differentiate $H = AV$:

$$\mathrm{d}H = (\mathrm{d}A)V + A(\mathrm{d}V).$$

Taking the Frobenius inner product with G_H and cyclically permuting traces gives

$$\begin{aligned}\mathrm{d}L &= \operatorname{tr}\big(G_H^\top(\mathrm{d}A)V\big) + \operatorname{tr}\big(G_H^\top A(\mathrm{d}V)\big) \\ &= \operatorname{tr}\big((G_H V^\top)^\top \mathrm{d}A\big) + \operatorname{tr}\big((A^\top G_H)^\top \mathrm{d}V\big).\end{aligned}$$

Therefore $G_A = G_H V^\top$ and $G_V = A^\top G_H$. Next, row-wise softmax does not mix different query rows. Applying the softmax Jacobian to each row yields exactly

$$g_{S,i} = \big(\operatorname{diag}(a_i) - a_i a_i^\top\big)\, g_{A,i} = a_i \odot \big(g_{A,i} - (a_i^\top g_{A,i})\mathbf{1}_n\big).$$

Now differentiate the score matrix:

$$\mathrm{d}S = \frac{(\mathrm{d}Q)K^\top + Q(\mathrm{d}K)^\top}{\sqrt{d_k}}.$$

It follows that

$$\begin{aligned}\mathrm{d}L &= \langle G_S, \mathrm{d}S\rangle_F \\ &= \left\langle \frac{G_S K}{\sqrt{d_k}}, \mathrm{d}Q \right\rangle_F + \left\langle \frac{G_S^\top Q}{\sqrt{d_k}}, \mathrm{d}K \right\rangle_F.\end{aligned}$$

Hence $G_Q = G_S K/\sqrt{d_k}$ and $G_K = G_S^\top Q/\sqrt{d_k}$. Finally,

$$\begin{aligned}\mathrm{d}Q &= (\mathrm{d}\widetilde{X})W^{(Q)} + \widetilde{X}(\mathrm{d}W^{(Q)}), \\ \mathrm{d}K &= (\mathrm{d}\widetilde{X})W^{(K)} + \widetilde{X}(\mathrm{d}W^{(K)}), \\ \mathrm{d}V &= (\mathrm{d}\widetilde{X})W^{(V)} + \widetilde{X}(\mathrm{d}W^{(V)}).\end{aligned}$$

Applying the matrix-product backward rule to each expression and accumulating all contributions to $\widetilde{X}$ proves the remaining identities. □

Remark 18.7 (Multiple gradient paths). The same input $\widetilde{X}$ is used by all three projection branches. Its gradient is therefore the *sum* of the contributions from the query, key, and value branches. This is precisely the gradient-accumulation rule for computational graphs introduced in Lecture 7. If the positions are added using $\widetilde{X} = X + P_{\text{pos}}$ with a learned position matrix, the upstream gradient is also passed to both X and P_{pos} (subject to any parameter sharing).

18.7 Masking, Cross-Attention, and Computational Cost

The handwritten lecture develops full self-attention, in which every token can potentially attend to every other token. Two closely related mechanisms are useful for understanding how the same formula appears in broader transformer architectures.

18.7.1 Causal Attention

For autoregressive sequence generation, the representation at position i must not use information from future positions $j > i$. This restriction can be imposed by modifying the scores before softmax.

Definition 18.9 (Causal attention mask)**.** Let $S \in \mathbb{R}^{n\times n}$ be a score matrix. Define the masked weights by

$$a_{ij} = \begin{cases} \dfrac{e^{s_{ij}}}{\sum_{r=1}^{i} e^{s_{ir}}}, & j \le i, \\ 0, & j > i. \end{cases}$$

This is equivalent, in the extended-real convention, to placing $-\infty$ at disallowed score locations and applying row-wise softmax.

The first position can attend only to itself, the second can attend to the first two positions, and so on. The resulting attention matrix has the form

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}, \qquad \sum_{j=1}^{i} a_{ij} = 1.$$

Causal attention is particularly natural for next-token prediction, since the output at time i may depend on the prefix $(w_1, \ldots, w_i)$ but not on unseen future words.

18.7.2 Cross-Attention

Self-attention uses the same input source for queries, keys, and values. *Cross-attention* instead allows queries to come from one sequence and keys/values from another. For a query representation $X_Q \in \mathbb{R}^{n_q\times d_q}$ and a context representation $X_C \in \mathbb{R}^{n_c\times d_c}$, choose projection matrices so that

$$Q \in \mathbb{R}^{n_q\times d_k}, \qquad K \in \mathbb{R}^{n_c\times d_k}, \qquad V \in \mathbb{R}^{n_c\times d_v}.$$

Then

$$\mathrm{CrossAttn}(Q, K, V) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{n_q\times d_v}.$$

The attention matrix now has shape $n_q \times n_c$: each query position selects information from the n_c context positions. The same single-head gradient theorem applies, with the corresponding rectangular dimensions.

18.7.3 Complexity and Parallelism

Proposition 18.8 (Cost of full attention)**.** *For a single head and a sequence of length n, forming $QK^\top$ takes $O(n^2 d_k)$ arithmetic operations, and forming AV takes $O(n^2 d_v)$ operations. The score and weight matrices each have n^2 entries. Thus the principal attention mixing cost is*

$$O\big(n^2(d_k + d_v)\big),$$

with $O(n^2)$ attention-weight storage in a straightforward implementation, in addition to storage for activations and projection weights.

Proof. Every one of the n^2 elements of $QK^\top$ is a dot product of two vectors with d_k entries, giving $O(n^2 d_k)$ operations. Every one of the nd_v elements of AV requires a length-n dot product, giving $O(n^2 d_v)$ operations. The score matrix and attention matrix are both $n \times n$. □

Remark 18.8 (Attention versus recurrent processing). RNNs require a temporal dependency chain across successive hidden-state updates, even though each update may be relatively inexpensive. Full self-attention computes all query–key interactions using matrix operations that can be parallelized across sequence positions, but pays a quadratic cost in the sequence length. More memory-efficient attention implementations can avoid storing the full attention matrix explicitly, without changing the mathematical attention map.

18.8 Connection to Transformer Blocks

A conventional transformer block uses multi-head attention together with other learned operations. Although the defining focus of this lecture is attention and positional encoding, it is useful to situate them within the larger computation.

Let $X \in \mathbb{R}^{n \times d}$ be a position-aware input. A common pre-normalization transformer block has the schematic form

$$\begin{aligned} Z &= X + \mathrm{MultiHead}(\mathrm{LN}(X)), \\ Y &= Z + \mathrm{FFN}(\mathrm{LN}(Z)), \end{aligned}$$

where LN denotes layer normalization and the position-wise feedforward network may be of the form

$$\mathrm{FFN}(z) = W_2 \phi(W_1 z + b_1) + b_2.$$

The feedforward map is applied independently to each token row with shared parameters. The attention operation mixes information *between* positions, while the feedforward operation transforms the feature coordinates *within* each position. Residual connections provide direct paths for both forward information and backward gradients.

Remark 18.9 (What attention does and does not guarantee). An attention head produces a data-dependent weighted average of projected values. This facilitates interactions among distant positions, but it does not guarantee that the learned weights correspond to a human-interpretable notion of explanation. The architecture's expressive power arises from the combined effect of multiple heads, learned projections, nonlinear feedforward transformations, positional representations, and training.

18.9 Exercises and Solutions

Exercise 1.

Consider a sequence of $n = 12$ tokens with embedding width $d = 64$. A single attention head uses $d_k = 16$ and $d_v = 20$. Determine the shapes of the input, projection matrices, query/key/value matrices, score matrix, attention matrix, and output. Find the number of projection weights before any output projection.

Solution. The shapes follow from the definitions:

$$\begin{aligned} X &\in \mathbb{R}^{12\times 64}, \\ W^{(Q)}, W^{(K)} &\in \mathbb{R}^{64\times 16}, \\ W^{(V)} &\in \mathbb{R}^{64\times 20}, \\ Q, K &\in \mathbb{R}^{12\times 16}, \\ V &\in \mathbb{R}^{12\times 20}, \\ S = QK^\top/\sqrt{16} &\in \mathbb{R}^{12\times 12}, \\ A = \mathrm{softmax}_{\mathrm{row}}(S) &\in \mathbb{R}^{12\times 12}, \\ H = AV &\in \mathbb{R}^{12\times 20}. \end{aligned}$$

The projection weight count, without biases, is

$$64 \cdot 16 + 64 \cdot 16 + 64 \cdot 20 = 1024 + 1024 + 1280 = 3328.$$

The sequence length affects the dimensions of the activations but not the number of these learned projection parameters. ■

Exercise 2.

Take $n = 2$, $d = d_k = d_v = 1$, and input embeddings $x_1 = 1$, $x_2 = 0$. Set $W^{(Q)} = W^{(K)} = W^{(V)} = [1]$. Calculate Q, K, V, S, A, and H exactly. Explain why the attention weights are not identical in the two rows.

Solution. In column-vector notation for this one-dimensional example,

$$Q = K = V = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Because $d_k = 1$, there is no nontrivial scaling:

$$S = QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Applying softmax independently to each row gives

$$A = \begin{bmatrix} \frac{e}{e+1} & \frac{1}{e+1} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix}.$$

The output is

$$H = AV = \begin{bmatrix} \frac{e}{e+1} & \frac{1}{e+1} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{e}{e+1} \\ \frac{1}{2} \end{bmatrix}.$$

The first query equals 1 and scores key 1 more highly than key 2. The second query equals 0 and assigns the same score to both keys, producing equal weights. The separate row normalizations are essential to this result. ■

Exercise 3.

For a fixed query position i, consider a perturbed vector of scores $s_i + c\mathbf{1}_n$, where $c \in \mathbb{R}$ is constant. Prove that the attention weights do not change. Deduce a formula for computing softmax that avoids exponentiating a positive score larger than the maximum in its row.

Solution. For every source position j,

$$\frac{\exp(s_{ij} + c)}{\sum_{r=1}^{n} \exp(s_{ir} + c)} = \frac{e^c e^{s_{ij}}}{e^c \sum_{r=1}^{n} e^{s_{ir}}} = \frac{e^{s_{ij}}}{\sum_{r=1}^{n} e^{s_{ir}}}.$$

Thus the softmax weights are unchanged by adding a common scalar to the entire row. Choosing $c = -\max_r s_{ir}$ yields

$$a_{ij} = \frac{\exp(s_{ij} - \max_r s_{ir})}{\sum_{r=1}^{n} \exp(s_{ir} - \max_r s_{ir})}.$$

Every exponent is nonpositive, which eliminates overflow from exponentiating a large positive number. The denominator is at least 1 because at least one shifted score equals zero. ■

Exercise 4.

Let $d = 96$ and $H_0 = 6$ attention heads. Assume $d_k = d_v = 16$, $d_{\text{out}} = 96$, and no biases. Determine the output dimensions of each head, the concatenation dimension, and the total number of trainable projection weights. Does this number depend on sequence length n?

Solution. For an input $X \in \mathbb{R}^{n \times 96}$, each query and key matrix is $n \times 16$, each attention matrix is $n \times n$, and each head output has shape $n \times 16$. Concatenating six heads produces a matrix in $\mathbb{R}^{n \times 96}$, which is multiplied by $W^{(O)} \in \mathbb{R}^{96 \times 96}$. For each head, the three input projections have

$$96 \cdot 16 + 96 \cdot 16 + 96 \cdot 16 = 4608$$

weights, giving $6 \cdot 4608 = 27648$ across six heads. The output projection contributes $96^2 = 9216$ weights, for a total

$$27648 + 9216 = 36864 = 4 \cdot 96^2.$$

No term depends on n. However, the score matrices have $6n^2$ entries across the six heads, so the activation memory does depend on n. ■

Exercise 5.

Let $a = \mathrm{softmax}(s) \in \mathbb{R}^n$, and suppose a scalar loss L has upstream derivative $g_a = \nabla_a L$. Show that its score gradient is

$$\nabla_s L = a \odot \big(g_a - (a^\top g_a)\mathbf{1}_n\big).$$

Prove that the components of this gradient sum to zero. Interpret this property.

Solution. From the softmax Jacobian,

$$\begin{aligned}\nabla_s L &= \big(\mathrm{diag}(a) - aa^\top\big)g_a \\ &= \mathrm{diag}(a)g_a - a(a^\top g_a) \\ &= a \odot \big(g_a - (a^\top g_a)\mathbf{1}_n\big).\end{aligned}$$

Summing the components,

$$\begin{aligned}\mathbf{1}_n^\top \nabla_s L &= \mathbf{1}_n^\top \,\mathrm{diag}(a)g_a - \mathbf{1}_n^\top a(a^\top g_a) \\ &= a^\top g_a - (\mathbf{1}_n^\top a)(a^\top g_a) \\ &= a^\top g_a - a^\top g_a = 0.\end{aligned}$$

The vanishing sum reflects shift invariance: changing all the scores by the same small amount does not change the output probabilities or the loss. Therefore the directional derivative of L in the all-ones direction is zero. ■

Exercise 6.

For a causal attention mechanism with $n = 3$, let the unmasked score matrix be

$$S = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}.$$

Write the masked attention matrix explicitly and verify that every row sums to one.

Solution. At position 1, only source position 1 is permitted. At position 2, source positions 1 and 2 are permitted. At position 3, all three sources are permitted. Thus

$$A = \begin{bmatrix} 1 & 0 & 0 \\ \frac{e}{e+1} & \frac{1}{e+1} & 0 \\ \frac{e^2}{e^2+e+1} & \frac{e}{e^2+e+1} & \frac{1}{e^2+e+1} \end{bmatrix}.$$

The first row sums to 1; the second sums to $(e+1)/(e+1) = 1$; and the third sums to $(e^2+e+1)/(e^2+e+1) = 1$. All disallowed future entries are exactly zero. Therefore an output at position i can depend only on values from positions $1, \ldots, i$. ■

19 Lecture 20: Normalization and the Transformer Encoder–Decoder

In Lecture 19, we introduced scaled dot-product attention, learned query–key–value projections, positional encodings, and multi-head attention. Attention by itself is not the entire Transformer architecture. A practical Transformer alternates attention with nonlinear feedforward transformations, combines each transformation with a residual connection, and normalizes intermediate representations. In the encoder–decoder setting, separate attention mechanisms process the source sequence, the partially generated target sequence, and the interactions between them.

This lecture first develops *batch normalization* and *layer normalization*, emphasizing the axes along which statistics are computed. We then assemble the residual, normalization, attention, and feedforward operations into Transformer encoder and decoder blocks. Finally, we derive the conditional language-model objective and explain the difference between parallel training and autoregressive generation.

19.1 Why Normalize Hidden Activations?

Let a hidden layer produce an activation vector $h \in \mathbb{R}^H$. The coordinates of h can have widely different magnitudes and can change substantially as the parameters in preceding layers are updated. This can make optimization sensitive to learning rates and parameter initialization. *Normalization* rescales activations using statistics computed from a prescribed collection of coordinates or observations.

A generic centering-and-scaling operation is

$$\widehat{h}_i = \frac{h_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \varepsilon > 0,$$

where μ and σ^2 are a mean and variance associated with the chosen normalization group. The small positive constant ε prevents division by zero and improves numerical stability.

Remark 19.1 (Normalization is not automatically an information-preserving map). Subtracting a mean removes a common offset, and dividing by a scale removes some magnitude information. The effect depends on which coordinates share the statistics, whether trainable scale and offset parameters are used, and whether residual paths preserve the original representation. Normalization can improve optimization behavior, but it is not a mathematical guarantee of faster convergence or better generalization.

19.2 Batch Normalization

19.2.1 Mini-Batch Statistics

Consider a mini-batch containing B observations. Suppose a neural-network layer outputs H coordinates per observation. Collect the hidden activations into the matrix

$$H_{\text{batch}} = \begin{bmatrix} (h^{(1)})^\top \\ \vdots \\ (h^{(B)})^\top \end{bmatrix} \in \mathbb{R}^{B \times H}, \qquad h^{(b)} = (h_1^{(b)}, \dots, h_H^{(b)})^\top.$$

We distinguish the batch size B from the feature dimension H.

Definition 19.1 (Batch normalization)**.** For every feature $j \in \{1, \dots, H\}$, define the *mini-batch mean* and *mini-batch variance* by

$$\mu_j = \frac{1}{B} \sum_{b=1}^{B} h_j^{(b)},$$
$$\sigma_j^2 = \frac{1}{B} \sum_{b=1}^{B} \big(h_j^{(b)} - \mu_j\big)^2.$$

The standardized activations are

$$\widehat{h}_j^{(b)} = \frac{h_j^{(b)} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}.$$

The common affine extension uses trainable parameters $\gamma_j, \beta_j \in \mathbb{R}$:

$$y_j^{(b)} = \gamma_j \widehat{h}_j^{(b)} + \beta_j.$$

The same μ_j, σ_j^2, γ_j, and β_j apply to all B observations in a given batch, but the statistics and affine parameters may differ across features.

Thus batch normalization fixes a feature index j and computes its statistics *across observations*. The arithmetic means and variances in the notes correspond to the first two expressions above. The additional affine transformation allows the layer to learn a useful scale and offset after standardization.

19.2.2 Mean and Variance After Standardization

Proposition 19.1 (Basic identities for batch normalization)**.** *For a fixed feature j, the standardized mini-batch has exactly zero empirical mean:*

$$\frac{1}{B} \sum_{b=1}^{B} \widehat{h}_j^{(b)} = 0.$$

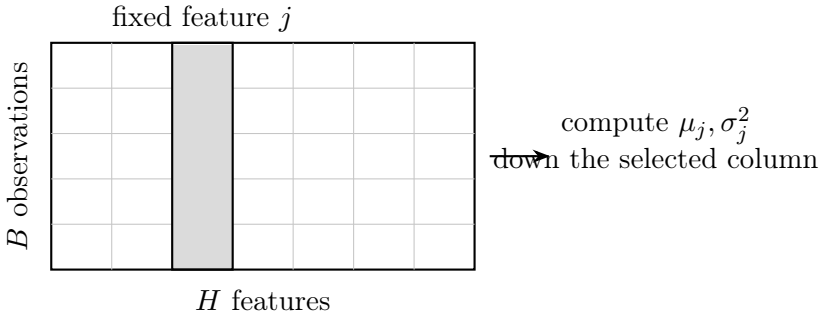


Figure 49: Batch normalization treats each feature column separately and uses observations in the mini-batch to estimate its mean and variance.

Its empirical second moment is

$$\frac{1}{B}\sum_{b=1}^{B}\big(\widehat{h}_j^{(b)}\big)^2 = \frac{\sigma_j^2}{\sigma_j^2+\varepsilon}.$$

In particular, when $\varepsilon > 0$ the empirical variance is generally not exactly one.

Proof. For the mean, by the definition of μ_j,

$$\begin{aligned}\frac{1}{B}\sum_{b=1}^{B}\widehat{h}_j^{(b)} &= \frac{1}{B\sqrt{\sigma_j^2+\varepsilon}}\sum_{b=1}^{B}(h_j^{(b)}-\mu_j)\\ &= \frac{\sum_{b=1}^{B}h_j^{(b)} - B\mu_j}{B\sqrt{\sigma_j^2+\varepsilon}} = 0.\end{aligned}$$

For the second moment,

$$\begin{aligned}\frac{1}{B}\sum_{b=1}^{B}(\widehat{h}_j^{(b)})^2 &= \frac{1}{B(\sigma_j^2+\varepsilon)}\sum_{b=1}^{B}(h_j^{(b)}-\mu_j)^2\\ &= \frac{\sigma_j^2}{\sigma_j^2+\varepsilon}.\end{aligned}$$

Since the standardized mean is zero, the second moment equals its empirical variance. □

Example 19.1 (Numerical mini-batch)**.** Suppose one feature takes values $(2,4,6,8)$ across a batch of size $B=4$. Then

$$\begin{aligned}\mu &= \frac{2+4+6+8}{4} = 5,\\ \sigma^2 &= \frac{(-3)^2+(-1)^2+1^2+3^2}{4} = 5.\end{aligned}$$

The standardized vector is

$$\widehat{h} = \frac{1}{\sqrt{5+\varepsilon}}(-3,-1,1,3)^\top.$$

If ε is negligible, the normalized values have mean zero and variance approximately one. They can then be rescaled and shifted using γ and β.

19.2.3 Training and Inference are Different

During training, a batch-normalization layer uses the statistics of the current mini-batch. Consequently, the normalized representation of one sample may change when the other samples in the same batch change, even if that sample itself is unchanged. At inference time, many implementations instead use running estimates of the training-population feature means and variances, so an individual prediction does not depend on the other examples in a test batch.

One common exponential-moving-average update is

$$\bar{\mu}_j^{(t)} = (1-\rho)\bar{\mu}_j^{(t-1)} + \rho\mu_j^{(t)},$$
$$\bar{v}_j^{(t)} = (1-\rho)\bar{v}_j^{(t-1)} + \rho\sigma_j^{2,(t)},$$

where $0 < \rho \leq 1$ is a momentum convention for the running statistics. The inference transformation is then

$$y_j = \gamma_j \frac{h_j - \bar{\mu}_j}{\sqrt{\bar{v}_j + \varepsilon}} + \beta_j.$$

Specific libraries may use different momentum or variance-estimator conventions; these do not change the distinction between training-batch statistics and stored inference statistics.

Remark 19.2 (Small-batch degeneracy). If $B = 1$, each feature mean is its only observed activation, so $\sigma_j^2 = 0$ and $\widehat{h}_j^{(1)} = 0$. The standardized output then loses the sample's information for that feature. More generally, very small mini-batches may give noisy feature statistics. These issues motivate normalization procedures that are independent of batch size, such as layer normalization.

19.3 Layer Normalization

19.3.1 Statistics Within Each Observation

Definition 19.2 (Layer normalization)**.** For an individual activation vector $h^{(b)} \in \mathbb{R}^H$, define the feature-wise mean and variance by

$$\mu^{(b)} = \frac{1}{H}\sum_{j=1}^{H} h_j^{(b)},$$
$$(\sigma^{(b)})^2 = \frac{1}{H}\sum_{j=1}^{H}\big(h_j^{(b)} - \mu^{(b)}\big)^2.$$

Its layer-normalized coordinates are

$$\widehat{h}_j^{(b)} = \frac{h_j^{(b)} - \mu^{(b)}}{\sqrt{(\sigma^{(b)})^2 + \varepsilon}}, \qquad j = 1, \ldots, H.$$

With learned affine parameters, the output is

$$y_j^{(b)} = \gamma_j \widehat{h}_j^{(b)} + \beta_j.$$

The key distinction is the axis of normalization: layer normalization fixes a *sample* and computes statistics across its *features*. It therefore does not require a batch of multiple samples. In Transformer models, layer normalization is ordinarily applied independently at each token position over the hidden feature dimension d_{model}.

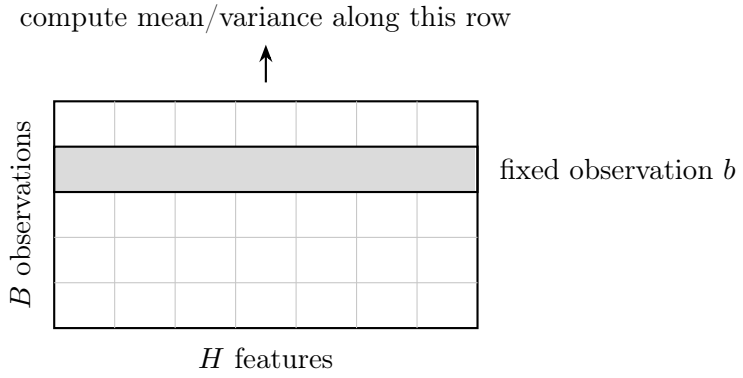


Figure 50: Layer normalization uses the coordinates within each observation rather than the observations within a batch.

Proposition 19.2 (Sample-wise centering)**.** *For every fixed sample b, layer normalization satisfies*

$$\sum_{j=1}^{H} \widehat{h}_j^{(b)} = 0, \qquad \frac{1}{H}\sum_{j=1}^{H} (\widehat{h}_j^{(b)})^2 = \frac{(\sigma^{(b)})^2}{(\sigma^{(b)})^2 + \varepsilon}.$$

Both identities are independent of the other samples in the mini-batch.

Proof. For fixed b, repeat the calculation for batch normalization with the feature index j as the summation index and H in place of B. Since $H\mu^{(b)} = \sum_j h_j^{(b)}$, the centered coordinates sum to zero. The variance identity follows by substituting the definition of $(\sigma^{(b)})^2$. □

Example 19.2 (Equal preactivations)**.** Suppose a layer contains two hidden units with outputs

$$h_1 = w_{11}x_1 + w_{12}x_2 + b_1, \qquad h_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

If $h_1 = h_2 = 5$ for a particular observation, then $\mu = 5$ and $\sigma^2 = 0$. Thus the normalized vector is $(0, 0)^\top$ and the affine output is $(\beta_1, \beta_2)^\top$. This simple example illustrates that layer normalization is performed within an observation, not across examples. If the hidden units have unequal magnitudes, normalization makes their relative deviations more comparable. It does not automatically cause the units' weights to learn at identical rates.

19.3.2 Comparing the Two Normalizations

	Batch normalization	Layer normalization
Statistics over	different samples, fixed feature	different features, fixed sample
Training depends on batch composition	yes	no
Typical inference convention	stored running feature statistics	current sample's features
Works with one sample	degenerate standardization	yes, if feature variance is nonzero
Common application	feedforward/convolutional layers	Transformers and sequence models

Table 1: The essential distinction is which axis supplies the mean and variance.

Remark 19.3 (Scale and translation invariance). When $\varepsilon = 0$ and the variance is positive, standardization is exactly invariant to adding a constant to every member of its normalization group and to multiplying every member by the same positive constant. The added ε preserves exact translation invariance but makes positive-scale invariance approximate rather than exact. These statements concern the pre-affine standardized values; learned γ and β then change the output scale and location.

19.4 Backpropagation Through Normalization

Because the mean and variance themselves depend on the inputs, a normalization layer does more than divide each component by an independent constant. Its backward pass must differentiate through both statistics. We derive the generic formula once and apply it to either normalization method.

Proposition 19.3 (Backward derivative of a normalization group)**.** *Let $x = (x_1, \dots, x_M)^\top \in \mathbb{R}^M$ be any normalization group. Define*

$$\begin{aligned} \mu &= \frac{1}{M}\sum_{i=1}^{M} x_i, \\ \sigma^2 &= \frac{1}{M}\sum_{i=1}^{M} (x_i - \mu)^2, \\ s &= \sqrt{\sigma^2 + \varepsilon}, \\ \widehat{x}_i &= \frac{x_i - \mu}{s}, \\ y_i &= \gamma \widehat{x}_i + \beta. \end{aligned}$$

Suppose a scalar differentiable loss L has upstream derivatives $g_i = \partial L / \partial y_i$. Then

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^{M} g_i, \qquad \frac{\partial L}{\partial \gamma} = \sum_{i=1}^{M} g_i \widehat{x}_i,$$

and for every k,

$$\frac{\partial L}{\partial x_k} = \frac{\gamma}{Ms}\left(Mg_k - \sum_{i=1}^{M} g_i - \widehat{x}_k \sum_{i=1}^{M} g_i \widehat{x}_i\right).$$

For batch normalization, take $M = B$ and fix a feature. For layer normalization, take $M = H$ and fix a sample; with feature-specific γ_j, first set $a_j = \gamma_j g_j$ and apply the same centered backward formula to a_j instead of γg_j.

Proof. The affine derivatives follow from $y_i = \gamma \widehat{x}_i + \beta$. To derive the input gradient, first note that

$$\begin{aligned}
\frac{\partial \mu}{\partial x_k} &= \frac{1}{M}, \\
\frac{\partial \sigma^2}{\partial x_k} &= \frac{2}{M}\sum_{i=1}^{M}(x_i - \mu)\left(\delta_{ik} - \frac{1}{M}\right) \\
&= \frac{2}{M}(x_k - \mu) - \frac{2}{M^2}\sum_{i=1}^{M}(x_i - \mu) \\
&= \frac{2}{M}(x_k - \mu),
\end{aligned}$$

where $\sum_i (x_i - \mu) = 0$. Since $s = (\sigma^2 + \varepsilon)^{1/2}$,

$$\frac{\partial s}{\partial x_k} = \frac{1}{2s}\frac{\partial \sigma^2}{\partial x_k} = \frac{x_k - \mu}{Ms}.$$

Differentiating $\widehat{x}_i = (x_i - \mu)s^{-1}$ gives

$$\begin{aligned}
\frac{\partial \widehat{x}_i}{\partial x_k} &= \frac{1}{s}\left(\delta_{ik} - \frac{1}{M}\right) - (x_i - \mu)\frac{1}{s^2}\frac{\partial s}{\partial x_k} \\
&= \frac{1}{s}\left(\delta_{ik} - \frac{1}{M}\right) - \frac{(x_i - \mu)(x_k - \mu)}{Ms^3} \\
&= \frac{1}{s}\left(\delta_{ik} - \frac{1}{M} - \frac{\widehat{x}_i \widehat{x}_k}{M}\right).
\end{aligned}$$

Therefore the chain rule gives

$$\begin{aligned}
\frac{\partial L}{\partial x_k} &= \sum_{i=1}^{M} \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial \widehat{x}_i}\frac{\partial \widehat{x}_i}{\partial x_k} \\
&= \frac{\gamma}{s}\sum_{i=1}^{M} g_i\left(\delta_{ik} - \frac{1}{M} - \frac{\widehat{x}_i \widehat{x}_k}{M}\right) \\
&= \frac{\gamma}{Ms}\left(Mg_k - \sum_i g_i - \widehat{x}_k \sum_i g_i \widehat{x}_i\right).
\end{aligned}$$

For feature-specific affine scales, simply replace the upstream derivative γg_i by $a_i = \gamma_i g_i$ before summing over the normalized coordinates. □

Remark 19.4 (Gradient coupling). Every input derivative contains the sums of upstream derivatives across the normalization group. Thus the gradient of one input coordinate depends on all the others in that group. Batch normalization couples examples within a batch, while layer normalization couples features within each example.

19.5 Residual Connections and Add & Norm

Recall that a Transformer attention head produces a contextualized output, and multi-head attention combines several such heads. A Transformer sublayer does not ordinarily replace its input by this output alone. Instead it uses a *residual connection* to add the sublayer input back to its transformed output.

Definition 19.3 (Residual transformation)**.** If $F : \mathbb{R}^d \to \mathbb{R}^d$ is a differentiable transformation, its residual counterpart is

$$R(x) = x + F(x).$$

This addition requires the two vectors to have the same dimension. If they do not, a learned linear projection or another dimension-matching operation is needed.

Proposition 19.4 (Jacobian of a residual map)**.** *For a differentiable $F : \mathbb{R}^d \to \mathbb{R}^d$,*

$$DR(x) = I_d + DF(x).$$

Thus, for a scalar loss L with upstream gradient $g = \nabla_R L$,

$$\nabla_x L = g + (DF(x))^\top g.$$

Proof. Differentiating $R(x) = x + F(x)$ gives $\mathrm{d}R = \mathrm{d}x + DF(x)\mathrm{d}x = (I_d + DF(x))\mathrm{d}x$. The gradient formula follows from the vector chain rule. □

The direct g term represents a gradient pathway that does not multiply through $DF(x)$. However, its presence does not imply gradients can never vanish or explode, because the terms may interact and the full network includes additional operations such as normalization.

19.5.1 Post-Norm and Pre-Norm Conventions

The handwritten lecture uses *Add & Norm* to refer to a residual addition followed by layer normalization. This is often called the *post-norm* convention:

$$y = \mathrm{LN}\big(x + F(x)\big).$$

Another common convention is *pre-norm*, which normalizes before the sublayer and adds afterward:

$$y = x + F\big(\mathrm{LN}(x)\big).$$

These two forms are mathematically distinct. For post-norm, the Jacobian is

$$Dy(x) = D\,\mathrm{LN}(x + F(x))\big(I + DF(x)\big),$$

whereas for pre-norm,

$$Dy(x) = I + DF(\mathrm{LN}(x))D\,\mathrm{LN}(x).$$

Consequently, the explicit identity pathway appears outside the normalized transformation only in the latter expression. Both arrangements are used in practice.

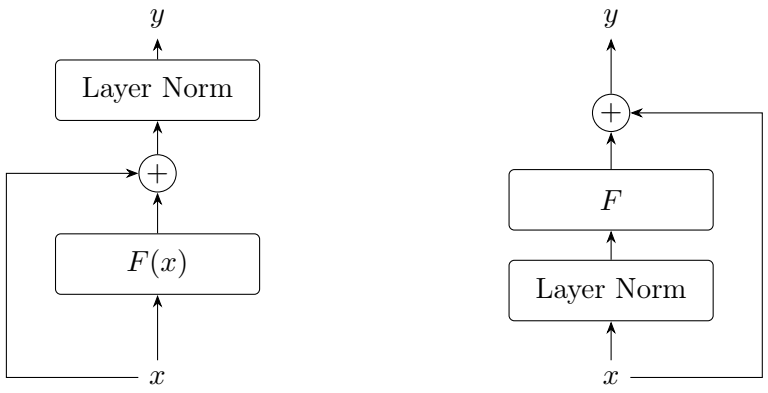


Figure 51: Two standard ways of combining layer normalization with a residual sublayer. The post-norm form matches the "Add & Norm" expression in the handwritten notes.

19.6 Position-Wise Feedforward Networks

After attention combines information across token positions, a *position-wise feedforward network* applies the same learned nonlinear transformation independently to every token representation.

Definition 19.4 (Transformer feedforward sublayer)**.** For token vector $x \in \mathbb{R}^d$, the two-layer feedforward map is

$$F(x) = W_2\phi(W_1x + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\mathrm{ff}}\times d}$, $b_1 \in \mathbb{R}^{d_{\mathrm{ff}}}$, $W_2 \in \mathbb{R}^{d\times d_{\mathrm{ff}}}$, $b_2 \in \mathbb{R}^d$, and ϕ is an elementwise nonlinearity such as ReLU or GELU. The intermediate dimension d_{ff} can be larger than d.

If the input sequence is represented by $X \in \mathbb{R}^{n\times d}$, use row-oriented matrices $\widetilde{W}_1 = W_1^\top \in \mathbb{R}^{d\times d_{\mathrm{ff}}}$ and $\widetilde{W}_2 = W_2^\top \in \mathbb{R}^{d_{\mathrm{ff}}\times d}$. The same transformation applied to all positions becomes

$$\mathrm{FFN}(X) = \phi\big(X\widetilde{W}_1 + \mathbf{1}_n b_1^\top\big)\widetilde{W}_2 + \mathbf{1}_n b_2^\top.$$

No summation over token positions occurs in this formula. Information from other positions enters a token's representation through attention, not through this feedforward map itself.

Proposition 19.5 (Permutation equivariance of position-wise feedforward layers)**.** *Let $P \in \mathbb{R}^{n\times n}$ be a permutation matrix acting on token positions. Then*

$$\mathrm{FFN}(PX) = P\,\mathrm{FFN}(X).$$

Proof. Since $P\mathbf{1}_n = \mathbf{1}_n$ and ϕ acts independently on entries,

$$\begin{aligned}
\mathrm{FFN}(PX) &= \phi\big(PX\widetilde{W}_1 + \mathbf{1}_n b_1^\top\big)\widetilde{W}_2 + \mathbf{1}_n b_2^\top \\
&= \phi\big(P(X\widetilde{W}_1 + \mathbf{1}_n b_1^\top)\big)\widetilde{W}_2 + P\mathbf{1}_n b_2^\top \\
&= P\phi(X\widetilde{W}_1 + \mathbf{1}_n b_1^\top)\widetilde{W}_2 + P\mathbf{1}_n b_2^\top \\
&= P\,\mathrm{FFN}(X).
\end{aligned}$$

□

Remark 19.5 (Parameter count). Ignoring the attention layers, the feedforward sublayer contains

$$d_{\mathrm{ff}}d + d_{\mathrm{ff}} + dd_{\mathrm{ff}} + d = 2dd_{\mathrm{ff}} + d_{\mathrm{ff}} + d$$

trainable weights and biases. This parameter count is independent of the sequence length n, because the same map is shared across token positions.

19.7 Transformer Encoder Architecture

19.7.1 Self-Attention and Residual Processing

Let the source sequence contain S tokens with embeddings $X_{\mathrm{src}} \in \mathbb{R}^{S\times d}$. Add positional information $P_{\mathrm{src}} \in \mathbb{R}^{S\times d}$, introduced in Lecture 19, and define

$$H^{(0)} = X_{\mathrm{src}} + P_{\mathrm{src}}.$$

An encoder layer computes attention among the source tokens, then applies a position-wise feedforward transformation. In the post-norm convention used in the notes,

$$\begin{aligned}
A^{(\ell)} &= \mathrm{LN}_1^{(\ell)}\left(H^{(\ell-1)} + \mathrm{MHA}_{\mathrm{self}}^{(\ell)}(H^{(\ell-1)})\right), \\
H^{(\ell)} &= \mathrm{LN}_2^{(\ell)}\left(A^{(\ell)} + \mathrm{FFN}^{(\ell)}(A^{(\ell)})\right).
\end{aligned}$$

Here $\mathrm{MHA}_{\mathrm{self}}$ constructs its queries, keys, and values from the same source representation. Its output has shape $S \times d$, so residual addition is defined.

After L_{enc} encoder layers, write

$$E = H^{(L_{\mathrm{enc}})} \in \mathbb{R}^{S\times d}.$$

This matrix is a context-dependent representation of the entire source sequence. In an encoder–decoder Transformer, the decoder will use E to compute its *cross-attention* keys and values.

Remark 19.6 (Padding masks). If the input sequences are padded to a common length, attention scores associated with padded source positions must be masked so that they receive no probability mass. This is separate from the causal mask used by the decoder: an encoder generally allows each real source token to attend to other real source tokens in both directions.

19.8 Transformer Decoder Architecture

The decoder predicts target tokens using both preceding target tokens and the encoder's representation of the source. Its layer has three principal transformations: masked self-attention, encoder–decoder cross-attention, and a position-wise feedforward map. Each transformation is accompanied by residual and normalization operations.

19.8.1 Masked Self-Attention

Let the target sequence contain T positions and let $Y^{(0)} \in \mathbb{R}^{T\times d}$ denote the embeddings of the shifted decoder input tokens, augmented with positional encodings. In an autoregressive decoder, target position t is not permitted to examine positions $j > t$.

For query/key matrices $Q, K \in \mathbb{R}^{T\times d_k}$, let $M \in (\mathbb{R} \cup \{-\infty\})^{T\times T}$ be the causal mask

$$M_{tj} = \begin{cases} 0, & j \le t, \\ -\infty, & j > t. \end{cases}$$

The attention weights are

$$A_{\text{causal}} = \operatorname{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right).$$

Thus $(A_{\text{causal}})_{tj} = 0$ for all $j > t$. A masked attention head can use only the target prefix available at its query position.

Proposition 19.6 (No dependence on future target tokens)**.** *Suppose every target-side attention sublayer uses a causal mask and every other target-side operation is applied independently at each token position. Then the representation at target position t after any finite number of decoder layers depends only on target inputs at positions $1, \dots, t$ and on the encoder output, not on any future target input at position $j > t$.*

Proof. At the initial layer, the vector at position t depends only on the target token and positional encoding at t. Assume the claim holds for every vector at some intermediate layer. Causal attention at position t combines only vectors at positions $j \le t$, each of which, by the induction hypothesis, depends only on target inputs at positions $1, \dots, j \subseteq \{1, \dots, t\}$. A position-wise feedforward map or layer normalization across features does not mix target positions. A residual sum also cannot introduce future dependence. Cross-attention depends additionally on encoder outputs, which are independent of future decoder inputs. Hence the claim remains true after the next layer, and induction completes the proof. □

19.8.2 Encoder–Decoder Cross-Attention

In *cross-attention*, the queries arise from the decoder's current hidden representation, while the keys and values arise from the encoder output.

Definition 19.5 (Cross-attention)**.** Let $D \in \mathbb{R}^{T\times d}$ denote decoder representations and $E \in \mathbb{R}^{S\times d}$ encoder representations. A single cross-attention head uses

$$\begin{aligned} Q &= DW^{(Q)} \in \mathbb{R}^{T\times d_k}, \\ K &= EW^{(K)} \in \mathbb{R}^{S\times d_k}, \\ V &= EW^{(V)} \in \mathbb{R}^{S\times d_v}, \end{aligned}$$

with output

$$\mathrm{CrossAttn}(D,E) = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{T\times d_v}.$$

Padding masks may restrict which encoder positions are visible. Cross-attention itself does not require a causal mask over source positions.

Remark 19.7 (The roles of queries, keys, and values). In decoder masked self-attention, Q, K, and V are derived from target-side hidden vectors. In decoder cross-attention, Q comes from the target side and K, V come from the *encoder*. This distinction is the key architectural point of the source-to-target attention drawing in the handwritten notes.

Example 19.3 (Matrix dimensions)**.** Suppose there are $S = 5$ source tokens and $T = 3$ decoder positions, with $d_k = 8$ and $d_v = 6$. Then

$$\begin{aligned} Q &\in \mathbb{R}^{3\times 8}, & K &\in \mathbb{R}^{5\times 8}, & V &\in \mathbb{R}^{5\times 6}, \\ QK^\top &\in \mathbb{R}^{3\times 5}, & & & AV &\in \mathbb{R}^{3\times 6}. \end{aligned}$$

The score matrix has one row for each decoder query and one column for each source position. Its tth row describes which source tokens the tth target position attends to.

19.8.3 Complete Decoder Layer

Let $D^{(\ell-1)} \in \mathbb{R}^{T\times d}$ denote the input to decoder layer ℓ and $E \in \mathbb{R}^{S\times d}$ the final encoder memory. A post-norm decoder layer can be written as

$$\begin{aligned} U^{(\ell)} &= \mathrm{LN}_1^{(\ell)}\left(D^{(\ell-1)} + \mathrm{MHA}^{(\ell)}_{\mathrm{masked}}(D^{(\ell-1)})\right), \\ C^{(\ell)} &= \mathrm{LN}_2^{(\ell)}\left(U^{(\ell)} + \mathrm{MHA}^{(\ell)}_{\mathrm{cross}}(U^{(\ell)}, E)\right), \\ D^{(\ell)} &= \mathrm{LN}_3^{(\ell)}\left(C^{(\ell)} + \mathrm{FFN}^{(\ell)}(C^{(\ell)})\right). \end{aligned}$$

Each map preserves the token count T and the model dimension d, even though cross-attention accesses S source positions.

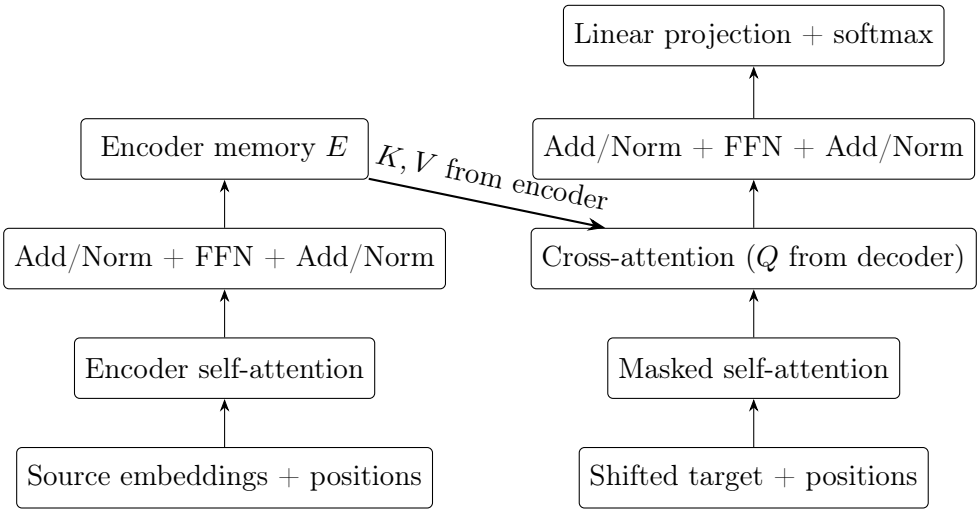


Figure 52: Conceptual encoder–decoder Transformer. For legibility, repeated encoder and decoder layers and individual residual paths are suppressed; every attention and feedforward sublayer is accompanied by residual and normalization operations.

19.9 From Decoder Hidden States to Token Probabilities

After the decoder's last layer, the hidden representation at target position t is $d_t \in \mathbb{R}^d$. Let the target vocabulary have size V_{vocab} and let $W_o \in \mathbb{R}^{V_{\text{vocab}} \times d}$ and $b_o \in \mathbb{R}^{V_{\text{vocab}}}$ be the output projection parameters. The logits are

$$z_t = W_o d_t + b_o \in \mathbb{R}^{V_{\text{vocab}}}.$$

Applying softmax gives a categorical distribution over the vocabulary:

$$p_\theta(y_t = k \mid y_{<t}, x) = \frac{e^{(z_t)_k}}{\sum_{r=1}^{V_{\text{vocab}}} e^{(z_t)_r}}.$$

Here $x = (x_1, \ldots, x_S)$ denotes the source sequence and $y_{<t}$ the previously generated target prefix. The parameters θ include embedding tables, attention projections, feedforward weights, normalization parameters, and output projections.

19.9.1 Autoregressive Likelihood

Proposition 19.7 (Chain-rule factorization)**.** *Every conditional distribution on a finite target sequence $y = (y_1, \ldots, y_T)$ admits the factorization*

$$p_\theta(y_1, \ldots, y_T \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x),$$

provided the conditionals are interpreted on prefixes with positive probability. Accordingly, the negative log-likelihood is

$$-\log p_\theta(y \mid x) = -\sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x).$$

Proof. For any events with positive conditioning probabilities, the product rule gives

$$p(y_1, y_2 \mid x) = p(y_1 \mid x)p(y_2 \mid y_1, x).$$

Applying it repeatedly yields

$$\begin{aligned} p(y_1, \dots, y_T \mid x) &= p(y_1, \dots, y_{T-1} \mid x)p(y_T \mid y_1, \dots, y_{T-1}, x) \\ &= \cdots \\ &= \prod_{t=1}^{T} p(y_t \mid y_{<t}, x). \end{aligned}$$

Taking logarithms converts the product into a sum; multiplying by -1 gives the objective. □

The next-token predictive distribution is learned by *maximum likelihood*, equivalently by minimizing this negative log-likelihood on the training data.

Definition 19.6 (Empirical sequence cross-entropy)**.** Let $\mathcal{D} = \{(x^{(n)}, y^{(n)})\}_{n=1}^{N}$ be N source–target sequence pairs, and let T_n denote the target length for sample n. One common token-averaged empirical loss is

$$\widehat{\mathcal{L}}(\theta) = -\frac{1}{\sum_{n=1}^{N} T_n} \sum_{n=1}^{N} \sum_{t=1}^{T_n} \log p_\theta\big(y_t^{(n)} \mid y_{<t}^{(n)}, x^{(n)}\big).$$

Another convention averages sequence losses instead of individual token losses. The choice affects example weighting but not the underlying probabilistic factorization.

At training time, the decoder input is typically the *shifted target sequence*: a beginning-of-sequence token followed by the previous ground-truth target tokens. This is called *teacher forcing*. Causal masking prevents a target position from reading future target tokens, even though the entire shifted input matrix is processed in parallel.

Example 19.4 (Shifted training sequence)**.** Suppose the desired target output is $(\texttt{A}, \texttt{B}, \texttt{EOS})$. The decoder inputs are $(\texttt{BOS}, \texttt{A}, \texttt{B})$, and the three output positions are trained to predict $(\texttt{A}, \texttt{B}, \texttt{EOS})$. The loss is

$$\begin{aligned} \ell(\theta) = \; & -\log p_\theta(\texttt{A} \mid \texttt{BOS}, x) \\ & -\log p_\theta(\texttt{B} \mid \texttt{BOS}, \texttt{A}, x) \\ & -\log p_\theta(\texttt{EOS} \mid \texttt{BOS}, \texttt{A}, \texttt{B}, x). \end{aligned}$$

The first prediction uses no generated target content, the second uses the first target token, and the third uses the first two target tokens.

19.9.2 Softmax–Cross-Entropy Gradient

Proposition 19.8 (Derivative with respect to output logits)**.** *Let $p = \mathrm{softmax}(z)$ and let c be the correct token index. For the token negative log-likelihood $\ell(z) = -\log p_c$,*

$$\frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}_{\{k=c\}}.$$

Proof. Since $p_c = e^{z_c} / \sum_r e^{z_r}$,

$$\ell(z) = -z_c + \log\left(\sum_{r=1}^{V_{\text{vocab}}} e^{z_r}\right),$$
$$\frac{\partial \ell}{\partial z_k} = -\delta_{ck} + \frac{e^{z_k}}{\sum_r e^{z_r}} = p_k - \delta_{ck}.$$

□

Let $e_c \in \mathbb{R}^{V_{\text{vocab}}}$ be the one-hot vector for the correct token. Writing $\delta = p - e_c$ and $z = W_o d + b_o$, the chain rule gives

$$\nabla_z \ell = \delta,$$
$$\nabla_{W_o} \ell = \delta d^\top,$$
$$\nabla_{b_o} \ell = \delta,$$
$$\nabla_d \ell = W_o^\top \delta.$$

These derivatives are propagated through the decoder sublayers by the reverse-mode differentiation rules studied in Lectures 6–8 and 19.

19.10 Parallel Training and Sequential Generation

An RNN computes its hidden states through the recurrence $h_t = F(h_{t-1}, x_t)$, forcing a dependency chain across time. In contrast, a Transformer self-attention layer computes all query–key scores and weighted sums through batched matrix operations. This allows substantial *parallelism across sequence positions* during training, because the ground-truth target prefixes are known in advance and causal masks enforce valid information flow.

Nevertheless, autoregressive *generation* remains sequential at the token level: before sampling y_{t+1}, the model must produce y_t, because the next distribution is conditional on the already generated prefix. Cached keys and values can reduce repeated attention computation at inference time, but they do not remove this logical dependence.

Remark 19.8 (Complexity). For sequence length n and hidden dimension d, standard dense self-attention computes an $n \times n$ similarity matrix, creating a quadratic dependence on n in its attention scores. A position-wise feedforward layer has cost proportional to n for fixed dimensions. The ability to parallelize training across positions is therefore distinct from having linear computational complexity in sequence length.

19.11 Worked Transformer Block Example

Consider a single token representation $x = (2, 0)^\top \in \mathbb{R}^2$ and a simple feedforward residual sublayer

$$F(x) = W_2\phi(W_1x), \qquad W_1 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}, \qquad W_2 = I_2,$$

with $\phi = \mathrm{ReLU}$. Then

$$\begin{aligned} W_1x &= \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \\ \phi(W_1x) &= \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \\ F(x) &= \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \\ x + F(x) &= \begin{bmatrix} 4 \\ 0 \end{bmatrix}. \end{aligned}$$

For layer normalization without learned affine parameters, the mean and variance across these two features are

$$\begin{aligned} \mu &= \frac{4+0}{2} = 2, \\ \sigma^2 &= \frac{(4-2)^2 + (0-2)^2}{2} = 4. \end{aligned}$$

Therefore the post-norm output is

$$y = \frac{1}{\sqrt{4+\varepsilon}} \begin{bmatrix} 2 \\ -2 \end{bmatrix}.$$

For small ε, the output is approximately $(1, -1)^\top$. This illustrates the sequence *transform* $\rightarrow$ *residual addition* $\rightarrow$ *layer normalization* used in Add & Norm.

19.12 Summary of the Architecture

For clarity, the main computations can be arranged in the following order:

1. The source tokens are embedded and given positional information.
2. Each encoder layer applies source self-attention and a feedforward map, together with residual connections and layer normalization.
3. The decoder receives a shifted target prefix and applies causally masked self-attention.
4. Encoder–decoder cross-attention uses target-side queries with encoder-side keys and values.
5. A feedforward map and normalization finish each decoder layer.

6. The final decoder hidden states are projected to vocabulary logits and passed through softmax.
7. The model is trained by minimizing negative log-likelihood, with a sum of token-level cross-entropies over the target sequence.

The architectural purpose of normalization is to control the scale and centering of hidden features; the purpose of attention is to mix information across positions; and the purpose of the feedforward network is to perform nonlinear feature transformations independently at each position. Residual connections provide direct routes for information and gradients, while causal masking ensures that the decoder represents a valid autoregressive conditional model.

19.13 Exercises

Exercise 1.

Let a single feature have mini-batch activations $(1, 3, 5, 7)^\top$. Compute its batch-normalized values for $\varepsilon = 0$ and then for general $\varepsilon > 0$. Verify its empirical mean and variance.

Solution. The mean and variance are

$$\mu = \frac{1+3+5+7}{4} = 4,$$
$$\sigma^2 = \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{4} = 5.$$

Thus

$$\widehat{h} = \frac{1}{\sqrt{5+\varepsilon}}(-3, -1, 1, 3)^\top.$$

At $\varepsilon = 0$, this is $(-3, -1, 1, 3)^\top/\sqrt{5}$. Its empirical mean is $(-3-1+1+3)/(4\sqrt{5}) = 0$ and its variance is $(9+1+1+9)/(4 \cdot 5) = 1$. For $\varepsilon > 0$, the mean stays zero but the empirical variance becomes $5/(5+\varepsilon)$. ■

Exercise 2.

Let $X \in \mathbb{R}^{B \times H}$. Describe the statistics used by batch normalization and layer normalization, and explain what happens in either method when the number of elements in its normalization group is one.

Solution. Batch normalization fixes a feature j and averages x_{bj} over $b = 1, \ldots, B$. Layer normalization fixes a sample b and averages x_{bj} over $j = 1, \ldots, H$. A normalization group with one element has mean equal to that element and variance zero. Its standardized value is therefore $0/\sqrt{\varepsilon} = 0$, so the affine output is simply its learned bias. Batch normalization has this degeneracy when $B = 1$; layer normalization has the analogous degeneracy when the normalized feature group has size $H = 1$. ■

Exercise 3.

Prove that the Jacobian of the basic standardization map $x \mapsto \widehat{x}$ annihilates the all-ones direction. Explain why.

Solution. From the normalization derivative with $\gamma = 1$,

$$J = \frac{1}{s}\left(I_M - \frac{1}{M}\mathbf{1}\mathbf{1}^\top - \frac{1}{M}\widehat{x}\widehat{x}^\top\right).$$

Because $\mathbf{1}^\top\widehat{x} = 0$, it follows that

$$\begin{aligned} J\mathbf{1} &= \frac{1}{s}\left(\mathbf{1} - \frac{1}{M}\mathbf{1}(\mathbf{1}^\top\mathbf{1}) - \frac{1}{M}\widehat{x}(\widehat{x}^\top\mathbf{1})\right) \\ &= \frac{1}{s}(\mathbf{1} - \mathbf{1} - 0) = 0. \end{aligned}$$

This is expected: adding the same constant to every coordinate changes both x_i and the mean μ by that constant, leaving each centered difference $x_i - \mu$ unchanged. ■

Exercise 4.

In encoder–decoder cross-attention, let $D \in \mathbb{R}^{4\times 64}$, $E \in \mathbb{R}^{7\times 64}$, $d_k = 16$, and $d_v = 8$. Write the dimensions of Q, K, V, S, A, and the output AV. Explain which axis of A sums to one.

Solution. The decoder-side query matrix is $Q \in \mathbb{R}^{4\times 16}$, the source-side key matrix is $K \in \mathbb{R}^{7\times 16}$, and the value matrix is $V \in \mathbb{R}^{7\times 8}$. Hence

$$\begin{aligned} S &= QK^\top/\sqrt{16} \in \mathbb{R}^{4\times 7}, \\ A &= \mathrm{softmax}_{\mathrm{row}}(S) \in \mathbb{R}^{4\times 7}, \\ AV &\in \mathbb{R}^{4\times 8}. \end{aligned}$$

For each decoder query row t, $\sum_{j=1}^{7} A_{tj} = 1$. Each output token is therefore a convex combination of the seven source value vectors. ■

Exercise 5.

Let $y = \mathrm{LN}(x + F(x))$ for a differentiable sublayer F. Derive its Jacobian, and compare it with the pre-norm variant $\widetilde{y} = x + F(\mathrm{LN}(x))$.

Solution. For post-norm, apply the chain rule to $u(x) = x + F(x)$ and $y = \mathrm{LN}(u)$:

$$Dy(x) = D\,\mathrm{LN}(u(x))(I + DF(x)).$$

For pre-norm, $\widetilde{y} = x + F(\mathrm{LN}(x))$, so

$$D\widetilde{y}(x) = I + DF(\mathrm{LN}(x))D\,\mathrm{LN}(x).$$

Only the second Jacobian displays the identity term additively outside the normalization Jacobian. They therefore have different gradient-propagation properties even if the same attention and feedforward functions are used. ■

Exercise 6.

Suppose a target vocabulary has three words and the decoder produces logits $z = (\log 2, 0, 0)^\top$ at a position whose correct word is index 1. Compute its categorical probabilities, negative log-likelihood, and gradient with respect to z.

Solution. The exponentials of the logits are $(2, 1, 1)$, so

$$p = \left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)^\top.$$

The correct-word probability is $p_1 = 1/2$, giving loss

$$\ell = -\log \frac{1}{2} = \log 2.$$

Since $\nabla_z \ell = p - e_1$, the logit gradient is

$$\nabla_z \ell = \left(-\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)^\top.$$

Increasing the correct logit locally decreases the loss; increasing either incorrect logit increases it. ■

Exercise 7.

Prove that a position-wise feedforward map cannot, by itself, transmit information from a source token at position j to a different position $i \neq j$, even when its intermediate hidden dimension is large.

Solution. For $X \in \mathbb{R}^{n \times d}$, the output at position i is

$$F_i(X) = W_2 \phi(W_1 x_i + b_1) + b_2.$$

This formula does not contain any x_j with $j \neq i$. Consequently, where the derivative exists,

$$\frac{\partial F_i}{\partial x_j} = 0 \qquad (i \neq j).$$

An increase in the intermediate dimension d_{ff} changes the expressive capacity of the map of x_i but does not introduce dependence on any other token. Cross-position interactions must come from attention or another operation that mixes positions. ■

20 Lecture 21: Adversarial Attacks and Robustness of Neural Networks

20.1 Classification and the Geometry of Decision Boundaries

The preceding lectures developed neural networks as compositions of differentiable operations and showed how automatic differentiation makes it possible to optimize their parameters. An important consequence of differentiability is that the same derivatives used to *train* a classifier can also be used to identify small changes to an input that substantially alter its prediction. These changes are called *adversarial perturbations*.

Definition 20.1 (Supervised classification data)**.** Let $\mathcal{X} \subseteq \mathbb{R}^d$ be an input space and let $\mathcal{Y} = \{1, \dots, K\}$ be a collection of $K \geq 2$ class labels. A supervised training dataset is

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}, \qquad x^{(i)} \in \mathcal{X}, \quad y^{(i)} \in \mathcal{Y}.$$

A neural network with parameters θ produces a logit vector $z_\theta(x) = (z_{\theta,1}(x), \dots, z_{\theta,K}(x)) \in \mathbb{R}^K$. Its predicted class probabilities are

$$p_{\theta,k}(x) = \frac{\exp(z_{\theta,k}(x))}{\sum_{j=1}^{K} \exp(z_{\theta,j}(x))}, \qquad k = 1, \dots, K.$$

The classifier is $f_\theta(x) = \arg\max_{1 \leq k \leq K} p_{\theta,k}(x)$, with a fixed tie-breaking convention.

Since softmax is strictly increasing in each logit with all other logits held fixed, its ordering coincides with the ordering of the logits:

$$f_\theta(x) = \arg\max_k z_{\theta,k}(x).$$

For a fixed class c, its classification region is $R_c = \{x \in \mathcal{X} : f_\theta(x) = c\}$. A *decision boundary* is a location where the identity of the maximizing class can change. In a binary classifier, a common representation is

$$f_\theta(x) = \begin{cases} 1, & s_\theta(x) \geq 0, \\ 0, & s_\theta(x) < 0, \end{cases}$$

where $s_\theta : \mathcal{X} \to \mathbb{R}$ is a score function, and the corresponding boundary is $\{x : s_\theta(x) = 0\}$.

Definition 20.2 (Cross-entropy classification loss)**.** For a single observation (x, y), the cross-entropy loss is

$$\ell(\theta; x, y) = -\log p_{\theta,y}(x) = -z_{\theta,y}(x) + \log\left(\sum_{k=1}^{K} e^{z_{\theta,k}(x)}\right).$$

The empirical training loss is $\widehat{L}_N(\theta) = N^{-1} \sum_{i=1}^{N} \ell(\theta; x^{(i)}, y^{(i)})$ and the population loss is $L(\theta) = \mathbb{E}_{(X,Y)\sim P} \ell(\theta; X, Y)$ whenever the expectation exists.

Proposition 20.1 (Cross-entropy derivative and input sensitivity)**.** *Write* $p_k = p_{\theta,k}(x)$ *and* $z_k = z_{\theta,k}(x)$*. For* $j = 1, \dots, K$*,*

$$\frac{\partial \ell}{\partial z_j} = p_j - \mathbf{1}_{\{j=y\}}.$$

If the logit map is differentiable in x*, with Jacobian* $J_{z_\theta}(x) \in \mathbb{R}^{K\times d}$*, then*

$$\nabla_x \ell(\theta; x, y) = J_{z_\theta}(x)^\top \big(p_\theta(x) - e_y\big),$$

where e_y *is the* y*th standard basis vector.*

Proof. For each j, differentiate the log-sum-exp representation:

$$\begin{aligned}\frac{\partial \ell}{\partial z_j} &= -\frac{\partial z_y}{\partial z_j} + \frac{1}{\sum_{k=1}^K e^{z_k}} \frac{\partial}{\partial z_j} \sum_{k=1}^K e^{z_k} \\ &= -\mathbf{1}_{\{j=y\}} + \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \\ &= p_j - \mathbf{1}_{\{j=y\}}.\end{aligned}$$

The multivariable chain rule then yields

$$\nabla_x \ell = J_{z_\theta}(x)^\top \nabla_z \ell = J_{z_\theta}(x)^\top (p_\theta(x) - e_y).$$

□

Remark 20.1 (Two different gradients). The *parameter gradient* $\nabla_\theta \ell$ is used to update the network's weights. By contrast, the *input gradient* $\nabla_x \ell$ describes how the loss changes if the input changes while the parameters remain fixed. Adversarial-example construction generally studies the latter quantity. These gradients are distinct even though both are computed by backpropagation.

20.2 Adversarial Examples and the Threat Model

Definition 20.3 (Adversarial perturbation)**.** Fix a trained classifier f_θ, an input x with true label y, and a set of allowable perturbations $\Delta(x) \subseteq \mathbb{R}^d$. An *adversarial example* is a perturbed input $x' = x + \delta \in \mathcal{X}$, with $\delta \in \Delta(x)$, that satisfies an attack objective such as $f_\theta(x') \neq y$.

A standard constraint is a norm ball,

$$\Delta_p(x; \varepsilon) = \{\delta \in \mathbb{R}^d : \|\delta\|_p \leq \varepsilon,\ x + \delta \in \mathcal{X}\}, \qquad \varepsilon > 0.$$

For a fixed ε, the choice of p determines what is meant by a *small* change.

Some common choices are

$$\|\delta\|_\infty = \max_{1 \le j \le d} |\delta_j|,$$
$$\|\delta\|_2 = \left(\sum_{j=1}^{d} \delta_j^2 \right)^{1/2},$$
$$\|\delta\|_1 = \sum_{j=1}^{d} |\delta_j|,$$
$$\|\delta\|_0 = \#\{j : \delta_j \neq 0\}.$$

Strictly speaking, $\|\cdot\|_0$ is a count of nonzero coordinates rather than a norm. For image pixels normalized to $[0,1]$, the input constraint is typically $\mathcal{X} = [0,1]^d$.

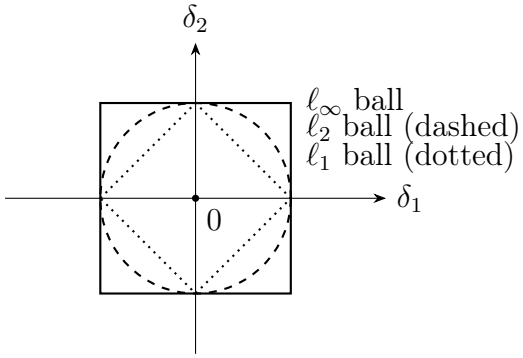


Figure 53: Schematic two-dimensional perturbation sets of radius ε. In two dimensions the ℓ_∞ ball is a square, the ℓ_2 ball is a disk, and the ℓ_1 ball is a diamond.

Definition 20.4 (Untargeted and targeted objectives)**.** An *untargeted* attack attempts to produce any incorrect predicted label. A common differentiable surrogate is

$$\max_{\delta \in \Delta_p(x;\varepsilon)} \ell(\theta; x + \delta, y).$$

For a chosen target label $t \neq y$, a *targeted* attack instead attempts to force $f_\theta(x + \delta) = t$. One common surrogate is

$$\min_{\delta \in \Delta_p(x;\varepsilon)} \ell(\theta; x + \delta, t).$$

The optimization surrogates need not produce a successful misclassification: maximizing the loss associated with y and changing the predicted class are related but not identical events. Likewise, reducing cross-entropy for a target class does not guarantee that class becomes the argmax.

Definition 20.5 (White-box and black-box access)**.** In a *white-box* setting the attacker is assumed to know the model structure and parameters and can usually compute $\nabla_x \ell$. In a *black-box* setting the attacker lacks direct access to the internal parameters or derivatives and may only observe the model's outputs, scores, or predicted labels through queries.

A threat model should specify the attacker's knowledge, allowable input changes, objective, and query or computational budget. Robustness cannot be meaningfully assessed without such constraints. In particular, a white-box robustness claim is stronger than simply reporting performance against one fixed perturbation chosen in advance.

20.3 Decision-Boundary Distance and Local Robustness

Definition 20.6 (Distance to a class-change region)**.** For a correctly classified point (x, y), define the adversarial distance under a norm $\|\cdot\|$ by

$$\rho_\theta(x, y) = \inf\{\|\delta\| : x + \delta \in \mathcal{X},\ f_\theta(x + \delta) \neq y\}.$$

The convention is $\inf \varnothing = +\infty$. The classifier is *locally robust at radius* ε if no allowed δ of norm at most ε changes its prediction from y.

This definition captures the geometry illustrated by the handwritten decision-boundary diagram: an ordinary observation may lie correctly classified but close to a boundary, whereas a small permitted perturbation can move it into another region. In general, low training loss does not imply a large adversarial distance.

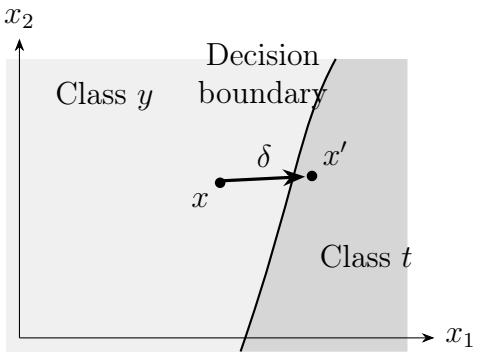


Figure 54: A small input perturbation may cross a decision boundary and cause a different prediction. The figure is schematic, not a plot of a learned classifier.

20.3.1 Exact Calculations for a Linear Classifier

Consider a binary linear score $s(x) = w^\top x + b$ with $w \neq 0$. The predicted label is $+1$ if $s(x) \geq 0$ and -1 otherwise. Let the correct label be $y \in \{-1, +1\}$. Define its signed margin as $m(x, y) = y(w^\top x + b)$. A correctly classified point with strict margin satisfies $m(x, y) > 0$.

Proposition 20.2 (Worst-case margin under ℓ_∞ perturbations)**.** *Ignoring any additional box constraint on $x + \delta$, the smallest margin over $\|\delta\|_\infty \leq \varepsilon$ is*

$$\min_{\|\delta\|_\infty \leq \varepsilon} y(w^\top(x + \delta) + b) = m(x, y) - \varepsilon\|w\|_1.$$

A minimizing perturbation is $\delta^* = -\varepsilon y \operatorname{sign}(w)$, with arbitrary choices for the signs of zero coordinates. In particular, $m(x, y) > \varepsilon\|w\|_1$ guarantees the prediction is unchanged throughout this ball.

Proof. Using linearity and $y^2 = 1$,

$$\begin{aligned}\min_{\|\delta\|_\infty \le \varepsilon} y(w^\top(x+\delta)+b) &= m(x,y) + \min_{\|\delta\|_\infty \le \varepsilon} y \sum_{j=1}^{d} w_j \delta_j \\ &= m(x,y) + \sum_{j=1}^{d} \min_{|\delta_j| \le \varepsilon} y w_j \delta_j \\ &= m(x,y) - \varepsilon \sum_{j=1}^{d} |w_j| \\ &= m(x,y) - \varepsilon \|w\|_1.\end{aligned}$$

For each coordinate the minimum is attained by $\delta_j = -\varepsilon y \operatorname{sign}(w_j)$ whenever $w_j \neq 0$. If the minimized margin remains strictly positive, no allowed perturbation changes the sign of the score. □

Proposition 20.3 (Distance to a linear boundary)**.** *Let $x \in \mathbb{R}^d$, $s(x) = w^\top x + b$ and $w \neq 0$. If $\|\cdot\|_p$ is a norm with dual norm $\|\cdot\|_q$, where $1/p + 1/q = 1$, then the distance from x to the hyperplane $\{u : w^\top u + b = 0\}$ is*

$$\operatorname{dist}_p(x, \{u : w^\top u + b = 0\}) = \frac{|w^\top x + b|}{\|w\|_q}.$$

In particular,

$$\operatorname{dist}_\infty = \frac{|w^\top x + b|}{\|w\|_1}, \qquad \operatorname{dist}_2 = \frac{|w^\top x + b|}{\|w\|_2}.$$

Proof. To reach the hyperplane, a perturbation must satisfy $w^\top \delta = -(w^\top x + b)$. By Hölder's inequality,

$$|w^\top x + b| = |w^\top \delta| \le \|w\|_q \|\delta\|_p.$$

Therefore every such perturbation has norm at least $|w^\top x + b|/\|w\|_q$. By the definition of the dual norm in finite-dimensional space, a unit ℓ_p vector u attains $|w^\top u| = \|w\|_q$. Choosing its sign and scaling it by $|w^\top x + b|/\|w\|_q$ attains equality and reaches the hyperplane. □

Remark 20.2. The result gives the exact geometric distance to a *linear* decision boundary, in the unconstrained input space. For nonlinear neural networks, one typically studies locally linear approximations or computes attacks numerically. An input-domain constraint, such as requiring image pixels to remain in $[0, 1]$, can prevent the unconstrained minimizing perturbation from being feasible.

20.4 The Fast Gradient Sign Method

The Fast Gradient Sign Method (FGSM) provides a simple, one-step approximation to the inner adversarial optimization problem. Its main justification comes from first-order Taylor expansion combined with a norm constraint.

Proposition 20.4 (First-order ℓ_∞ attack)**.** *Let $g = \nabla_x \ell(\theta; x, y)$ and suppose the loss is differentiable near x. The first-order approximation to the untargeted adversarial objective is*

$$\ell(\theta; x + \delta, y) \approx \ell(\theta; x, y) + g^\top \delta.$$

Over the unconstrained ℓ_∞ ball $\|\delta\|_\infty \le \varepsilon$, this linearized loss is maximized by

$$\delta_{\mathrm{FGSM}} = \varepsilon \operatorname{sign}(g),$$

and the largest predicted first-order increase is $\varepsilon \|g\|_1$.

Proof. Because $|\delta_j| \le \varepsilon$ for every coordinate,

$$\begin{aligned} g^\top \delta &= \sum_{j=1}^{d} g_j \delta_j \\ &\le \sum_{j=1}^{d} |g_j| \, |\delta_j| \\ &\le \varepsilon \sum_{j=1}^{d} |g_j| \\ &= \varepsilon \|g\|_1. \end{aligned}$$

Equality holds by selecting $\delta_j = \varepsilon \operatorname{sign}(g_j)$ for nonzero g_j. This solves the linearized optimization exactly, although it need not solve the original nonlinear maximization problem. □

Definition 20.7 (FGSM attack)**.** The untargeted FGSM perturbation and adversarial input are

$$\begin{aligned} \delta_{\mathrm{FGSM}} &= \varepsilon \operatorname{sign}\big(\nabla_x \ell(\theta; x, y)\big), \\ x_{\mathrm{adv}} &= \Pi_{\mathcal{X}}\big(x + \delta_{\mathrm{FGSM}}\big), \end{aligned}$$

where $\Pi_{\mathcal{X}}$ clips or projects the perturbed input onto the valid input domain. For $\mathcal{X} = [0, 1]^d$, the projection is coordinatewise clipping to the interval $[0, 1]$.

The perturbation is in the *input space*. It leaves θ unchanged. Standard training updates θ in the negative parameter-gradient direction, whereas the untargeted attack updates x in the positive input-gradient direction:

$$\begin{aligned} \text{Gradient-descent training:} &\quad \theta \leftarrow \theta - \eta \nabla_\theta \ell(\theta; x, y), \\ \text{Untargeted gradient ascent:} &\quad x \leftarrow x + \eta \nabla_x \ell(\theta; x, y). \end{aligned}$$

FGSM replaces the raw input-gradient direction with its sign and takes one norm-constrained step.

Remark 20.3 (Targeted FGSM). For a desired target $t \neq y$, a common one-step targeted variant *decreases* the target-label loss:

$$x_{\text{adv}}^{\text{target}} = \Pi_{\mathcal{X}}\left(x - \varepsilon\,\text{sign}\big(\nabla_x \ell(\theta; x, t)\big)\right).$$

The minus sign is essential: maximizing the true-label cross-entropy and minimizing the target-label cross-entropy correspond to different optimization problems.

20.4.1 A Direct Numerical Example

Example 20.1 (An FGSM step with clipping)**.** Suppose an input has three normalized coordinates,

$$x = (0.45,\, 0.10,\, 0.98), \qquad \nabla_x \ell = (-2,\, 3,\, 0.5), \qquad \varepsilon = 0.03.$$

The sign vector is $(-1, 1, 1)$, so the unprojected perturbation is

$$\begin{aligned} \delta_{\text{FGSM}} &= 0.03(-1, 1, 1) = (-0.03, 0.03, 0.03), \\ x + \delta_{\text{FGSM}} &= (0.42, 0.13, 1.01). \end{aligned}$$

For valid pixels in $[0, 1]^3$, projection gives

$$x_{\text{adv}} = (0.42, 0.13, 1.00).$$

The perturbation respects the ℓ_∞ radius after clipping; the third coordinate moves by 0.02 rather than 0.03.

Example 20.2 (Image perturbations and their units)**.** An MNIST image contains $28 \times 28 = 784$ grayscale pixel coordinates. If the pixel values are rescaled from integer intensities $\{0, \ldots, 255\}$ to real values in $[0, 1]$, an ℓ_∞ budget often written $\varepsilon = 8/255$ allows each pixel to change by at most approximately 0.0314 on the normalized scale:

$$\|x_{\text{adv}} - x\|_\infty \leq \frac{8}{255}.$$

This does not assert that the perturbed image is imperceptible in every setting. Visibility depends on the image, display, perturbation structure, and perceptual criteria. The normalization and attack budget must always be stated together.

20.4.2 Other Norm-Constrained First-Order Attacks

Proposition 20.5 (Dual-norm first-order optimum)**.** *Let $g \in \mathbb{R}^d$, and let $1 \leq p \leq \infty$ with dual exponent q satisfying $1/p + 1/q = 1$. Then*

$$\max_{\|\delta\|_p \leq \varepsilon} g^\top \delta = \varepsilon \|g\|_q.$$

For $g \neq 0$, examples of maximizing perturbations include

$$\begin{aligned}\delta_\infty^* &= \varepsilon \operatorname{sign}(g),\\ \delta_2^* &= \varepsilon \frac{g}{\|g\|_2},\\ \delta_1^* &= \varepsilon \operatorname{sign}(g_{j^*}) e_{j^*}, \quad j^* \in \arg\max_j |g_j|.\end{aligned}$$

Proof. Hölder's inequality implies $g^\top \delta \leq \|g\|_q \|\delta\|_p \leq \varepsilon \|g\|_q$. In finite dimensions equality can be achieved by an appropriate vector on the boundary of the ball. For $p = 2$, the Cauchy–Schwarz equality case yields $\delta^* = \varepsilon g / \|g\|_2$. For $p = \infty$, each component may independently use its maximum signed magnitude. For $p = 1$, concentrating the entire budget in a coordinate with largest $|g_j|$ achieves the bound. □

Remark 20.4. An FGSM-type sign step is tailored to the ℓ_∞ constraint. The ℓ_2-optimal first-order direction is the *normalized raw gradient*, not the sign vector. Different perturbation norms represent different attack models and should not be interchanged.

20.5 Why High Input Dimension Matters

The handwritten lecture emphasizes the dimension of image space. A 28×28 image occupies 784 input coordinates, whereas a toy two-dimensional plot shows only a tiny part of that geometry. The following calculation explains why many individually small perturbations can accumulate into a substantial change in a model's score.

Proposition 20.6 (Accumulation of coordinatewise changes)**.** *For an affine function $s(x) = w^\top x + b$, the largest possible change in its score under an ℓ_∞ perturbation of radius ε is*

$$\max_{\|\delta\|_\infty \leq \varepsilon} |s(x + \delta) - s(x)| = \varepsilon \|w\|_1.$$

If $w = (1, \ldots, 1) \in \mathbb{R}^d$, this maximum is εd. If $w = (1/d, \ldots, 1/d)$, it is ε.

Proof. Since $s(x + \delta) - s(x) = w^\top \delta$, Hölder's inequality gives $|w^\top \delta| \leq \|w\|_1 \|\delta\|_\infty \leq \varepsilon \|w\|_1$. Equality is attained by $\delta = \varepsilon \operatorname{sign}(w)$ or its negative. The examples follow by evaluating $\|w\|_1$. □

In the first example, an ε-sized change in each of d coordinates can combine to a score shift of order d. In the second example the normalization of the weights prevents that same scaling. Thus *high dimension alone does not prove vulnerability*; the effect depends on how the classifier's gradients or weights scale with dimension.

For comparison, if δ_j are independent, zero-mean random perturbations with variance σ^2, then

$$\begin{aligned}\mathbb{E}[w^\top \delta] &= 0,\\ \operatorname{Var}(w^\top \delta) &= \sum_{j=1}^{d} w_j^2 \operatorname{Var}(\delta_j)\\ &= \sigma^2 \|w\|_2^2.\end{aligned}$$

An adversarial perturbation aligns with the gradient direction; random zero-mean noise usually does not. This helps explain why robustness to generic random noise is not equivalent to adversarial robustness.

20.6 Iterative Attacks and Projection

Although the lecture introduces one-step FGSM, the same principle extends to multiple constrained gradient steps. Multiple steps can better account for curvature or changes in the loss gradient away from the original input.

Definition 20.8 (Projected gradient ascent)**.** For a fixed clean example (x, y), a step size $\alpha > 0$, and a set $\mathcal{C} = \{u \in \mathcal{X} : \|u - x\|_\infty \le \varepsilon\}$, the iterative update is

$$x^{(k+1)} = \Pi_{\mathcal{C}}\left(x^{(k)} + \alpha \operatorname{sign}\left(\nabla_u \ell(\theta; u, y)|_{u=x^{(k)}}\right)\right).$$

The iteration is a form of projected gradient ascent for the inner maximization, commonly called an ℓ_∞ projected-gradient-descent (PGD) attack by convention, despite ascending the loss.

For $\mathcal{X} = [0, 1]^d$, projection onto $\mathcal{C}$ is coordinatewise:

$$[\Pi_{\mathcal{C}}(u)]_j = \min\left\{\min(1, x_j + \varepsilon),\ \max\left\{\max(0, x_j - \varepsilon), u_j\right\}\right\}.$$

The center of this feasible set is always the *original* input x, not the latest iterate. This ensures that successive steps do not silently exceed the total attack budget. When gradients are poorly behaved, gradient-based attacks can underestimate vulnerability; stronger and diverse evaluations are needed before making a robustness claim.

20.7 Adversarial Robustness as an Optimization Problem

Ordinary supervised learning minimizes average loss on unperturbed examples. A natural robust-learning objective modifies the per-example loss by considering the worst permissible perturbation.

Definition 20.9 (Adversarial or robust risk)**.** For a threat set $\Delta(x)$, define the robust per-example loss and population risk by

$$\ell_{\text{rob}}(\theta; x, y) = \sup_{\delta \in \Delta(x)} \ell(\theta; x + \delta, y),$$

$$R_{\mathrm{rob}}(\theta) = \mathbb{E}_{(X,Y)\sim P}\left[\sup_{\delta\in\Delta(X)} \ell(\theta; X+\delta, Y)\right].$$

The corresponding robust training problem is

$$\min_{\theta} R_{\mathrm{rob}}(\theta) = \min_{\theta} \mathbb{E}_{(X,Y)\sim P}\left[\sup_{\delta\in\Delta(X)} \ell(\theta; X+\delta, Y)\right].$$

For a finite dataset, the expectation can be replaced by the empirical average.

This is a *minimax* objective: the inner step searches for difficult perturbations, while the outer step changes the network parameters to reduce their loss. The two optimization variables should be kept distinct:

$$\begin{aligned} \text{Inner attack:} \quad & \delta^*(\theta; x, y) \in \arg\max_{\delta\in\Delta(x)} \ell(\theta; x+\delta, y), \\ \text{Outer training:} \quad & \theta \in \arg\min_{\vartheta} \mathbb{E}\big[\ell(\vartheta; X+\delta^*(\vartheta; X, Y), Y)\big]. \end{aligned}$$

The inner maximizer may not be unique and may not be found exactly in practice.

Proposition 20.7 (Robust loss dominates clean loss)**.** *If $0 \in \Delta(x)$ for every input x, then*

$$\ell_{\mathrm{rob}}(\theta; x, y) \geq \ell(\theta; x, y) \qquad \textit{and} \qquad R_{\mathrm{rob}}(\theta) \geq L(\theta).$$

Proof. Since $\delta = 0$ is feasible,

$$\sup_{\delta\in\Delta(x)} \ell(\theta; x+\delta, y) \geq \ell(\theta; x+0, y) = \ell(\theta; x, y).$$

Taking expectations proves the population inequality, provided the risks are defined. □

Remark 20.5 (Robustness and generalization). A low average clean training loss does not imply low robust loss. Likewise, success against a particular attack algorithm does not imply the supremum over all allowed attacks is small. Evaluation should report the clean accuracy, attack family, ε and norm, input preprocessing, and the measured adversarial accuracy under the stated threat model.

20.7.1 Differentiating Through the Worst-Case Loss

Proposition 20.8 (Envelope rule for the robust loss)**.** *Suppose $\Delta(x)$ is compact, the loss is continuous in (θ, δ) and differentiable in θ, and the maximizer $\delta^*(\theta)$ is unique at the parameter value under consideration. Under the regularity hypotheses of Danskin's theorem,*

$$\nabla_\theta \ell_{\mathrm{rob}}(\theta; x, y) = \nabla_\theta \ell(\theta; x+\delta^*(\theta), y).$$

The right-hand side differentiates the loss with the maximizing perturbation held fixed; it contains no explicit derivative $\partial\delta^*/\partial\theta$.

Proof. Write $F(\theta) = \max_{\delta \in \Delta(x)} f(\theta, \delta)$ with $f(\theta, \delta) = \ell(\theta; x + \delta, y)$. For sufficiently small h, optimality at θ and $\theta + h$ yields

$$F(\theta + h) - F(\theta) \geq f(\theta + h, \delta^*(\theta)) - f(\theta, \delta^*(\theta)),$$
$$F(\theta + h) - F(\theta) \leq f(\theta + h, \delta^*(\theta + h)) - f(\theta, \delta^*(\theta + h)).$$

By compactness and continuity, uniqueness implies that maximizing perturbations at nearby parameters converge to $\delta^*(\theta)$ under the stated regularity conditions. Taking directional difference quotients, both bounds converge to the directional derivative $\nabla_\theta f(\theta, \delta^*(\theta))^\top h / \|h\|$, yielding the formula. □

Remark 20.6. In practical adversarial training, the inner maximizer is only approximated by FGSM, PGD, or another attack. The exact envelope conclusion may therefore not apply to the resulting approximate surrogate, and nondifferentiability may occur when several perturbations are equally optimal.

20.8 Robust Classification Margins and Certificates

A loss-based attack is an optimization surrogate. A complementary approach gives sufficient conditions under which the actual predicted class provably cannot change.

Definition 20.10 (Logit margin)**.** For input x with correct label y, define the logit margin

$$M_\theta(x, y) = z_{\theta,y}(x) - \max_{k \neq y} z_{\theta,k}(x).$$

The classifier predicts y uniquely whenever $M_\theta(x, y) > 0$.

Proposition 20.9 (Lipschitz margin certificate)**.** *Suppose that, for each $k \neq y$, the logit difference $g_k(u) = z_{\theta,y}(u) - z_{\theta,k}(u)$ satisfies*

$$|g_k(u) - g_k(v)| \leq L_k \|u - v\|_p$$

for all u, v in the neighborhood of interest. Then every δ with $\|\delta\|_p \leq \varepsilon$ preserves prediction y whenever

$$z_{\theta,y}(x) - z_{\theta,k}(x) > L_k \varepsilon \qquad \text{for all } k \neq y.$$

Proof. For every competing class k, the Lipschitz assumption gives

$$\begin{aligned} z_{\theta,y}(x + \delta) - z_{\theta,k}(x + \delta) &= g_k(x + \delta) \\ &\geq g_k(x) - L_k \|\delta\|_p \\ &\geq z_{\theta,y}(x) - z_{\theta,k}(x) - L_k \varepsilon \\ &> 0. \end{aligned}$$

Thus the true-class logit strictly exceeds every competing logit throughout the specified ball. □

Remark 20.7. This is a *sufficient* robustness certificate; failure to satisfy it does not establish that an adversarial example exists. Deriving tight Lipschitz bounds for a deep network can be challenging, so certificates may be conservative.

20.9 A Small Worked Linear Attack

Example 20.3 (A clean prediction that is not robust)**.** Consider $w = (1, 2)^\top$, $b = -0.5$, $x = (0.3, 0.2)^\top$, and true binary class $y = +1$. The clean margin is

$$\begin{aligned} m(x, +1) &= w^\top x + b \\ &= 1(0.3) + 2(0.2) - 0.5 \\ &= 0.2 > 0. \end{aligned}$$

The minimal ℓ_∞ distance to the hyperplane is

$$\rho_\infty = \frac{0.2}{|1| + |2|} = \frac{1}{15} \approx 0.0667.$$

Choose $\varepsilon = 0.08$ and the perturbation

$$\delta = -\varepsilon \operatorname{sign}(w) = (-0.08, -0.08)^\top.$$

Then $x' = x + \delta = (0.22, 0.12)^\top$ and

$$\begin{aligned} s(x') &= 1(0.22) + 2(0.12) - 0.5 \\ &= 0.22 + 0.24 - 0.5 \\ &= -0.04 < 0. \end{aligned}$$

Thus an input originally classified as $+1$ is now classified as -1, despite a coordinatewise perturbation of no more than 0.08.

20.10 Implementation and Evaluation

The following PyTorch-style pseudocode illustrates a single untargeted FGSM step on inputs already normalized to the interval $[0, 1]$. The code does not update any model weights while producing an adversarial example.

```
def fgsm(model, x, y, epsilon):
    x0 = x.detach()
    x_attack = x0.clone().requires_grad_(True)
    logits = model(x_attack)
    loss = torch.nn.functional.cross_entropy(logits, y)
    grad = torch.autograd.grad(loss, x_attack)[0]
    x_adv = x0 + epsilon * grad.sign()
    x_adv = torch.clamp(x_adv, min=0.0, max=1.0)
    return x_adv.detach()
```

Remark 20.8 (Implementation details). To compare clean and adversarial predictions, put stochastic evaluation components such as dropout in the intended evaluation mode. If the network internally standardizes its inputs, the input-gradient and clipping computations must respect the chosen coordinate system. Avoid applying a pixel-space ε directly to a differently normalized tensor without transforming the budget accordingly.

A simple attack evaluation computes both ordinary accuracy and attacked accuracy. With N labeled evaluation observations and one allowable perturbation δ_i per observation, the empirical clean and attacked accuracies are

$$\widehat{A}_{\text{clean}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}_{\{f_\theta(x^{(i)})=y^{(i)}\}},$$
$$\widehat{A}_{\text{attack}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}_{\{f_\theta(x^{(i)}+\delta_i)=y^{(i)}\}}.$$

These are measurements against the *particular constructed perturbations*. A claim about worst-case robust accuracy would require a much stronger argument or evaluation protocol.

20.11 Exercises

Exercise 1.

Let $\ell(\theta; x, y) = -\log p_{\theta,y}(x)$, where $p_\theta(x) = \operatorname{softmax}(z_\theta(x))$. Derive the input gradient of cross-entropy when the logits are affine, $z_\theta(x) = Wx + b$ with $W \in \mathbb{R}^{K\times d}$.

Solution. By the earlier cross-entropy derivative,

$$\nabla_z \ell = p - e_y.$$

The affine logit map has Jacobian $J_z(x) = W$. Consequently,

$$\begin{aligned}\nabla_x \ell &= W^\top (p - e_y) \\ &= \sum_{k=1}^{K} p_k W_{k,:}^\top - W_{y,:}^\top.\end{aligned}$$

The input gradient is a probability-weighted average of class weight vectors, minus the weight vector of the true class. It has dimension d, whereas $\nabla_b \ell = p - e_y$ has dimension K. ■

Exercise 2.

For an ℓ_2 adversarial budget $\|\delta\|_2 \le \varepsilon$, determine the first-order perturbation that maximizes a differentiable loss and compare it with the ℓ_∞ FGSM step.

Solution. Let $g = \nabla_x \ell$ and assume $g \neq 0$. The local objective is to maximize $g^\top \delta$. Cauchy–Schwarz gives

$$g^\top \delta \leq \|g\|_2 \|\delta\|_2 \leq \varepsilon \|g\|_2.$$

Equality holds when δ is positively proportional to g, giving

$$\delta_2^* = \varepsilon \frac{g}{\|g\|_2}.$$

The ℓ_∞ optimizer instead uses $\delta_\infty^* = \varepsilon \operatorname{sign}(g)$. The first perturbation distributes an overall Euclidean budget proportional to gradient magnitude; the second independently saturates coordinatewise bounds. ■

Exercise 3.

Consider a binary linear classifier with $w = (2, -1, 3)^\top$, $b = 0.1$, $x = (0.2, 0.3, 0.1)^\top$, and true label $y = +1$. Compute its clean margin and smallest unconstrained ℓ_∞ perturbation radius required to reach the decision boundary. Determine whether $\varepsilon = 0.05$ certifies robustness.

Solution. The margin is

$$\begin{aligned} m &= y(w^\top x + b) \\ &= 2(0.2) - (0.3) + 3(0.1) + 0.1 \\ &= 0.5. \end{aligned}$$

Also $\|w\|_1 = |2| + |-1| + |3| = 6$. Hence

$$\rho_\infty = \frac{m}{\|w\|_1} = \frac{0.5}{6} = \frac{1}{12} \approx 0.0833.$$

For $\varepsilon = 0.05$, the worst-case margin is

$$m - \varepsilon \|w\|_1 = 0.5 - 0.05(6) = 0.2 > 0.$$

Thus the classifier is robust at that radius under the stated unconstrained linear model; any additional valid-input restriction can only remove candidate perturbations. ■

Exercise 4.

Explain why the dimension d does not by itself determine the magnitude of the adversarial effect. Compare $w = (1, \ldots, 1) \in \mathbb{R}^d$ with $\widetilde{w} = (1/d, \ldots, 1/d)$ under the same ℓ_∞ budget.

Solution. For the affine score change,

$$\sup_{\|\delta\|_\infty \leq \varepsilon} |w^\top \delta| = \varepsilon \|w\|_1.$$

The first weight vector satisfies $\|w\|_1 = d$ and gives a maximum score change εd. The normalized vector has $\|\widetilde{w}\|_1 = 1$, giving maximum score change ε. The same input dimension therefore

produces different sensitivity depending on the weight scaling. A complete robustness claim also requires the clean margin, not only the worst-case score change. ■

Exercise 5.

For an untargeted attack, explain why the update $\delta = -\varepsilon \operatorname{sign}(\nabla_x \ell(\theta; x, y))$ generally has the wrong sign, and describe when that sign is appropriate.

Solution. The Taylor approximation is

$$\ell(\theta; x + \delta, y) \approx \ell(\theta; x, y) + g^\top \delta, \qquad g = \nabla_x \ell(\theta; x, y).$$

With $\delta = -\varepsilon \operatorname{sign}(g)$, the linearized change is

$$g^\top \delta = -\varepsilon \sum_j |g_j| = -\varepsilon \|g\|_1 \leq 0.$$

It therefore *decreases* the true-label loss to first order, whereas an untargeted attack ordinarily seeks to increase it. The negative sign is appropriate for a targeted attack if the differentiated loss is the cross-entropy for the desired target label t. ■

Exercise 6.

Assume $0 \in \Delta(x)$ for every x. Prove that a parameter vector minimizing the clean population risk need not minimize robust risk. Construct a simple finite hypothesis-class example.

Solution. Consider two candidate models θ_A and θ_B on a distribution concentrated at a single observation (x, y). Suppose their losses are, respectively,

$$\ell(\theta_A; x, y) = 0.01, \qquad \sup_{\delta \in \Delta(x)} \ell(\theta_A; x + \delta, y) = 10,$$

$$\ell(\theta_B; x, y) = 0.10, \qquad \sup_{\delta \in \Delta(x)} \ell(\theta_B; x + \delta, y) = 0.20.$$

Such a pattern can occur if one model is highly confident at the clean input but sharply sensitive nearby, while the second has a smoother local response. Among the two models, θ_A uniquely minimizes clean loss because $0.01 < 0.10$, while θ_B uniquely minimizes robust loss because $0.20 < 10$. Both respect the pointwise requirement that the robust loss dominate the clean loss. Thus the optimizers of the two objectives need not coincide. ■

Exercise 7.

Suppose the logit difference $z_y(u) - z_k(u)$ is L_k-Lipschitz under an ℓ_2 norm near x, and its clean value is 0.6. If $L_k = 2$ and $\varepsilon = 0.2$, can this competitor overtake the true class within the allowed ball? What if $\varepsilon = 0.4$?

Solution. For every $\|\delta\|_2 \le \varepsilon$,

$$z_y(x+\delta) - z_k(x+\delta) \ge 0.6 - 2\varepsilon.$$

At $\varepsilon = 0.2$, the lower bound is $0.6 - 0.4 = 0.2 > 0$, so this competing class cannot overtake the true class. At $\varepsilon = 0.4$, the lower bound is $0.6 - 0.8 = -0.2$, which fails to certify robustness. The failed certificate is not proof that an actual misclassification exists. ■

21 Lecture 22: Adversarial Defense and TRADES Robust Training

21.1 From Adversarial Attacks to Adversarial Defense

Lecture 21 introduced *adversarial examples*: inputs obtained by small, deliberately chosen perturbations that cause a classifier to make incorrect predictions. An ordinary classifier is trained to perform well on examples sampled from a data distribution, yet it may behave quite differently at points only a small distance away. We now study how to train a model whose predictions remain stable throughout a prescribed neighborhood of each observation.

A direct approach is to incorporate adversarially perturbed examples into the training objective. Rather than adjusting network parameters only in response to clean training inputs, we repeatedly search for challenging nearby inputs and train the network to produce appropriate predictions there as well.

Definition 21.1 (Statistical classification problem)**.** Let (X, Y) have joint distribution P on $\mathcal{X} \times \mathcal{Y}$, where $\mathcal{X} \subseteq \mathbb{R}^d$ and, initially, $\mathcal{Y} = \{-1, +1\}$. The training data are

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}, \qquad (x^{(i)}, y^{(i)}) \in \mathcal{X} \times \mathcal{Y}.$$

A real-valued score $s_\theta : \mathcal{X} \to \mathbb{R}$ determines a classifier

$$f_\theta(x) = \operatorname{sign}(s_\theta(x)),$$

with a fixed convention at zero. For a norm $\|\cdot\|_p$ and perturbation radius $\varepsilon \geq 0$, define the admissible neighborhood

$$\mathcal{B}_p(x, \varepsilon) = \{x' \in \mathcal{X} : \|x' - x\|_p \leq \varepsilon\}.$$

We assume that $x \in \mathcal{B}_p(x, \varepsilon)$.

The predicted label is correct when $y s_\theta(x) > 0$; the quantity $y s_\theta(x)$ is called the *signed classification margin*. A decision boundary occurs at points where the score changes sign. The purpose of a defense is to discourage decision boundaries from cutting through neighborhoods of correctly classified observations.

Definition 21.2 (Natural classification risk)**.** The population misclassification probability is

$$R_{\mathrm{nat}}(\theta) = \mathbb{P}\big(f_\theta(X) \neq Y\big) = \mathbb{E}\big[\mathbf{1}_{\{Y s_\theta(X) \leq 0\}}\big],$$

where the second equality uses a tie convention that treats zero margin as an error. Its empirical analogue is

$$\widehat{R}_{\mathrm{nat}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}_{\{y^{(i)} s_\theta(x^{(i)}) \leq 0\}}.$$

Definition 21.3 (Robust classification risk)**.** The ε-robust zero–one risk is

$$R_{\mathrm{rob}}(\theta;\varepsilon) = \mathbb{P}\left(\exists X' \in \mathcal{B}_p(X,\varepsilon) : f_\theta(X') \neq Y\right)$$
$$= \mathbb{E}\left[\sup_{x' \in \mathcal{B}_p(X,\varepsilon)} \mathbf{1}_{\{f_\theta(x') \neq Y\}}\right].$$

In this expression the supremum is taken inside the expectation: the attacker is allowed to choose a different perturbation for each input observation.

For technical precision, the displayed random events are assumed measurable. This holds in common settings with continuous scores and compact norm balls after adopting an appropriate boundary convention.

Proposition 21.1 (Robust error dominates natural error)**.** *For any classifier, perturbation norm, and $\varepsilon \geq 0$,*

$$R_{\mathrm{nat}}(\theta) \leq R_{\mathrm{rob}}(\theta;\varepsilon).$$

Moreover, if $0 \leq \varepsilon_1 \leq \varepsilon_2$, then

$$R_{\mathrm{rob}}(\theta;\varepsilon_1) \leq R_{\mathrm{rob}}(\theta;\varepsilon_2).$$

Proof. Since $x \in \mathcal{B}_p(x,\varepsilon)$, the loss at x is one of the terms over which the supremum is taken. Hence, pointwise,

$$\mathbf{1}_{\{f_\theta(x) \neq y\}} \leq \sup_{x' \in \mathcal{B}_p(x,\varepsilon)} \mathbf{1}_{\{f_\theta(x') \neq y\}}.$$

Taking expectations proves the first inequality. The ball inclusion $\mathcal{B}_p(x,\varepsilon_1) \subseteq \mathcal{B}_p(x,\varepsilon_2)$ implies that the supremum over the smaller ball cannot exceed the supremum over the larger ball. Taking expectations yields the second inequality. □

Remark 21.1 (What robustness measures). A network can classify the clean input correctly with high confidence and still have a large robust error if a small allowed perturbation crosses its decision boundary. Conversely, an attack algorithm that fails to find a misclassification does not prove that none exists. The robust risk is defined using a true supremum over an entire perturbation set, which is generally much harder to evaluate than a finite set of attacks.

21.2 Classification Boundaries and the Robustness Trade-off

The handwritten lecture illustrates a clean point close to a curved decision boundary. A defense seeks to shift or smooth this boundary so that the ball centered at the clean point lies entirely in its correct classification region. This geometric picture suggests a tension between fitting clean data and maintaining stable predictions nearby.

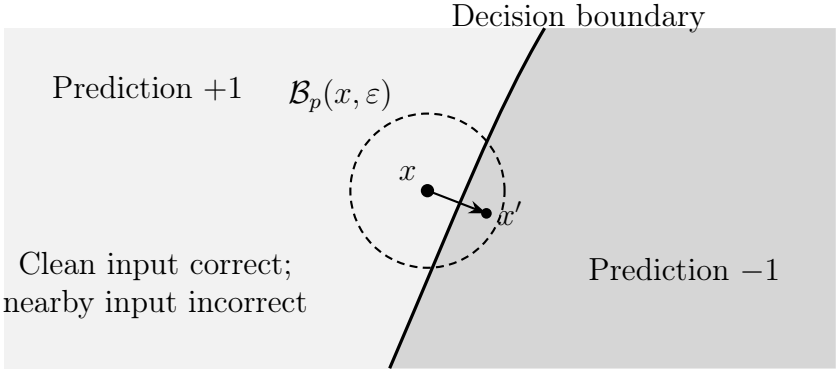


Figure 55: A clean input may be correctly classified while its perturbation neighborhood crosses the decision boundary. The dashed circle is a schematic perturbation ball.

Definition 21.4 (Robustly correct and boundary-vulnerable inputs)**.** For any observation (x, y) define two events:

$$\begin{aligned} C(x, y) &= \{f_\theta(x) = y\}, \\ B(x) &= \{\exists x' \in \mathcal{B}_p(x, \varepsilon) : f_\theta(x') \neq f_\theta(x)\}. \end{aligned}$$

The *boundary error* is the probability of a cleanly correct input lying near a region with a different prediction:

$$R_{\mathrm{bdy}}(\theta; \varepsilon) = \mathbb{P}\big(C(X, Y) \cap B(X)\big).$$

Theorem 21.1 (Exact decomposition of robust risk)**.** *For any fixed classifier and admissible perturbation neighborhoods containing the center point,*

$$R_{\mathrm{rob}}(\theta; \varepsilon) = R_{\mathrm{nat}}(\theta) + R_{\mathrm{bdy}}(\theta; \varepsilon).$$

Proof. Write

$$A = \{f_\theta(X) \neq Y\}, \qquad D = \{\exists X' \in \mathcal{B}_p(X, \varepsilon) : f_\theta(X') \neq Y\}.$$

Since X is itself in the neighborhood, $A \subseteq D$. Decompose D into disjoint events:

$$D = A \,\dot\cup\, (D \cap A^c).$$

On A^c, the clean prediction equals the true label, so

$$\begin{aligned} D \cap A^c &= \{f_\theta(X) = Y,\ \exists X' \in \mathcal{B}_p(X, \varepsilon) : f_\theta(X') \neq Y\} \\ &= \{f_\theta(X) = Y,\ \exists X' \in \mathcal{B}_p(X, \varepsilon) : f_\theta(X') \neq f_\theta(X)\} \\ &= C(X, Y) \cap B(X). \end{aligned}$$

Thus

$$\begin{aligned} \mathbb{P}(D) &= \mathbb{P}(A) + \mathbb{P}(D \cap A^c) \\ &= R_{\mathrm{nat}}(\theta) + R_{\mathrm{bdy}}(\theta; \varepsilon), \end{aligned}$$

which proves the identity. □

The decomposition is particularly useful: robust error can occur either because the model already misclassifies the original point, or because a correctly classified point has an unstable prediction nearby. These are different deficiencies, and they motivate different parts of the training objective below.

Remark 21.2 (Not every disagreement is a robust error). The unconditional probability of local prediction disagreement, $\mathbb{P}(B(X))$, is not in general the same as R_{bdy}. An input that is already incorrectly classified can also have a nearby region with a different prediction, but its robust error is already counted by the natural-risk term. Including the condition $f_\theta(X) = Y$ makes the decomposition exact.

21.3 Why the Zero–One Objective is Difficult to Optimize

Classification accuracy is defined using an indicator. For binary scores, the ordinary zero–one loss is

$$\ell_{01}(s, y) = \mathbf{1}_{\{ys \le 0\}}.$$

Away from its discontinuity, this is constant and has zero derivative. At the decision boundary it is discontinuous. Consequently, direct gradient descent on empirical zero–one error provides essentially no useful local gradient signal.

Definition 21.5 (Margin-based surrogate loss)**.** A *margin surrogate* is a function $\phi : \mathbb{R} \to [0, \infty)$ used in place of the indicator $\mathbf{1}_{\{u \le 0\}}$. Its argument is the signed margin $u = ys_\theta(x)$, and the surrogate classification loss is

$$\ell_\phi(\theta; x, y) = \phi\big(ys_\theta(x)\big).$$

Common examples include

$$\begin{aligned} \phi_{\log}(u) &= \log(1 + e^{-u}), \\ \phi_{\mathrm{hinge}}(u) &= \max\{0, 1 - u\}, \\ \phi_{\exp}(u) &= e^{-u}. \end{aligned}$$

Each listed loss penalizes a small or negative margin more heavily than a large positive margin. The logistic loss is smooth; the hinge loss is convex but not differentiable at $u = 1$. One can also use a smooth bounded proxy such as

$$\phi_{\mathrm{smooth}}(u) = \frac{1}{1 + e^{a(u-c)}}, \qquad a > 0,$$

which approximates a decreasing step centered at $u = c$. The bounded proxy is useful for visualization, but it is not the same as the unbounded logistic cross-entropy used in most modern classification training.

Proposition 21.2 (Derivatives of the logistic margin loss)**.** *For $\phi(u) = \log(1 + e^{-u})$,*

$$\phi'(u) = -\frac{1}{1 + e^{u}},$$
$$\phi''(u) = \frac{e^{u}}{(1 + e^{u})^2} > 0.$$

Therefore the logistic loss is decreasing and strictly convex as a function of its margin. If s_θ is differentiable, then

$$\nabla_\theta \ell_\phi(\theta; x, y) = -\frac{y}{1 + e^{y s_\theta(x)}} \nabla_\theta s_\theta(x).$$

Proof. The chain rule gives

$$\frac{\mathrm{d}}{\mathrm{d}u} \log(1 + e^{-u}) = \frac{-e^{-u}}{1 + e^{-u}} = -\frac{1}{1 + e^{u}},$$
$$\frac{\mathrm{d}}{\mathrm{d}u} \left(-\frac{1}{1 + e^{u}} \right) = \frac{e^{u}}{(1 + e^{u})^2}.$$

Apply the multivariable chain rule to $\phi(y s_\theta(x))$ for the last expression. □

Remark 21.3 (Surrogates are not the target metric)*.* Minimizing a differentiable surrogate provides usable gradients; it does not mean that the surrogate equals the actual zero–one risk. The same distinction is important in adversarial training: the maximized surrogate discrepancy between two outputs is not automatically a certified upper bound on their worst-case classification disagreement without additional assumptions.

21.4 Adversarial Training as Minimax Optimization

The most direct robust training method replaces the clean loss by its maximum over the admissible perturbation neighborhood:

$$L_{\mathrm{AT}}(\theta) = \mathbb{E}_{(X,Y)\sim P} \left[\sup_{x' \in \mathcal{B}_p(X, \varepsilon)} \ell(\theta; x', Y) \right],$$
$$\theta^* \in \arg\min_{\theta} L_{\mathrm{AT}}(\theta).$$

For a finite dataset, the objective is approximated by

$$\widehat{L}_{\mathrm{AT}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sup_{x' \in \mathcal{B}_p(x^{(i)}, \varepsilon)} \ell(\theta; x', y^{(i)}).$$

This formulation introduces two nested optimization problems:

- The *inner maximization* searches for a difficult allowed perturbation with the model parameters fixed.
- The *outer minimization* updates the model parameters to reduce the loss on those perturbed inputs.

Proposition 21.3 (Clean-loss lower bound)**.** *For any nonnegative loss ℓ and any θ,*

$$\mathbb{E}[\ell(\theta; X, Y)] \leq L_{\mathrm{AT}}(\theta).$$

Proof. The admissible ball includes X, so

$$\ell(\theta; X, Y) \leq \sup_{x' \in \mathcal{B}_p(X,\varepsilon)} \ell(\theta; x', Y)$$

pointwise. Take expectations. □

Remark 21.4 (Clean accuracy and robust accuracy). Optimizing worst-case loss changes the effective training distribution and can change the location of the decision boundary. This may reduce clean accuracy in some settings as the model gains robustness, but a strict performance trade-off is not a theorem for every architecture and distribution. The outcome depends on the data, capacity, perturbation constraints, training procedure, and evaluation metric.

21.5 TRADES: Separating Accuracy from Boundary Stability

TRADES is short for *TRadeoff-inspired Adversarial DEfense via Surrogate-loss minimization.* Its central idea is to train for both ordinary prediction accuracy and local output consistency. Rather than maximizing the true-label classification loss alone, it penalizes changes in the model's prediction distribution between the clean input and nearby perturbed inputs.

In the binary score formulation, the risk decomposition suggests a loss of the form

$$\text{classification surrogate} + \beta \, \text{boundary-stability surrogate}, \qquad \beta \geq 0.$$

The hyperparameter β controls how strongly the training algorithm penalizes output changes within the perturbation ball. It does not change the radius ε itself.

Definition 21.6 (Probabilistic multiclass classifier)**.** For K classes, a network with parameters θ returns logits $z_\theta(x) \in \mathbb{R}^K$ and a probability vector

$$p_\theta(x) = \mathrm{softmax}(z_\theta(x)) \in \Delta^{K-1},$$

where

$$\Delta^{K-1} = \left\{ p \in [0,1]^K : \sum_{k=1}^{K} p_k = 1 \right\}.$$

The clean cross-entropy for a true label y is

$$\mathrm{CE}(y, p_\theta(x)) = -\log p_{\theta,y}(x).$$

Definition 21.7 (Kullback–Leibler prediction discrepancy)**.** For probability vectors $p, q \in \Delta^{K-1}$ with strictly positive components, define

$$D_{\mathrm{KL}}(p\|q) = \sum_{k=1}^{K} p_k \log \frac{p_k}{q_k}.$$

In adversarial consistency training, the first distribution is the prediction at a clean input and the second is the prediction at a perturbed input:

$$D_\theta(x, x') = D_{\mathrm{KL}}\big(p_\theta(x)\|p_\theta(x')\big).$$

Proposition 21.4 (Nonnegativity of KL divergence)**.** *For strictly positive probability vectors,*

$$D_{\mathrm{KL}}(p\|q) \geq 0,$$

with equality if and only if $p = q$.

Proof. By the elementary inequality $-\log t \geq 1 - t$ for $t > 0$,

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= \sum_{k=1}^{K} p_k \left(-\log \frac{q_k}{p_k}\right) \\ &\geq \sum_{k=1}^{K} p_k \left(1 - \frac{q_k}{p_k}\right) \\ &= \sum_{k=1}^{K} p_k - \sum_{k=1}^{K} q_k = 0. \end{aligned}$$

Equality in the logarithmic inequality requires $q_k = p_k$ for every k. □

Definition 21.8 (Practical TRADES objective)**.** A standard multiclass TRADES training objective is

$$\begin{aligned} L_{\mathrm{TRADES}}(\theta) = \mathbb{E}_{(X,Y)\sim P}\Big[&\mathrm{CE}\big(Y, p_\theta(X)\big) \\ &+ \beta \sup_{X' \in \mathcal{B}_p(X,\varepsilon)} D_{\mathrm{KL}}\big(p_\theta(X)\|p_\theta(X')\big)\Big]. \end{aligned}$$

For a dataset, this becomes

$$\begin{aligned} \widehat{L}_{\mathrm{TRADES}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[&-\log p_{\theta,y^{(i)}}(x^{(i)}) \\ &+ \beta \max_{x' \in \mathcal{B}_p(x^{(i)},\varepsilon)} D_{\mathrm{KL}}\big(p_\theta(x^{(i)})\|p_\theta(x')\big)\Big]. \end{aligned}$$

The clean cross-entropy addresses ordinary classification errors. The inner maximization identifies a nearby point whose predicted distribution is least consistent with the prediction at the clean input. The outer training step pushes those two distributions closer together while continuing to learn the correct label at the clean input.

Remark 21.5 (Binary and multiclass versions). The original theoretical TRADES analysis uses surrogate losses of binary classification margins and boundary error, while the commonly implemented multiclass loss above uses cross-entropy plus KL divergence between predictive distributions. They share the same guiding principle, but one should not identify the binary zero–one boundary event with a KL divergence without further analysis.

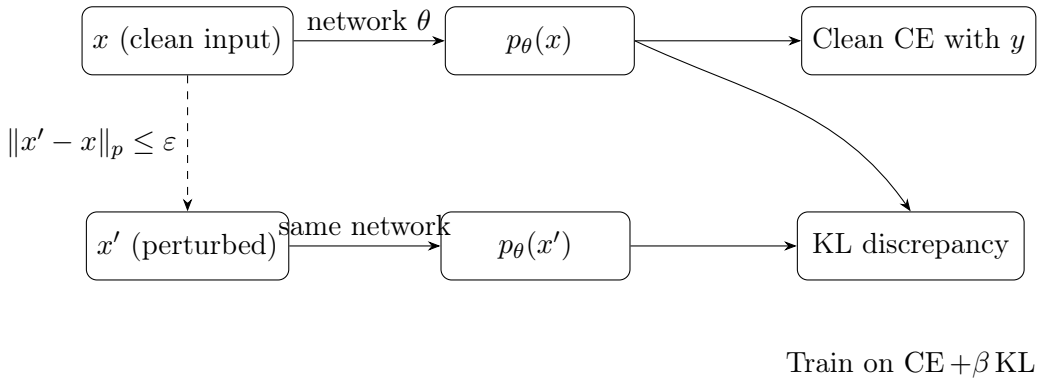


Figure 56: TRADES combines supervised clean-input training with adversarial consistency between two predictions produced by the same model.

21.6 How Prediction Stability Relates to Classification Robustness

The KL penalty controls a discrepancy between output distributions, while robustness is defined in terms of whether the maximizing class changes. The following argument makes the relationship precise when the clean prediction has a positive probability margin.

Definition 21.9 (Clean probability margin)**.** Suppose the clean predicted label is $y = \arg\max_k p_k$. Define its probability margin

$$\gamma(p, y) = p_y - \max_{j \neq y} p_j.$$

The prediction is unique precisely when $\gamma(p, y) > 0$.

Proposition 21.5 (Prediction stability under an ℓ_1 probability perturbation)**.** *Let $p, q \in \Delta^{K-1}$ and suppose y is the unique maximizing class of p with margin $\gamma = \gamma(p, y) > 0$. If*

$$\|p - q\|_1 < \gamma,$$

then y is also the unique maximizing class of q.

Proof. For any competing class $j \neq y$,

$$\begin{aligned} q_y - q_j &= (p_y - p_j) + (q_y - p_y) - (q_j - p_j) \\ &\geq \gamma - |q_y - p_y| - |q_j - p_j| \\ &\geq \gamma - \|p - q\|_1 > 0. \end{aligned}$$

Thus $q_y > q_j$ for every $j \neq y$, proving that the maximizing class remains y. □

Theorem 21.2 (A KL-based sufficient condition for local prediction stability)**.** *Let $p = p_\theta(x)$ have unique maximizing class y and margin $\gamma > 0$. If*

$$\sup_{x' \in \mathcal{B}_p(x,\varepsilon)} D_{\mathrm{KL}}\big(p_\theta(x) \| p_\theta(x')\big) < \frac{\gamma^2}{2},$$

then $f_\theta(x') = y$ for every $x' \in \mathcal{B}_p(x, \varepsilon)$.

Proof. Pinsker's inequality for probability vectors states that

$$\|p - q\|_1 \leq \sqrt{2 D_{\mathrm{KL}}(p \| q)}.$$

Apply the inequality with $q = p_\theta(x')$. Under the theorem's condition,

$$\begin{aligned} \|p_\theta(x) - p_\theta(x')\|_1 &\leq \sqrt{2 D_{\mathrm{KL}}\big(p_\theta(x) \| p_\theta(x')\big)} \\ &< \sqrt{2(\gamma^2/2)} = \gamma. \end{aligned}$$

The previous proposition then implies that the maximizing class cannot change anywhere in the allowed neighborhood. □

Remark 21.6 (Not a free robustness certificate). This theorem requires a *certified bound on the supremum* of KL divergence over the whole perturbation ball, as well as a positive clean margin. Merely observing a small KL divergence at one or several adversarial samples does not establish its hypothesis. The theorem explains the motivation for a consistency penalty; it does not transform heuristic PGD training into a formal certificate by itself.

21.7 Inner Optimization: Finding Discrepant Nearby Inputs

Fix parameters θ and a clean input x. Denote the clean probability vector by

$$p = p_\theta(x).$$

The inner TRADES problem is

$$\max_{x' \in \mathcal{B}_p(x,\varepsilon)} D_{\mathrm{KL}}\big(p \| p_\theta(x')\big).$$

For images with coordinates in $[0, 1]$, the feasible set for an ℓ_∞ attack is

$$\mathcal{C}(x) = \left\{ x' : \|x' - x\|_\infty \leq \varepsilon, \quad 0 \leq x'_j \leq 1 \right\}.$$

21.7.1 Differentiating the KL Objective

Let $u(x') = z_\theta(x') \in \mathbb{R}^K$ be the perturbed logits, and set $q = \mathrm{softmax}(u)$. During the input-ascent step the clean vector p is treated as fixed. Then

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= \sum_{k=1}^{K} p_k \log p_k - \sum_{k=1}^{K} p_k \log q_k \\ &= \sum_{k=1}^{K} p_k \log p_k - \sum_{k=1}^{K} p_k u_k + \log\left(\sum_{j=1}^{K} e^{u_j}\right), \end{aligned}$$

since $\sum_k p_k = 1$.

Proposition 21.6 (Input gradient of TRADES KL divergence)**.** *If the perturbed-logit map is differentiable with Jacobian $J_u(x') \in \mathbb{R}^{K\times d}$, then*

$$\nabla_u D_{\mathrm{KL}}(p\|q) = q - p, \qquad \nabla_{x'} D_{\mathrm{KL}}(p\|q) = J_u(x')^\top (q - p).$$

Proof. For the jth logit,

$$\begin{aligned} \frac{\partial}{\partial u_j} D_{\mathrm{KL}}(p\|q) &= -p_j + \frac{e^{u_j}}{\sum_{k=1}^{K} e^{u_k}} \\ &= q_j - p_j. \end{aligned}$$

The chain rule therefore gives

$$\nabla_{x'} D_{\mathrm{KL}}(p\|q) = J_u(x')^\top \nabla_u D_{\mathrm{KL}}(p\|q) = J_u(x')^\top (q - p).$$

□

Remark 21.7 (Why a random start is useful). If the inner optimization is initialized at $x' = x$, then $q = p$ and the exact KL input gradient is zero. Hence gradient ascent started exactly at the clean input does not move on its first iteration, in ordinary differentiable arithmetic. A small random initialization inside the allowed ball is therefore especially important for KL-based adversarial example generation.

21.7.2 Projected Gradient Ascent

Given a step size $\alpha > 0$, the projected ascent iteration is

$$x'_{t+1} = \Pi_{\mathcal{C}(x)}\left(x'_t + \alpha\, \mathrm{sign}\left(\nabla_{x'} D_{\mathrm{KL}}(p_\theta(x)\|p_\theta(x'_t))\right)\right),$$

where $\Pi_{\mathcal{C}(x)}$ is the Euclidean projection onto the feasible input set. The sign step is a convenient ℓ_∞ update; it is not standard Euclidean gradient ascent.

Proposition 21.7 (Projection onto an image ℓ_∞ ball)**.** *Let* $\mathcal{C}(x) = [0,1]^d \cap \{x' : \|x' - x\|_\infty \le \varepsilon\}$. *For any* $u \in \mathbb{R}^d$*, its Euclidean projection is coordinatewise*

$$[\Pi_{\mathcal{C}(x)}(u)]_j = \min\left\{\min(1, x_j + \varepsilon), \max\big(\max(0, x_j - \varepsilon), u_j\big)\right\}.$$

Proof. The constraints decouple into intervals

$$\max(0, x_j - \varepsilon) \le x'_j \le \min(1, x_j + \varepsilon).$$

Euclidean projection minimizes $\sum_j (x'_j - u_j)^2$ over the Cartesian product of these intervals. Minimization is therefore coordinatewise, and the projection of a scalar onto an interval is obtained by clipping it to that interval's endpoints. □

The inner optimization is nonconcave for ordinary neural networks. Multiple starts and more steps may discover more damaging perturbations, but neither guarantees the global maximum. Robustness evaluation should not rely only on the same attack settings used during training.

21.8 Outer Optimization and Gradients of the TRADES Loss

After producing an approximate maximizer x'_{adv}, we update the network parameters to minimize

$$\mathcal{L}_\theta(x, y, x'_{\text{adv}}) = \text{CE}(y, p_\theta(x)) + \beta D_{\text{KL}}(p_\theta(x) \| p_\theta(x'_{\text{adv}})).$$

Unlike the inner ascent step, both predictive distributions depend on θ in the outer objective. The adversarial input is usually treated as a fixed sample during the final parameter-gradient computation.

21.8.1 The Clean Cross-Entropy Gradient

If $z = z_\theta(x)$ and $p = \text{softmax}(z)$, then

$$\frac{\partial}{\partial z_j} \text{CE}(y, p) = p_j - \mathbf{1}_{\{j=y\}}.$$

This is the usual supervised-learning gradient from Lecture 5. It increases the correct-class probability and decreases the others through the softmax coupling.

21.8.2 Derivatives of KL with Respect to Both Logit Vectors

Denote clean logits by z and perturbed logits by u, with probability vectors $p = \text{softmax}(z)$ and $q = \text{softmax}(u)$. We already established

$$\frac{\partial}{\partial u_j} D_{\text{KL}}(p \| q) = q_j - p_j.$$

For the clean logits, the derivative is more subtle because the probabilities p_k occur both as coefficients and inside the logarithm.

Proposition 21.8 (Two-sided softmax–KL gradient)**.** *Let $D = D_{\mathrm{KL}}(p\|q)$, where $p = \mathrm{softmax}(z)$ and $q = \mathrm{softmax}(u)$. Then*

$$\frac{\partial D}{\partial u_j} = q_j - p_j,$$
$$\frac{\partial D}{\partial z_j} = p_j \left(\log \frac{p_j}{q_j} - D \right).$$

Proof. The first identity was proved above. For the second, differentiate $D = \sum_k p_k(\log p_k - \log q_k)$:

$$\begin{aligned}
\frac{\partial D}{\partial z_j} &= \sum_{k=1}^{K} \frac{\partial p_k}{\partial z_j} \left(\log \frac{p_k}{q_k} + 1 \right) \\
&= \sum_{k=1}^{K} p_k(\mathbf{1}_{\{k=j\}} - p_j) \left(\log \frac{p_k}{q_k} + 1 \right) \\
&= p_j \left(\log \frac{p_j}{q_j} + 1 \right) - p_j \sum_{k=1}^{K} p_k \left(\log \frac{p_k}{q_k} + 1 \right) \\
&= p_j \left(\log \frac{p_j}{q_j} + 1 \right) - p_j(D + 1) \\
&= p_j \left(\log \frac{p_j}{q_j} - D \right).
\end{aligned}$$

Here we used the softmax Jacobian $\partial p_k / \partial z_j = p_k(\mathbf{1}_{\{k=j\}} - p_j)$. □

Consequently, with $J_z(x) = \partial z_\theta(x)/\partial\theta$ and $J_u(x') = \partial z_\theta(x')/\partial\theta$ interpreted as parameter Jacobians, the full outer gradient for fixed x' is

$$\begin{aligned}
\nabla_\theta \mathcal{L}_\theta = {} & J_z(x)^\top (p - e_y) \\
& + \beta J_z(x)^\top \left[p \odot (\log(p/q) - D\mathbf{1}) \right] \\
& + \beta J_u(x')^\top (q - p),
\end{aligned}$$

where $\odot$ denotes componentwise multiplication, the logarithm and division are componentwise, and e_y is the yth coordinate vector.

Remark 21.8 (Stop-gradient conventions). During the *inner maximization*, $p_\theta(x)$ is held fixed relative to changes in x'. During the *outer minimization*, the mathematical TRADES objective generally differentiates through both $p_\theta(x)$ and $p_\theta(x')$ with respect to the network parameters, while treating the computed adversarial input as fixed. Detaching the clean probability vector during the outer step produces a related but different optimization gradient.

21.8.3 The Envelope View of an Inner Maximum

Proposition 21.9 (Envelope derivative under a unique maximizer)**.** *Let $F(\theta, \delta)$ be differentiable in θ and continuous in (θ, δ), with δ restricted to a fixed compact feasible set Δ. Suppose the maximizer*

$$\delta^*(\theta) = \arg\max_{\delta \in \Delta} F(\theta, \delta)$$

is unique at the parameter value under consideration and suitable regularity hypotheses of Danskin's theorem hold. Then

$$\nabla_\theta \max_{\delta \in \Delta} F(\theta, \delta) = \nabla_\theta F(\theta, \delta^*(\theta)).$$

Thus, for an *exact* maximizing perturbation, the derivative of the maximized value can be obtained by holding the maximizer fixed while differentiating the loss. This does not guarantee correct gradients for an inexact or locally optimized adversarial example, although the stop-gradient procedure is a common practical approximation.

21.9 Training Algorithm

A practical training epoch alternates adversarial input search and network parameter updates. Let $\mathcal{B}$ be a mini-batch, K_{PGD} the number of inner steps, α the inner step size, and η the outer learning rate.

1. Draw a mini-batch $\mathcal{B} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{B}$.
2. Compute clean distributions $p^{(i)} = p_\theta(x^{(i)})$.
3. Initialize each adversarial example within its constraint set, typically $x_0'^{(i)} = x^{(i)} + \delta_0^{(i)}$ with a random admissible perturbation.
4. For $t = 0, \ldots, K_{\mathrm{PGD}} - 1$, update

$$x_{t+1}'^{(i)} = \Pi_{\mathcal{C}(x^{(i)})}\Big(x_t'^{(i)} + \alpha \operatorname{sign}\big(abla_{x'} D_{\mathrm{KL}}(p^{(i)} \| p_\theta(x_t'^{(i)}))\big)\Big).$$

5. Form the mini-batch objective

$$\widehat{\mathcal{L}}_{\mathcal{B}}(\theta) = \frac{1}{B}\sum_{i=1}^{B}\Big[-\log p_{\theta, y^{(i)}}(x^{(i)}) + \beta D_{\mathrm{KL}}\big(p_\theta(x^{(i)}) \| p_\theta(x_{K_{\mathrm{PGD}}}'^{(i)})\big)\Big].$$

6. Update the network parameters using gradient descent or a gradient-based optimizer:

$$\theta \longleftarrow \theta - \eta \nabla_\theta \widehat{\mathcal{L}}_{\mathcal{B}}(\theta).$$

Remark 21.9 (Numerical conventions). The input domain must be respected during every adversarial step. For image tensors this often means clipping to both the ℓ_∞ ball and $[0, 1]^d$. The KL term can be implemented with log-softmax for numerical stability. During PGD, clean probabilities are detached from the input-ascent computation; during the final network update, the adversarial sample is detached as an input, but the clean and perturbed forward passes are differentiated with respect to model weights.

21.10 A Detailed Binary Logistic Example

Consider a linear binary classifier with score

$$s_w(x) = w^\top x + b, \qquad p_w(+1 \mid x) = \sigma(s_w(x)), \quad \sigma(u) = \frac{1}{1+e^{-u}}.$$

Let $y \in \{-1, +1\}$ and take a clean input with positive signed margin $m = y(w^\top x + b) > 0$. The negative log-likelihood is

$$\mathrm{CE}(y, x) = \log(1 + e^{-m}).$$

For an ℓ_∞ perturbation with no further box constraint,

$$\begin{aligned}\min_{\|\delta\|_\infty \le \varepsilon} y(w^\top(x+\delta)+b) &= m + \min_{\|\delta\|_\infty \le \varepsilon} yw^\top\delta \\ &= m - \varepsilon\|w\|_1.\end{aligned}$$

Consequently, the exact worst-case logistic classification loss is

$$\max_{\|\delta\|_\infty \le \varepsilon} \log(1 + e^{-y(w^\top(x+\delta)+b)}) = \log(1 + e^{-m+\varepsilon\|w\|_1}).$$

The ordinary adversarial training objective therefore directly penalizes a small worst-case signed margin.

TRADES uses a different inner objective. If $p = \sigma(s_w(x))$ and $q = \sigma(s_w(x+\delta))$, then the Bernoulli KL divergence is

$$D_{\mathrm{KL}}(p\|q) = p\log\frac{p}{q} + (1-p)\log\frac{1-p}{1-q}.$$

The perturbed score is

$$s_w(x+\delta) = s_w(x) + w^\top\delta, \qquad |w^\top\delta| \le \varepsilon\|w\|_1.$$

Writing $a = s_w(x)$, the inner objective is thus reduced to

$$\max_{t \in [-\varepsilon\|w\|_1, \varepsilon\|w\|_1]} D_{\mathrm{KL}}\big(\mathrm{Bern}(\sigma(a))\|\,\mathrm{Bern}(\sigma(a+t))\big).$$

Proposition 21.10 (Endpoint attainment in a binary linear TRADES inner problem)**.** *For fixed $a \in \mathbb{R}$ and $p = \sigma(a)$, define*

$$H(t) = D_{\mathrm{KL}}\big(\mathrm{Bern}(p)\|\,\mathrm{Bern}(\sigma(a+t))\big).$$

Then H is convex in t, has its unique minimum at $t = 0$, and its maximum on a closed interval $[-c, c]$ occurs at one or both endpoints.

Proof. Put $q(t) = \sigma(a+t)$. Differentiating the Bernoulli KL with respect to the perturbed logit gives

$$\begin{aligned}H'(t) &= q(t) - p, \\ H''(t) &= q(t)(1-q(t)) > 0.\end{aligned}$$

Thus H is strictly convex and $H'(0) = 0$. A convex function on a compact interval attains a maximum at an endpoint: for each interior $t = \lambda(-c) + (1-\lambda)c$,

$$H(t) \le \lambda H(-c) + (1-\lambda)H(c) \le \max\{H(-c), H(c)\}.$$

Therefore it is sufficient to compare the endpoint divergences. □

This example emphasizes the conceptual difference between the two training objectives. Standard adversarial training directly searches for a perturbation that increases the label-dependent loss; TRADES searches for a perturbation that changes the predictive distribution relative to the clean input, regardless of its true label during the inner maximization.

21.11 Choosing the Robustness Parameters

Three principal controls influence the behavior of the algorithm:

- *Perturbation radius ε.* Determines the neighborhood in which output stability is encouraged, for a specific input norm and scaling convention.
- *Trade-off coefficient β.* Determines the relative weight assigned to local prediction consistency. At $\beta = 0$, the objective reduces to clean cross-entropy training.
- *Inner optimization strength.* The number of ascent steps, step size, and random restarts affect how accurately the numerical procedure finds high-KL points within the specified ball.

Remark 21.10 (Effect of very large β). A large consistency coefficient encourages nearly equal predictions across local neighborhoods, but excessive smoothing can harm classification if neighborhoods from different classes overlap or if the model lacks capacity. Constant predictions are perfectly invariant, yet they are generally not useful classifiers. The clean loss term is therefore essential.

Remark 21.11 (Loss values are not a direct accuracy guarantee). The evaluation of robust accuracy is a separate adversarial testing problem. Reporting only the adversarial training objective or the performance against a weak attack can give a misleading impression of robustness. Evaluations should state the perturbation norm, budget, input scaling, attack strength, and whether a formal certificate was computed.

21.12 Robust Risk, Empirical Risk, and Generalization

At training time, we observe a finite sample, and the empirical TRADES objective approximates a population expectation. A low empirical value need not imply low population robust risk unless the learned function generalizes.

Definition 21.10 (Clean and robust accuracies)**.** Define

$$\mathrm{Acc}_{\mathrm{clean}}(\theta) = 1 - R_{\mathrm{nat}}(\theta), \qquad \mathrm{Acc}_{\mathrm{rob}}(\theta;\varepsilon) = 1 - R_{\mathrm{rob}}(\theta;\varepsilon).$$

Then $\mathrm{Acc}_{\mathrm{rob}}(\theta;\varepsilon) \le \mathrm{Acc}_{\mathrm{clean}}(\theta)$ for a fixed model evaluated under the same distribution.

The inequality does not mean that adversarial training must reduce clean accuracy relative to a differently trained model. It simply says that, for any fixed classifier, correctly classifying *every* permitted perturbation is a stronger condition than correctly classifying the original point.

Proposition 21.11 (Robust risk under a union of threat models)**.** *Let $\mathcal{B}_1(x)$ and $\mathcal{B}_2(x)$ contain x, and define $\mathcal{B}_\cup(x) = \mathcal{B}_1(x) \cup \mathcal{B}_2(x)$. Then*

$$R_{\mathrm{rob}}(\theta;\mathcal{B}_\cup) \ge \max\{R_{\mathrm{rob}}(\theta;\mathcal{B}_1), R_{\mathrm{rob}}(\theta;\mathcal{B}_2)\}.$$

Proof. The event that some perturbation in $\mathcal{B}_j(X)$ causes a misclassification is contained in the event that some perturbation in the union causes a misclassification. Probabilities preserve this inclusion for both $j = 1, 2$. □

Thus the robustness claim depends on exactly which perturbations are allowed. A model can be robust under one norm and budget but not another, or robust to a restricted family of perturbations but not to unrestricted semantic changes.

21.13 Numerical Example: Robust and Clean Accuracy

Suppose a classifier is evaluated on a collection of 100 observations. On the clean inputs, it misclassifies 8 points. Among the remaining 92 correctly classified points, an allowed perturbation changes the prediction for 17 additional points. Then

$$\begin{aligned}
\widehat{R}_{\mathrm{nat}} &= \frac{8}{100} = 0.08, \\
\widehat{R}_{\mathrm{bdy}} &= \frac{17}{100} = 0.17, \\
\widehat{R}_{\mathrm{rob}} &= \frac{8+17}{100} = 0.25.
\end{aligned}$$

Hence

$$\widehat{\mathrm{Acc}}_{\mathrm{clean}} = 0.92, \qquad \widehat{\mathrm{Acc}}_{\mathrm{rob}} = 0.75.$$

If a defense reduces boundary-vulnerable points from 17 to 5 while causing the clean-error count to increase from 8 to 10, then

$$\begin{aligned}
\widehat{R}_{\mathrm{nat,new}} &= 0.10, \\
\widehat{R}_{\mathrm{bdy,new}} &= 0.05, \\
\widehat{R}_{\mathrm{rob,new}} &= 0.15.
\end{aligned}$$

The clean accuracy falls from 92% to 90%, while robust accuracy improves from 75% to 85%. These are hypothetical illustrative counts, not empirical results from the handwritten lecture.

21.14 Worked Exercises

Exercise 1.

Prove that, for a fixed binary classifier, robust zero–one risk equals natural zero–one risk plus the probability of an adversarial label change at a correctly classified point.

Solution. Let $A = \{f(X) \neq Y\}$ and $B = \{\exists X' \in \mathcal{B}(X, \varepsilon) : f(X') \neq Y\}$. Because X belongs to its ball, $A \subseteq B$. Therefore,

$$\begin{aligned} R_{\mathrm{rob}} &= \mathbb{P}(B) \\ &= \mathbb{P}(A) + \mathbb{P}(B \cap A^c) \\ &= R_{\mathrm{nat}} + \mathbb{P}\big(f(X) = Y, \exists X' \in \mathcal{B}(X, \varepsilon) : f(X') \neq f(X)\big). \end{aligned}$$

The last probability is precisely the boundary error. ∎

Exercise 2.

For $p = \mathrm{softmax}(z)$ and $q = \mathrm{softmax}(u)$, derive the gradient of $D_{\mathrm{KL}}(p\|q)$ with respect to the perturbed logits u, treating p as fixed. What happens if the perturbed input is initialized at the clean input?

Solution. Since $\log q_k = u_k - \log \sum_j e^{u_j}$,

$$\begin{aligned} D_{\mathrm{KL}}(p\|q) &= \sum_k p_k \log p_k - \sum_k p_k u_k + \log \sum_j e^{u_j}, \\ \frac{\partial D}{\partial u_j} &= -p_j + \frac{e^{u_j}}{\sum_k e^{u_k}} = q_j - p_j. \end{aligned}$$

When $x' = x$, we have $q = p$, hence $\nabla_u D = 0$ and, by the chain rule, $\nabla_{x'} D = 0$ at the initialization. This motivates a small random start for KL-based projected ascent. ∎

Exercise 3.

Let $p = (0.8, 0.15, 0.05)$ be a clean three-class prediction. Find a sufficient upper bound on $D_{\mathrm{KL}}(p\|q)$ that guarantees the predicted class remains class 1.

Solution. The clean top-two probability margin is

$$\gamma = 0.8 - 0.15 = 0.65.$$

By Pinsker's inequality, it suffices that

$$\sqrt{2 D_{\mathrm{KL}}(p\|q)} < 0.65,$$

or equivalently,

$$D_{\mathrm{KL}}(p\|q) < \frac{(0.65)^2}{2} = 0.21125.$$

This is a sufficient condition, not a necessary condition; the class might remain unchanged even for a larger KL divergence. ■

Exercise 4.

Let $x = (0.2, 0.9)$, $\varepsilon = 0.15$, and an unprojected update produce $u = (0.5, 1.2)$. Compute the projected point in $[0,1]^2 \cap \mathcal{B}_\infty(x, \varepsilon)$.

Solution. The allowed intervals are

$$x_1' \in [0.2 - 0.15,\ 0.2 + 0.15] = [0.05, 0.35],$$
$$x_2' \in [0.9 - 0.15,\ \min(1, 0.9 + 0.15)] = [0.75, 1].$$

Projecting $u = (0.5, 1.2)$ coordinatewise gives

$$\Pi_{\mathcal{C}(x)}(u) = (0.35, 1).$$

The result stays within both the pixel range and the perturbation budget. ■

Exercise 5.

For Bernoulli probabilities $p = \sigma(a)$ and $q(t) = \sigma(a + t)$, consider

$$H(t) = D_{\mathrm{KL}}\big(\mathrm{Bern}(p) \| \,\mathrm{Bern}(q(t))\big).$$

Show that H is convex and has its minimum at $t = 0$.

Solution. Expanding the KL gives

$$H(t) = p \log \frac{p}{q(t)} + (1 - p) \log \frac{1 - p}{1 - q(t)}.$$

Because $q'(t) = q(t)(1 - q(t))$,

$$\begin{aligned} H'(t) &= -\frac{p}{q(t)} q'(t) + \frac{1 - p}{1 - q(t)} q'(t) \\ &= -p(1 - q(t)) + (1 - p)q(t) \\ &= q(t) - p, \\ H''(t) &= q(t)(1 - q(t)) > 0. \end{aligned}$$

Since $q(0) = p$, $H'(0) = 0$, so strict convexity implies $t = 0$ is the unique minimizer. ■

Exercise 6.

A model has 1000 clean test examples. It incorrectly classifies 40 of them. Among the 960 cleanly correct examples, 110 have a successful perturbation within the threat model. Compute natural risk, boundary risk, and robust accuracy. Why should the 110 not be divided by 960 when using the decomposition?

Solution. Each risk in the decomposition is an unconditional probability over the original test distribution:

$$\widehat{R}_{\mathrm{nat}} = \frac{40}{1000} = 0.04,$$
$$\widehat{R}_{\mathrm{bdy}} = \frac{110}{1000} = 0.11,$$
$$\widehat{R}_{\mathrm{rob}} = \frac{150}{1000} = 0.15.$$

Thus the robust accuracy is $1 - 0.15 = 0.85$. The quantity $110/960$ estimates the *conditional* fraction of vulnerable examples among cleanly correct inputs, not the unconditional boundary error in the exact decomposition. ■

Exercise 7.

Explain why an attack that takes sign-gradient steps from the exact clean input can fail to move when the inner objective is KL divergence, even though nearby higher-KL points may exist.

Solution. For $D(x') = D_{\mathrm{KL}}(p_\theta(x) \| p_\theta(x'))$, the gradient is

$$\nabla_{x'} D(x') = J_{z_\theta}(x')^\top (p_\theta(x') - p_\theta(x)).$$

At $x' = x$, the probability difference vanishes exactly, so the gradient is zero. A deterministic update based only on that gradient therefore remains at x. The point can nevertheless be a strict local minimum of KL with larger values nearby. Random initialization moves the iterate away from this stationary point so that subsequent ascent steps can exploit a nonzero gradient. ■

Exercise 8.

Suppose the inner adversarial search only finds x'_{adv} satisfying

$$D_\theta(x, x'_{\mathrm{adv}}) = 0.01,$$

while another feasible point has KL divergence 0.4. Is the first value a valid upper bound on the local divergence? Could it certify stability with a clean probability margin of 0.5?

Solution. No. The observed value 0.01 is a *lower bound* on the supremum over feasible perturbations, not an upper bound:

$$\sup_{x' \in \mathcal{B}(x,\varepsilon)} D_\theta(x, x') \geq 0.4.$$

A sufficient Pinsker-based stability condition with margin 0.5 would require

$$\sup_{x' \in \mathcal{B}(x,\varepsilon)} D_\theta(x, x') < \frac{0.5^2}{2} = 0.125.$$

The existence of a feasible point with divergence 0.4 shows that the sufficient condition fails. It does not itself prove a label change, but the value 0.01 cannot certify stability. ■

22 Lecture 23: Generative Adversarial Networks and Adversarial Generative Learning

22.1 From Discriminative Learning to Generative Learning

The previous two lectures studied *adversarial attacks and defenses*: an adversary changes an input in order to make a classifier behave incorrectly, while the classifier is trained to withstand such changes. We now consider a different, but related, kind of competition. In a *generative adversarial network* (GAN), one network produces synthetic observations and another network attempts to distinguish synthetic observations from genuine training data. Both networks improve through their interaction.

The handwritten lecture introduces precisely these two components: a *generator* G, which transforms a noise vector into a synthetic sample, and a *discriminator* D, which assigns a high score to real observations and a low score to generated observations. Unlike adversarial perturbation methods, the generator does not start from a particular real observation and choose a nearby point. Instead, it starts with a latent random variable and learns to produce a distribution of new observations.

Definition 22.1 (Data and latent distributions)**.** Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the observation space and $\mathcal{Z} \subseteq \mathbb{R}^k$ the latent space. We distinguish two probability distributions:

- P_{data}, the unknown distribution generating real observations $X \in \mathcal{X}$;
- P_Z, a simple, prescribed distribution for the latent input $Z \in \mathcal{Z}$, commonly $\mathcal{N}(0, I_k)$ or a uniform distribution.

Training observations $x^{(1)}, \dots, x^{(N)}$ are sampled from P_{data}. Independently, latent vectors $z^{(1)}, \dots, z^{(M)}$ can be drawn from P_Z whenever needed.

Definition 22.2 (Generator and generated distribution)**.** A generator with parameters θ is a measurable map

$$G_\theta : \mathcal{Z} \longrightarrow \mathcal{X}.$$

Given $Z \sim P_Z$, it produces a generated observation $\widetilde{X} = G_\theta(Z)$. The induced distribution is the *pushforward* measure

$$P_\theta = (G_\theta)_{\#} P_Z, \qquad P_\theta(A) = P_Z\big(G_\theta^{-1}(A)\big), \quad A \subseteq \mathcal{X} \text{ measurable.}$$

Whenever a generated density exists, it is denoted by p_θ; the corresponding data density, when it exists, is p_{data}.

The pushforward formulation is important: a GAN need not provide a tractable formula for $p_\theta(x)$. It is an *implicit generative model*, specified by its sampling procedure $Z \mapsto G_\theta(Z)$. The generator may have fewer latent dimensions than the observation space; in that case

the generated distribution can be supported on a lower-dimensional set even when the data distribution has a density in $\mathbb{R}^d$.

Definition 22.3 (Discriminator)**.** A discriminator with parameters ϕ is a measurable function

$$D_\phi : \mathcal{X} \longrightarrow (0,1),$$

where $D_\phi(x)$ is interpreted as the network's predicted probability that x came from the real-data source rather than the generator, under the balanced real-versus-fake classification experiment. In practice it is often written as

$$D_\phi(x) = \sigma\big(a_\phi(x)\big), \qquad \sigma(u) = \frac{1}{1+e^{-u}},$$

with a real-valued logit $a_\phi(x)$.

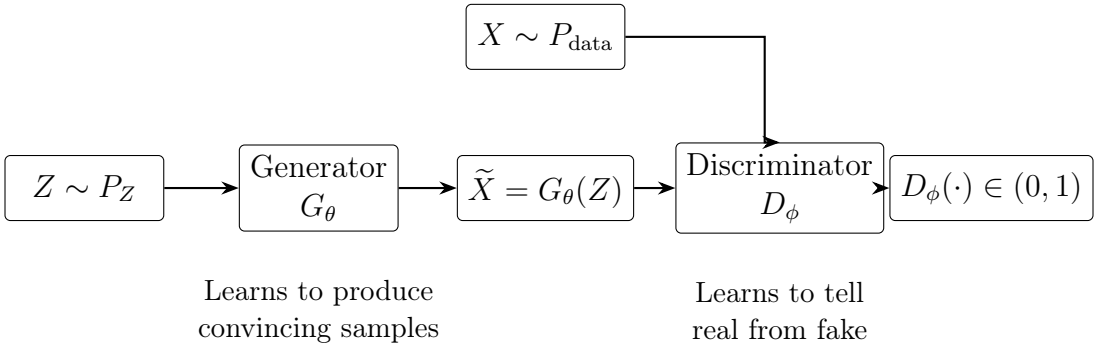


Figure 57: Basic GAN architecture. The generator transforms latent noise into a synthetic observation; the discriminator processes real or generated observations. Their objectives compete during training.

Remark 22.1 (What it means to fool the discriminator)*.* A small discriminator score $D_\phi(\widetilde{x})$ suggests that the discriminator recognizes $\widetilde{x}$ as generated; a large score suggests that the generated observation appears real to that discriminator. This score is not a general-purpose measure of physical similarity between $\widetilde{x}$ and any particular real observation. It depends on the learned discriminator and the current distributions.

22.2 The Real-versus-Fake Classification Problem

To train a discriminator, present it with a balanced mixture of examples: half are drawn from P_{data} and labeled $Y = 1$, while the remainder are drawn from P_θ and labeled $Y = 0$. Conditional on the example $X = x$, the discriminator predicts $D_\phi(x) \approx \mathbb{P}(Y = 1 \mid X = x)$.

Definition 22.4 (Binary cross-entropy loss)**.** For a target $y \in \{0,1\}$ and a predicted real probability $d \in (0,1)$, the binary cross-entropy is

$$\ell_{\mathrm{BCE}}(d,y) = -y \log d - (1-y)\log(1-d).$$

In particular,

$$\begin{aligned}\ell_{\mathrm{BCE}}(d,1) &= -\log d,\\ \ell_{\mathrm{BCE}}(d,0) &= -\log(1-d).\end{aligned}$$

Fix a generator G_θ. The discriminator's expected classification loss is

$$\begin{aligned}C_D(\phi;\theta) &= \mathbb{E}_{X\sim P_{\mathrm{data}}}\big[-\log D_\phi(X)\big] + \mathbb{E}_{Z\sim P_Z}\big[-\log(1-D_\phi(G_\theta(Z)))\big]\\ &= -\mathbb{E}_{X\sim P_{\mathrm{data}}}\log D_\phi(X) - \mathbb{E}_{\widetilde{X}\sim P_\theta}\log(1-D_\phi(\widetilde{X})).\end{aligned}$$

The factor $1/2$ from balanced class probabilities has been omitted because multiplying the loss by a positive constant does not change its minimizers. The discriminator wants to make C_D small: real inputs should receive scores near one and generated inputs scores near zero.

Definition 22.5 (Original GAN minimax value)**.** The original GAN value function is

$$V(\theta,\phi) = \mathbb{E}_{X\sim P_{\mathrm{data}}}\log D_\phi(X) + \mathbb{E}_{Z\sim P_Z}\log\big(1-D_\phi(G_\theta(Z))\big).$$

The idealized adversarial learning objective is

$$\min_\theta \max_\phi V(\theta,\phi).$$

Since $C_D = -V$, maximizing V with respect to ϕ is equivalent to minimizing binary cross-entropy with respect to ϕ.

For a fixed discriminator, the generator can affect only the second expectation. In the *minimax* (also called *saturating*) formulation it minimizes

$$L_G^{\mathrm{sat}}(\theta;\phi) = \mathbb{E}_{Z\sim P_Z}\log\big(1-D_\phi(G_\theta(Z))\big).$$

Because $\log(1-d)$ decreases as d increases, minimizing this loss encourages generated inputs to obtain larger real-probability scores.

Remark 22.2 (Sign conventions). In the handwritten lecture the discriminator performs gradient descent on C_D, whereas the generator is encouraged to move in the opposite direction of the discriminator's classification objective. These are consistent descriptions of the same original minimax game:

$$\begin{aligned}\phi &\leftarrow \phi - \eta_D \nabla_\phi C_D(\phi;\theta) = \phi + \eta_D \nabla_\phi V(\theta,\phi),\\ \theta &\leftarrow \theta + \eta_G \nabla_\theta C_D(\phi;\theta) = \theta - \eta_G \nabla_\theta V(\theta,\phi).\end{aligned}$$

The generator step treats ϕ as fixed, and the discriminator step treats θ as fixed. This alternation is essential when computing the partial gradients of the two objectives.

22.2.1 Empirical objectives and Monte Carlo estimation

Let $x^{(1)}, \dots, x^{(m)}$ be a mini-batch of real examples and $z^{(1)}, \dots, z^{(m)}$ independent latent samples. The empirical value is

$$\widehat{V}_m(\theta, \phi) = \frac{1}{m} \sum_{i=1}^{m} \log D_\phi(x^{(i)}) + \frac{1}{m} \sum_{i=1}^{m} \log\bigl(1 - D_\phi(G_\theta(z^{(i)}))\bigr).$$

If the required expectations and gradients are integrable, differentiation and expectation can be interchanged, and the mini-batch gradients are unbiased estimates of the gradients of V when the samples are freshly drawn from their stated distributions. Sampling uniformly from a fixed finite training dataset instead targets the empirical data distribution.

22.3 The Optimal Discriminator for a Fixed Generator

The objective of the discriminator can be solved exactly in an idealized setting in which it may represent an arbitrary measurable function taking values in $[0, 1]$. The resulting formula explains why a GAN compares whole distributions rather than paired individual samples.

Theorem 22.1 (Optimal discriminator)**.** *Suppose P_{data} and P_θ admit densities p and q, respectively, with respect to a common reference measure μ. For a fixed G_θ, the maximizer of*

$$V(D; G_\theta) = \int \bigl[p(x) \log D(x) + q(x) \log(1 - D(x))\bigr] \, \mu(\mathrm{d}x)$$

over all measurable $D : \mathcal{X} \to [0, 1]$ is, wherever $p(x) + q(x) > 0$,

$$D^*(x) = \frac{p(x)}{p(x) + q(x)}.$$

At points with $p(x) = q(x) = 0$, the value of $D(x)$ is irrelevant to the objective. At points where both densities are positive, the optimizer is unique.

Proof. Since $D(x)$ appears only in the integrand at the same point x, it suffices to maximize the scalar function

$$F_x(d) = p(x) \log d + q(x) \log(1 - d), \qquad 0 \le d \le 1.$$

For $p(x), q(x) > 0$, calculate

$$F_x'(d) = \frac{p(x)}{d} - \frac{q(x)}{1 - d},$$
$$F_x''(d) = -\frac{p(x)}{d^2} - \frac{q(x)}{(1 - d)^2} < 0.$$

Thus F_x is strictly concave. Its unique stationary point satisfies

$$0 = \frac{p(x)}{d} - \frac{q(x)}{1 - d},$$

$$
\begin{aligned}
p(x)(1-d) &= q(x)d, \\
p(x) &= (p(x)+q(x))d, \\
d &= \frac{p(x)}{p(x)+q(x)}.
\end{aligned}
$$

Strict concavity makes this the unique maximum. If $p(x) > 0$ and $q(x) = 0$, the maximum occurs at $d = 1$; if $p(x) = 0$ and $q(x) > 0$, it occurs at $d = 0$, agreeing with the same formula. Values multiplied by zero are interpreted using $0 \log 0 = 0$. Choosing these pointwise maximizers measurably maximizes the integral. □

Remark 22.3 (Conditional probability interpretation). With equal prior probabilities for the real and generated classes, Bayes' rule gives

$$
\begin{aligned}
\mathbb{P}(Y = 1 \mid X = x) &= \frac{p(x)\mathbb{P}(Y=1)}{p(x)\mathbb{P}(Y=1) + q(x)\mathbb{P}(Y=0)} \\
&= \frac{p(x)/2}{p(x)/2 + q(x)/2} = \frac{p(x)}{p(x)+q(x)}.
\end{aligned}
$$

Hence the optimal discriminator is exactly the Bayes classifier for distinguishing the two sources. If one uses unequal class priors, the corresponding formula contains those priors.

Proposition 22.1 (Density-ratio form)**.** *Where $p(x), q(x) > 0$, the optimal discriminator obeys*

$$
\frac{D^*(x)}{1 - D^*(x)} = \frac{p(x)}{q(x)}, \qquad \log \frac{D^*(x)}{1 - D^*(x)} = \log p(x) - \log q(x).
$$

Proof. Substitution of $D^*(x) = p(x)/(p(x) + q(x))$ yields

$$
\frac{D^*(x)}{1 - D^*(x)} = \frac{p(x)/(p(x)+q(x))}{q(x)/(p(x)+q(x))} = \frac{p(x)}{q(x)}.
$$

Taking logarithms gives the second expression. □

The density-ratio interpretation makes the distribution-level competition transparent: $D^*(x)$ exceeds $1/2$ precisely when real data are more likely than generated data near x, as measured by the densities. If $p = q$, the optimal discriminator cannot distinguish the two sources and returns $1/2$.

22.4 The Jensen–Shannon Divergence and the GAN Equilibrium

We now evaluate the value function at the optimal discriminator. This produces an exact connection to a statistical divergence.

Definition 22.6 (Kullback–Leibler and Jensen–Shannon divergences)**.** For probability measures P, Q with densities p, q, define

$$D_{\mathrm{KL}}(P\|Q) = \int p(x) \log \frac{p(x)}{q(x)} \, \mu(\mathrm{d}x),$$

with the usual absolute-continuity conventions. The *Jensen–Shannon divergence* is

$$D_{\mathrm{JS}}(P\|Q) = \frac{1}{2} D_{\mathrm{KL}}(P\|M) + \frac{1}{2} D_{\mathrm{KL}}(Q\|M), \qquad M = \frac{1}{2}(P+Q).$$

Unlike $D_{\mathrm{KL}}(P\|Q)$, the Jensen–Shannon divergence is symmetric in P and Q and always lies in $[0, \log 2]$, using natural logarithms.

Theorem 22.2 (Optimal GAN value as a Jensen–Shannon divergence)**.** *Under the assumptions of the optimal-discriminator theorem, let P_θ denote the generated distribution. Then*

$$\sup_D V(D; G_\theta) = -\log 4 + 2 D_{\mathrm{JS}}(P_{\mathrm{data}} \| P_\theta).$$

Consequently, if the generator family can realize P_{data}, the global optimum of the idealized minimax problem is $-\log 4$, attained when $P_\theta = P_{\mathrm{data}}$.

Proof. Let $p = p_{\mathrm{data}}$, $q = p_\theta$, and $m = (p+q)/2$. Substituting the optimal discriminator yields

$$\begin{aligned}
V(D^*; G_\theta) &= \int p(x) \log \frac{p(x)}{p(x)+q(x)} \, \mu(\mathrm{d}x) \\
&\quad + \int q(x) \log \frac{q(x)}{p(x)+q(x)} \, \mu(\mathrm{d}x) \\
&= \int p(x) \log \frac{p(x)}{2m(x)} \, \mu(\mathrm{d}x) \\
&\quad + \int q(x) \log \frac{q(x)}{2m(x)} \, \mu(\mathrm{d}x) \\
&= \int p(x) \log \frac{p(x)}{m(x)} \, \mu(\mathrm{d}x) - \log 2 \int p(x) \, \mu(\mathrm{d}x) \\
&\quad + \int q(x) \log \frac{q(x)}{m(x)} \, \mu(\mathrm{d}x) - \log 2 \int q(x) \, \mu(\mathrm{d}x) \\
&= D_{\mathrm{KL}}(P_{\mathrm{data}}\|M) + D_{\mathrm{KL}}(P_\theta\|M) - 2\log 2 \\
&= 2 D_{\mathrm{JS}}(P_{\mathrm{data}}\|P_\theta) - \log 4.
\end{aligned}$$

Both KL terms are nonnegative by Gibbs' inequality; hence $D_{\mathrm{JS}} \geq 0$. Equality requires both distributions to equal M, which is equivalent to $P_{\mathrm{data}} = P_\theta$. Therefore the minimum optimal-discriminator value is $-\log 4$ exactly when the generator matches the data distribution. $\square$

Proposition 22.2 (Why the discriminator is uncertain at distribution matching)**.** *If $P_\theta = P_{\text{data}}$, then an optimal discriminator is*

$$D^*(x) = \frac{1}{2}$$

on the common support, and its expected classification loss in the unscaled convention is $C_D = \log 4$. No classifier can infer from x alone which of two identically distributed sources produced it when the sources have equal prior probabilities.

Proof. With $p = q$, the optimal-discriminator formula gives $D^* = p/(2p) = 1/2$ wherever $p > 0$. Consequently,

$$\begin{aligned} C_D &= -\mathbb{E}_{P_{\text{data}}} \log(1/2) - \mathbb{E}_{P_\theta} \log(1/2) \\ &= \log 2 + \log 2 = \log 4. \end{aligned}$$

The source label is independent of the observed value under identical conditional distributions, so its posterior probability is the class prior $1/2$. □

Remark 22.4 (A global result, not a convergence guarantee). The idealized theorem assumes an unrestricted discriminator and a generator capable of matching the data law. It identifies the distribution-level global optimum, but does not establish that alternating gradient optimization will find it. Finite data, restricted architectures, nonconvex parameterizations, inaccurate discriminator optimization, and numerical instability all affect actual learning.

22.5 Generator Gradients and the Saturation Problem

The preceding distributional results explain what a GAN would learn under ideal optimization. To implement the model, however, we need gradients through the discriminator and then through the generator. These gradients also reveal why a literal minimax generator objective can train poorly when the discriminator becomes very accurate.

Definition 22.7 (Saturating and non-saturating generator objectives)**.** For a fixed discriminator D_ϕ, the original generator objective is

$$L_G^{\text{sat}}(\theta; \phi) = \mathbb{E}_Z \log\big(1 - D_\phi(G_\theta(Z))\big).$$

A widely used alternative is the *non-saturating generator loss*

$$L_G^{\text{ns}}(\theta; \phi) = -\mathbb{E}_Z \log D_\phi(G_\theta(Z)).$$

Both are minimized by raising discriminator scores on generated observations, but their derivatives are different when those scores are very small. The non-saturating objective changes the generator's optimization rule; it is not algebraically identical to the original zero-sum minimax objective.

Proposition 22.3 (Gradient signals at the discriminator logit)**.** *Write $D_\phi(x) = \sigma(a_\phi(x))$ and let a be the discriminator logit at a generated observation. Then*

$$\frac{\mathrm{d}}{\mathrm{d}a}\log(1-\sigma(a)) = -\sigma(a),$$
$$\frac{\mathrm{d}}{\mathrm{d}a}\big[-\log\sigma(a)\big] = -(1-\sigma(a)).$$

Thus when $D_\phi(G_\theta(z)) \approx 0$, the scalar gradient factor of the saturating loss approaches zero, whereas that of the non-saturating loss approaches -1.

Proof. Since $\sigma'(a) = \sigma(a)(1-\sigma(a))$, the chain rule gives

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}a}\log(1-\sigma(a)) &= -\frac{\sigma'(a)}{1-\sigma(a)}\\ &= -\frac{\sigma(a)(1-\sigma(a))}{1-\sigma(a)}\\ &= -\sigma(a),\\ \frac{\mathrm{d}}{\mathrm{d}a}[-\log\sigma(a)] &= -\frac{\sigma'(a)}{\sigma(a)}\\ &= -(1-\sigma(a)).\end{aligned}$$

The limits follow as $\sigma(a) \downarrow 0$. □

The complete generator gradients, rather than only their scalar factors, are obtained by composing Jacobians. Let $J_\theta G_\theta(z)$ be the Jacobian of the generator output with respect to its parameters, and let $\nabla_x a_\phi(x)$ be the input gradient of the discriminator logit. Under differentiability and the usual interchange conditions,

$$\nabla_\theta L_G^{\mathrm{sat}} = -\mathbb{E}_Z\left[D_\phi(G_\theta(Z))J_\theta G_\theta(Z)^\top \nabla_x a_\phi(G_\theta(Z))\right],$$
$$\nabla_\theta L_G^{\mathrm{ns}} = -\mathbb{E}_Z\left[(1-D_\phi(G_\theta(Z)))J_\theta G_\theta(Z)^\top \nabla_x a_\phi(G_\theta(Z))\right].$$

Remark 22.5 (What non-saturation does not guarantee). The alternative loss removes the particular small factor $D_\phi(G_\theta(Z))$ associated with an accurate sigmoid discriminator. It does not ensure a useful gradient if $J_\theta G_\theta(Z)$ is degenerate, the discriminator logit is locally flat as a function of x, or optimization is otherwise ill conditioned.

22.5.1 Discriminator gradients

For a data example x and fake example $\widetilde{x}$, define $d_r = D_\phi(x)$ and $d_f = D_\phi(\widetilde{x})$. The binary cross-entropy derivatives with respect to logits are

$$\frac{\partial}{\partial a_r}[-\log\sigma(a_r)] = d_r - 1,$$
$$\frac{\partial}{\partial a_f}[-\log(1-\sigma(a_f))] = d_f.$$

Therefore, for a finite real/fake batch, holding θ fixed,

$$\nabla_\phi \widehat{C}_D = \frac{1}{m}\sum_{i=1}^{m}(D_\phi(x^{(i)}) - 1)\nabla_\phi a_\phi(x^{(i)}) + \frac{1}{m}\sum_{i=1}^{m} D_\phi(G_\theta(z^{(i)}))\nabla_\phi a_\phi(G_\theta(z^{(i)})).$$

This formula clarifies that discriminator training pushes real logits upward and fake logits downward. When computing it, generated images may be treated as constants: no generator parameter gradient is needed for this update.

22.6 Training GANs by Alternating Optimization

GAN training differs from ordinary supervised learning because the objective seen by either network changes as the other network improves. A common practical algorithm repeatedly updates the discriminator using real and generated mini-batches and then updates the generator through the trained discriminator.

Definition 22.8 (Alternating GAN updates)**.** Fix step sizes $\eta_D, \eta_G > 0$ and a discriminator-update count $k_D \geq 1$. At iteration t, one alternates

$$\phi_{t,j+1} = \phi_{t,j} - \eta_D \widehat{\nabla}_\phi C_D(\phi_{t,j}; \theta_t), \qquad j = 0, \ldots, k_D - 1,$$
$$\theta_{t+1} = \theta_t - \eta_G \widehat{\nabla}_\theta L_G(\theta_t; \phi_{t,k_D}),$$

where $\phi_{t,0} = \phi_t$, $\phi_{t+1} = \phi_{t,k_D}$, and L_G is either the saturating or the non-saturating generator objective.

22.6.1 Explicit training procedure

1. Initialize generator parameters θ and discriminator parameters ϕ.
2. For each of k_D discriminator steps, sample a real mini-batch $\{x^{(i)}\}_{i=1}^m$ and a noise mini-batch $\{z^{(i)}\}_{i=1}^m$, generate $\widetilde{x}^{(i)} = G_\theta(z^{(i)})$, and compute

   $$\widehat{C}_D = -\frac{1}{m}\sum_{i=1}^{m}\big[\log D_\phi(x^{(i)}) + \log(1 - D_\phi(\widetilde{x}^{(i)}))\big].$$

   Take a descent step in ϕ while holding θ fixed.
3. Draw another latent mini-batch and compute the non-saturating generator loss

   $$\widehat{L}_G^{\mathrm{ns}} = -\frac{1}{m}\sum_{i=1}^{m}\log D_\phi(G_\theta(z^{(i)})).$$

   Take a descent step in θ, differentiating through both G_θ and D_ϕ but leaving ϕ unchanged.
4. Repeat, evaluating both the distribution of generated samples and the ability of the discriminator to distinguish sources.

Remark 22.6 (Do not backpropagate through an optimizer step unnecessarily). Ordinary alternating GAN training calculates the partial derivative of each current loss with the other parameter vector fixed. It does not calculate the total derivative of $V(\theta, \phi^*(\theta))$ through every discriminator optimization step. Differentiating through an optimizer is a separate, more elaborate bilevel-learning procedure.

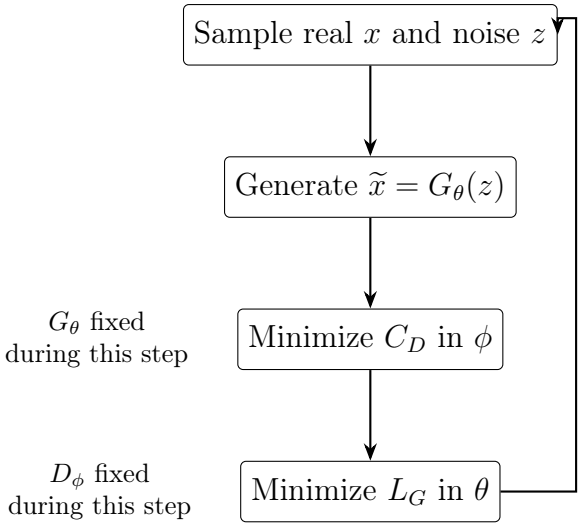


Figure 58: Alternating discriminator and generator optimization. One can perform several discriminator updates for each generator update.

22.7 Why GAN Optimization Can Be Unstable

The min–max game involves two parameter vectors and an objective that is neither necessarily convex in θ nor concave in ϕ. Ordinary gradient descent intuition for minimizing one fixed objective is not sufficient to prove convergence.

Example 22.1 (Rotation and divergence in a bilinear game)**.** Consider the simple zero-sum game

$$\min_{u\in\mathbb{R}} \max_{v\in\mathbb{R}} F(u, v), \qquad F(u, v) = uv.$$

The stationary point is $(u, v) = (0, 0)$. Simultaneous gradient descent in u and gradient ascent in v, using a constant step size $\eta > 0$, gives

$$u_{t+1} = u_t - \eta \frac{\partial F}{\partial u}(u_t, v_t) = u_t - \eta v_t,$$
$$v_{t+1} = v_t + \eta \frac{\partial F}{\partial v}(u_t, v_t) = v_t + \eta u_t.$$

Writing $r_t^2 = u_t^2 + v_t^2$, expand

$$\begin{aligned} r_{t+1}^2 &= (u_t - \eta v_t)^2 + (v_t + \eta u_t)^2 \\ &= u_t^2 - 2\eta u_t v_t + \eta^2 v_t^2 + v_t^2 + 2\eta u_t v_t + \eta^2 u_t^2 \end{aligned}$$

$$= (1+\eta^2)(u_t^2 + v_t^2)$$
$$= (1+\eta^2) r_t^2.$$

Thus the distance from the stationary point *increases* under these simultaneous updates unless the algorithm starts exactly at the origin. This elementary example demonstrates that local stationary conditions alone do not guarantee convergence of a competitive optimization method.

Remark 22.7 (Alternating is not the same as simultaneous). In practice the discriminator and generator parameters are often updated sequentially, and optimizers such as Adam modify the basic gradient steps. The bilinear example does not prove that all GAN algorithms diverge. It illustrates the underlying game-theoretic difficulty and why one should not infer convergence from ordinary loss-minimization arguments.

22.8 Separated Supports and Weak Learning Signals

The picture in the handwritten lecture shows two separated clusters of real and generated observations. When these clusters are easy to distinguish, the discriminator can make nearly perfect predictions while providing the generator with a poor learning signal. This is distinct from requiring generated points to be close to particular real points: the generator must learn the full distribution of shapes, locations, and modes.

Proposition 22.4 (Disjoint supports)**.** *If P_{data} and P_θ are mutually singular, then the optimal discriminator has $D^* = 1$ on a set supporting the real distribution and $D^* = 0$ on a disjoint set supporting the generated distribution. In this case*

$$D_{\text{JS}}(P_{\text{data}} \| P_\theta) = \log 2, \qquad \sup_D V(D; G_\theta) = 0.$$

Proof. Mutual singularity means there exists a measurable A with $P_{\text{data}}(A) = 1$ and $P_\theta(A) = 0$. Choose $D^*(x) = 1$ on A and $D^*(x) = 0$ on A^c. The discriminator value is then

$$V(D^*; G_\theta) = \mathbb{E}_{P_{\text{data}}} \log 1 + \mathbb{E}_{P_\theta} \log(1-0) = 0.$$

Every discriminator term $\log D$ and $\log(1-D)$ is nonpositive, so zero is the largest possible value. The JS identity gives

$$0 = -\log 4 + 2 D_{\text{JS}}(P_{\text{data}} \| P_\theta),$$
$$D_{\text{JS}}(P_{\text{data}} \| P_\theta) = \log 2.$$

□

If the distributions stay mutually singular throughout a region of generator parameter space, their optimal-discriminator JS objective is constant throughout that region. It consequently provides no distributional gradient there. At finite discriminator capacity, optimization may nevertheless yield some gradient; its usefulness depends on the discriminator's behavior between the supports.

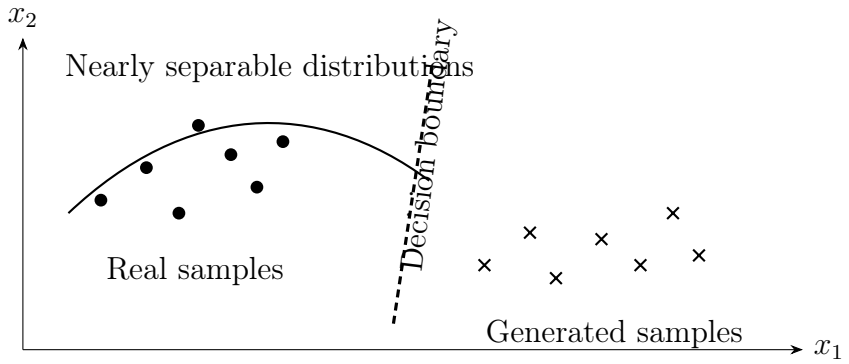


Figure 59: Schematic real and generated sample clouds that are easily distinguished. A nearly perfect discriminator is not automatically a useful teacher for the generator.

22.9 Mode Collapse and Distributional Coverage

The handwritten intuition asks the generator to make fake observations indistinguishable from real observations. Importantly, this requires more than producing *one* plausible observation. The generator should reproduce the diversity of the data distribution.

Definition 22.9 (Mode collapse)**.** A generator exhibits *mode collapse* when it maps many distinct latent inputs to the same observation or a restricted subset of plausible observations, so that the generated distribution fails to reproduce important modes or regions of P_{data}.

For example, imagine a dataset consisting of equal numbers of handwritten digits 0 through 9. If a generator produces only convincing digit 3s, individual outputs may look realistic, but the distribution still differs substantially from the data law. A discriminator that learns the missing modes should detect the discrepancy at the distribution level.

Proposition 22.5 (Two-mode example)**.** *Suppose the data distribution gives probability* $1/2$ *to each of two observations* a, b*, while the generator always outputs* a*. Then the Jensen–Shannon divergence is strictly positive:*

$$D_{\text{JS}}(P_{\text{data}} \| P_\theta) = \frac{3}{4} \log \frac{4}{3} > 0.$$

Proof. The probability vectors on $\{a, b\}$ are

$$p = (1/2, 1/2), \qquad q = (1, 0), \qquad m = (3/4, 1/4).$$

The first KL divergence is

$$\begin{aligned} D_{\text{KL}}(p \| m) &= \frac{1}{2} \log \frac{1/2}{3/4} + \frac{1}{2} \log \frac{1/2}{1/4} \\ &= \frac{1}{2} \log \frac{2}{3} + \frac{1}{2} \log 2 = \frac{1}{2} \log \frac{4}{3}. \end{aligned}$$

The second is

$$D_{\mathrm{KL}}(q\|m) = \log \frac{1}{3/4} = \log \frac{4}{3}.$$

Therefore,

$$\begin{aligned} D_{\mathrm{JS}}(p\|q) &= \frac{1}{2}\left(\frac{1}{2}\log\frac{4}{3}\right) + \frac{1}{2}\log\frac{4}{3} \\ &= \frac{3}{4}\log\frac{4}{3} > 0. \end{aligned}$$

□

Remark 22.8 (Distributional realism versus memorization). The optimal discriminator measures whether the real and generated *distributions* differ. It does not directly ensure that generated samples are novel, and a generator can overfit a finite training set. Diversity, coverage, and memorization therefore require additional evaluation beyond inspecting a few realistic outputs or a discriminator score close to 1/2.

22.10 A Worked Discrete GAN Example

An exact two-point example makes the discriminator objective and generator signal concrete. Let $\mathcal{X} = \{0, 1\}$, and suppose

$$P_{\mathrm{data}}(X = 1) = p, \qquad P_\theta(\widetilde{X} = 1) = r, \qquad 0 < p, r < 1.$$

The generator distribution is determined by the probability parameter r. Write $D(1) = d_1$ and $D(0) = d_0$. Then

$$\begin{aligned} V(D; r) = {} & p \log d_1 + (1-p)\log d_0 \\ & + r\log(1-d_1) + (1-r)\log(1-d_0). \end{aligned}$$

By differentiating in d_1 and d_0 separately,

$$\begin{aligned} \frac{\partial V}{\partial d_1} &= \frac{p}{d_1} - \frac{r}{1-d_1}, \\ \frac{\partial V}{\partial d_0} &= \frac{1-p}{d_0} - \frac{1-r}{1-d_0}. \end{aligned}$$

Setting these derivatives to zero yields

$$d_1^* = \frac{p}{p+r}, \qquad d_0^* = \frac{1-p}{(1-p)+(1-r)} = \frac{1-p}{2-p-r}.$$

If $r = p$ then both scores equal $1/2$. If instead $p = 0.8$ and $r = 0.2$, then

$$d_1^* = \frac{0.8}{1.0} = 0.8, \qquad d_0^* = \frac{0.2}{1.0} = 0.2.$$

Thus an observation of 1 is more likely real, while an observation of 0 is more likely generated. The generator must move r toward p if it wants its distribution to match the data distribution.

Remark 22.9 (Discrete generators and pathwise gradients). If the generator samples a discrete Bernoulli observation directly, the sample is not a differentiable function of r in the ordinary pathwise sense. The example describes the *distributional objective* exactly, but backpropagation through a hard discrete sample requires an estimator or relaxation. By contrast, the usual image GAN generates continuous tensors by differentiable neural-network operations.

22.11 A Gaussian Classification Example

For further intuition, suppose the real and generated distributions are one-dimensional Gaussians with the same variance:

$$P_{\text{data}} = \mathcal{N}(\mu_r, \sigma^2), \qquad P_\theta = \mathcal{N}(\mu_g, \sigma^2).$$

Their log density ratio can be calculated without approximation:

$$\begin{aligned}
\log \frac{p_{\text{data}}(x)}{p_\theta(x)} &= -\frac{(x-\mu_r)^2}{2\sigma^2} + \frac{(x-\mu_g)^2}{2\sigma^2} \\
&= \frac{(x^2 - 2x\mu_g + \mu_g^2) - (x^2 - 2x\mu_r + \mu_r^2)}{2\sigma^2} \\
&= \frac{\mu_r - \mu_g}{\sigma^2} x + \frac{\mu_g^2 - \mu_r^2}{2\sigma^2}.
\end{aligned}$$

Therefore the optimal discriminator is logistic in x:

$$D^*(x) = \sigma\left(\frac{\mu_r - \mu_g}{\sigma^2} x + \frac{\mu_g^2 - \mu_r^2}{2\sigma^2}\right).$$

When $\mu_r = \mu_g$, the logit vanishes at every x, and $D^*(x) = 1/2$. This is a simple instance of an optimal discriminator learning a density ratio using a linear logit, even though the two underlying distributions are Gaussian.

22.12 Practical Stabilization and Alternative Generative Objectives

Several common difficulties are now visible: a discriminator can become much stronger than the generator; mode collapse can reduce the diversity of generated observations; and the minimax optimization can oscillate. Practical methods therefore adjust architecture, regularization, or the objective itself.

22.12.1 Non-saturating training and discriminator regularization

The non-saturating objective described earlier is the simplest modification. Other training choices include balancing the number of generator and discriminator updates, constraining discriminator complexity, and regularizing discriminator gradients. For example, a differentiable penalty of the schematic form

$$\lambda \mathbb{E}_{X\sim\nu} \|\nabla_x a_\phi(X)\|_2^2, \qquad \lambda \geq 0,$$

for an appropriate chosen sampling law ν, discourages excessively sharp changes in the discriminator logit. This is an additional modeling and optimization choice; its effect depends on the actual network and sampling distribution.

22.12.2 The Wasserstein perspective (extension)

The original minimax GAN objective involves the Jensen–Shannon divergence only after optimizing the discriminator exactly. A different approach is to compare probability distributions with an optimal-transport distance. For probability measures with finite first moments on a metric space, the 1-Wasserstein distance is

$$W_1(P, Q) = \inf_{\pi \in \Pi(P,Q)} \int \|x - y\| \, \pi(\mathrm{d}x, \mathrm{d}y),$$

where $\Pi(P, Q)$ is the set of couplings with marginals P, Q. Under the standard assumptions of Kantorovich–Rubinstein duality,

$$W_1(P, Q) = \sup_{\|f\|_{\mathrm{Lip}} \le 1} \big(\mathbb{E}_P f(X) - \mathbb{E}_Q f(X) \big).$$

A Wasserstein GAN replaces the probabilistic discriminator with a real-valued *critic* approximating a 1-Lipschitz function. Its idealized game becomes

$$\min_{\theta} \sup_{\|f\|_{\mathrm{Lip}} \le 1} \left[\mathbb{E}_{P_{\mathrm{data}}} f(X) - \mathbb{E}_Z f(G_\theta(Z)) \right].$$

Unlike the original GAN discriminator, the critic score is not interpreted as a probability in $[0, 1]$. Enforcing the Lipschitz restriction is part of the method. Wasserstein objectives can distinguish separated distributions by a geometry-sensitive quantity even when Jensen–Shannon divergence is saturated.

Example 22.2 (Separated point masses)**.** Let $P = \delta_0$ and $Q = \delta_a$ on $\mathbb{R}$ for some $a \neq 0$. Their supports do not intersect, so

$$D_{\mathrm{JS}}(\delta_0 \| \delta_a) = \log 2 \quad \text{for every } a \neq 0.$$

But there is only one possible coupling of these point masses, and its transportation cost is $|a|$. Hence

$$W_1(\delta_0, \delta_a) = |a|.$$

The Wasserstein distance changes continuously with the displacement of the generated point mass, while Jensen–Shannon divergence remains constant until the point masses coincide.

Remark 22.10 (Relation to the handwritten lecture)*.* The optimal discriminator, minimax loss, and alternating updates are the core content of the uploaded lecture. The Wasserstein formulation is a mathematical extension that helps explain the support-separation problem; it is not needed to define the original GAN.

22.12.3 Conditional generation (extension)

Suppose each observation comes with a class or side-information variable C. A *conditional GAN* can take C as input to both networks:

$$\widetilde{X} = G_\theta(Z, C), \qquad D_\phi(X, C) \in (0, 1).$$

Its conditional objective is

$$\begin{aligned} V_c(\theta, \phi) = \mathbb{E}_{(X,C)\sim P_{\text{data}}} \log D_\phi(X, C) \\ + \mathbb{E}_{C\sim P_C,\, Z\sim P_Z} \log(1 - D_\phi(G_\theta(Z, C), C)). \end{aligned}$$

If both networks have sufficient capacity and optimization is ideal, this compares the conditional data and generated distributions, enabling the user to request samples with specified attributes rather than drawing every sample unconditionally.

22.13 Worked Exercises

Exercise 1.

Let $p(x) = 3/4$ and $q(x) = 1/4$ at some point with nonzero density under both distributions. Find the optimal real probability, its odds, and its logit. Interpret the answer.

Solution. The optimal-discriminator formula yields

$$\begin{aligned} D^*(x) &= \frac{3/4}{3/4 + 1/4} = \frac{3}{4}, \\ \frac{D^*(x)}{1 - D^*(x)} &= \frac{3/4}{1/4} = 3, \\ \operatorname{logit}(D^*(x)) &= \log 3. \end{aligned}$$

Thus the observation is three times as likely under the data density as under the generated density, assuming balanced source priors, and its posterior probability of being real is $3/4$. ■

Exercise 2.

Suppose $P_\theta = P_{\text{data}}$. Find D^*, $V(D^*; G_\theta)$, and $C_D(D^*; \theta)$, and explain why these are not all zero.

Solution. The density-ratio formula gives $D^* = 1/2$. Hence

$$\begin{aligned} V(D^*; G_\theta) &= \log(1/2) + \log(1/2) = -\log 4, \\ C_D(D^*; \theta) &= -V(D^*; G_\theta) = \log 4. \end{aligned}$$

The positive cross-entropy is unavoidable because identical real and fake distributions carry no information about their source labels. The classification uncertainty is maximized rather than eliminated. ■

Exercise 3.

Derive the discriminator and non-saturating generator gradients with respect to the discriminator logit a for a single generated observation. What happens when $D = 0.01$?

Solution. For a fake discriminator example the BCE loss is $-\log(1 - \sigma(a))$, so

$$\frac{\partial C_D^{\text{fake}}}{\partial a} = \sigma(a) = D.$$

For the non-saturating generator loss $-\log \sigma(a)$,

$$\frac{\partial L_G^{\text{ns}}}{\partial a} = -(1 - \sigma(a)) = -(1 - D).$$

When $D = 0.01$, the two derivatives are 0.01 and -0.99, respectively. They have opposite intended effects in their respective optimization steps: the discriminator tries to reduce fake scores, while the generator tries to raise those scores by moving its outputs. ■

Exercise 4.

Suppose $P_{\text{data}} = \delta_{-1}$ and $P_\theta = \delta_b$ on $\mathbb{R}$ with $b \neq -1$. Calculate the JS divergence and the 1-Wasserstein distance. Discuss why these quantities behave differently as b approaches -1.

Solution. Because the two measures are concentrated at distinct points, they are mutually singular, and

$$D_{\text{JS}}(P_{\text{data}} \| P_\theta) = \log 2.$$

The unique coupling transports mass one from -1 to b, so

$$W_1(P_{\text{data}}, P_\theta) = |b + 1|.$$

As $b \to -1$ through unequal values, $W_1 \to 0$, whereas the JS divergence stays at $\log 2$ until the two point masses coincide; at equality it drops to zero. Hence the Wasserstein distance reflects geometric displacement more directly for these singular distributions. ■

Exercise 5.

Verify the discriminator optimum directly for the two-point distribution $p = (0.6, 0.4)$ and $q = (0.3, 0.7)$.

Solution. The pointwise optimal scores are

$$d_1^* = \frac{0.6}{0.6 + 0.3} = \frac{2}{3}, \qquad d_2^* = \frac{0.4}{0.4 + 0.7} = \frac{4}{11}.$$

The corresponding objective is

$$V(D^*; G) = 0.6 \log \frac{2}{3} + 0.4 \log \frac{4}{11} + 0.3 \log \frac{1}{3} + 0.7 \log \frac{7}{11}.$$

Since all four source probabilities are positive, the pointwise second derivatives are negative and these are the unique optimal discriminator scores. ■

Exercise 6.

For the bilinear game $F(u, v) = uv$, show that the norm of the parameter pair after T simultaneous updates with step size $\eta > 0$ satisfies $r_T = (1 + \eta^2)^{T/2} r_0$.

Solution. The calculation in the lecture shows $r_{t+1}^2 = (1 + \eta^2) r_t^2$. Iterating,

$$\begin{aligned} r_T^2 &= (1 + \eta^2) r_{T-1}^2 \\ &= (1 + \eta^2)^2 r_{T-2}^2 \\ &= \cdots = (1 + \eta^2)^T r_0^2. \end{aligned}$$

Taking square roots and using $r_T \geq 0$ gives $r_T = (1 + \eta^2)^{T/2} r_0$. ■

Exercise 7.

Consider the generator $G_\theta(z) = \theta + z$ with $Z \sim \mathcal{N}(0, 1)$ and real data $X \sim \mathcal{N}(2, 1)$. Assuming the discriminator is always optimal, determine the matching value of θ and write the optimal discriminator as a logistic function of x for general θ.

Solution. The generator produces $P_\theta = \mathcal{N}(\theta, 1)$, so exact distribution matching occurs at $\theta = 2$. By the equal-variance Gaussian formula with $\mu_r = 2$, $\mu_g = \theta$, and $\sigma^2 = 1$,

$$D^*(x) = \sigma\left((2 - \theta)x + \frac{\theta^2 - 4}{2}\right).$$

At $\theta = 2$ the logit is zero for every x and $D^*(x) = 1/2$. ■

Exercise 8.

A generator makes realistic samples from only one of three equally probable data modes. Explain why realistic appearance of individual outputs does not establish distribution matching and state what the ideal discriminator would detect.

Solution. Let the three modes be a, b, c with $p = (1/3, 1/3, 1/3)$, and suppose the generator produces only a, so $q = (1, 0, 0)$. The optimal discriminator at the generated mode is

$$D^*(a) = \frac{1/3}{1/3 + 1} = \frac{1}{4},$$

while at the two absent modes $D^*(b) = D^*(c) = 1$. Thus the missing modes provide decisive distribution-level evidence separating real and fake sources even though every generated observation might resemble a plausible data point. This is an example of mode collapse. ■